

High-Level System Design

Interview Preparation Guide

System Design Notes

July 28, 2026

Contents

1	Design a URL Shortener (TinyURL / Bitly)	24
1.1	Table of Contents	24
1.2	1. Problem Statement and Scope	24
1.2.1	1.2.1 Problem Statement	24
1.2.2	1.2.2 Why is This Problem Important?	24
1.2.3	1.2.3 Scope Definition	25
1.3	2. Functional Requirements	25
1.4	3. Non-Functional Requirements	26
1.4.1	1.4.1 Consistency vs. Availability Trade-off	26
1.5	4. Back-of-Envelope Estimation	26
1.5.1	1.5.1 4.1 Traffic Estimation	27
1.5.2	1.5.2 4.2 Storage Estimation	28
1.5.3	1.5.3 4.3 Bandwidth Estimation	28
1.5.4	1.5.4 4.4 Cache/Memory Estimation	29
1.6	5. API Design	29
1.6.1	1.6.1 RESTful API Endpoints	30
1.6.2	1.6.2 API Design Considerations	32
1.7	6. Data Model and Database Selection	33
1.7.1	1.7.1 6.1 Schema Design	33
1.7.2	1.7.2 6.2 Database Selection Rationale	35
1.8	7. High-Level Architecture	36
1.8.1	1.8.1 Complete System Architecture	36
1.8.2	1.8.2 7.1 Component Breakdown	38
1.9	8. Deep Dive: Core Components	39
1.9.1	1.9.1 8.1 URL Shortening Algorithm	39
1.9.2	1.9.2 8.2 Key Generation Service (KGS)	44
1.9.3	1.9.3 8.3 Redirection Service	47
1.9.4	1.9.4 8.4 Analytics Pipeline	49
1.10	9. Data Partitioning and Sharding	51
1.10.1	1.10.1 Sharding Strategy	51
1.10.2	1.10.2 DynamoDB Partition Key Design	52
1.10.3	1.10.3 Cross-Shard Operations	52
1.11	10. Caching Strategy	53
1.11.1	1.11.1 Multi-Layer Cache Architecture	53
1.11.2	1.11.2 Cache-Aside Pattern (Implementation)	54
1.11.3	1.11.3 Cache Stampede Prevention	55
1.12	11. Replication and Consistency	55
1.12.1	1.12.1 Consistency Model	56
1.12.2	1.12.2 Replication Architecture	57
1.12.3	1.12.3 Redis Replication	57

1.13	12. Fault Tolerance and Failure Handling	58
1.13.1	Rate Limiting	59
1.13.2	Graceful Degradation Strategy	59
1.14	13. Scalability	59
1.14.1	Horizontal Scaling of Stateless Components	60
1.14.2	Scaling to 10x and 100x	61
1.14.3	Read Replicas and CQRS	62
1.15	14. Monitoring and Observability	62
1.15.1	Key Metrics Dashboard	63
1.15.2	Alerting Rules	64
1.15.3	Distributed Tracing	65
1.16	15. Trade-offs and Design Decisions	65
1.16.1	Algorithm Comparison	65
1.16.2	Database Comparison	66
1.16.3	301 vs 302 Summary	67
1.16.4	SQL vs NoSQL Decision Framework	67
1.17	16. Interview Deep-Dive Questions	67
1.17.1	Q1: How do you handle custom aliases?	68
1.17.2	Q2: How do you handle URL expiration?	68
1.17.3	Q3: What if two users submit the same long URL?	68
1.17.4	Q4: How do you prevent abuse and spam?	68
1.17.5	Q5: How do you generate globally unique keys in a distributed system?	69
1.17.6	Q6: How do you handle hot URLs (viral links)?	69
1.17.7	Q7: How do you handle the read/write ratio imbalance?	69
1.17.8	Q8: What happens if the database is temporarily unavailable?	69
1.17.9	Q9: How do you ensure short URLs are not predictable?	70
1.17.10	Q10: How do you handle database schema migrations?	70
1.17.11	Q11: How would you implement A/B testing with short URLs?	70
1.17.12	Q12: How do you handle the “thundering herd” problem when a popular URL’s cache expires?	71
1.17.13	Q13: What are the security considerations for a URL shortener?	71
1.17.14	Q14: How would you migrate from one database to another (e.g., PostgreSQL to DynamoDB)?	71
1.17.15	Q15: How would you design URL shortener for multi-tenancy (enterprise white- label)?	72
1.18	Summary: System at a Glance	73
2	Design a Distributed Rate Limiter	74
2.1	1. Problem Statement and Scope	74
2.1.1	What is Rate Limiting?	74
2.1.2	Problem	74
2.1.3	Scope	75
2.1.4	Real-World Examples	75
2.2	2. Functional Requirements	75
2.2.1	Core Requirements	75
2.2.2	Request Flow	76
2.3	3. Non-Functional Requirements	76
2.3.1	Fail-Open vs Fail-Closed	76
2.4	4. Back-of-Envelope Estimation	77
2.4.1	Traffic Assumptions	77
2.4.2	Counter Storage	77
2.4.3	Redis Operations	77

2.4.4	Network Bandwidth	77
2.4.5	Rule Storage	78
2.5	5. API Design	78
2.5.1	5.1 Rate Limit Check (Internal Middleware API)	78
2.5.2	5.2 Admin API (External, for Rule Management)	79
2.5.3	5.3 Rate Limit Response Headers	79
2.6	6. Data Model and Database Selection	80
2.6.1	6.1 Database Selection Rationale	80
2.6.2	6.2 Rules Schema (PostgreSQL)	80
2.6.3	6.3 Allow/Deny List Schema	80
2.6.4	6.4 Redis Counter Key Design	81
2.6.5	6.5 Entity Relationship	81
2.7	7. High-Level Architecture	81
2.7.1	7.1 System Overview	82
2.7.2	7.2 Centralized vs Distributed Rate Limiting	84
2.7.3	7.3 Component Breakdown	84
2.8	8. Deep Dive: Core Components	84
2.8.1	8.1 Rate Limiting Algorithms	85
2.8.2	8.2 Distributed Counter Management	92
2.8.3	8.3 Rule Engine	94
2.8.4	8.4 Edge Rate Limiting vs Centralized	96
2.9	9. Data Partitioning and Sharding	98
2.9.1	9.1 Redis Cluster Sharding	98
2.9.2	9.2 Sharding Strategy	99
2.9.3	9.3 Handling Hot Shards	100
2.10	10. Caching Strategy	100
2.10.1	10.1 Rule Caching	101
2.10.2	10.2 Counter Approximation with Local Caching	101
2.10.3	10.3 Cache Warm-Up	102
2.11	11. Replication and Consistency	102
2.11.1	11.1 Counter Consistency Model	103
2.11.2	11.2 Multi-Data-Center Consistency	104
2.11.3	11.3 Rule Consistency	105
2.12	12. Fault Tolerance and Failure Handling	105
2.12.1	12.1 Failure Scenarios	105
2.12.2	12.2 Circuit Breaker for Redis	106
2.12.3	12.3 Local Fallback Counters	107
2.13	13. Scalability	107
2.13.1	13.1 Horizontal Scaling	108
2.13.2	13.2 Performance Characteristics	109
2.13.3	13.3 Scaling Bottlenecks and Mitigations	110
2.14	14. Monitoring and Observability	110
2.14.1	14.1 Key Metrics	111
2.14.2	14.2 Logging Strategy	112
2.14.3	14.3 Distributed Tracing Integration	112
2.15	15. Trade-offs and Design Decisions	113
2.15.1	15.1 Key Trade-offs	113
2.15.2	15.2 Technology Choices Rationale	115
2.16	16. Interview Deep-Dive Questions	115
2.16.1	Q1: How do you rate limit across multiple data centers?	115
2.16.2	Q2: What happens during a Redis failover?	116

2.16.3	Q3: How do you handle the fixed window boundary burst problem?	116
2.16.4	Q4: How do you rate limit WebSocket connections vs HTTP requests?	116
2.16.5	Q5: How would you implement a “leaky bucket” at global scale?	116
2.16.6	Q6: How do you prevent a single hot tenant from affecting other tenants?	117
2.16.7	Q7: What is the cost of this system at scale?	117
2.16.8	Q8: How do you handle rate limit key collision?	118
2.16.9	Q9: How would you implement tiered rate limiting (free/pro/enterprise)?	118
2.16.10	Q10: How do you test a rate limiter in production?	118
2.16.11	Q11: Should rate limiting be implemented as a library or a service?	119
2.16.12	Q12: How do you handle rate limiting for distributed transactions?	119
2.16.13	Q13: How do you handle sudden traffic spikes (flash crowds)?	119
2.16.14	Q14: What is the difference between rate limiting and throttling?	120
2.16.15	Q15: How would you migrate from a local rate limiter to a distributed one?	120
2.17	Appendix A: Complete Request Flow	121
2.18	Appendix B: Production Configuration Example	122
2.19	Appendix C: Quick Reference Card	123
3	Design a Chat Messaging System (WhatsApp / Messenger)	124
3.1	1. Problem Statement and Scope	124
3.1.1	1.1 In Scope	124
3.1.2	1.2 Out of Scope	124
3.2	2. Functional Requirements	125
3.2.1	2.1 Core Messaging	125
3.2.2	2.2 User Stories	125
3.3	3. Non-Functional Requirements	125
3.3.1	3.1 SLA Breakdown	126
3.4	4. Back-of-Envelope Estimation	126
3.4.1	4.1 Traffic Estimates	126
3.4.2	4.2 Storage Estimates	126
3.4.3	4.3 Connection Estimates	126
3.4.4	4.4 Bandwidth Estimates	127
3.4.5	4.5 Summary Table	127
3.5	5. API Design	127
3.5.1	5.1 WebSocket Protocol (Real-Time)	127
3.5.2	5.2 REST APIs	129
3.6	6. Data Model and Database Selection	130
3.6.1	6.1 Database Selection Rationale	131
3.6.2	6.2 Messages Table (Cassandra)	132
3.6.3	6.3 Users and Groups (MySQL via Vitess)	133
3.6.4	6.4 Session and Presence (Redis)	134
3.6.5	6.5 Entity Relationship Diagram	135
3.7	7. High-Level Architecture	135
3.7.1	7.0 System Overview Diagram	136
3.7.2	7.1 Component Breakdown	137
3.8	8. Deep Dive: Core Components	139
3.8.1	8.1 WebSocket Connection Management	139
3.8.2	8.2 Message Delivery and Ordering	142
3.8.3	8.3 Presence Service	145
3.8.4	8.4 Group Messaging	147
3.9	9. Data Partitioning and Sharding	149
3.9.1	9.1 Message Data (Cassandra)	150
3.9.2	9.2 User Data (MySQL via Vitess)	151

3.9.3	9.3 Connection Server Routing	152
3.10	10. Caching Strategy	152
3.10.1	10.1 Cache Architecture	153
3.10.2	10.2 What We Do NOT Cache	154
3.10.3	10.3 Cache Invalidation Strategy	154
3.11	11. Replication and Consistency	154
3.11.1	11.1 Multi-Region Architecture	155
3.11.2	11.2 Consistency Model per Data Type	156
3.11.3	11.3 Cross-Region Message Delivery	157
3.12	12. Fault Tolerance and Failure Handling	157
3.12.1	12.1 Failure Scenarios and Mitigations	158
3.12.2	12.2 Client-Side Reliability	159
3.12.3	12.3 Message Delivery Guarantee	160
3.13	13. Scalability	160
3.13.1	13.1 Scaling Strategy per Component	161
3.13.2	13.2 Bottleneck Analysis	162
3.13.3	13.3 Connection Server Scaling Detail	163
3.14	14. Monitoring and Observability	163
3.14.1	14.1 Key Metrics Dashboard	164
3.14.2	14.2 Distributed Tracing	165
3.14.3	14.3 Health Check Strategy	165
3.15	15. Trade-offs and Design Decisions	166
3.15.1	15.1 Push vs Pull for Message Delivery	166
3.15.2	15.2 WebSocket vs Long Polling vs SSE	167
3.15.3	15.3 Cassandra vs HBase vs MongoDB	167
3.15.4	15.4 End-to-End Encryption Trade-offs	168
3.15.5	15.5 Message Storage: Store Per-Conversation vs Per-User	169
3.16	16. Interview Deep-Dive Questions	169
3.16.1	Q1: How do you handle message ordering in group chats?	169
3.16.2	Q2: How does end-to-end encryption work in this system?	169
3.16.3	Q3: How do you sync messages across multiple devices?	170
3.16.4	Q4: What happens when a user comes back online after being offline for days?	170
3.16.5	Q5: How do you handle message deletion for everyone?	171
3.16.6	Q6: How do you handle the “thundering herd” problem when a popular user comes online?	171
3.16.7	Q7: How do you ensure exactly-once message delivery?	171
3.16.8	Q8: How would you implement message search?	172
3.16.9	Q9: How do you handle rate limiting and abuse prevention?	172
3.16.10	Q10: How do you handle schema evolution and backward compatibility?	172
3.16.11	Q11: How do you handle typing indicators efficiently?	173
3.16.12	Q12: How do you implement read receipts for group chats?	173
3.16.13	Q13: How would you handle message reactions (emoji reactions)?	173
3.16.14	Q14: How do you design the notification batching system?	174
3.16.15	Q15: How would you migrate from a monolithic chat system to this microservices architecture?	174
3.17	17. End-to-End Message Flow Summary	176
3.18	18. Quick Reference: Interview Checklist	177
3.19	19. References and Further Reading	177
4	Design a News Feed System (Twitter / Facebook)	178
4.1	Table of Contents	178
4.2	1. Problem Statement and Scope	178

4.2.1	What is a News Feed?	178
4.2.2	Scope Definition	179
4.2.3	Key Actors	179
4.3	2. Functional Requirements	179
4.3.1	Core Features	179
4.3.2	User Flows	180
4.4	3. Non-Functional Requirements	181
4.4.1	Consistency vs Availability Trade-off	181
4.5	4. Back-of-Envelope Estimation	181
4.5.1	User and Traffic Assumptions	181
4.5.2	QPS Calculations	182
4.5.3	Storage Estimation	183
4.5.4	Bandwidth Estimation	183
4.6	5. API Design	183
4.6.1	5.1 Post Creation	184
4.6.2	5.2 Get News Feed (Cursor-Based Pagination)	185
4.6.3	5.3 Follow / Unfollow	186
4.6.4	5.4 Like / Comment	186
4.6.5	5.5 Trending / Hashtags	186
4.6.6	Rate Limiting	187
4.7	6. Data Model and Database Selection	187
4.7.1	6.1 Database Selection Rationale	187
4.7.2	6.2 Schema Definitions	188
4.7.3	6.3 Entity Relationship Overview	191
4.8	7. High-Level Architecture	191
4.8.1	7.1 System Overview	192
4.8.2	7.2 Post Creation Path (Write Path)	193
4.8.3	7.3 Feed Retrieval Path (Read Path)	194
4.8.4	7.4 Component Breakdown	195
4.9	8. Deep Dive: Core Components	196
4.9.1	8.1 Fan-out Service	196
4.9.2	8.2 Feed Ranking Algorithm	200
4.9.3	8.3 Post Storage and Retrieval	204
4.9.4	8.4 Social Graph Service	206
4.10	9. Data Partitioning and Sharding	208
4.10.1	Sharding Strategy Overview	208
4.10.2	Cassandra Partitioning for Posts	209
4.10.3	Redis Cluster Sharding for Feed Cache	210
4.10.4	MySQL Sharding for Social Graph	211
4.11	10. Caching Strategy	211
4.11.1	Multi-Layer Cache Architecture	212
4.11.2	Cache Invalidation Strategy	214
4.12	11. Replication and Consistency	214
4.12.1	Consistency Model Per Data Store	215
4.12.2	Multi-Region Architecture	216
4.12.3	Read-After-Write Consistency	216
4.13	12. Fault Tolerance and Failure Handling	217
4.13.1	Failure Scenarios and Mitigations	218
4.13.2	Degraded Mode Strategy	219
4.13.3	Idempotency in Fan-out	219
4.14	13. Scalability	220

4.14.1	Horizontal Scaling Strategy	220
4.14.2	Fan-out Worker Scaling	221
4.14.3	Handling Traffic Spikes (Viral Posts)	222
4.15	14. Monitoring and Observability	222
4.15.1	Key Metrics Dashboard	223
4.15.2	Distributed Tracing Example	224
4.16	15. Trade-offs and Design Decisions	224
4.16.1	Decision 1: Fan-out Strategy	225
4.16.2	Decision 2: Feed Ranking	225
4.16.3	Decision 3: Pre-compute vs On-Demand Feed	226
4.16.4	Decision 4: Database Selection for Posts	226
4.16.5	Decision 5: Cursor-Based vs Offset-Based Pagination	226
4.17	16. Interview Deep-Dive Questions	227
4.17.1	Q1: How do you handle the celebrity problem?	227
4.17.2	Q2: What happens when a user unfollows someone?	227
4.17.3	Q3: How do you handle deleted or hidden posts in pre-computed feeds?	227
4.17.4	Q4: How do you rank posts for new users with no interaction history?	227
4.17.5	Q5: How do you prevent feed staleness?	228
4.17.6	Q6: How does the system handle a Kafka outage? Will users not see new posts?	228
4.17.7	Q7: How do you handle pagination when the feed is being modified concurrently?	228
4.17.8	Q8: What if a user follows 10,000 accounts? How does fan-out on read scale?	228
4.17.9	Q9: How do you handle media (images/videos) in posts?	229
4.17.10	Q10: How do you implement trending topics?	229
4.17.11	Q11: How do you handle a user who has been inactive for months and returns?	229
4.17.12	Q12: How do you prevent spam in the news feed?	229
4.17.13	Q13: How would you add “Stories” (ephemeral content) to this system?	230
4.17.14	Q14: How do you ensure data consistency when the same post is stored in Cassandra and cached in Redis?	230
4.17.15	Q15: How would you design the system differently if the requirement changed to a bidirectional “friend” model (like Facebook) instead of unidirectional “fol- low” (like Twitter)?	230
4.18	Summary	231
5	Design a Video Streaming Platform (YouTube / Netflix)	232
5.1	1. Problem Statement and Scope	232
5.1.1	In Scope	232
5.1.2	Out of Scope	232
5.2	2. Functional Requirements	233
5.3	3. Non-Functional Requirements	233
5.4	4. Back-of-Envelope Estimation	233
5.4.1	Traffic Estimates	233
5.4.2	Storage Estimates	234
5.4.3	Bandwidth Estimates	234
5.4.4	Compute Estimates (Transcoding)	234
5.4.5	Summary Table	234
5.5	5. API Design	235
5.5.1	5.1 Video Upload	235
5.5.2	5.2 Video Streaming	236
5.5.3	5.3 Video Metadata and Interactions	237
5.6	6. Data Model and Database Selection	237
5.6.1	6.1 Data Models	237
5.6.2	6.2 Database Selection	240

5.7	7. High-Level Architecture	240
5.7.1	Overall System Architecture	241
5.7.2	Upload Path (Write Path)	243
5.7.3	Watch Path (Read Path)	243
5.7.4	7.1 Component Breakdown	244
5.8	8. Deep Dive: Core Components	244
5.8.1	8.1 Video Upload and Processing Pipeline	244
5.8.2	8.2 Adaptive Bitrate Streaming (ABR)	248
5.8.3	8.3 CDN Architecture	251
5.8.4	8.4 Recommendation Engine	253
5.9	9. Data Partitioning and Sharding	255
5.9.1	Sharding Strategy by Data Type	256
5.9.2	Cross-Shard Query Handling	257
5.10	10. Caching Strategy	258
5.10.1	Multi-Layer Cache Architecture	259
5.10.2	Cache Invalidation Strategy	260
5.11	11. Replication and Consistency	260
5.11.1	Consistency Requirements by Data Type	261
5.11.2	Replication Topology	262
5.12	12. Fault Tolerance and Failure Handling	263
5.12.1	Failure Scenarios and Mitigations	264
5.12.2	Degraded Mode Strategy	265
5.13	13. Scalability	265
5.13.1	Horizontal Scaling Approach	266
5.13.2	Handling Viral Videos	267
5.14	14. Monitoring and Observability	267
5.14.1	Key Metrics Dashboard	268
5.14.2	Distributed Tracing for Video Playback	269
5.15	15. Trade-offs and Design Decisions	269
5.15.1	Decision 1: Pre-Transcode All Resolutions vs On-Demand Transcoding	270
5.15.2	Decision 2: HLS vs DASH vs Both (CMAF)	270
5.15.3	Decision 3: Push vs Pull CDN Strategy	271
5.15.4	Decision 4: Monolith vs Microservices	272
5.15.5	Decision 5: Segment Duration	272
5.16	16. Interview Deep-Dive Questions	273
5.16.1	Q1: How do you handle viral videos with sudden traffic spikes?	273
5.16.2	Q2: How does adaptive bitrate streaming work end-to-end?	273
5.16.3	Q3: How do you detect and handle copyrighted content?	273
5.16.4	Q4: How would you design live streaming on top of this architecture?	274
5.16.5	Q5: How do you optimize for mobile vs desktop clients?	275
5.16.6	Q6: How do you handle view counting at scale without losing accuracy?	275
5.16.7	Q7: What is your content moderation strategy?	276
5.16.8	Q8: How do you design the subscription feed (timeline)?	276
5.16.9	Q9: How do you ensure smooth playback across different network conditions?	276
5.16.10	Q10: How do you handle video search at scale?	277
5.16.11	Q11: What is the data retention and storage tiering strategy?	277
5.16.12	Q12: How do you handle DRM (Digital Rights Management)?	277
5.16.13	Q13: How do you handle multi-language and subtitles?	278
5.16.14	Q14: What is your approach to A/B testing the video player and recommendation algorithm?	278
5.16.15	Q15: How would you estimate the total infrastructure cost?	278

5.17 Appendix: Quick Reference Architecture	279
6 Design a Notification System	280
6.1 Table of Contents	280
6.2 1. Problem Statement and Scope	280
6.2.1 Scope Boundaries	281
6.3 2. Functional Requirements	281
6.3.1 Core Features	281
6.3.2 Notification Lifecycle	282
6.3.3 Channel-Specific Features	282
6.4 3. Non-Functional Requirements	282
6.4.1 Latency Budget Breakdown	283
6.5 4. Back-of-Envelope Estimation	283
6.5.1 Traffic Estimates	284
6.5.2 Storage Estimates	285
6.5.3 Bandwidth Estimates	285
6.6 5. API Design	285
6.6.1 5.1 Send Notification	286
6.6.2 5.2 Get In-App Notifications	287
6.6.3 5.3 Update User Preferences	288
6.6.4 5.4 Template Management	289
6.6.5 5.5 Delivery Status Webhook	290
6.7 6. Data Model and Database Selection	290
6.7.1 Database Selection Rationale	291
6.7.2 Schema Design	292
6.8 7. High-Level Architecture	294
6.8.1 Overall System Architecture	295
6.8.2 7.1 Component Breakdown	297
6.9 8. Deep Dive: Core Components	300
6.9.1 8.1 Notification Processing Pipeline	300
6.9.2 8.2 Priority Queue and Ordering	302
6.9.3 8.3 Push Notification Delivery	304
6.9.4 8.4 Email Delivery	308
6.9.5 8.5 Deduplication and Idempotency	310
6.9.6 8.6 Rate Limiting	311
6.9.7 8.7 In-App Notification Delivery	313
6.9.8 8.8 Digest and Aggregation	315
6.10 9. Data Partitioning and Sharding	316
6.10.1 Partitioning Strategy	316
6.10.2 Hot Partition Mitigation	318
6.11 10. Caching Strategy	319
6.12 11. Replication and Consistency	321
6.12.1 Multi-Region Replication	322
6.13 12. Fault Tolerance and Failure Handling	322
6.13.1 Failure Scenarios and Mitigations	323
6.13.2 Dead Letter Queue (DLQ) Processing	325
6.14 13. Scalability	326
6.14.1 Scaling Strategy	326
6.15 14. Monitoring and Observability	327
6.15.1 Key Metrics Dashboard	328
6.15.2 Distributed Tracing	330
6.16 15. Trade-offs and Design Decisions	330

6.16.1	Decision Matrix	331
6.17	16. Interview Deep-Dive Questions	334
6.17.1	Q1: How do you handle notification storms (e.g., everyone gets notified at once)?	334
6.17.2	Q2: How do you prevent duplicate notifications?	334
6.17.3	Q3: How do you handle timezone-aware scheduling?	335
6.17.4	Q4: What happens when APNs/FCM is down?	335
6.17.5	Q5: How do you A/B test notification content?	335
6.17.6	Q6: How do you ensure ordering of related notifications?	336
6.17.7	Q7: How do you handle user preference updates propagating to in-flight notifications?	336
6.17.8	Q8: How do you handle multi-device push delivery?	336
6.17.9	Q9: How would you add a new notification channel (e.g., WhatsApp)?	336
6.17.10	Q10: How do you handle cost optimization for SMS?	337
6.17.11	Q11: What is the retry strategy, and how do you prevent retry storms?	337
6.17.12	Q12: How do you handle notification preferences at scale (500M users)? . . .	337
6.17.13	Q13: How do you ensure compliance with regulations (CAN-SPAM, GDPR)? .	337
6.17.14	Q14: How do you design the collapse/replace notification feature?	338
6.17.15	Q15: How would you migrate from a monolithic notification system to this architecture?	338
6.18	Summary: System at a Glance	339
7	Design a Web Search Engine (Google Search)	340
7.1	1. Problem Statement and Scope	340
7.1.1	In Scope	340
7.1.2	Out of Scope	340
7.2	2. Functional Requirements	341
7.3	3. Non-Functional Requirements	341
7.4	4. Back-of-the-Envelope Estimation	341
7.4.1	4.1 Query Traffic	341
7.4.2	4.2 Web Scale	342
7.4.3	4.3 Inverted Index Size	342
7.4.4	4.4 Crawler Throughput	342
7.4.5	4.5 Storage Summary	342
7.4.6	4.6 Serving Infrastructure	342
7.5	5. API Design	343
7.5.1	5.1 Web Search	343
7.5.2	5.2 Autocomplete / Suggestions	344
7.5.3	5.3 Spell Correction	344
7.6	6. Data Model and Database Selection	344
7.6.1	6.1 Core Data Structures	345
7.6.2	6.2 Database Selection	346
7.6.3	6.3 Why Not a Traditional RDBMS?	346
7.7	7. High-Level Architecture	347
7.7.1	7.1 System Overview - Three Major Pipelines	347
7.7.2	7.2 Query Pipeline - Detailed Flow	349
7.7.3	7.3 Component Breakdown	350
7.8	8. Deep Dive: Core Components	350
7.8.1	8.1 Web Crawler	350
7.8.2	8.2 Inverted Index	354
7.8.3	8.3 Query Processing and Ranking	361
7.8.4	8.4 PageRank Algorithm	366
7.8.5	8.5 Autocomplete and Query Suggestions	369

7.8.6	8.6 Spell Correction	370
7.8.7	8.7 Snippet Generation	371
7.9	9. Data Partitioning and Sharding	371
7.9.1	9.1 Index Sharding Strategy	372
7.9.2	9.2 Crawler Data Partitioning	373
7.10	10. Caching Strategy	373
7.11	11. Replication and Consistency	374
7.11.1	11.1 Index Replication	375
7.11.2	11.2 Real-Time Index for Freshness	376
7.12	12. Fault Tolerance and Failure Handling	377
7.13	13. Scalability	378
7.13.1	13.1 Scaling Each Pipeline	378
7.13.2	13.2 Scaling Numbers	379
7.14	14. Monitoring and Observability	380
7.15	15. Trade-offs and Design Decisions	381
7.15.1	15.1 Key Design Decisions	382
7.15.2	15.2 Freshness vs Completeness Trade-off	383
7.15.3	15.3 Relevance vs Speed Trade-off	383
7.16	16. Interview Deep-Dive Questions	383
7.16.1	Q1: How does PageRank work and how do you compute it at scale?	383
7.16.2	Q2: How do you handle duplicate or near-duplicate content?	384
7.16.3	Q3: How do you keep the index fresh?	384
7.16.4	Q4: How do you handle multi-word queries and phrase matching?	384
7.16.5	Q5: How do you prevent search result manipulation (SEO spam)?	385
7.16.6	Q6: How do you scale the scatter-gather query architecture?	385
7.16.7	Q7: How would you design the autocomplete system?	385
7.16.8	Q8: How do you handle different languages and internationalization?	386
7.16.9	Q9: What is the role of the link graph in ranking?	386
7.16.10	Q10: How do you handle query intent classification?	386
7.16.11	Q11: How do you build knowledge panels?	387
7.16.12	Q12: How do you handle image and video search?	387
7.16.13	Q13: What is the index serving data path for a single query?	387
7.16.14	Q14: How do you handle index compaction and garbage collection?	388
7.16.15	Q15: How would you design the system differently for a vertical search engine (e.g., e-commerce)?	388
7.17	17. Summary: Key Takeaways for Interviews	390
7.18	References and Further Reading	390
8	Design a Distributed Cache (Redis / Memcached)	392
8.1	Table of Contents	392
8.2	1. Problem Statement	392
8.3	2. Functional Requirements	393
8.3.1	Core Operations	393
8.3.2	Eviction Policies	393
8.3.3	Data Structures (Redis-style)	393
8.3.4	Pub/Sub	393
8.3.5	Atomic Operations	394
8.3.6	Persistence Options	394
8.3.7	Cluster Mode	394
8.4	3. Non-Functional Requirements	394
8.5	4. Back-of-Envelope Estimation	394
8.5.1	Traffic Estimation	395

8.5.2	Storage Estimation	395
8.5.3	Node Calculation	395
8.5.4	Network Estimation	395
8.5.5	Memory Breakdown Per Node (128 GB)	395
8.6	5. API Design	396
8.6.1	Data Operations	396
8.6.2	Cluster Management	397
8.6.3	Pub/Sub	397
8.6.4	Transactions and Scripting	398
8.6.5	Administrative	398
8.7	6. Data Model	398
8.7.1	Core Data Structures (Internal Implementation)	398
8.8	7. High-Level Architecture	403
8.8.1	System Overview	404
8.8.2	Request Flow	406
8.8.3	Single Node Internal Architecture	407
8.9	8. Deep Dive	407
8.9.1	8.1 Hash Slot Architecture	408
8.9.2	8.2 Eviction Policies	409
8.9.3	8.3 Replication	413
8.9.4	8.4 Cache Patterns	415
8.10	9. Data Partitioning	419
8.10.1	Hash Slot Partitioning (Redis Cluster)	420
8.11	10. Caching Strategy	422
8.11.1	Client-Side Caching (Redis 6.0+ Tracking)	422
8.11.2	Two-Tier L1/L2 Caching	423
8.12	11. Replication and Consistency	423
8.12.1	Asynchronous Replication (Default)	424
8.12.2	Split-Brain Scenario	425
8.13	12. Fault Tolerance	425
8.13.1	Automatic Failover	426
8.13.2	Retry and Recovery Strategies	428
8.14	13. Scalability	429
8.14.1	Horizontal Scaling (Adding Nodes + Migrating Slots)	429
8.14.2	Read Replicas for Read Scaling	430
8.14.3	Pipelining for Throughput	431
8.15	14. Monitoring	431
8.15.1	Key Metrics	432
8.16	15. Trade-offs	434
8.16.1	Redis vs Memcached	434
8.16.2	Persistence vs Pure In-Memory	435
8.16.3	Single-Threaded vs Multi-Threaded	436
8.17	16. Interview Questions	436
8.17.1	Q1: How does Redis achieve sub-millisecond latency?	437
8.17.2	Q2: How does Redis Cluster handle resharding without downtime?	437
8.17.3	Q3: Explain the differences between cache avalanche, cache stampede, and cache penetration.	437
8.17.4	Q4: How can Redis be single-threaded yet handle 100K+ ops/sec?	438
8.17.5	Q5: How do you handle hot keys in a distributed cache?	438
8.17.6	Q6: Explain Redis persistence options and when to use each.	438
8.17.7	Q7: How does the Redis eviction pool work?	439

8.17.8 Q8: What happens during a Redis Cluster failover?	439
8.17.9 Q9: How would you implement a distributed rate limiter using Redis?	439
8.17.10Q10: How do you handle cache warming after a Redis restart?	440
8.17.11Q11: What is the difference between MOVED and ASK redirections?	441
8.17.12Q12: How would you design cache invalidation for a microservices architecture?	441
8.17.13Q13: Explain Redis Cluster's gossip protocol.	441
8.17.14Q14: How do you prevent cache penetration attacks?	442
8.17.15Q15: Design a leaderboard system using Redis sorted sets.	442
8.18 Summary	443
9 Design an Object Storage Service (Amazon S3)	445
9.1 1. Problem Statement and Scope	445
9.1.1 Problem	445
9.1.2 Scope	445
9.2 2. Functional Requirements	445
9.3 3. Non-Functional Requirements	446
9.3.1 Durability Math: What Does 11 Nines Mean?	446
9.4 4. Back-of-Envelope Estimation	446
9.4.1 Storage Scale	447
9.4.2 Traffic Estimates	447
9.4.3 Hardware Estimates	447
9.4.4 Erasure Coding vs Replication Cost	447
9.5 5. API Design	448
9.5.1 5.1 Object Operations	448
9.5.2 5.2 Bucket Operations	449
9.5.3 5.3 Multipart Upload	450
9.5.4 5.4 Pre-signed URL Generation (Server-side)	450
9.6 6. Data Model and Database Selection	451
9.6.1 6.1 Data Categories and Storage Choices	451
9.6.2 6.2 Object Metadata Schema	452
9.6.3 6.3 Bucket Metadata Schema	452
9.6.4 6.4 Why Not a Filesystem for Object Data?	453
9.7 7. High-Level Architecture	453
9.7.1 7.1 System Overview	454
9.7.2 7.2 Request Flow: PUT Object	455
9.7.3 7.3 Request Flow: GET Object	456
9.7.4 7.4 Component Breakdown	456
9.8 8. Deep Dive: Core Components	457
9.8.1 8.1 Data Storage and Durability	457
9.8.2 8.2 Metadata Service	460
9.8.3 8.3 Data Placement and Rebalancing	461
9.8.4 8.4 Multipart Upload	464
9.9 9. Data Partitioning and Sharding	465
9.9.1 9.1 Data Layer Partitioning	466
9.9.2 9.2 Cross-Region Replication Architecture	467
9.10 10. Caching Strategy	467
9.10.1 10.1 Multi-Layer Caching Architecture	468
9.10.2 10.2 Cache Invalidation Strategy	469
9.11 11. Replication and Consistency	469
9.11.1 11.1 Strong Read-After-Write Consistency	469
9.11.2 11.2 Consistency Levels by Operation	471
9.11.3 11.3 Cross-Region Consistency	471

9.12	12. Fault Tolerance and Failure Handling	471
9.12.1	12.1 Failure Modes and Recovery	472
9.12.2	12.2 Data Scrubbing and Integrity	473
9.12.3	12.3 Automatic Repair Pipeline	474
9.13	13. Scalability	474
9.13.1	13.1 Scaling Dimensions	475
9.13.2	13.2 Storage Class Tiering	476
9.14	14. Monitoring and Observability	476
9.14.1	14.1 Key Metrics Dashboard	477
9.14.2	14.2 Distributed Tracing	478
9.15	15. Trade-offs and Design Decisions	479
9.15.1	15.1 Key Trade-offs	479
9.16	16. Interview Deep-Dive Questions	481
9.16.1	Q1: How do you achieve 11 nines of durability?	481
9.16.2	Q2: How does erasure coding work, and why prefer it over replication?	482
9.16.3	Q3: How do you handle consistency for read-after-write?	482
9.16.4	Q4: How do you implement storage classes (hot/warm/cold/archive)?	482
9.16.5	Q5: How does multipart upload handle failures mid-upload?	482
9.16.6	Q6: How do you handle the “hot bucket” problem (one bucket with billions of objects)?	483
9.16.7	Q7: How do you design the garbage collection system?	483
9.16.8	Q8: How do you handle encryption at rest?	483
9.16.9	Q9: How does listing with prefix and delimiter work efficiently?	483
9.16.10	Q10: What happens when you need to add a new availability zone?	484
9.16.11	Q11: How do you implement pre-signed URLs securely?	484
9.16.12	Q12: How do you handle the “small file problem” in an erasure-coded system?	484
9.16.13	Q13: How does the system handle a correlated failure (e.g., software bug that corrupts all replicas)?	484
9.16.14	Q14: How do you design rate limiting for a multi-tenant object storage service?	485
9.16.15	Q15: How would you design the system to support WORM (Write Once Read Many) compliance?	485
9.17	17. References and Further Reading	485
9.18	18. Summary Cheat Sheet	486
10	Design a Ride-Sharing Service (Uber / Lyft)	487
10.1	1. Problem Statement and Scope	487
10.1.1	1.1 In Scope	487
10.1.2	1.2 Out of Scope	487
10.2	2. Functional Requirements	487
10.2.1	2.1 Rider Functions	487
10.2.2	2.2 Driver Functions	488
10.2.3	2.3 Admin / Platform Functions	488
10.3	3. Non-Functional Requirements	488
10.4	4. Back-of-Envelope Estimation	488
10.4.1	4.1 Traffic Estimates	489
10.4.2	4.2 Location Update Throughput	489
10.4.3	4.3 Storage Estimates	489
10.4.4	4.4 Bandwidth Estimates	489
10.5	5. API Design	490
10.5.1	5.1 Ride APIs	490
10.5.2	5.2 Driver Location API (High Frequency)	491
10.5.3	5.3 Driver Availability API	492

10.5.4	5.4 Rating API	492
10.6	6. Data Model and Database Selection	492
10.6.1	6.1 Core Entities and Schema	493
10.6.2	6.2 Database Selection per Data Type	495
10.6.3	6.3 Geospatial Index Design	496
10.7	7. High-Level Architecture	496
10.7.1	7.1 System Architecture Overview	497
10.7.2	7.2 Ride Request and Dispatch Flow	498
10.7.3	7.3 Component Breakdown	499
10.8	8. Deep Dive: Core Components	499
10.8.1	8.1 Location Service and Geospatial Indexing	500
10.8.2	8.2 Matching / Dispatch Algorithm	503
10.8.3	8.3 Dynamic Pricing (Surge)	506
10.8.4	8.4 ETA Calculation	508
10.9	9. Data Partitioning and Sharding	509
10.9.1	9.1 Sharding Strategy by Data Type	510
10.9.2	9.2 Cross-Shard Query Handling	511
10.10	10. Caching Strategy	511
10.10.1	10.1 Cache Layers	512
10.10.2	10.2 Cache Invalidation	513
10.11	11. Replication and Consistency	513
10.11.1	11.1 Consistency Model per Data Type	514
10.11.2	11.2 Replication Topology	515
10.11.3	11.3 Trip State Machine with Consistency	516
10.12	12. Fault Tolerance and Failure Handling	517
10.12.1	12.1 Failure Scenarios and Mitigations	518
10.12.2	12.2 Circuit Breaker Pattern	519
10.13	13. Scalability	519
10.13.1	13.1 Scaling Strategy	520
10.13.2	13.2 Handling 10x Traffic Spikes (New Year's Eve)	521
10.14	14. Monitoring and Observability	521
10.14.1	14.1 Key Metrics Dashboard	522
10.14.2	14.2 Alerting Rules	523
10.14.3	14.3 Distributed Tracing	523
10.15	15. Trade-offs and Design Decisions	523
10.15.1	15.1 Key Trade-offs	524
10.15.2	15.2 Why Not a Single Database?	526
10.16	16. Interview Deep-Dive Questions	526
10.16.1	16.1 Q1: How do you handle drivers at city boundaries?	526
10.16.2	16.2 Q2: How do you prevent surge pricing abuse?	526
10.16.3	16.3 Q3: How do you handle ride cancellation mid-trip?	527
10.16.4	16.4 Q4: How would you design ride pooling/sharing?	527
10.16.5	16.5 Q5: How do you ensure payment consistency?	528
10.16.6	16.6 Q6: How does the system handle GPS inaccuracy?	528
10.16.7	16.7 Q7: How do you handle a complete data center failure?	529
10.16.8	16.8 Q8: How do you calculate the optimal pickup point?	529
10.16.9	16.9 Q9: How does the system handle peak events (concerts, sports)?	530
10.16.10	16.10 Q10: How do you prevent fraud (fake rides, GPS spoofing)?	530
10.16.11	16.11 Q11: How do you design the notification system for ride updates?	530
10.16.12	16.12 Q12: How do you handle international expansion?	531
10.16.13	16.13 Q13: How do you ensure driver and rider safety?	531

10.16.10	14: How do you handle scheduled (future) rides?	532
10.16.10	15: How would you redesign the matching algorithm using ML?	532
10.17	17. System Evolution and Future Considerations	533
10.17.1	Phase 1: MVP (Months 1-3)	533
10.17.2	Phase 2: Scale (Months 4-8)	533
10.17.3	Phase 3: Optimization (Months 9-12)	533
10.17.4	Phase 4: Platform (Year 2+)	533
10.18	18. Quick Reference Card	534

11	Design a Payment System (Stripe / PayPal)	535
11.1	1. Problem Statement and Scope	535
11.1.1	In Scope	535
11.1.2	Out of Scope	535
11.2	2. Functional Requirements	536
11.3	3. Non-Functional Requirements	536
11.4	4. Back-of-Envelope Estimation	536
11.4.1	Traffic Estimates	537
11.4.2	Storage Estimates	537
11.4.3	Compute Estimates	537
11.4.4	Financial Estimates	537
11.5	5. API Design	538
11.5.1	5.1 Create Payment (Auth + Capture or Auth Only)	538
11.5.2	5.2 Get Payment	539
11.5.3	5.3 Capture Payment (for Auth-only Flows)	539
11.5.4	5.4 Refund Payment	540
11.5.5	5.5 List Payments	540
11.5.6	5.6 Create Payment Method (Tokenization)	541
11.5.7	5.7 Webhook Registration	541
11.5.8	5.8 Idempotency Key Handling	542
11.6	6. Data Model and Database Selection	542
11.6.1	6.1 Database Selection Rationale	542
11.6.2	6.2 Core Schema	543
11.6.3	6.3 Double-Entry Ledger: How Entries Balance	546
11.7	7. High-Level Architecture	546
11.7.1	7.1 System Overview	547
11.7.2	7.2 Complete Payment Flow	549
11.7.3	7.3 Component Breakdown	550
11.8	8. Deep Dive: Core Components	552
11.8.1	8.1 Payment Processing Flow and State Machine	552
11.8.2	8.2 Idempotency and Exactly-Once Processing	554
11.8.3	8.3 Double-Entry Ledger	557
11.8.4	8.4 Payment Security and PCI Compliance	559
11.9	9. Data Partitioning and Sharding	563
11.9.1	Sharding Strategy	563
11.9.2	Cross-Shard Transactions via Saga	565
11.10	10. Caching Strategy	566
11.11	11. Replication and Consistency	567
11.11.1	Consistency Requirements by Data Type	568
11.11.2	PostgreSQL Replication Topology	569
11.11.3	Distributed Transaction: Saga Pattern	569
11.12	12. Fault Tolerance and Failure Handling	570
11.12.1	Payment Timeout Handling	571

11.12.2	PSP Failover	572
11.12.3	Dead Letter Queue for Webhooks	573
11.12.4	Reconciliation Catches Discrepancies	574
11.13	Scalability	574
11.13.1	Horizontal Scaling Strategy	575
11.13.2	Database Scaling Path	575
11.13.3	Async Settlement Processing	576
11.14	Monitoring and Observability	576
11.14.1	Key Metrics Dashboard	577
11.14.2	Distributed Tracing	577
11.14.3	Alerting Strategy	578
11.15	Trade-offs and Design Decisions	578
11.15.1	Synchronous vs. Asynchronous Payment Processing	579
11.15.2	Saga vs. Two-Phase Commit (2PC)	580
11.15.3	Single PSP vs. Multi-PSP	580
11.15.4	Real-Time vs. Batch Settlement	581
11.15.5	Additional Design Decisions	581
11.16	Interview Deep-Dive Questions	581
11.16.1	Q1: How do you prevent double charging?	581
11.16.2	Q2: How do you handle a payment that succeeds at the bank but your system crashes before recording it?	582
11.16.3	Q3: How does reconciliation work?	582
11.16.4	Q4: How do you handle currency conversion?	583
11.16.5	Q5: How do you design for PCI compliance without handling raw card numbers?	583
11.16.6	Q6: How do you handle partial captures and partial refunds?	584
11.16.7	Q7: How does the webhook signing and verification work?	584
11.16.8	Q8: How do you handle subscription/recurring payments?	584
11.16.9	Q9: How do you ensure exactly-once settlement (no double payouts to merchants)?	585
11.16.10	Q10: How do you handle chargebacks/disputes?	585
11.16.11	Q11: How does the payment router decide which PSP to use?	585
11.16.12	Q12: How do you handle multi-region deployment for a payment system?	586
11.16.13	Q13: How do you handle rate limiting for payment APIs?	586
11.16.14	Q14: What happens if Kafka goes down? Are payments lost?	586
11.16.15	Q15: How would you migrate from a monolithic payment system to this microservices architecture?	587
11.17	Summary: Key Takeaways for Interview	588
12	Design a Ticket Booking System (BookMyShow / Ticketmaster)	589
12.1	1. Problem Statement and Scope	589
12.1.1	In Scope	589
12.1.2	Out of Scope	589
12.2	2. Functional Requirements	590
12.2.1	User Journeys	590
12.3	3. Non-Functional Requirements	590
12.3.1	Consistency Requirements by Data Type	591
12.4	4. Back-of-Envelope Estimation	591
12.4.1	Traffic Estimates	591
12.4.2	Storage Estimates	591
12.4.3	Bandwidth Estimates	592
12.4.4	Redis Memory for Seat Holds	592
12.5	5. API Design	592

12.5.1	5.1 Event Discovery APIs	592
12.5.2	5.2 Seat Map API	593
12.5.3	5.3 Booking APIs	594
12.5.4	5.4 Search API	596
12.5.5	Rate Limiting Strategy	596
12.6	6. Data Model and Database Selection	596
12.6.1	6.1 Entity-Relationship Overview	597
12.6.2	6.2 Database Selection	598
12.6.3	6.3 Seat Status State Machine	599
12.6.4	6.4 Key Table Schemas	600
12.7	7. High-Level Architecture	601
12.7.1	7.1 System Architecture Diagram	602
12.7.2	7.2 Booking Flow – Sequence Diagram	604
12.7.3	7.3 Component Breakdown	605
12.8	8. Deep Dive: Core Components	605
12.8.1	8.1 Seat Inventory and Locking	605
12.8.2	8.2 Flash Sale / High-Demand Event Handling	610
12.8.3	8.3 Payment Integration	614
12.8.4	8.4 Search and Discovery	618
12.9	9. Data Partitioning and Sharding	620
12.9.1	Sharding Strategy	621
12.9.2	Hot Partition Mitigation for Popular Events	622
12.10	10. Caching Strategy	622
12.10.1	Multi-Layer Cache Architecture	623
12.10.2	Cache Invalidation Strategy	624
12.11	11. Replication and Consistency	624
12.11.1	Replication Topology	625
12.11.2	Consistency Guarantees by Operation	626
12.12	12. Fault Tolerance and Failure Handling	626
12.12.1	Failure Modes and Recovery	627
12.12.2	Graceful Degradation Strategy	629
12.12.3	Expired Hold Cleanup	630
12.13	13. Scalability	630
12.13.1	Horizontal Scaling Architecture	631
12.13.2	Scaling Bottleneck Analysis	632
12.14	14. Monitoring and Observability	632
12.14.1	Key Metrics Dashboard	633
12.14.2	Anomaly Detection	635
12.15	15. Trade-offs and Design Decisions	635
12.15.1	Decision 1: Optimistic vs Pessimistic Locking	636
12.15.2	Decision 2: Queue vs First-Come-First-Served	637
12.15.3	Decision 3: Pre-Allocated vs Dynamic Seat Selection	638
12.15.4	Decision 4: Monolith vs Microservices	639
12.15.5	Decision 5: Hold Duration	640
12.16	16. Interview Deep-Dive Questions	640
12.16.1	Q1: How do you prevent double booking?	640
12.16.2	Q2: How do you handle 500K users trying to book 50K seats simultaneously?	640
12.16.3	Q3: What happens if payment fails after seats are held?	641
12.16.4	Q4: How do you handle seat selection for group bookings (10+ seats)?	641
12.16.5	Q5: How would you design dynamic pricing?	641

12.16.6Q6: How do you handle the thundering herd problem when a hold expires during a flash sale?	642
12.16.7Q7: How do you ensure exactly-once payment processing?	642
12.16.8Q8: How do you handle multi-region deployment?	643
12.16.9Q9: How do you handle partial failures in the booking confirmation flow?	643
12.16.10Q10: How would you handle a scenario where Redis loses data (e.g., server crash without persistence)?	643
12.16.11Q11: How do you prevent bots and scalpers from hoarding tickets?	644
12.16.12Q12: How do you design the seat map for real-time updates?	644
12.16.13Q13: How do you handle timezone and scheduling for global events?	644
12.16.14Q14: How do you handle inventory sync between the online system and box office (walk-in) sales?	645
12.16.15Q15: How would you add a waitlist feature for sold-out events?	645
12.1717. Summary	646

13 HLD 13: Design a Typeahead / Autocomplete System 647

13.1 Table of Contents	647
13.2 1. Problem Statement	647
13.3 2. Functional Requirements	648
13.4 3. Non-Functional Requirements	648
13.5 4. Back-of-Envelope Estimation	649
13.6 5. API Design	649
13.6.1 5.1 Get Suggestions	649
13.6.2 5.2 Report Selection (Analytics)	650
13.6.3 5.3 Client-Side Optimizations	650
13.7 6. Data Model	650
13.7.1 6.1 Trie Node (In-Memory)	650
13.7.2 6.2 Query Log (Kafka Topic: query-logs)	651
13.7.3 6.3 Aggregated Query Frequency (HDFS / S3)	651
13.7.4 6.4 User Search History (Cassandra)	651
13.7.5 6.5 Trending Queries (Redis Sorted Set)	651
13.8 7. High-Level Architecture	651
13.8.1 7.1 System Overview	652
13.8.2 7.2 Request Flow	653
13.9 8. Deep Dive	653
13.9.1 8.1 Trie Data Structure	653
13.9.2 8.2 Data Collection Pipeline	655
13.9.3 8.3 Ranking and Scoring	656
13.9.4 8.4 Trie Sharding	657
13.10 9. Partitioning	657
13.11 10. Caching	657
13.11.1 Multi-Layer Architecture	658
13.12 11. Replication	658
13.12.1 Trie Replication Model	658
13.13 12. Fault Tolerance	659
13.14 13. Scalability	659
13.15 14. Monitoring and Observability	660
13.15.1 Key Metrics	660
13.15.2 Alerting Rules	660
13.16 15. Trade-offs and Alternatives	660
13.16.1 15.1 Trie vs Inverted Index	660
13.16.2 15.2 Prefix Match vs Fuzzy Match	661

13.16.315.3 Real-Time vs Batch Updates	661
13.16.415.4 Caching Strategy	661
13.1716. Interview Questions	661
13.18Summary	663
14 Design a Distributed Key-Value Store (Dynamo-Style)	664
14.1 Table of Contents	664
14.2 1. Problem Statement	664
14.3 2. Functional Requirements	665
14.3.1 Core Operations	665
14.3.2 Consistency Levels (per-request tunable)	665
14.3.3 Additional Features	665
14.4 3. Non-Functional Requirements	666
14.4.1 SLA Targets	666
14.5 4. Back-of-Envelope Estimation	666
14.5.1 Traffic	666
14.5.2 Storage	666
14.5.3 Bandwidth	666
14.5.4 Node Count	667
14.5.5 Memory	667
14.6 5. API Design	667
14.6.1 Put (Write)	667
14.6.2 Get (Read)	668
14.6.3 Delete	668
14.6.4 Internal RPC API (Node-to-Node)	669
14.7 6. Data Model	669
14.7.1 6.1 LSM Tree (Log-Structured Merge Tree)	669
14.7.2 6.2 Storage Engine	670
14.8 7. High-Level Architecture	674
14.8.1 Request Flow (Write)	675
14.9 8. Deep Dive	676
14.9.1 8.1 Consistent Hashing with Virtual Nodes	676
14.9.2 8.2 Quorum Consensus	677
14.9.3 8.3 Conflict Resolution	679
14.9.4 8.4 Failure Detection	680
14.109. Data Partitioning	683
14.10.1Consistent Hash Ring with Token Ranges	683
14.10.2Virtual Node Configuration	684
14.10.3Range Ownership and Rebalancing	684
14.1110. Caching	684
14.11.1Multi-Layer Caching Architecture	685
14.11.2Row Cache	685
14.11.3Key Cache	686
14.11.4Bloom Filter Tuning	686
14.1211. Replication and Consistency	686
14.12.1Replication Strategy	686
14.12.2Tunable Consistency Levels	687
14.12.3Read Repair	687
14.12.4Anti-Entropy (Background Repair)	687
14.1312. Fault Tolerance	688
14.13.1Failure Scenarios and Recovery Mechanisms	688
14.13.2Hinted Handoff (Recovery Flow)	688

14.13.3	Read Repair (Consistency Recovery)	689
14.13.4	Merkle Tree Synchronization (Anti-Entropy)	689
14.13.5	Re-Replication on Permanent Failure	689
14.14	Scalability	690
14.14.1	Linear Scaling	690
14.14.2	Auto Rebalancing	691
14.14.3	Cross-Datacenter Replication	691
14.15	Monitoring	691
14.15.1	Key Metrics Dashboard	692
14.15.2	Critical Alerts	692
14.15.3	Operational Runbooks	693
14.16	Trade-offs	693
14.16.1	AP vs CP	693
14.16.2	Vector Clocks vs Last-Write-Wins (LWW)	693
14.16.3	LSM Tree vs B-Tree	694
14.16.4	Compaction Strategies Comparison	694
14.17	Interview Questions	694
14.17.1	Q1: How do you handle hotspot keys?	694
14.17.2	Q2: What happens during a network partition?	694
14.17.3	Q3: Explain vector clocks and when they produce conflicts.	695
14.17.4	Q4: How does read repair work? When is it insufficient?	695
14.17.5	Q5: How does LSM differ from B-Tree for this use case?	695
14.17.6	Q6: How do you handle deletes in a distributed KV store?	695
14.17.7	Q7: What is the impact of compaction on latency?	695
14.17.8	Q8: How do you tune N, W, R for different use cases?	695
14.17.9	Q9: How does consistent hashing handle heterogeneous hardware?	696
14.17.10	Q10: What is a sloppy quorum and why is it important?	696
14.17.11	Q11: How do Merkle trees help with anti-entropy?	696
14.17.12	Q12: How do you handle cross-datacenter replication?	696
14.17.13	Q13: What is the difference between hinted handoff and anti-entropy?	696
14.17.14	Q14: How do you prevent tombstone accumulation?	697
14.17.15	Q15: How would you migrate from a single-node KV store to this distributed design?	697
14.18	Summary	698
15	Design a Metrics and Monitoring System (Datadog / Prometheus)	699
15.1	Table of Contents	699
15.2	1. Problem Statement	699
15.2.1	Real-World Context	700
15.3	2. Functional Requirements	700
15.3.1	Core Features	700
15.3.2	Out of Scope (for initial design)	700
15.4	3. Non-Functional Requirements	700
15.5	4. Back-of-Envelope Estimation	701
15.5.1	Ingestion Volume	701
15.5.2	Storage	701
15.5.3	Network Bandwidth	702
15.5.4	Node Count Estimates	702
15.6	5. API Design	702
15.6.1	5.1 Metrics Ingestion API	703
15.6.2	5.2 Prometheus Remote Write (Pull-to-Push Bridge)	704
15.6.3	5.3 Query API	704

15.6.4	5.4 Alerting API	705
15.6.5	5.5 Dashboard API	706
15.7	6. Data Model	706
15.7.1	6.1 Time-Series Data Model	706
15.7.2	6.2 TSDB On-Disk Layout	707
15.7.3	6.3 Inverted Index for Labels	708
15.7.4	6.4 Metadata Store (MySQL / PostgreSQL)	709
15.8	7. High-Level Architecture	710
15.8.1	7.1 System Overview	711
15.8.2	7.2 Query Path	714
15.8.3	7.3 Alerting Path	715
15.9	8. Deep Dive	716
15.9.1	8.1 TSDB Storage Engine	716
15.9.2	8.2 Ingestion Pipeline	719
15.9.3	8.3 Query Engine	722
15.9.4	8.4 Alerting Engine	725
15.10	9. Partitioning Strategy	728
15.10.1	9.1 Partitioning Dimensions	729
15.10.2	9.2 Consistent Hashing for Shard Assignment	730
15.11	10. Caching Strategy	731
15.12	11. Replication	732
15.13	12. Fault Tolerance	733
15.13.1	12.1 Failure Scenarios and Mitigations	734
15.13.2	12.2 WAL Recovery Process	735
15.14	13. Scalability	735
15.14.1	13.1 Horizontal Scaling Strategy	736
15.15	14. Monitoring the Monitoring System (Meta)	738
15.16	15. Trade-offs and Design Decisions	740
15.16.1	15.1 Push vs Pull Ingestion	741
15.16.2	15.2 Custom TSDB vs Existing Solutions	741
15.16.3	15.3 PromQL vs SQL for Query Language	742
15.16.4	15.4 Memory vs Disk Trade-off	742
15.16.5	15.5 Summary of Key Trade-offs	743
15.17	16. Interview Questions and Answers	743
15.17.1	Q1: How do you handle high cardinality labels that cause a cardinality explosion?	743
15.17.2	Q2: Explain Gorilla compression and why it works so well for time-series data.	744
15.17.3	Q3: How do you prevent alert fatigue in a large-scale monitoring system?	744
15.17.4	Q4: How do you handle clock skew when collecting metrics from thousands of hosts?	744
15.17.5	Q5: How would you design multi-tenant isolation in a shared monitoring platform?	745
15.17.6	Q6: How does the system handle a sudden 10x spike in ingestion rate?	745
15.17.7	Q7: How do you implement efficient range queries over months of data?	746
15.17.8	Q8: What happens when a TSDB storage node runs out of disk space?	746
15.17.9	Q9: How would you implement recording rules for pre-computed aggregations?	747
15.17.10	Q10: How do you handle the “thundering herd” problem when many dashboards refresh simultaneously?	747
15.17.11	Q11: How would you implement anomaly detection without ML?	747
15.17.12	Q12: How do you ensure alert notifications are delivered even when the monitoring system is partially down?	748

15.17.1Q13: How would you migrate from Prometheus to this distributed system with zero downtime?	748
15.17.1Q14: How do you handle metrics from ephemeral containers and serverless functions?	749
15.17.1Q15: What are the most critical SLOs for a monitoring system itself, and how do you measure them?	749
15.18Summary	750

Chapter 1

Design a URL Shortener (TinyURL / Bitly)

Difficulty: Medium-Hard | **Companies:** Google, Amazon, Microsoft, Meta, Uber

1.1 Table of Contents

1. Problem Statement and Scope
 2. Functional Requirements
 3. Non-Functional Requirements
 4. Back-of-Envelope Estimation
 5. API Design
 6. Data Model and Database Selection
 7. High-Level Architecture
 8. Deep Dive: Core Components
 9. Data Partitioning and Sharding
 10. Caching Strategy
 11. Replication and Consistency
 12. Fault Tolerance and Failure Handling
 13. Scalability
 14. Monitoring and Observability
 15. Trade-offs and Design Decisions
 16. Interview Deep-Dive Questions
-

1.2 1. Problem Statement and Scope

1.2.1 Problem Statement

Design a URL shortening service similar to TinyURL or Bitly that takes a long URL (e.g., https://www.example.com/articles/2024/distributed-systems/cap-theorem?ref=newsletter&utm_source=twitter) and converts it to a short, unique alias (e.g., <https://short.ly/aB3x7Kq>) that redirects to the original URL when accessed.

1.2.2 Why is This Problem Important?

1. **Shareability:** Long URLs are unwieldy in tweets, text messages, and print media.
2. **Analytics:** Shortened links enable click tracking, geographic analysis, and A/B testing.

3. **Branding:** Custom short domains (e.g., `amzn.to`, `goo.gl`) reinforce brand identity.
4. **Link management:** Centralized control over redirects allows updating destination URLs without changing the shared link.

1.2.3 Scope Definition

IN SCOPE
- Shorten a given URL to a unique short alias - Redirect short URL to the original long URL - Custom short aliases (user-chosen slugs) - Link expiration (TTL-based) - Basic analytics (click count, referrer, geo) - Rate limiting to prevent abuse - API-based access (RESTful)
OUT OF SCOPE
- Full user management / OAuth / social login - Payment and subscription tiers - Link-in-bio pages or landing page builders - QR code generation (mention as extension) - Deep link handling for mobile apps

1.3 2. Functional Requirements

- **F1 - Shorten URL:** Given a long URL, generate a unique short URL (7-character key by default).
- **F2 - Redirect:** When a user visits the short URL, redirect them (HTTP 301/302) to the original long URL.
- **F3 - Custom Aliases:** Allow users to optionally specify a custom short key (e.g., `short.ly/my-brand`).
- **F4 - Link Expiration:** Support optional expiration time (TTL). Default lifetime: 5 years. Expired links return HTTP 410.
- **F5 - Link Deletion:** Authenticated users can delete their own shortened URLs.
- **F6 - Analytics:** Track click count, timestamp, referrer, user-agent, geographic location per short URL.
- **F7 - Duplicate Detection:** If the same long URL is submitted by the same user, return the existing short URL instead of creating a new one.
- **F8 - Rate Limiting:** Limit URL creation to prevent spam (e.g., 100 URLs/hour per API key, 1000 redirects/sec per IP).
- **F9 - API Key Authentication:** All write operations require a valid API key. Read (redirect) is public.
- **F10 - Preview Mode:** Appending + to a short URL (e.g., `short.ly/aB3x7Kq+`) shows a preview page instead of redirecting.

1.4 3. Non-Functional Requirements

- **Latency:** p50 < 5ms, p99 < 50ms redirect; Rationale: Redirect must feel instant; any delay loses users.
- **Throughput:** 50K reads/sec, 500 writes/sec; Rationale: Read-heavy workload (100:1 read-to-write ratio).
- **Availability:** 99.99% (52 min downtime/year); Rationale: Revenue-generating links must always resolve; use multi-region.
- **Consistency:** Eventual consistency (reads); Rationale: A newly created short URL may take ~1-2s to propagate to all replicas.
- **Durability:** 99.999999999% (11 nines); Rationale: Once a URL is stored, it must not be lost. Replicate across AZs/regions.
- **Scalability:** 100B+ URLs over 5 years; Rationale: Must handle internet-scale growth without re-architecture.
- **Security:** No enumeration attacks; Rationale: Short keys must not be sequential/predictable.

1.4.1 Consistency vs. Availability Trade-off

For a URL shortener, **availability is more critical** than strong consistency. If a user creates a short URL and it takes 1-2 seconds to propagate, that is acceptable. However, if the redirect service is down, every shared link on the internet pointing to our service is broken. This places us firmly in the **AP** side of the CAP theorem during network partitions.

1.5 4. Back-of-Envelope Estimation

1.5.1 4.1 Traffic Estimation

TRAFFIC ASSUMPTIONS	
Daily Active Users (DAU):	10 million
Monthly Active Users (MAU):	100 million
URL shortenings per day:	100 million (10 per DAU avg)
But most users only read:	~1M writes/day, 100M reads/day
Read:Write ratio:	100 : 1
WRITE PATH	
New URLs per day:	1,000,000
Write QPS:	$1M / 86400 \approx 12$ QPS
Peak write QPS:	$12 \times 5 = \sim 60$ QPS
READ PATH	
Redirects per day:	100,000,000
Read QPS:	$100M / 86400 \approx 1200$
Peak read QPS:	$1200 \times 5 = \sim 6000$
(Bitly processes ~600M clicks/month $\approx$ 230 QPS average)	

1.5.2 4.2 Storage Estimation

STORAGE ESTIMATION	
Per URL record:	
short_key (7 chars)	: 7 bytes
original_url (avg 200 chars)	: 200 bytes
user_id (UUID)	: 16 bytes
created_at (timestamp)	: 8 bytes
expires_at (timestamp)	: 8 bytes
click_count (counter)	: 8 bytes
metadata (JSON blob avg)	: ~250 bytes
Total per record	: ~500 bytes
New URLs per month: 1M/day × 30 = 30 million	
New URLs per year: 30M × 12 = 360 million	
Over 5 years: 360M × 5 = 1.8 billion URLs	
Storage for URLs: 1.8B × 500 bytes = 900 GB ≈ ~1 TB	
With replication (×3): ~3 TB	
With analytics data: ~10-15 TB (click events are voluminous)	

Key insight: 1 TB of URL data is modest. Even a single modern server with NVMe SSDs can hold this. The challenge is not raw storage but **read throughput** and **latency at the tail** (p99).

1.5.3 4.3 Bandwidth Estimation

BANDWIDTH ESTIMATION	
Incoming (write):	
12 QPS × 500 bytes = 6 KB/s	
(Negligible)	
Outgoing (read/redirect):	
1200 QPS × 500 bytes = 600 KB/s ≈ ~5 Mbps	
(Redirect responses are just HTTP headers, ~200 bytes each)	
Actual: 1200 × 200 bytes = 240 KB/s ≈ ~2 Mbps	
Peak outgoing:	
6000 QPS × 200 bytes = 1.2 MB/s ≈ ~10 Mbps	
Verdict: Bandwidth is NOT a bottleneck for this system.	

1.5.4 4.4 Cache/Memory Estimation

CACHE ESTIMATION
Applying the 80-20 rule: 20% of URLs generate 80% of traffic. Total URLs to cache (hot set): Daily unique URLs accessed: ~20M Hot 20%: $20M \times 0.2 = 4$ million URLs Memory needed: $4M \times 500 \text{ bytes} = 2 \text{ GB}$ With overhead (hash table, pointers, metadata): ~4-5 GB per cache node Recommendation: A single Redis instance (64 GB) can hold the entire hot set with room to spare. Use 2-3 Redis replicas for HA. Expected cache hit ratio: 85-95% (Viral/popular URLs dominate traffic)

1.6 5. API Design

1.6.1 RESTful API Endpoints

1.6.1.1 5.1 Create Short URL

POST /api/v1/urls

Headers:

```
Authorization: Bearer <api_key>
Content-Type: application/json
```

Request Body:

```
{
  "long_url": "https://www.example.com/very/long/path?with=params",
  "custom_alias": "my-link",           // optional
  "expires_at": "2027-01-01T00:00:00Z", // optional, default 5 years
  "metadata": {                       // optional
    "campaign": "spring-sale",
    "source": "email"
  }
}
```

Response: 201 Created

```
{
  "short_url": "https://short.ly/my-link",
  "short_key": "my-link",
  "long_url": "https://www.example.com/very/long/path?with=params",
  "created_at": "2026-04-09T10:30:00Z",
  "expires_at": "2027-01-01T00:00:00Z",
  "user_id": "usr_abc123"
}
```

Error Responses:

```
400 - Invalid URL format or custom alias contains invalid characters
409 - Custom alias already taken
429 - Rate limit exceeded
```

1.6.1.2 5.2 Redirect (Get Original URL)

GET /{short_key}

Response: 301 Moved Permanently (or 302 Found)

```
Location: https://www.example.com/very/long/path?with=params
Cache-Control: private, max-age=3600
```

Error Responses:

```
404 - Short URL not found
410 - Short URL has expired
```

1.6.1.3 5.3 Delete Short URL

DELETE /api/v1/urls/{short_key}

Headers:

Authorization: Bearer <api_key>

Response: 204 No Content

Error Responses:

401 - Unauthorized

403 - Forbidden (not the owner)

404 - Short URL not found

1.6.1.4 5.4 Get Analytics

GET /api/v1/urls/{short_key}/analytics

Headers:

Authorization: Bearer <api_key>

Query Parameters:

?from=2026-01-01&to=2026-04-01

&granularity=day // hour, day, week, month

&group_by=country // country, referrer, device, browser

Response: 200 OK

```
{
  "short_key": "aB3x7Kq",
  "total_clicks": 142857,
  "unique_visitors": 98234,
  "time_series": [
    { "date": "2026-04-01", "clicks": 1523, "unique": 1102 },
    { "date": "2026-04-02", "clicks": 1687, "unique": 1230 },
    ...
  ],
  "top_referrers": [
    { "referrer": "twitter.com", "clicks": 45023 },
    { "referrer": "facebook.com", "clicks": 32100 },
    ...
  ],
  "top_countries": [
    { "country": "US", "clicks": 62000 },
    { "country": "IN", "clicks": 28000 },
    ...
  ],
  "devices": {
    "mobile": 85000,
    "desktop": 52000,
    "tablet": 5857
  }
}
```

1.6.1.5 5.5 Bulk Create (Batch API)

POST /api/v1/urls/batch

Headers:

Authorization: Bearer <api_key>
Content-Type: application/json

Request Body:

```
{
  "urls": [
    { "long_url": "https://example.com/page1" },
    { "long_url": "https://example.com/page2", "custom_alias": "pg2" },
    ...
  ]
}
```

Response: 200 OK (partial success possible)

```
{
  "results": [
    { "status": "created", "short_url": "https://short.ly/xK9mP2q", ... },
    { "status": "conflict", "error": "Alias 'pg2' already exists" },
    ...
  ]
}
```

1.6.2 API Design Considerations

API DESIGN PRINCIPLES

1. Idempotency: POST with same long_url + user returns existing short URL rather than creating a duplicate.
2. Rate Limiting Headers:
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 73
X-RateLimit-Reset: 1617235200
3. Versioning: /api/v1/ prefix for backward compatibility.
4. Pagination: Analytics endpoints use cursor-based pagination (not offset-based) for consistency under concurrent writes.
5. The redirect endpoint (GET /{short_key}) is intentionally separate from the API namespace (/api/v1/) to keep URLs as short as possible.

1.7 6. Data Model and Database Selection

1.7.1 6.1 Schema Design

1.7.1.1 URL Table (Primary)

urls		
Column	Type	Notes
short_key	VARCHAR(16)	PRIMARY KEY, indexed
original_url	TEXT	The destination URL (max 2048 chars)
user_id	UUID	Foreign key to users table
created_at	TIMESTAMP	Immutable, set on creation
expires_at	TIMESTAMP	NULL = never expires
is_active	BOOLEAN	Soft delete flag, default TRUE
click_count	BIGINT	Denormalized counter for fast reads
custom_alias	BOOLEAN	Whether the key is user-chosen
metadata	JSONB	Campaign tags, source, etc.
INDEXES:		
- PRIMARY KEY (short_key) -- 0(1) lookups		
- INDEX idx_user_urls (user_id, created_at DESC)		
- INDEX idx_expiry (expires_at) WHERE expires_at IS NOT NULL		
- INDEX idx_original (user_id, original_url) -- duplicate detect		

1.7.1.2 User Table

users		
Column	Type	Notes
user_id	UUID	PRIMARY KEY
api_key	VARCHAR(64)	UNIQUE, hashed
email	VARCHAR(255)	UNIQUE
tier	ENUM	free, pro, enterprise
rate_limit	INT	URLs per hour (varies by tier)
created_at	TIMESTAMP	
is_active	BOOLEAN	Account status
INDEXES:		
- PRIMARY KEY (user_id)		
- UNIQUE INDEX (api_key)		
- UNIQUE INDEX (email)		

1.7.1.3 Click Events Table (Analytics)

click_events		
Column	Type	Notes
event_id	UUID / ULID	PRIMARY KEY (time-sortable)
short_key	VARCHAR(16)	Partition key
clicked_at	TIMESTAMP	Event time (sort key in time-series)
ip_hash	VARCHAR(64)	SHA256 of IP (privacy-preserving)
country	CHAR(2)	ISO country code from GeoIP
city	VARCHAR(100)	From GeoIP lookup
referrer	TEXT	HTTP Referer header
user_agent	TEXT	Browser/device info
device_type	ENUM	mobile, desktop, tablet, bot
browser	VARCHAR(50)	Parsed from user-agent
os	VARCHAR(50)	Parsed from user-agent
PARTITION BY: RANGE (clicked_at) -- monthly partitions		
INDEXES:		
- PRIMARY KEY (short_key, clicked_at, event_id)		
- INDEX (short_key, country)		
- INDEX (short_key, referrer)		

1.7.2 6.2 Database Selection Rationale

DATABASE SELECTION MATRIX			
Data Store	Use Case	Technology	Why
URL Mapping (Primary)	Key-value lookups	DynamoDB / Cassandra	- Simple key→value lookup - Horizontally scalable - Single-digit ms latency - No complex joins needed - Tunable consistency
User Data	Relational with joins	PostgreSQL (RDS)	- ACID transactions - Complex queries - Small dataset (~1M) - Strong consistency
Click Events (Analytics)	Time-series append-only	ClickHouse / Apache Druid / TimescaleDB	- Columnar storage - Fast aggregations - Partition by time - Billions of rows
Cache	Hot URL lookups	Redis (Cluster)	- Sub-millisecond reads - Built-in TTL support - LRU eviction policy
Message Queue	Click event streaming	Apache Kafka	- Durable event stream - High throughput - Consumer groups
Note on DynamoDB vs Cassandra: - DynamoDB: Managed, auto-scaling, pay-per-request. Best for AWS. - Cassandra: Multi-DC replication, tunable consistency, open source. - Both offer $O(1)$ key lookups at scale with predictable latency.			

1.7.2.1 Why Not a Traditional RDBMS for URL Storage?

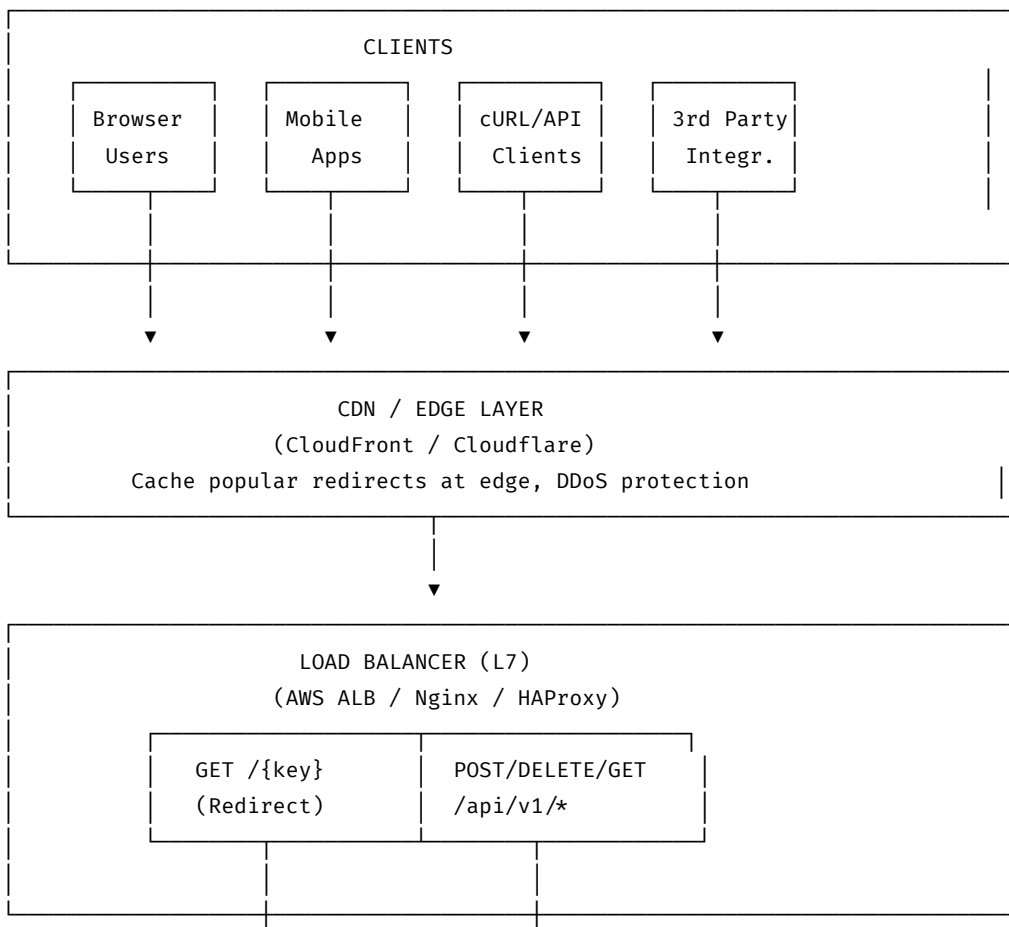
1. **No complex queries:** URL lookup is a pure key-value operation. No joins, no aggregations, no WHERE clauses beyond the primary key.
2. **Scale ceiling:** A single PostgreSQL instance tops out at ~10K-50K QPS for simple reads. With 1.8B rows and 6K peak QPS, we would need multiple read replicas and the operational burden grows.
3. **Sharding complexity:** RDBMS sharding is operationally painful (schema changes, cross-shard queries). DynamoDB/Cassandra handle partitioning natively.

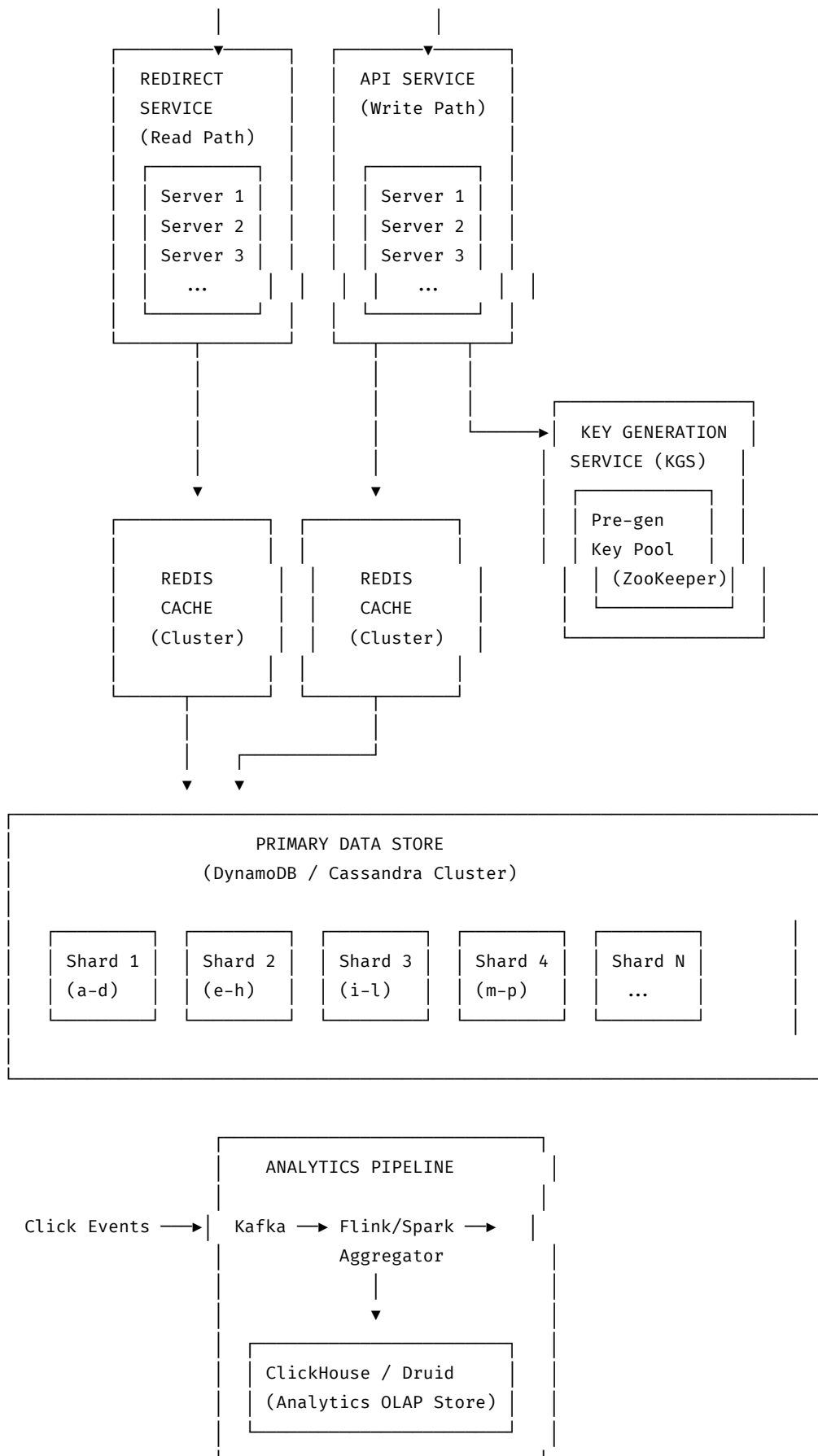
1.7.2.2 Why Keep PostgreSQL for Users?

The user table is small (~1M rows), requires ACID guarantees (e.g., API key rotation), and benefits from relational queries (e.g., “find all URLs by user X with expiry before date Y”). A single PostgreSQL instance handles this trivially.

1.8 7. High-Level Architecture

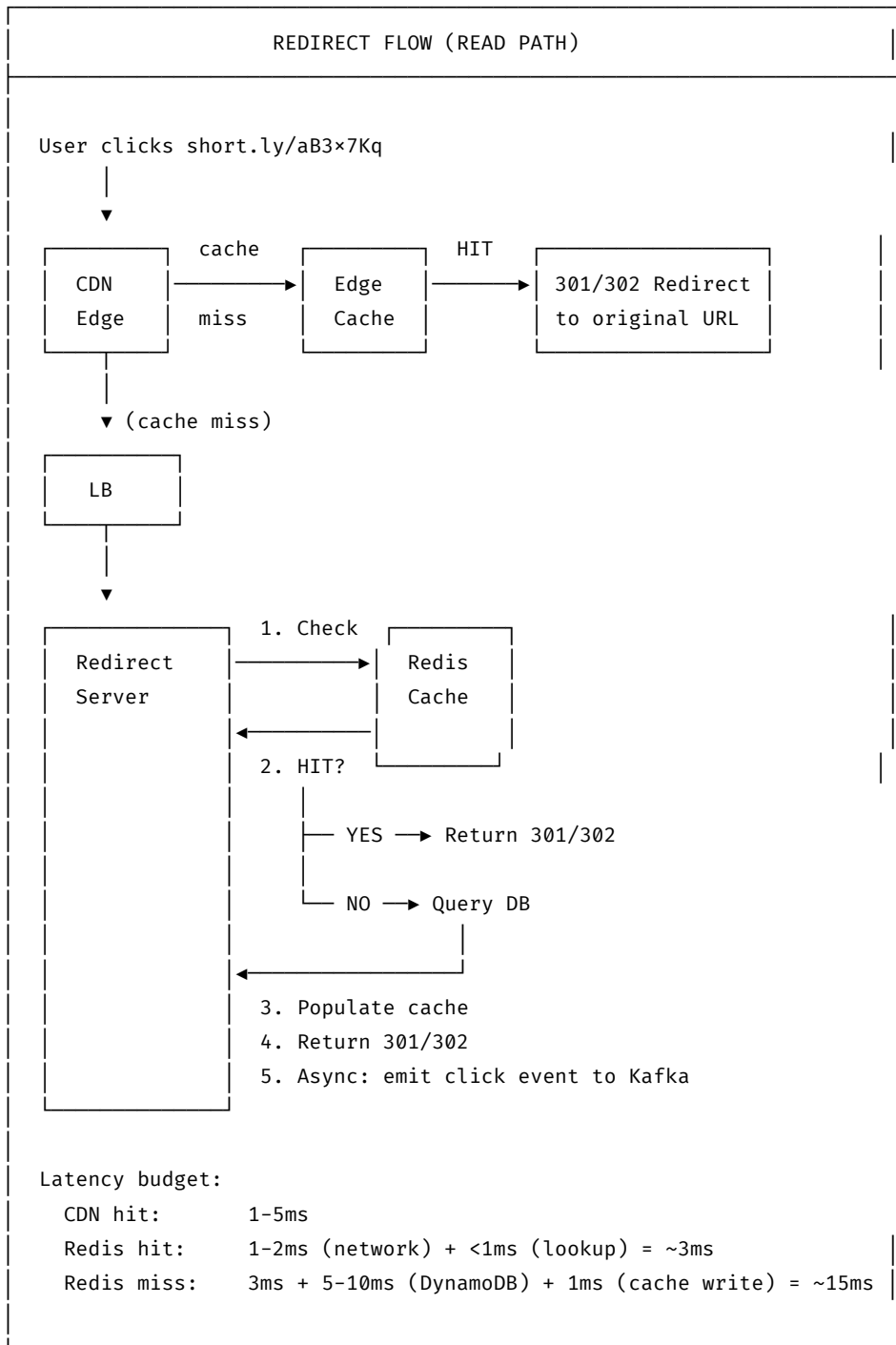
1.8.1 Complete System Architecture



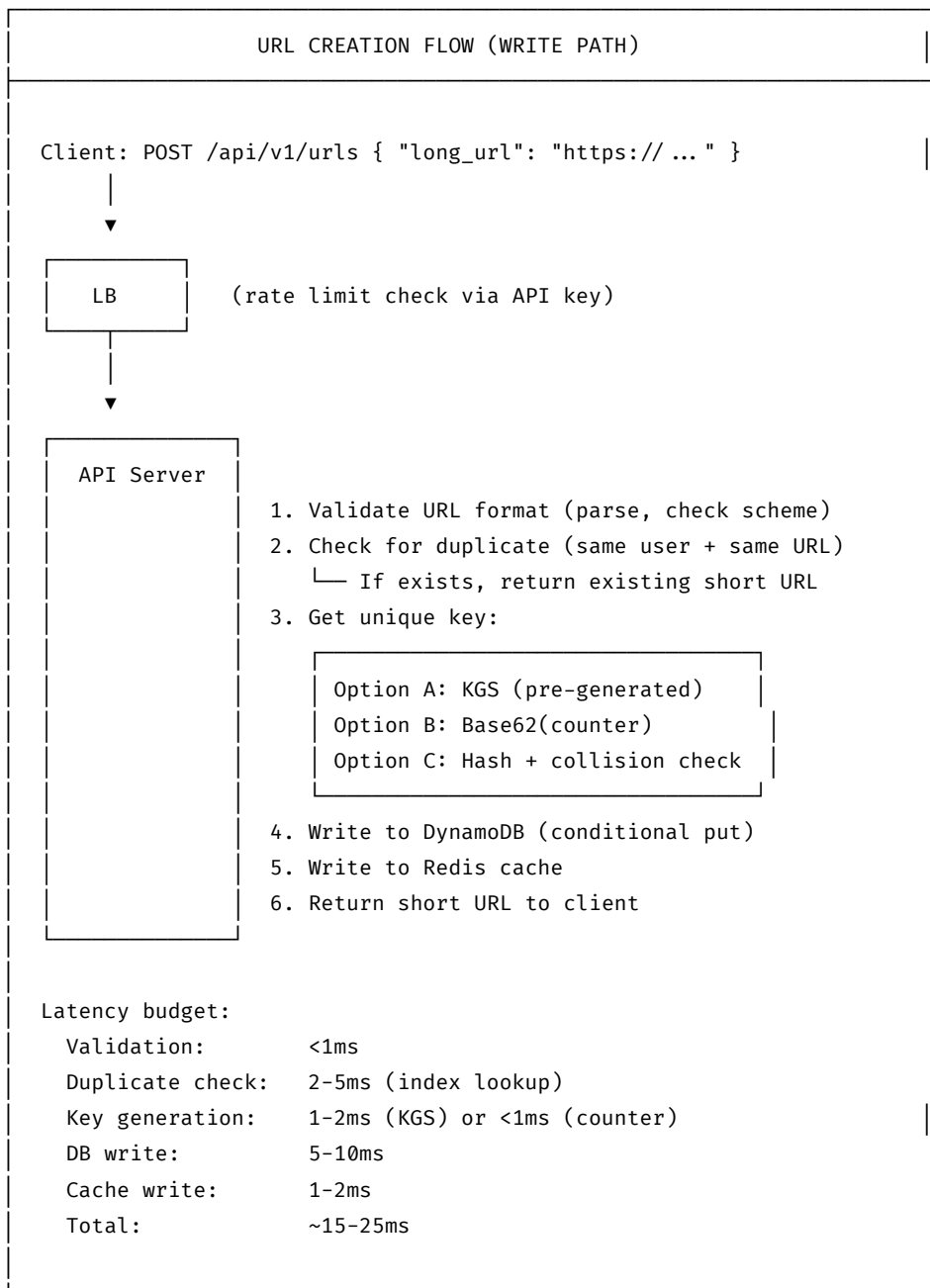


1.8.2 7.1 Component Breakdown

1.8.2.1 Read Path (Redirect) - Latency-Critical



1.8.2.2 Write Path (URL Creation)



1.9 8. Deep Dive: Core Components

1.9.1 8.1 URL Shortening Algorithm

The choice of shortening algorithm is the most critical design decision. Below are the four main approaches, with trade-offs.

1.9.1.1 Key Space Analysis

KEY SPACE ANALYSIS		
Character set: [a-z, A-Z, 0-9] = 62 characters (Base62)		
Key length vs. unique combinations:		
Length	Combinations	Sufficient for
5 chars	$62^5 = 916\text{M}$	~1 year at 1M/day
6 chars	$62^6 = 56.8\text{B}$	~155 years at 1M/day
7 chars	$62^7 = 3.5\text{T}$	~9600 years
8 chars	$62^8 = 218\text{T}$	~600K years
Recommendation: 7 characters		
- 3.5 trillion combinations is more than sufficient for 1.8B URLs		
- Utilization: $1.8\text{B} / 3.5\text{T} = 0.05\%$ (extremely sparse, low collision)		
- Short enough for readability and sharing		

1.9.1.2 Approach 1: Base62 Encoding of Auto-Increment Counter

BASE62 ENCODING OF COUNTER

Algorithm:

1. Maintain a global auto-incrementing counter.
2. Convert counter value to Base62 string.
3. Use the Base62 string as the short key.

Example:

Counter = 1000000

Base62(1000000) = "4c92"

Base62 encoding:

chars = "0123456789abcdef ... ABCDEF ... Z"

while n > 0:

 remainder = n % 62

 result = chars[remainder] + result

 n = n // 62

Pros:

- + Zero collisions (unique counter = unique key)
- + Simple and deterministic
- + Keys grow gradually (short keys first)

Cons:

- Sequential keys are predictable (security concern)
- Single point of failure (counter service)
- Distributed counter coordination is hard
- Competitors can estimate your volume

Mitigation for predictability:

Shuffle bits or XOR with a secret before Base62 encoding.

e.g., key = Base62(counter XOR 0x5DEECE66D)

1.9.1.3 Approach 2: MD5/SHA256 Hash + Truncation

HASH-BASED APPROACH

Algorithm:

1. Compute hash = MD5(long_url) or SHA256(long_url)
2. Take first 43 bits of the hash (sufficient for 7 Base62 chars)
3. Encode those bits as Base62 → 7-character key
4. Check for collision in DB:
 - If key exists with DIFFERENT URL → append salt and rehash
 - If key exists with SAME URL → return existing
 - If key doesn't exist → insert

Collision handling detail:

```

hash_input = long_url
while True:
    key = Base62(MD5(hash_input)[:7])
    if not exists_in_db(key):
        insert(key, long_url)
        break
    elif db_lookup(key).url == long_url:
        return key // same URL, reuse
    else:
        hash_input = hash_input + str(attempt_number) // rehash

```

Collision probability (Birthday Paradox):

```

With 1.8B URLs and 3.5T keyspace:
P(collision) ≈ n^2 / (2 × keyspace)
              ≈ (1.8×10^9)^2 / (2 × 3.5×10^12)
              ≈ 46% cumulative collision chance!

```

This is TOO HIGH. Need collision detection + resolution.

Pros:

- + No centralized counter needed
- + Same URL always maps to same hash (dedup-friendly)
- + Stateless computation

Cons:

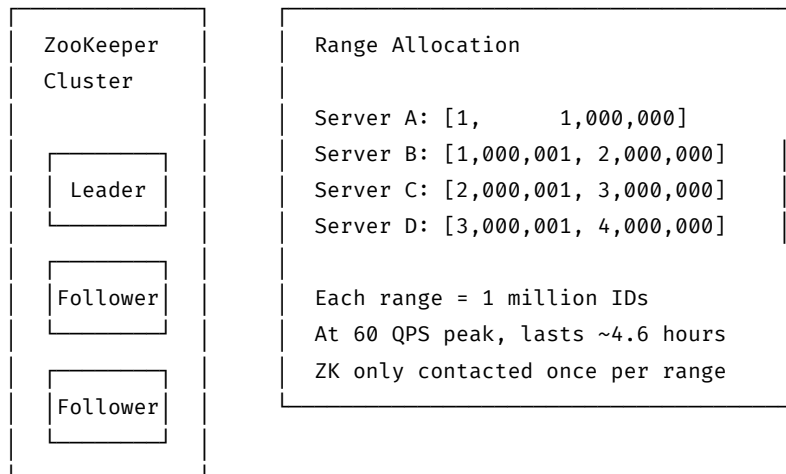
- Collision handling adds latency and complexity
- Requires DB read before write (check existence)
- Hash computation cost (minor but nonzero)

1.9.1.4 Approach 3: Counter with ZooKeeper Range Allocation

ZOOKEEPER RANGE-BASED COUNTER

Algorithm:

1. ZooKeeper allocates counter ranges to each app server.
2. Each server independently increments within its range.
3. When a range is exhausted, server requests a new range.



Pros:

- + No collisions (guaranteed unique ranges)
- + Minimal coordination (only on range exhaustion)
- + Each server works independently within its range
- + Horizontally scalable

Cons:

- Gaps in sequence if server crashes mid-range (acceptable)
- ZooKeeper dependency (operational complexity)
- Slightly predictable within a range

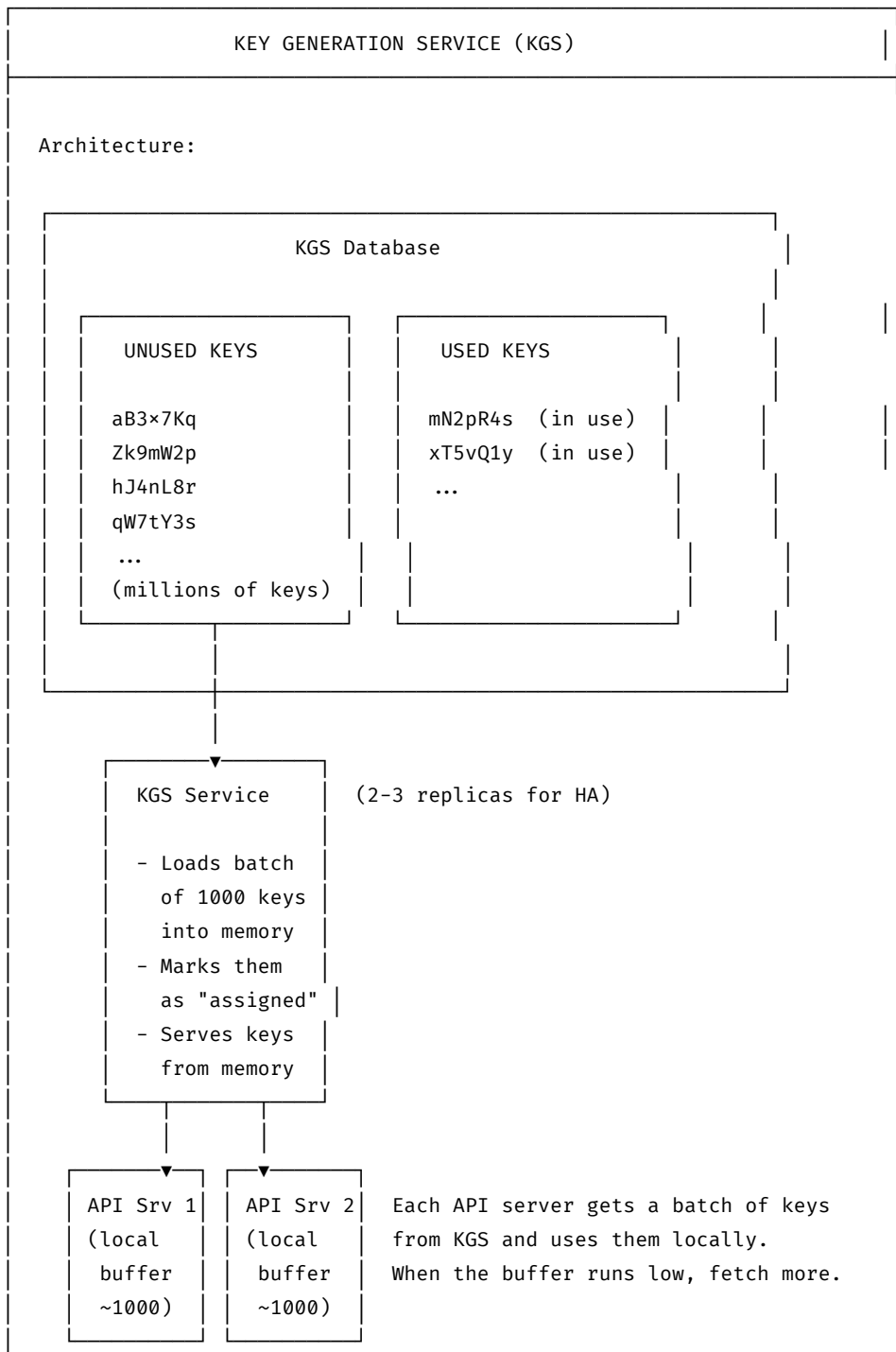
Real-world: Twitter Snowflake uses a similar concept for tweet IDs.

1.9.1.5 Approach 4: Snowflake ID

SNOWFLAKE-STYLE ID GENERATION			
64-bit ID structure:			
Timestamp (41 bit) ms since epoch	DC ID (5b) 32 DCs	Worker ID (5b) 32 workers	Sequence(12b) 4096/ms
Total: $41 + 5 + 5 + 12 = 63$ bits + 1 sign bit = 64 bits			
Capacity:			
- 32 data centers × 32 workers × 4096 IDs/ms			
- = 4,194,304 IDs per millisecond			
- = 4.19 billion IDs per second			
- Timestamp lasts: 2^{41} ms $\approx$ 69 years			
For URL shortener:			
Convert 64-bit Snowflake ID to Base62 → 11 characters			
Truncate to 7 characters → still unique enough			
(discard high-order timestamp bits that rarely change)			
Pros:			
+ Time-sortable (useful for analytics and debugging)			
+ No coordination needed (worker ID baked in)			
+ Battle-tested at Twitter scale			
+ k-sortable: rough chronological ordering			
Cons:			
- 11 Base62 chars is longer than 7 (can truncate with care)			
- Clock skew across servers can cause issues			
- Worker ID management adds operational overhead			

1.9.2 8.2 Key Generation Service (KGS)

The **Key Generation Service** is the recommended approach for production systems. It pre-generates unique keys offline and hands them out on demand, completely avoiding runtime collision checks.



Key generation (offline batch job):

- Generate random 7-char Base62 strings
- Check uniqueness against existing keys
- Insert into UNUSED_KEYS table
- Run periodically to maintain pool of ~10M unused keys

Failure handling:

- If API server crashes, its buffer of ~1000 keys is lost.
- 1000 keys out of 3.5T key space is negligible waste.
- KGS marks keys as "assigned" atomically (no double-issue).

Concurrency control:

- KGS uses SELECT ... FOR UPDATE or atomic batch operations
- Each key is moved from UNUSED → USED atomically
- Two KGS replicas never issue the same key

1.9.2.1 KGS Sizing

Storage for pre-generated keys:

Each key: 7 bytes

10 million keys: $7 \times 10M = 70$ MB

100 million keys: 700 MB

This easily fits in a single database table.

With an index on (status, key): ~1 GB total.

Throughput:

At 60 QPS peak write rate, 1 million pre-generated keys

last: $1,000,000 / 60 = \sim 16,666$ seconds ≈ 4.6 hours.

KGS can lazily generate in batches of 100K keys every few hours.

1.9.3 8.3 Redirection Service

1.9.3.1 301 vs 302: A Critical Decision

301 vs 302 REDIRECT COMPARISON		
	301 Moved Permanently	302 Found (Temporary Redirect)
Browser behavior	Caches redirect. Future visits go directly to dest.	Does NOT cache. Always hits our server first.
Server load	Lower (browser caches)	Higher (every click hits our servers)
Analytics accuracy	Under-counts! Repeat visits not tracked.	Accurate click counts. Every visit is tracked.
SEO impact	Passes link juice to destination	Does not pass link juice to destination
URL updates	Hard to change destination later	Easy to change destination anytime
RECOMMENDATION: Use 302 (temporary redirect) by default.		
Reasoning:		
1. Analytics is a core feature - we NEED every click to hit us.		
2. We want the ability to update/expire links dynamically.		
3. The additional server load is manageable with caching at our end.		
4. Offer 301 as an opt-in for users who prioritize SEO.		
Note: Bitly uses 301. TinyURL uses 301. Google's goo.gl used 301. This is because they prioritize reducing server load at massive scale. For a new service, start with 302 for better analytics.		

1.9.3.2 Redirect Response Headers

```
HTTP/1.1 302 Found
Location: https://www.example.com/original-very-long-url
Cache-Control: private, max-age=0, no-cache
X-Robots-Tag: noindex
Server: URLShortener/1.0
```

```
# For 301 (when user opts in):
HTTP/1.1 301 Moved Permanently
Location: https://www.example.com/original-very-long-url
Cache-Control: public, max-age=86400
```

1.9.3.3 Redirect Server Implementation (Pseudocode)

```
function handleRedirect(short_key):
    // 1. Check Redis cache
    long_url = redis.get("url:" + short_key)

    if long_url is null:
        // 2. Cache miss - check database
        record = db.get(short_key)

        if record is null:
            return HTTP 404 "Short URL not found"

        if record.expires_at < now():
            return HTTP 410 "This link has expired"

        if not record.is_active:
            return HTTP 410 "This link has been deleted"

        long_url = record.original_url

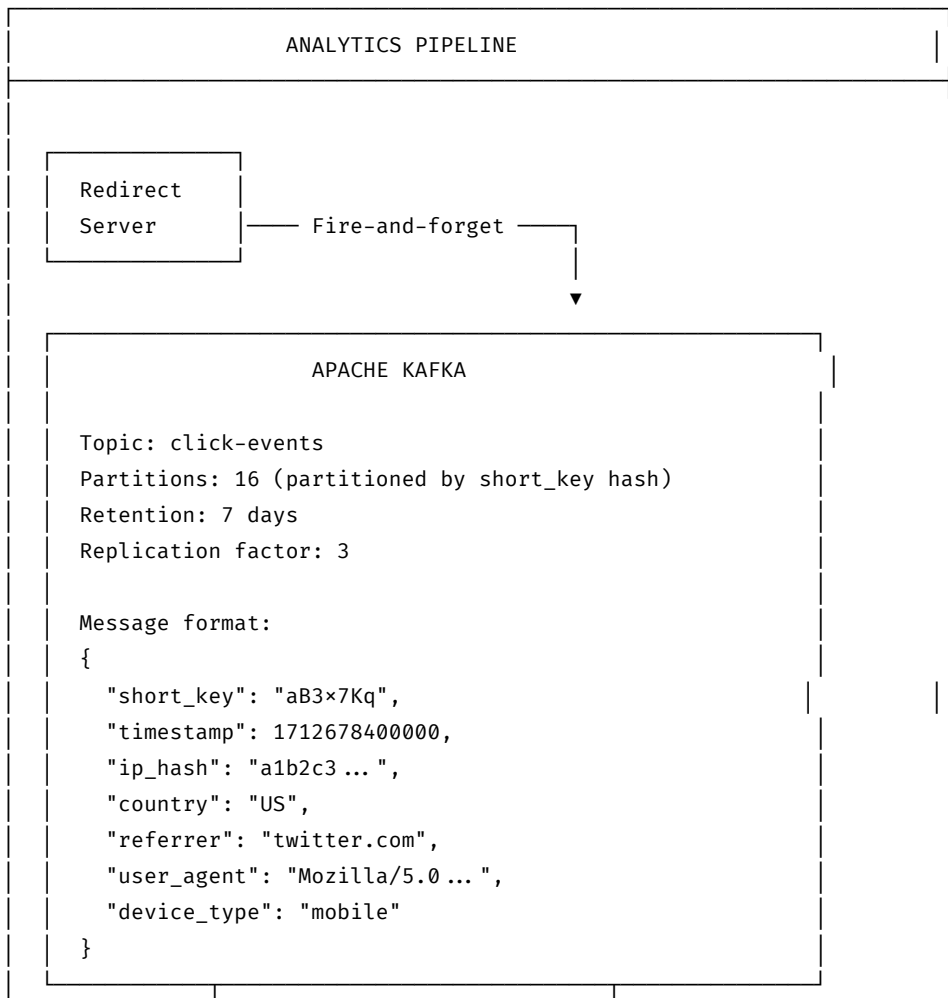
    // 3. Populate cache with TTL
    redis.setex("url:" + short_key, 3600, long_url)

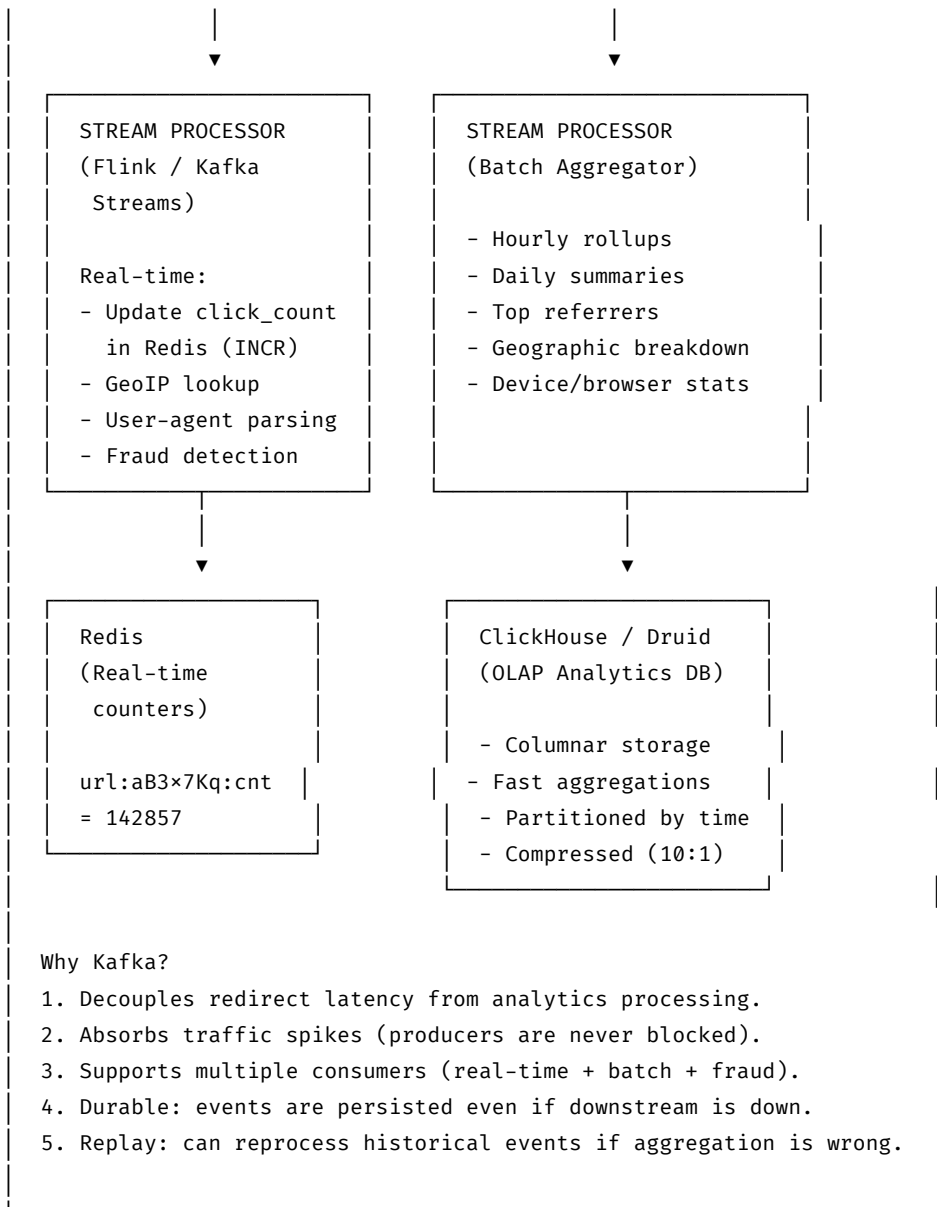
    // 4. Async: emit click event (non-blocking)
    kafka.produce("click-events", {
        short_key: short_key,
        timestamp: now(),
        ip: request.remote_addr,
        referrer: request.headers["Referer"],
        user_agent: request.headers["User-Agent"]
    })

    // 5. Return redirect
    return HTTP 302, Location: long_url
```

1.9.4 8.4 Analytics Pipeline

Click tracking must be **asynchronous** to avoid adding latency to the redirect path. Every millisecond of redirect latency is user-visible.





1.9.4.1 Click Count Consistency

The click_count field in the URL table is a denormalized counter for fast reads. It is updated asynchronously via the analytics pipeline, so it may lag behind the true count by a few seconds. This is acceptable for display purposes.

For exact counts, query the analytics OLAP store directly.

Redis counter update (real-time):

```

INCR url:aB3x7Kq:clicks      → total clicks
INCR url:aB3x7Kq:clicks:2026-04 → monthly clicks
PFADD url:aB3x7Kq:uniq <ip_hash> → HyperLogLog for unique visitors
  
```

Periodic flush to primary DB:

Every 5 minutes, batch-update click_count in DynamoDB from Redis counters.

1.10 9. Data Partitioning and Sharding

1.10.1 Sharding Strategy

SHARDING STRATEGIES COMPARED

Option 1: Hash-Based Partitioning on short_key (RECOMMENDED)

`shard_id = hash(short_key) % num_shards`

Shard 0	Shard 1	Shard 2	Shard 3
hash=0	hash=1	hash=2	hash=3
aB3 ...	mK7 ...	xP2 ...	qR9 ...
zW1 ...	hJ4 ...	cN6 ...	tY5 ...

Pros: Uniform distribution, simple routing

Cons: Adding shards requires data migration (mitigated by consistent hashing)

Option 2: Range-Based Partitioning on short_key

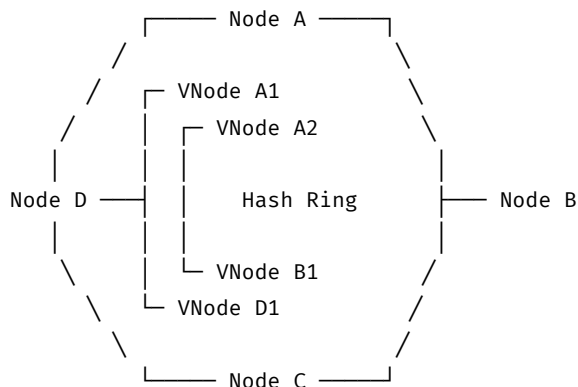
`shard_id` based on first character of short_key

Shard 0	Shard 1	Shard 2	Shard 3
a-f	g-n	o-v	w-Z,0-9

Pros: Range queries possible, simple to understand

Cons: Uneven distribution if keys aren't uniformly distributed
Hot spots if some ranges are more popular

Option 3: Consistent Hashing (RECOMMENDED FOR PRODUCTION)



Each physical node has 100-200 virtual nodes on the ring.
 Adding/removing a node only affects adjacent keys ($\sim 1/N$ data).

When Node E is added:

- Only keys between Node D and Node E migrate to Node E
- All other keys remain on their current nodes
- Data movement: $\sim 1/N$ of total data (not $1/2$ as in mod hashing)

1.10.2 DynamoDB Partition Key Design

If using DynamoDB:

- Partition key: short_key
- No sort key needed (each item is a single URL record)
- DynamoDB automatically distributes across partitions based on hash of partition key
- With random short_keys, distribution is naturally uniform

Throughput per partition: 3000 RCU + 1000 WCU

For 6000 read QPS: need ~ 2 partitions minimum (DynamoDB auto-scales)

DynamoDB handles sharding transparently - one of its key advantages.

1.10.3 Cross-Shard Operations

Operations that touch multiple shards are rare:

- Redirect lookup: single-key, single-shard ✓
- URL creation: single-key, single-shard ✓
- "List all URLs by user X": requires scatter-gather across shards
 - Use a secondary index (GSI in DynamoDB: user_id → short_keys)
 - Or maintain a separate user_urls table in PostgreSQL
- Analytics: handled by separate OLAP store, not the URL store

1.11 10. Caching Strategy

1.11.1 Multi-Layer Cache Architecture

MULTI-LAYER CACHING

Layer 1: CDN Edge Cache (CloudFront / Cloudflare)

- Caches 302 responses at edge locations worldwide
- Cache key: short_key
- TTL: 5 minutes (short, to preserve analytics accuracy)
- Covers: viral links clicked millions of times
- Hit ratio: 30-50% (only the most popular links)

Layer 2: Application-Level Cache (Local in-process)

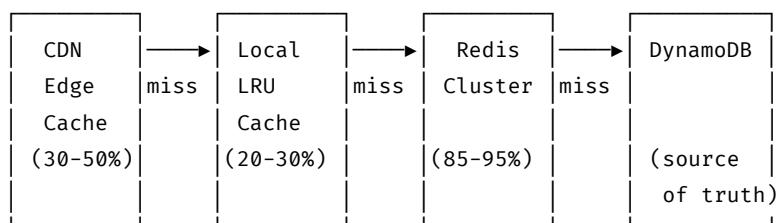
- LRU cache in each redirect server (e.g., Caffeine/Guava)
- Size: 100K entries (~50 MB per server)
- TTL: 60 seconds
- Avoids Redis network round-trip for ultra-hot URLs
- Hit ratio: 20-30%

Layer 3: Distributed Cache (Redis Cluster)

- Central cache shared across all redirect servers
- Size: 4M entries (~5 GB) covering the hot 20%
- TTL: 1 hour (refreshed on access)
- Pattern: Cache-aside (read-through with lazy population)
- Hit ratio: 85-95%
- Eviction: allkeys-lru

Layer 4: Database

- Only reached on triple cache miss (~1-5% of requests)
- DynamoDB with DAX (DynamoDB Accelerator) for additional caching



Effective DB hit rate after all cache layers:

~1-3% of total requests reach the database.

At 6000 QPS peak: only 60-180 QPS hit the DB. Easily handled.

1.11.2 Cache-Aside Pattern (Implementation)

CACHE-ASIDE PATTERN

READ (Redirect):

1. Look up key in Redis.
2. If HIT → return cached URL, done.
3. If MISS → query database.
4. If found in DB → write to Redis with TTL, return URL.
5. If not in DB → cache "not found" sentinel for 60s (prevents cache stampede on invalid URLs).

WRITE (Create URL):

1. Write to database first (source of truth).
2. Then write to Redis cache.
3. If cache write fails, it's OK - next read will populate it.

DELETE:

1. Delete from database.
2. Delete from Redis cache.
3. CDN cache will expire naturally (short TTL).

Cache Invalidation:

- URL updates (rare): delete from Redis, CDN purge API.
- Expiration: TTL-based eviction handles most cases.
- The mantra: "There are only two hard things in computer science: cache invalidation and naming things."

1.11.3 Cache Stampede Prevention

When a popular URL's cache entry expires, hundreds of simultaneous requests may all miss the cache and hit the database concurrently. This is a "thundering herd" or "cache stampede."

Mitigation strategies:

1. Probabilistic early expiration (jittered TTL):
 `actual_ttl = base_ttl + random(-60, +60) seconds`

2. Lock-based recomputation:
 Only one thread fetches from DB; others wait on the lock.
 Redis SETNX "lock:aB3×7Kq" with short expiry.

3. Background refresh:
 Refresh cache entries proactively before they expire.
 If TTL < 10% remaining, trigger async refresh.

4. Negative caching:
 Cache "not found" responses for 60 seconds to prevent repeated DB lookups for nonexistent keys (e.g., bots scanning random short URLs).
-

1.12 11. Replication and Consistency

1.12.1 Consistency Model

CONSISTENCY REQUIREMENTS		
URL Shortener consistency needs are RELAXED:		
Operation	Consistency	Rationale
Create URL	Strong (write)	Must not lose data. Use quorum write (W=2, N=3).
Read/Redirect	Eventual	1-2s propagation delay is acceptable. Read from any replica (R=1 for speed).
Delete URL	Eventual	Soft-delete; cache TTL handles propagation.
Analytics	Eventual	Aggregated async; exact real-time not required.

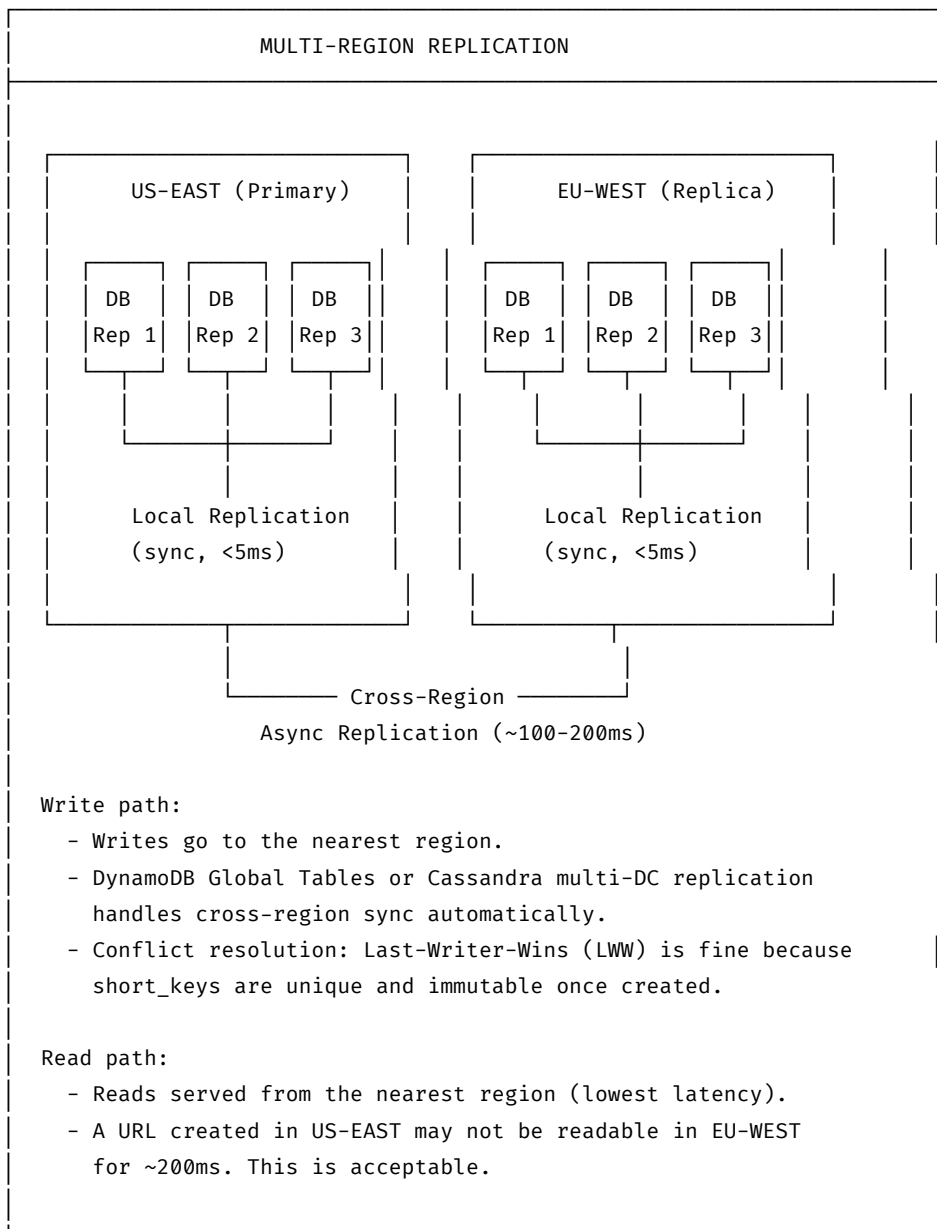
Quorum configuration (Cassandra/DynamoDB):

- N = 3 (replication factor)
- W = 2 (write to 2 of 3 replicas before acknowledging)
- R = 1 (read from any 1 replica for lowest latency)

$W + R > N? \rightarrow 2 + 1 = 3 > 3?$ NO, so NOT strongly consistent.
This is intentional: we trade consistency for read latency.

For strong consistency when needed: use R=2 (but adds latency).

1.12.2 Replication Architecture



1.12.3 Redis Replication

Redis cache is replicated independently per region:

- Each region has its own Redis cluster (not cross-region replicated)
- Cache misses in one region simply fall through to the local DB replica
- This avoids cross-region cache synchronization complexity
- Cache warm-up after deployment: pre-populate from DB scan of recent hot URLs

1.13 12. Fault Tolerance and Failure Handling

FAILURE SCENARIOS AND HANDLING	
Failure	Handling Strategy
App server crash	LB detects via health check (every 5s). Routes traffic to healthy servers. Pre-fetched keys from KGS are lost (OK, ~1000 keys out of trillions).
Redis cache down	Redirect servers fall through to DB directly. Latency increases from ~3ms to ~15ms. DB can handle the load temporarily. Circuit breaker: stop trying Redis after 3 consecutive failures; retry every 30s.
Primary DB down	Failover to read replica (automatic with DynamoDB/Cassandra multi-node setup). Writes may be briefly unavailable (seconds). Reads continue from replicas + cache.
KGS down	API servers use their local key buffer. Buffer holds ~1000 keys → lasts ~16 minutes at 60 QPS. Fallback: generate keys inline using hash-based approach until KGS recovers.
Kafka down	Click events are lost (acceptable trade-off). Analytics may have gaps but redirects work. Alternative: buffer events locally on disk and replay when Kafka recovers.
Region outage	DNS failover to secondary region. Route53 health checks detect within 30s. All data is replicated to secondary region. RTO: ~60 seconds. RPO: ~200ms of writes.
CDN outage	LB receives traffic directly. Higher latency but functional. DNS TTL (60s) determines switchover speed.

1.13.1 Rate Limiting

RATE LIMITING		
Algorithm: Token Bucket (implemented via Redis)		
Limits:		
Tier	URL Creation	Redirect (per IP)
Free	50/hour	1000/min per short URL
Pro	500/hour	10,000/min per short URL
Enterprise	10,000/hour	No limit
Implementation:		
Key: ratelimit:{api_key}:{window}		
Redis: INCR + EXPIRE (sliding window counter)		
Anti-abuse for redirects:		
- IP-based rate limiting on redirect endpoint - CAPTCHA challenge after threshold - Block known bot user-agents - Flag URLs with suspiciously high click rates for review		

1.13.2 Graceful Degradation Strategy

Under extreme load, shed non-critical work in this priority order:

1. KEEP: Redirects (core functionality - always available)
2. KEEP: URL creation (primary write path)
3. SHED: Real-time analytics (buffer in Kafka, process later)
4. SHED: Analytics API (return cached/stale data, HTTP 503 if needed)
5. SHED: Custom alias validation (fall back to system-generated keys)
6. SHED: Duplicate URL detection (allow duplicates temporarily)

Circuit breaker states:

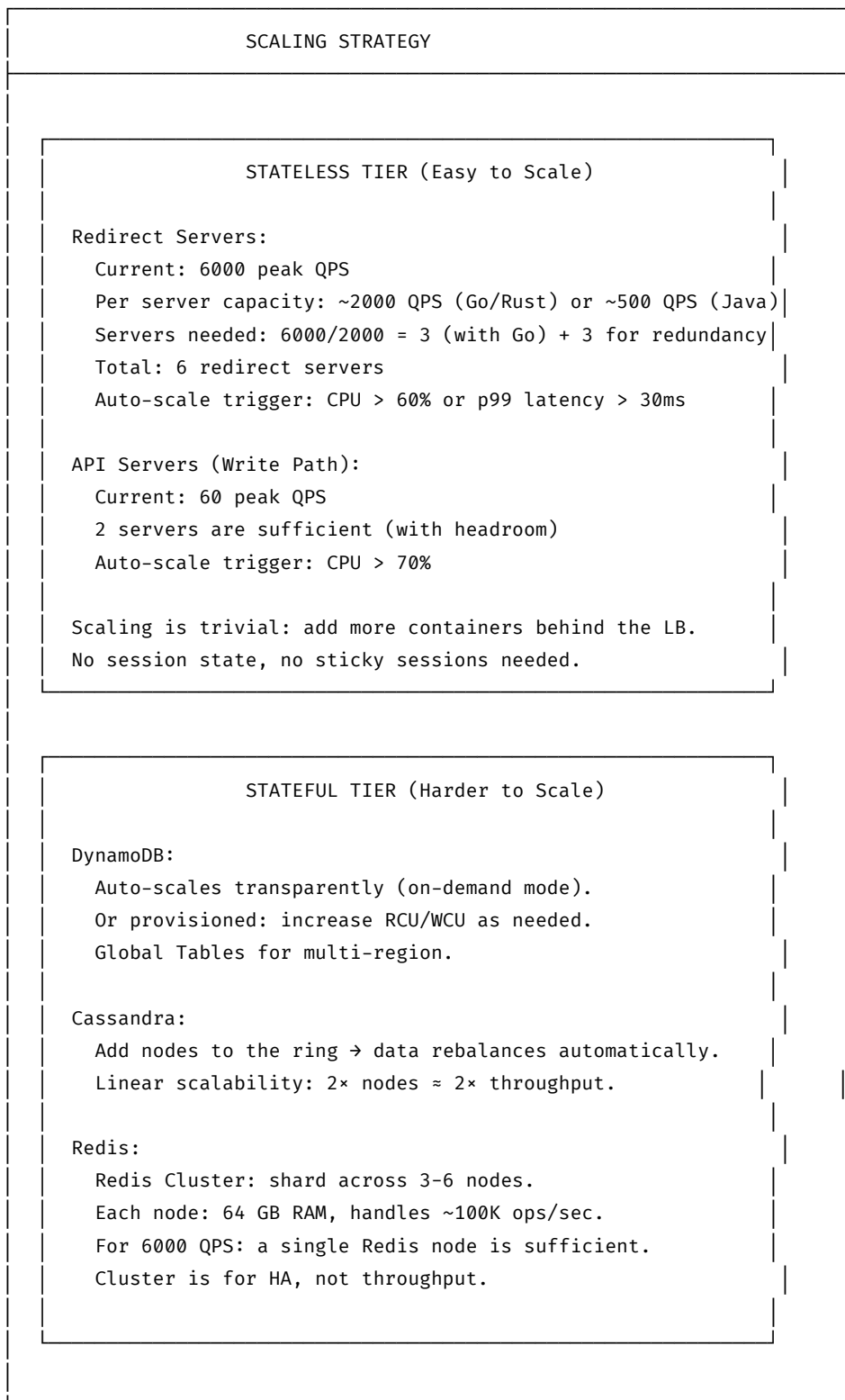
CLOSED → Normal operation

OPEN → Skip the failing dependency, use fallback

HALF-OPEN → Periodically test if dependency has recovered

1.14 13. Scalability

1.14.1 Horizontal Scaling of Stateless Components



1.14.2 Scaling to 10x and 100x

SCALING PROJECTIONS

Current design handles: ~6K read QPS, ~60 write QPS

10× scale (60K read QPS):

- Add more redirect servers (30 instances)
- Redis Cluster with 3 shards (20K QPS each)
- DynamoDB: increase RCU to 60K or use DAX
- CDN handles most of the increase for viral URLs
- Kafka: increase partitions to 64
- No architectural changes needed

100× scale (600K read QPS):

- Multi-region deployment (US, EU, APAC) - each region handles local traffic independently
- CDN becomes critical (cache hit ratio must be >80%)
- Redis Cluster: 10+ shards per region
- Consider edge computing (Cloudflare Workers / Lambda@Edge) to resolve redirects at the edge without hitting origin
- DynamoDB Global Tables or Cassandra multi-DC
- Separate read and write services completely
- Analytics pipeline: dedicated Kafka cluster, Flink cluster

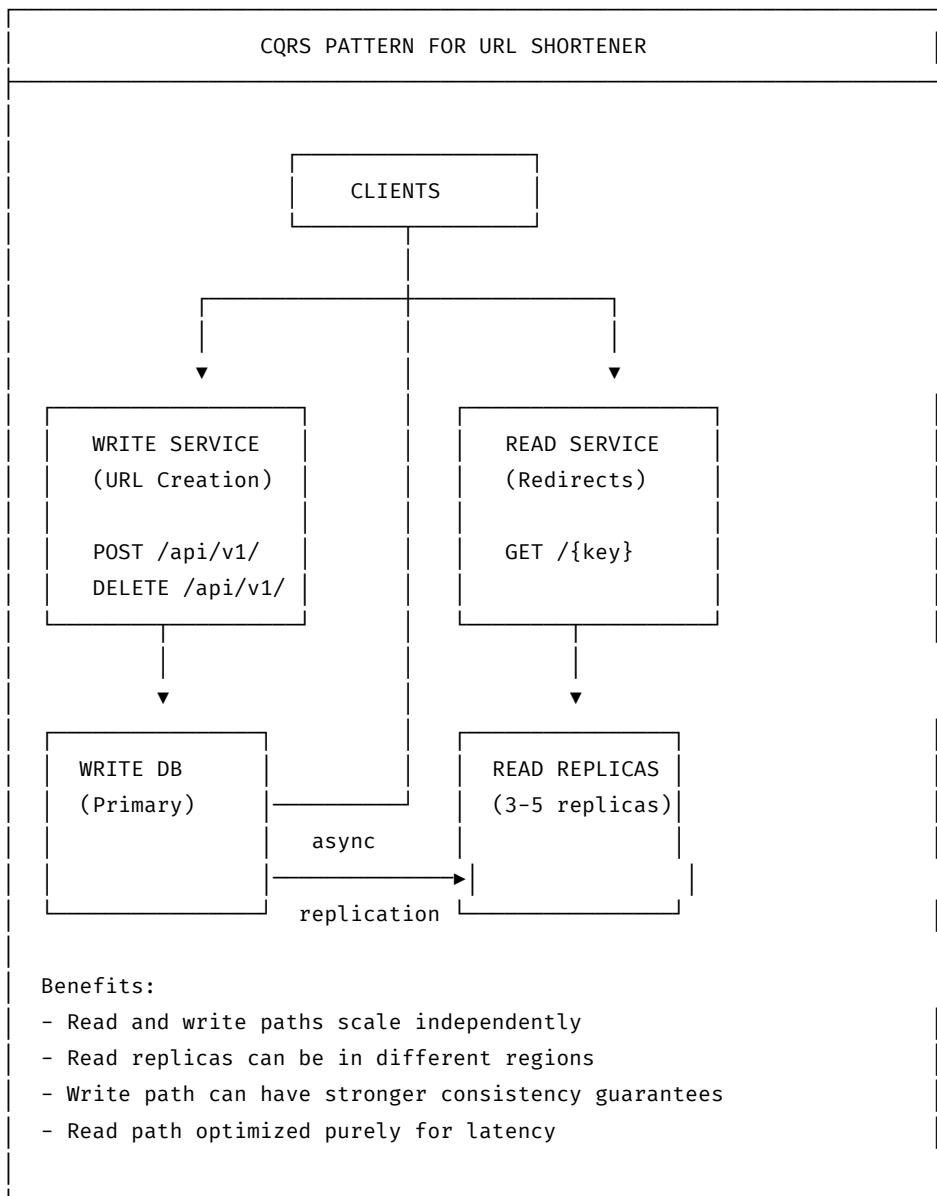
Bitly scale reference:

~600M clicks/month ≈ 230 QPS average

Our 6K QPS design is already ~26× Bitly's average.

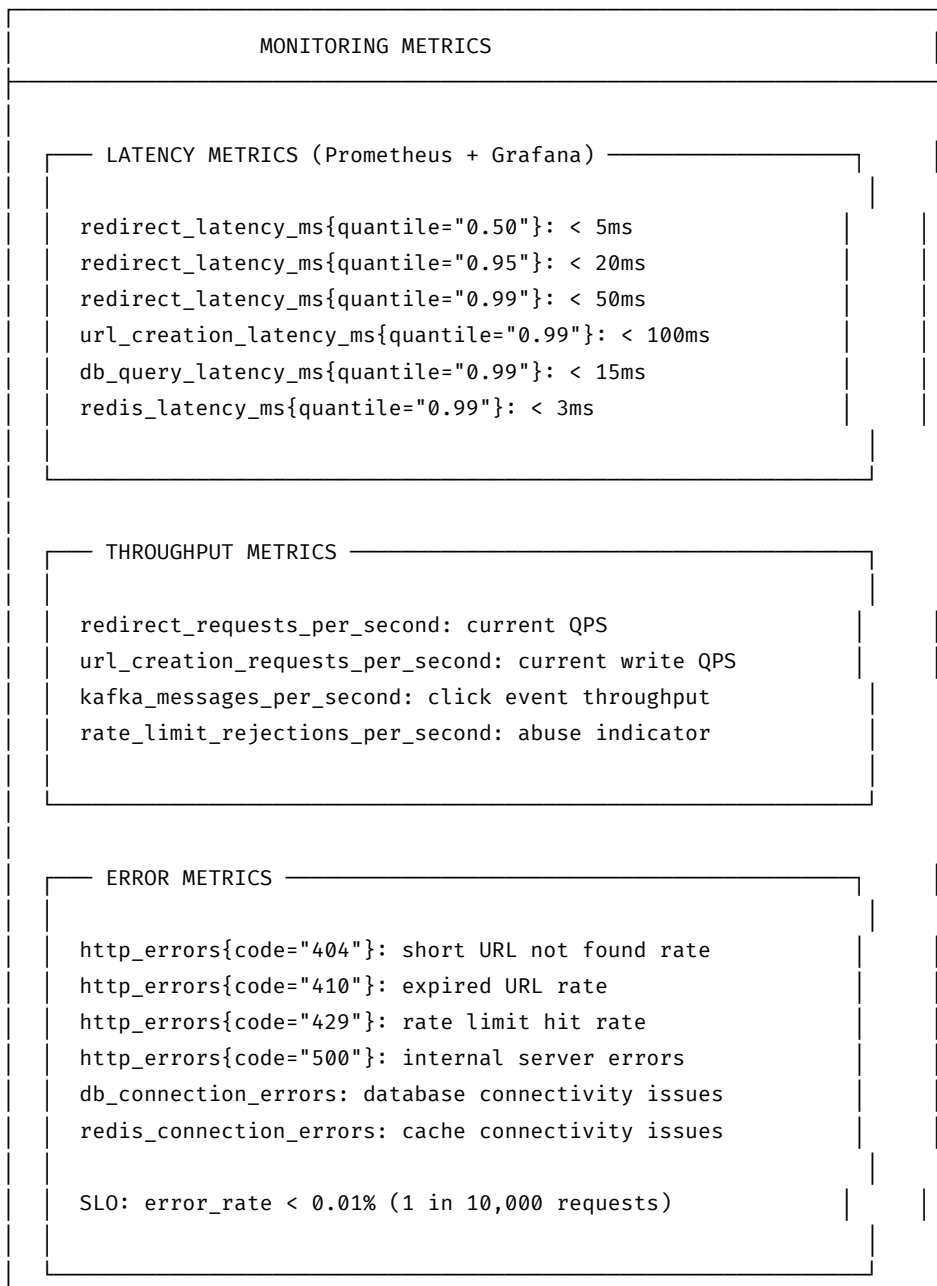
With 100× scaling, we handle ~2600× Bitly.

1.14.3 Read Replicas and CQRS



1.15 14. Monitoring and Observability

1.15.1 Key Metrics Dashboard



CACHE METRICS

```
redis_cache_hit_ratio: target > 90%
redis_cache_evictions_per_second: should be low
redis_memory_usage_bytes: vs. max memory
cdn_cache_hit_ratio: target > 40%
local_cache_hit_ratio: target > 20%
```

STORAGE METRICS

```
total_urls_count: total URLs in system
urls_created_today: daily creation rate
expired_urls_count: URLs past expiration
db_storage_bytes: disk usage growth trend
kgs_available_keys: must stay above threshold (>100K)
```

BUSINESS METRICS

```
daily_active_users: DAU trend
urls_per_user: engagement metric
clicks_per_url: viral coefficient
top_domains_shortened: content analysis
```

1.15.2 Alerting Rules

ALERTING THRESHOLDS		
Alert	Threshold	Action
Redirect p99 > 100ms	CRITICAL	Page on-call, check DB/cache
Error rate > 1%	CRITICAL	Page on-call immediately
Cache hit < 70%	WARNING	Investigate cache evictions
KGS keys < 10K	WARNING	Trigger key generation job
KGS keys < 1K	CRITICAL	Page on-call, fallback to hash
DB CPU > 80%	WARNING	Consider scaling/optimization
Kafka consumer lag > 100K messages	WARNING	Scale consumers, check proc.
Redis memory > 80%	WARNING	Review eviction policy/sizing
Disk usage > 80%	WARNING	Plan storage expansion

1.15.3 Distributed Tracing

Each request gets a trace ID that flows through all components:

```
Browser → CDN → LB → Redirect Server → Redis → DB → Kafka
      t=0   t=1ms t=2ms           t=3ms   t=8ms t=9ms
```

Tools: Jaeger / Zipkin / AWS X-Ray

Sampling rate: 1% for redirects (high volume), 100% for writes

1.16 15. Trade-offs and Design Decisions

1.16.1 Algorithm Comparison

URL SHORTENING ALGORITHM COMPARISON				
Criterion	Base62 Counter	Hash+Trunc (MD5/SHA)	ZK Counter Range	KGS (Pre-gen)
Collision	None	Possible (needs check)	None	None
Predictable	Yes (bad)	No	Within range (semi-bad)	No
Latency	~1ms	~2-5ms (with check)	~1ms	~1ms (from local buf)
Scalability	SPOF (counter)	Stateless (no coord)	Good (ZK needed)	Excellent (pre-computed)
Dedup (same URL)	No (new key)	Natural (same hash)	No (new key)	No (new key)
Complexity	Low	Medium	Medium	Medium
Dependencies	Counter DB	None	ZooKeeper	KGS DB + service
Recommended?	For small scale only	If dedup is critical	For large distributed systems	BEST for production

Winner: KGS (Key Generation Service)

- Zero runtime collisions
- O(1) key retrieval from local buffer
- Unpredictable keys (security)
- Decoupled from write path latency
- Trade-off: operational overhead of running KGS

1.16.2 Database Comparison

DATABASE COMPARISON			
Criterion	DynamoDB	Cassandra	PostgreSQL
Ops overhead	Fully managed	Self-managed	Managed (RDS) or self
Latency	<10ms single-digit ms at any scale	<10ms	<5ms (single node) Degrades with scale
Scalability	Infinite (auto)	Linear (add nodes)	Vertical + read replicas Sharding is painful
Multi-region	Global Tables (built-in)	Native multi-DC (battle-tested)	Logical replication (limited)
Cost	Pay-per-request or provisioned	Infra + ops team cost	Cheapest at low scale
Consistency	Eventual or strong (per-req)	Tunable (per-query)	Strong (ACID)
Query flexibility	Key-value only (GSI for others)	CQL (limited)	Full SQL, joins, etc.
Best for	AWS-native, variable traffic	Multi-cloud, write-heavy	Small-medium scale, complex queries

Recommendation:

- DynamoDB if you're on AWS (simplest ops, auto-scaling)
- Cassandra if multi-cloud or you need fine-grained control
- PostgreSQL ONLY if your scale is <1M URLs and you value simplicity

1.16.3 301 vs 302 Summary

REDIRECT STATUS CODE DECISION

Use 302 when:

- ✓ Analytics accuracy is critical (track every click)
- ✓ You need ability to change destination URL
- ✓ Link expiration is a feature
- ✓ You're building a new service and want maximum flexibility

Use 301 when:

- ✓ Minimizing server load is the top priority
- ✓ SEO pass-through matters to your users
- ✓ You're at extreme scale and every request counts
- ✓ Analytics can tolerate undercounting

Our choice: 302 by default, 301 as opt-in per URL

1.16.4 SQL vs NoSQL Decision Framework

"If your primary access pattern is key-value lookup and you need to scale beyond a single machine, use a purpose-built key-value or wide-column store."

URL shortener's primary access pattern:

INPUT: short_key (string)
OUTPUT: original_url (string)

This is the textbook use case for a key-value store.

No joins. No transactions across keys. No complex WHERE clauses.

SQL makes sense for:

- User management (ACID, complex queries)
- Billing and subscriptions
- Admin dashboards

Polyglot persistence: use the right DB for each use case.

1.17 16. Interview Deep-Dive Questions

1.17.1 Q1: How do you handle custom aliases?

Answer: Custom aliases follow the same write path but skip the key generation step. The user-provided alias is validated (alphanumeric, hyphens, 3-30 characters, no profanity filter hit) and then checked against the database with a conditional write (DynamoDB `ConditionExpression: attribute_not_exists(short_key)` or Cassandra `IF NOT EXISTS`). If the alias already exists, return HTTP 409 Conflict. Custom aliases are stored in the same table as auto-generated keys; a `custom_alias` boolean flag distinguishes them. To prevent custom aliases from colliding with future auto-generated keys, reserve a character pattern (e.g., auto-generated keys always start with a lowercase letter, custom aliases can contain hyphens which auto-generated keys never do). Alternatively, use separate key namespaces.

1.17.2 Q2: How do you handle URL expiration?

Answer: Each URL record has an optional `expires_at` timestamp. Expiration is checked in two places:

1. **At read time (lazy):** When a redirect is requested, check `expires_at < now()`. If expired, return HTTP 410 Gone and delete from cache.
 2. **Background cleanup (eager):** A daily cron job scans for expired URLs using the `idx_expiry` index (`SELECT short_key FROM urls WHERE expires_at < now() AND is_active = true`). It soft-deletes them (`is_active = false`), removes them from cache, and moves the `short_key` back to the KGS unused pool for recycling. In DynamoDB, use TTL feature which automatically deletes expired items.
-

1.17.3 Q3: What if two users submit the same long URL?

Answer: There are two valid approaches:

- **Per-user dedup:** Before generating a new key, query `(user_id, original_url)` index. If a match exists, return the existing short URL. Different users get different short URLs for the same long URL. This allows per-user analytics and independent expiration. This is the recommended approach.
 - **Global dedup:** All users share the same short URL for the same long URL. Saves storage but makes deletion and expiration complex (who “owns” it?). Analytics become ambiguous. TinyURL uses this approach for simplicity. Not recommended for a Bitly-style service.
-

1.17.4 Q4: How do you prevent abuse and spam?

Answer: Multiple layers of defense:

1. **Rate limiting:** Token bucket per API key (creation) and per IP (redirects). Implemented via Redis with sliding window counters.
 2. **URL validation:** Check submitted URLs against Google Safe Browsing API or a local blocklist of known malware/phishing domains. Reject URLs pointing to our own domain (redirect loops).
 3. **CAPTCHA:** Require CAPTCHA for anonymous URL creation (no API key).
 4. **Link scanning:** Async job that follows created URLs and checks destination content. Flag suspicious content for human review. Auto-disable links with confirmed malware.
 5. **Monitoring:** Alert on sudden spikes in URL creation from a single user or IP range.
-

6. **Cooldown:** After rate limit is hit, require exponential backoff before new requests.
-

1.17.5 Q5: How do you generate globally unique keys in a distributed system?

Answer: The KGS approach avoids distributed coordination entirely. Pre-generate millions of random keys offline and store them in a database. Each application server fetches a batch of keys (e.g., 1000) into a local in-memory buffer. Key assignment uses an atomic database operation (e.g., `DELETE FROM unused_keys LIMIT 1000 RETURNING short_key`). If two KGS instances run simultaneously, they get different batches because the operation is atomic. If an app server crashes and loses its buffer, those keys are simply never used – wasting 1000 out of 3.5 trillion keys is negligible. No ZooKeeper, no distributed locks, no coordination needed beyond a simple database.

1.17.6 Q6: How do you handle hot URLs (viral links)?

Answer: A single URL going viral (millions of clicks per hour) creates a hot key problem:

1. **CDN caching:** Cache the redirect at the edge. With a 5-minute CDN TTL, a viral link with 1M clicks/hour generates only ~12 origin requests per edge location per hour.
 2. **Local in-memory cache:** Each redirect server caches the top 100K URLs locally. A viral URL will be in every server's local cache after the first request.
 3. **Redis read replicas:** If using Redis Cluster, the hot key lands on one shard. Add read replicas for that shard, or use client-side replication where the client randomly picks from multiple replicas.
 4. **Request coalescing:** If multiple concurrent requests hit the same cache miss, only one request fetches from DB. Others wait on the result (singleflight pattern in Go).
-

1.17.7 Q7: How do you handle the read/write ratio imbalance?

Answer: The 100:1 read-to-write ratio is a defining characteristic. The architecture exploits this by:

- **Separating read and write services** (CQRS-lite): Redirect servers are scaled independently from API servers. Scale redirect servers 10-50x more than API servers.
 - **Aggressive caching:** With 90%+ cache hit ratio, only 10% of reads hit the database. The effective DB read QPS is 600, not 6000.
 - **Read replicas:** Database read replicas serve redirect queries; the primary handles writes.
 - **CDN:** Offloads the most popular URLs entirely from origin infrastructure.
-

1.17.8 Q8: What happens if the database is temporarily unavailable?

Answer: Defense in depth:

1. **Cache serves most reads:** With 90%+ hit ratio and 1-hour TTL, the cache can serve reads for up to an hour without the database. This covers most short outages.
 2. **Circuit breaker on DB:** After 3 consecutive DB failures, open the circuit. Return cached results if available, or HTTP 503 with `Retry-After` header.
 3. **Write path:** Queue write requests in a local buffer or SQS queue. Process them when DB recovers. Return HTTP 202 Accepted (the short URL will work once DB is back).
-

4. **Multi-AZ / multi-region:** DynamoDB multi-AZ or Cassandra multi-DC means a single AZ failure does not cause DB unavailability.
-

1.17.9 Q9: How do you ensure short URLs are not predictable?

Answer: Predictability is a security concern because attackers could enumerate valid short URLs and discover private/sensitive links. Solutions:

1. **KGS with random generation:** Keys are randomly generated Base62 strings, not sequential. Enumerating $62^7 = 3.5$ trillion possibilities is infeasible.
 2. **No sequential patterns:** Avoid auto-increment counters without obfuscation.
 3. **Rate limiting on redirect:** Limit redirect requests per IP to prevent scanning.
 4. **Private links:** For sensitive URLs, require authentication to access (token in query param or cookie). This is a premium feature.
 5. **Monitoring:** Detect and block IPs making rapid sequential requests to different short URLs.
-

1.17.10 Q10: How do you handle database schema migrations?

Answer: With a NoSQL store like DynamoDB or Cassandra:

- **Schema-less:** Adding new fields requires no migration. Just start writing the new field; old records will have it as null. Application code handles both old and new formats.
 - **Backward compatibility:** Always make additive changes. Never remove or rename fields in-place. Deprecate fields by adding new ones alongside.
 - **For PostgreSQL (user table):** Use online migration tools like `pg-osc` or `gh-ost` to apply schema changes without locking tables. Perform rolling migrations: add new column, backfill, deploy code that writes both old and new, deploy code that reads new, drop old.
-

1.17.11 Q11: How would you implement A/B testing with short URLs?

Answer: Enable a single short URL to redirect to different destinations based on rules:

URL record:

```
{
  "short_key": "promo2026",
  "destinations": [
    { "url": "https://example.com/landing-v1", "weight": 50 },
    { "url": "https://example.com/landing-v2", "weight": 50 }
  ],
  "split_type": "weighted_random" // or "cookie_based" for consistency
}
```

The redirect server picks a destination based on the weight distribution. For consistent user experience, hash the user's IP or cookie to deterministically assign them to a variant. Track which variant each click went to in the analytics pipeline for conversion analysis.

1.17.12 Q12: How do you handle the “thundering herd” problem when a popular URL’s cache expires?

Answer: When a cache entry for a viral URL expires, thousands of simultaneous requests may all miss the cache and hit the database concurrently.

Solutions (in order of sophistication): 1. **Jittered TTL:** Add random jitter to TTL so entries expire at slightly different times. $TTL = base_ttl + random(0, 120)$ seconds. 2. **Mutex/lock-based refresh:** First thread to detect a miss acquires a distributed lock (`SETNX lock:key 1 EX 5`), fetches from DB, and populates cache. Other threads wait briefly (50ms) and retry the cache. The `singleflight` pattern in Go is ideal. 3. **Proactive refresh:** A background process refreshes cache entries before they expire. If remaining TTL < 10%, trigger an async refresh. The stale value is served until the refresh completes. 4. **Two-tier TTL:** Soft TTL (1 hour) triggers background refresh. Hard TTL (2 hours) is when the entry is actually evicted. Between soft and hard, stale data is served while refresh happens async.

1.17.13 Q13: What are the security considerations for a URL shortener?

Answer:

1. **Open redirect vulnerability:** Short URLs can be used to disguise malicious destinations. Mitigation: scan URLs against malware/phishing databases, show preview pages.
 2. **Data exfiltration:** Attackers encode data in URL parameters to exfiltrate through the shortener (it bypasses DLP tools). Mitigation: log and monitor URL patterns.
 3. **Denial of service:** Mass-creation of URLs to exhaust key space or storage. Mitigation: rate limiting, API key requirements, CAPTCHA.
 4. **Information disclosure:** Short URLs in browser history, server logs, and Referer headers may leak sensitive information. Mitigation: offer “private” URLs that require authentication.
 5. **Clickjacking:** Short URL landing page (preview mode) must include `X-Frame-Options: DENY` header.
 6. **API key security:** Store API keys hashed (bcrypt/argon2), never in plaintext. Use HTTPS everywhere. Rotate keys periodically.
-

1.17.14 Q14: How would you migrate from one database to another (e.g., PostgreSQL to DynamoDB)?

Answer: A phased migration approach:

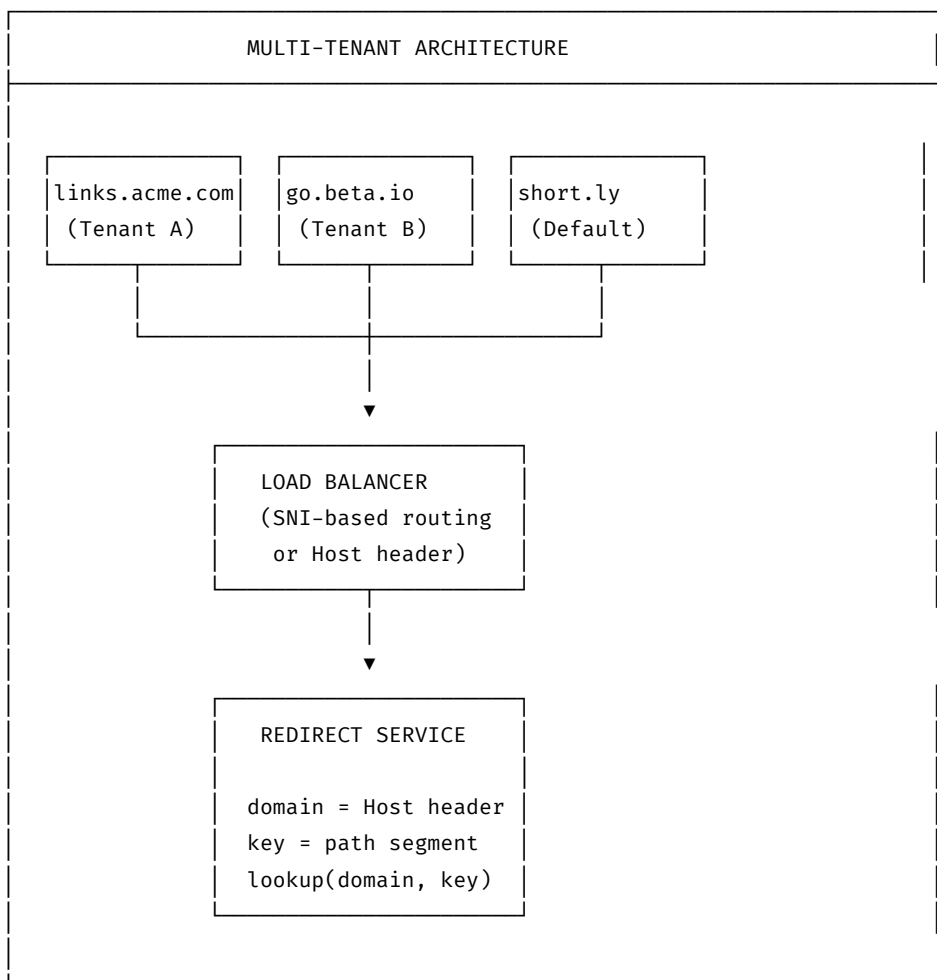
1. **Dual-write phase:** Modify the write path to write to both old (PostgreSQL) and new (DynamoDB) databases simultaneously. The old DB remains the source of truth.
2. **Backfill:** Run a one-time migration job that copies all existing records from PostgreSQL to DynamoDB. Handle conflicts (dual-write may have already copied some).
3. **Shadow-read phase:** Read from both databases and compare results. Log any discrepancies. Fix inconsistencies. This runs for days/weeks until confidence is high.
4. **Cutover reads:** Switch the read path to DynamoDB. PostgreSQL is still receiving writes as a safety net.
5. **Cutover writes:** Stop writing to PostgreSQL. DynamoDB is now the sole data store.
6. **Decommission:** After a soak period (e.g., 2 weeks with no issues), shut down PostgreSQL.

Use feature flags to control each phase. Rollback at any step is trivial: flip the flag back.

1.17.15 Q15: How would you design URL shortener for multi-tenancy (enterprise white-label)?

Answer: Enterprise customers want their own custom domains (e.g., `links.company.com`) with isolated analytics:

1. **Custom domain mapping:** Add a `domain` field to the URL table. The redirect service checks the `Host` header and looks up `(domain, short_key)` as the composite key. Same `short_key` can exist under different domains without collision.
2. **DNS setup:** Customer creates a CNAME record: `links.company.com` → `shortener.ourservice.com`. Our load balancer accepts traffic for any configured custom domain. TLS certificates are provisioned via Let's Encrypt with DNS-01 challenge or AWS Certificate Manager.
3. **Tenant isolation:**
 - **Logical isolation:** Same database, partitioned by `tenant_id`. DynamoDB GSI on `(tenant_id, short_key)`. Cheaper but “noisy neighbor” risk.
 - **Physical isolation (enterprise):** Dedicated Redis instance and database partition per tenant. Higher cost but guaranteed SLAs.
4. **Analytics isolation:** Each tenant sees only their own click data. Analytics queries are always filtered by `tenant_id`. Use row-level security in the OLAP store.
5. **Rate limiting per tenant:** Each tenant has their own rate limit pool, not shared with others.



1.18 Summary: System at a Glance

SYSTEM SUMMARY	
Scale:	1.8B URLs over 5 years, 6K peak read QPS
Storage:	~1 TB URLs + ~10 TB analytics
Key algo:	KGS (pre-generated random Base62, 7 chars)
Database:	DynamoDB (URLs) + PostgreSQL (users) + ClickHouse (analytics)
Cache:	Redis Cluster (5 GB, 90%+ hit ratio)
Queue:	Kafka (click events, 16 partitions)
Redirect:	302 by default (analytics-first)
Consistency:	Eventual for reads, quorum for writes
Availability:	99.99% (multi-AZ, multi-region)
Latency:	p50 < 5ms, p99 < 50ms
Key trade-offs made:	
1. Availability over consistency (AP in CAP)	
2. 302 over 301 (analytics accuracy over server load)	
3. KGS over hash (no collisions, slight operational overhead)	
4. NoSQL over SQL for URL store (scale over query flexibility)	
5. Async analytics over sync (latency over real-time accuracy)	

Chapter 2

Design a Distributed Rate Limiter

Difficulty: Medium-Hard | **Companies:** Google, Amazon, Cloudflare, Stripe, Netflix

For the low-level class design, see [LLD: Rate Limiter](#)

2.1 1. Problem Statement and Scope

2.1.1 What is Rate Limiting?

Rate limiting is a technique to control the rate of requests a client can make to a server within a given time window. It protects services from abuse, prevents resource starvation, manages costs, and ensures fair usage across tenants.

2.1.2 Problem

Design a **distributed rate limiting system** that can:

- Throttle API requests across hundreds of application servers
- Support multiple rate limiting algorithms (token bucket, sliding window, etc.)
- Handle multi-tenant workloads with per-tenant, per-endpoint, and per-user limits
- Process millions of requests per second with sub-millisecond overhead
- Remain consistent (or gracefully degrade) during partial failures

2.1.3 Scope

IN SCOPE
- API request rate limiting (HTTP, gRPC) - Multiple algorithms (token bucket, sliding window, etc.) - Multi-tenant support with per-tenant configuration - Distributed counters across server fleet - Rate limit rule management (CRUD) - Rate limit response headers (X-RateLimit-*) - Allow/deny lists for bypass or hard block - Monitoring and observability dashboard
OUT OF SCOPE
- DDoS protection (layer 3/4 network-level filtering) - Request queuing / throttling with retries (that's a queue) - Cost-based billing or metering - Authentication / authorization (assumed handled upstream)

2.1.4 Real-World Examples

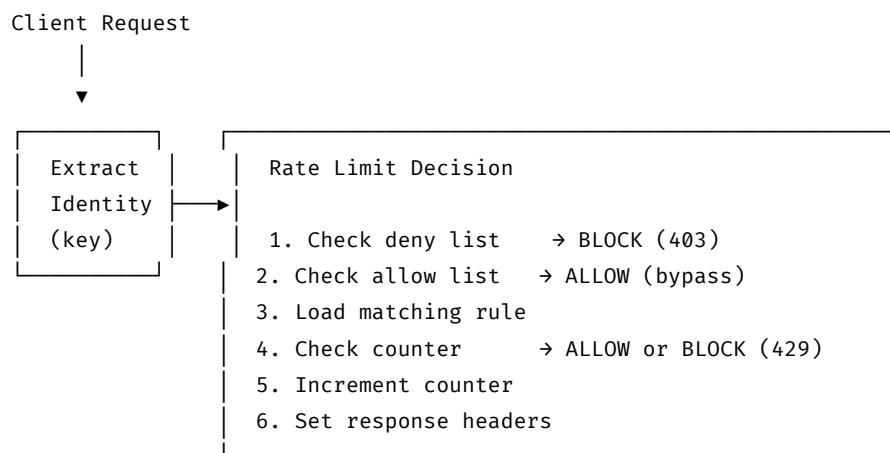
- **Cloudflare:** Use Case: Edge rate limiting for 25M+ internet properties; Scale: Billions of req/sec
- **Stripe:** Use Case: API rate limiting per merchant; Scale: Millions of API calls/day
- **Google:** Use Case: Cloud API quotas per project; Scale: Millions of projects
- **Netflix:** Use Case: Zuul gateway rate limiting; Scale: 100B+ requests/day
- **Amazon:** Use Case: API Gateway throttling per API key; Scale: Millions of APIs

2.2 2. Functional Requirements

2.2.1 Core Requirements

- **FR1 - Rate limit by identity:** Limit by user ID, API key, IP address, or any combination
- **FR2 - Multiple time windows:** Per-second, per-minute, per-hour, per-day limits simultaneously
- **FR3 - Per-endpoint configuration:** Different limits for `/api/search` (100/min) vs `/api/upload` (10/min)
- **FR4 - Rate limit headers:** Return `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset` in every response
- **FR5 - Allow/deny lists:** Bypass rate limiting for trusted partners; hard-block known abusers
- **FR6 - Multi-tenant support:** Each tenant (SaaS customer) has independent rate limit configurations
- **FR7 - Rule management:** CRUD operations on rate limit rules with hot-reload (no restart)
- **FR8 - Usage dashboard:** Real-time view of rate limit counters, hit rates, and rejected requests
- **FR9 - Graceful rejection:** Return HTTP 429 with `Retry-After` header when limit exceeded
- **FR10 - Multiple algorithms:** Support token bucket, sliding window counter, fixed window, leaky bucket

2.2.2 Request Flow



2.3 3. Non-Functional Requirements

- **Latency overhead:** < 1ms p99; Rationale: Rate limiter sits in the hot path of every request
- **Throughput:** 10M+ decisions/sec; Rationale: Aggregated across all servers in the fleet
- **Availability:** 99.99% (52 min/year); Rationale: Downtime means either all traffic blocked or all allowed
- **Consistency:** Near-exact (< 1% drift); Rationale: Slight over-counting acceptable; under-counting is not
- **Scalability:** Horizontal; Rationale: Add more Redis nodes / servers linearly
- **Fault tolerance:** Configurable fail mode; Rationale: Fail-open (allow) or fail-closed (deny) per rule
- **Durability:** Rules durable, counters ephemeral; Rationale: Counters naturally expire; rules must survive restarts

2.3.1 Fail-Open vs Fail-Closed

FAIL-OPEN (Default)
Redis down → allow all requests → risk of overload Use for: non-critical APIs, internal services Rationale: availability > protection
FAIL-CLOSED
Redis down → deny all requests → service appears down Use for: payment APIs, security-sensitive endpoints Rationale: protection > availability
FAIL-LOCAL (Hybrid)
Redis down → fall back to local in-memory counters Use for: balanced approach, most production systems Rationale: approximate limiting > no limiting

2.4 4. Back-of-Envelope Estimation

2.4.1 Traffic Assumptions

Tenants: 1,000 (SaaS customers)
API endpoints/tenant: 50
Users/tenant: 100,000
Total users: 100M
Peak requests/sec: 10M (across all tenants)
Avg request size: ~1 KB

2.4.2 Counter Storage

Unique rate limit keys:

- = users * endpoints * time_windows
- = 100M * 1 (most active endpoint) * 3 (sec/min/hour windows)
- = 300M keys (worst case)
- = ~100M keys (realistic active set at any time)

Per-key storage:

Key: ~80 bytes (e.g., "tenant42:api/search:min:user12345678")
Value: ~16 bytes (counter + TTL metadata)
Total: ~100 bytes/key

Total memory:

- = 100M keys * 100 bytes
- = 10 GB

This fits comfortably in a Redis cluster with 3-5 nodes
(each node: 4-8 GB, with replication overhead)

2.4.3 Redis Operations

Requests/sec: 10M
Redis ops/request: 2 (GET counter + INCR counter, or 1 Lua script)
Total Redis ops: 10-20M ops/sec

Single Redis node: ~100K-300K ops/sec (depending on operation)
Nodes needed: 10M / 200K = 50 shards (conservative)

With Redis Cluster: 50 primary + 50 replica = 100 nodes
Or with Lua scripts: 10M / 200K = 50 shards

2.4.4 Network Bandwidth

Rate limit check payload: ~200 bytes/request
10M req/sec * 200 bytes = 2 GB/sec between app servers and Redis
Distributed across 50 shards = ~40 MB/sec per shard (manageable)

2.4.5 Rule Storage

Rules: $1,000 \text{ tenants} * 50 \text{ endpoints} * 3 \text{ windows} = 150,000 \text{ rules}$

Size: $150K * 500 \text{ bytes} = 75 \text{ MB}$

Fits easily in any relational DB + local cache

2.5 5. API Design

2.5.1 5.1 Rate Limit Check (Internal Middleware API)

This is the hot-path API called on every incoming request. It is not exposed externally but invoked by the API gateway or service mesh sidecar.

`checkRateLimit(request: RateLimitRequest) -> RateLimitResponse`

```
RateLimitRequest {
    client_id:  string    // user ID, API key, or IP
    tenant_id:  string    // SaaS customer identifier
    endpoint:   string    // e.g., "POST /api/orders"
    http_method: string    // GET, POST, PUT, DELETE
    timestamp:  int64     // Unix timestamp in millis
    cost:       int       // weight of this request (default 1)
}
```

```
RateLimitResponse {
    allowed:    bool      // true = proceed, false = reject
    limit:      int       // max requests in window
    remaining:  int       // requests remaining
    reset_at:   int64     // Unix timestamp when window resets
    retry_after: int      // seconds to wait (if denied)
    rule_id:    string    // which rule triggered the decision
}
```

2.5.2 5.2 Admin API (External, for Rule Management)

```
# Create or update a rate limit rule
PUT /v1/rules/{rule_id}
{
  "tenant_id":      "tenant-42",
  "endpoint_pattern": "POST /api/orders",
  "identifier_type": "API_KEY",          // USER | API_KEY | IP | GLOBAL
  "algorithm":      "TOKEN_BUCKET",     // TOKEN_BUCKET | SLIDING_WINDOW | FIXED_WINDOW | LEAKY_BUCKET
  "limit":          1000,
  "window_seconds": 60,
  "burst_capacity": 50,                  // for token bucket
  "fail_mode":      "OPEN",             // OPEN | CLOSED | LOCAL
  "priority":       10,
  "enabled":        true
}
Response: 200 OK { "rule_id": "rule-123", "version": 5 }

# Get a specific rule
GET /v1/rules/{rule_id}
Response: 200 OK { ... rule object ... }

# List rules for a tenant
GET /v1/rules?tenant_id=tenant-42&page=1&size=50
Response: 200 OK { "rules": [ ... ], "total": 150 }

# Delete a rule
DELETE /v1/rules/{rule_id}
Response: 204 No Content

# Get usage statistics for a client
GET /v1/usage/{client_id}?tenant_id=tenant-42&endpoint=POST+/api/orders&window=60
Response: 200 OK {
  "client_id": "user-789",
  "endpoint":  "POST /api/orders",
  "limit":     1000,
  "used":      423,
  "remaining": 577,
  "reset_at":  1680000060,
  "window":    60
}
```

2.5.3 5.3 Rate Limit Response Headers

Every API response includes these headers, regardless of whether the request was allowed:

HTTP/1.1 200 OK	HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 1000	X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 577	X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1680000060	X-RateLimit-Reset: 1680000060
	Retry-After: 37

2.6 6. Data Model and Database Selection

2.6.1 6.1 Database Selection Rationale

Data Type	Storage	Rationale
Rate limit rules	PostgreSQL	ACID, relational, durable
Rate limit counters	Redis Cluster	In-memory, atomic, TTL
Allow/deny lists	Redis + Postgres	Redis for fast lookup, Postgres for persistence
Usage analytics	ClickHouse/Kafka	Time-series, high write
Audit log (rule changes)	PostgreSQL	Append-only, queryable

2.6.2 6.2 Rules Schema (PostgreSQL)

```
CREATE TABLE rate_limit_rules (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   VARCHAR(64) NOT NULL,
  endpoint_pattern VARCHAR(256) NOT NULL,      -- e.g., "POST /api/orders" or "/api/*"
  identifier_type VARCHAR(32) NOT NULL,        -- USER, API_KEY, IP, GLOBAL
  algorithm    VARCHAR(32) NOT NULL,          -- TOKEN_BUCKET, SLIDING_WINDOW, etc.
  max_requests INT NOT NULL,
  window_seconds INT NOT NULL,
  burst_capacity INT DEFAULT 0,               -- for token bucket
  fail_mode    VARCHAR(16) DEFAULT 'OPEN',
  priority     INT DEFAULT 0,                 -- higher = checked first
  enabled      BOOLEAN DEFAULT TRUE,
  version      INT DEFAULT 1,
  created_at   TIMESTAMP DEFAULT NOW(),
  updated_at   TIMESTAMP DEFAULT NOW(),

  UNIQUE (tenant_id, endpoint_pattern, identifier_type)
);

CREATE INDEX idx_rules_tenant ON rate_limit_rules(tenant_id, enabled);
CREATE INDEX idx_rules_lookup ON rate_limit_rules(tenant_id, endpoint_pattern, enabled);
```

2.6.3 6.3 Allow/Deny List Schema

```
CREATE TABLE access_lists (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   VARCHAR(64) NOT NULL,
  identifier   VARCHAR(256) NOT NULL,          -- IP, user ID, API key
  list_type    VARCHAR(8) NOT NULL,            -- ALLOW or DENY
  reason       TEXT,
  expires_at   TIMESTAMP,                     -- NULL = permanent
  created_at   TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_access_list ON access_lists(tenant_id, identifier, list_type);
```

2.6.4 6.4 Redis Counter Key Design

Key format:

```
rl:{tenant_id}:{endpoint_hash}:{window}:{identifier}
```

Examples:

```
rl:tenant42:a1b2c3:60:user-789      # 60-second window for user
rl:tenant42:a1b2c3:3600:user-789    # 1-hour window for user
rl:tenant42:a1b2c3:60:ip-192.168.1.1 # 60-second window for IP
```

Token bucket keys:

```
tb:{tenant_id}:{endpoint_hash}:{identifier}:tokens # current token count
tb:{tenant_id}:{endpoint_hash}:{identifier}:ts      # last refill timestamp
```

Sliding window log keys:

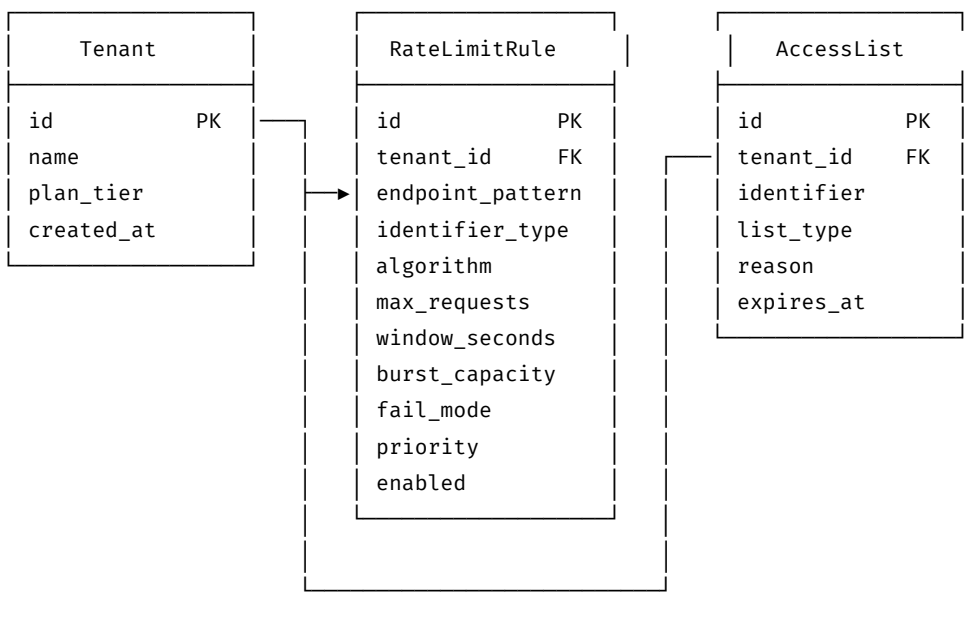
```
swl:{tenant_id}:{endpoint_hash}:{identifier}      # sorted set of timestamps
```

TTL:

Each key gets TTL = window_seconds + buffer (e.g., window + 10s)

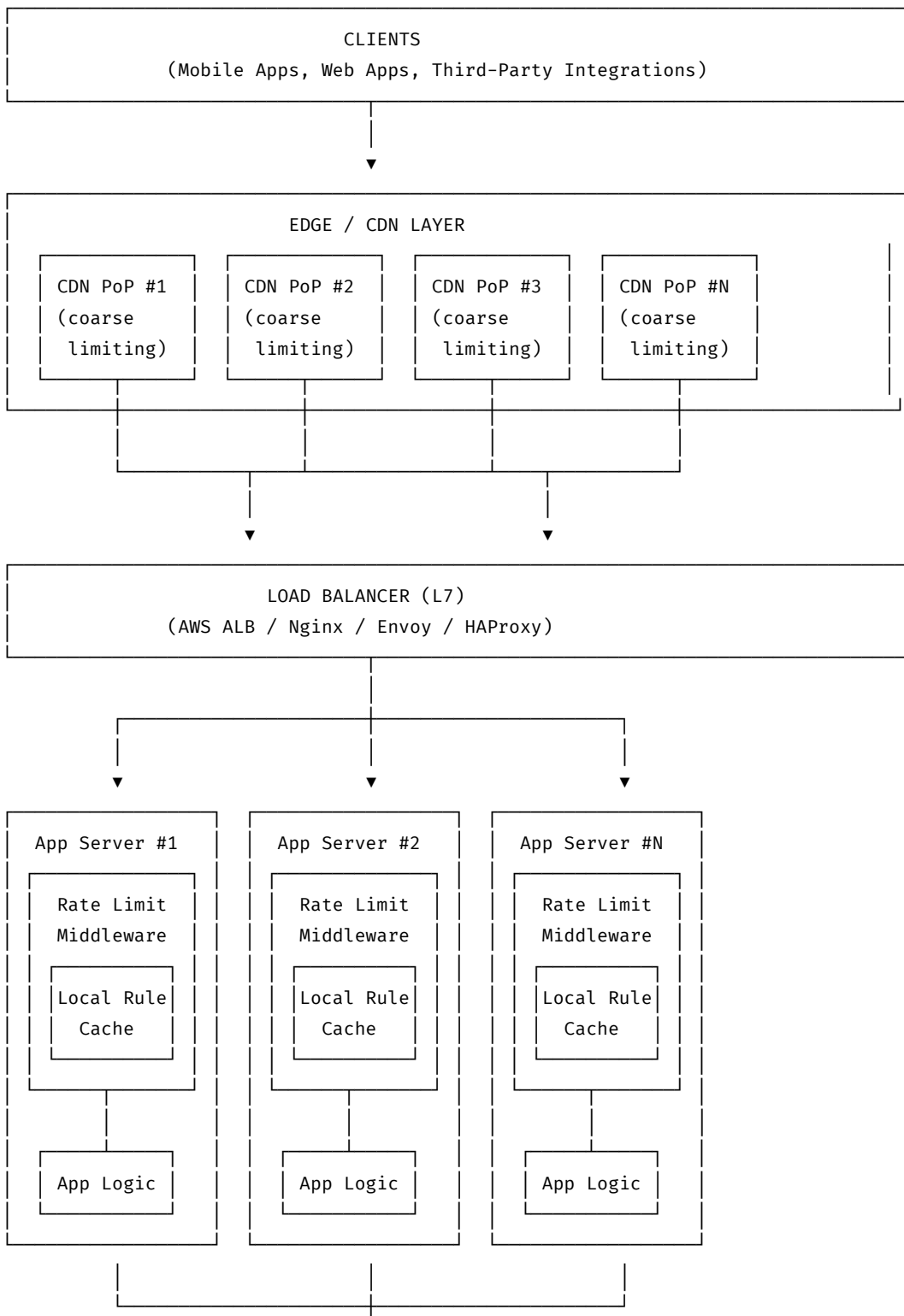
Redis automatically evicts expired keys

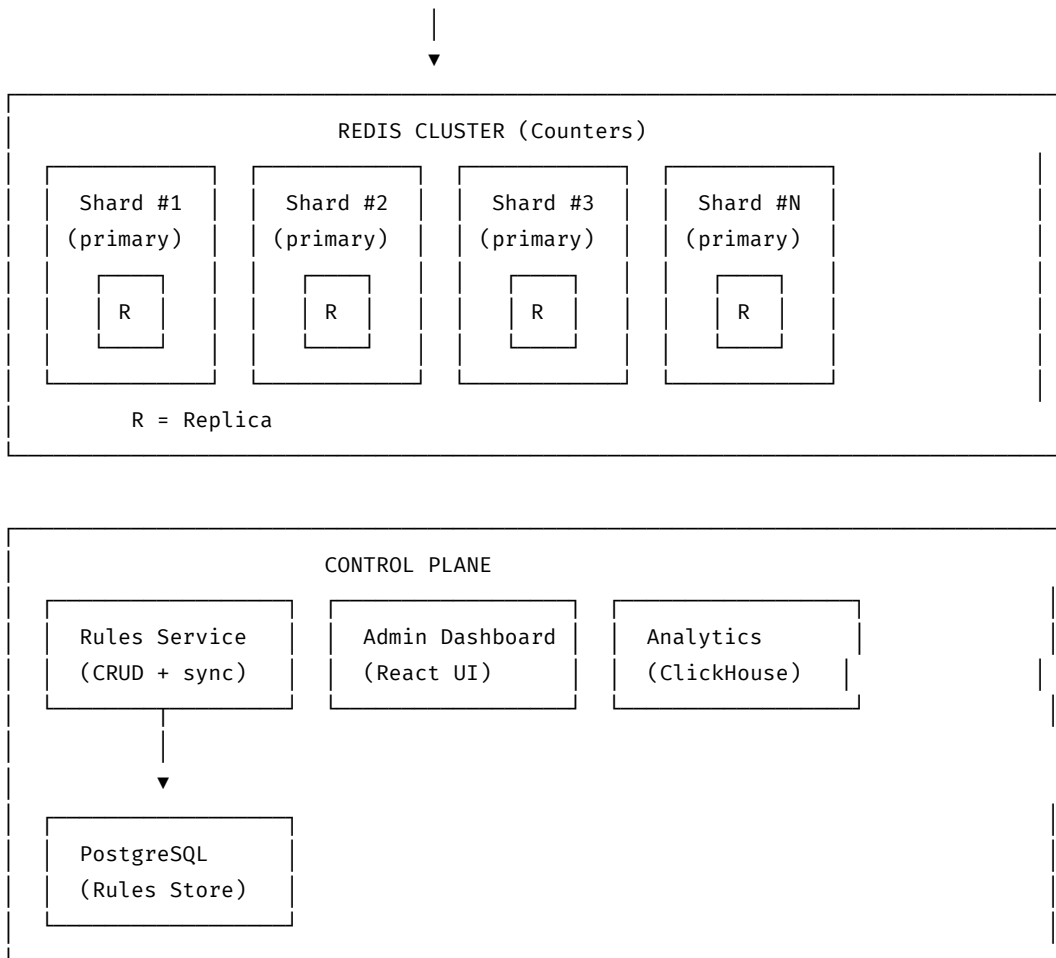
2.6.5 6.5 Entity Relationship



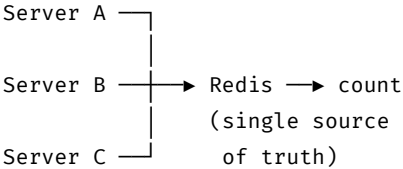
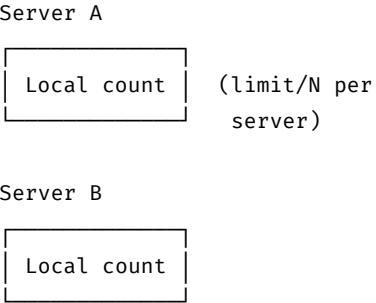
2.7 7. High-Level Architecture

2.7.1 7.1 System Overview





2.7.2 7.2 Centralized vs Distributed Rate Limiting

CENTRALIZED (via Redis)	DISTRIBUTED (local only)
 Server A Server B Server C Redis → count (single source of truth) Pros: + Exact global count + Simple mental model Cons: - Redis is SPOF - Network latency per request - Redis scaling challenges Best for: Exact limits, financial APIs	 Server A Local count (limit/N per server) Server B Local count Pros: + No network hop + No SPOF + Ultra-low latency Cons: - Inaccurate if traffic uneven - Scaling changes break limits Best for: Approximate limits, high volume

2.7.3 7.3 Component Breakdown

- **Rate Limit Middleware:** Intercepts every request, calls rate limit logic; Tech Choice: In-process library
- **Rule Cache:** Local in-memory cache of rate limit rules; Tech Choice: Caffeine / Guava
- **Rate Limit Service:** Core logic: algorithm execution, counter management; Tech Choice: Java / Go library
- **Redis Cluster:** Distributed counter storage with atomic operations; Tech Choice: Redis 7+ Cluster
- **Rules Service:** CRUD API for rate limit rules, pushes updates; Tech Choice: REST microservice
- **Rules Database:** Persistent storage for rate limit configurations; Tech Choice: PostgreSQL
- **Admin Dashboard:** UI for managing rules, viewing usage, analytics; Tech Choice: React + REST
- **Analytics Pipeline:** Collects rate limit events for monitoring and reporting; Tech Choice: Kafka + ClickHouse

2.8 8. Deep Dive: Core Components

2.8.1 8.1 Rate Limiting Algorithms

2.8.1.1 8.1.1 Token Bucket

The token bucket is the most commonly used algorithm (used by AWS API Gateway, Stripe). Tokens are added to a bucket at a fixed rate. Each request consumes a token. If the bucket is empty, the request is rejected.

Parameters: - capacity (bucket size): maximum burst size - refill_rate: tokens added per second

Token Bucket Visualization (capacity=5, refill_rate=1/sec)

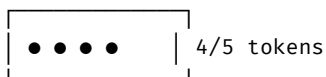
Time 0s: Bucket full



Time 3s: 3 requests consumed



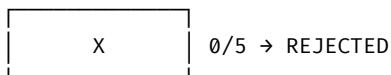
Time 5s: Refilled +2 tokens



Time 6s: Burst of 4 requests



Time 7s: Request DENIED!



Time 8s: 1 token refilled



Redis Implementation (Lua Script):

```

-- Token Bucket via Redis Lua script (atomic)
-- KEYS[1] = bucket key
-- ARGV[1] = capacity, ARGV[2] = refill_rate, ARGV[3] = now (ms), ARGV[4] = cost

local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local cost = tonumber(ARGV[4])

local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(bucket[1]) or capacity
local last_refill = tonumber(bucket[2]) or now

-- Refill tokens
local elapsed = (now - last_refill) / 1000.0
local new_tokens = math.min(capacity, tokens + elapsed * refill_rate)

-- Try to consume
if new_tokens ≥ cost then
    new_tokens = new_tokens - cost
    redis.call('HMSET', key, 'tokens', new_tokens, 'last_refill', now)
    redis.call('PEXPIRE', key, capacity / refill_rate * 1000 + 1000)
    return {1, math.floor(new_tokens)} -- allowed, remaining
else
    redis.call('HMSET', key, 'tokens', new_tokens, 'last_refill', now)
    redis.call('PEXPIRE', key, capacity / refill_rate * 1000 + 1000)
    local wait_time = math.ceil((cost - new_tokens) / refill_rate)
    return {0, 0, wait_time} -- denied, remaining=0, retry_after
end

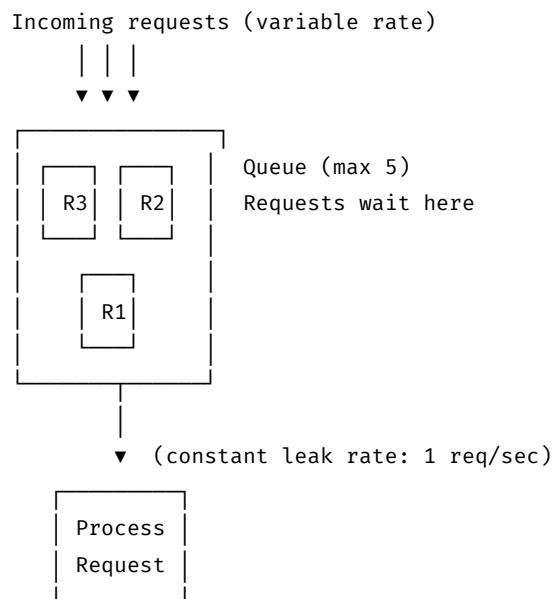
```

Pros: Allows bursts up to bucket capacity, smooth rate limiting, memory efficient. **Cons:** Two parameters to tune, harder to reason about exact window counts.

2.8.1.2 8.1.2 Leaky Bucket

The leaky bucket processes requests at a constant rate. Incoming requests are queued, and the bucket “leaks” (processes) at a fixed rate. If the queue is full, new requests are dropped.

Leaky Bucket (queue_size=5, leak_rate=1/sec)



State transitions:

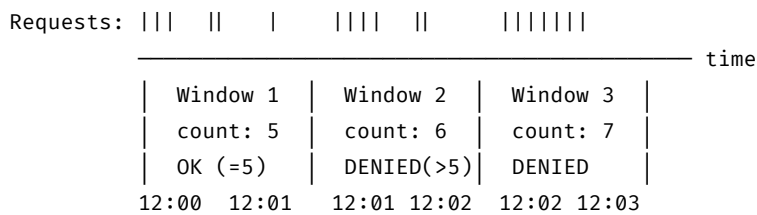
t=0	[R1, R2, R3, R4, R5]	Queue full → R6 DROPPED
t=1	[R2, R3, R4, R5]	R1 processed, space for 1 more
t=2	[R3, R4, R5, R6]	R2 processed, R6 now accepted
t=3	[R4, R5, R6]	R3 processed

Pros: Constant output rate, prevents bursts entirely. **Cons:** Slow processing during burst traffic, stale requests if queue is old, not commonly used for API rate limiting (more common in network traffic shaping).

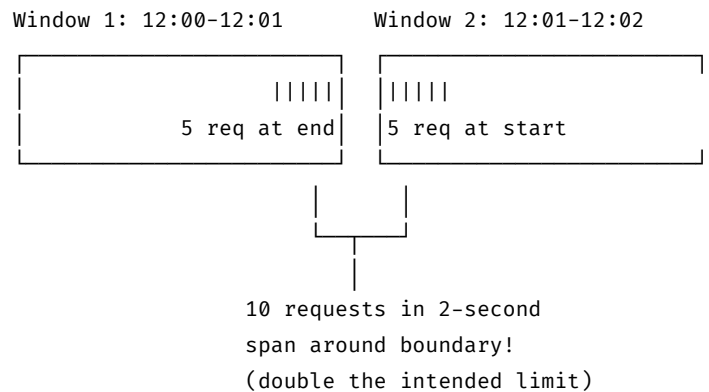
2.8.1.3 8.1.3 Fixed Window Counter

Divide time into fixed windows (e.g., 60-second intervals). Count requests in the current window. Reject when count exceeds the limit.

Fixed Window (limit=5 per 60s)



BOUNDARY BURST PROBLEM:



Redis Implementation:

```
-- Fixed Window via Redis (simple INCR + EXPIRE)
-- KEYS[1] = base key
-- ARGV[1] = window_seconds, ARGV[2] = limit, ARGV[3] = now

local window = math.floor(tonumber(ARGV[3]) / tonumber(ARGV[1]))
local key = KEYS[1] .. ':' .. window
local count = redis.call('INCR', key)

if count == 1 then
    redis.call('EXPIRE', key, tonumber(ARGV[1]) + 1)
end

if count > tonumber(ARGV[2]) then
    return {0, 0}    -- denied
else
    return {1, tonumber(ARGV[2]) - count} -- allowed, remaining
end
```

Pros: Simple, memory efficient (one counter per window), O(1) operations. **Cons:** Boundary burst problem allows 2x limit at window edges.

2.8.1.4 8.1.4 Sliding Window Log

Track every request timestamp in a sorted set. For each new request, remove timestamps outside the window and count remaining entries.

Sliding Window Log (limit=5, window=60s)

Current time: 12:01:30

Sorted Set of timestamps:

12:00:45	12:01:02	12:01:15	12:01:22	12:01:28
----------	----------	----------	----------	----------

count=5

Window: [12:00:30, 12:01:30]
(now - 60s) (now)

New request at 12:01:30:

1. Remove entries before 12:00:30 → none removed
2. Count entries in window → 5
3. $5 \geq \text{limit}(5)$ → DENIED

At 12:01:46 (12:00:45 falls out of window):

1. Remove entries before 12:00:46 → remove 12:00:45
2. Count entries in window → 4
3. $4 < \text{limit}(5)$ → ALLOWED, add 12:01:46

Redis Implementation:

```
-- Sliding Window Log via Redis Sorted Set
-- KEYS[1] = key, ARGV[1] = now_ms, ARGV[2] = window_ms, ARGV[3] = limit

local key = KEYS[1]
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])

-- Remove expired entries
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count current entries
local count = redis.call('ZCARD', key)

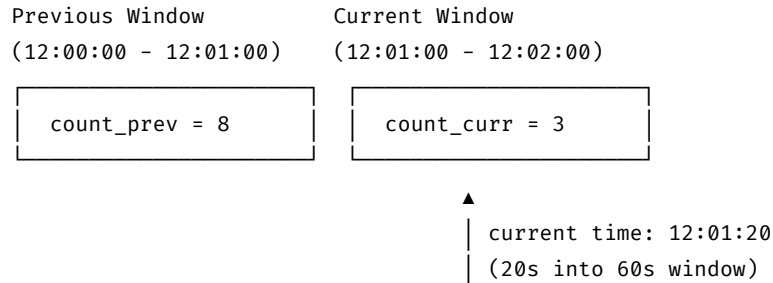
if count < limit then
    -- Add new entry with timestamp as score and unique member
    redis.call('ZADD', key, now, now .. ':' .. math.random(1000000))
    redis.call('PEXPIRE', key, window + 1000)
    return {1, limit - count - 1} -- allowed, remaining
else
    redis.call('PEXPIRE', key, window + 1000)
    return {0, 0} -- denied
end
```

Pros: Exact counting, no boundary problems. **Cons:** High memory usage (stores every timestamp), $O(N)$ cleanup per request.

2.8.1.5 8.1.5 Sliding Window Counter (Recommended)

A hybrid approach that combines the simplicity of fixed windows with the accuracy of sliding windows. Uses weighted counters from the current and previous windows.

Sliding Window Counter (limit=10, window=60s)

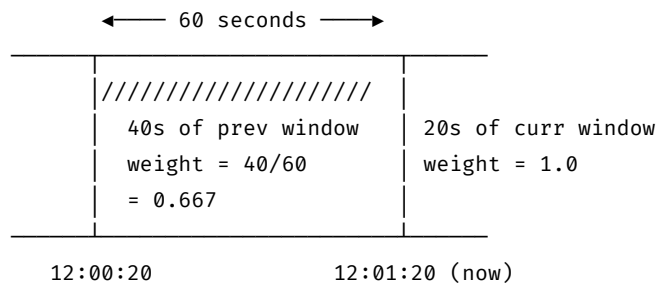


Weighted count formula:

```
weighted = count_prev * (1 - elapsed/window) + count_curr
          = 8 * (1 - 20/60) + 3
          = 8 * 0.667 + 3
          = 5.33 + 3
          = 8.33
```

8.33 < limit(10) → ALLOWED

Visual of the sliding window:



Redis Implementation:

```
-- Sliding Window Counter
-- KEYS[1] = base key
-- ARGV[1] = window_sec, ARGV[2] = limit, ARGV[3] = now_sec

local window = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

local curr_window = math.floor(now / window) * window
local prev_window = curr_window - window
local elapsed = now - curr_window

local curr_key = KEYS[1] .. ':' .. curr_window
local prev_key = KEYS[1] .. ':' .. prev_window

local curr_count = tonumber(redis.call('GET', curr_key) or '0')
local prev_count = tonumber(redis.call('GET', prev_key) or '0')

local weight = 1 - (elapsed / window)
local weighted = prev_count * weight + curr_count

if weighted < limit then
    redis.call('INCR', curr_key)
    redis.call('EXPIRE', curr_key, window * 2 + 1)
    return {1, math.floor(limit - weighted - 1)}
else
    return {0, 0}
end
```

Pros: Memory efficient (2 counters), $O(1)$ operations, no boundary burst problem, good accuracy.

Cons: Approximation (not exact), assumes uniform request distribution in previous window.

2.8.1.6 8.1.6 Algorithm Comparison

Algorithm	Memory per key	Accuracy	Burst Handling	Complexity	Best For
Token Bucket	$O(1)$ ~32 B	Good	Allows bursts	Medium	API gateways (AWS, Stripe)
Leaky Bucket	$O(N)$ ~queue	Exact	Smooths output	Medium	Traffic shaping
Fixed Window	$O(1)$ ~16 B	Poor	2x at boundary	Simple	Simple use cases
Sliding Window Log	$O(N)$ ~req*16B	Exact	None	High	When exact count needed
Sliding Window Counter	$O(1)$ ~32 B	~99.7%	Minimal	Simple	Best default choice

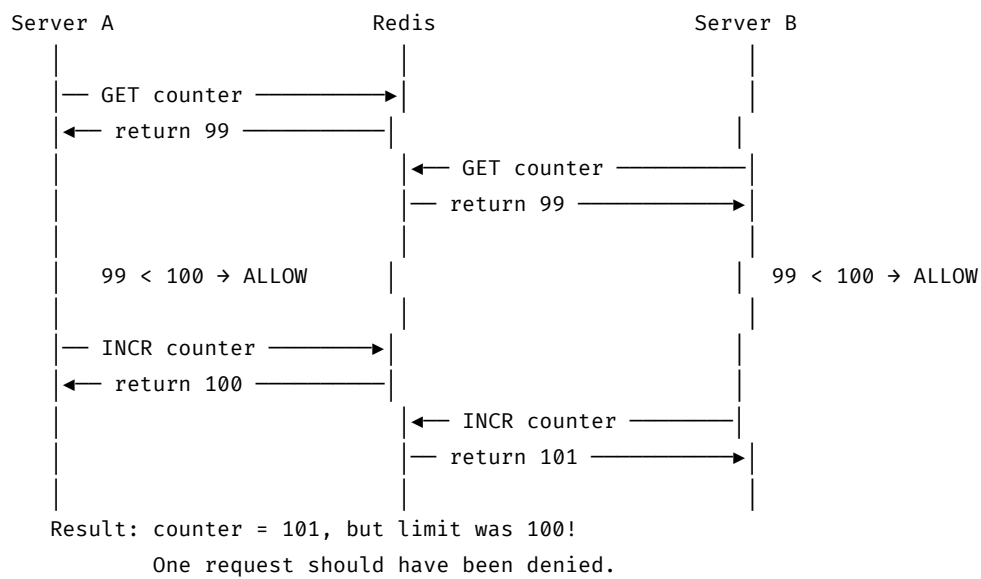
Recommendation: Use Sliding Window Counter as default, Token Bucket when burst control is needed, Sliding Window Log only for financial/compliance use cases.

2.8.2 8.2 Distributed Counter Management

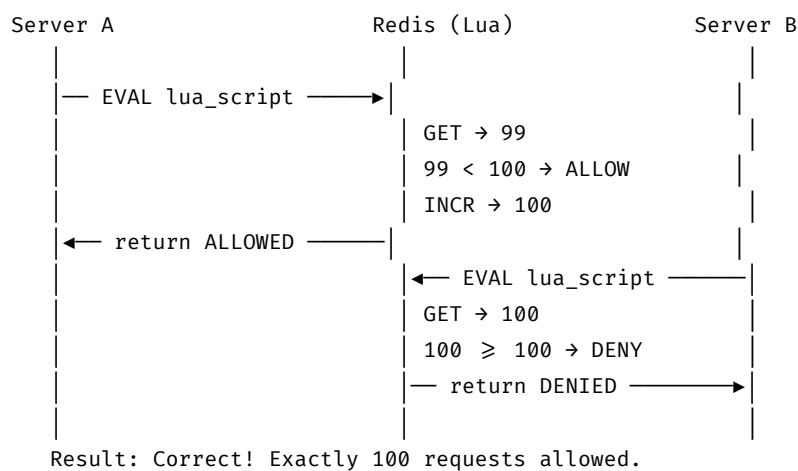
2.8.2.1 8.2.1 Atomic Operations with Redis

All counter operations must be atomic to avoid race conditions. Redis provides two mechanisms: single atomic commands and Lua scripts.

Race Condition WITHOUT atomicity:



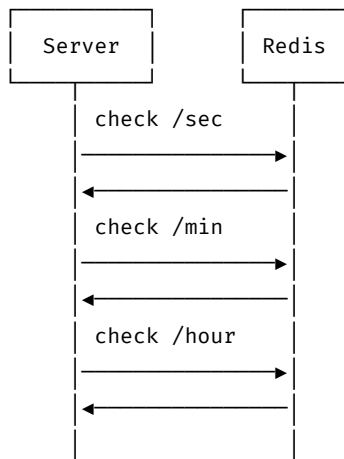
SOLUTION: Lua script runs atomically on Redis



2.8.2.2 Pipeline Optimization

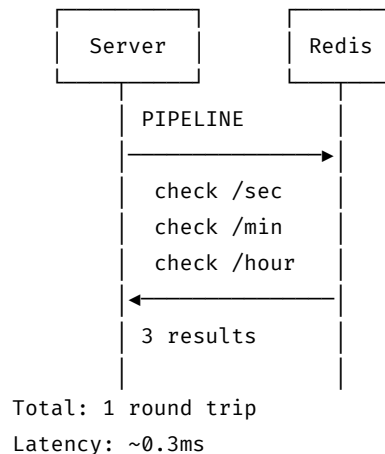
For multiple rate limit checks (e.g., per-second AND per-minute), use Redis pipelines:

Without pipeline (3 round trips):



Total: 3 round trips
Latency: ~0.9ms

With pipeline (1 round trip):



Total: 1 round trip
Latency: ~0.3ms

2.8.2.3 8.2.3 Clock Synchronization

In a distributed system, servers have slightly different clocks. This matters for window-based algorithms.

Problem: Clock skew between servers

Server A (clock: 12:01:00.000)

Server B (clock: 12:00:59.950)

```

Window 2 starts at 12:01:00
curr_count = 0
NEW request → count = 1
  
```

```

Still in Window 1
curr_count = 99
NEW request → count = 100
  
```

Server A thinks it's a new window (count=1)

Server B thinks it's the same window (count=100)

Both write to different Redis keys!

Solutions:

1. Use Redis server time: EVAL with `redis.call('TIME')`
→ All decisions use the same clock
2. Use NTP: Keep servers within 10ms of each other
→ Acceptable for most rate limiting use cases
3. Use sequence numbers instead of timestamps
→ Eliminates clock dependency entirely

Recommended approach: Use Redis server time via Lua scripts. This ensures all rate limit decisions reference the same clock, regardless of app server clock skew.

2.8.3 8.3 Rule Engine

2.8.3.1 8.3.1 Rule Matching

When a request arrives, the rule engine must find the most specific matching rule.

Rule Priority Matching:

Request: POST /api/v2/orders from user-789, tenant-42

Rule evaluation order (highest priority first):

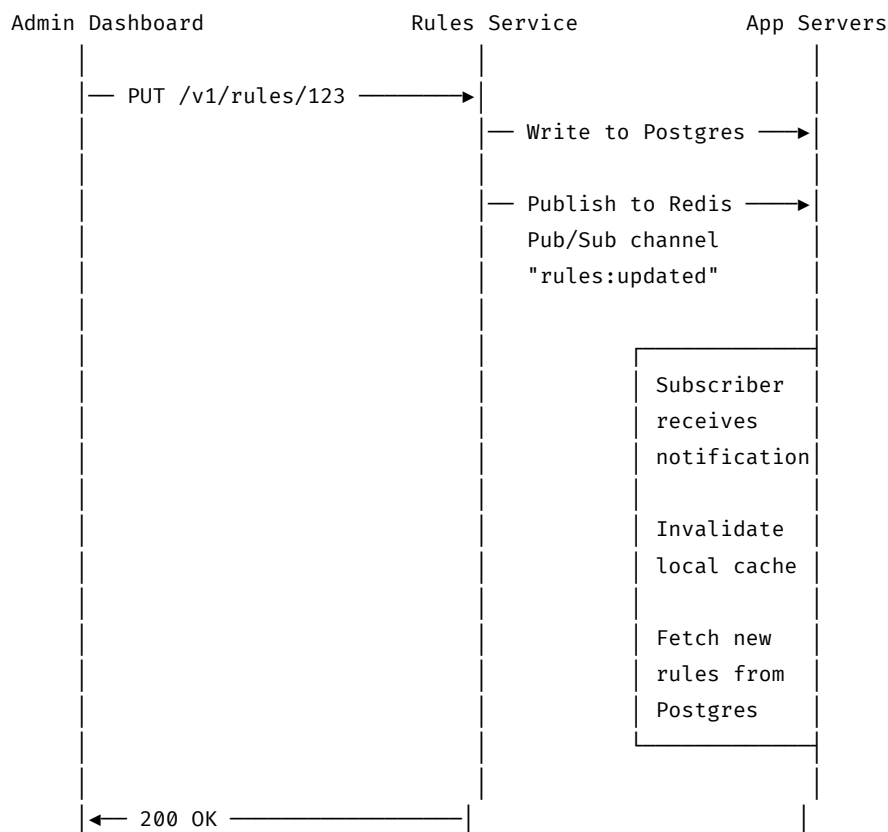
Priority 100:	tenant-42 + POST /api/v2/orders + user-789	← EXACT match
Priority 80:	tenant-42 + POST /api/v2/orders + *	← endpoint match
Priority 60:	tenant-42 + POST /api/v2/* + *	← wildcard path
Priority 40:	tenant-42 + POST /api/** + *	← glob pattern
Priority 20:	tenant-42 + * + *	← tenant default
Priority 0:	* + * + *	← global default

First matching rule wins.

2.8.3.2 8.3.2 Rule Cache and Hot Reload

Rules are stored in PostgreSQL but cached locally for performance. Changes are propagated in near real-time.

Rule Update Flow:



Propagation time: < 1 second (Pub/Sub) + cache TTL fallback (30s)

2.8.3.3 8.3.3 Rule Conflict Resolution

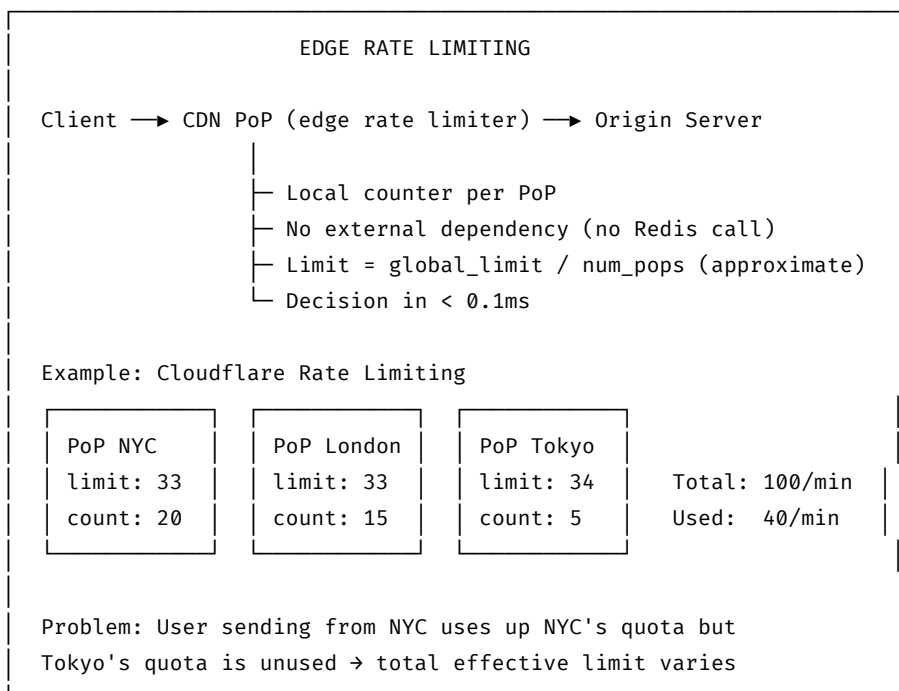
When multiple rules match, apply these resolution rules:

1. Most specific endpoint wins over wildcard
`POST /api/v2/orders > POST /api/v2/* > POST /** > *`
2. Most specific identifier wins
`user:user-789 > apikey:key-456 > ip:10.0.0.1 > global`
3. Higher priority number wins (explicit admin override)
`priority: 100 > priority: 50`
4. Shortest window wins (if same priority)
`10 req/sec evaluated before 1000 req/min`
5. Most restrictive limit applies when ambiguous
 If two rules match with same priority, the lower limit wins

2.8.4 8.4 Edge Rate Limiting vs Centralized

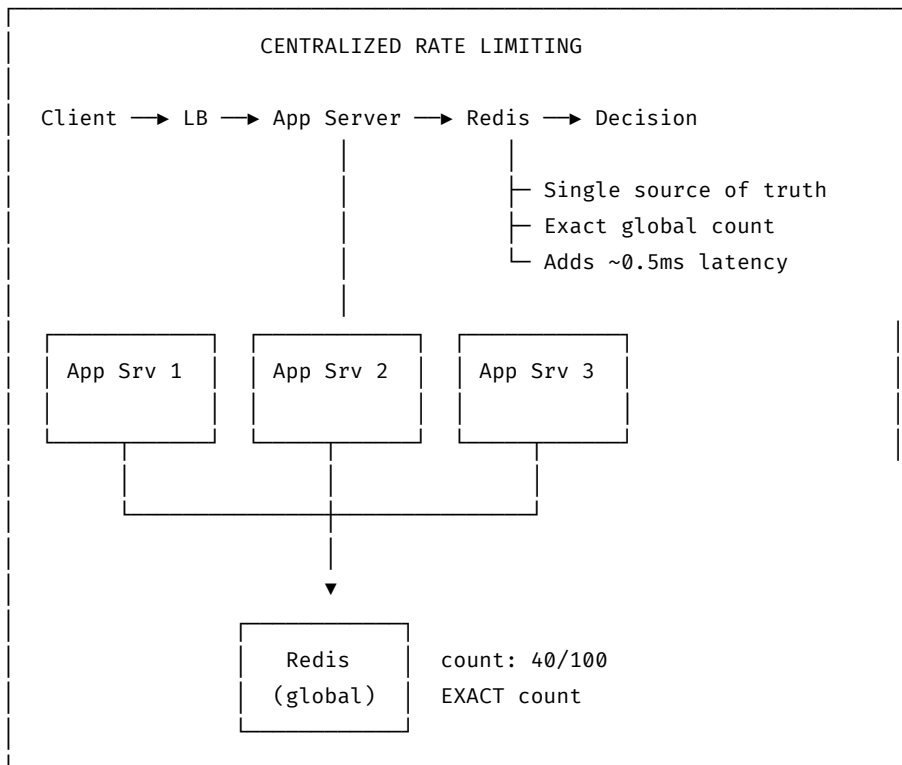
2.8.4.1 8.4.1 Edge Rate Limiting

Rate limiting at the CDN/load balancer layer, before traffic reaches application servers.

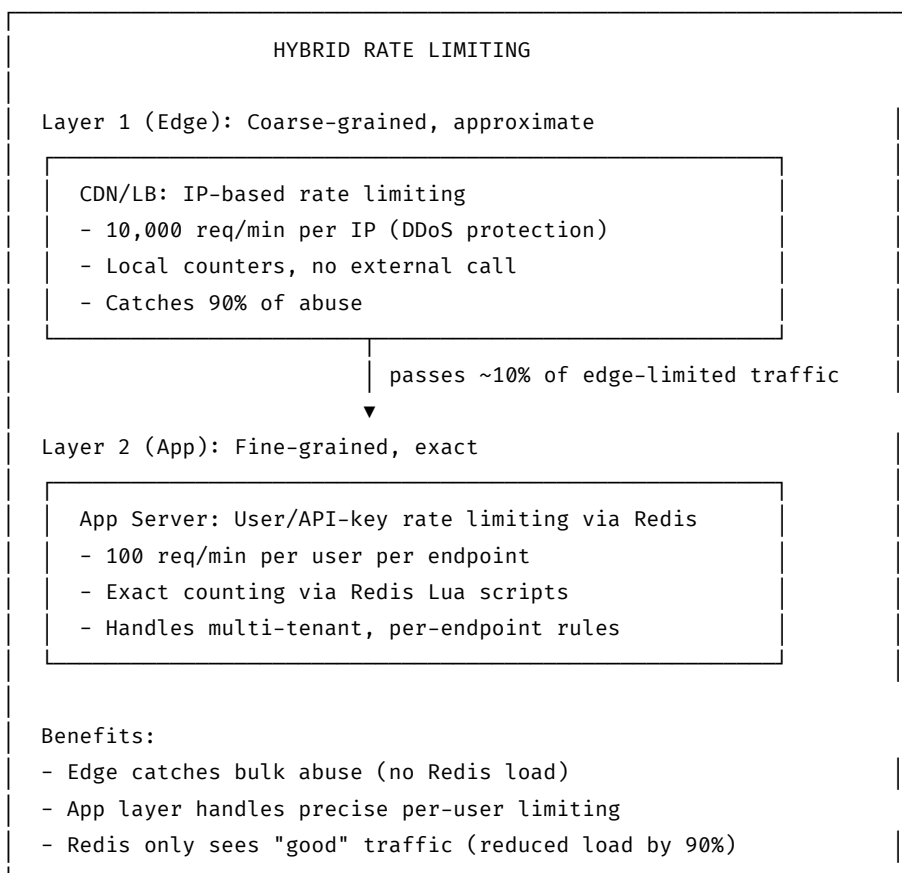


2.8.4.2 8.4.2 Centralized Rate Limiting

Rate limiting at the application layer, using a shared counter store (Redis).



2.8.4.3 8.4.3 Hybrid Approach (Recommended)

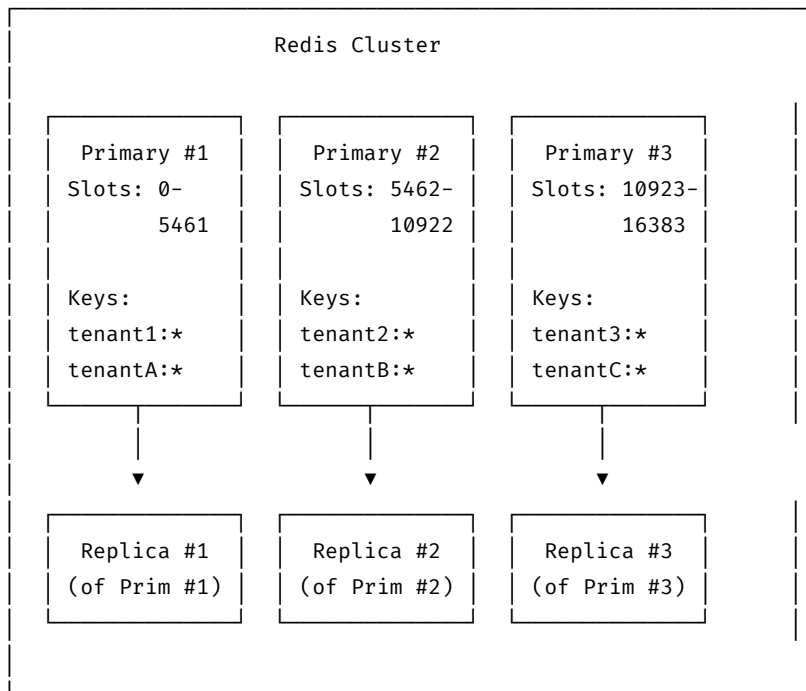


2.9 9. Data Partitioning and Sharding

2.9.1 9.1 Redis Cluster Sharding

Redis Cluster automatically partitions data across shards using hash slots (16,384 slots).

Redis Cluster Topology (6 nodes: 3 primary + 3 replica)



Key distribution using hash tags:

```
rl:{tenant42}:endpoint>window:user → hash("tenant42") → slot 8234 → Primary #2
```

Hash tags {tenant42} ensure all keys for the same tenant land on the same shard, enabling Lua scripts to operate on multiple keys atomically.

2.9.2 9.2 Sharding Strategy

SHARDING STRATEGIES

Strategy 1: Shard by `client_id` (Recommended)

Key: `rl:{client_id_hash}:endpoint:window`

Pros: All limits for one client on same shard

Cons: Hot clients create hot shards

Strategy 2: Shard by `tenant_id`

Key: `rl:{tenant_id}:endpoint:window:client`

Pros: Tenant isolation, Lua scripts work

Cons: Large tenants = hot shards

Strategy 3: Shard by composite key

Key: `rl:{tenant_id:endpoint_hash}:window:client`

Pros: Better distribution, avoids mega-tenant hotspots

Cons: Multi-key Lua scripts won't work across hash tags

Recommendation: Strategy 2 for multi-tenant SaaS (tenant isolation + Lua compatibility), Strategy 1 for single-tenant.

2.9.3 9.3 Handling Hot Shards

Problem: A viral API key generates 1M req/sec, overwhelming one shard.

Solution 1: Local rate limiting for hot keys

```
App Server detects hot key (>1K req/sec)
→ Switch to local counter
→ Sync with Redis every 100ms
→ Accept ~N% over-counting (N = servers)
```

Solution 2: Key splitting

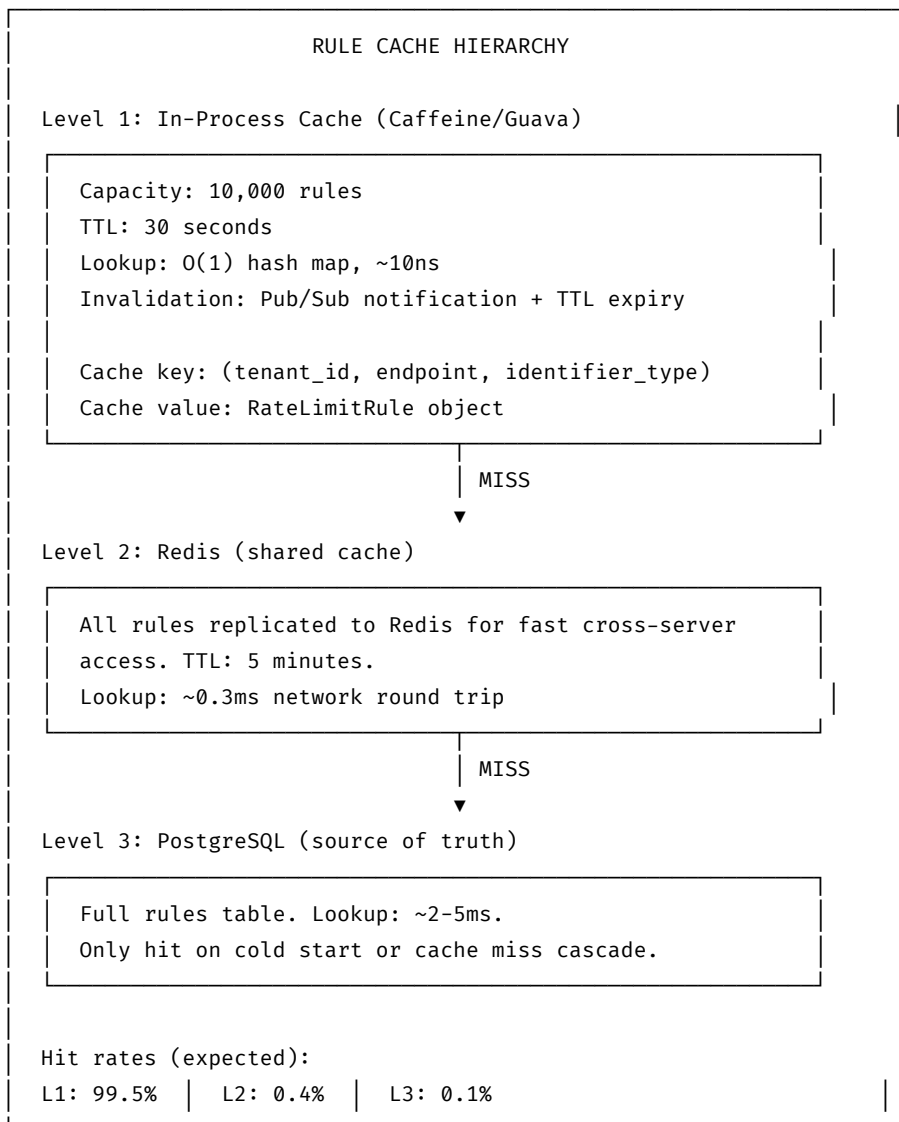
```
Instead of 1 key for a hot client:
rl:{tenant}:endpoint>window:user

Split into K sub-keys:
rl:{tenant}:endpoint>window:user:0
rl:{tenant}:endpoint>window:user:1
rl:{tenant}:endpoint>window:user:2
...
rl:{tenant}:endpoint>window:user:K-1

Server picks sub-key = hash(server_id)%K
Limit per sub-key = global_limit / K
Keys distribute across K different shards
```

2.10 10. Caching Strategy

2.10.1 10.1 Rule Caching



2.10.2 10.2 Counter Approximation with Local Caching

For extremely high throughput, avoid calling Redis on every request by batching counter updates locally.

Local Counter Batching:

```

App Server (in-memory)

local_count = 0
last_sync_count = 850 (from Redis)
limit = 1000

On request:
    local_count++
    estimated_total = last_sync_count
                    + local_count
    if estimated_total > limit → DENY

Every 100ms or 50 local requests:
    sync with Redis:
    redis_count = INCRBY key local_count
    last_sync_count = redis_count
    local_count = 0

```

Trade-off:

- Reduces Redis calls by 50x (batch of 50)
- Introduces counting error of up to:
 $\text{num_servers} * \text{batch_size} = 20 * 50 = 1000$ extra requests
- Acceptable for high-limit rules (10,000/min)
- NOT acceptable for low-limit rules (10/min)

2.10.3 10.3 Cache Warm-Up

On server startup:

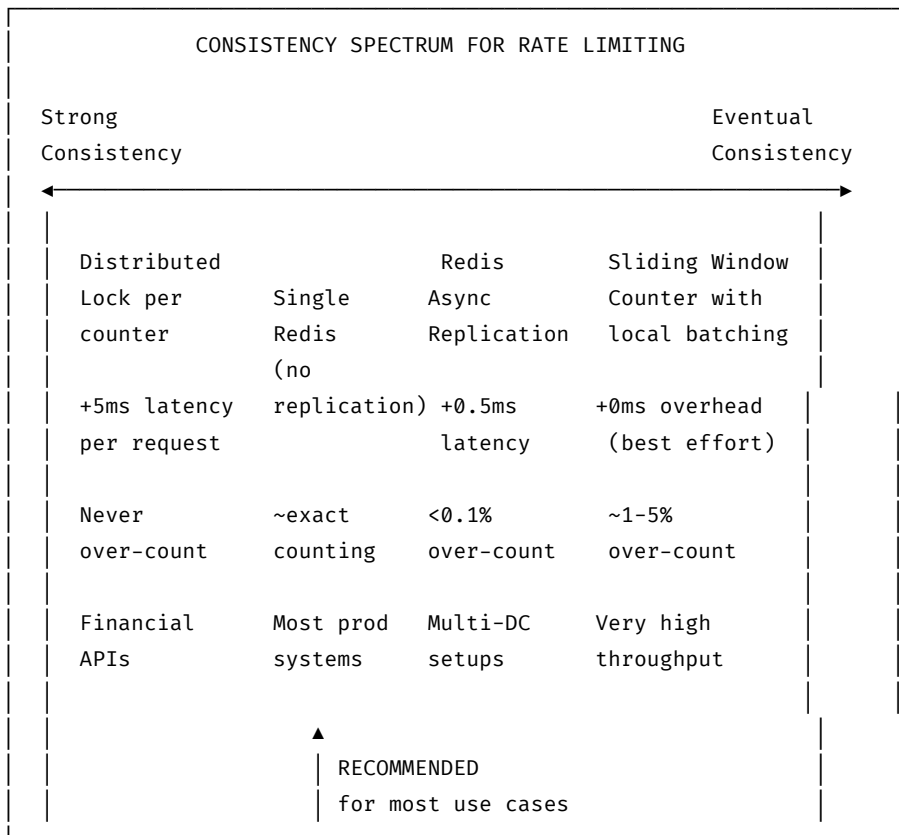
1. Fetch all active rules from PostgreSQL
`SELECT * FROM rate_limit_rules WHERE enabled = true;`
 → ~150K rules, takes ~500ms
2. Populate local cache (Caffeine)
 → All rules available immediately, no cold-start misses
3. Subscribe to Redis Pub/Sub for rule updates
 → Receive real-time invalidations
4. Start accepting traffic
 → First requests don't need to hit Postgres

Startup time: ~1 second (acceptable for rolling deploys)

2.11 11. Replication and Consistency

2.11.1 11.1 Counter Consistency Model

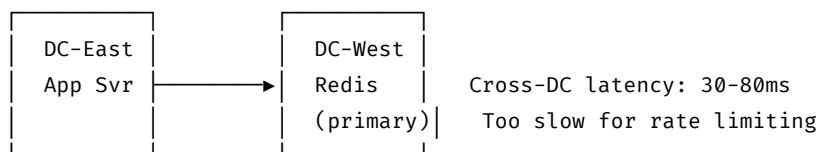
Rate limiting uses **eventual consistency** for counters. This is an intentional design decision: strong consistency would require distributed locks, adding unacceptable latency.



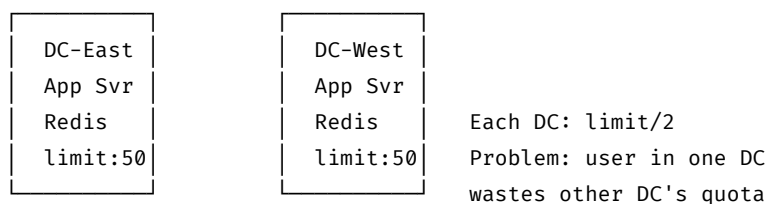
2.11.2 11.2 Multi-Data-Center Consistency

Multi-DC Rate Limiting Options:

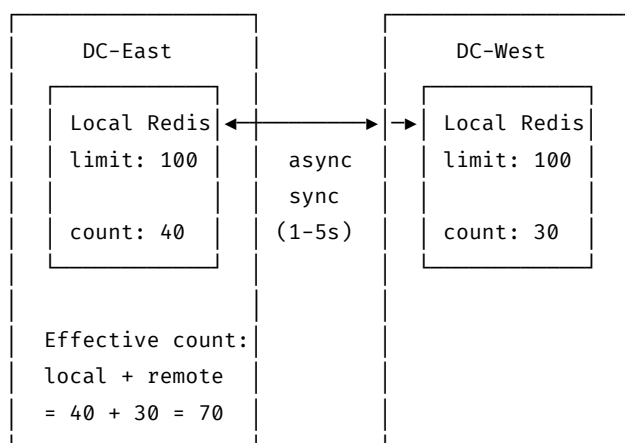
Option A: Shared global counter (high latency)



Option B: Independent counters (inaccurate)



Option C: Async sync with local counters (RECOMMENDED)

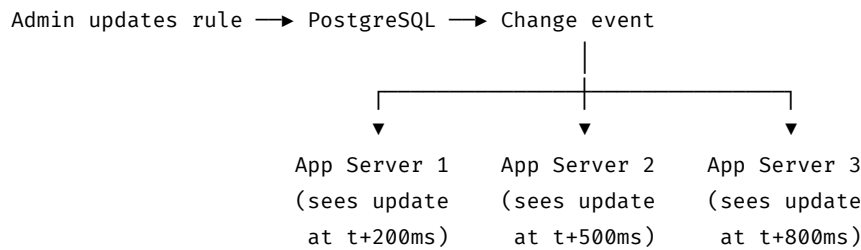


Sync mechanism:

- Every 1-5 seconds, each DC publishes its local delta
- Other DCs adjust their view of the global count
- Worst-case over-count: $\text{num_DCs} * \text{sync_interval} * \text{request_rate}$
- Example: $3 \text{ DCs} * 5 \text{ s} * 100 \text{ req/s} = 1500 \text{ extra requests (1.5\% of 100K limit)}$

2.11.3 11.3 Rule Consistency

Rules propagation (eventually consistent):



During the propagation window (up to ~1 second):

- Some servers use old rule, others use new rule
- This is acceptable: rule changes are rare and not latency-sensitive
- Safety measure: version number on rules prevents reverting to older versions

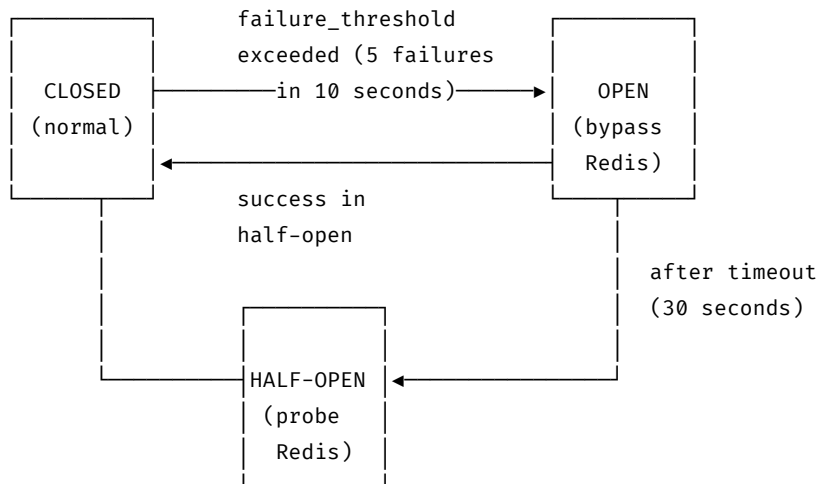
2.12 12. Fault Tolerance and Failure Handling

2.12.1 12.1 Failure Scenarios

Failure Scenario	Handling Strategy
Redis primary down	Automatic failover to replica (Redis Sentinel or Cluster). Failover time: 5-30 seconds. During failover: use fail mode (open/closed).
Redis cluster partially down (1 shard)	Keys on failed shard unavailable. Other shards still work. Affected keys use fail mode.
Network partition (app ↔ Redis)	App servers can't reach Redis. Fall back to local in-memory counters with reduced limits.
PostgreSQL down	Rules already cached locally. No impact on rate limiting. Rule updates queued until back.
App server crash	Stateless middleware. New server picks up traffic immediately. Counters in Redis.
Clock drift	Use Redis server time in Lua scripts. App server clocks irrelevant for counting.
Counter data loss (Redis restart)	Counters are ephemeral (TTL-based). Loss means some requests bypass limits temporarily. Counters rebuild within one window period.

2.12.2 12.2 Circuit Breaker for Redis

Circuit Breaker State Machine:



CLOSED: Normal operation, all requests go to Redis

OPEN: Redis calls skipped, use fail mode (open/closed/local)

HALF-OPEN: Send 1 probe request to Redis every 5 seconds
If success → CLOSED, if failure → OPEN

Implementation:

```

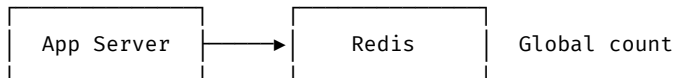
class RateLimiterWithCircuitBreaker {
    CircuitBreaker cb = new CircuitBreaker(
        failureThreshold: 5,
        timeout: 30s,
        halfOpenProbes: 1
    );

    RateLimitResponse check(request) {
        if (cb.isOpen()) {
            return applyFailMode(request.rule.failMode);
        }
        try {
            result = redis.eval(luaScript, ...);
            cb.recordSuccess();
            return result;
        } catch (RedisException e) {
            cb.recordFailure();
            return applyFailMode(request.rule.failMode);
        }
    }
}
  
```

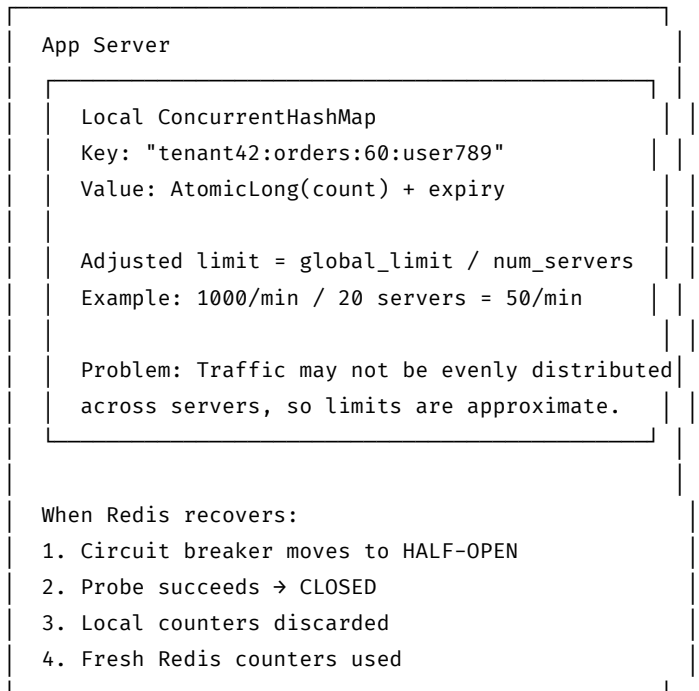
2.12.3 12.3 Local Fallback Counters

When Redis is unavailable, fall back to local in-memory counters:

Normal operation:

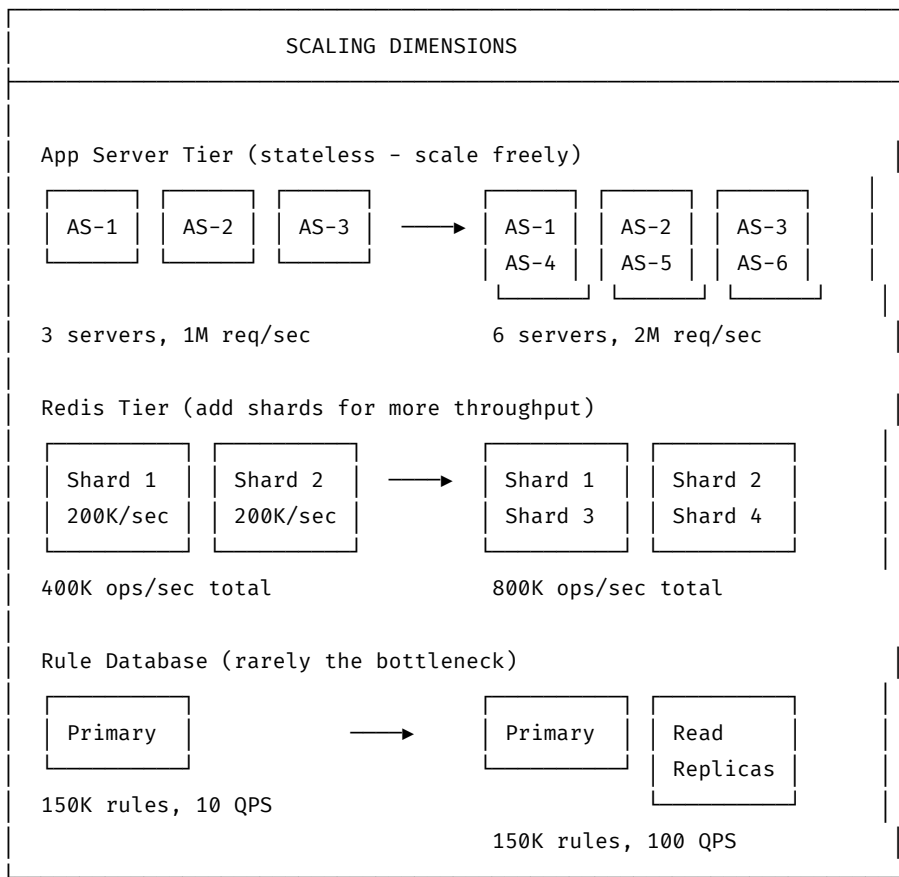


Redis down (circuit breaker OPEN):



2.13 13. Scalability

2.13.1 13.1 Horizontal Scaling



2.13.2 13.2 Performance Characteristics

Operation Complexity:

Operation	Time	Space
Rule lookup (local cache)	$O(1)$	$O(R)$
Rule lookup (Redis)	$O(1)$	$O(R)$
Token bucket check	$O(1)$	$O(1)/\text{key}$
Fixed window check	$O(1)$	$O(1)/\text{key}$
Sliding window counter	$O(1)$	$O(1)/\text{key}$
Sliding window log	$O(N)$	$O(N)/\text{key}$
Allow/deny list check	$O(1)$	$O(L)$
Redis Lua script	$O(1)*$	$O(1)$

R = number of rules, L = list size, N = requests in window

* $O(1)$ assuming constant-time Redis operations

Latency Breakdown (p99):

Rule cache lookup:	0.01 ms
Allow/deny list check:	0.01 ms
Network to Redis:	0.20 ms
Redis Lua script exec:	0.05 ms
Network from Redis:	0.20 ms
Response header injection:	0.01 ms
Total overhead:	0.48 ms (< 1ms target)

2.13.3 13.3 Scaling Bottlenecks and Mitigations

Bottleneck	Symptom	Mitigation
Redis throughput	High latency on Redis ops	Add more shards, use local batching
Redis memory	OOM errors	Shorter TTLs, fewer windows, eviction policy
Network (app→Redis)	Increased latency	Pipeline, connection pool, colocate Redis with app
Hot keys (viral user/endpoint)	One shard overloaded	Key splitting, local counter for hot keys
Rule evaluation (regex matching)	High CPU on app servers	Precompile patterns, use hash lookup
Pub/Sub fan-out (many subscribers)	Slow rule updates	Batch updates, use polling with ETag

2.14 14. Monitoring and Observability

2.14.1 14.1 Key Metrics

MONITORING DASHBOARD	
REQUEST METRICS	
Total requests checked:	10,234,567 / sec
Requests allowed:	9,876,543 / sec (96.5%)
Requests denied (429):	358,024 / sec (3.5%)
Requests bypassed (allow list):	12,345 / sec
Requests blocked (deny list):	1,234 / sec
LATENCY METRICS	
Rate limit check p50:	0.15 ms
Rate limit check p99:	0.48 ms
Rate limit check p999:	1.20 ms
Redis round trip p50:	0.12 ms
Redis round trip p99:	0.35 ms
Rule cache hit rate:	99.7%
REDIS CLUSTER METRICS	
Total ops/sec:	8,234,567
Memory usage:	6.2 GB / 10 GB (62%)
Connected clients:	1,200
Key count:	45,678,901
Evicted keys:	0 / sec
Cluster health:	OK (all slots covered)
ACCURACY METRICS	
Counter accuracy drift:	0.3% (target < 1%)
False allows (over-count):	~12 / million requests
False denies (under-count):	~3 / million requests
Clock skew (max):	8ms across fleet
ALERTING THRESHOLDS	
WARN: Denial rate > 5% (potential abuse or bad rule)	
WARN: Redis p99 > 1ms (network/capacity issue)	
CRIT: Redis unavailable (circuit breaker triggered)	
CRIT: Rule cache hit rate < 95%	
WARN: Counter drift > 2%	

2.14.2 14.2 Logging Strategy

Log levels for rate limiting events:

DEBUG: Every rate limit check (only in dev/staging)

```
{
  "level": "DEBUG",
  "client": "user-789",
  "endpoint": "POST /orders",
  "count": 45,
  "limit": 100,
  "remaining": 55,
  "allowed": true
}
```

INFO: Rate limit rule changes

```
{
  "level": "INFO",
  "action": "RULE_UPDATED",
  "rule_id": "rule-123",
  "old_limit": 100,
  "new_limit": 200,
  "updated_by": "admin@co.com"
}
```

WARN: Client approaching limit (> 80% used)

```
{
  "level": "WARN",
  "client": "user-789",
  "endpoint": "POST /orders",
  "count": 85,
  "limit": 100,
  "pct_used": 85
}
```

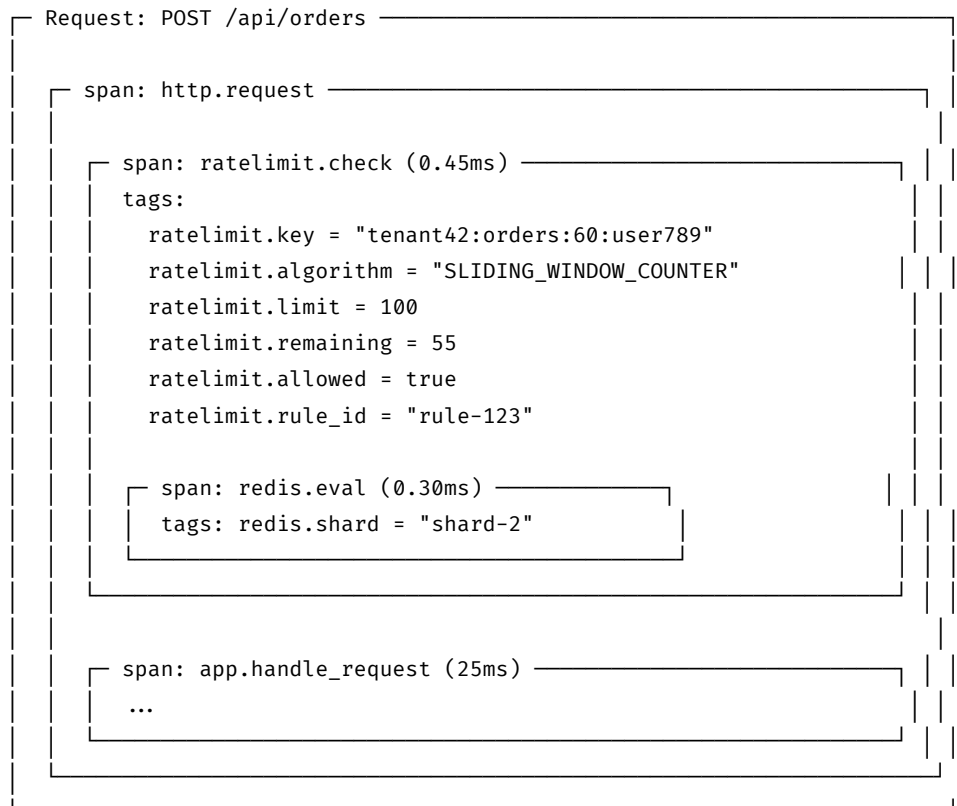
ERROR: Redis connection failures, circuit breaker state changes

```
{
  "level": "ERROR",
  "event": "CIRCUIT_BREAKER_OPEN",
  "redis_shard": "shard-3",
  "fail_mode": "OPEN",
  "duration": "30s"
}
```

Sampling: In production, log 1% of checks (DEBUG), 100% of denials (INFO)

2.14.3 14.3 Distributed Tracing Integration

Request trace with rate limiting span:



2.15 15. Trade-offs and Design Decisions

2.15.1 15.1 Key Trade-offs

TRADE-OFF 1: Token Bucket vs Sliding Window Counter	
Token Bucket	Sliding Window Counter
+ Natural burst handling + Industry standard (AWS, Stripe) + Two knobs (capacity + rate) - Harder to reason about - "100 req/min" doesn't mean exactly 100 in any minute	+ Simpler mental model + Easy to explain limits + One knob (count/window) - No built-in burst control - "100 req/min" means exactly that in any sliding minute
Decision: Use Sliding Window Counter as default for simplicity. Offer Token Bucket as option for APIs that need burst control.	
TRADE-OFF 2: Centralized (Redis) vs Distributed (Local)	
Centralized (Redis)	Distributed (Local)
+ Exact counts + Works with any server count - Redis is a dependency - Network hop per request	+ No SPOF + Zero network latency - Inaccurate with uneven traffic - Scaling changes break limits
Decision: Centralized with local fallback (circuit breaker). Redis for accuracy; local counters only during Redis outages.	
TRADE-OFF 3: Accuracy vs Performance	

High Accuracy

- + Exact enforcement
- + No over/under counting
- Redis call per request
- Higher latency

High Performance

- + Lower latency
- + Less Redis load
- Up to N% over-counting
- Batched syncs

Decision: Per-request Redis check for low-limit rules (< 100/min).
Local batching for high-limit rules (> 1000/min) where 1-5%
over-counting is acceptable.

TRADE-OFF 4: Fail-Open vs Fail-Closed

Fail-Open

- + Service stays available
- + Better user experience
- Abuse possible during outage
- Downstream overload risk

Fail-Closed

- + Protects downstream systems
- + Security-safe
- Service appears down
- Customer impact

Decision: Configurable per rule. Default fail-open for most APIs.
Fail-closed for payment and security-sensitive endpoints.
Fail-local (in-memory fallback) as middle ground.

2.15.2 15.2 Technology Choices Rationale

Decision	Choice	Why Not Alternatives
Counter store	Redis Cluster	Memcached: no Lua scripts, no cluster mode, no persistence DynamoDB: too slow (5-10ms), expensive at 10M ops/sec In-memory only: not distributed
Rule store	PostgreSQL	MySQL: less feature-rich for JSON rules / pattern matching Redis: not durable enough for configuration data
Rule propagation	Redis Pub/Sub	Kafka: overkill for config changes (low volume) Polling: higher latency (acceptable as fallback)
Local cache	Caffeine	Guava Cache: slower, less memory efficient EhCache: heavier, disk-based
Analytics	Kafka + ClickHouse	Direct DB writes: too slow Prometheus only: no drill-down Elasticsearch: expensive at 10M events/sec

2.16 16. Interview Deep-Dive Questions

2.16.1 Q1: How do you rate limit across multiple data centers?

Answer: Use a hybrid approach with local Redis clusters per data center and asynchronous cross-DC counter synchronization.

Each DC maintains its own Redis cluster with full rate limit counters. Periodically (every 1-5 seconds), each DC publishes its counter deltas to other DCs via a cross-DC message bus (e.g., Kafka with cross-region replication, or a purpose-built gossip protocol).

Each DC calculates the effective count as: $\text{effective} = \text{local_count} + \text{sum}(\text{remote_dc_counts})$

This means the system is eventually consistent across DCs with a sync delay of 1-5 seconds. The worst-case over-count is: $\text{num_DCs} * \text{sync_interval} * \text{request_rate_per_DC}$

For most use cases, this is acceptable. For strict global limits (e.g., financial transaction caps), you would need a single global Redis cluster, accepting the cross-DC latency penalty (30-80ms).

2.16.2 Q2: What happens during a Redis failover?

Answer: Redis Sentinel or Redis Cluster handles automatic failover. The timeline:

1. **Detection** (5-15s): Sentinel detects primary is down via heartbeats.
2. **Election** (1-5s): Sentinels vote on a new primary from replicas.
3. **Promotion** (1-2s): Replica promoted, clients redirected.

During this 10-30 second window, the rate limiter circuit breaker opens, and the configured fail mode activates. With fail-open, all requests are allowed temporarily. With fail-local, in-memory counters enforce approximate limits.

Data loss: Redis async replication means the promoted replica may be a few seconds behind. Some counter increments from the old primary are lost. This means the system slightly under-counts after failover. For rate limiting, this is preferable to over-counting (which would wrongly block users).

2.16.3 Q3: How do you handle the fixed window boundary burst problem?

Answer: The boundary burst problem occurs when a client sends their full quota at the end of one window and the beginning of the next, effectively getting 2x the limit in a short burst.

Solution: Sliding Window Counter. Instead of a single counter per window, maintain counters for both the current and previous windows. The effective count is:

```
weighted_count = prev_count * (1 - elapsed/window) + curr_count
```

This creates a virtual sliding window by interpolating between the two fixed windows. The approximation assumes uniform distribution within the previous window, which introduces ~0.3% error in practice – far better than the 100% burst error of pure fixed windows.

2.16.4 Q4: How do you rate limit WebSocket connections vs HTTP requests?

Answer: WebSocket rate limiting has two dimensions:

1. **Connection rate limiting:** Limit how many WebSocket connections a client can establish per time window (e.g., 10 connections/minute). This uses the same mechanisms as HTTP rate limiting, triggered on the `Upgrade` handshake.
2. **Message rate limiting:** Once connected, limit the number of messages a client can send per time window (e.g., 100 messages/second). This requires the WebSocket handler to call the rate limiter on each incoming frame.

Key difference: HTTP rate limiting is stateless (each request independent), while WebSocket message limiting is stateful (tied to a long-lived connection). For WebSocket messages, a **token bucket** is ideal because it allows message bursts (e.g., rapid typing) while enforcing an average rate.

Implementation: Maintain a per-connection token bucket in local memory (no need for Redis since the connection is pinned to one server). Only use Redis for connection establishment rate limiting.

2.16.5 Q5: How would you implement a “leaky bucket” at global scale?

Answer: A true leaky bucket queues requests and processes them at a constant rate. At global scale, this requires a distributed queue.

Architecture: - Incoming requests are pushed to a partitioned queue (e.g., Kafka topic, partitioned by client_id) - Consumer group processes messages at a fixed drain rate (e.g., 100 msg/sec per client) - If the queue for a client is full (configurable depth), new messages are rejected (429)

Challenges at scale: - Each client needs its own logical queue partition – millions of partitions - Constant drain rate requires a timer per client – use a priority queue ordered by next-allowed-time - Memory overhead: storing queued requests is expensive vs just counting them

Recommendation: For API rate limiting, the token bucket achieves similar smoothing with far less complexity. The leaky bucket is better suited for network traffic shaping where actual packet queuing is needed.

2.16.6 Q6: How do you prevent a single hot tenant from affecting other tenants?

Answer: Multi-tenant isolation requires multiple strategies:

1. **Separate Redis key namespaces:** Each tenant's counters use {tenant_id} as a Redis hash tag, ensuring they land on predictable shards.
2. **Per-tenant Redis connection pools:** Prevent one tenant's traffic from exhausting another tenant's Redis connections.
3. **Hot tenant detection:** Monitor per-tenant request rates. When a tenant exceeds a threshold (e.g., 10x their limit), trigger local rate limiting for that tenant (skip Redis entirely, use in-memory counters).
4. **Shard isolation for premium tenants:** Large enterprise tenants get dedicated Redis shards, ensuring their counter operations never compete with other tenants.
5. **Admission control:** At the load balancer level, apply coarse per-tenant limits before traffic reaches the app layer.

2.16.7 Q7: What is the cost of this system at scale?

Answer: For 10M requests/second:

Redis Cluster:

50 shards * 2 (primary + replica) = 100 nodes
 r6g.xlarge (4 vCPU, 26 GB RAM): ~\$0.25/hr per node
 Monthly: 100 * \$0.25 * 730 = ~\$18,250/month

App Server overhead:

Rate limiting adds ~0.5ms per request
 20 app servers, rate limiting uses ~5% CPU
 Marginal cost: ~\$500/month

PostgreSQL (rules):

Single db.r6g.large: ~\$300/month

Kafka + ClickHouse (analytics):

~\$2,000/month

Total: ~\$21,000/month for 10M req/sec rate limiting

Per request: ~\$0.000000068 (essentially free per request)

2.16.8 Q8: How do you handle rate limit key collision?

Answer: Key collisions happen when different clients map to the same Redis key (e.g., hash collision in key generation).

Prevention: - Use full identifiers in keys (not hashes): `rl:{tenant}:{endpoint}:{window}:{user_id}` - If key length is a concern, use SHA-256 hash with collision probability $\sim 1/2^{128}$ - Include the identifier type in the key to prevent cross-type collisions

Detection: - Monitor anomalies where a client's actual request count differs significantly from the counter value - Log the full key on every 429 response for debugging

In practice, using full identifiers (not hashes) in Redis keys eliminates this problem entirely. Redis supports keys up to 512MB, so a 200-byte key is fine.

2.16.9 Q9: How would you implement tiered rate limiting (free/pro/enterprise)?

Answer: Map tier to rate limit rules:

Tier	/api/search	/api/upload	/api/export
Free	10/min	5/hour	1/day
Pro	100/min	50/hour	10/day
Enterprise	10,000/min	1,000/hour	unlimited

Implementation: 1. Each tenant has a `plan_tier` field in the tenant table. 2. Rate limit rules have a `tier` field: `WHERE tier = tenant.plan_tier`. 3. On tier upgrade, update the tenant's tier and invalidate their cached rules. 4. Rules engine resolves: tier-specific rule > default rule.

Edge case: When a user upgrades mid-window, should the new limit apply immediately or at the next window? Recommendation: apply immediately by resetting the counter for that window. This provides a better user experience and incentivizes upgrades.

2.16.10 Q10: How do you test a rate limiter in production?

Answer:

1. **Shadow mode:** Run the rate limiter in observe-only mode, logging decisions without enforcing them. Compare shadow decisions against expected behavior.
2. **Canary deployment:** Enable enforcement on 1% of traffic first. Monitor 429 rates and customer complaints.
3. **Load testing:** Use tools like Locust or k6 to simulate burst traffic patterns. Verify counters are accurate and latency stays under 1ms.
4. **Chaos testing:**
 - Kill Redis shards and verify circuit breaker activates
 - Inject network latency between app servers and Redis
 - Simulate clock skew between servers
5. **Accuracy verification:** Periodically compare Redis counter values against actual request counts from access logs. Alert if drift exceeds 1%.

2.16.11 Q11: Should rate limiting be implemented as a library or a service?

Answer:

	Library (SDK)	Service (sidecar or standalone)
Latency	Lower (in-proc)	Higher (network hop)
Language support	Per-language SDK	Language-agnostic (HTTP/gRPC)
Deployment	Per-app upgrade	Independent deployment
Consistency	Rule sync needed	Centralized rules
Complexity	Simpler	More infrastructure
Best for	Homogeneous stack (all Java)	Polyglot microservices or service mesh (Envoy, Istio)

Recommendation for most teams: Start with a library (lower complexity), move to a sidecar if you have multiple languages or need centralized control. Envoy proxy with its rate limiting filter is a good middle ground.

2.16.12 Q12: How do you handle rate limiting for distributed transactions?

Answer: When a single user action triggers multiple API calls (e.g., placing an order calls inventory, payment, and notification services), you must decide: does each internal call count toward the limit, or only the user-facing request?

Approach: Use a **request cost** model. The user-facing request has `cost=1`. Internal service-to-service calls carry a context header (`X-RateLimit-Internal: true`) and are either excluded from rate limiting or counted with `cost=0`.

Alternatively, rate limit only at the **edge** (API gateway). Internal calls bypass the rate limiter entirely since they are already bounded by the edge limit.

2.16.13 Q13: How do you handle sudden traffic spikes (flash crowds)?

Answer:

1. **Token bucket with burst capacity:** Set bucket capacity higher than the sustained rate. E.g., `limit=100/min`, `burst=50` means 50 requests can arrive instantly, then 100/min sustained.
2. **Adaptive rate limiting:** Monitor backend health (CPU, latency, error rate). When backends are stressed, dynamically lower rate limits. When healthy, restore them.

```

if backend_error_rate > 5%:
    dynamic_limit = base_limit * 0.5
elif backend_p99_latency > 500ms:
    dynamic_limit = base_limit * 0.75
else:
    dynamic_limit = base_limit

```

3. **Prioritized rate limiting:** Under load, tier limits differently. Enterprise customers keep their full limits, while free-tier limits are reduced. This is effectively quality-of-service (QoS) based rate limiting.
-

2.16.14 Q14: What is the difference between rate limiting and throttling?

Answer:

- **Rate limiting:** Binary decision. Request is either allowed or rejected (429). The client must retry later. Protects the server.
- **Throttling:** Delays the request rather than rejecting it. The request is queued and processed when capacity is available. Better for batch/async workloads.
- **Backpressure:** Server signals the client to slow down (e.g., via flow control in gRPC or TCP window size). Client cooperatively reduces its rate.

In practice, API rate limiting uses rejection (429 + Retry-After), while internal service-to-service communication may use throttling or backpressure to avoid cascading failures.

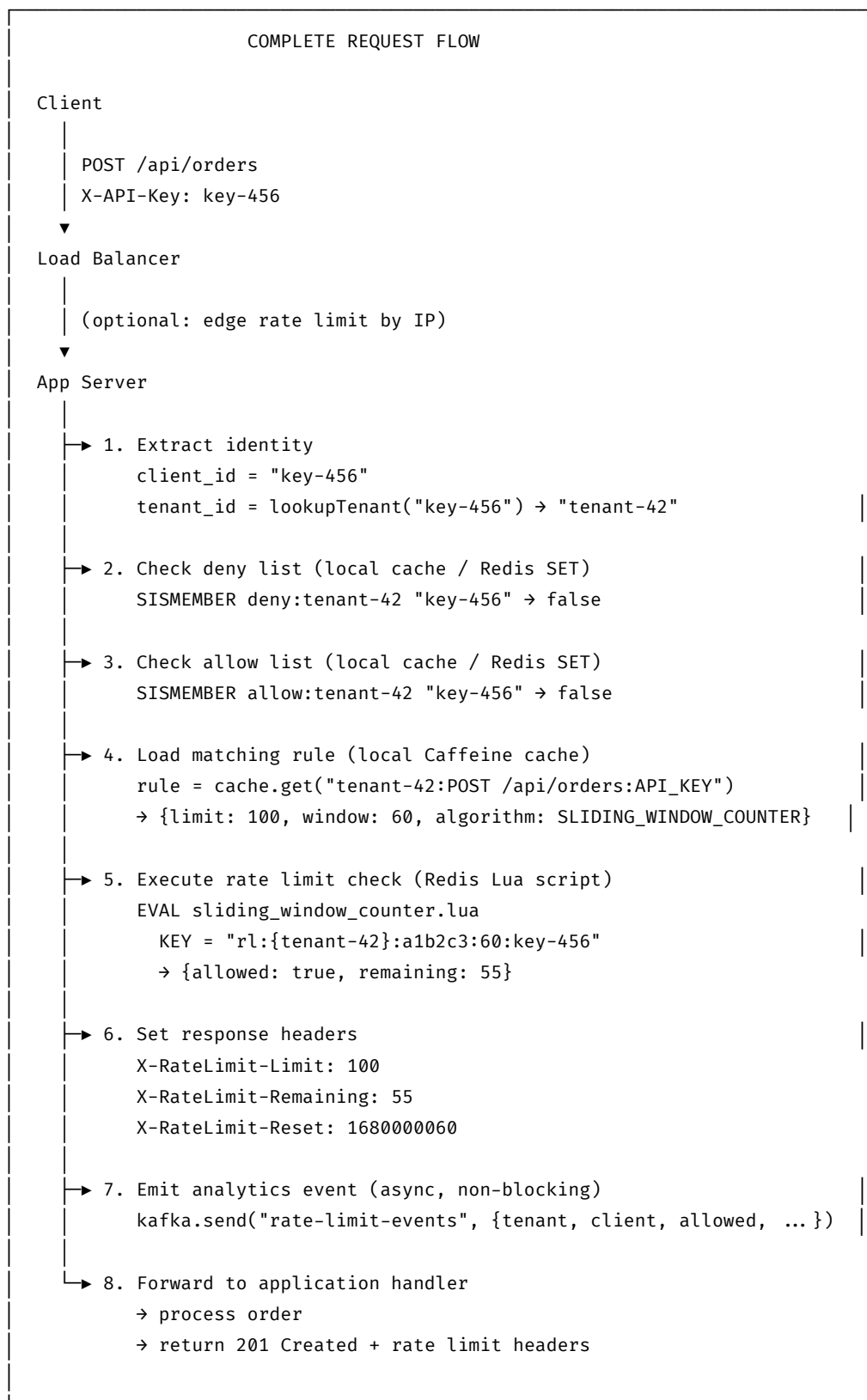
2.16.15 Q15: How would you migrate from a local rate limiter to a distributed one?

Answer: Phased migration:

1. **Phase 1 - Dual write:** Deploy Redis-based rate limiter alongside existing local limiter. Both make decisions, but only the local limiter's decision is enforced. Log discrepancies.
2. **Phase 2 - Shadow enforcement:** Switch enforcement to the distributed limiter for 5% of traffic. Monitor for correctness and latency impact.
3. **Phase 3 - Gradual rollout:** Increase to 25%, 50%, 100% over 2 weeks. Keep local limiter as fallback (circuit breaker).
4. **Phase 4 - Cleanup:** Remove local limiter code once distributed limiter has been stable for 30 days.

Key concern: During migration, both limiters run simultaneously. A request that passes the local limiter but fails the distributed one (or vice versa) reveals a discrepancy. Track these as a metric and investigate before increasing rollout.

2.17 Appendix A: Complete Request Flow



2.18 Appendix B: Production Configuration Example

```
# rate-limiter-config.yaml
rate_limiter:
  # Global settings
  default_algorithm: SLIDING_WINDOW_COUNTER
  default_fail_mode: OPEN
  local_cache_ttl_seconds: 30
  local_cache_max_size: 10000

  # Redis connection
  redis:
    cluster_nodes:
      - redis-shard-1.internal:6379
      - redis-shard-2.internal:6379
      - redis-shard-3.internal:6379
    connection_pool_size: 50
    connection_timeout_ms: 100
    read_timeout_ms: 50
    write_timeout_ms: 50
    max_retries: 2

  # Circuit breaker
  circuit_breaker:
    failure_threshold: 5
    failure_window_seconds: 10
    open_duration_seconds: 30
    half_open_probes: 1

  # Local fallback (when Redis is down)
  local_fallback:
    enabled: true
    server_count: 20 # used to divide global limit
    cleanup_interval_seconds: 60
```

```
# Analytics
analytics:
  enabled: true
  kafka_topic: rate-limit-events
  sampling_rate: 0.01 # 1% of allowed requests
  denied_sampling_rate: 1.0 # 100% of denied requests

# Default rules (overridden by tenant-specific rules)
default_rules:
  - endpoint: "*"
    identifier_type: IP
    limit: 10000
    window_seconds: 60
    algorithm: FIXED_WINDOW
    fail_mode: OPEN

  - endpoint: "POST /api/auth/login"
    identifier_type: IP
    limit: 5
    window_seconds: 60
    algorithm: SLIDING_WINDOW_COUNTER
    fail_mode: CLOSED # security: fail-closed for login
```

2.19 Appendix C: Quick Reference Card

RATE LIMITER DESIGN - QUICK REFERENCE	
DEFAULT ALGORITHM:	Sliding Window Counter
COUNTER STORE:	Redis Cluster (50+ shards for 10M rps)
RULE STORE:	PostgreSQL + local Caffeine cache
FAIL MODE:	Fail-open (default), configurable
LATENCY:	< 1ms p99 overhead per request
ACCURACY:	~99.7% (sliding window approximation)
MULTI-DC:	Local Redis + async cross-DC sync
HOT KEY HANDLING:	Local counter + periodic Redis sync
KEY FORMULA:	rl:{tenant}:{ep_hash}:{window}:{client}
TTL:	window_seconds + 10s buffer
RULE PROPAGATION:	Redis Pub/Sub + 30s cache TTL fallback
CRITICAL DECISIONS:	
1. Sliding window counter over token bucket (simpler)	
2. Redis over local-only (accuracy over latency)	
3. Fail-open over fail-closed (availability over safety)	
4. Async analytics over sync logging (performance)	
5. Lua scripts over multi-command (atomicity)	

Chapter 3

Design a Chat Messaging System (WhatsApp / Messenger)

Difficulty: Hard | **Category:** Real-Time Communication | **Time:** 45 min

Companies: Meta, Google, Microsoft, Uber, Slack, Discord, LinkedIn, Telegram

3.1 1. Problem Statement and Scope

Design a real-time chat messaging system similar to WhatsApp or Facebook Messenger that supports billions of users with low-latency message delivery, reliable storage, and rich communication features including media sharing and group chats.

3.1.1 In Scope

- 1-on-1 private messaging with real-time delivery
- Group messaging (up to 256 members per group)
- Message delivery status tracking (sent, delivered, read)
- Online/offline presence indicators
- Push notifications for offline users
- Message history retrieval and search
- Media sharing (images, videos, documents)
- End-to-end encryption (E2E)
- Multi-device synchronization

3.1.2 Out of Scope

- Voice and video calling (separate system)
 - Stories / Status features
 - Payment integration
 - Bot platform and business API
 - Sticker / GIF marketplace
-

3.2 2. Functional Requirements

3.2.1 Core Messaging

- **1 - Send and receive 1-on-1 text messages in real-time:** Priority: P0
- **2 - Group chat with up to 256 members:** Priority: P0
- **3 - Message ordering within a conversation:** Priority: P0
- **4 - Delivery receipts: sent / delivered / read:** Priority: P0
- **5 - Push notifications for offline users:** Priority: P0
- **6 - Message history and scroll-back:** Priority: P0
- **7 - Media sharing (images, video, files up to 100MB):** Priority: P1
- **8 - Online / offline / last-seen presence:** Priority: P1
- **9 - End-to-end encryption:** Priority: P1
- **10 - Multi-device sync (phone, tablet, web):** Priority: P1
- **11 - Message search across conversations:** Priority: P2
- **12 - Message deletion (for me / for everyone):** Priority: P2
- **13 - Typing indicators:** Priority: P2
- **14 - Message reactions (emoji):** Priority: P3

3.2.2 User Stories

1. **Send Message:** User A types a message and sends it to User B. User B receives it instantly if online, or via push notification and offline queue if not.
 2. **Group Chat:** User creates a group, adds members. Each message sent is delivered to all active members in real-time. Offline members get notifications and catch up later.
 3. **Read Receipts:** When User B opens the chat and views the message, User A sees a “read” indicator with a timestamp.
 4. **Media Share:** User selects a photo, the client uploads it to the media service, gets a URL, and sends a message referencing the media URL.
-

3.3 3. Non-Functional Requirements

- **Latency:** < 100ms message delivery (same region); Rationale: Real-time conversation feel
- **Availability:** 99.99% (52 min downtime/year); Rationale: Critical communication service
- **Durability:** 99.9999% (no message loss); Rationale: Users expect zero data loss
- **Throughput:** 230K+ messages/sec sustained; Rationale: 500M DAU, 40 msgs/user/day
- **Consistency:** Causal ordering within conversation; Rationale: Messages must appear in order
- **Presence:** Eventual consistency (< 10s lag); Rationale: Acceptable for online status
- **Scalability:** Support 2B registered, 500M DAU; Rationale: WhatsApp-scale
- **Message size:** Up to 64KB text, 100MB media; Rationale: Support rich content
- **Connection scale:** 500M concurrent WebSocket connections; Rationale: One per active user
- **Security:** End-to-end encryption for all messages; Rationale: Privacy requirement

3.3.1 SLA Breakdown

Message Delivery Latency (p50/p95/p99):

Same region: 30ms / 80ms / 150ms
Cross-region: 80ms / 200ms / 500ms

Availability per component:

WebSocket Gateway: 99.99%
Chat Service: 99.99%
Message Store: 99.999%
Push Notifications: 99.9%

3.4 4. Back-of-Envelope Estimation

3.4.1 Traffic Estimates

Registered users: 2 Billion
Daily Active Users (DAU): 500 Million
Avg messages sent/day: 40 per user

Daily messages: $500M \times 40 = 20$ Billion messages/day
Messages per second: $20B / 86400 = \sim 230,000$ msg/sec
Peak (3x average): $\sim 700,000$ msg/sec

Group messages: 30% of total = 6B/day
Avg group size: 20 members
Fan-out messages: $6B \times 20 = 120B$ fan-out deliveries/day
Fan-out per second: $120B / 86400 = \sim 1.4M$ deliveries/sec

3.4.2 Storage Estimates

Average message size: 1 KB (text + metadata)
Daily message storage: $20B \times 1KB = 20$ TB/day
Yearly message storage: $20TB \times 365 = 7.3$ PB/year
With replication (3x): ~ 22 PB/year

Media messages: 5% of messages = 1B/day
Average media size: 200 KB (thumbnails) + 2 MB (original)
Daily media storage: $1B \times 2.2MB = 2.2$ PB/day
Yearly media storage: ~ 800 PB/year (with CDN offloading)

3.4.3 Connection Estimates

Concurrent connections: 500 Million WebSocket connections
Per connection memory: ~ 10 KB (buffers, session state)
Total connection memory: $500M \times 10KB = 5$ TB

Connection servers needed: $500M / 100K$ connections each = 5,000 servers
(each server: 64GB RAM, 100K concurrent connections)

Heartbeat traffic: 500M heartbeats every 30s = 16.7M/sec

3.4.4 Bandwidth Estimates

Inbound messages: $230\text{K/sec} \times 1\text{KB} = 230\text{ MB/sec} = 1.84\text{ Gbps}$
 Fan-out outbound: $1.4\text{M/sec} \times 1\text{KB} = 1.4\text{ GB/sec} = 11.2\text{ Gbps}$
 Media uploads: $1\text{B/day} \times 2\text{MB} / 86400 = 23\text{ GB/sec} = 184\text{ Gbps}$
 Media downloads: $3\text{x uploads (views)} = \sim 550\text{ Gbps (served from CDN)}$

3.4.5 Summary Table

- **Messages:** Daily: 20 Billion; Yearly: 7.3T
- **Message storage:** Daily: 20 TB; Yearly: 7.3 PB
- **Media storage:** Daily: 2.2 PB; Yearly: 800 PB
- **Fan-out deliveries:** Daily: 120 Billion; Yearly: 43.8T
- **WebSocket connections:** Daily: 500M peak; Yearly: -
- **Connection servers:** Daily: 5,000; Yearly: -

3.5 5. API Design

3.5.1 5.1 WebSocket Protocol (Real-Time)

The primary communication channel uses WebSocket with a custom binary protocol for efficiency. Messages are framed with a type byte followed by protobuf payload.

WebSocket Frame Format:

Type (1B)	Len (4B)	Protobuf Payload (variable)
-----------	----------	-----------------------------

Message Types:

0×01 = SEND_MESSAGE
 0×02 = ACK_MESSAGE
 0×03 = DELIVER_MESSAGE
 0×04 = READ_RECEIPT
 0×05 = TYPING_INDICATOR
 0×06 = PRESENCE_UPDATE
 0×07 = HEARTBEAT / PONG

3.5.1.1 sendMessage

```
Client -> Server (0x01 SEND_MESSAGE):
{
  "client_msg_id": "uuid-v4",           // Client-generated for dedup
  "conversation_id": "conv_abc123",
  "sender_id": "user_123",
  "recipient_id": "user_456",          // null for group messages
  "group_id": "grp_789",               // null for 1-on-1
  "content": {
    "type": "TEXT | IMAGE | VIDEO | FILE",
    "text": "Hello!",
    "media_url": null,
    "thumbnail_url": null,
    "encryption_key": "base64_key"
  },
  "reply_to_msg_id": null,
  "timestamp": 1680000000000
}
```

```
Server -> Client (0x02 ACK_MESSAGE):
{
  "client_msg_id": "uuid-v4",
  "server_msg_id": "msg_snowflake_id",
  "server_timestamp": 1680000000005,
  "status": "SENT"
}
```

3.5.1.2 deliverMessage

```
Server -> Recipient (0x03 DELIVER_MESSAGE):
{
  "server_msg_id": "msg_snowflake_id",
  "conversation_id": "conv_abc123",
  "sender_id": "user_123",
  "content": { ... },
  "server_timestamp": 1680000000005,
  "sequence_number": 42                // Per-conversation ordering
}
```

```
Recipient -> Server (0x02 ACK_MESSAGE):
{
  "server_msg_id": "msg_snowflake_id",
  "status": "DELIVERED",
  "timestamp": 16800000000050
}
```

3.5.1.3 readReceipt

```
Client -> Server (0x04 READ_RECEIPT):
{
  "conversation_id": "conv_abc123",
  "last_read_msg_id": "msg_snowflake_id",
  "last_read_seq": 42,
  "timestamp": 1680000000100
}
```

3.5.2 5.2 REST APIs

3.5.2.1 Group Management

```
POST /api/v1/groups
{
  "name": "Engineering Team",
  "members": ["user_123", "user_456", "user_789"],
  "avatar_url": "https://media.example.com/ ..."
}
Response: 201 Created
{
  "group_id": "grp_abc",
  "conversation_id": "conv_grp_abc",
  "created_at": "2024-01-01T00:00:00Z"
}

PUT /api/v1/groups/{group_id}/members
{
  "action": "ADD | REMOVE",
  "user_ids": ["user_999"]
}

GET /api/v1/groups/{group_id}
Response: { group metadata, member list, admin list }
```

3.5.2.2 Message History

```
GET /api/v1/conversations/{conv_id}/messages
  ?before_seq=100
  &limit=50
  &direction=BACKWARD

Response: 200 OK
{
  "messages": [ ... ],
  "has_more": true,
  "oldest_seq": 51
}
```


3.5.2.3 Media Upload

```
POST /api/v1/media/upload
Content-Type: multipart/form-data
```

Fields:

```
file: <binary>
type: "IMAGE | VIDEO | FILE"
conversation_id: "conv_abc123"
```

Response: 200 OK

```
{
  "media_id": "media_xyz",
  "url": "https://media-cdn.example.com/ ... ",
  "thumbnail_url": "https://media-cdn.example.com/ ... /thumb",
  "encryption_key": "base64_key",
  "size_bytes": 2048000,
  "expires_at": null
}
```

3.5.2.4 Presence

```
PUT /api/v1/users/{user_id}/presence
{
  "status": "ONLINE | OFFLINE | AWAY",
  "last_seen": "2024-01-01T12:00:00Z"
}
```

GET /api/v1/users/{user_id}/presence

Response: { "status": "ONLINE", "last_seen": " ... " }

3.6 6. Data Model and Database Selection

3.6.1 6.1 Database Selection Rationale

Data	Database	Rationale
Messages	Apache Cassandra	Write-heavy, time-series, horizontal scaling, tunable consistency
Users, Groups	MySQL (Vitess)	Strong consistency, relational data, complex queries, ACID transactions
Sessions/Presence	Redis Cluster	In-memory, sub-ms reads, TTL-based expiry for sessions
Message Queue	Apache Kafka	Ordered, durable, high-throughput message delivery pipeline
Media/Files	S3 + CloudFront	Blob storage with CDN for global distribution
Search Index	Elasticsearch	Full-text search across messages (async indexing via Kafka)

3.6.2 6.2 Messages Table (Cassandra)

```
-- Primary message store: partitioned by conversation for efficient retrieval
CREATE TABLE messages (
  conversation_id  UUID,
  sequence_number  BIGINT,           -- Monotonically increasing per conversation
  message_id       UUID,           -- Globally unique (Snowflake ID)
  sender_id        UUID,
  content_type      TEXT,           -- TEXT, IMAGE, VIDEO, FILE
  content_text      TEXT,           -- Encrypted text body
  media_url         TEXT,
  thumbnail_url     TEXT,
  encryption_key    BLOB,
  reply_to_msg_id   UUID,
  created_at        TIMESTAMP,
  deleted_at        TIMESTAMP,      -- Soft delete
  PRIMARY KEY ((conversation_id), sequence_number)
) WITH CLUSTERING ORDER BY (sequence_number DESC);

-- Partition key: conversation_id
-- Clustering key: sequence_number DESC (newest first for scroll-back)

-- Per-user inbox: tracks conversations and unread counts
CREATE TABLE user_conversations (
  user_id          UUID,
  last_activity     TIMESTAMP,
  conversation_id   UUID,
  last_msg_seq      BIGINT,
  last_read_seq     BIGINT,
  unread_count      INT,
  is_muted          BOOLEAN,
  PRIMARY KEY ((user_id), last_activity, conversation_id)
) WITH CLUSTERING ORDER BY (last_activity DESC, conversation_id ASC);

-- Offline message queue: messages pending delivery
CREATE TABLE pending_messages (
  recipient_id      UUID,
  message_id        UUID,
  conversation_id    UUID,
  sender_id         UUID,
  content            BLOB,          -- Serialized message
  created_at        TIMESTAMP,
  retry_count        INT,
  PRIMARY KEY ((recipient_id), created_at, message_id)
) WITH CLUSTERING ORDER BY (created_at ASC);

-- Messages delivered oldest-first when user comes online
```

3.6.3 6.3 Users and Groups (MySQL via Vitess)

```
-- Users table: sharded by user_id
CREATE TABLE users (
  user_id          BIGINT PRIMARY KEY AUTO_INCREMENT,
  phone_number     VARCHAR(20) UNIQUE NOT NULL,
  display_name     VARCHAR(100),
  avatar_url       VARCHAR(500),
  public_key       BLOB,           -- For E2E encryption key exchange
  status_text      VARCHAR(200),
  created_at       DATETIME NOT NULL,
  updated_at       DATETIME NOT NULL,
  INDEX idx_phone (phone_number)
);

-- Groups table
CREATE TABLE groups (
  group_id         BIGINT PRIMARY KEY AUTO_INCREMENT,
  conversation_id  BIGINT UNIQUE NOT NULL,
  name             VARCHAR(100),
  description      VARCHAR(500),
  avatar_url       VARCHAR(500),
  max_members      INT DEFAULT 256,
  created_by       BIGINT NOT NULL,
  created_at       DATETIME NOT NULL,
  updated_at       DATETIME NOT NULL,
  FOREIGN KEY (created_by) REFERENCES users(user_id)
);

-- Group membership: sharded by group_id
CREATE TABLE group_members (
  group_id         BIGINT,
  user_id          BIGINT,
  role            ENUM('ADMIN', 'MEMBER') DEFAULT 'MEMBER',
  joined_at       DATETIME NOT NULL,
  PRIMARY KEY (group_id, user_id),
  INDEX idx_user_groups (user_id)
);

-- Conversations: maps 1-on-1 and groups to conversation_id
CREATE TABLE conversations (
  conversation_id  BIGINT PRIMARY KEY AUTO_INCREMENT,
  type            ENUM('DIRECT', 'GROUP') NOT NULL,
  created_at      DATETIME NOT NULL
);

-- Direct conversation participants
CREATE TABLE conversation_participants (
  conversation_id  BIGINT,
  user_id         BIGINT,
  PRIMARY KEY (conversation_id, user_id),
  INDEX idx_user_convs (user_id, conversation_id)
);
```

3.6.4 6.4 Session and Presence (Redis)

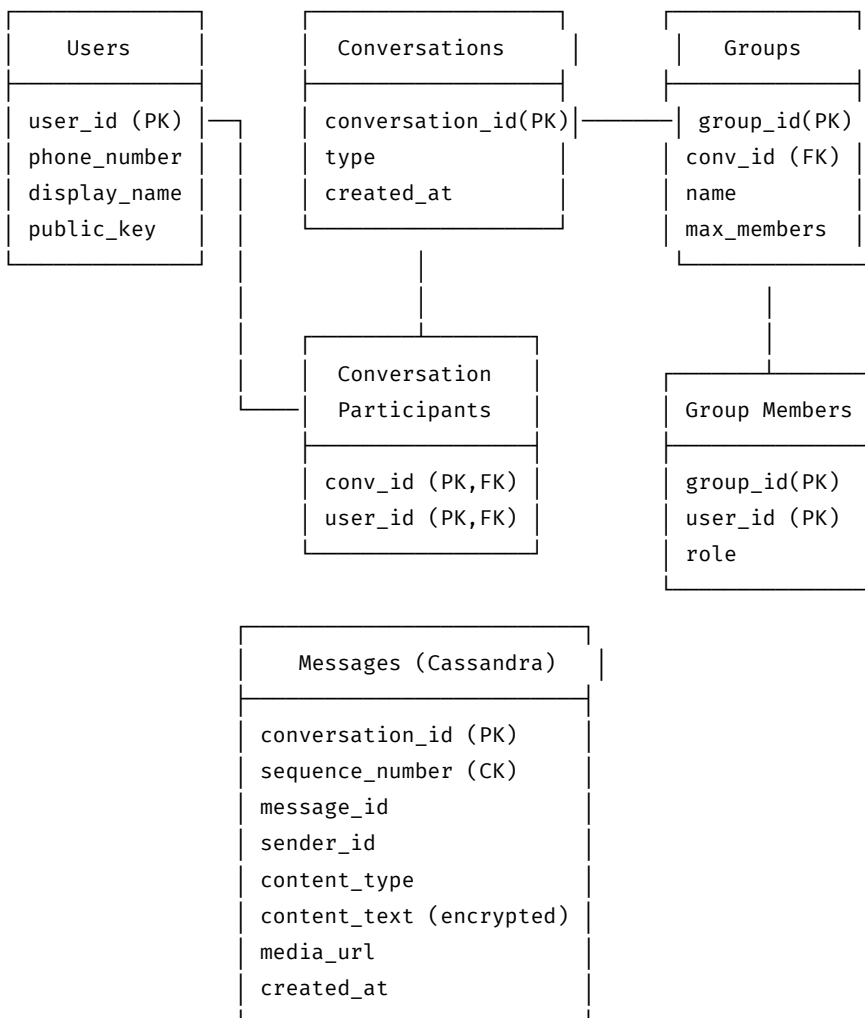
```
# User session: maps user to their WebSocket connection server
SET user:session:{user_id} -> {
    "server_id": "ws-server-042",
    "connection_id": "conn_abc123",
    "device_type": "MOBILE",
    "connected_at": 1680000000000
} EX 300    # 5-minute TTL, refreshed by heartbeat

# Multi-device: set of active sessions
SADD user:devices:{user_id} -> ["device_1", "device_2"]

# Presence
SET user:presence:{user_id} -> {
    "status": "ONLINE",
    "last_seen": 1680000000000
} EX 60    # 60-second TTL, refreshed by heartbeat

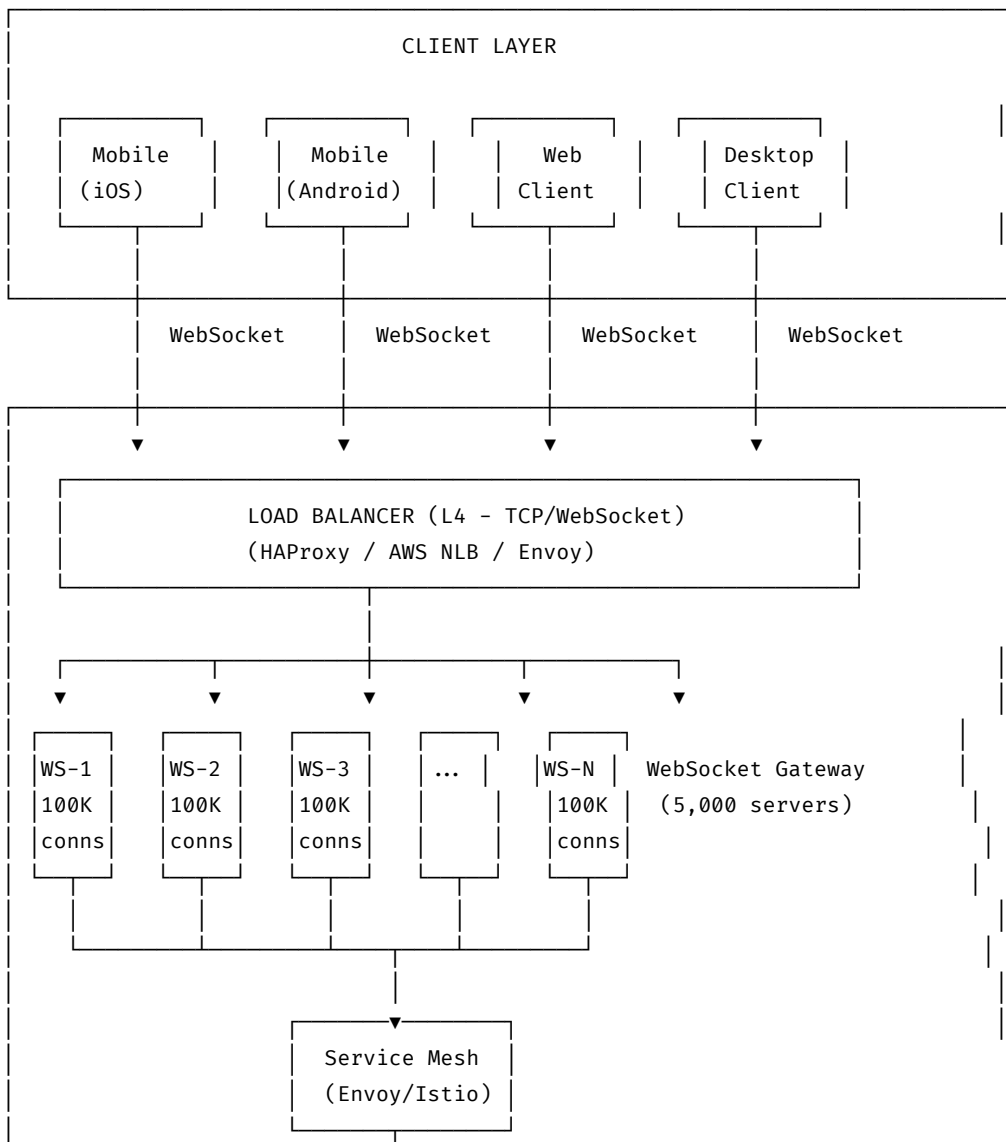
# Conversation recent cache (last message for conversation list)
HSET conv:last_msg:{user_id} {conversation_id} -> {serialized_last_msg}
```

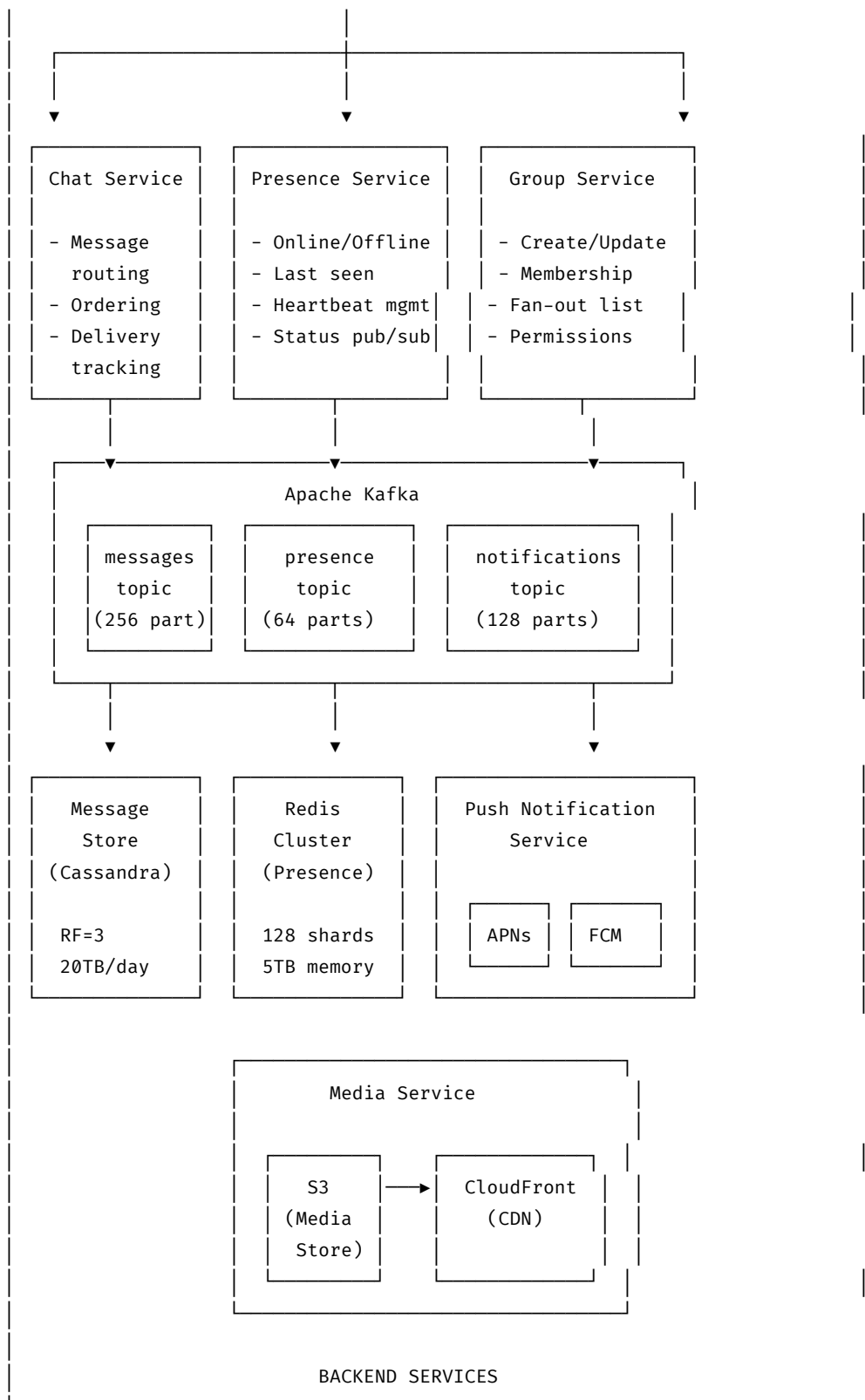
3.6.5 6.5 Entity Relationship Diagram



3.7 7. High-Level Architecture

3.7.1 7.0 System Overview Diagram





3.7.2 7.1 Component Breakdown

3.7.2.1 WebSocket Gateway (Connection Servers)

- **Purpose:** Maintain persistent WebSocket connections with clients
- **Scale:** 5,000 servers, each handling ~100K concurrent connections

- **Responsibilities:**
 - Accept and authenticate WebSocket connections
 - Route incoming messages to the Chat Service
 - Push outgoing messages to connected clients
 - Manage heartbeat and connection lifecycle
 - TLS termination and protocol handling

3.7.2.2 Chat Service

- **Purpose:** Core message processing, routing, and delivery logic
- **Responsibilities:**
 - Validate and process incoming messages
 - Assign server-side message IDs (Snowflake) and sequence numbers
 - Route messages to recipient connection servers
 - Persist messages to Cassandra
 - Track delivery status (sent, delivered, read)
 - Handle message deduplication using `client_msg_id`

3.7.2.3 Presence Service

- **Purpose:** Track and distribute user online/offline status
- **Responsibilities:**
 - Process heartbeats from connection servers
 - Update user presence in Redis with TTL
 - Publish presence changes to interested subscribers
 - Optimize fan-out for users with many contacts

3.7.2.4 Group Service

- **Purpose:** Manage group metadata and membership
- **Responsibilities:**
 - CRUD operations for groups
 - Member management (add, remove, promote)
 - Provide fan-out list for group messages
 - Enforce group policies (max members, admin permissions)

3.7.2.5 Push Notification Service

- **Purpose:** Deliver notifications to offline users
- **Responsibilities:**
 - Consume from notification topic in Kafka
 - Format and send via APNs (iOS) and FCM (Android)
 - Manage device tokens and notification preferences
 - Rate limiting and notification batching

3.7.2.6 Media Service

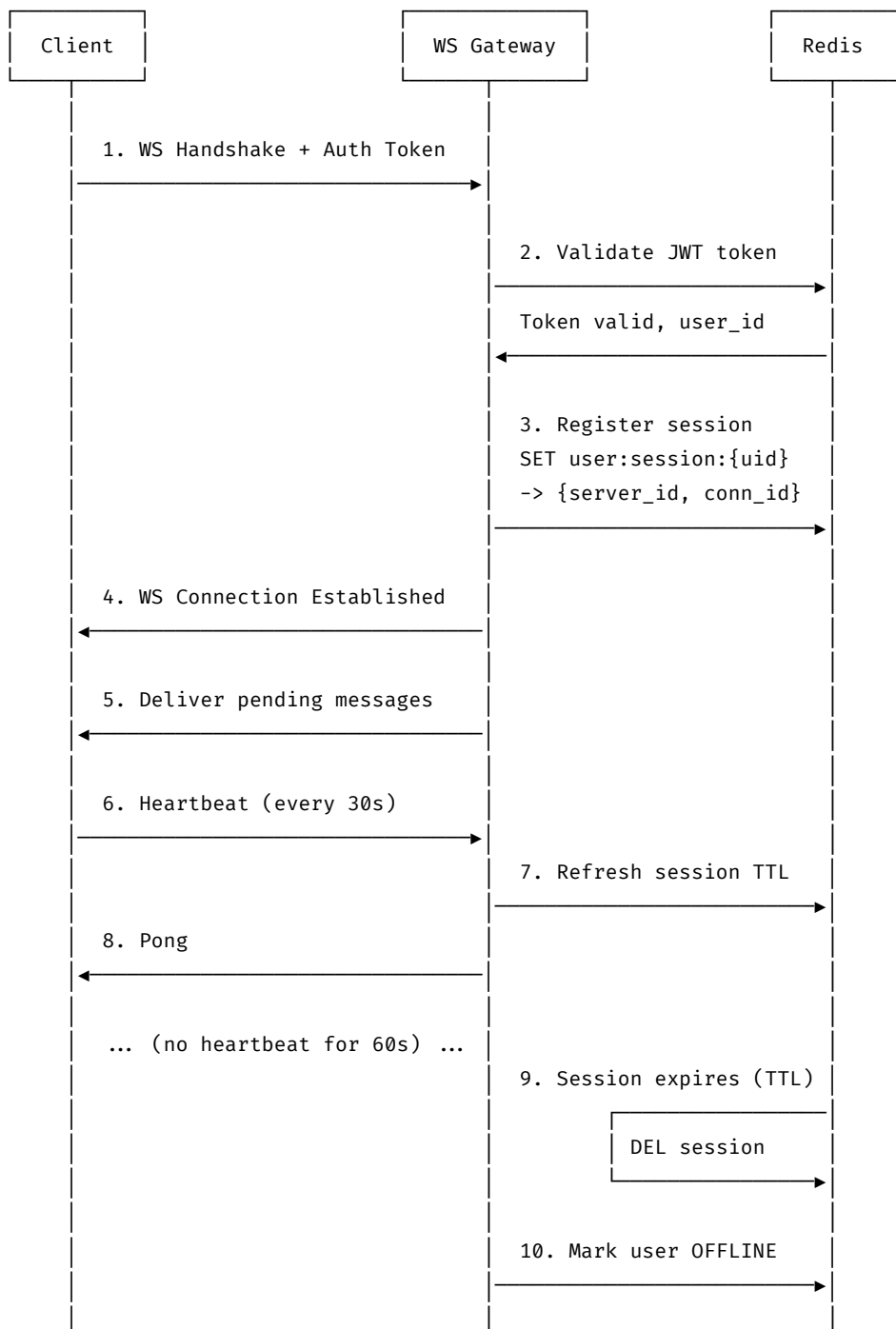
- **Purpose:** Handle file uploads and media processing
- **Responsibilities:**
 - Accept media uploads via HTTP multipart
 - Generate thumbnails and compressed versions
 - Store originals in S3, serve via CDN

- Client-side encryption key management

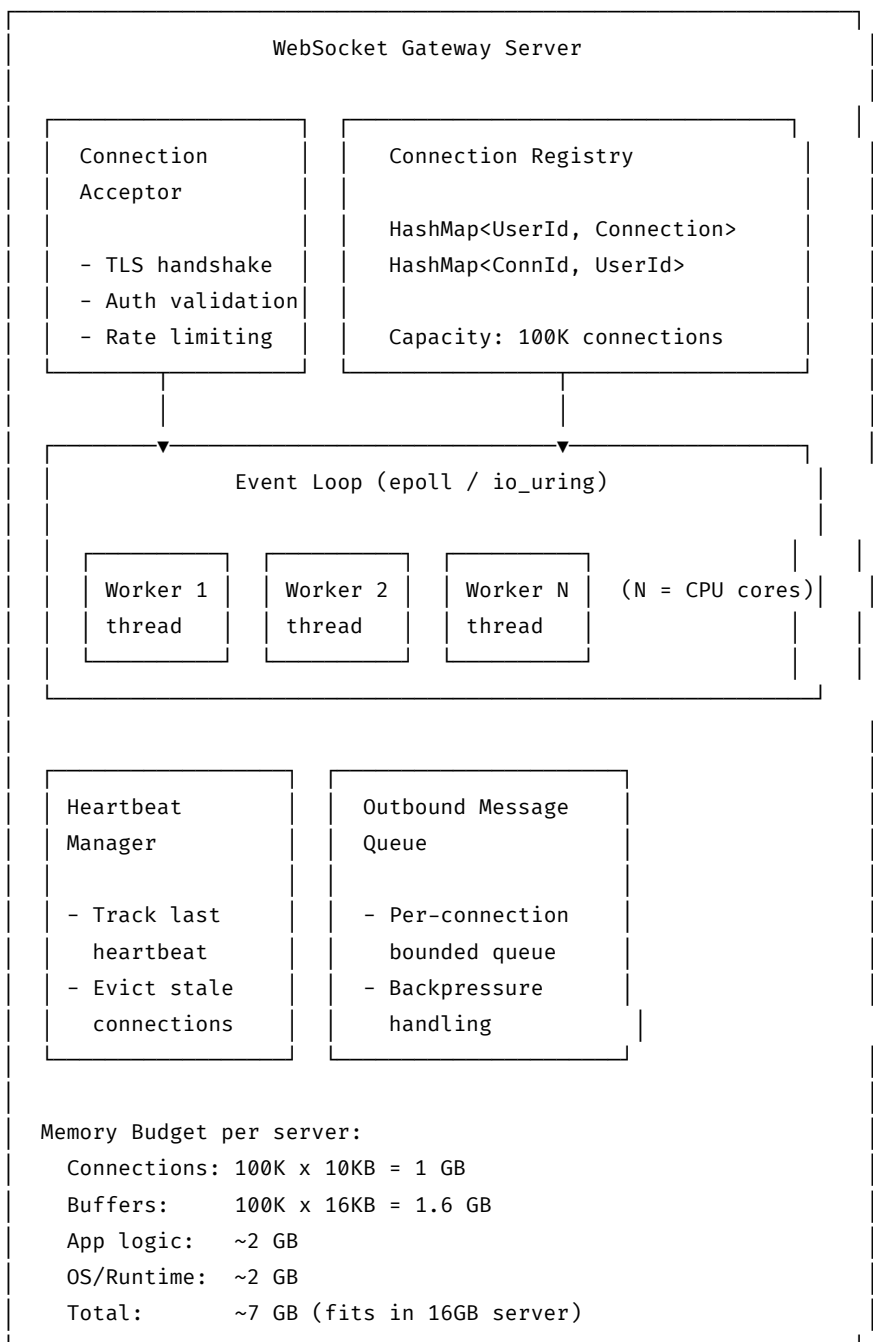
3.8 8. Deep Dive: Core Components

3.8.1 8.1 WebSocket Connection Management

3.8.1.1 Connection Lifecycle



3.8.1.2 Connection Server Internal Architecture



3.8.1.3 Session Management for Multi-Device

When a user has multiple devices (phone, tablet, web), each device maintains its own WebSocket connection. The session registry tracks all active sessions:

Redis Multi-Device Session:

HSET user:sessions:user_123

"device_mobile_ios" -> {"server": "ws-042", "conn": "c1", "ts": 168 ... }

"device_tablet_ipad" -> {"server": "ws-107", "conn": "c2", "ts": 168 ... }

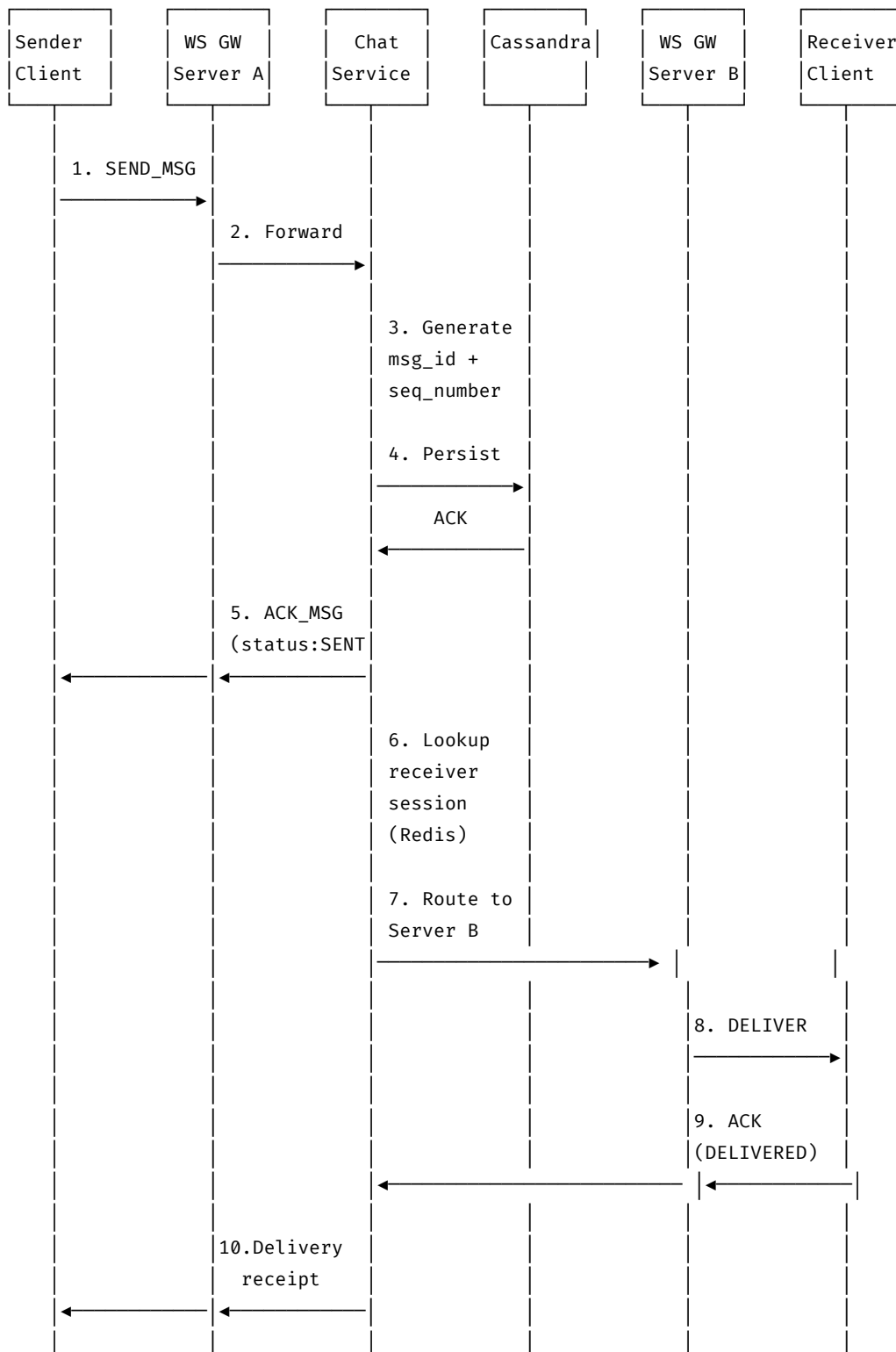
"device_web_chrome" -> {"server": "ws-233", "conn": "c3", "ts": 168 ... }

Message delivery: fan-out to ALL active devices

Read receipt sync: propagate across devices via sync protocol

3.8.2 8.2 Message Delivery and Ordering

3.8.2.1 1-on-1 Message Flow



Key steps explained:

1. Sender transmits message via WebSocket with a client-generated `client_msg_id`.
2. WS Gateway forwards to Chat Service (via gRPC or internal queue).

3. Chat Service generates a globally unique `msg_id` (Snowflake) and assigns a monotonically increasing `sequence_number` for the conversation.
4. Message is persisted to Cassandra (write quorum: `LOCAL_QUORUM = 2` of 3 replicas).
5. ACK with status `SENT` returned to sender – message is now durable.
6. Look up receiver's active session in Redis.
7. If receiver is online: route to their connection server.
8. Connection server pushes message to receiver's WebSocket.
9. Receiver ACKs delivery – status changes from `SENT` to `DELIVERED`.
10. Delivery receipt pushed back to sender.

3.8.2.2 Sequence Number Generation

Each conversation maintains an atomic counter for message ordering:

Sequence Number Service

Approach: Redis `INCR` per conversation

`INCR conv:seq:{conversation_id}` -> next sequence number

For high-throughput conversations (large groups):

- Pre-allocate ranges: `INCRBY conv:seq:{cid} 100`
- Chat Service caches range `[n, n+100)` locally
- Assign from local range without Redis round-trip

Alternative: Hybrid Logical Clocks (HLC) for cross-datacenter ordering without centralized counter

Ordering guarantee:

- Within a conversation: total order via `sequence_number`
- Across conversations: no ordering guarantee needed
- Ties broken by `(sequence_number, msg_id)`

3.8.2.3 At-Least-Once Delivery and Deduplication

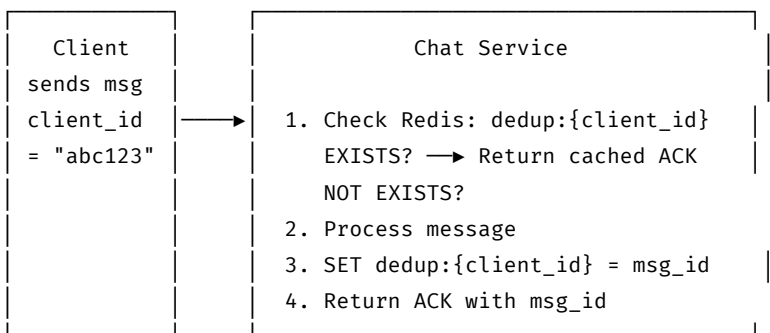
Sender-side retry:

1. Client sends message with `client_msg_id` (UUID)
2. If no ACK within 5 seconds, retry (exponential backoff)
3. Server deduplicates using `client_msg_id`
Redis SET: `dedup:{client_msg_id} -> msg_id EX 3600` (1-hour TTL)

Server-side retry:

1. After persisting message, attempt delivery
2. If recipient offline or delivery fails, enqueue to `pending_messages`
3. Delivery consumer retries with exponential backoff
4. Recipient deduplicates using server `msg_id`

Dedup flow:



3.8.2.4 Offline Message Handling

When recipient is OFFLINE:

1. Chat Service persists message to Cassandra
2. ACK SENT to sender
3. Lookup recipient session in Redis -> NOT FOUND
4. Enqueue to `pending_messages` table (Cassandra)
5. Publish to Kafka notifications topic
6. Push Notification Service sends APNs/FCM

When recipient comes ONLINE:

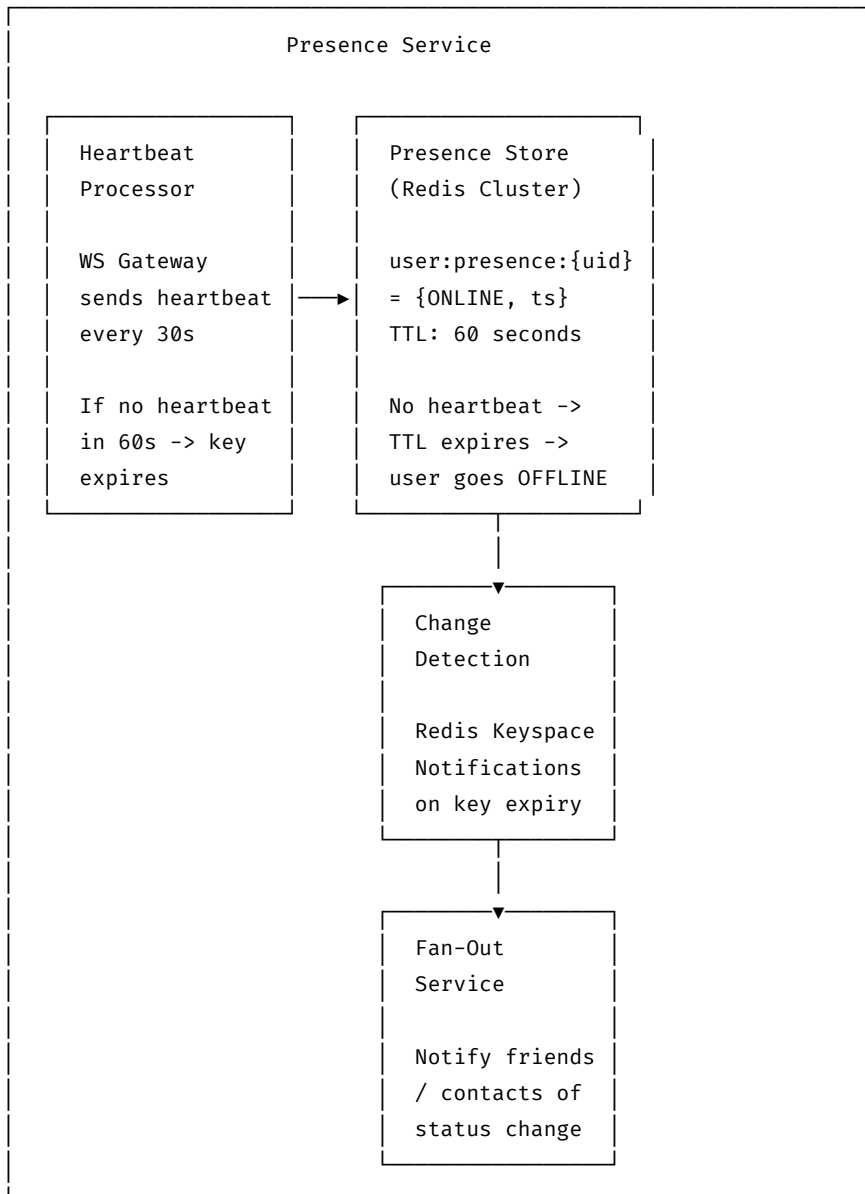
1. WS Gateway registers session in Redis
2. Chat Service queries `pending_messages` for user
3. Deliver messages in `sequence_number` order
4. Recipient ACKs each -> update status to DELIVERED
5. Delete from `pending_messages`
6. Fan out delivery receipts to senders

Optimization: batch delivery of pending messages

- Fetch up to 100 pending messages per batch
- Single WebSocket frame with multiple messages
- Single batch ACK from client

3.8.3 8.3 Presence Service

3.8.3.1 Architecture



3.8.3.2 Presence Fan-Out Optimization

The naive approach of notifying all contacts when a user comes online creates a fan-out storm for popular users. Optimizations:

Problem: User with 1000 friends comes online

Naive: push ONLINE to all 1000 friends = 1000 messages instantly

At scale: 500M users x avg 200 friends = 100B presence events/day

Optimization 1: Lazy Presence (Pull-based)

- Don't push presence changes proactively
- When User A opens chat with User B, query B's presence
- Cache locally for 30 seconds
- Reduces presence traffic by ~95%

Optimization 2: Active-Contact Subscription

- Only subscribe to presence of users you've chatted with recently
- Presence Service maintains subscription list per user
- Fan-out only to subscribed users (typically 10-20, not 200+)

Subscription Model

User A opens chat with User B:

SADD presence:subs:B -> A

(A subscribes to B's presence)

User A closes chat / goes idle (5min):

SREM presence:subs:B -> A

(A unsubscribes from B's presence)

B comes online:

SMEMBERS presence:subs:B

-> notify only active subscribers

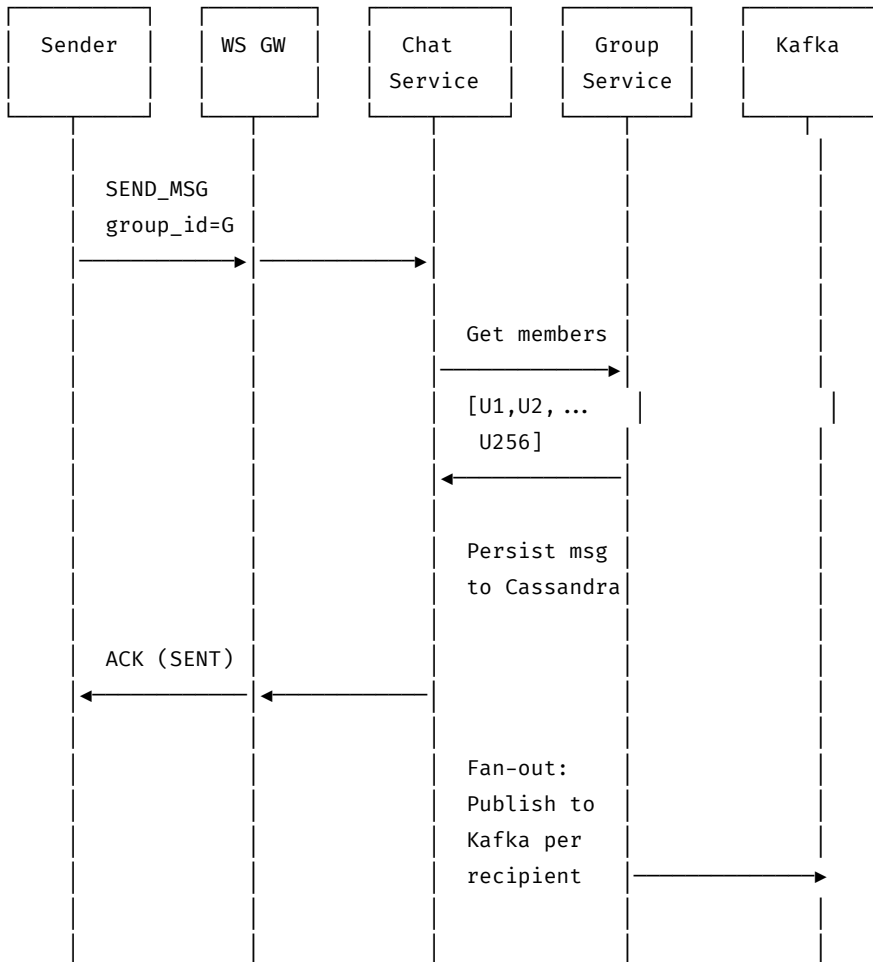
(typically 5-10 users, not 200+)

Optimization 3: Presence Batching

- Batch presence updates every 5 seconds
- Aggregate ONLINE/OFFLINE transitions
- If user toggles rapidly (flaky connection), only send final state

3.8.4 8.4 Group Messaging

3.8.4.1 Group Message Delivery Flow



Kafka Fan-Out Detail:

Group G has 256 members.
Message M sent to group G.

Option A: Write-time fan-out (small groups, < 256 members)

- Chat Service writes 256 delivery tasks to Kafka
- Each task: {msg_id, recipient_id, conversation_id}
- Delivery workers consume and push to each recipient's WS Gateway
- Pro: simple, low read amplification
- Con: 256x write amplification per message

Option B: Read-time fan-out (large broadcast channels)

- Store message once in group timeline
- Each user reads from group timeline on demand
- Pro: single write, efficient for large groups
- Con: read amplification, harder real-time delivery

WhatsApp approach: Write-time fan-out (groups max 256)

Facebook Messenger: Hybrid (small groups: write-time, pages: read-time)

3.8.4.2 Group Message Ordering

Challenge: Ensuring all group members see messages in the same order

Solution: Single-writer sequencing per group

Each group has a single sequence counter:

Redis: INCR group:seq:{group_id}

All messages for a group are serialized through a single partition in Kafka:

Kafka partition key = group_id

-> All messages for a group go to same partition

-> Partition guarantees ordering

Sequence assignment:

1. Chat Service receives group message
2. INCR group:seq:{group_id} -> seq = 42
3. Message stored with seq=42
4. All members receive message with seq=42
5. Clients insert message at correct position based on sequence_number

Handling gaps (missed seq):

Client detects gap: has seq 40, receives seq 42

-> Request message for seq 41 from server

-> Hold seq 42 in buffer until gap filled

3.8.4.3 Group Membership Management

Operations and their effects:

ADD MEMBER:

1. Update group_members table (MySQL)
2. Invalidate cached member list in Redis
3. Send system message to group: "User X was added"
4. New member receives conversation history (last 100 messages)

REMOVE MEMBER:

1. Update group_members table
2. Invalidate cache
3. System message: "User X was removed"
4. Removed user still has local copy (E2E: rotate group encryption key)

ADMIN PROMOTION:

1. Update role in group_members
2. System message: "User X is now an admin"

Consistency concern:

- Member list cached in Redis with 5-minute TTL
 - Cache invalidated on membership changes
 - Stale cache risk: message might be delivered to removed member
(acceptable: E2E encryption key rotation prevents decryption)
-

3.9 9. Data Partitioning and Sharding

3.9.1 9.1 Message Data (Cassandra)

Cassandra Partitioning Strategy

Partition Key: conversation_id

Clustering Key: sequence_number DESC

Why conversation_id?

- All messages in a conversation are co-located
- Efficient scroll-back queries (pagination by seq range)
- Natural time-series data within a conversation

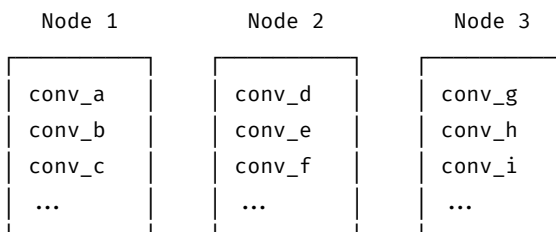
Why NOT user_id?

- User has many conversations -> hot partition for active users
- Group messages would be duplicated per user
- conversation_id distributes more evenly

Partition sizing:

- Avg conversation: 1000 messages x 1KB = 1MB per partition
- Active conversation: 100K msgs x 1KB = 100MB (well within Cassandra's ~100MB recommended partition limit)

Ring distribution example (12-node cluster):



Replication: NetworkTopologyStrategy

DC-East: 3 replicas

DC-West: 3 replicas

Read consistency: LOCAL_QUORUM (2 of 3)

Write consistency: LOCAL_QUORUM (2 of 3)

Compaction: TimeWindowCompactionStrategy (TWCS)

- Optimized for time-series append-only data
- 1-day time windows
- Old SSTables compacted into larger files

TTL: Messages older than 1 year auto-expire (configurable)

3.9.2 9.2 User Data (MySQL via Vitess)

Vitess Sharding Strategy

Shard key: `user_id` (hash-based sharding)

Number of shards: 256 (expandable via Vitess resharding)

Shard 0: `user_id % 256 = 0`

Shard 1: `user_id % 256 = 1`

...

Shard 255: `user_id % 256 = 255`

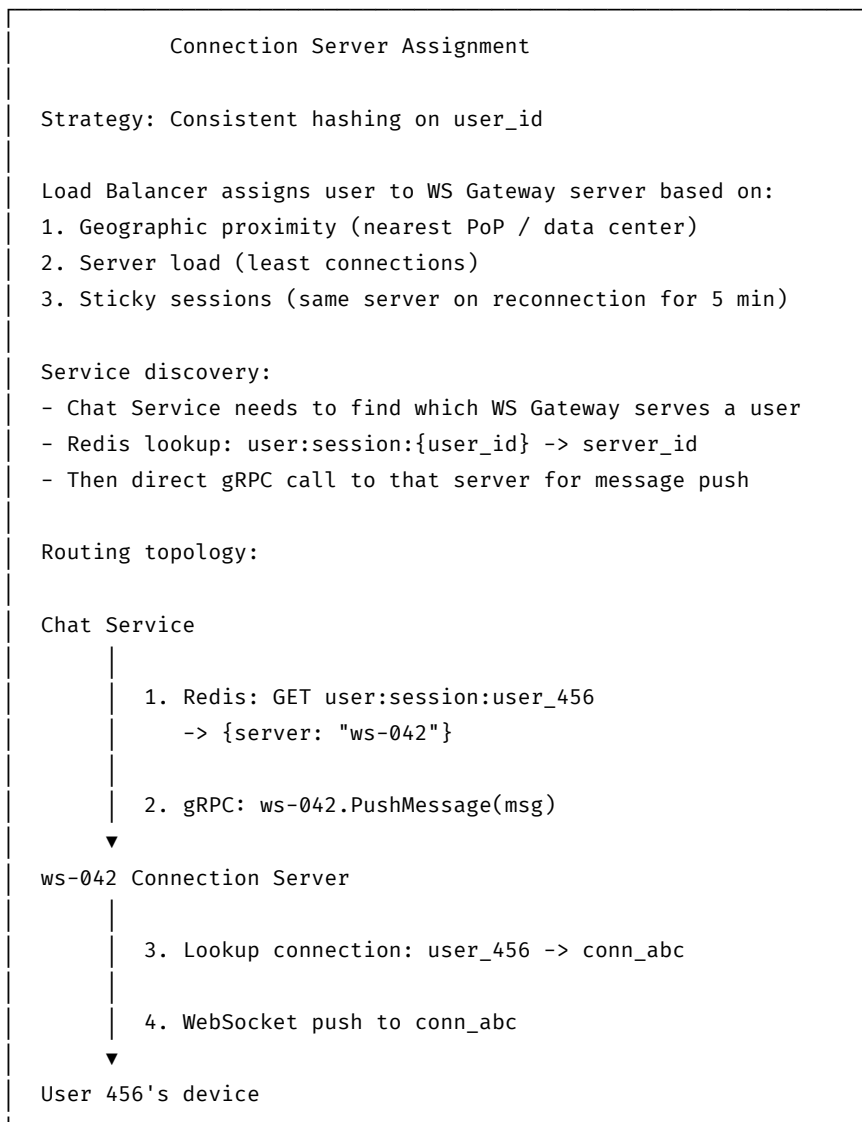
Cross-shard queries (e.g., group members from different shards):

- Handled by Vitess VTGate with scatter-gather
- `Group_members` table co-sharded by `group_id` (separate keyspace) for efficient member lookups

Each shard: MySQL primary + 2 replicas

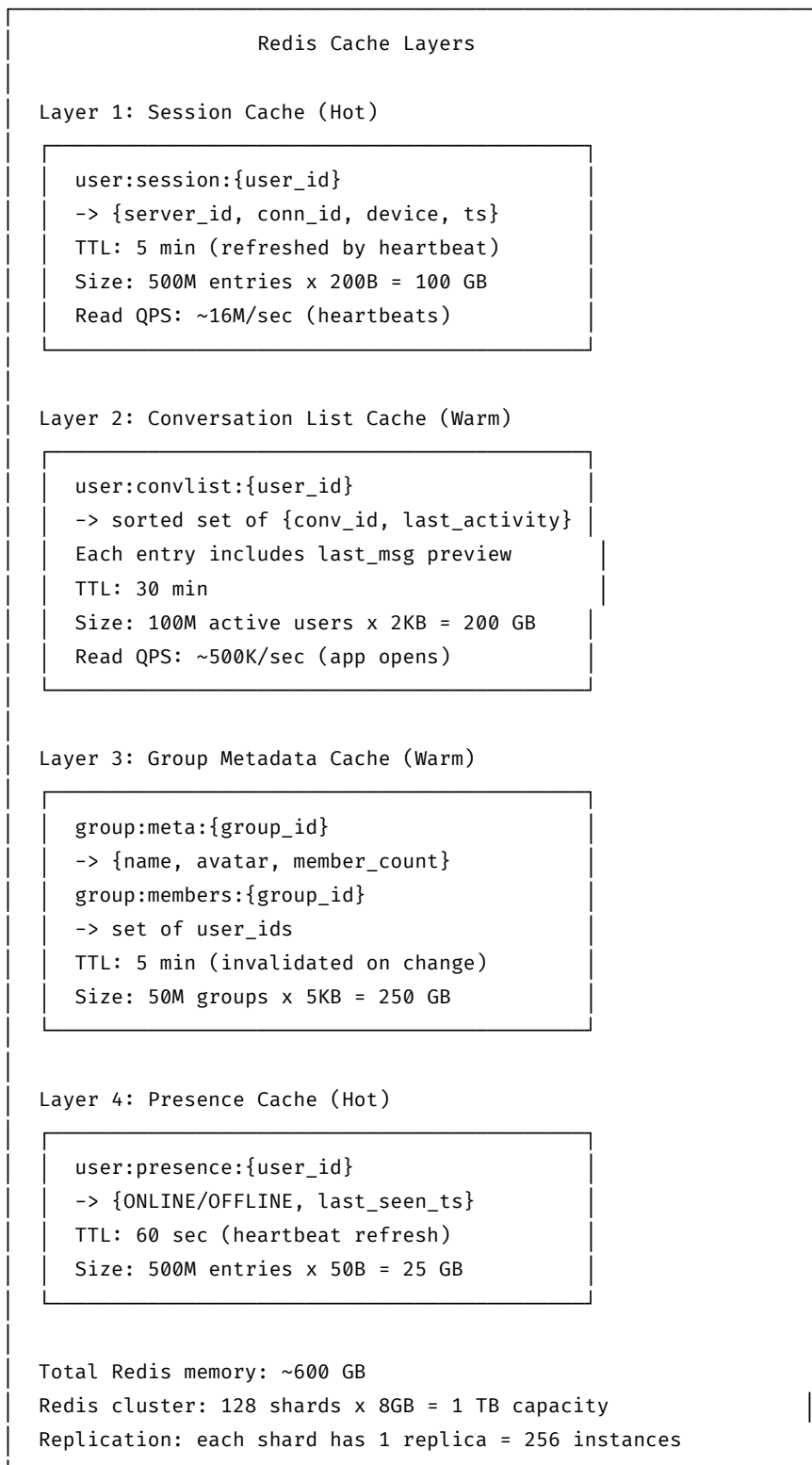
Total: $256 \times 3 = 768$ MySQL instances

3.9.3 9.3 Connection Server Routing



3.10 10. Caching Strategy

3.10.1 10.1 Cache Architecture



3.10.2 10.2 What We Do NOT Cache

Message content:

- Too much volume: 20B messages/day x 1KB = 20TB/day
- Low cache hit rate: messages are read once and rarely revisited
- Cassandra with SSD storage provides ~5ms reads for recent messages
- SSTable caching in Cassandra serves as an implicit cache

User profile data (full):

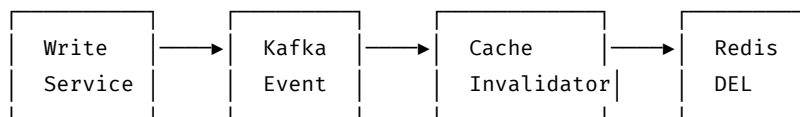
- Profile photo URLs and display names change rarely
- Client-side caching with ETag/If-None-Match is sufficient
- CDN caches avatars

Message search results:

- Highly personalized, low reuse potential
- Elasticsearch maintains its own internal caches

3.10.3 10.3 Cache Invalidation Strategy

Strategy: Event-driven invalidation via Kafka



Events triggering invalidation:

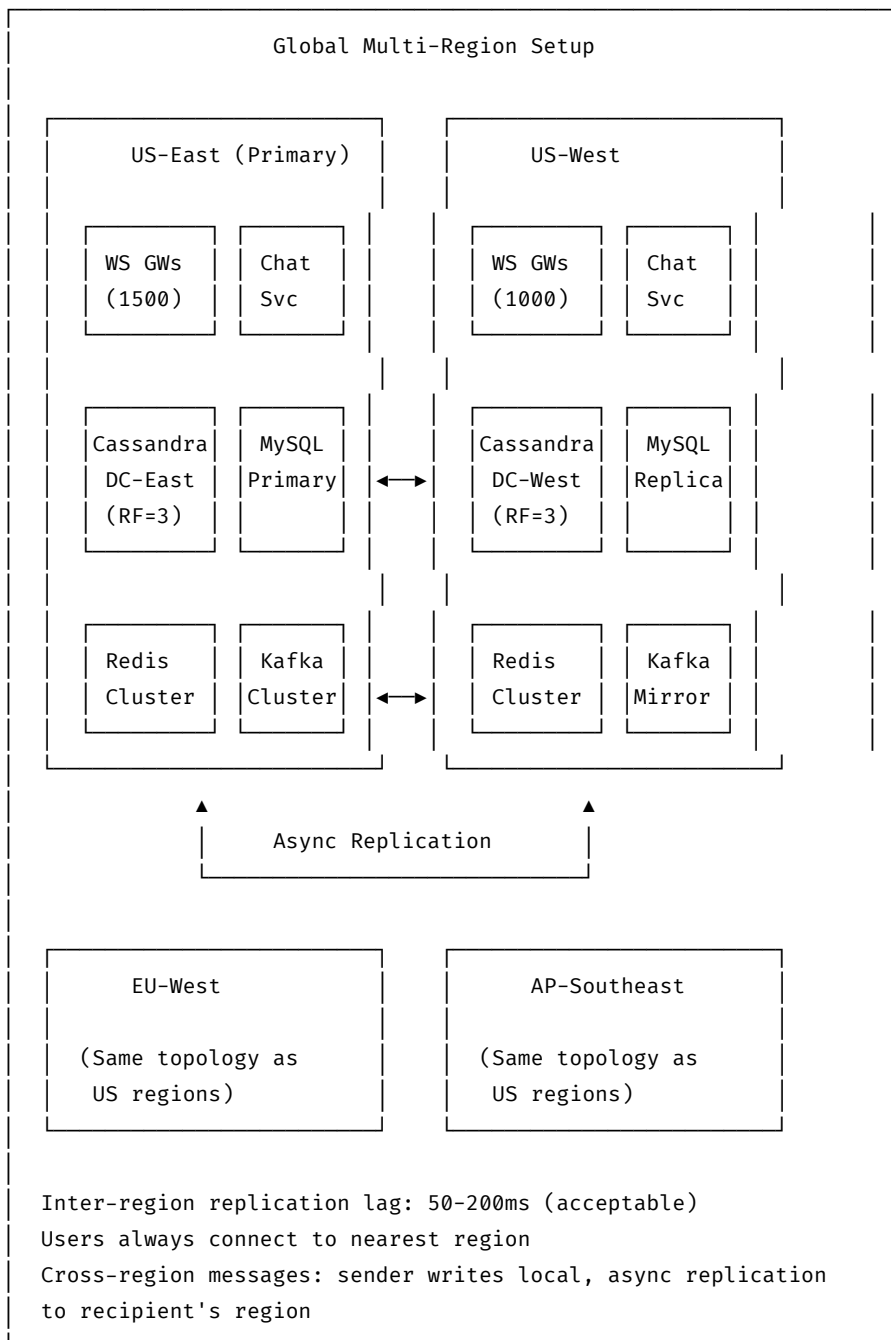
- New message: Update user:convlist:{uid} (add conv to top)
- Group change: DEL group:meta:{gid}, group:members:{gid}
- Profile update: DEL user:profile:{uid}
- Message delete: Update user:convlist if last_msg affected

Consistency guarantee:

- Cache is eventually consistent (writes to DB first, then invalidate)
- Stale cache window: typically < 1 second
- For session data: immediate write-through (critical for routing)

3.11 11. Replication and Consistency

3.11.1 11.1 Multi-Region Architecture

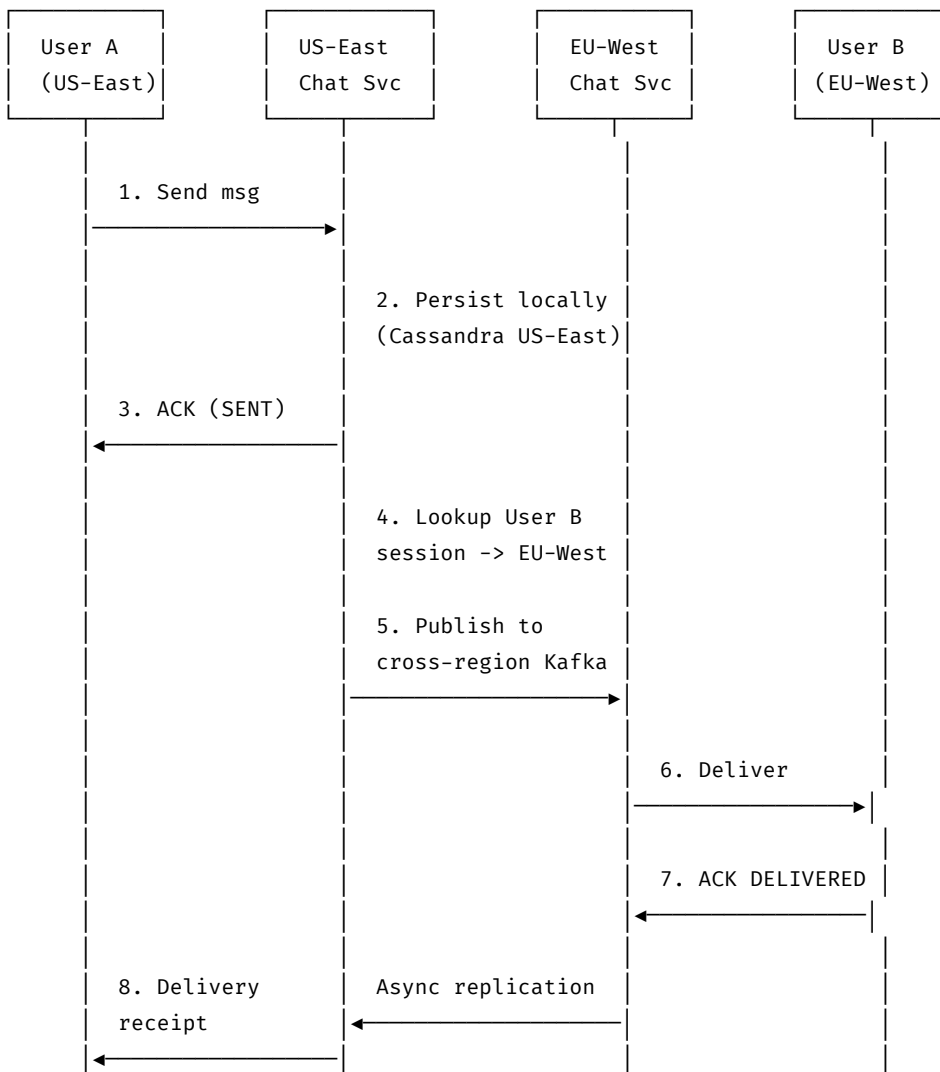


3.11.2 11.2 Consistency Model per Data Type

Data Type	Consistency Level	Rationale
Messages	Causal consistency within conversation	Messages must appear in correct order within a chat but not across chats
Delivery status	Eventual consistency	DELIVERED/READ receipts can have seconds-level lag
Presence	Eventual consistency	5-10 second lag acceptable; users tolerate "last seen" being slightly stale
User profiles	Eventual consistency	Name/avatar changes can take minutes to propagate
Group membership	Strong within DC, eventual across DC	Critical for message routing but brief staleness OK
User sessions	Strong per DC (Redis)	Must be accurate for message routing; stale = missed msg

3.11.3 11.3 Cross-Region Message Delivery

Scenario: User A (US-East) sends message to User B (EU-West)



Latency: Steps 1-3 (~30ms local), Steps 4-7 (~150ms cross-region)

Total: ~180ms end-to-end for cross-region delivery

3.12 12. Fault Tolerance and Failure Handling

3.12.1 12.1 Failure Scenarios and Mitigations

Failure Scenario	Detection	Mitigation
WS Gateway crashes	Heartbeat timeout (60s)	Client auto-reconnects to different server via load balancer. Pending messages delivered from offline queue.
Chat Service instance goes down	Health checks (5s interval)	Load balancer routes to healthy instances. Kafka retains unprocessed messages.
Cassandra node failure	Gossip protocol	RF=3: other 2 replicas serve reads/writes. Hinted handoff on recovery.
Redis shard failure	Sentinel / Cluster monitoring	Automatic failover to replica. Sessions re-registered on next heartbeat.
Kafka broker failure	ZooKeeper / KRaft	Partition leadership transferred. ISR replicas continue.
Entire DC failure	Cross-DC health monitoring	DNS failover to other DC. Users reconnect. Cassandra serves from remaining DCs.
Network partition between data centers	Split-brain detection	Each DC continues independently. Messages reconciled after partition heals via Cassandra anti-entropy.

3.12.2 12.2 Client-Side Reliability

Client Reconnection Protocol

1. Connection drops detected (no pong for 60s)
2. Client enters reconnection mode:
 - Attempt 1: immediate retry
 - Attempt 2: 1 second delay
 - Attempt 3: 2 seconds delay
 - Attempt 4: 4 seconds delay
 - ...
 - Max delay: 30 seconds (with jitter)
3. On reconnection:
 - Re-authenticate with JWT
 - Send last_received_seq per conversation
 - Server sends missed messages (gap fill)
4. While disconnected:
 - Messages queued locally (SQLite on device)
 - Sent in order on reconnection
 - Each has client_msg_id for dedup
5. Idempotency guarantee:
 - Client retries with same client_msg_id
 - Server deduplicates: if already processed, returns cached ACK without re-processing

3.12.3 12.3 Message Delivery Guarantee

Guarantee: At-least-once delivery with client-side dedup

End-to-end flow with failure handling:

Sender -> Server:

- ✓ Client retries until ACK received (at-least-once send)
- ✓ Server deduplicates via `client_msg_id` (exactly-once processing)

Server -> Recipient:

- ✓ If online: push via WebSocket, wait for DELIVERED ACK
- ✓ If ACK not received: retry from Kafka consumer
- ✓ If offline: queue in `pending_messages` table
- ✓ On reconnection: deliver all pending messages
- ✓ Recipient deduplicates via `server msg_id` (exactly-once display)

Failure during persist:

- ✓ Write to Cassandra with LOCAL_QUORUM
- ✓ If write fails: do NOT ACK to sender -> sender retries
- ✓ If ACK to sender fails after write: sender retries, server deduplicates and returns cached ACK

Result: Messages are never lost (durability)

Messages are never duplicated (dedup at both ends)

Messages may be delivered out of order briefly
(client reorders by `sequence_number`)

3.13 13. Scalability

3.13.1 13.1 Scaling Strategy per Component

Horizontal Scaling Plan			
Component	Current	2x Scale	Scaling Method
WS Gateways	5,000	10,000	Add servers, LB distributes
Chat Service	500	1,000	Stateless, add pods behind LB
Cassandra	100 nodes (12 racks)	200 nodes	Add nodes, auto rebalance ring
MySQL (Vitess)	256 shards	512 shards	Vitess reshard (online)
Redis	128 shards	256 shards	Redis Cluster resharding
Kafka	30 brokers 256 parts	60 brokers 512 parts	Add brokers, rebalance parts
Push Notification	50 workers	100 workers	Add consumers
Media Service	100 pods	200 pods	Stateless, S3 scales auto

3.13.2 13.2 Bottleneck Analysis

Bottleneck 1: WebSocket connections

Problem: 500M concurrent connections need 5,000 servers

Solution: Connection pooling, binary protocol, OS tuning
(net.core.somaxconn, ulimit -n 1000000)

Growth: Linear scaling -- add servers as users grow

Bottleneck 2: Group message fan-out

Problem: Large group (256 members) = 256 Kafka messages per send

Solution: Batch fan-out, Kafka partitioning by recipient
Pre-compute fan-out list, cache in Redis

Bottleneck 3: Sequence number generation

Problem: Redis INCR for per-conversation seq is centralized

Solution: Range pre-allocation (INCRBY 100)
Local caching of ranges in Chat Service
At 230K msg/sec, Redis handles this comfortably
(Redis can do ~1M ops/sec per shard)

Bottleneck 4: Cross-region latency

Problem: User A in US, User B in EU = 150ms+ network latency

Solution: Write locally, async replication
Accept 150-200ms delivery latency for cross-region
Local read for message history

Bottleneck 5: Media upload throughput

Problem: 1B media uploads/day = ~12K uploads/sec

Solution: Direct-to-S3 upload with pre-signed URLs
Client uploads directly, bypassing our servers
Server only stores the media reference

3.13.3 13.3 Connection Server Scaling Detail

Connection Server Auto-Scaling

Scaling triggers:

- Connection count > 80K per server -> scale up
- Connection count < 30K per server -> scale down
- CPU > 70% or Memory > 80% -> scale up

Graceful scaling down:

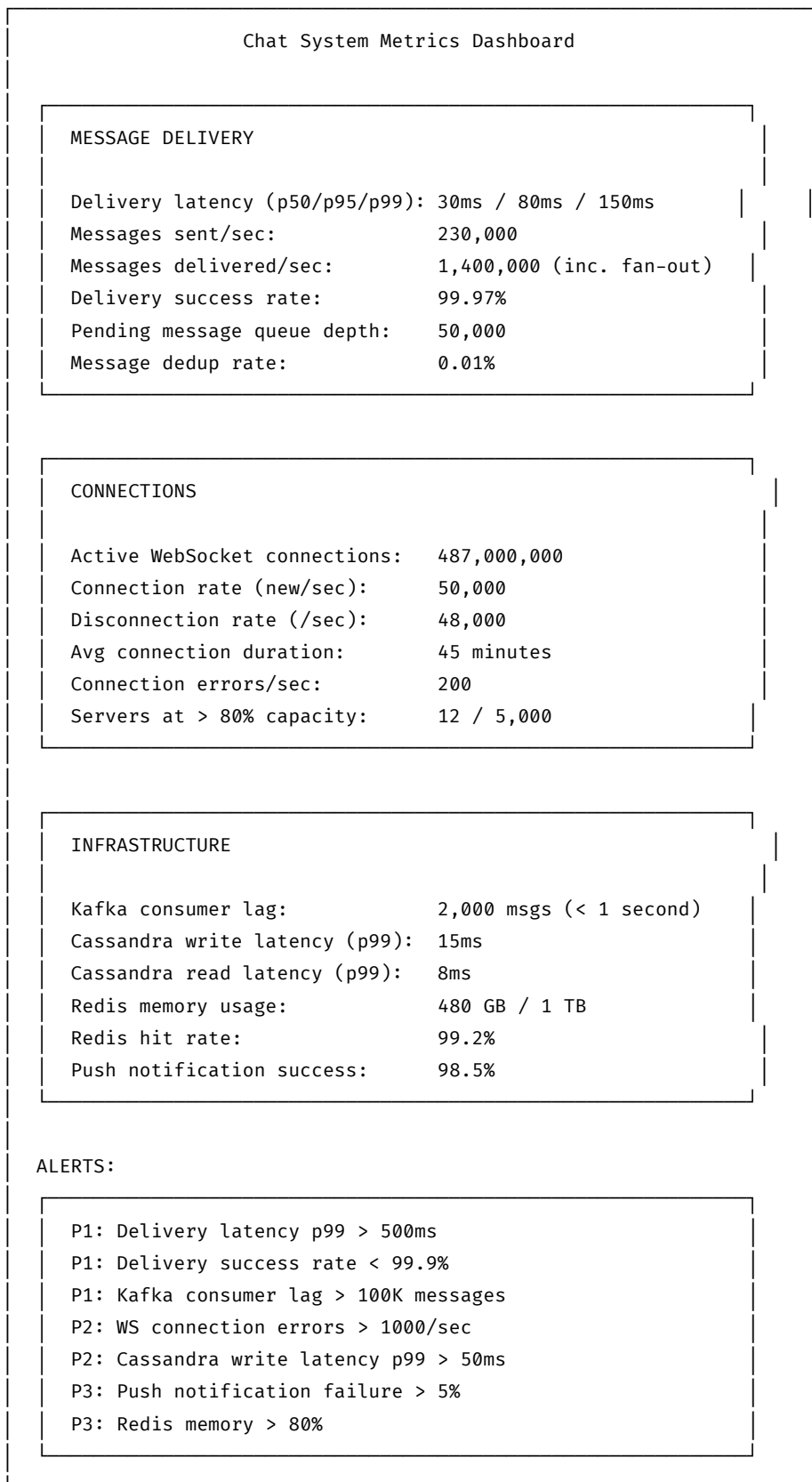
1. Mark server as "draining" in service registry
2. Stop accepting new connections
3. Wait for existing connections to naturally disconnect
(heartbeat timeout: 60 seconds)
4. After 5 minutes: forcefully close remaining connections
5. Clients auto-reconnect to other servers
6. Decommission server

Rolling deployment:

1. Deploy to 1% of servers (canary)
2. Monitor error rates, latency for 15 minutes
3. If healthy: roll to 10%, then 50%, then 100%
4. If unhealthy: rollback canary, investigate
5. Connections gracefully migrate during rollout

3.14 14. Monitoring and Observability

3.14.1 14.1 Key Metrics Dashboard



3.14.2 14.2 Distributed Tracing

Trace: Send Message (end-to-end)

TraceID: abc-123-def-456

```

┌ Client: sendMessage [0ms]
├─ WS Gateway: receiveFrame [2ms]
│   ┌ Auth: validateToken [1ms]
│   └─ Forward: toChatService (gRPC) [3ms]
│       ┌ ChatService: dedup check (Redis) [1ms]
│       ┌ ChatService: assignSeqNum (Redis INCR) [1ms]
│       ┌ ChatService: persist (Cassandra) [8ms]
│       ┌ ChatService: ACK to sender [1ms] ← SENT
│       ┌ ChatService: lookupSession (Redis) [1ms]
│       ┌ ChatService: route to WS Gateway B (gRPC) [5ms]
│       │   ┌ WS Gateway B: pushToClient [2ms]
│       │   │   └─ Client B: ACK (DELIVERED) [3ms]
│       └─ ChatService: update delivery status [2ms]
└─ Total: 28ms (same region)

Spans breakdown:
Network hops:    4 x 2ms = 8ms
Redis operations: 3 x 1ms = 3ms
Cassandra write: 1 x 8ms = 8ms
Processing:      ~9ms
Total:           ~28ms (p50 target: 30ms) ✓

```

3.14.3 14.3 Health Check Strategy

Component health checks:

WS Gateway:

- /health/live -> process is running (Kubernetes liveness)
- /health/ready -> can accept connections (readiness)
- /health/deep -> Redis connectivity, can register sessions

Chat Service:

- /health/live -> process running
- /health/ready -> Kafka producer connected, Cassandra reachable
- Dependency check: Redis, Cassandra, Kafka all reachable

Presence Service:

- /health/live -> process running
- /health/ready -> Redis cluster connected

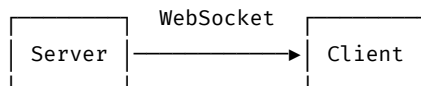
Circuit Breaker Pattern:

- Each service wraps downstream calls in circuit breakers
- Thresholds: 50% failure rate over 30-second window
- Open state: fail fast, return cached/default response
- Half-open: allow 10% traffic to probe recovery
- Closed: normal operation

3.15 15. Trade-offs and Design Decisions

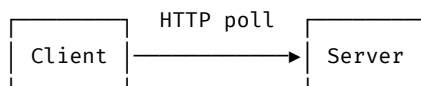
3.15.1 15.1 Push vs Pull for Message Delivery

PUSH Model (chosen):



- + Real-time delivery (< 100ms)
- + No unnecessary network requests
- + Server controls delivery rate
- Requires persistent connections (resource-intensive)
- Complex connection management at scale

PULL Model (rejected):



- + Simpler server architecture
- + Stateless servers
- High latency (polling interval)
- Wasted bandwidth (empty polls)
- At 500M users polling every 1s = 500M req/sec (unsustainable)

HYBRID (what we actually do):

- Push via WebSocket when user is online (primary)
- Pull via REST when loading message history (on-demand)
- Push notification (APNs/FCM) when user is offline

Decision: Push for real-time, Pull for historical data

3.15.2 15.2 WebSocket vs Long Polling vs SSE

Criterion	WebSocket	Long Polling	SSE
Latency	~1ms	~100ms	~1ms
Direction	Bidirectional	Client->Srvr	Server->Clnt
Overhead	2B per frame	HTTP headers	HTTP headers
Connections	Persistent	Repeated	Persistent
Mobile battery	Good (w/ heartbeat)	Battery-intensive	One-way only
Proxy/FW	Some issues	Works always	Works mostly
Binary data	Yes	Yes	No (text)
Verdict	CHOSEN ✓	Fallback	Not suitable

Decision: WebSocket primary with long-polling fallback for restrictive network environments (corporate proxies).

3.15.3 15.3 Cassandra vs HBase vs MongoDB

Criterion	Cassandra	HBase	MongoDB
Write perf	Excellent	Good	Good
Read perf	Good (by PK)	Good (by RK)	Flexible
Scalability	Linear	Linear	Complex
Multi-DC	Native	Complex	Supported
Consistency	Tunable	Strong (row)	Tunable
Operations	Simple	Complex (HDP)	Moderate
Schema flex	Limited	Limited	Flexible
Time-series	Excellent	Good	Moderate
Compaction	TWCS ideal	Manual	WiredTiger
Verdict	CHOSEN ✓	Viable alt	Not ideal

Decision: Cassandra for its native multi-DC replication, tunable consistency, and excellent write throughput.

Write path is critical -- 230K messages/sec sustained.

3.15.4 15.4 End-to-End Encryption Trade-offs

E2E Encryption (Signal Protocol)

How it works (simplified):

1. Each user generates identity key pair (Curve25519)
2. Key exchange via server (public keys only)
3. Derive shared secret using Double Ratchet Algorithm
4. Each message encrypted with unique key (forward secrecy)
5. Server stores only encrypted blobs, cannot decrypt

Trade-offs:

Benefit	Cost
Privacy: server cannot read msgs	No server-side search (search must be client-side)
Security: even if DB breached, msgs are unreadable	Complex key management (key exchange, rotation)
Forward secrecy: past msgs safe if key compromised	Multi-device sync is harder (each device has own keys)
Regulatory compliance	Cannot comply with lawful interception (by design)

Group E2E encryption:

- Sender encrypts message once with group key
- Group key rotated when members join/leave
- Pairwise keys for 256 members = key management overhead
- Alternative: Sender Key protocol (one encrypt, all decrypt)

Decision: Implement E2E for 1-on-1 (mandatory), optional for groups (performance vs security trade-off)

3.15.5 15.5 Message Storage: Store Per-Conversation vs Per-User

Option A: Per-Conversation (chosen)

Partition key: `conversation_id`

- + Group messages stored once (not per-member)
- + Efficient conversation scroll-back
- + Natural data locality
- Cross-conversation queries (search) require secondary index

Option B: Per-User Inbox

Partition key: `user_id`

- + All user's messages in one partition
- + Easy "all conversations" listing
- Group messages duplicated per member (256x for large group)
- Hot partition for active users
- Massive storage amplification

Decision: Per-conversation storage with a separate `user_conversations` table for the conversation list.

3.16 16. Interview Deep-Dive Questions

3.16.1 Q1: How do you handle message ordering in group chats?

Answer: We use a per-conversation monotonic sequence counter stored in Redis. When a message arrives for a group, the Chat Service atomically increments the counter (`INCR group:seq:{group_id}`) and assigns the resulting sequence number to the message. Since all messages for a group are routed through the same Kafka partition (keyed by `group_id`), we get total ordering within the partition.

On the client side, messages are displayed sorted by `sequence_number`. If the client detects a gap (e.g., receives seq 42 but last seen was 40), it requests the missing message (seq 41) from the server before displaying seq 42. This handles out-of-order delivery while maintaining causal consistency.

For very high-throughput groups, we can pre-allocate sequence ranges (`INCRBY 100`) to reduce Redis round-trips, though this creates non-contiguous sequences that the client must handle gracefully.

3.16.2 Q2: How does end-to-end encryption work in this system?

Answer: We implement the Signal Protocol (Double Ratchet Algorithm):

1. **Key Generation:** Each user generates an identity key pair (Curve25519), a signed pre-key, and a set of one-time pre-keys. Public keys are uploaded to the server.
 2. **Key Exchange:** When User A wants to message User B for the first time, A fetches B's public keys from the server and performs an X3DH (Extended Triple Diffie-Hellman) key agreement to establish a shared secret.
 3. **Message Encryption:** Each message is encrypted with a unique symmetric key derived from the Double Ratchet. The ratchet advances with each message, providing forward secrecy (compromising the current key does not reveal past messages).
 4. **Server Role:** The server only stores encrypted ciphertext. It cannot decrypt messages. It acts as a relay and key distribution service (for public keys only).
-

5. **Group Encryption:** Uses the Sender Key protocol. The sender generates a Sender Key, distributes it to all group members (encrypted with pairwise keys), then encrypts the message once with the Sender Key. All members can decrypt. When a member leaves, the Sender Key is rotated.
-

3.16.3 Q3: How do you sync messages across multiple devices?

Answer: Multi-device sync requires careful handling:

1. **Session Registry:** Each device maintains its own WebSocket connection. Redis tracks all active sessions: `HSET user:sessions:{uid} {device_id} → {server_id, conn_id}`.
 2. **Fan-out to All Devices:** When a message arrives for a user, the Chat Service looks up all active sessions and delivers to each device.
 3. **Sync Protocol:** Each device tracks its `last_synced_seq` per conversation. On reconnection, the device sends its sync state, and the server sends only messages with `seq > last_synced_seq`.
 4. **Conflict Resolution:** If a user reads a message on Device A, the read receipt is propagated to Device B. We use “latest writer wins” for conflicting actions (e.g., muting a conversation on two devices simultaneously).
 5. **E2E Encryption Complication:** Each device has its own encryption keys. Messages must be encrypted separately for each device. We use the multi-device Signal Protocol, where each device is treated as a separate recipient.
-

3.16.4 Q4: What happens when a user comes back online after being offline for days?

Answer: We handle this through a staged sync process:

1. **Pending Queue:** While offline, messages are stored in the `pending_messages` table in Cassandra, keyed by `(recipient_id, created_at)`.
 2. **Reconnection Sync:** When the user reconnects:
 - The client sends its `last_received_seq` per conversation.
 - The server queries `pending_messages` for the user.
 - Messages are delivered in batches of 100, oldest first.
 - Each batch is ACKed before the next is sent.
 3. **Large Backlog:** If the user has been offline for days, there might be thousands of pending messages. We prioritize:
 - First: conversation list with unread counts (fast UI rendering).
 - Then: messages from recent conversations first.
 - Background: older conversations load on-demand.
 4. **Storage Limits:** Pending messages are retained for 30 days. Beyond that, the client fetches from the permanent message store (Cassandra) when the user scrolls back.
 5. **Push Notifications:** While offline, the user received push notifications (batched to avoid notification spam). On reconnect, push delivery is suppressed in favor of WebSocket delivery.
-

3.16.5 Q5: How do you handle message deletion for everyone?

Answer:

1. **Soft Delete:** We do not physically delete the message immediately. Instead, we set a `deleted_at` timestamp and a `deleted_for` field (ALL or specific `user_ids`).
 2. **Deletion Message:** A “delete” control message is sent to all conversation participants via the normal message delivery pipeline. This message contains the `msg_id` to delete and the deletion type.
 3. **Time Window:** WhatsApp allows “Delete for Everyone” only within a window (e.g., 1 hour 8 minutes after sending). We enforce this server-side.
 4. **Client Handling:** Recipients’ clients receive the delete event and remove the message from local display, replacing it with “This message was deleted.”
 5. **Consistency:** If a recipient is offline when the delete is issued, the delete event is queued. When they come online, they receive the delete event and remove the message from their local store. There is a window where the recipient might have already read and cached the message locally.
 6. **Media Cleanup:** If the deleted message had media, the media file is scheduled for deletion from S3 (with a grace period for CDN cache expiration).
-

3.16.6 Q6: How do you handle the “thundering herd” problem when a popular user comes online?

Answer: A celebrity with millions of followers coming online could trigger millions of presence update fan-outs. We mitigate this:

1. **Subscription-Based Presence:** Only users who have the chat window open subscribe to another user’s presence. This reduces fan-out from “all contacts” to “users actively viewing the chat” (typically < 10).
 2. **Presence Coalescing:** If a user toggles online/offline rapidly (flaky connection), we debounce presence updates with a 5-second window. Only the final state is broadcast.
 3. **Lazy Presence:** For the conversation list view, presence is fetched on-demand (pull model) rather than pushed. The client requests presence for visible conversations only.
 4. **Rate Limiting:** Cap presence fan-out at 1,000 recipients per second. If a user has more subscribers, updates are spread over multiple seconds.
-

3.16.7 Q7: How do you ensure exactly-once message delivery?

Answer: We achieve exactly-once semantics through at-least-once delivery combined with idempotency:

1. **Sender Side:** The client generates a `client_msg_id` (UUID v4) for each message. If the server ACK is not received within 5 seconds, the client retries with the same `client_msg_id`.
2. **Server Dedup:** The Chat Service maintains a dedup cache in Redis: `SET dedup:{client_msg_id} → server_msg_id EX 3600`. If a duplicate arrives, the server returns the cached ACK without re-processing.

3. **Recipient Side:** The server assigns a globally unique `server_msg_id` (Snowflake ID). The recipient client deduplicates incoming messages by `server_msg_id`. This handles the case where the server delivered the message but the recipient's ACK was lost, causing the server to retry.
 4. **Idempotent Persistence:** Cassandra writes are naturally idempotent (same primary key overwrites). Writing the same message twice is a no-op.
-

3.16.8 Q8: How would you implement message search?

Answer:

1. **Architecture:** Message content is indexed asynchronously via Kafka into Elasticsearch. The search index is separate from the message store.
 2. **Index Design:** Documents are indexed per-message with fields: `user_id`, `conversation_id`, `content_text`, `sender_name`, `timestamp`. The index is sharded by `user_id` so each user's search is a single-shard query.
 3. **E2E Encryption Challenge:** With E2E encryption, the server cannot index message content. Two approaches:
 - **Client-side search:** Search is performed locally on the device using a local index (SQLite FTS). This works for recent messages.
 - **Encrypted search index:** The client builds an encrypted search index and uploads it. The server can perform token-based search without seeing plaintext. This is complex and has limitations.
 4. **Performance:** Elasticsearch returns message IDs. Full message content is fetched from Cassandra. Results are paginated (20 per page) and sorted by relevance/recency.
-

3.16.9 Q9: How do you handle rate limiting and abuse prevention?

Answer:

1. **Per-User Rate Limits:**
 - Messages: 100 messages per minute per user.
 - Group creation: 10 groups per day.
 - Media upload: 50 files per hour.
 2. **Implementation:** Token bucket algorithm in Redis. `DECR ratelimit:{user_id}:msg` with TTL reset.
 3. **Spam Detection:** ML model scores messages based on:
 - Send rate pattern (burst detection).
 - Number of unique recipients in short window.
 - Message content similarity (duplicate messages).
 - Account age and verification status.
 4. **Abuse Actions:** Graduated response: warning, temporary mute (1 hour), temporary ban (24 hours), permanent ban.
-

3.16.10 Q10: How do you handle schema evolution and backward compatibility?

Answer:

1. **Protocol Buffers:** All wire formats use protobuf, which supports backward-compatible schema evolution (new fields have defaults, old clients ignore unknown fields).
 2. **API Versioning:** REST APIs are versioned (`/api/v1/`, `/api/v2/`). Old versions are supported for at least 2 years.
 3. **Cassandra Schema:** New columns are added (not removed or renamed). Cassandra supports adding columns without downtime. Data migration is handled via dual-write during transition periods.
 4. **Feature Flags:** New features are gated behind feature flags. Clients advertise their capabilities during WebSocket handshake, and the server adapts its behavior accordingly.
-

3.16.11 Q11: How do you handle typing indicators efficiently?

Answer:

1. **Ephemeral Messages:** Typing indicators are NOT persisted. They are fire-and-forget via WebSocket.
 2. **Throttling:** The client sends “typing” events at most once every 3 seconds. The event has a 5-second implicit TTL (if no new typing event, UI shows “stopped typing”).
 3. **Fan-out:** For 1-on-1 chats, it is a single message to the recipient. For groups, it fans out only to members who currently have the chat open (determined by presence subscription).
 4. **No Reliability Needed:** If a typing indicator is lost, the impact is negligible. No retries, no persistence, no ACKs.
-

3.16.12 Q12: How do you implement read receipts for group chats?

Answer:

1. **Per-User Tracking:** Each user’s read position is tracked via `last_read_seq` in the `user_conversations` table.
 2. **Aggregation:** For a group message, the sender sees “delivered to 15/20 members, read by 10/20 members.” This is computed by aggregating `last_read_seq` across all group members.
 3. **Optimization:** We do not push individual read receipts for every member in a group. Instead, the read count is fetched on-demand when the sender views the message info screen.
 4. **Privacy:** Some users disable read receipts. The server respects the `read_receipts_enabled` flag per user and omits them from the count.
-

3.16.13 Q13: How would you handle message reactions (emoji reactions)?

Answer:

1. **Data Model:** Reactions are stored as a separate table: `reactions(message_id, user_id, emoji, timestamp)`.
 2. **Delivery:** Reactions are sent as control messages via the same delivery pipeline. They reference the original `message_id`.
-

3. **Aggregation:** The client displays aggregated reactions (e.g., “5 people reacted with thumbs-up”). The aggregation can be computed client-side from the reaction events.
 4. **Removal:** Users can remove their reaction. A “remove reaction” event is sent and processed the same way.
 5. **Scale:** Reactions add relatively low traffic compared to messages. A popular message might get 256 reactions in a group (one per member), which is manageable.
-

3.16.14 Q14: How do you design the notification batching system?

Answer:

1. **Problem:** A user in 50 active groups might receive hundreds of notifications while offline, which is overwhelming.
 2. **Batching Logic:**
 - First message in a conversation: immediate notification.
 - Subsequent messages within 30 seconds: batch into “5 new messages in Engineering Group.”
 - If more than 5 conversations have unread messages: collapse into “You have messages in 5 conversations.”
 3. **Implementation:** The Push Notification Service maintains a per-user notification buffer in Redis with a 30-second flush window. When the timer fires, it sends the batched notification.
 4. **Priority:** Direct messages have higher priority than group messages. Mentions (“@user”) in groups are treated as high-priority.
 5. **Muting:** Muted conversations do not generate push notifications, but unread counts still update (visible when the user opens the app).
-

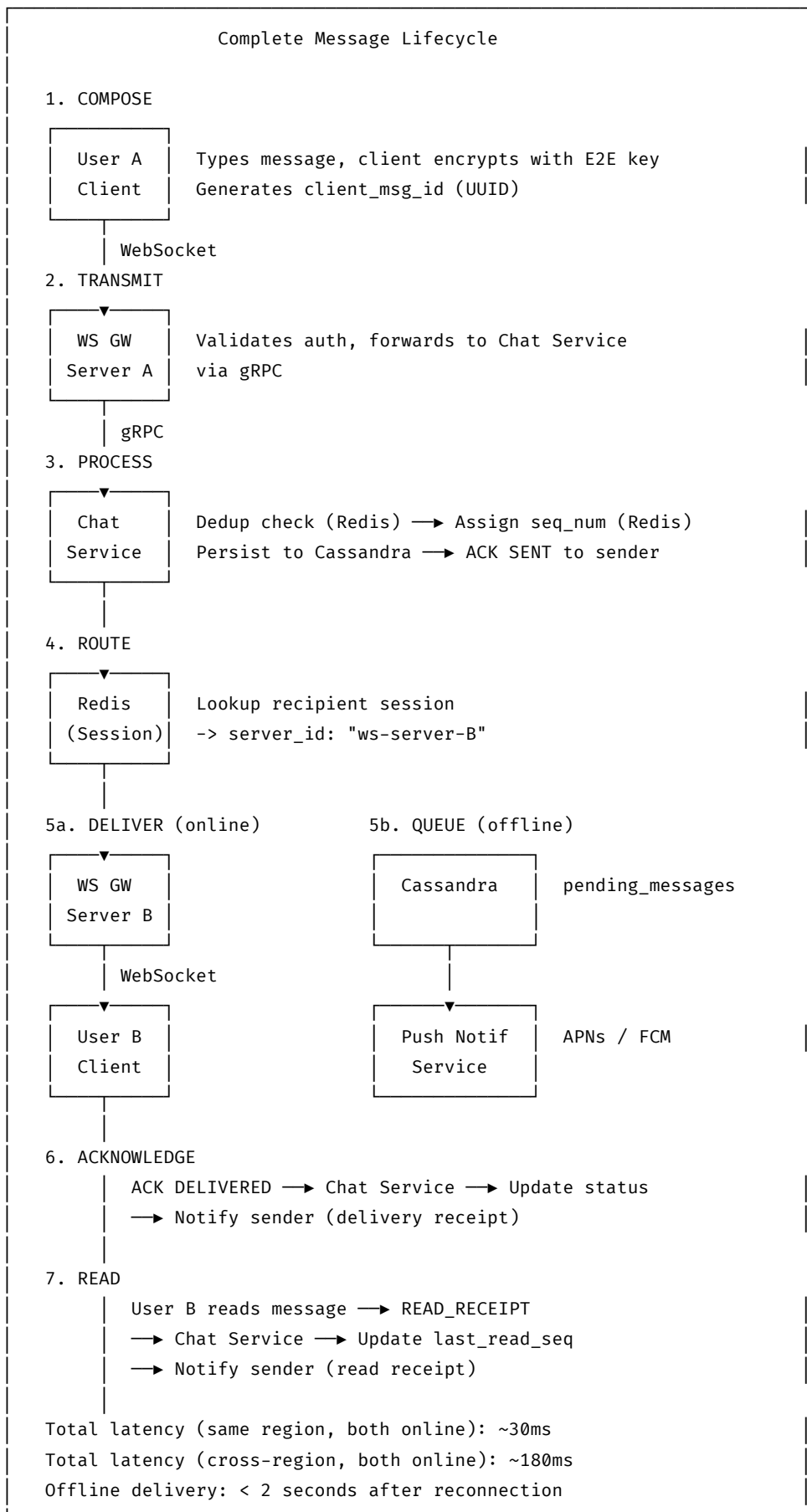
3.16.15 Q15: How would you migrate from a monolithic chat system to this microservices architecture?

Answer:

1. **Strangler Fig Pattern:** Gradually route traffic from monolith to microservices, one feature at a time.
2. **Migration Order:**
 - Phase 1: Extract Media Service (lowest risk, independent).
 - Phase 2: Extract Push Notification Service.
 - Phase 3: Extract Presence Service.
 - Phase 4: Extract Group Service.
 - Phase 5: Migrate message storage from MySQL to Cassandra (dual-write).
 - Phase 6: Extract Chat Service and WebSocket Gateway.
3. **Dual-Write:** During migration, write to both old and new systems. Read from old system. Verify consistency. Switch reads to new system. Stop writing to old system.
4. **Feature Flags:** Each migration phase is gated behind feature flags. Roll out to 1% of users, validate, then expand.

5. **Rollback Plan:** Each phase has a rollback procedure. The old system remains operational until the new system is validated at full scale.
-

3.17 17. End-to-End Message Flow Summary



3.18 18. Quick Reference: Interview Checklist

45-Minute Interview Time Allocation	
[0-5 min]	Requirements & scope clarification - Clarify: 1-on-1 vs group, scale, E2E encryption - State assumptions: 500M DAU, 40 msgs/day
[5-10 min]	Back-of-envelope estimation - 230K msg/sec, 20TB/day, 500M WS connections - Identify key bottlenecks early
[10-15 min]	API design (WebSocket + REST) - Show WebSocket protocol for real-time - REST for history, groups, media
[15-25 min]	High-level architecture - Draw the architecture diagram - Explain each component's role - Message flow: send -> process -> deliver
[25-40 min]	Deep dives (interviewer's choice) - Message ordering and delivery guarantees - WebSocket connection management - Group messaging fan-out - Presence service optimization - E2E encryption
[40-45 min]	Scalability and trade-offs - How each component scales horizontally - Key trade-offs: push vs pull, consistency model - Monitoring and failure handling

3.19 19. References and Further Reading

- **WhatsApp Architecture:** “The WhatsApp Architecture Facebook Bought For \$19 Billion” (High Scalability blog)
- **Signal Protocol:** Double Ratchet Algorithm specification (signal.org/docs)
- **Facebook Messenger:** “Building Mobile-First Infrastructure for Messenger” (Engineering at Meta blog)
- **Cassandra at Discord:** “How Discord Stores Trillions of Messages” (Discord Engineering blog)
- **Apache Kafka:** “Kafka: a Distributed Messaging System for Log Processing” (LinkedIn Engineering)
- **Consistent Hashing:** Karger et al., “Consistent Hashing and Random Trees”

Chapter 4

Design a News Feed System (Twitter / Facebook)

Difficulty: Hard | **Companies:** Meta, Twitter/X, Google, LinkedIn, Pinterest, ByteDance

4.1 Table of Contents

1. Problem Statement and Scope
 2. Functional Requirements
 3. Non-Functional Requirements
 4. Back-of-Envelope Estimation
 5. API Design
 6. Data Model and Database Selection
 7. High-Level Architecture
 8. Deep Dive: Core Components
 9. Data Partitioning and Sharding
 10. Caching Strategy
 11. Replication and Consistency
 12. Fault Tolerance and Failure Handling
 13. Scalability
 14. Monitoring and Observability
 15. Trade-offs and Design Decisions
 16. Interview Deep-Dive Questions
-

4.2 1. Problem Statement and Scope

4.2.1 What is a News Feed?

A **news feed** is a continuously updating list of stories in the middle of a social network's homepage. It includes status updates, photos, videos, links, app activity, and "likes" from people, pages, and groups that a user follows or is connected to.

The core challenge: given hundreds of millions of users who each follow hundreds or thousands of accounts, generate a personalized, ranked, real-time feed for every single user – and do it in under 200 milliseconds.

4.2.2 Scope Definition

SYSTEM SCOPE	
IN SCOPE	OUT OF SCOPE
Post creation	Ads insertion (separate system)
Feed generation	Content moderation (separate pipeline)
Feed ranking	Full-text search
Follow/unfollow	Direct messaging
Like/comment	Stories/reels (ephemeral content)
Real-time updates	Marketplace/shopping
Media handling	Video streaming infrastructure
Pagination	User authentication (assume handled)
Trending/hashtags	Recommendation engine (mention briefly)

4.2.3 Key Actors

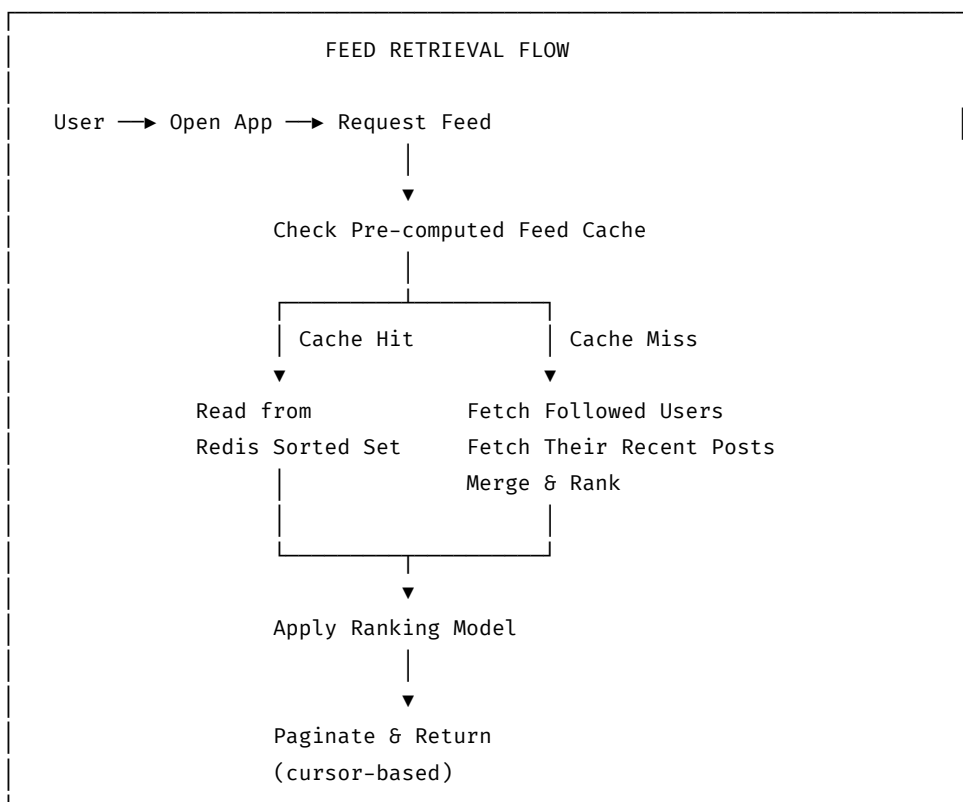
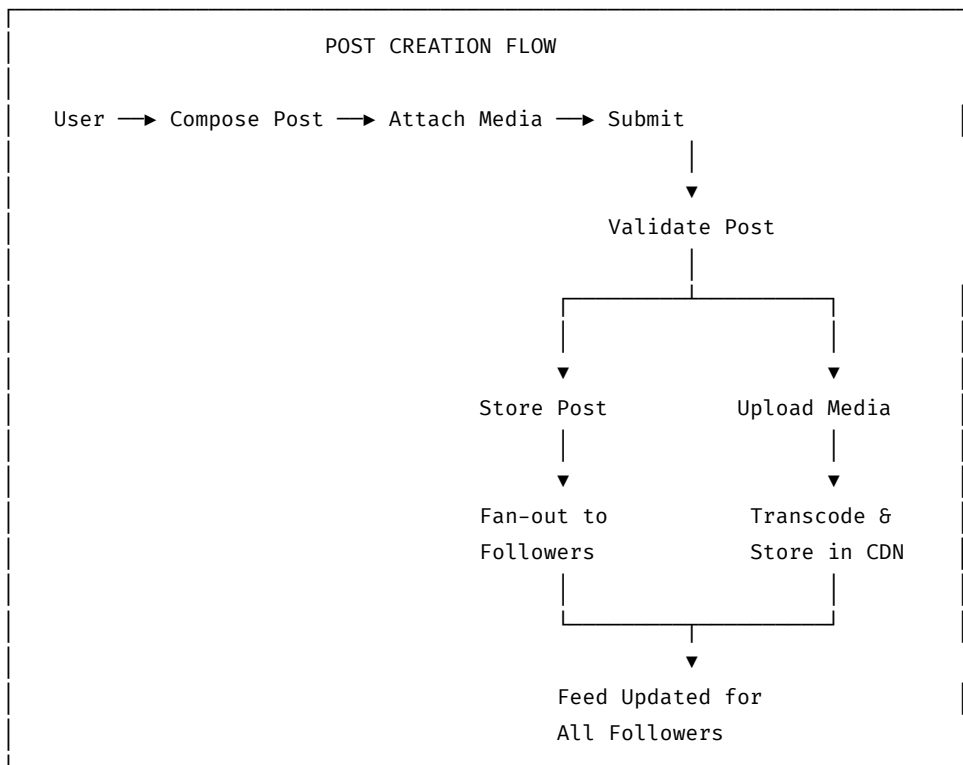
- **Content Creator:** Creates posts (text, images, video)
- **Content Consumer:** Views the news feed, scrolls, interacts
- **Celebrity/Influencer:** Has millions of followers (special handling)
- **System (Internal):** Fan-out workers, ranking pipelines, caches

4.3 2. Functional Requirements

4.3.1 Core Features

- **FR-1 - Post Creation:** Users can create posts with text (up to 10K chars), images, and videos
- **FR-2 - News Feed Generation:** Generate a personalized feed from posts by followed users
- **FR-3 - Follow / Unfollow:** Users can follow/unfollow other users; asymmetric relationship
- **FR-4 - Like / Comment / Share:** Users can interact with posts; interactions influence ranking
- **FR-5 - Real-time Feed Updates:** New posts from followed users appear in the feed with minimal delay
- **FR-6 - Feed Ranking:** Posts are ranked by relevance, not just chronological order
- **FR-7 - Pagination:** Cursor-based infinite scroll with consistent ordering
- **FR-8 - Hashtags / Trending:** Aggregate trending topics from recent posts
- **FR-9 - Media Support:** Support images (JPEG, PNG, WebP) and videos (MP4) with transcoding
- **FR-10 - Notifications:** Notify users of new interactions on their posts

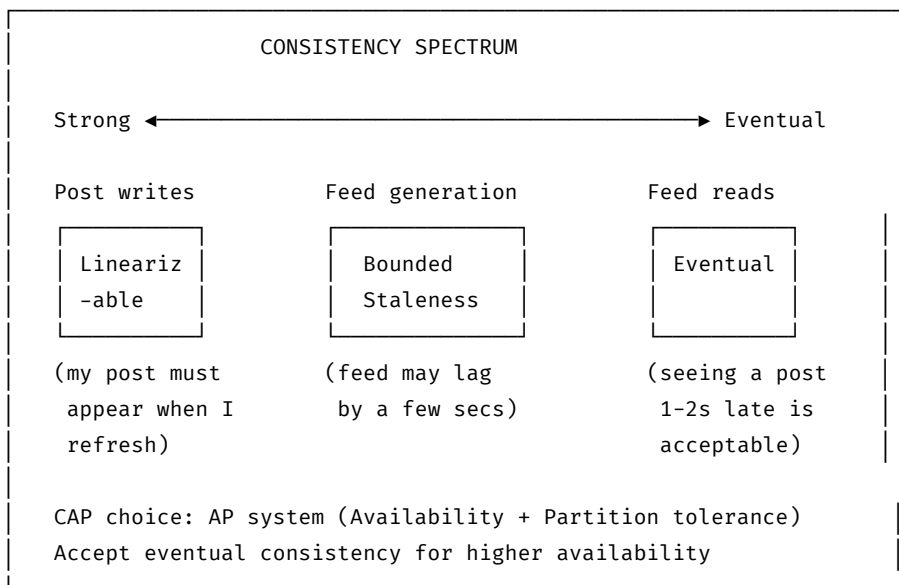
4.3.2 User Flows



4.4 3. Non-Functional Requirements

- **Latency:** Feed generation <200ms; Rationale: Users expect instant feed loading
- **Availability:** 99.99% uptime; Rationale: Core engagement feature; downtime = revenue loss
- **Consistency:** Eventual (seconds); Rationale: Feed does not need strong consistency
- **Throughput:** ~6K feed reads/sec; Rationale: Based on 500M DAU estimation
- **Durability:** No data loss for posts; Rationale: Posts are user-generated content; must persist
- **Scalability:** Handle 500M DAU; Rationale: Must scale horizontally
- **Global Reach:** Multi-region; Rationale: Users are worldwide; CDN for media
- **Fan-out Handling:** Celebrity w/ 100M followers; Rationale: Must not block on massive fan-out

4.4.1 Consistency vs Availability Trade-off



4.5 4. Back-of-Envelope Estimation

4.5.1 User and Traffic Assumptions

- **Total registered users:** Value: 2 billion; Derivation: Given
- **Daily Active Users (DAU):** Value: 500 million; Derivation: ~25% of total
- **Avg posts per user per day:** Value: 2; Derivation: Most users post less, power users more
- **Total posts per day:** Value: 1 billion; Derivation: 500M x 2
- **Avg followers per user:** Value: 300; Derivation: Power-law distribution
- **Feed refreshes per user/day:** Value: 10; Derivation: Open app ~10 times
- **Avg feed page size:** Value: 20 posts; Derivation: Per page/scroll

4.5.2 QPS Calculations

QPS ESTIMATION
Post Creation: $1\text{B posts/day} \div 86,400 \text{ sec} = \sim 11,574 \text{ QPS}$ $\text{Peak (3x avg)} = \sim 35\text{K QPS}$
Feed Reads: $500\text{M DAU} \times 10 \text{ refreshes/day} = 5\text{B requests/day}$ $5\text{B} \div 86,400 = \sim 58,000 \text{ QPS}$ $\text{Peak (3x avg)} = \sim 174\text{K QPS}$
Fan-out Writes (push model): $1\text{B posts} \times 300 \text{ avg followers} = 300\text{B fan-out operations/day}$ $300\text{B} \div 86,400 = \sim 3.47\text{M operations/sec}$ $\text{Peak} = \sim 10.4\text{M operations/sec}$
Read:Write ratio $\approx 5:1$ (reads dominate, but fan-out amplifies writes massively)

4.5.3 Storage Estimation

STORAGE ESTIMATION
Post Metadata (text + metadata): Avg post size: $\sim 1 \text{ KB (text)} + 0.5 \text{ KB (metadata)} = 1.5 \text{ KB}$ Daily: $1\text{B posts} \times 1.5 \text{ KB} = 1.5 \text{ TB/day}$ Yearly: $1.5 \text{ TB} \times 365 = \sim 548 \text{ TB/year}$ Media Storage: 20% of posts have images: $200\text{M} \times 500 \text{ KB avg} = 100 \text{ TB/day}$ 5% of posts have videos: $50\text{M} \times 10 \text{ MB avg} = 500 \text{ TB/day}$ Total media: $\sim 600 \text{ TB/day}$ Feed Cache (Redis): Each user's feed: $500 \text{ post IDs} \times 8 \text{ bytes} = 4 \text{ KB}$ 500M users: $500\text{M} \times 4 \text{ KB} = 2 \text{ TB}$ With scores + overhead: $\sim 6 \text{ TB total Redis}$ Social Graph: $2\text{B users} \times 300 \text{ avg connections} \times 16 \text{ bytes} = \sim 9.6 \text{ TB}$ TOTAL (5 years): Posts: $\sim 2.7 \text{ PB}$ Media: $\sim 1 \text{ EB (with dedup/compression savings)}$ Feed cache: $\sim 6 \text{ TB (rolling, not cumulative)}$

4.5.4 Bandwidth Estimation

- **Ingress:** Calculation: $35\text{K posts/sec} \times 1.5\text{KB} + \text{media uploads}$; Result: $\sim 500 \text{ MB/s}$
- **Egress:** Calculation: $174\text{K feed reads/sec} \times 20 \text{ posts} \times 1.5\text{KB}$; Result: $\sim 5 \text{ GB/s (text)}$
- **Media CDN:** Calculation: Images + video serving; Result: $\sim 50\text{-}100 \text{ GB/s}$

4.6 5. API Design

4.6.1 5.1 Post Creation

POST /v1/feed/posts

Authorization: Bearer <token>

Content-Type: multipart/form-data

```
{
  "content": "string (max 10,000 chars)",
  "media_ids": ["media_id_1", "media_id_2"], // pre-uploaded via media service
  "visibility": "public | followers | private",
  "reply_to": "post_id (optional)",
  "hashtags": ["tech", "systemdesign"]
}
```

Response 201:

```
{
  "post_id": "uuid-v7",
  "created_at": "2026-04-09T10:30:00Z",
  "author_id": "user_123",
  "content": "...",
  "media": [
    {
      "media_id": "m_456",
      "type": "image",
      "url": "https://cdn.example.com/images/m_456.webp",
      "thumbnail_url": "https://cdn.example.com/thumbs/m_456.webp"
    }
  ],
  "stats": { "likes": 0, "comments": 0, "shares": 0 }
}
```

4.6.2 5.2 Get News Feed (Cursor-Based Pagination)

GET /v1/feed?cursor=<cursor>&limit=20&ranking=ranked

Authorization: Bearer <token>

Response 200:

```
{
  "posts": [
    {
      "post_id": "uuid-v7",
      "author": {
        "user_id": "user_789",
        "username": "janedoe",
        "avatar_url": "https://cdn.example.com/avatars/user_789.webp"
      },
      "content": "...",
      "media": [...],
      "created_at": "2026-04-09T10:25:00Z",
      "stats": { "likes": 142, "comments": 23, "shares": 5 },
      "user_interaction": {
        "liked": false,
        "shared": false,
        "bookmarked": true
      },
      "ranking_score": 0.87
    },
    ...
  ],
  "pagination": {
    "next_cursor": "eyJ0IjoxNjk... ", // opaque, base64-encoded
    "has_more": true
  }
}
```

Why Cursor-Based Over Offset-Based Pagination?

PAGINATION COMPARISON	
Offset-Based GET /feed?offset=20	Cursor-Based GET /feed?cursor=abc123
❌ Skips/duplicates on concurrent inserts	✓ Stable across inserts/deletes
❌ O(N) for large offsets	✓ O(1) seek using indexed cursor
❌ Poor for real-time	✓ Works with real-time data
✓ Easy to implement	❌ Can't jump to arbitrary page
✓ Supports "go to page"	✓ Perfect for infinite scroll

Cursor encoding: `base64({ "ts": 1712654400, "post_id": "abc123" })`

4.6.3 5.3 Follow / Unfollow

POST /v1/users/{user_id}/follow

Authorization: Bearer <token>

Response 200: { "status": "following", "follower_count": 1024 }

DELETE /v1/users/{user_id}/follow

Authorization: Bearer <token>

Response 200: { "status": "unfollowed", "follower_count": 1023 }

4.6.4 5.4 Like / Comment

POST /v1/feed/posts/{post_id}/like

Authorization: Bearer <token>

Response 200: { "liked": true, "like_count": 143 }

POST /v1/feed/posts/{post_id}/comments

Authorization: Bearer <token>

```
{
  "content": "Great post!",
  "reply_to_comment_id": "comment_456 (optional)"
}
```

Response 201:

```
{
  "comment_id": "comment_789",
  "content": "Great post!",
  "author_id": "user_123",
  "created_at": "2026-04-09T10:35:00Z"
}
```

GET /v1/feed/posts/{post_id}/comments?cursor=<cursor>&limit=20

4.6.5 5.5 Trending / Hashtags

GET /v1/feed/trending?region=US&limit=10

Response 200:

```
{
  "trending": [
    { "hashtag": "#systemdesign", "post_count": 45200, "trend_score": 0.95 },
    { "hashtag": "#tech", "post_count": 38100, "trend_score": 0.88 },
    ...
  ],
  "updated_at": "2026-04-09T10:30:00Z"
}
```

4.6.6 Rate Limiting

- **POST /feed/posts:** Rate Limit: 50 requests; Window: 1 hour
- **GET /feed:** Rate Limit: 300 requests; Window: 1 min
- **POST /like:** Rate Limit: 200 requests; Window: 1 hour
- **POST /follow:** Rate Limit: 100 requests; Window: 1 hour
- **POST /comments:** Rate Limit: 100 requests; Window: 1 hour

4.7 6. Data Model and Database Selection

4.7.1 6.1 Database Selection Rationale

DATABASE SELECTION MAP		
Data Type	Database	Why
Posts	Cassandra (or DynamoDB)	High write throughput, tunable consistency, time-series friendly
Social Graph (follow edges)	Neo4j + MySQL (adjlist)	Graph queries (followers of followers), MySQL for simple follower lookups
Feed Cache	Redis (Cluster mode)	Sorted sets for ranked feeds, sub-ms reads, TTL support
User Profiles	MySQL / Aurora	Relational data, strong consistency for account data
Media Files	S3 + CloudFront (CDN)	Blob storage + CDN for global low-latency delivery
Counters (likes, views)	Redis (+ async sync)	Atomic increments for likes, comments; async persist to DB
Analytics/ Events	Apache Kafka + ClickHouse	Event streaming for engagement data, trending computation

4.7.2 6.2 Schema Definitions

4.7.2.1 Posts Table (Cassandra)

```
CREATE TABLE posts (
    post_id        UUID,           -- UUID v7 (time-ordered)
    author_id      UUID,
    content        TEXT,
    media_ids      LIST<UUID>,
    visibility     TEXT,           -- 'public', 'followers', 'private'
    hashtags       SET<TEXT>,
    reply_to       UUID,
    like_count     COUNTER,
    comment_count  COUNTER,
    share_count    COUNTER,
    created_at     TIMESTAMP,
    updated_at     TIMESTAMP,
    is_deleted     BOOLEAN,
    PRIMARY KEY ((author_id), created_at, post_id)
) WITH CLUSTERING ORDER BY (created_at DESC, post_id DESC);

-- Secondary index for post lookups by ID
CREATE TABLE posts_by_id (
    post_id        UUID,
    author_id      UUID,
    content        TEXT,
    media_ids      LIST<UUID>,
    visibility     TEXT,
    hashtags       SET<TEXT>,
    reply_to       UUID,
    like_count     COUNTER,
    comment_count  COUNTER,
    share_count    COUNTER,
    created_at     TIMESTAMP,
    updated_at     TIMESTAMP,
    is_deleted     BOOLEAN,
    PRIMARY KEY (post_id)
);
```

Partition key rationale: `author_id` groups all posts by a single user on the same partition, making “get user’s posts” a single-partition query. `created_at DESC` allows efficient time-ordered retrieval.

4.7.2.2 Social Graph (MySQL + Graph DB)

```
-- MySQL adjacency list for simple lookups
CREATE TABLE follows (
  follower_id BIGINT NOT NULL,
  followee_id BIGINT NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status      ENUM('active', 'muted', 'blocked') DEFAULT 'active',
  PRIMARY KEY (follower_id, followee_id),
  INDEX idx_followee (followee_id, follower_id),
  INDEX idx_created (follower_id, created_at)
) ENGINE=InnoDB;

-- Follower counts (denormalized for performance)
CREATE TABLE user_follow_counts (
  user_id          BIGINT PRIMARY KEY,
  follower_count   INT UNSIGNED DEFAULT 0,
  following_count  INT UNSIGNED DEFAULT 0
) ENGINE=InnoDB;
```

4.7.2.3 User Profiles (MySQL / Aurora)

```
CREATE TABLE users (
  user_id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  username         VARCHAR(50) UNIQUE NOT NULL,
  display_name     VARCHAR(100),
  email            VARCHAR(255) UNIQUE,
  avatar_url       VARCHAR(500),
  bio              VARCHAR(500),
  is_verified      BOOLEAN DEFAULT FALSE,
  is_celebrity     BOOLEAN DEFAULT FALSE,  -- flag for fan-out strategy
  follower_count   INT UNSIGNED DEFAULT 0,
  created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_username (username)
) ENGINE=InnoDB;
```

4.7.2.4 Feed Cache Structure (Redis)

```
-- Each user has a Redis Sorted Set for their feed
-- Key: feed:{user_id}
-- Members: post_id
-- Score: timestamp (epoch milliseconds) or ranking_score

ZADD feed:user_123 1712654400000 "post_abc"
ZADD feed:user_123 1712654300000 "post_def"
ZADD feed:user_123 1712654200000 "post_ghi"

-- Retrieve latest 20 posts (descending by score)
ZREVRANGE feed:user_123 0 19 WITHSCORES

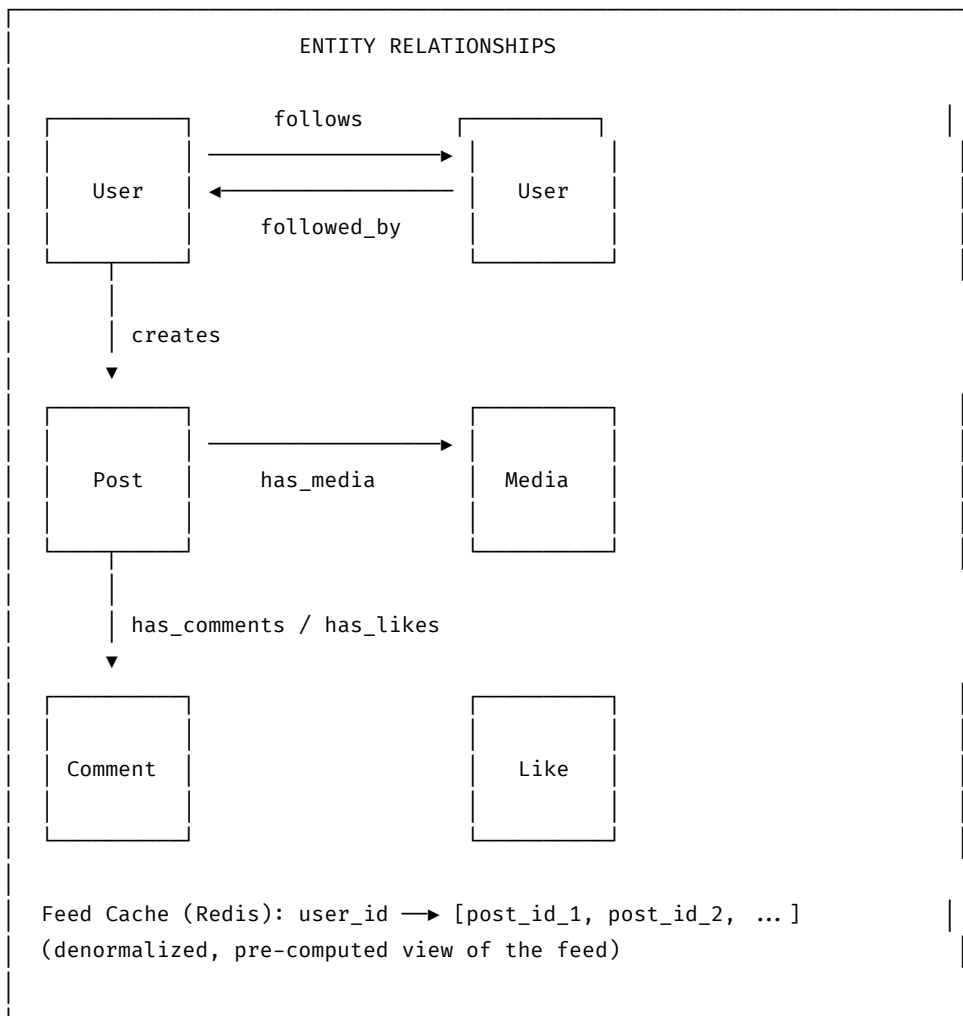
-- Cursor-based: get posts older than cursor timestamp
ZREVRANGEBYSCORE feed:user_123 (1712654300000 -inf LIMIT 0 20

-- TTL: feed expires after 7 days of inactivity
EXPIRE feed:user_123 604800
```

4.7.2.5 Comments Table (Cassandra)

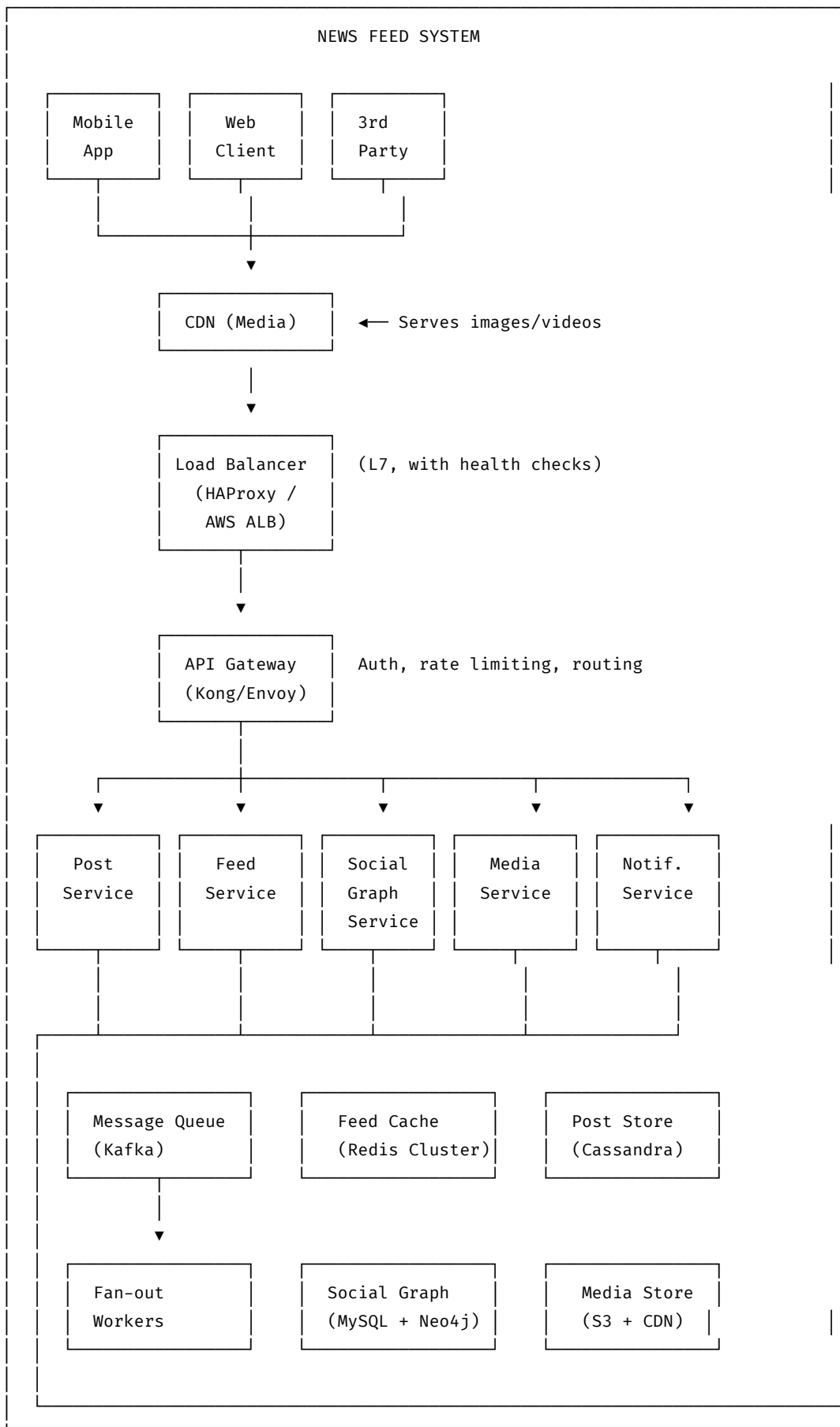
```
CREATE TABLE comments (
    post_id        UUID,
    comment_id     UUID,           -- UUID v7 (time-ordered)
    author_id      UUID,
    content        TEXT,
    reply_to       UUID,           -- null for top-level comments
    like_count     COUNTER,
    created_at     TIMESTAMP,
    is_deleted     BOOLEAN,
    PRIMARY KEY ((post_id), created_at, comment_id)
) WITH CLUSTERING ORDER BY (created_at ASC, comment_id ASC);
```

4.7.3 6.3 Entity Relationship Overview

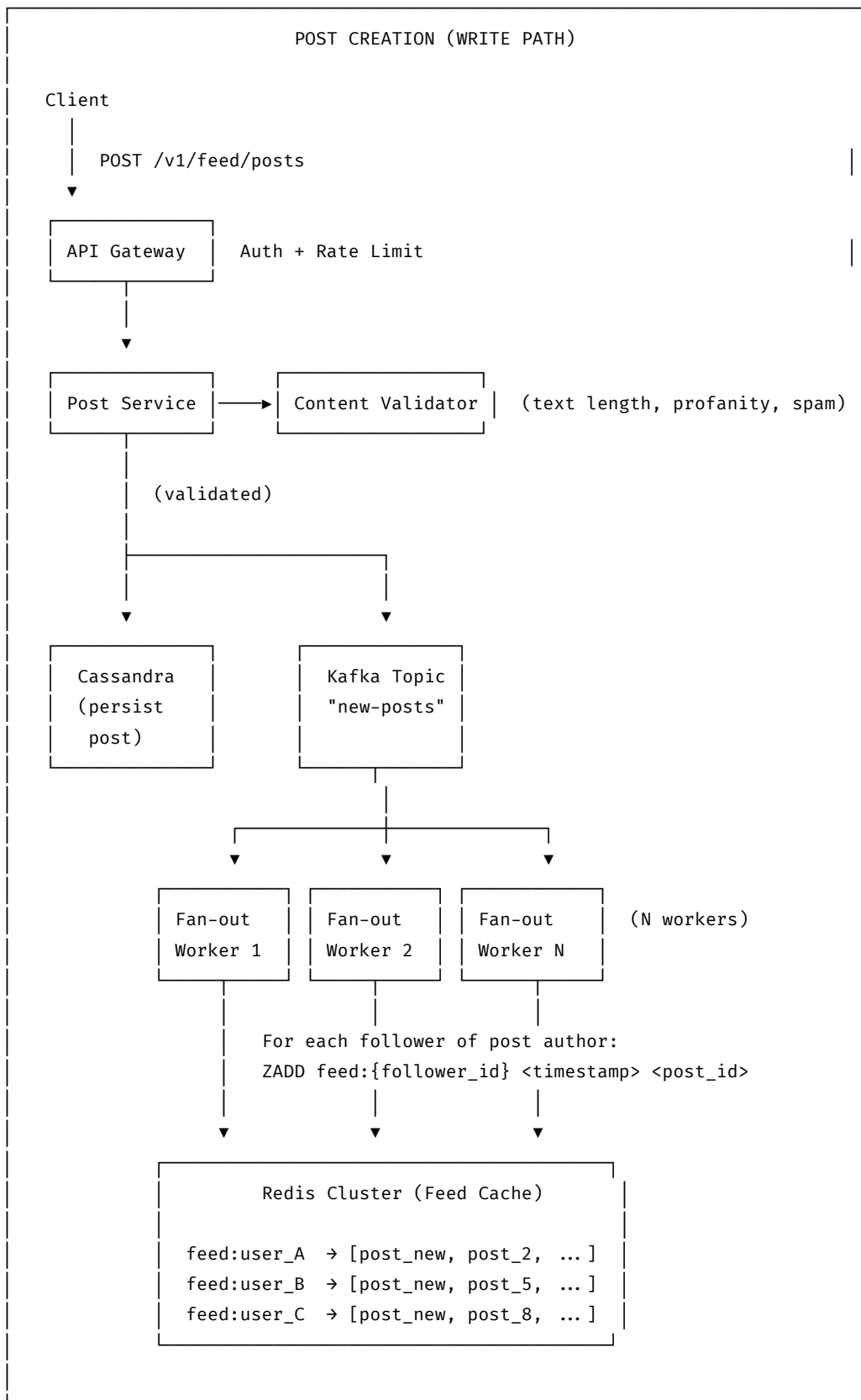


4.8 7. High-Level Architecture

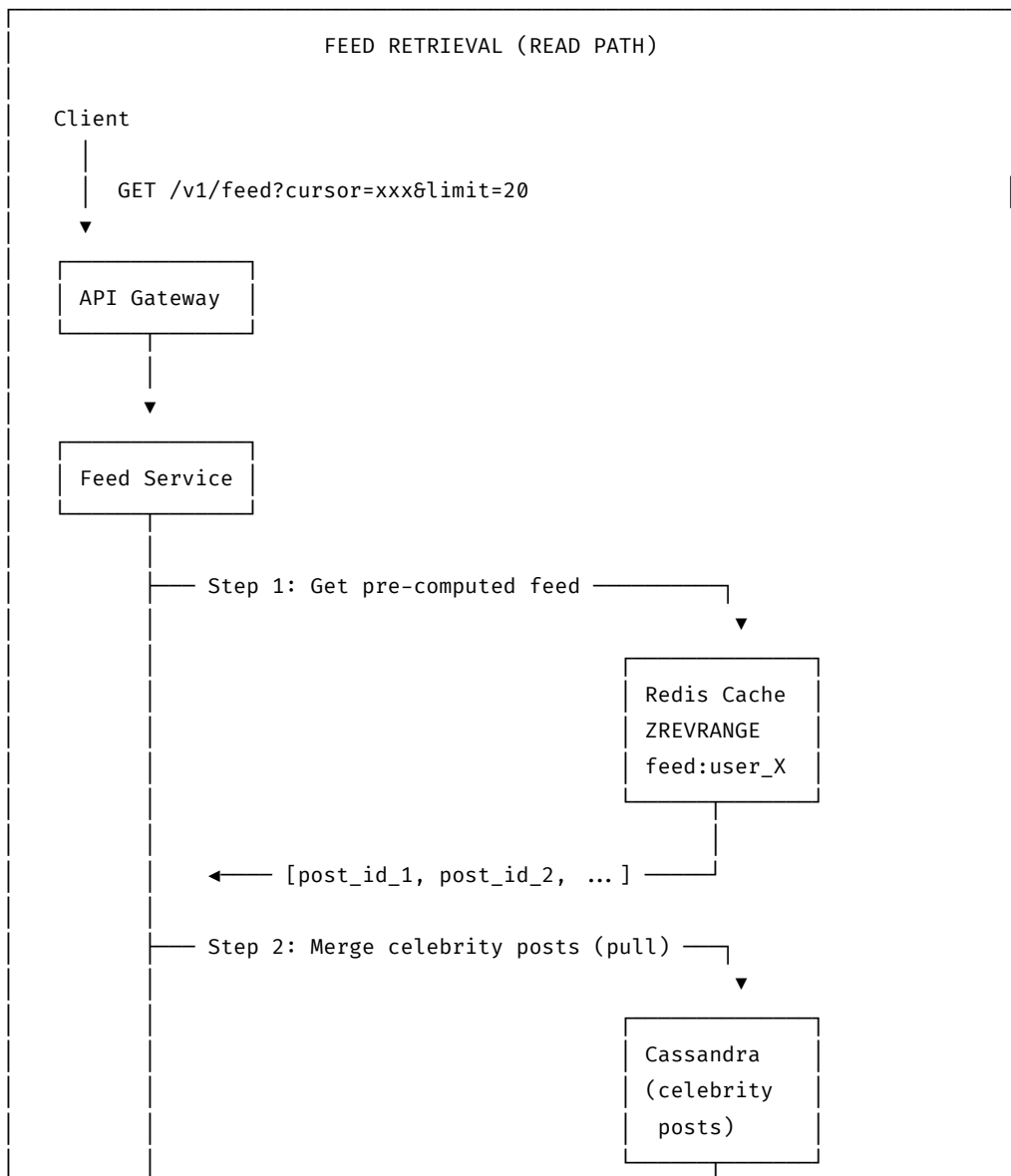
4.8.1 7.1 System Overview

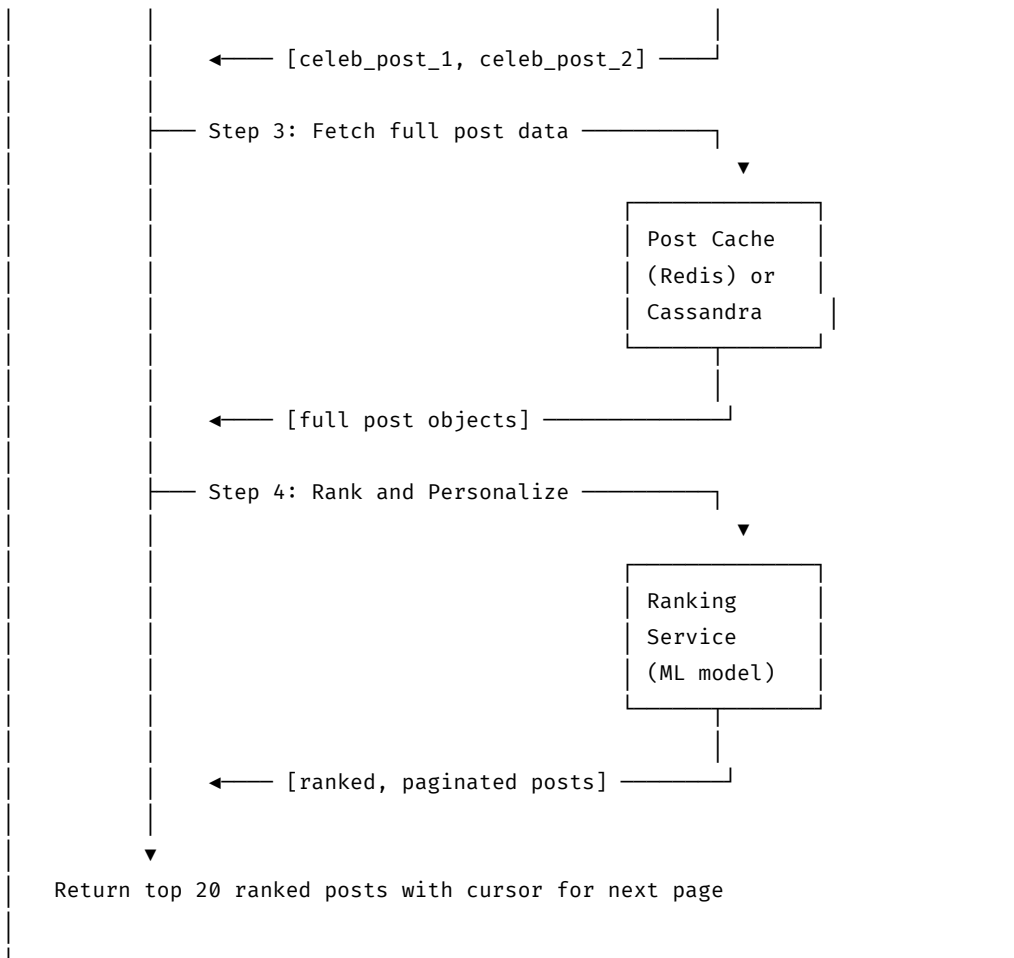


4.8.2 7.2 Post Creation Path (Write Path)



4.8.3 7.3 Feed Retrieval Path (Read Path)





4.8.4 7.4 Component Breakdown

- **API Gateway:** Responsibilities: Auth, rate limiting, request routing, SSL termination; Scale: 50+ instances
- **Post Service:** Responsibilities: CRUD for posts, validation, media orchestration; Scale: 100+ instances
- **Feed Service:** Responsibilities: Feed retrieval, celebrity merge, pagination; Scale: 200+ instances
- **Fan-out Service:** Responsibilities: Distribute new posts to follower feed caches; Scale: 500+ workers
- **Social Graph Svc:** Responsibilities: Follow/unfollow, follower lists, recommendations; Scale: 50+ instances
- **Media Service:** Responsibilities: Upload, transcode, thumbnail generation; Scale: 100+ instances
- **Notification Svc:** Responsibilities: Push notifications for interactions; Scale: 50+ instances
- **Ranking Service:** Responsibilities: ML-based feed ranking and personalization; Scale: 100+ instances
- **Trending Service:** Responsibilities: Compute trending hashtags from event stream; Scale: 20+ instances

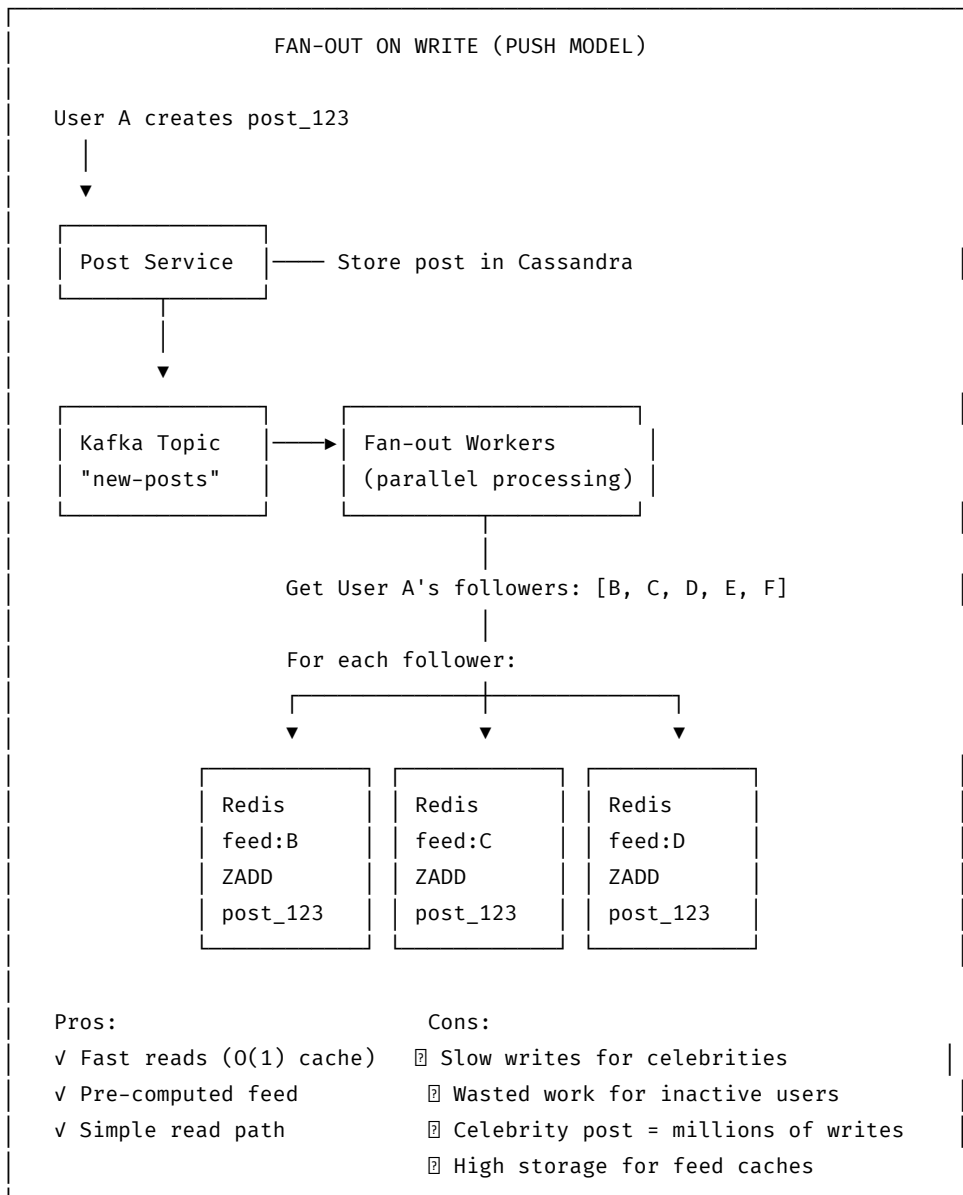
4.9 8. Deep Dive: Core Components

4.9.1 8.1 Fan-out Service

The fan-out service is the heart of the news feed system. When a user creates a post, the system must deliver that post to all followers' feeds. There are three fundamental strategies:

4.9.1.1 Strategy 1: Fan-out on Write (Push Model)

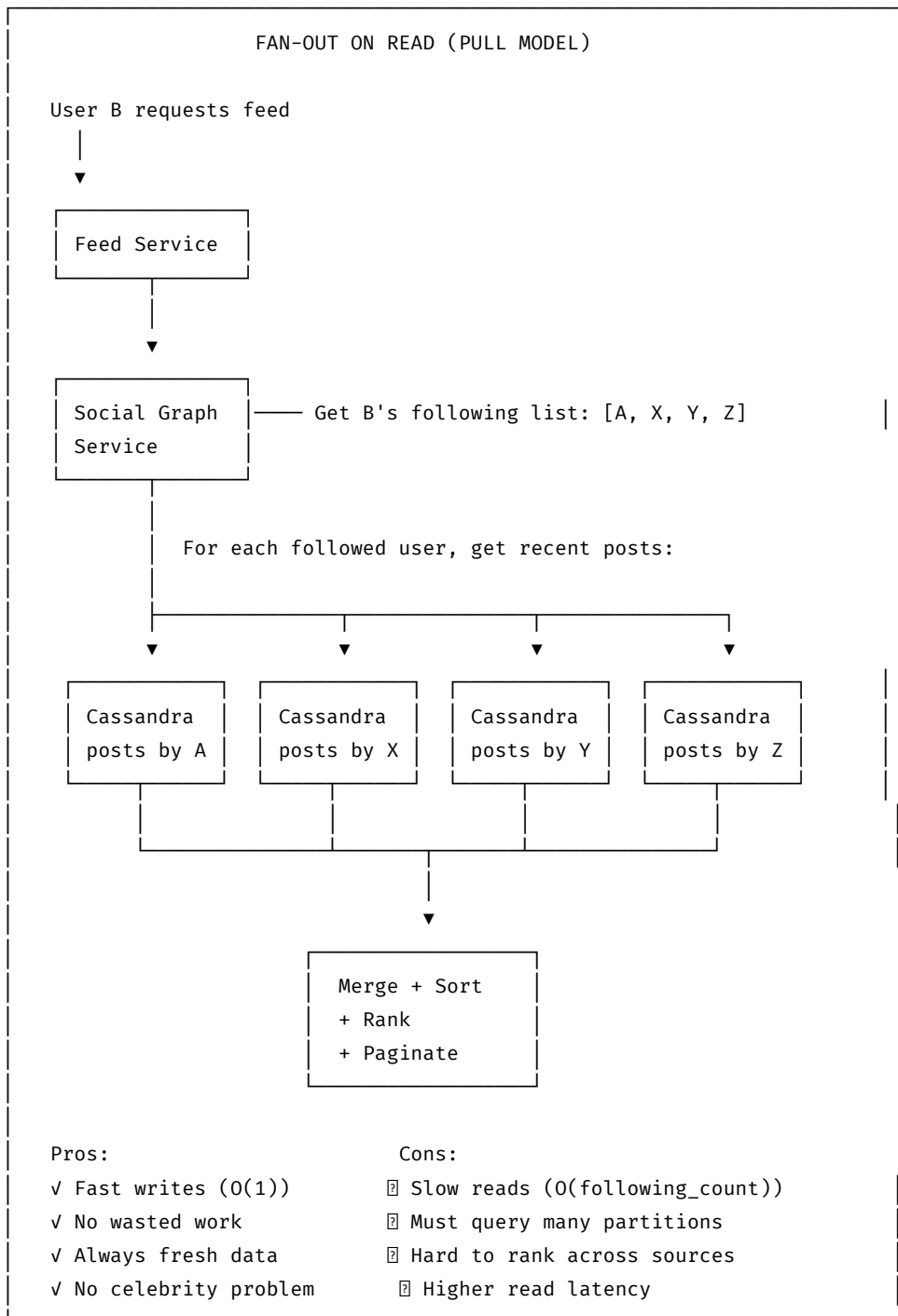
When a user publishes a post, immediately push it to all followers' feed caches.



Time complexity for a user with N followers: - Write: $O(N)$ – one write to each follower's feed cache - Read: $O(1)$ – single sorted set query

4.9.1.2 Strategy 2: Fan-out on Read (Pull Model)

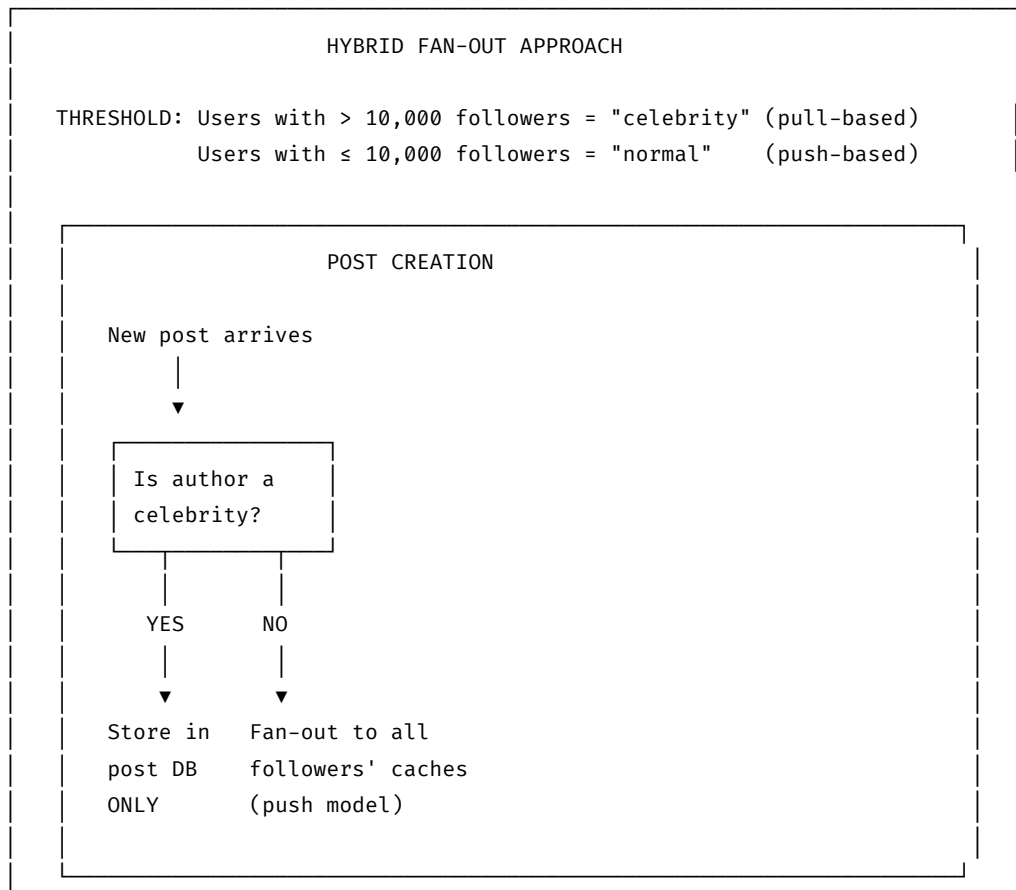
When a user requests their feed, fetch posts from all followed users at read time.

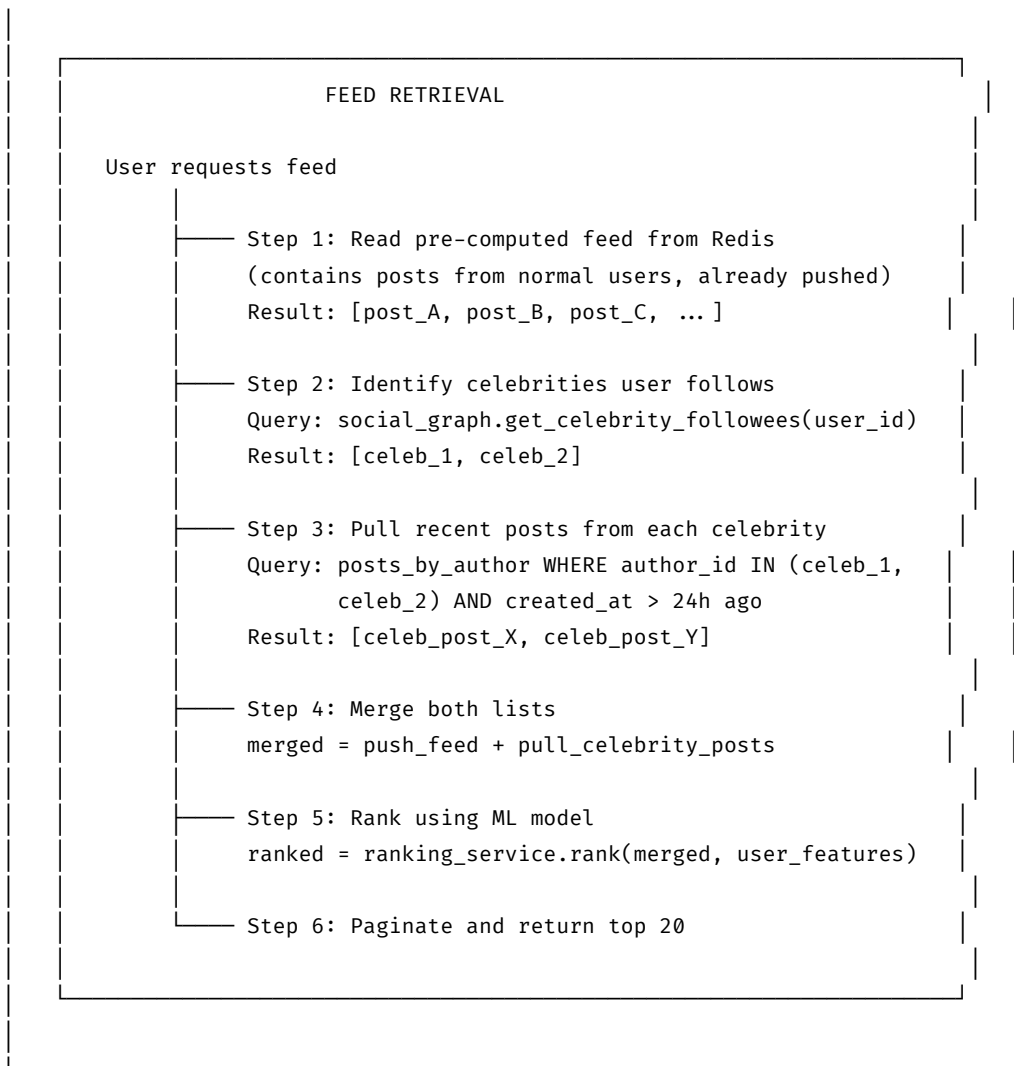


Time complexity for a user following M users: - Write: $O(1)$ – just store the post - Read: $O(M)$ – fetch from each followed user's partition

4.9.1.3 Strategy 3: Hybrid Approach (Production Solution)

This is what Twitter and Facebook actually use. Combine push for normal users and pull for celebrities.





4.9.1.4 Fan-out Strategy Comparison

Dimension	Fan-out Write	Fan-out Read	Hybrid
Write latency	High ($O(N)$)	$O(1)$	Low-Medium
Read latency	$O(1)$	High ($O(M)$)	Low
Celebrity problem	Severe	None	Solved
Storage (cache)	Very high	Low	Moderate
Freshness	Near real-time	Always fresh	Near real-time
Complexity	Low	Medium	High
Wasted work	Inactive users	None	Minimal
Used by	Early Twitter	(rarely pure)	Twitter, Meta

4.9.1.5 The Celebrity Problem in Detail

THE CELEBRITY PROBLEM

Scenario: Celebrity C has 100 million followers, posts once.

Fan-out on Write:

100,000,000 Redis ZADD operations
At 100K ops/sec per Redis node = 1,000 seconds = ~17 minutes
Even with 100 fan-out workers: ~10 seconds lag
Memory: 100M × 8 bytes = 800 MB per celebrity post

Hybrid Solution:

Celebrity C's post stored ONLY in Cassandra
Write cost: O(1) -- single DB write
Read cost: each follower pulls from C's post timeline
Since user follows ~5-10 celebrities on average:
Extra read cost per feed request = 5-10 Cassandra queries
~5ms extra latency (well within 200ms budget)

How to determine the threshold?

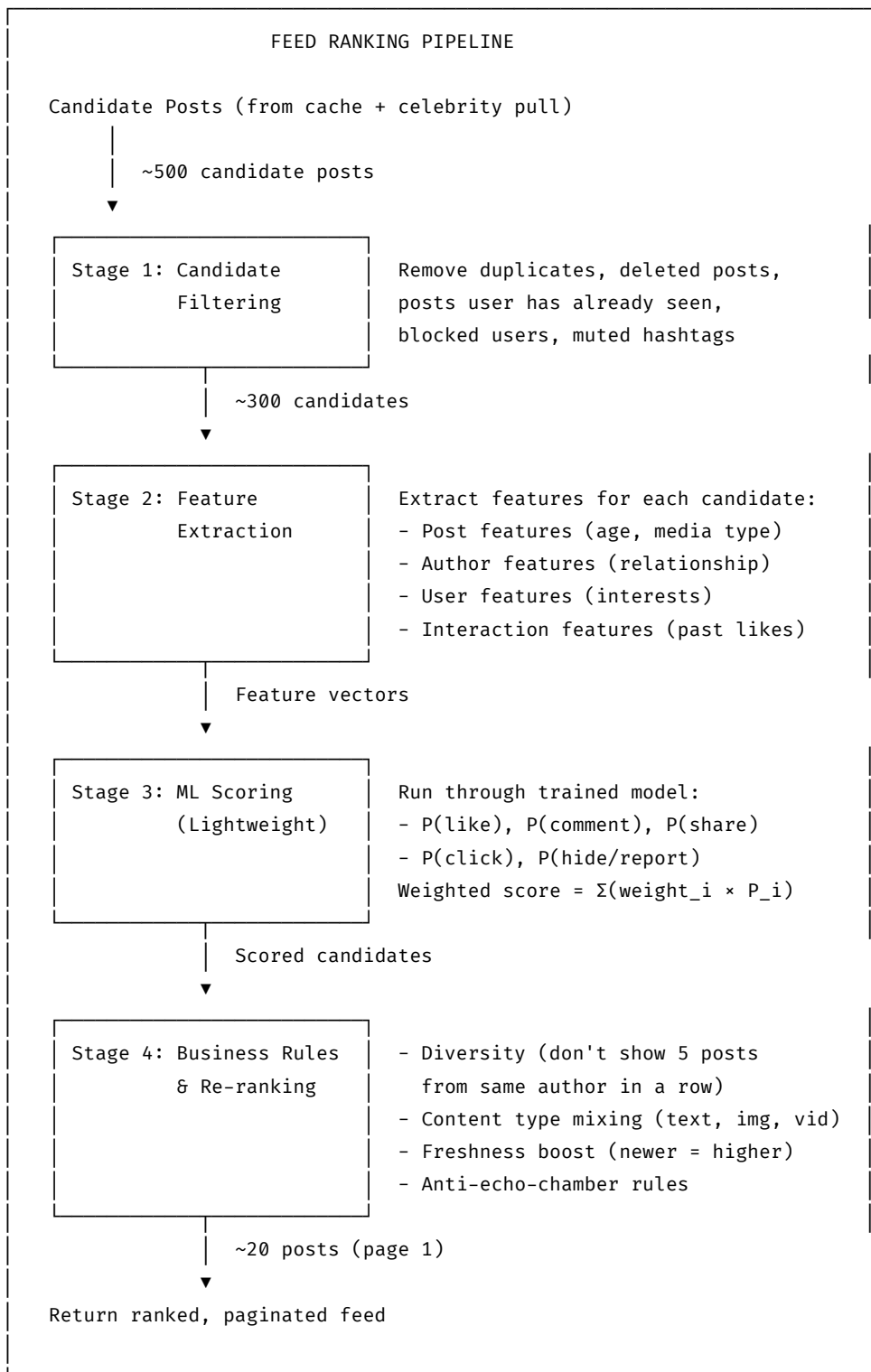
Followers < 1K : Always push (fast fan-out)
1K - 10K : Push (manageable fan-out)
10K - 100K : Configurable (analyze cost)
> 100K : Always pull (celebrity)

The threshold is tunable. Twitter reportedly uses ~5K-10K.
The flag `is_celebrity` on the user record controls this.

4.9.2 8.2 Feed Ranking Algorithm

Modern news feeds do not show posts in pure chronological order. They use ML-based ranking to maximize user engagement and relevance.

4.9.2.1 Ranking Architecture



4.9.2.2 Feature Vector and Scoring Function

RANKING FEATURE VECTOR

For each candidate post, we compute:

Post Features:

post_age_hours	: float	(0.5 = 30 min ago)
has_image	: bool	(1/0)
has_video	: bool	(1/0)
text_length	: int	(normalized 0-1)
hashtag_count	: int	
current_like_count	: int	(log-normalized)
current_comment_count	: int	(log-normalized)
current_share_count	: int	(log-normalized)
engagement_velocity	: float	(likes/min in last hour)

Author Features:

is_verified	: bool	
follower_count	: int	(log-normalized)
avg_engagement_rate	: float	(past 30 days)
post_frequency	: float	(posts per day)

User-Author Relationship Features:

relationship_strength	: float	(0-1, based on past interactions)
times_liked_author	: int	(in last 30 days)
times_commented	: int	(in last 30 days)
times_shared	: int	(in last 30 days)
profile_visits	: int	(visited author's profile?)
is_close_friend	: bool	
mutual_friends_count	: int	

Scoring Function:

```
score = w1 * P(like)
      + w2 * P(comment)
      + w3 * P(share)
      + w4 * P(click_through)
      - w5 * P(hide)
      - w6 * P(report)
      + w7 * freshness_decay(post_age)
      + w8 * relationship_strength
```

Where:

```
w1=1.0, w2=2.0, w3=3.0, w4=0.5, w5=5.0, w6=10.0
w7=0.3, w8=1.5
```

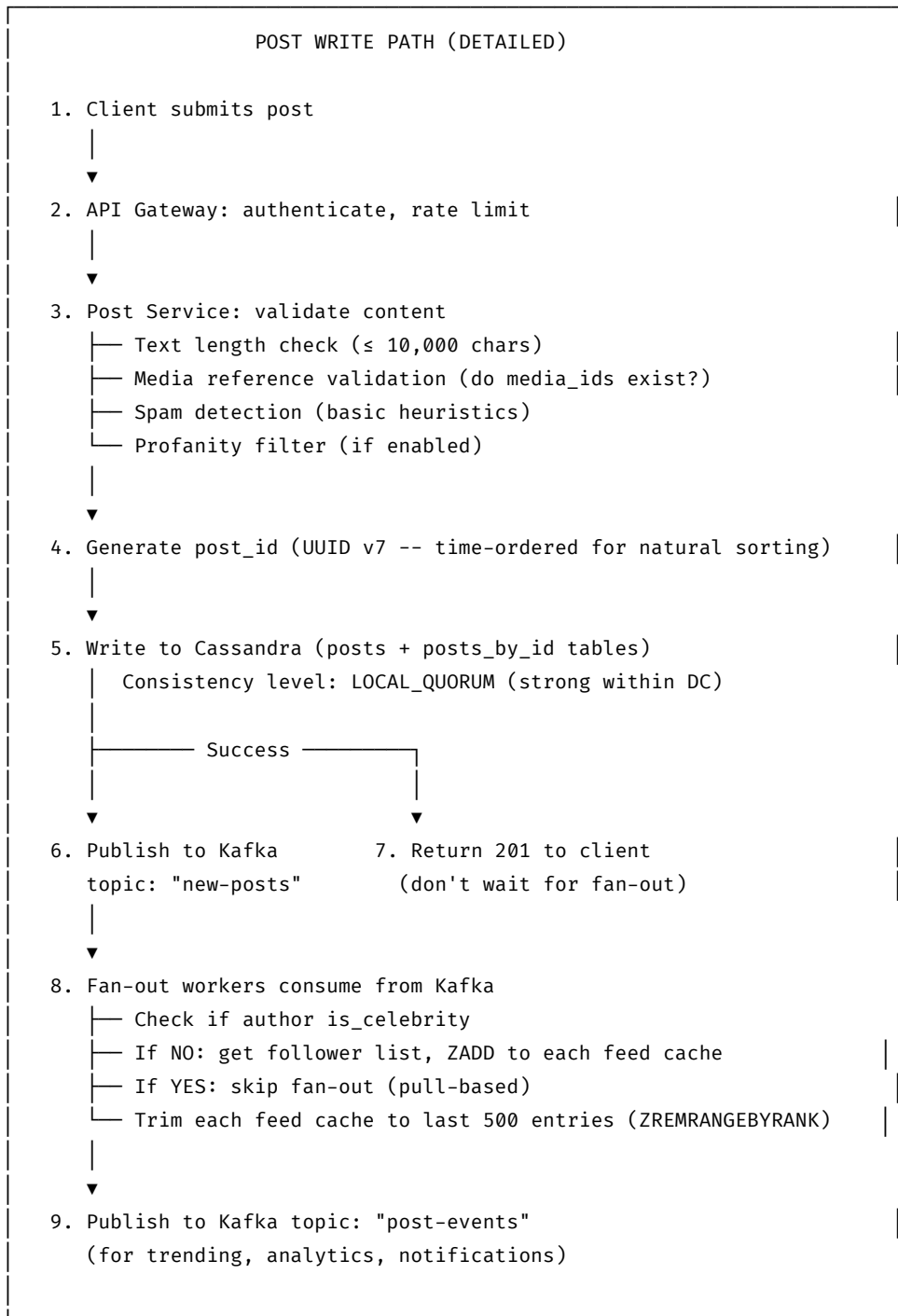
```
freshness_decay(age) = 1 / (1 + age_hours/24)
(posts older than 24h get significantly lower scores)
```

4.9.2.3 Chronological vs Ranked Feed

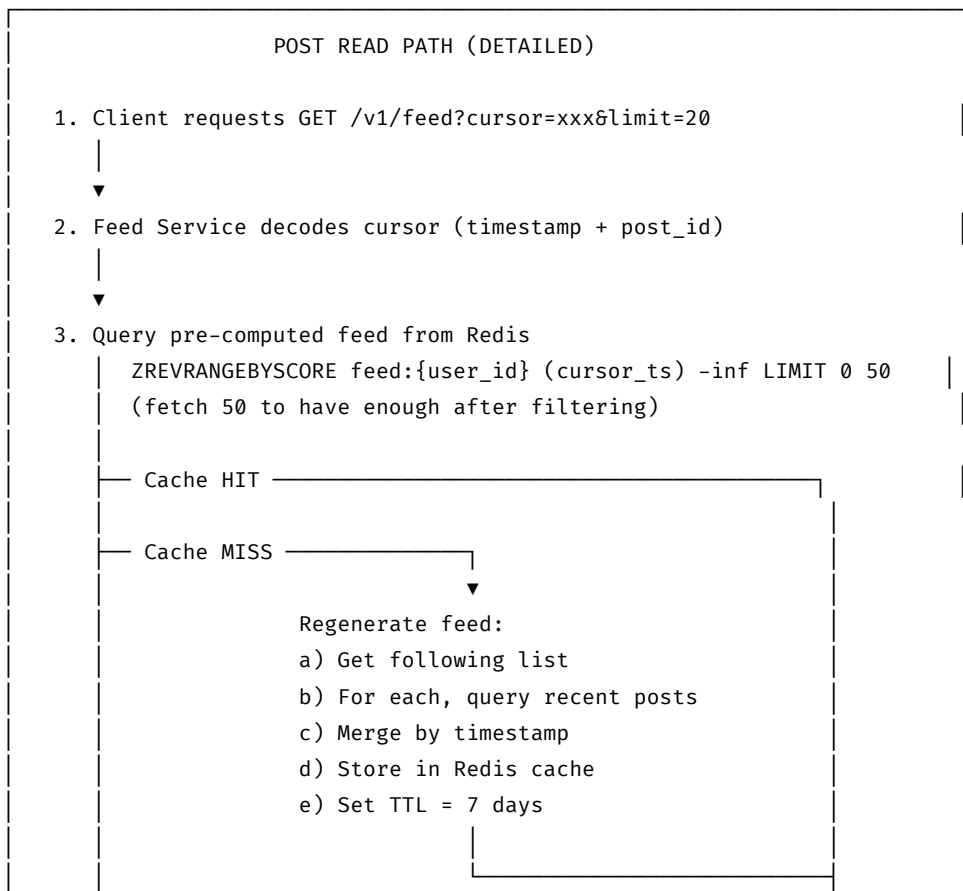
- **Ordering:** Chronological: Newest first; Ranked (ML-based): Relevance score
- **User expectation:** Chronological: Simple, predictable; Ranked (ML-based): “Best” content surfaces
- **Engagement:** Chronological: Lower; Ranked (ML-based): Higher (typically +10-30%)
- **Cold start:** Chronological: Works perfectly; Ranked (ML-based): Needs interaction history
- **Transparency:** Chronological: High (user understands); Ranked (ML-based): Low (black box)
- **Implementation:** Chronological: Simple; Ranked (ML-based): Complex (ML pipeline)
- **Celebrity advantage:** Chronological: Low (buried quickly); Ranked (ML-based): High (engagement begets visibility)
- **Used by:** Chronological: Mastodon, RSS readers; Ranked (ML-based): Facebook, Twitter, Instagram, TikTok

4.9.3 8.3 Post Storage and Retrieval

4.9.3.1 Write Path (Detailed)



4.9.3.2 Read Path (Detailed)



- ```

 4. Merge with celebrity posts (pull-based)
 | Get user's celebrity followee list (cached in Redis)
 | For each celebrity: query Cassandra for recent posts
 | Merge into candidate list
 |
 5. Hydrate post IDs into full post objects
 | Multi-get from post cache (Redis) → fallback to Cassandra
 | Batch fetch author profiles (user cache)
 | Fetch engagement counts (counter cache)
 |
 6. Filter out:
 | - Deleted posts (is_deleted = true)
 | - Posts from blocked users
 | - Posts user has already seen (seen_posts bloom filter)
 | - Muted hashtags
 |
 7. Apply ranking model
 |
 8. Take top 20, encode next cursor, return response

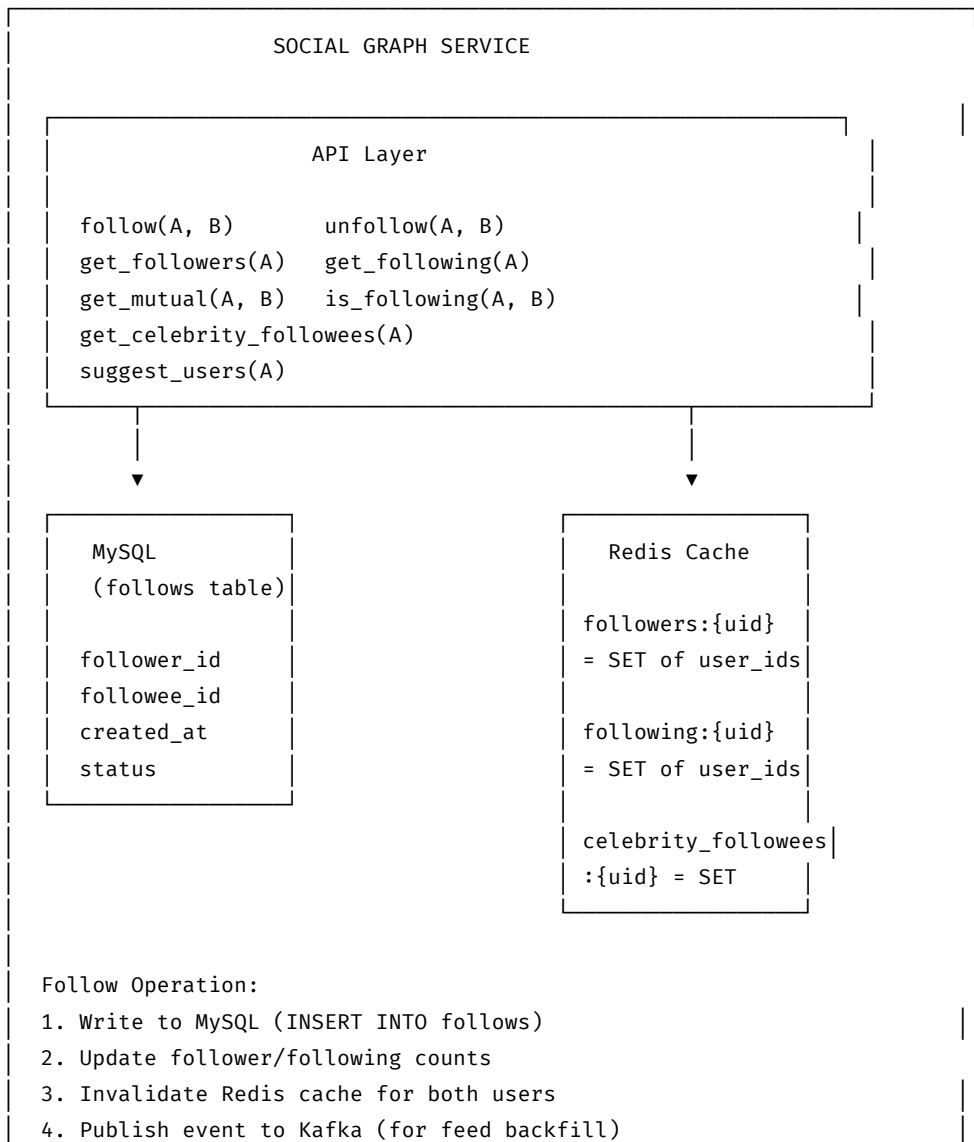
```

Total latency budget:

| Step                    | Target (ms) |
|-------------------------|-------------|
| Redis feed read         | 5           |
| Celebrity post fetch    | 20          |
| Post hydration (batch)  | 30          |
| Filtering               | 5           |
| Ranking model inference | 50          |
| Serialization + network | 20          |
| Buffer                  | 70          |
| TOTAL                   | 200         |

#### 4.9.4 8.4 Social Graph Service

The social graph manages follow relationships and powers follower list queries for fan-out.



Unfollow Operation:

1. Soft-delete in MySQL (UPDATE status = 'inactive')
2. Update counts
3. Invalidate caches
4. Remove unfollowed user's posts from follower's feed cache (async, via Kafka consumer)

Friend Recommendations (Graph Traversal):

User A follows [B, C]

B follows [D, E, F]

C follows [D, G, H]

"Friends of friends" = {D(2), E(1), F(1), G(1), H(1)}

Recommend D first (common connection = 2)

This is a 2-hop BFS -- efficient with Neo4j

For simple use: MySQL with JOIN on follows table

## 4.10 9. Data Partitioning and Sharding

### 4.10.1 Sharding Strategy Overview

SHARDING STRATEGY

| Data Store               | Shard Key                    | Rationale                                      |
|--------------------------|------------------------------|------------------------------------------------|
| Posts<br>(Cassandra)     | author_id<br>(partition key) | "Get user's posts" is a single-partition query |
| Feed Cache<br>(Redis)    | user_id<br>(hash slot)       | Each user's feed lives on one Redis node       |
| Social Graph<br>(MySQL)  | user_id<br>(range shard)     | Follower/following lists accessed per-user     |
| User Profiles<br>(MySQL) | user_id<br>(hash shard)      | Direct lookups by user                         |

### 4.10.2 Cassandra Partitioning for Posts

#### CASSANDRA PARTITION LAYOUT FOR POSTS

Partition Key: `author_id`

Clustering Keys: (`created_at` DESC, `post_id` DESC)

Partition: `author_id = user_A`

|                  |          |     |
|------------------|----------|-----|
| 2026-04-09 10:30 | post_abc | ... |
| 2026-04-09 08:15 | post_def | ... |
| 2026-04-08 22:00 | post_ghi | ... |
| ...              |          |     |

Partition: `author_id = user_B`

|                  |          |     |
|------------------|----------|-----|
| 2026-04-09 09:45 | post_jkl | ... |
| 2026-04-08 14:30 | post_mno | ... |

Hot partition risk: celebrity with 50K posts/year → large partition

Mitigation: Add time bucket to partition key

PRIMARY KEY ((`author_id`, `year_month`), `created_at`, `post_id`)

This caps each partition to ~1 month of posts per user

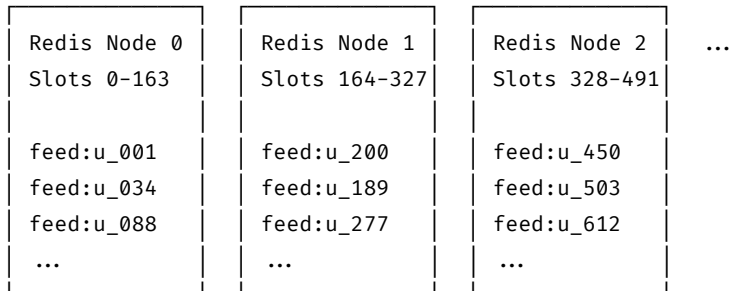


### 4.10.3 Redis Cluster Sharding for Feed Cache

#### REDIS CLUSTER FOR FEED CACHE

Redis Cluster: 100 nodes, 16384 hash slots

Key: `feed:{user_id}` → `hash(user_id) mod 16384` → slot → node



Each node:

- Primary: handles reads and writes
- 2 Replicas: handle read scaling + failover
- Memory: ~60 GB per node (6 TB total across 100 nodes)

Each sorted set:

- Max 500 entries (`post_ids` with timestamp scores)
- Size: ~4-8 KB per user's feed
- `ZREMRANGEBYRANK` trims to keep size bounded

#### 4.10.4 MySQL Sharding for Social Graph

##### MYSQL SHARDING FOR SOCIAL GRAPH

Strategy: Hash-based sharding on follower\_id  
Shards: 256 MySQL shards (Vitess or ProxySQL for routing)

Query: "Get all users that user\_A follows"

→ Route to shard:  $\text{hash}(\text{user\_A}) \bmod 256$

→ Single-shard query (efficient)

Query: "Get all followers of user\_A"

→ Needs secondary index on followee\_id

→ Option A: Global secondary index (scatter-gather)

→ Option B: Denormalized reverse table (followers\_of)  
sharded by followee\_id

Table: follows (sharded by follower\_id)

"Who does user X follow?" → single shard

Table: followers\_of (sharded by followee\_id)

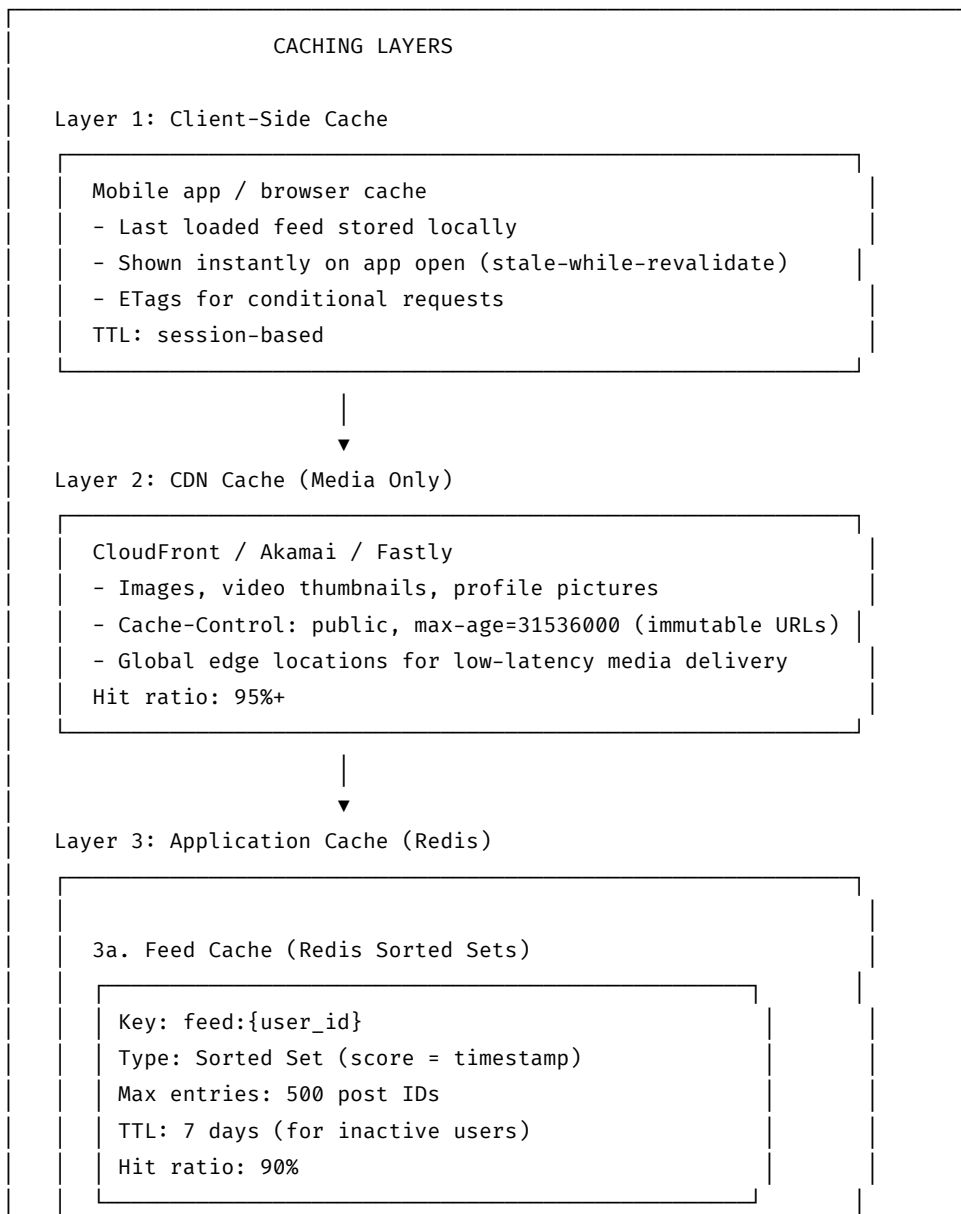
"Who follows user X?" → single shard

(denormalized copy, maintained via async replication)

This denormalization doubles storage but eliminates scatter-gather queries, which is critical for fan-out performance.

## 4.11 10. Caching Strategy

### 4.11.1 Multi-Layer Cache Architecture



## 3b. Post Cache (Redis Hash)

```
Key: post:{post_id}
Type: Hash (all post fields)
TTL: 24 hours (hot posts stay cached)
Hit ratio: 85%
```

## 3c. User Cache (Redis Hash)

```
Key: user:{user_id}
Type: Hash (profile fields)
TTL: 1 hour
Hit ratio: 95%
```

## 3d. Follower List Cache (Redis Set)

```
Key: followers:{user_id}
Type: Set of user_ids
TTL: 1 hour
Invalidated on follow/unfollow
```

## 3e. Engagement Counters (Redis)

```
Key: likes:{post_id}, comments:{post_id}
Type: Integer (INCR/DECR)
Async-synced to Cassandra every 30 seconds
```



## Layer 4: Database (Source of Truth)

```
Cassandra (posts), MySQL (users, social graph)
Only hit on cache miss
```

### 4.11.2 Cache Invalidation Strategy

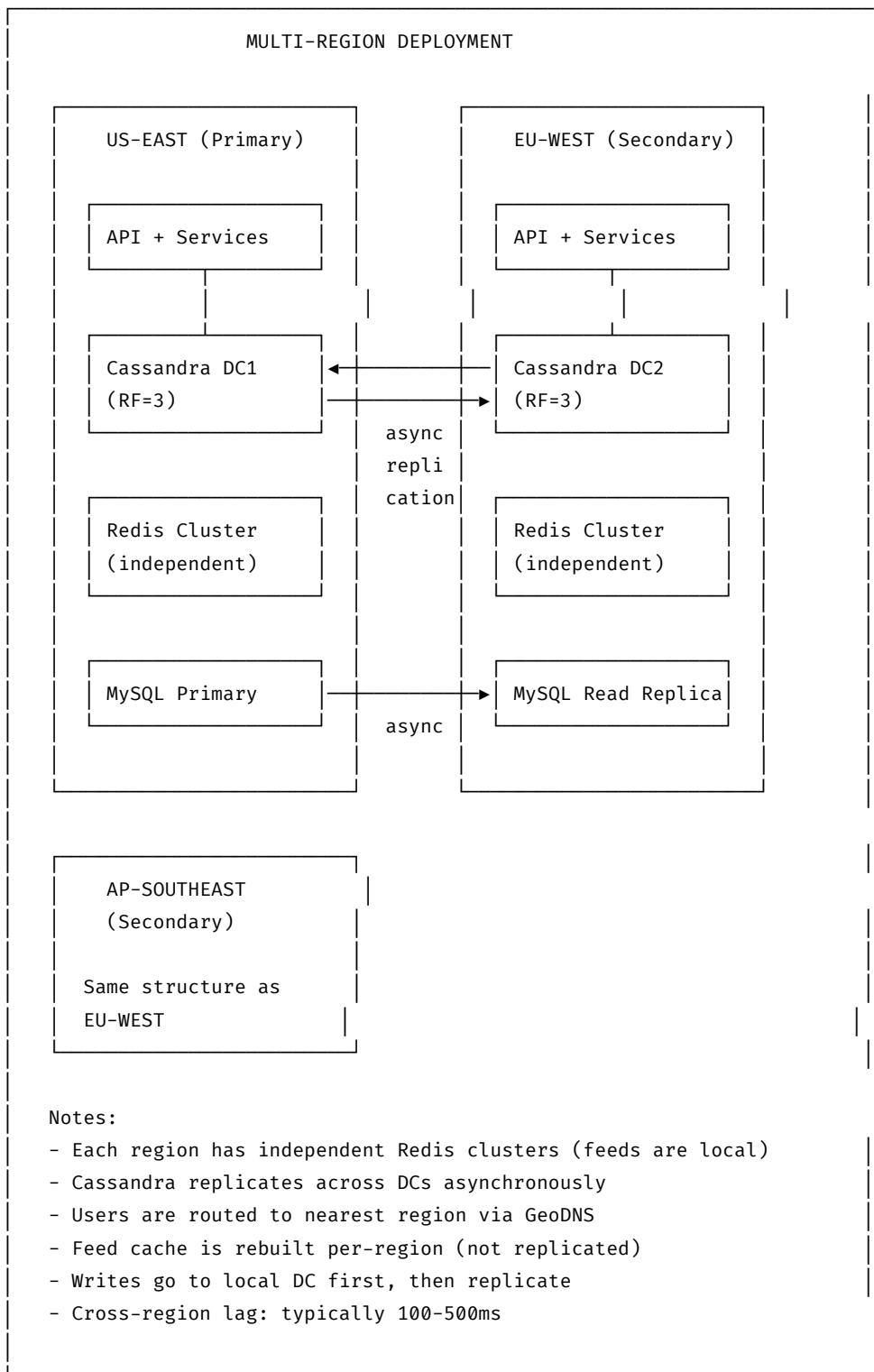
| CACHE INVALIDATION STRATEGIES                                                                               |                                                                                  |
|-------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Event                                                                                                       | Invalidation Action                                                              |
| New post created                                                                                            | Fan-out adds to followers' feed caches<br>No invalidation needed (additive)      |
| Post deleted                                                                                                | ZREM feed:{follower_id} post_id<br>DEL post:{post_id}<br>(async via Kafka event) |
| Post edited                                                                                                 | Update post:{post_id} hash in Redis<br>Feed cache unchanged (only has post_id)   |
| User unfollowed                                                                                             | Remove unfollowed user's posts from<br>follower's feed cache (async scan + ZREM) |
| User blocked                                                                                                | Same as unfollow + add to block list                                             |
| Profile updated                                                                                             | DEL user:{user_id} (TTL will repopulate)                                         |
| Like/comment                                                                                                | INCR likes:{post_id}<br>(no feed cache change needed)                            |
| Pattern: Write-through for feed cache, write-behind for counters,<br>TTL-based expiry for user/post caches. |                                                                                  |

## 4.12 11. Replication and Consistency

### 4.12.1 Consistency Model Per Data Store

| CONSISTENCY AND REPLICATION     |                                                 |                                                        |                                               |
|---------------------------------|-------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|
| Data Store                      | Consistency                                     | Replication                                            | Notes                                         |
| Posts<br>(Cassandra)            | LOCAL_QUORUM<br>(write)<br>LOCAL_ONE<br>(read)  | RF=3 per DC<br>3 DCs                                   | Strong within DC,<br>eventually across<br>DCs |
| Feed Cache<br>(Redis)           | Eventual                                        | 1 primary +<br>2 replicas                              | Feed is a view,<br>can be rebuilt             |
| Social Graph<br>(MySQL)         | Strong<br>(single DC)<br>Eventual<br>(cross DC) | Primary +<br>2 replicas<br>+ cross-DC<br>async replica | Follow/unfollow<br>must be correct            |
| User Profiles<br>(MySQL/Aurora) | Strong                                          | Primary +<br>2 read<br>replicas                        | Account data is<br>critical                   |
| Media<br>(S3)                   | Eventual                                        | S3 cross-<br>region<br>replication                     | Immutable; once<br>uploaded, never<br>changes |

### 4.12.2 Multi-Region Architecture



### 4.12.3 Read-After-Write Consistency

A critical requirement: when a user creates a post, they must see it in their own feed immediately (read-your-writes consistency). Solutions:

1. **Optimistic UI:** Client adds the post to local feed immediately before server confirms.
2. **Sticky sessions:** Route the user to the same DC/replica that processed the write.

3. **Direct injection:** After successful write, the Post Service directly returns the post object. The client injects it into the local feed view.
  4. **Write to own feed synchronously:** In the fan-out step, always write to the author's own feed cache synchronously (before async fan-out to followers).
- 

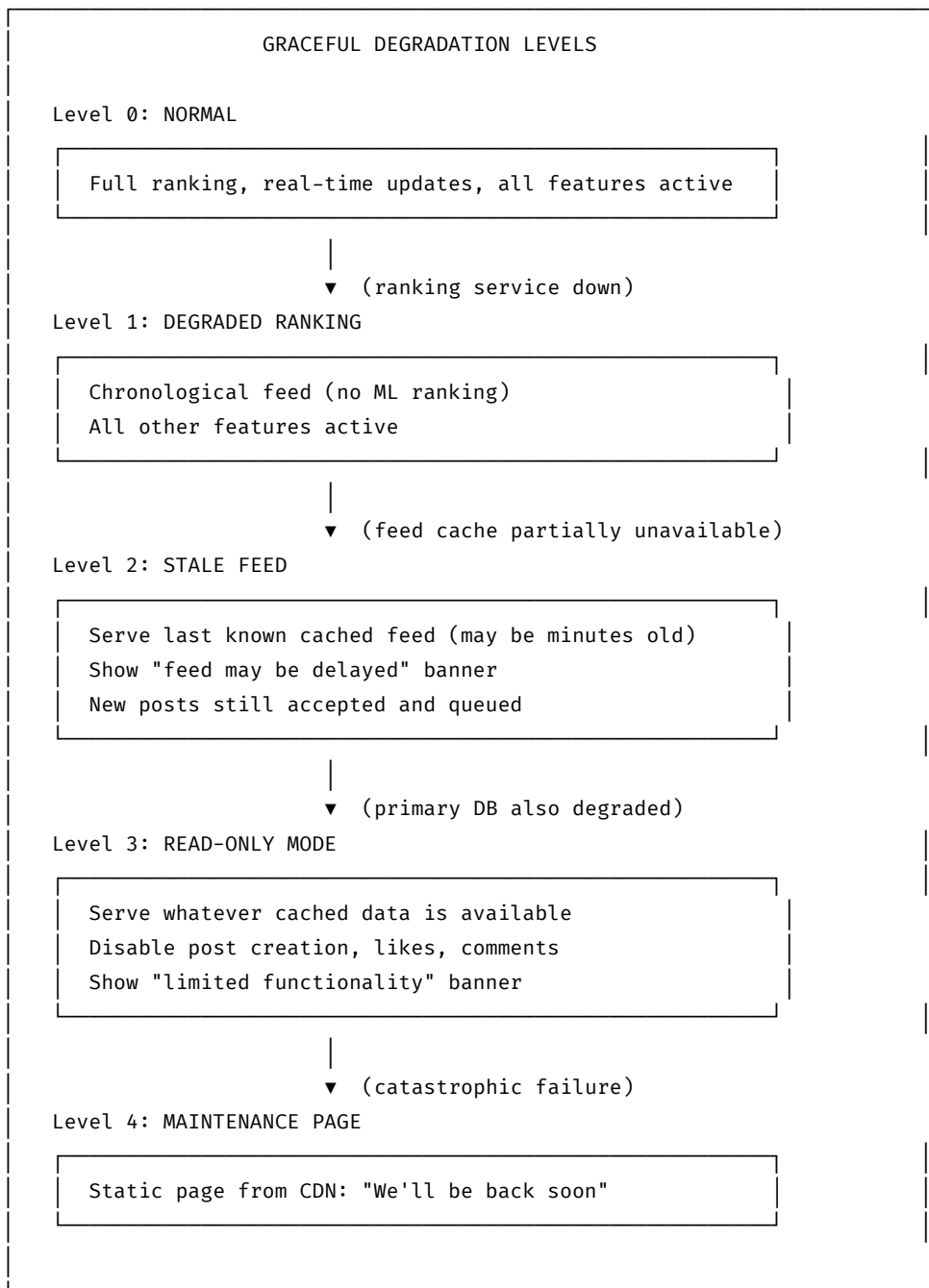
## 4.13 12. Fault Tolerance and Failure Handling



### 4.13.1 Failure Scenarios and Mitigations

| FAILURE SCENARIOS AND HANDLING     |                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure                            | Mitigation                                                                                                                                                                                                                                           |
| Redis feed cache node dies         | Regenerate from DB on-demand:<br>1. Get following list<br>2. Fetch recent posts from each<br>3. Merge, store in new Redis node<br>Cost: ~500ms for first request, then cached<br>Redis replicas auto-promote (Sentinel/<br>Cluster failover in ~15s) |
| Kafka broker fails                 | Kafka replication (RF=3) handles this.<br>Consumer group rebalancing.<br>Fan-out paused but not lost.<br>Feeds become stale until Kafka recovers.                                                                                                    |
| Fan-out worker crashes mid-process | Kafka retains messages until consumed.<br>Worker restart resumes from last committed offset. Some feeds may get duplicate entries (idempotent ZADD handles this).                                                                                    |
| Cassandra node failure             | RF=3 with LOCAL_QUORUM ensures reads/<br>writes continue with 2/3 nodes available.<br>Hinted handoff for temporary failures.<br>Anti-entropy repair for longer outages.                                                                              |
| Ranking service unavailable        | Fall back to chronological ordering.<br>Set circuit breaker with 5s timeout.<br>Degraded but functional experience.                                                                                                                                  |
| MySQL (social graph) primary fails | Promote read replica to primary.<br>Follower lists served from Redis cache during failover (~30s).                                                                                                                                                   |
| Complete region outage             | GeoDNS routes traffic to another region.<br>Users see slightly stale feeds.<br>Writes queued and replicated when region comes back.                                                                                                                  |

### 4.13.2 Degraded Mode Strategy



### 4.13.3 Idempotency in Fan-out

Fan-out operations must be idempotent because Kafka consumers can receive the same message multiple times (at-least-once delivery):

Operation: ZADD feed:{follower\_id} <timestamp> <post\_id>

ZADD is naturally idempotent:

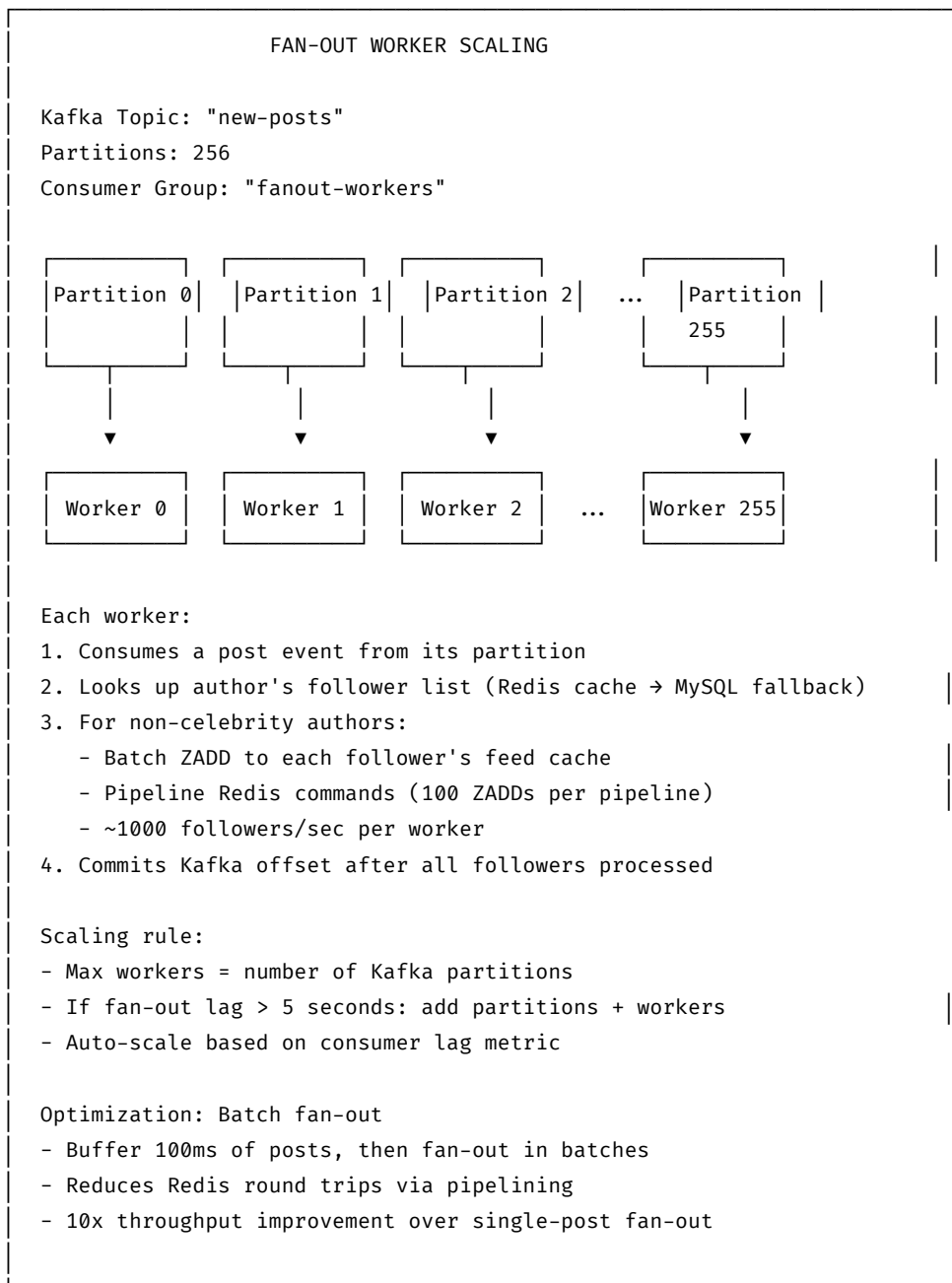
- If post\_id already exists in the sorted set, the score is updated.
- If it doesn't exist, it is added.
- Running ZADD twice with the same args = same result.
- This makes the fan-out worker safe to retry without side effects.

### 4.14 13. Scalability

#### 4.14.1 Horizontal Scaling Strategy

| SCALABILITY DIMENSIONS |                          |              |
|------------------------|--------------------------|--------------|
| Component              | Scaling Strategy         | Scale Target |
| API Gateway            | Horizontal (stateless)   | 50-200 nodes |
| Post Service           | Horizontal (stateless)   | 100+ nodes   |
| Feed Service           | Horizontal (stateless)   | 200+ nodes   |
| Fan-out Workers        | Horizontal (Kafka CG)    | 500-2000     |
| Ranking Service        | GPU-backed, horizontal   | 100+ nodes   |
| Redis Cluster          | Add shards + replicas    | 100+ nodes   |
| Cassandra              | Add nodes to ring        | 200+ nodes   |
| MySQL                  | Add shards (Vitess)      | 256+ shards  |
| Kafka                  | Add brokers + partitions | 50+ brokers  |

### 4.14.2 Fan-out Worker Scaling



### 4.14.3 Handling Traffic Spikes (Viral Posts)

#### VIRAL POST HANDLING

Scenario: A post goes viral -- 1M likes in 1 hour

Problem: Counter hotspot on single Redis key likes:{post\_id}

Solution: Distributed counters

Instead of: INCR likes:post\_abc

Use N counter shards:

INCR likes:post\_abc:shard\_0

INCR likes:post\_abc:shard\_1

...

INCR likes:post\_abc:shard\_15

Read: SUM(all shards) -- mget + sum

Write: random shard assignment

16 shards → 16x write throughput

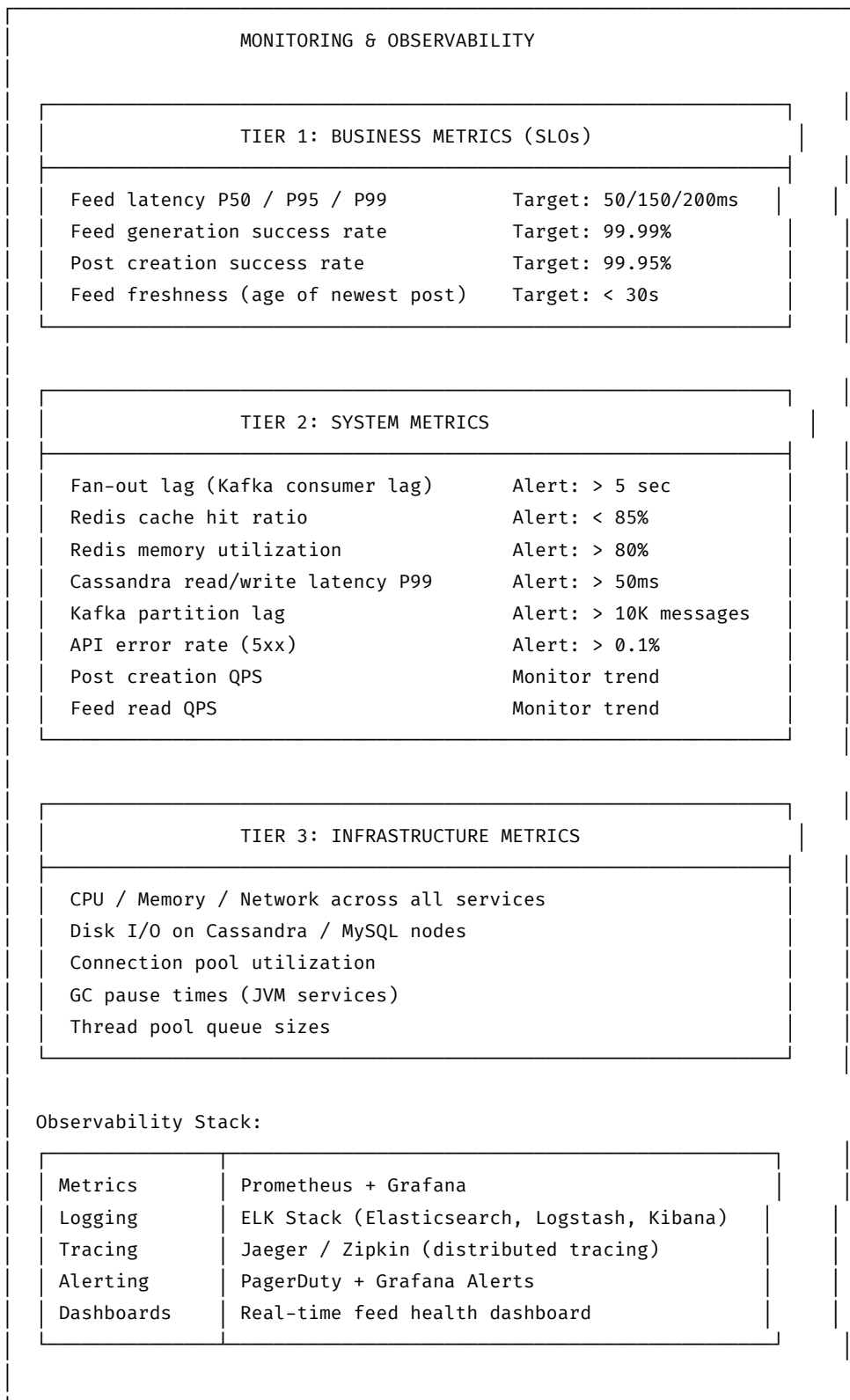
Read cost: 16 gets instead of 1 (still fast with mget)

Additional mitigation:

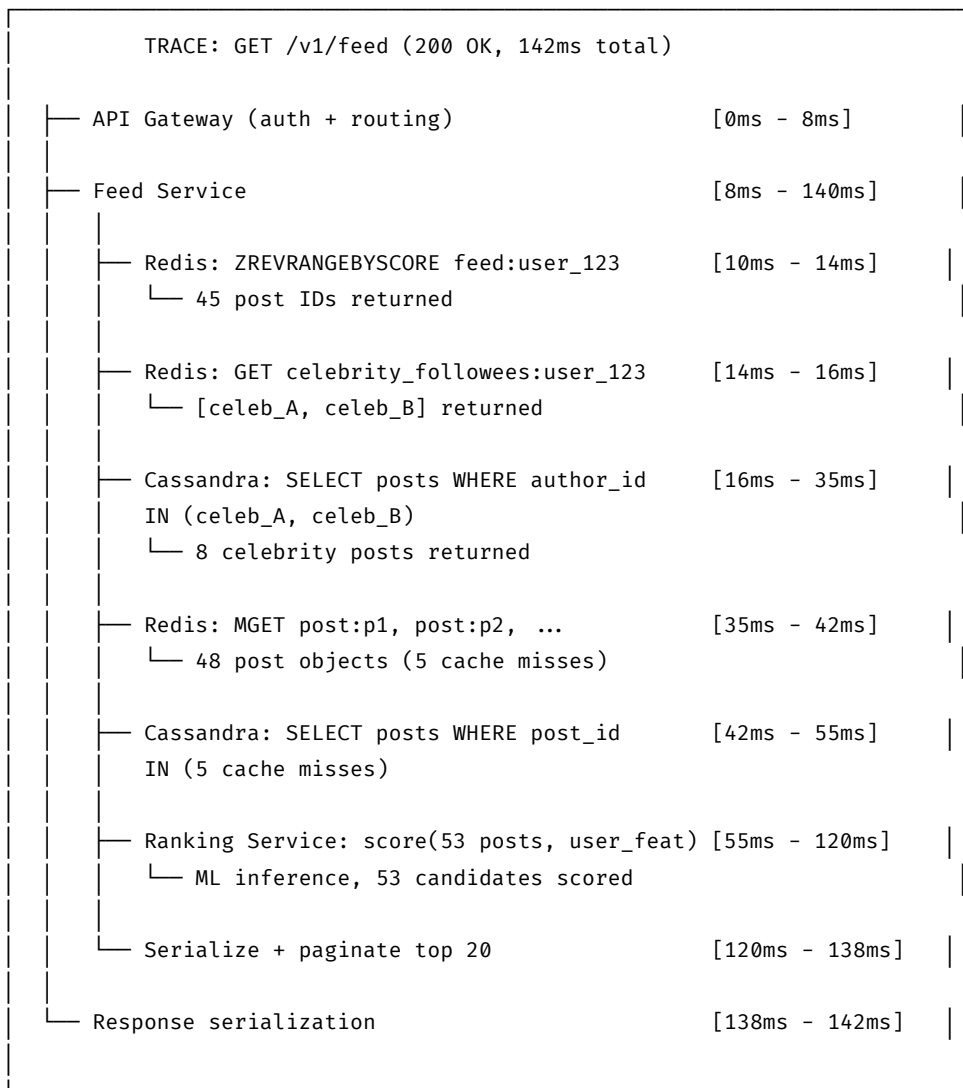
- Rate limit like/comment API per user (200/hour)
- Circuit breaker on fan-out for viral posts
- Pre-warm popular posts in all cache layers
- Async counter sync to DB (batch every 30s)

## 4.15 14. Monitoring and Observability

### 4.15.1 Key Metrics Dashboard



### 4.15.2 Distributed Tracing Example



## 4.16 15. Trade-offs and Design Decisions

### 4.16.1 Decision 1: Fan-out Strategy

DECISION: Use hybrid fan-out (push for normal, pull for celebrities)

Alternatives Considered:

| Option                       | Rejected? | Reason                                                                   |
|------------------------------|-----------|--------------------------------------------------------------------------|
| Pure push (fan-out on write) | Yes       | Celebrity problem makes this $O(100M)$ per post. Unsustainable at scale. |
| Pure pull (fan-out on read)  | Yes       | Every feed read requires querying 300 partitions. P99 latency too high.  |
| Hybrid (selected)            | No        | Best of both: fast reads + manageable writes. Complexity is worth it.    |

### 4.16.2 Decision 2: Feed Ranking

DECISION: ML-based ranking with chronological fallback

|                              |                                                                                 |
|------------------------------|---------------------------------------------------------------------------------|
| Chronological                | Simpler, transparent, but lower engagement and important posts get buried.      |
| ML-ranked (selected)         | Higher engagement (+15-30%), but opaque, requires ML infra, cold start problem. |
| User-toggable (nice to have) | Best UX but doubles serving complexity. Twitter offers this as an option.       |

Fallback: If ranking service is down, serve chronological feed. This is a circuit-breaker-protected degradation.



### 4.16.3 Decision 3: Pre-compute vs On-Demand Feed

DECISION: Pre-compute feeds in Redis with on-demand fallback

| Dimension          | Pre-computed        | On-Demand    |
|--------------------|---------------------|--------------|
| Read latency       | ~5ms (Redis ZRANGE) | ~200-500ms   |
| Write cost         | Fan-out per post    | None         |
| Storage            | ~6 TB Redis         | Minimal      |
| Freshness          | Near real-time      | Always fresh |
| Wasted computation | Inactive users      | None         |
| Cache miss penalty | High (rebuild feed) | N/A          |

Selected: Pre-computed with on-demand rebuild as fallback

Rationale: Read-heavy workload (5B reads vs 1B writes/day)

The 6 TB Redis cost is justified by the 100x read latency improvement.

### 4.16.4 Decision 4: Database Selection for Posts

DECISION: Cassandra for posts (over DynamoDB, MongoDB, PostgreSQL)

|                         |                                                                                                                                                                                                                                                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Cassandra<br>(selected) | <ul style="list-style-type: none"> <li>✓ Linear horizontal scaling</li> <li>✓ Tunable consistency (LOCAL_QUORUM)</li> <li>✓ Time-series friendly (clustering by timestamp)</li> <li>✓ Multi-DC replication built-in</li> <li>✗ No secondary indexes (need materialized views)</li> <li>✗ Operational complexity</li> </ul> |
| DynamoDB                | <ul style="list-style-type: none"> <li>✓ Managed (less ops)</li> <li>✓ Auto-scaling</li> <li>✗ AWS vendor lock-in</li> <li>✗ Expensive at this scale</li> </ul>                                                                                                                                                            |
| PostgreSQL              | <ul style="list-style-type: none"> <li>✓ Rich queries, ACID</li> <li>✗ Does not scale writes horizontally</li> <li>✗ Cannot handle 35K writes/sec natively</li> </ul>                                                                                                                                                      |

### 4.16.5 Decision 5: Cursor-Based vs Offset-Based Pagination

- **Stability:** Cursor-Based (selected): Stable on inserts; Offset-Based: Rows shift on inserts
- **Performance:** Cursor-Based (selected):  $O(1)$  with index; Offset-Based:  $O(\text{offset})$  scan
- **Infinite scroll:** Cursor-Based (selected): Perfect fit; Offset-Based: Works but fragile

- **Jump to page N:** Cursor-Based (selected): Not supported; Offset-Based: Supported
- **Cache-friendliness:** Cursor-Based (selected): High (immutable cursor); Offset-Based: Low (offset changes)

**Selected:** Cursor-based because feeds are infinite-scroll UIs where jump-to-page is not needed, and data changes constantly.

---

## 4.17 16. Interview Deep-Dive Questions

### 4.17.1 Q1: How do you handle the celebrity problem?

**Answer:** Use the hybrid fan-out approach. Maintain a threshold (e.g., 10K followers). Users below the threshold: fan-out on write (push their posts to all followers' feed caches). Users above the threshold: fan-out on read (store the post only, and merge it into the feed at read time). The `is_celebrity` flag on the user record controls the routing. At read time, the Feed Service fetches the user's pre-computed feed from Redis, then separately pulls recent posts from the ~5-10 celebrities the user follows, merges them, ranks, and returns. The extra 5-10 Cassandra queries add ~20ms, well within the 200ms budget.

---

### 4.17.2 Q2: What happens when a user unfollows someone?

**Answer:** Three things must happen: 1. **Social graph update:** Delete the follow edge in MySQL (or soft-delete). 2. **Cache invalidation:** Invalidate the follower list cache in Redis for both users. 3. **Feed cleanup (async):** A Kafka event triggers a worker to scan the unfollower's feed cache and remove all `post_ids` from the unfollowed user. This is done via `ZRANGEBYSCORE + ZREM` and can be deferred (seconds-level delay is acceptable).

If the unfollowed user is a celebrity, no feed cleanup is needed because their posts were never pushed – they are simply no longer pulled at read time.

---

### 4.17.3 Q3: How do you handle deleted or hidden posts in pre-computed feeds?

**Answer:** Two strategies work together: 1. **Lazy deletion:** When a post is deleted, set `is_deleted = true` in Cassandra and publish a deletion event to Kafka. Fan-out workers remove the `post_id` from all followers' feed caches (same as fan-out but in reverse – this is also a push operation). Since feeds only store `post_ids`, the actual deletion is a simple `ZREM`. 2. **Read-time filtering:** As a safety net, when hydrating `post_ids` into full objects, check the `is_deleted` flag. If true, skip the post. This handles the window between deletion and cache cleanup.

---

### 4.17.4 Q4: How do you rank posts for new users with no interaction history?

**Answer:** Cold start problem. Solutions: 1. **Default ranking signals:** Use non-personalized signals: post recency, global engagement rate (likes/impressions), author popularity, media type preferences from demographic data. 2. **Explore/exploit:** Show a diverse mix of content types and track what the new user interacts with. After ~20-30 interactions, the model starts personalizing. 3. **Onboarding flow:** Ask users to pick topics/interests during signup. Use these as initial features. 4. **Social**

**bootstrapping:** If the user imported contacts or follows some accounts, use “users similar to you liked these posts” collaborative filtering.

---

#### 4.17.5 Q5: How do you prevent feed staleness?

**Answer:** Multiple mechanisms: 1. **Pull-refresh:** Client polls every 30-60 seconds (or on user scroll-to-top). 2. **WebSocket/SSE:** Maintain a persistent connection for real-time updates. When a new post is added to the user’s feed cache, send a “new posts available” signal. The client shows a “N new posts” banner. 3. **Feed TTL:** Feed cache entries have TTL. If a user is inactive for 7 days, the feed cache expires and is rebuilt on next visit. 4. **Background refresh:** For highly active users, a background job re-ranks their feed every few minutes to ensure the “best” posts float to the top.

---

#### 4.17.6 Q6: How does the system handle a Kafka outage? Will users not see new posts?

**Answer:** During a Kafka outage: 1. **Post creation still works:** Posts are written to Cassandra synchronously. The Kafka publish is async. If Kafka is down, the post is persisted but fan-out is paused. 2. **Feeds become stale:** New posts from non-celebrity users won’t appear in followers’ feeds until Kafka recovers and fan-out catches up. 3. **Fallback:** The Feed Service can detect fan-out lag. If lag exceeds a threshold (e.g., 5 minutes), it temporarily switches to pull-based feed generation for all users – querying Cassandra directly for recent posts from followed users. 4. **Recovery:** When Kafka recovers, consumer group rebalancing resumes processing from the last committed offset. Backlog is processed at accelerated speed.

---

#### 4.17.7 Q7: How do you handle pagination when the feed is being modified concurrently?

**Answer:** Cursor-based pagination with composite cursor = (timestamp, post\_id). When the user requests the next page, the cursor points to the exact position in the sorted set. Even if new posts are inserted (higher timestamps), the cursor is stable because it uses `ZREVRANGEBYSCORE (cursor_ts) -inf LIMIT 0 20` – it always fetches posts *older* than the cursor. New posts appear when the user scrolls back to the top, not in the middle of their scroll. This gives a consistent reading experience.

---

#### 4.17.8 Q8: What if a user follows 10,000 accounts? How does fan-out on read scale?

**Answer:** In the hybrid model, only celebrity posts are pulled at read time. A user following 10,000 accounts might follow ~50-100 celebrities (10K+ follower threshold). The pull query fetches recent posts from each celebrity’s timeline in parallel (50-100 Cassandra queries). With async I/O and connection pooling, this completes in ~30-50ms. Posts from the other 9,900+ non-celebrity accounts are already in the pre-computed feed cache via push. The merge step combines both sources and ranks them.

If even 100 pull queries is too slow, we can cache the “merged celebrity feed” for each user with a short TTL (~60 seconds), avoiding repeated Cassandra hits.

---

#### 4.17.9 Q9: How do you handle media (images/videos) in posts?

**Answer:** Media is handled separately from text to avoid bloating the feed pipeline: 1. **Upload:** Client uploads media to the Media Service *before* creating the post. The Media Service stores the file in S3 and returns a `media_id`. 2. **Transcoding:** An async pipeline (triggered via SQS/Kafka) transcodes images to multiple sizes (thumbnail, medium, full) and videos to multiple bitrates. Stored in S3 with deterministic URLs. 3. **CDN:** All media is served via CloudFront/Akamai. URLs are immutable (content- addressed), so cache TTLs can be infinite. 4. **Feed integration:** The post object only stores `media_ids`. At hydration time, the Feed Service resolves `media_ids` to CDN URLs (simple lookup, cacheable). 5. **Lazy loading:** The client renders text first, then loads media progressively.

---

#### 4.17.10 Q10: How do you implement trending topics?

**Answer:** Trending is computed from a real-time event stream: 1. **Event stream:** Every post creation publishes hashtags to a Kafka topic `post-events`. 2. **Sliding window aggregation:** A Flink/Spark Streaming job maintains a sliding window (e.g., last 1 hour, 4 hours, 24 hours) counting hashtag occurrences. 3. **Trend detection:** A hashtag is “trending” if its current frequency significantly exceeds its historical baseline (z-score > 2.0). This filters out always-popular tags like #love and surfaces genuinely spiking topics. 4. **Regional trending:** Trending is computed per-region using the user’s location from the post metadata. 5. **Caching:** Trending lists are computed every 5 minutes and cached in Redis with a 5-minute TTL. No real-time requirement – a few minutes of staleness is fine.

---

#### 4.17.11 Q11: How do you handle a user who has been inactive for months and returns?

**Answer:** Their feed cache has likely expired (7-day TTL). On their first request: 1. **Cache miss detected:** Redis returns empty for `feed:{user_id}`. 2. **On-demand rebuild:** Feed Service fetches the user’s following list, queries recent posts from each followed user (last 24-48 hours), merges them, and stores the result in Redis. 3. **Performance:** This rebuild takes 200-500ms (higher than the normal 50ms), but it only happens once. Subsequent requests are served from cache. 4. **Optimization:** For the initial load, show the top 20 posts from the rebuild immediately while the full 500-post cache is populated asynchronously.

---

#### 4.17.12 Q12: How do you prevent spam in the news feed?

**Answer:** Multi-layered approach: 1. **Rate limiting:** Post creation capped at 50/hour per user. 2. **Content filtering:** NLP-based spam detection at post creation time (block known spam patterns, URLs, excessive hashtags). 3. **Behavioral signals:** Accounts that post identical content, follow/unfollow rapidly, or have low engagement ratios are flagged. 4. **User reporting:** “Report spam” adds negative signals to the ranking model, reducing the post’s visibility. 5. **Throttled fan-out:** New accounts have limited fan-out (posts not pushed to all followers until the account is N days old or verified).

---

#### 4.17.13 Q13: How would you add “Stories” (ephemeral content) to this system?

**Answer:** Stories differ from feed posts in two key ways: they expire after 24 hours and are displayed in a separate UI (horizontal scroll). The architecture adapts: 1. **Separate storage:** Stories go in a Redis sorted set per user with TTL=24h. No Cassandra persistence needed. 2. **Separate feed:** Stories feed is a simple aggregation of followed users who have active stories. No ranking needed – just chronological per-user. 3. **Fan-out:** Same hybrid approach. Normal users push, celebrities pull. 4. **Compact payload:** Stories are just media\_ids + timestamps. Very lightweight. 5. **View tracking:** Track which stories a user has seen (Redis bitmap per user per day) to show the unread indicator.

---

#### 4.17.14 Q14: How do you ensure data consistency when the same post is stored in Cassandra and cached in Redis?

**Answer:** The post in Cassandra is the source of truth. Redis caches are derived views. 1. **Write path:** Write to Cassandra first (synchronous), then update Redis (async). If Redis update fails, the cache has stale data but will expire via TTL. 2. **Read path:** Always try Redis first (fast). On cache miss, read from Cassandra and populate Redis. 3. **Inconsistency window:** Between a Cassandra write and Redis update, there is a brief window (~10-100ms) where Redis has stale data. This is acceptable for a feed system (eventual consistency). 4. **Counter consistency:** Like/comment counts are incremented in Redis atomically and async-synced to Cassandra every 30 seconds. The Redis value is the “hot” value shown to users. If Redis loses data, the last-synced Cassandra value is used.

---

#### 4.17.15 Q15: How would you design the system differently if the requirement changed to a bidirectional “friend” model (like Facebook) instead of unidirectional “follow” (like Twitter)?

**Answer:** Key differences: 1. **Social graph:** The follows table becomes symmetric. A friendship is a single edge, not two directional edges. This halves storage but adds complexity for friend requests (pending/accepted states). 2. **Fan-out volume:** Friend counts are typically lower than follower counts (avg 300 friends vs. potentially millions of followers). This makes pure fan-out on write more viable – no celebrity problem if friend count is capped at ~5,000. 3. **Privacy:** Every post has an implicit “friends only” visibility. No public posts by default. This simplifies ranking (no need to handle public viral content). 4. **Feed diversity:** With fewer sources (~300 friends vs. potentially 10K+ follows), feed diversity is lower. Ranking must balance showing enough from each friend without being repetitive. 5. **Fan-out strategy:** Pure push becomes feasible because max friends ~5K means max 5K writes per post. No need for the hybrid model.

---

## 4.18 Summary

### NEWS FEED SYSTEM: KEY TAKEAWAYS

1. HYBRID FAN-OUT is the production-grade solution.  
Push for normal users, pull for celebrities.
2. PRE-COMPUTED FEEDS in Redis give sub-10ms reads.  
On-demand rebuild as fallback for cache misses.
3. ML-BASED RANKING maximizes engagement.  
Chronological as fallback for ranking service failures.
4. CASSANDRA for posts (write throughput + multi-DC).  
Redis for caches. MySQL for relational data.
5. KAFKA decouples post creation from fan-out.  
Enables async, scalable, retryable fan-out.
6. EVENTUAL CONSISTENCY is acceptable for feeds.  
Strong consistency only for post writes and account data.
7. GRACEFUL DEGRADATION through circuit breakers.  
Always serve something, even if stale.
8. The CELEBRITY PROBLEM is the defining challenge.  
Everything else is standard distributed systems.

*Last updated: 2026-04-09*

## Chapter 5

# Design a Video Streaming Platform (YouTube / Netflix)

**Difficulty:** Hard | **Companies:** Google, Netflix, Amazon, Meta, Apple

---

### 5.1 1. Problem Statement and Scope

Design a globally distributed video streaming platform that supports video upload, transcoding, storage, and adaptive bitrate streaming to hundreds of millions of concurrent viewers. The system must handle the full lifecycle of video content – from the moment a creator uploads a raw file to the instant a viewer presses play on any device, anywhere in the world.

#### 5.1.1 In Scope

- Video upload (resumable, chunked) with progress tracking
- Video transcoding pipeline (multiple resolutions and codecs)
- On-demand video streaming with adaptive bitrate (ABR)
- Video search, discovery, and recommendation
- User interactions: likes, comments, subscriptions, playlists
- Content moderation and copyright detection
- Live streaming (extension)
- Analytics and view counting
- Global CDN distribution

#### 5.1.2 Out of Scope

- Payment/subscription billing (Stripe/payment gateway integration)
  - Detailed ad-serving pipeline (ad tech is its own system)
  - Creator Studio analytics dashboard (BI system)
  - Social features beyond comments (messaging, community posts)
-

## 5.2 2. Functional Requirements

- **1 - Upload videos:** Creators upload videos up to 12 hours / 256GB via resumable upload
  - **2 - Stream/watch videos:** Viewers watch on-demand with instant playback start
  - **3 - Adaptive bitrate streaming:** Automatic quality adjustment based on network conditions
  - **4 - Video quality selection:** Manual override to choose 240p through 4K
  - **5 - Search videos:** Full-text search on titles, descriptions, tags, captions
  - **6 - Like / dislike videos:** Engagement signals stored per user per video
  - **7 - Comment on videos:** Threaded comments with pagination
  - **8 - Subscribe to channels:** Follow creators, get feed updates
  - **9 - Recommendations:** Personalized home feed and “up next” suggestions
  - **10 - Playlists:** User-created ordered video collections
  - **11 - Video thumbnails:** Auto-generated + custom upload thumbnails
  - **12 - Live streaming:** Real-time broadcast to large audiences
  - **13 - Content moderation:** Automated + manual review for policy violations
  - **14 - Copyright detection:** Content ID / fingerprinting for copyrighted material
- 

## 5.3 3. Non-Functional Requirements

- **Playback start latency:** < 2 seconds (time to first frame)
  - **Rebuffering ratio:** < 0.5% of total playback time
  - **Availability:** 99.99% (< 52.6 minutes downtime/year)
  - **Upload success rate:** > 99.5% for files under 10GB
  - **Global distribution:** Content served from edge within 50ms RTT of 95% of users
  - **Scale:** 100M+ daily video views, 800M DAU
  - **Transcoding latency:** < 30 minutes for a 1-hour video (all resolutions)
  - **Search latency:** < 200ms p99
  - **Storage durability:** 99.999999999% (11 nines, matching S3)
  - **Content moderation:** Automated scan within 10 minutes of upload
  - **Consistency:** Eventual consistency acceptable for views/likes; strong for uploads
- 

## 5.4 4. Back-of-Envelope Estimation

### 5.4.1 Traffic Estimates

|                          |                                                                        |
|--------------------------|------------------------------------------------------------------------|
| DAU:                     | 800M users                                                             |
| Videos watched/user/day: | 5                                                                      |
| Total daily views:       | $800M * 5 = 4 \text{ Billion views/day}$                               |
| Views per second:        | $4B / 86400 = \sim 46,000 \text{ views/sec}$                           |
| Peak views/sec:          | $\sim 46K * 3 \text{ (peak multiplier)} = \sim 140K \text{ views/sec}$ |
| Uploads per day:         | 500K new videos/day                                                    |
| Uploads per second:      | $500K / 86400 = \sim 6 \text{ uploads/sec}$                            |
| Peak uploads/sec:        | $\sim 18 \text{ uploads/sec}$                                          |



### 5.4.2 Storage Estimates

Average raw upload size: 1 GB  
 Transcoded output per video: 5 resolutions \* avg 60MB each = 300MB total  
 (240p: 15MB, 360p: 30MB, 480p: 50MB,  
 720p: 80MB, 1080p: 125MB)  
 4K videos (10% of uploads): Additional 500MB per 4K video

Daily new storage:  
 Base:  $500K * 300MB = 150 \text{ TB/day}$   
 4K:  $50K * 500MB = 25 \text{ TB/day}$   
 Thumbnails:  $500K * 5 * 50KB = 125 \text{ GB/day}$  (5 thumbnails per video)  
 Total:  $\sim 175 \text{ TB/day}$

Annual storage:  $175 \text{ TB} * 365 = \sim 64 \text{ PB/year}$   
 After 5 years:  $\sim 320 \text{ PB total}$

### 5.4.3 Bandwidth Estimates

Average video duration: 3 minutes  
 Average bitrate served: 5 Mbps (mix of qualities)  
 Data per view:  $5 \text{ Mbps} * 180 \text{ sec} / 8 = \sim 112 \text{ MB per view}$

Daily egress:  $4B * 112 \text{ MB} = \sim 448 \text{ PB/day}$   
 Peak egress bandwidth:  $448 \text{ PB} / 86400 * 3 \text{ (peak)} = \sim 155 \text{ Tbps peak}$

CDN cache hit ratio target:  $> 95\%$   
 Origin bandwidth:  $155 \text{ Tbps} * 0.05 = \sim 7.8 \text{ Tbps from origin}$

### 5.4.4 Compute Estimates (Transcoding)

Transcoding time per video:  
 1 hour video, single resolution =  $\sim 30 \text{ min}$  on modern hardware  
 5 resolutions =  $\sim 2.5 \text{ hours}$  of compute per video (parallelized to  $\sim 30 \text{ min}$ )

Daily transcoding compute:  
 $500K \text{ videos} * 2.5 \text{ hours} = 1.25M \text{ compute-hours/day}$   
 $= \sim 52,000$  concurrent transcoding workers (if each runs 24h)  
 With spot/preemptible: target  $\sim 20,000$  workers at peak

### 5.4.5 Summary Table

- **DAU:** Value: 800M
- **Daily video views:** Value: 4 Billion
- **Views/sec (avg / peak):** Value: 46K / 140K
- **Daily uploads:** Value: 500K videos
- **Daily new storage:** Value:  $\sim 175 \text{ TB}$
- **Daily egress bandwidth:** Value:  $\sim 448 \text{ PB}$
- **Peak egress:** Value:  $\sim 155 \text{ Tbps}$
- **CDN cache hit target:** Value:  $> 95\%$
- **Transcoding workers:** Value:  $\sim 20K$  concurrent

---

## 5.5 5. API Design

### 5.5.1 5.1 Video Upload

POST /v1/videos/upload/initiate

Headers:

Authorization: Bearer <token>

Content-Type: application/json

Body:

```
{
 "title": "My Video",
 "description": "A great video about ... ",
 "tags": ["tutorial", "coding"],
 "visibility": "public", // public | unlisted | private
 "file_size_bytes": 1073741824, // 1GB
 "file_type": "video/mp4",
 "thumbnail": "<base64 or URL>" // optional custom thumbnail
}
```

Response 201:

```
{
 "video_id": "v_abc123",
 "upload_url": "https://upload.streamcdn.com/v_abc123",
 "upload_token": "tok_xyz789",
 "chunk_size": 8388608, // 8MB recommended chunk size
 "expires_at": "2026-04-09T13:00:00Z"
}
```

PUT /v1/videos/upload/{video\_id}/chunks/{chunk\_number}

Headers:

Authorization: Bearer <token>

Content-Range: bytes 0-8388607/1073741824

Content-Type: application/octet-stream

X-Upload-Token: tok\_xyz789

Body: <binary chunk data>

Response 200:

```
{
 "chunk_number": 0,
 "bytes_received": 8388608,
 "total_received": 8388608,
 "status": "uploading" // uploading | processing | complete
}
```

```
POST /v1/videos/upload/{video_id}/complete
Headers:
 Authorization: Bearer <token>
Body:
{
 "upload_token": "tok_xyz789",
 "checksum_sha256": "a1b2c3d4 ..."
}
Response 200:
{
 "video_id": "v_abc123",
 "status": "processing",
 "estimated_ready_at": "2026-04-09T12:45:00Z"
}
```

### 5.5.2 5.2 Video Streaming

```
GET /v1/videos/{video_id}/manifest.m3u8
Headers:
 Authorization: Bearer <token> // optional for public videos
Response 200:
 Content-Type: application/vnd.apple.mpegurl
 #EXTM3U
 #EXT-X-STREAM-INF:BANDWIDTH=800000,RESOLUTION=640x360
 360p/playlist.m3u8
 #EXT-X-STREAM-INF:BANDWIDTH=1400000,RESOLUTION=854x480
 480p/playlist.m3u8
 #EXT-X-STREAM-INF:BANDWIDTH=2800000,RESOLUTION=1280x720
 720p/playlist.m3u8
 #EXT-X-STREAM-INF:BANDWIDTH=5000000,RESOLUTION=1920x1080
 1080p/playlist.m3u8

GET /v1/videos/{video_id}/segments/{quality}/{segment_number}.ts
Headers:
 Range: bytes=0-1048575 // optional range request
Response 200/206:
 Content-Type: video/MP2T
 <binary segment data>
```

### 5.5.3 5.3 Video Metadata and Interactions

```
GET /v1/videos/{video_id}
Response 200:
{
 "video_id": "v_abc123",
 "title": "My Video",
 "description": "...",
 "channel": { "id": "ch_456", "name": "TechCreator" },
 "duration_seconds": 624,
 "view_count": 1542387,
 "like_count": 45200,
 "published_at": "2026-04-01T10:00:00Z",
 "thumbnails": {
 "default": "https://cdn.stream.com/thumbs/v_abc123/default.jpg",
 "medium": "https://cdn.stream.com/thumbs/v_abc123/medium.jpg",
 "high": "https://cdn.stream.com/thumbs/v_abc123/high.jpg"
 },
 "available_qualities": ["240p", "360p", "480p", "720p", "1080p"],
 "tags": ["tutorial", "coding"]
}

GET /v1/videos/search?q=system+design&page=1&page_size=20&sort=relevance
POST /v1/videos/{video_id}/like // toggle like
DELETE /v1/videos/{video_id}/like // remove like
POST /v1/videos/{video_id}/comments // add comment
GET /v1/videos/{video_id}/comments?page=1&sort=top
GET /v1/users/{user_id}/feed?page=1 // subscription feed
GET /v1/videos/recommendations?video_id={id}&count=20 // related videos
GET /v1/home/feed?page=1 // personalized home feed
```

---

## 5.6 6. Data Model and Database Selection

### 5.6.1 6.1 Data Models

#### Video Metadata

```

videos {
 video_id VARCHAR(16) PK -- globally unique, base62 encoded
 creator_id VARCHAR(16) FK -- references users.user_id
 title VARCHAR(500)
 description TEXT
 duration_seconds INT
 status ENUM('uploading','processing','ready','failed','removed')
 visibility ENUM('public','unlisted','private')
 upload_timestamp TIMESTAMP
 publish_timestamp TIMESTAMP
 raw_file_url VARCHAR(1024) -- S3 path to original upload
 manifest_url VARCHAR(1024) -- S3 path to HLS manifest
 file_size_bytes BIGINT
 codec VARCHAR(32) -- h264, h265, vp9, av1
 resolution_max VARCHAR(16) -- highest available resolution
 language VARCHAR(8)
 category_id INT FK
 is_age_restricted BOOLEAN
 copyright_status ENUM('clear','claimed','blocked')
 created_at TIMESTAMP
 updated_at TIMESTAMP
}
INDEX: (creator_id, publish_timestamp), (status), (category_id, publish_timestamp)

```

## User

```

users {
 user_id VARCHAR(16) PK
 username VARCHAR(64) UNIQUE
 email VARCHAR(256) UNIQUE
 display_name VARCHAR(128)
 avatar_url VARCHAR(1024)
 subscriber_count BIGINT DEFAULT 0
 total_views BIGINT DEFAULT 0
 created_at TIMESTAMP
 updated_at TIMESTAMP
}

```

## Subscription

```

subscriptions {
 subscriber_id VARCHAR(16) PK -- user who subscribes
 channel_id VARCHAR(16) PK -- user being subscribed to
 subscribed_at TIMESTAMP
 notifications BOOLEAN DEFAULT true
}
INDEX: (channel_id, subscribed_at)

```

## Comment

```

comments {
 comment_id VARCHAR(16) PK
 video_id VARCHAR(16) -- partition key
 user_id VARCHAR(16)
 parent_id VARCHAR(16) NULL -- for threaded replies
 content TEXT
 like_count INT DEFAULT 0
 created_at TIMESTAMP
 updated_at TIMESTAMP
}
PARTITION KEY: video_id
CLUSTERING KEY: created_at DESC

```

### Engagement (Likes)

```

video_likes {
 user_id VARCHAR(16)
 video_id VARCHAR(16)
 like_type ENUM('like','dislike')
 created_at TIMESTAMP
 PRIMARY KEY (user_id, video_id)
}

```

### View Count (Redis + Async Flush)

```

-- Redis structure:
video:{video_id}:views -> INT (atomic increment)
video:{video_id}:views:daily -> SORTED SET { date -> count }

```

-- Periodic flush to persistent store:

```

video_view_counts {
 video_id VARCHAR(16) PK
 total_views BIGINT
 views_24h BIGINT
 views_7d BIGINT
 views_30d BIGINT
 last_updated TIMESTAMP
}

```

### Watch History

```

watch_history {
 user_id VARCHAR(16) -- partition key
 video_id VARCHAR(16)
 watched_at TIMESTAMP -- clustering key
 watch_duration INT -- seconds watched
 completed BOOLEAN
}

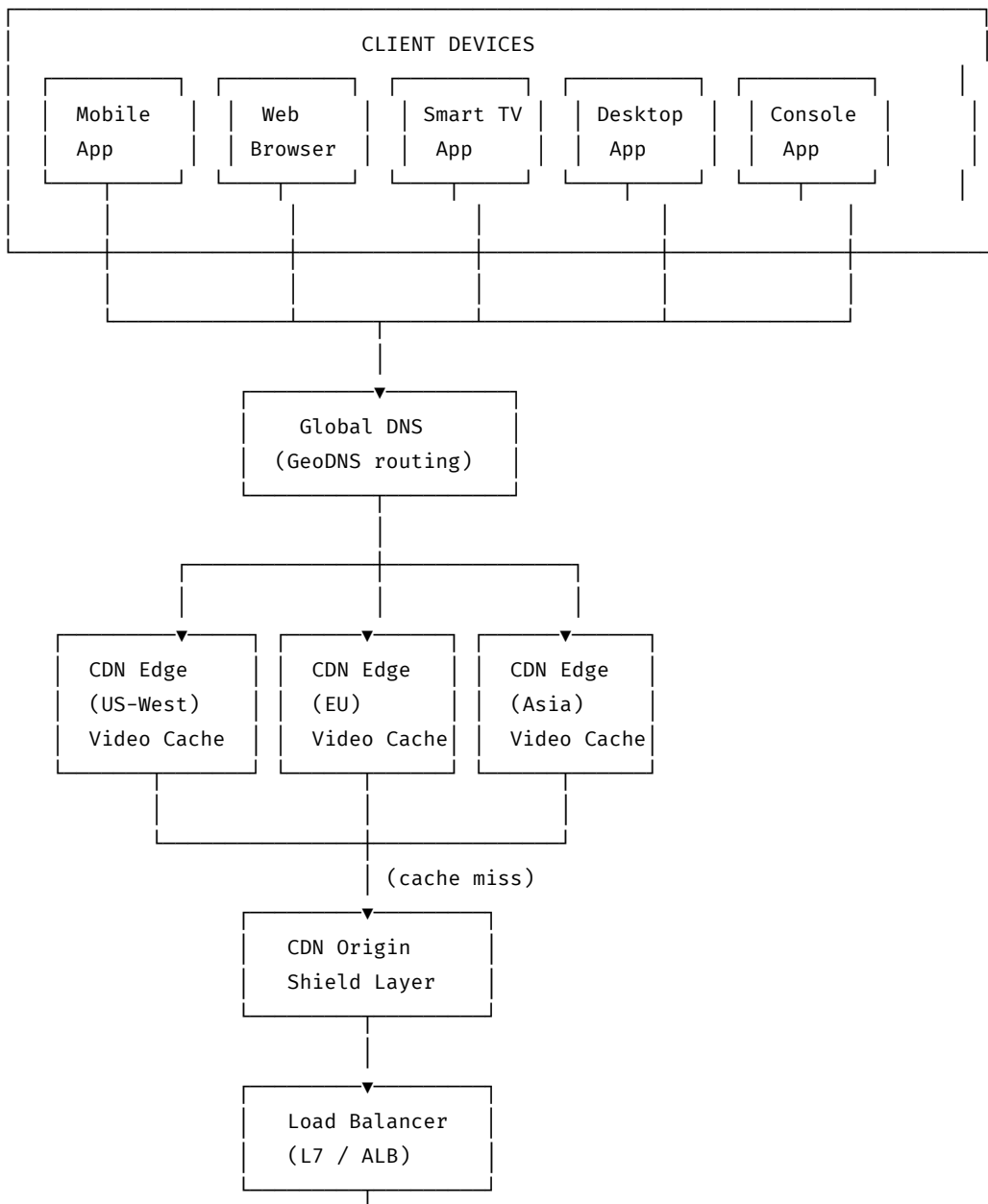
```

## 5.6.2 6.2 Database Selection

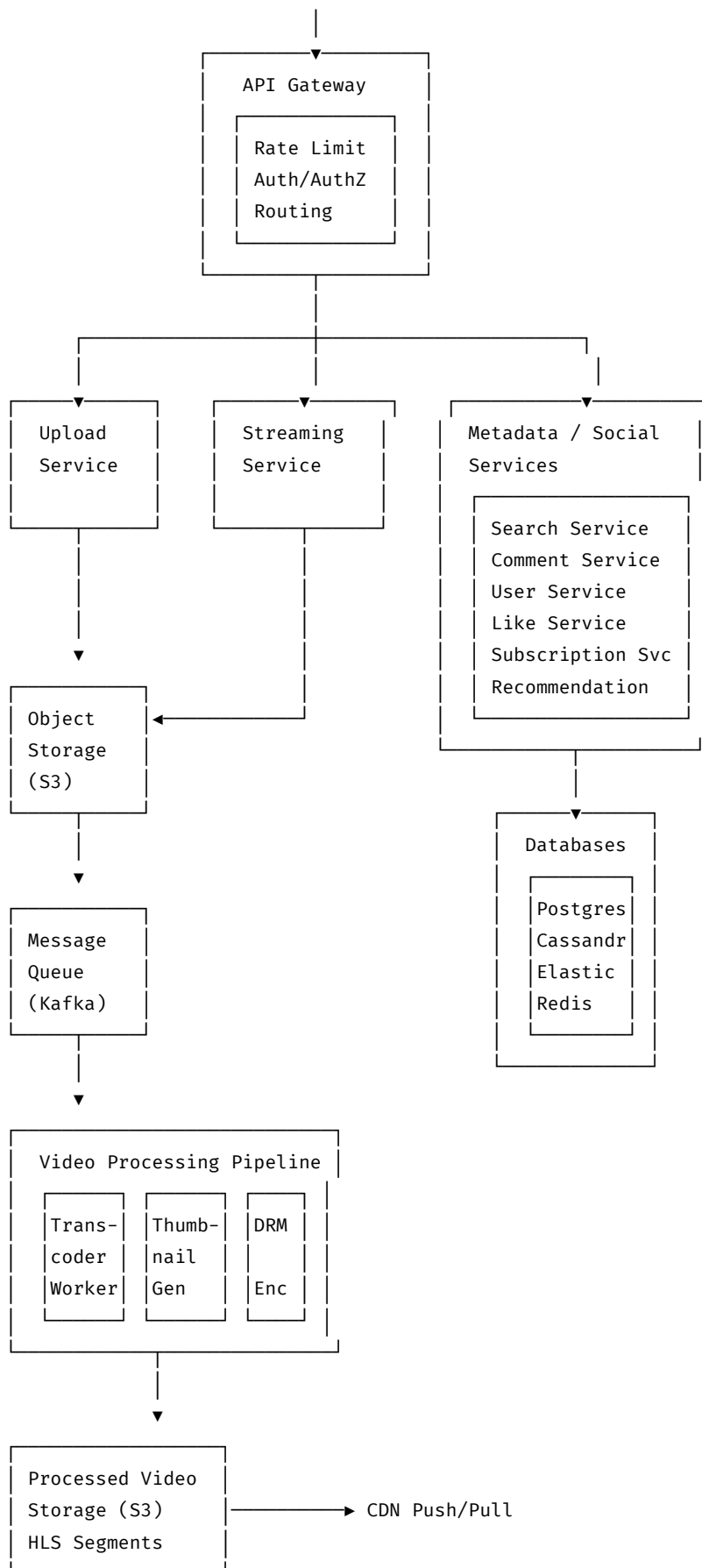
| Data Type            | Database           | Justification                    |
|----------------------|--------------------|----------------------------------|
| Video metadata       | PostgreSQL         | ACID, complex queries, joins     |
| User data            | PostgreSQL         | Relational integrity, auth       |
| Video files          | S3 / Blob Storage  | Scalable object store, 11-nines  |
| Thumbnails           | S3 + CDN           | Static assets, high read volume  |
| Comments             | Cassandra          | Write-heavy, partition by video  |
| Watch history        | Cassandra          | Append-heavy, time-series data   |
| Search index         | Elasticsearch      | Full-text search, faceted        |
| View counts          | Redis + PostgreSQL | Real-time counts + durable store |
| Session/cache        | Redis              | Low latency, TTL support         |
| Recommendations      | Redis + Feature DB | Pre-computed, real-time serving  |
| Subscriptions feed   | Redis (fan-out)    | Timeline generation              |
| Content fingerprints | Dedicated DB       | Audio/video hash matching        |

## 5.7 7. High-Level Architecture

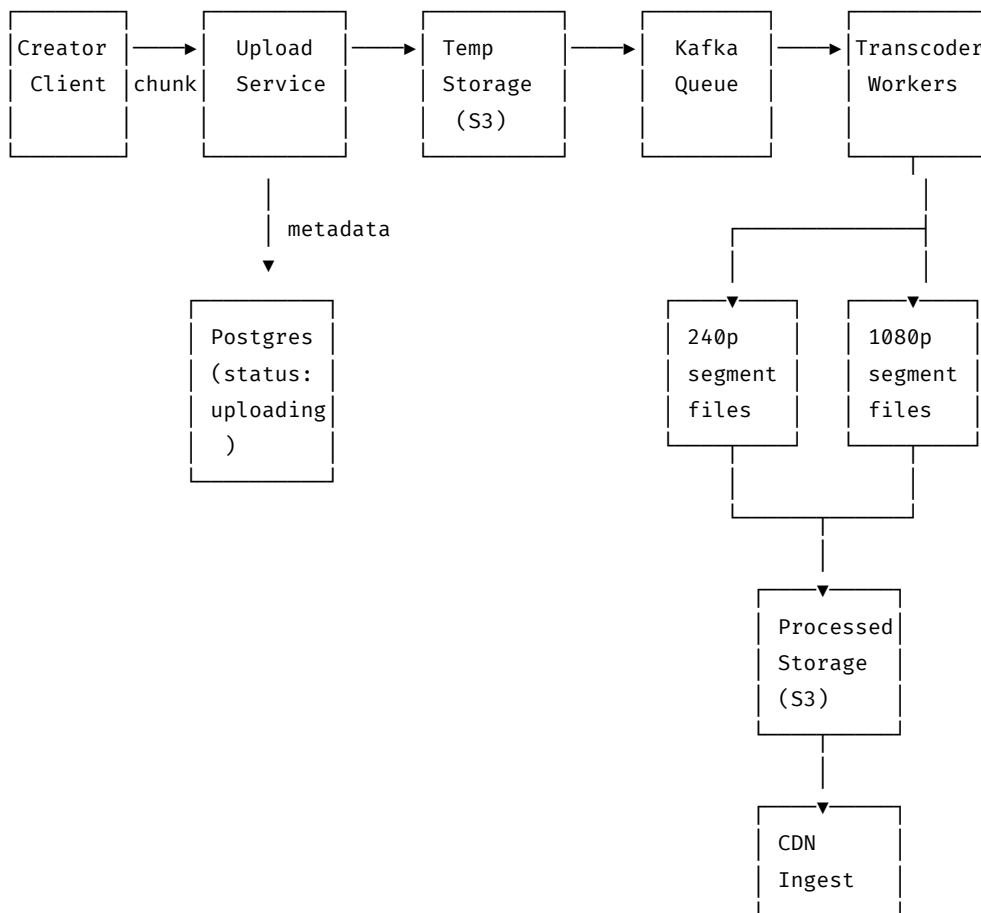
### 5.7.1 Overall System Architecture



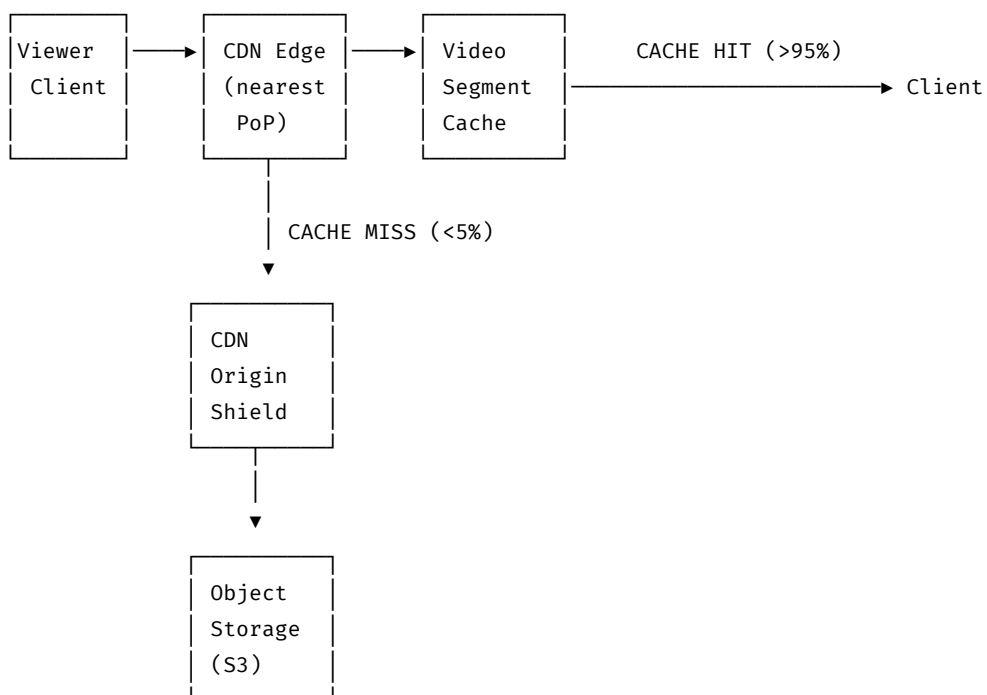




### 5.7.2 Upload Path (Write Path)



### 5.7.3 Watch Path (Read Path)



## 5.7.4 7.1 Component Breakdown

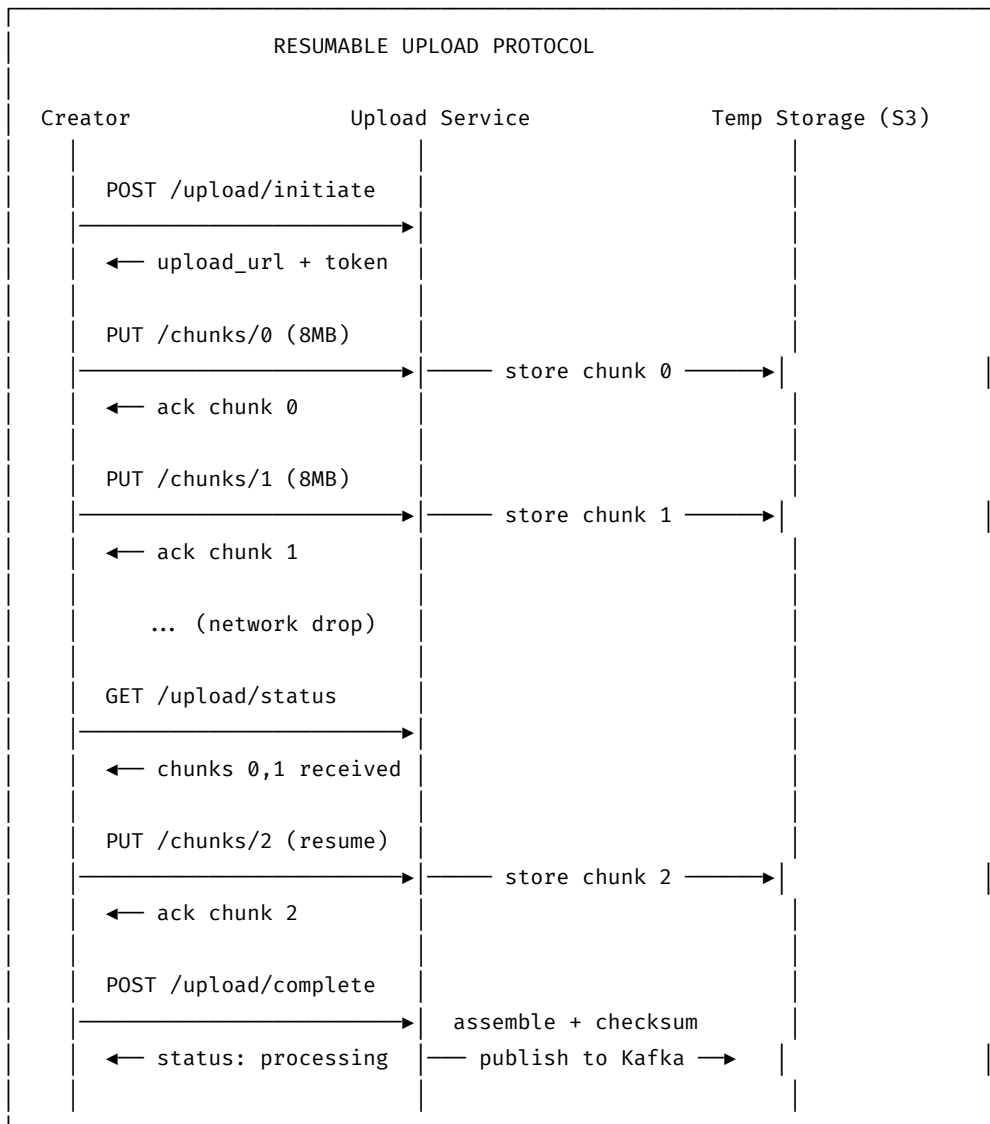
- **API Gateway:** Auth, rate limiting, request routing; Tech Stack: Kong / Envoy
  - **Upload Service:** Chunked upload, checksum validation; Tech Stack: Go / Java
  - **Video Processing Pipeline:** Transcoding, thumbnails, DRM, watermark; Tech Stack: FFmpeg + custom DAG
  - **Streaming Service:** Manifest generation, segment serving; Tech Stack: Go / Rust
  - **Search Service:** Full-text video search, autocomplete; Tech Stack: Elasticsearch
  - **Recommendation Service:** Personalized feeds, related videos; Tech Stack: Python + ML models
  - **User Service:** Registration, auth, profile management; Tech Stack: Java / Go
  - **Comment Service:** CRUD comments, threading, moderation; Tech Stack: Java
  - **Like Service:** Like/dislike toggle, aggregation; Tech Stack: Go
  - **Subscription Service:** Subscribe/unsubscribe, feed fan-out; Tech Stack: Go
  - **Notification Service:** Push notifications for new uploads; Tech Stack: Go + FCM/APNs
  - **Content Moderation Service:** Automated policy checks, copyright detection; Tech Stack: Python + ML
  - **CDN:** Edge caching, geo-distributed delivery; Tech Stack: CloudFront / Akamai
  - **Object Storage:** Raw and processed video storage; Tech Stack: S3 / GCS
  - **Message Queue:** Async pipeline orchestration; Tech Stack: Apache Kafka
- 

## 5.8 8. Deep Dive: Core Components

### 5.8.1 8.1 Video Upload and Processing Pipeline

#### 5.8.1.1 Resumable Chunked Upload

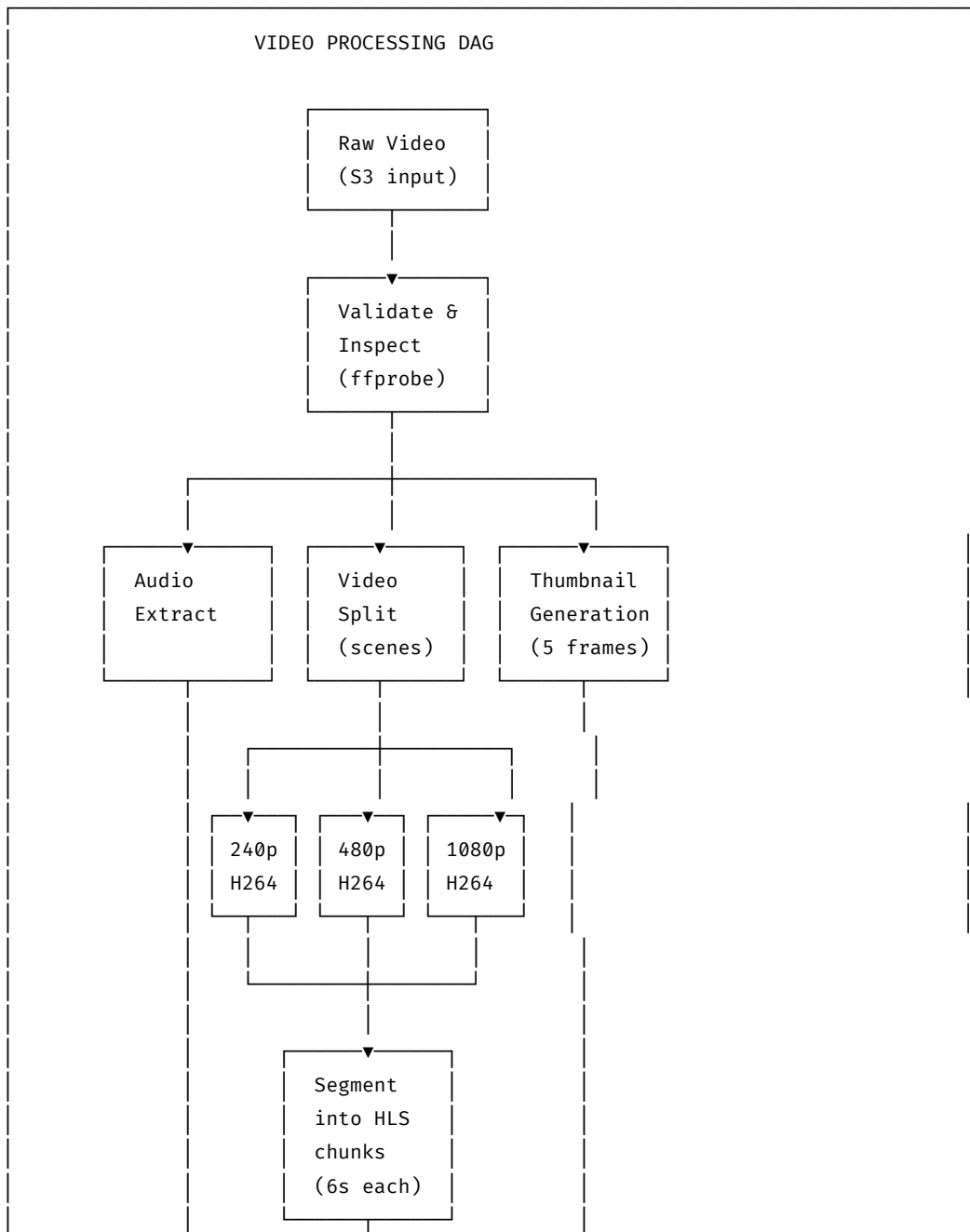
Large video files cannot be uploaded in a single HTTP request reliably. We use a chunked, resumable upload protocol inspired by the tus protocol:

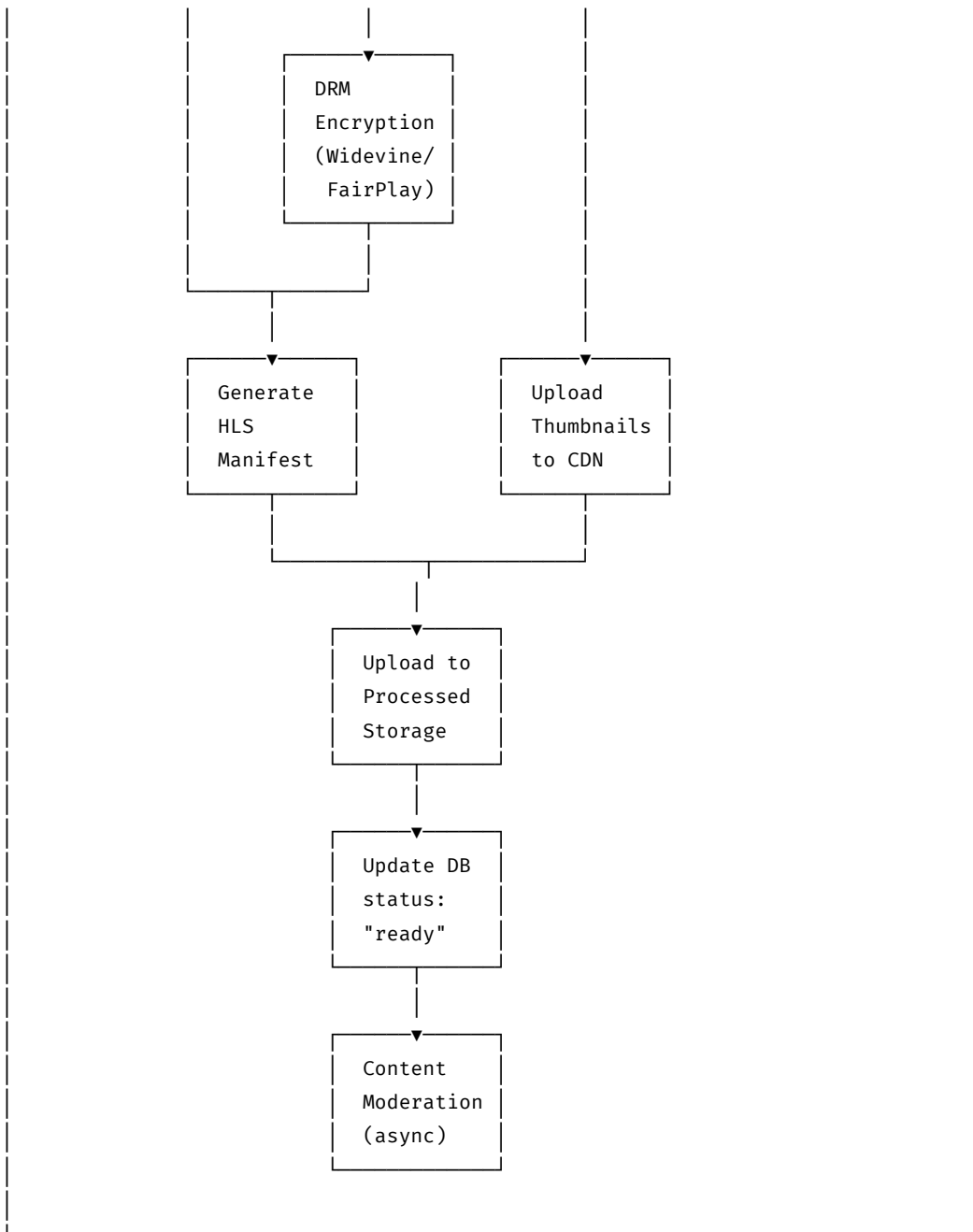


**Key design decisions:** - **Chunk size:** 8 MB default, adjustable based on client bandwidth - **Checksums:** Each chunk has a CRC32 checksum; final file has SHA-256 verification - **Expiry:** Incomplete uploads expire after 24 hours; temp storage is cleaned up - **Multipart upload to S3:** Chunks map directly to S3 multipart upload parts - **Idempotency:** Chunk numbers are idempotent keys; re-uploading same chunk is safe

### 5.8.1.2 Video Processing Pipeline (DAG-based)

Once the raw video is uploaded, a Directed Acyclic Graph (DAG) of processing tasks is created. This allows parallelism where possible and clear dependency management.





### 5.8.1.3 Transcoding Details

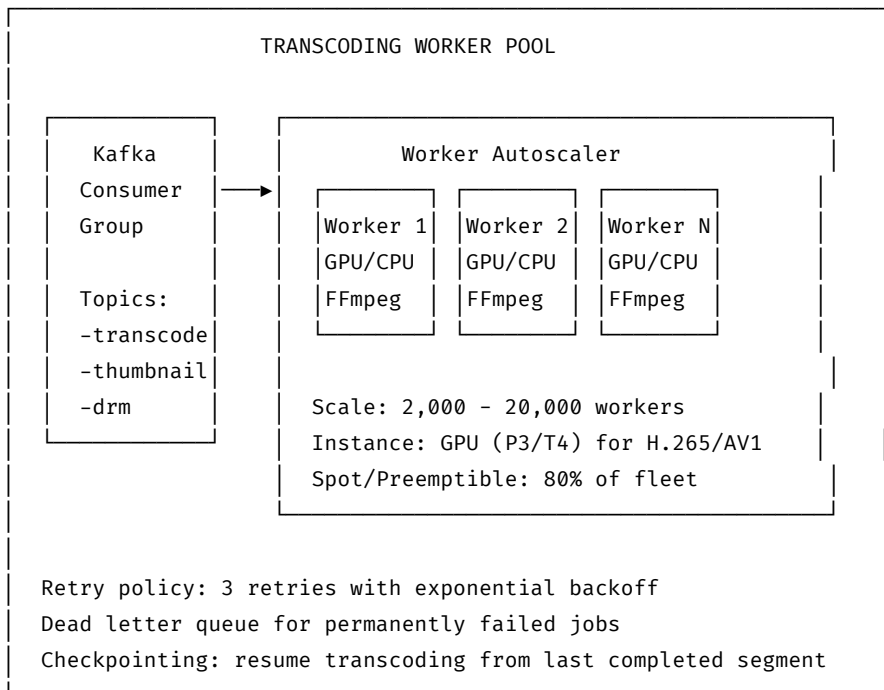
- **240p:** Bitrate (H.264): 400 Kbps; Bitrate (H.265): 250 Kbps; Bitrate (AV1): 150 Kbps; Segment Size (6s): ~300 KB
- **360p:** Bitrate (H.264): 800 Kbps; Bitrate (H.265): 500 Kbps; Bitrate (AV1): 300 Kbps; Segment Size (6s): ~600 KB
- **480p:** Bitrate (H.264): 1.5 Mbps; Bitrate (H.265): 1.0 Mbps; Bitrate (AV1): 600 Kbps; Segment Size (6s): ~1.1 MB
- **720p:** Bitrate (H.264): 3.0 Mbps; Bitrate (H.265): 2.0 Mbps; Bitrate (AV1): 1.2 Mbps; Segment Size (6s): ~2.3 MB
- **1080p:** Bitrate (H.264): 5.0 Mbps; Bitrate (H.265): 3.5 Mbps; Bitrate (AV1): 2.0 Mbps; Segment Size (6s): ~3.8 MB
- **4K:** Bitrate (H.264): 15.0 Mbps; Bitrate (H.265): 10.0 Mbps; Bitrate (AV1): 6.0 Mbps; Segment Size (6s): ~11.3 MB

**Codec strategy:**

- **H.264 (AVC):** Universal baseline – supported on all devices. Always generated.
- **H.265 (HEVC):** 40-50% better compression. Generated for 720p+ content. Requires licensing fees.
- **VP9:** Royalty-free Google codec. Generated for YouTube-like platforms.
- **AV1:** Next-gen royalty-free codec. 30% better than HEVC but very slow to encode. Generated only for popular content or as background job.

**Per-title encoding:** Netflix pioneered analyzing each video’s complexity to determine optimal bitrate ladders. A simple animation may only need 1.5 Mbps at 1080p, while an action scene needs 6+ Mbps. We analyze each title and generate custom encoding parameters.

#### 5.8.1.4 Transcoding Worker Architecture



## 5.8.2 8.2 Adaptive Bitrate Streaming (ABR)

Adaptive bitrate streaming is the core technology that enables smooth playback across varying network conditions. The video is pre-segmented into small chunks at multiple quality levels, and the client player dynamically switches between quality levels based on available bandwidth.

### 5.8.2.1 HLS (HTTP Live Streaming) vs DASH

- **Developer:** HLS: Apple; DASH: MPEG consortium (open std)
- **Container format:** HLS: MPEG-TS (.ts) or fMP4; DASH: fMP4 (.m4s)
- **Manifest format:** HLS: .m3u8 (playlist); DASH: .mpd (XML)
- **Segment duration:** HLS: 6 seconds typical; DASH: 2-6 seconds
- **DRM support:** HLS: FairPlay; DASH: Widevine, PlayReady
- **Browser support:** HLS: Safari native, others MSE; DASH: All browsers via MSE
- **Latency:** HLS: 15-30 sec (standard); DASH: 3-10 sec (low latency)
- **Industry adoption:** HLS: Apple ecosystem, broad; DASH: YouTube, Netflix

**Our choice:** Support both HLS and DASH. Generate fMP4 segments that work with both protocols (CMAF – Common Media Application Format). The manifest is generated dynamically per protocol.

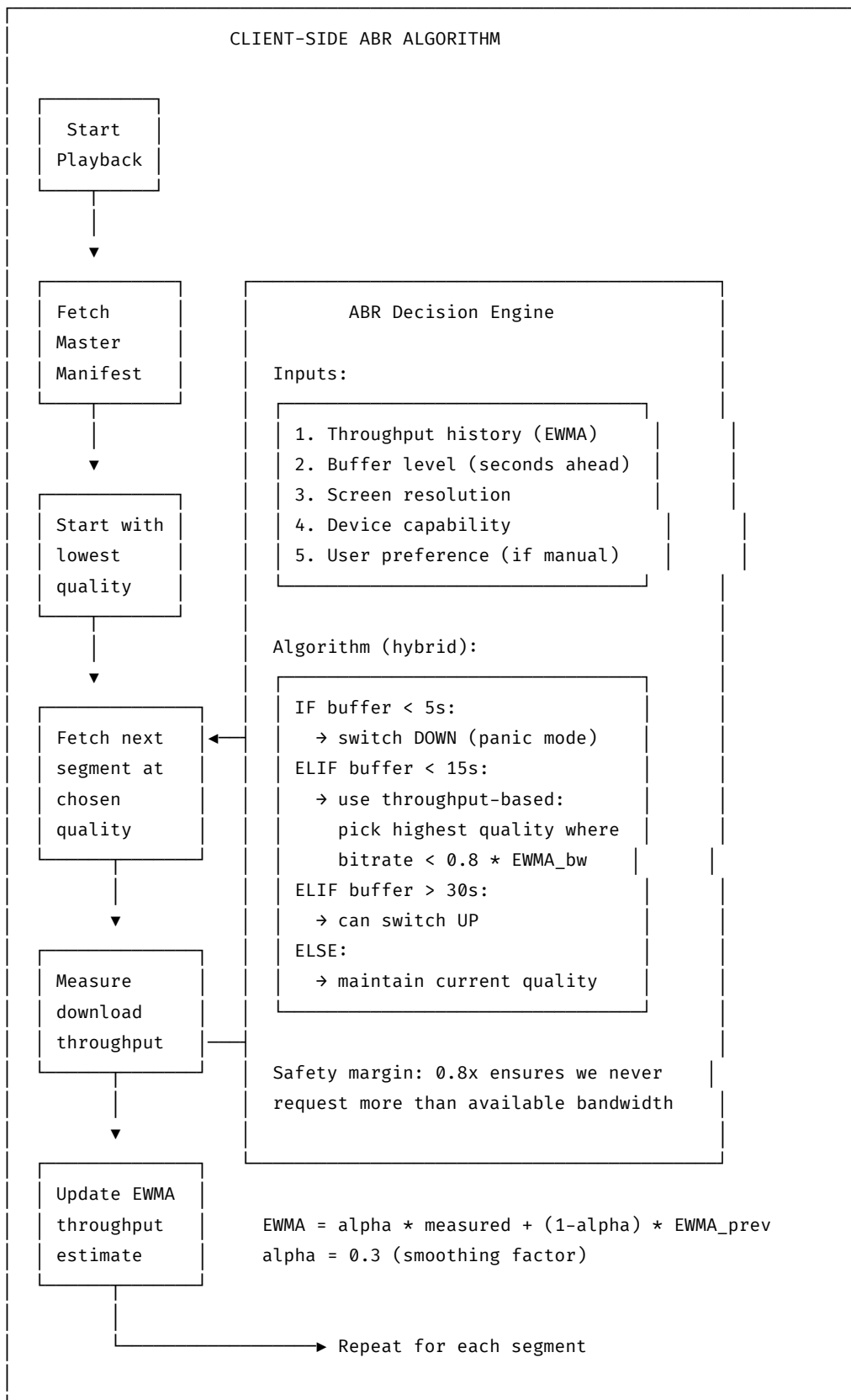
### 5.8.2.2 HLS Manifest Structure

Master Playlist (master.m3u8)

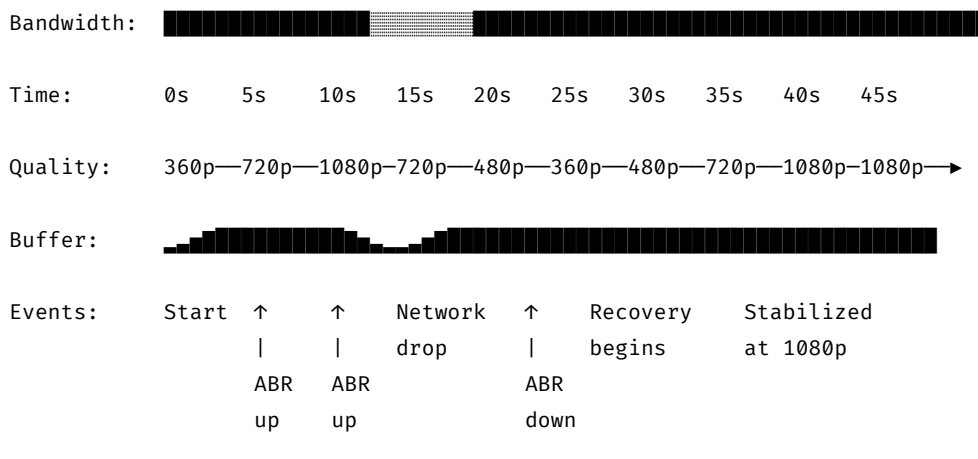
```
|— 240p/playlist.m3u8 → 240p/seg-0.ts, seg-1.ts, seg-2.ts, ...
|— 360p/playlist.m3u8 → 360p/seg-0.ts, seg-1.ts, seg-2.ts, ...
|— 480p/playlist.m3u8 → 480p/seg-0.ts, seg-1.ts, seg-2.ts, ...
|— 720p/playlist.m3u8 → 720p/seg-0.ts, seg-1.ts, seg-2.ts, ...
|— 1080p/playlist.m3u8 → 1080p/seg-0.ts, seg-1.ts, seg-2.ts, ...
└— audio/playlist.m3u8 → audio/seg-0.aac, seg-1.aac, seg-2.aac, ...
```



## 5.8.2.3 ABR Algorithm Flow



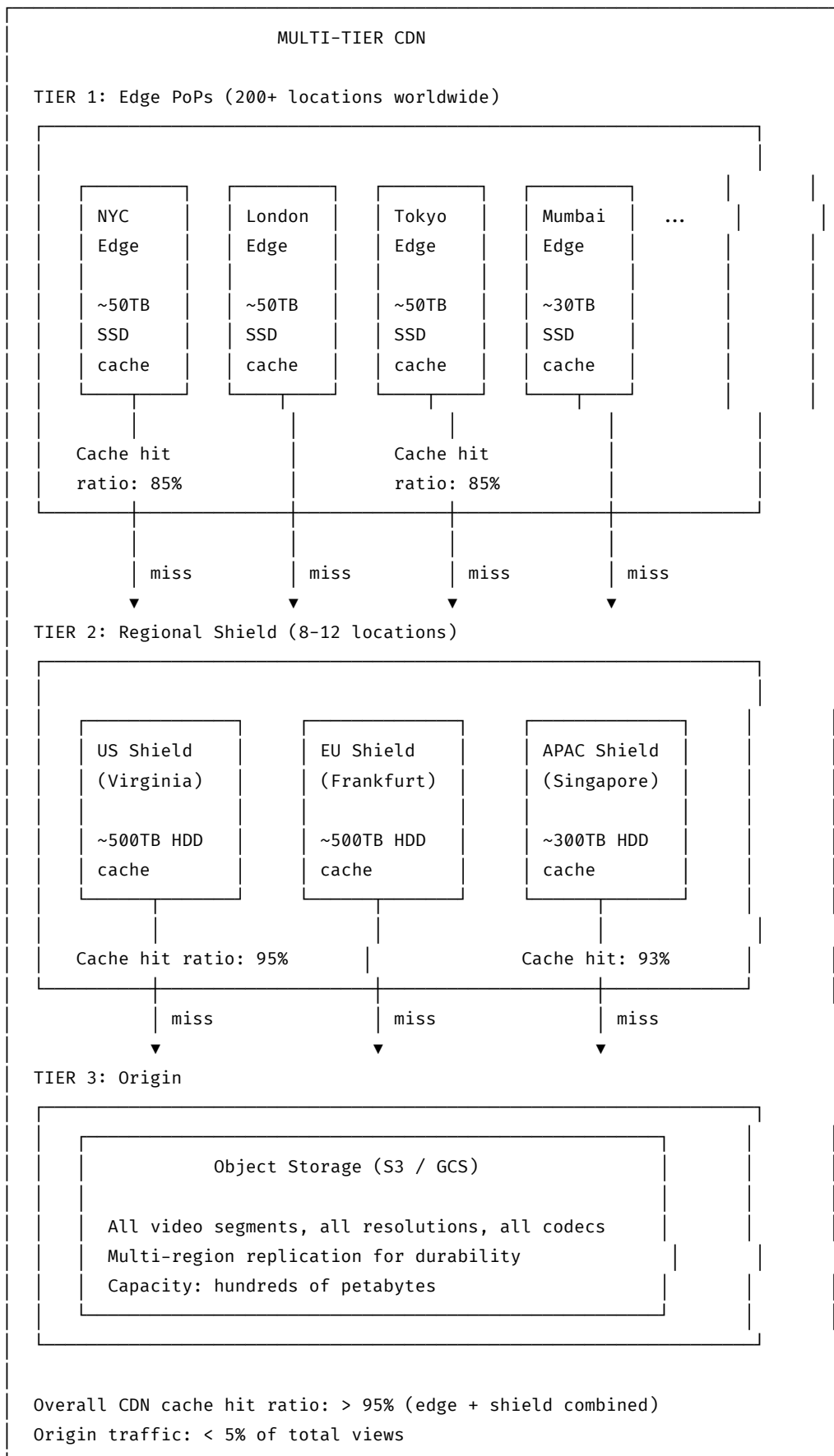
### 5.8.2.4 Quality Switching Timeline Example



## 5.8.3 8.3 CDN Architecture

Content Delivery Networks are absolutely critical for a video streaming platform. Without CDN, every view would hit origin storage, requiring petabits of bandwidth from a single region. CDN caches video segments at edge locations close to viewers.

### 5.8.3.1 Multi-Tier CDN Architecture



### 5.8.3.2 CDN Cache Warming and Popularity

Not all content is equal. The top 10% of videos typically account for 80%+ of views (Zipf distribution). We optimize caching based on content popularity:

| CONTENT POPULARITY TIERS |             |            |                           |
|--------------------------|-------------|------------|---------------------------|
| Tier                     | % of Videos | % of Views | CDN Strategy              |
| Hot                      | 0.1%        | 30%        | Pre-push to ALL edge PoPs |
| Warm                     | 5%          | 50%        | Push to regional shields  |
| Cool                     | 20%         | 15%        | Pull-through cache        |
| Cold                     | 74.9%       | 5%         | Origin-only, no caching   |

Viral detection:

|                          |                        |
|--------------------------|------------------------|
| Views/min > 10x baseline | → Promote to Hot tier  |
| Views/min > 3x baseline  | → Promote to Warm tier |
| No views for 7 days      | → Demote to Cold tier  |

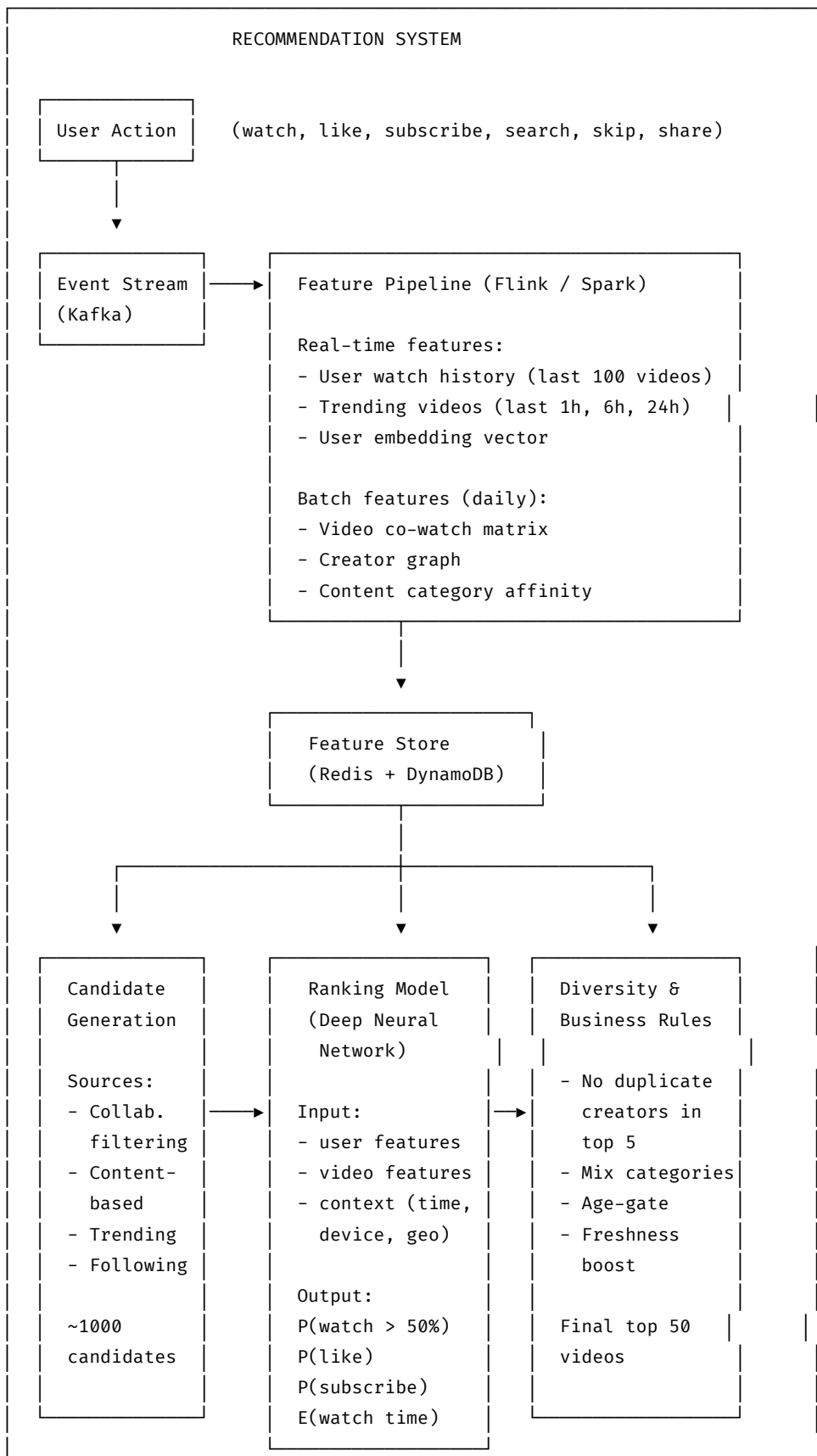
### 5.8.3.3 Cache Eviction Policy

- **Edge PoPs:** LRU with frequency boost (LFU-hybrid). Segments from popular videos get a “stickiness” bonus to avoid premature eviction.
- **Shield layer:** TTL-based (24h for hot, 72h for warm) + LRU fallback.
- **Segment granularity:** Cache individual segments (6-second chunks), not entire videos. First few segments of popular videos are prioritized since many viewers watch only the first 30 seconds.

## 5.8.4 8.4 Recommendation Engine

The recommendation engine is what keeps users engaged. It powers the home feed, “Up Next” sidebar, and search ranking signals.

### 5.8.4.1 Architecture Overview



Cold start strategy:

New user: → Show trending + popular by geo + onboarding quiz  
 New video: → Use content features (title, tags, description)  
                   → Boost exposure to small sample for signal  
                   → Creator's subscriber base as initial audience

#### 5.8.4.2 Collaborative vs Content-Based Filtering

##### COLLABORATIVE FILTERING

"Users who watched X also watched Y"

User-Item Matrix:

|    | V1  | V2 | V3 | V4 | V5  |
|----|-----|----|----|----|-----|
| U1 | [ 5 | .  | 3  | .  | 1 ] |
| U2 | [ 4 | .  | .  | 3  | .   |
| U3 | [ . | 3  | .  | .  | 4 ] |
| U4 | [ . | .  | 5  | 4  | .   |
| U5 | [ 1 | .  | .  | .  | 5 ] |

→ Matrix factorization (ALS, SVD)  
 → Predict missing values

Pros: Serendipitous finds  
 Cons: Cold start problem

##### CONTENT-BASED FILTERING

"This video is similar to videos you liked"

Video Feature Vector:

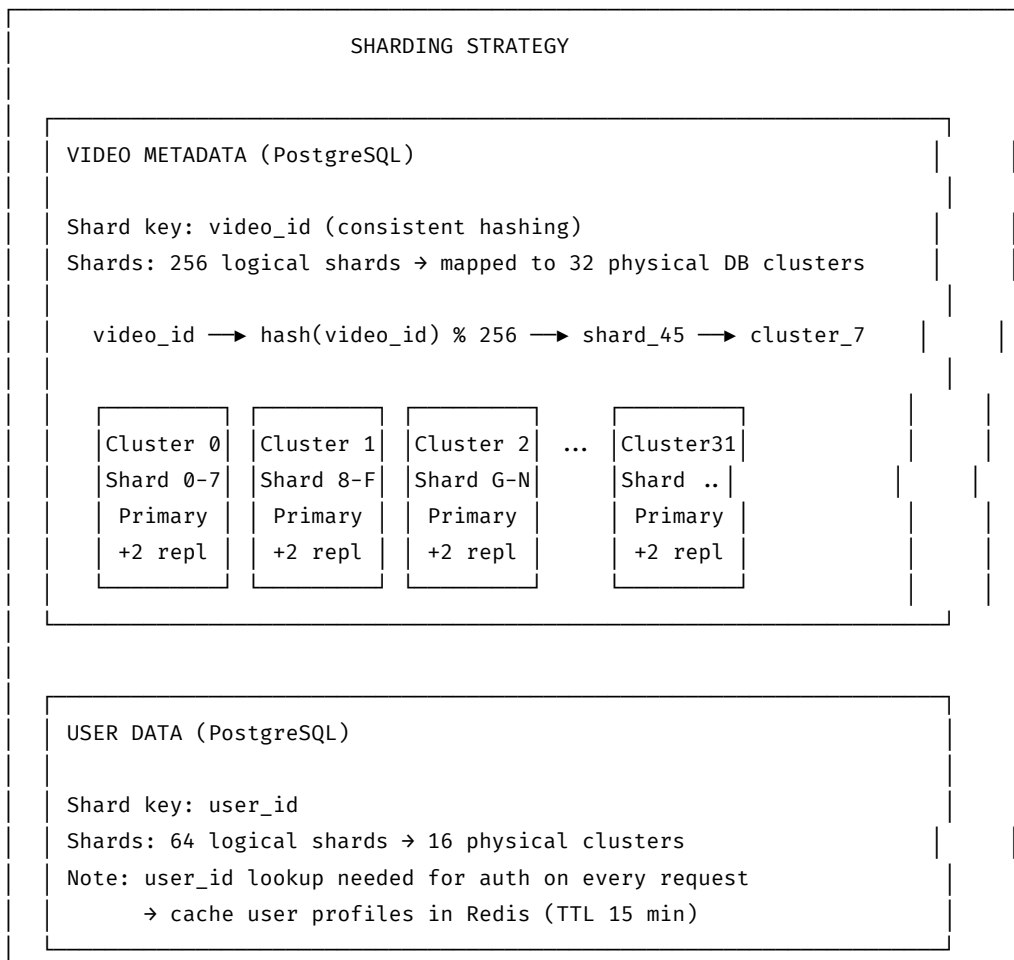
Title embedding: [0.2, ...]  
 Visual features: [0.8, ...]  
 Audio features: [0.1, ...]  
 Tags/categories: [1,0, ...]  
 Duration: [0.3]  
 Creator profile: [0.5, ...]

→ Cosine similarity to user's positive history

Pros: Works for new videos  
 Cons: Filter bubble risk

## 5.9 9. Data Partitioning and Sharding

### 5.9.1 Sharding Strategy by Data Type



#### COMMENTS (Cassandra)

Partition key: video\_id

Clustering key: created\_at DESC

Replication factor: 3 (across availability zones)

Hot partition risk: viral videos with millions of comments

Mitigation: bucket partition key as (video\_id, bucket\_id)

bucket\_id = comment\_count / 10000

→ Each partition holds at most ~10K comments

#### VIDEO FILES (Object Storage / S3)

Path structure:

s3://video-processed/{shard}/{video\_id}/{quality}/{segment}.ts

Shard prefix: first 2 chars of video\_id hash

→ Avoids S3 partition hotspots (S3 partitions by key prefix)

→ 256 prefix shards (00-ff)

Multi-region replication:

Primary: us-east-1

Replicas: eu-west-1, ap-southeast-1

→ CDN pulls from nearest region's S3

#### SEARCH INDEX (Elasticsearch)

Index: videos (50 shards, 1 replica)

Routing: video\_id (co-locate with metadata for efficient updates)

Fields indexed: title, description, tags, captions, category

Refresh interval: 30 seconds (near-real-time, not instant)

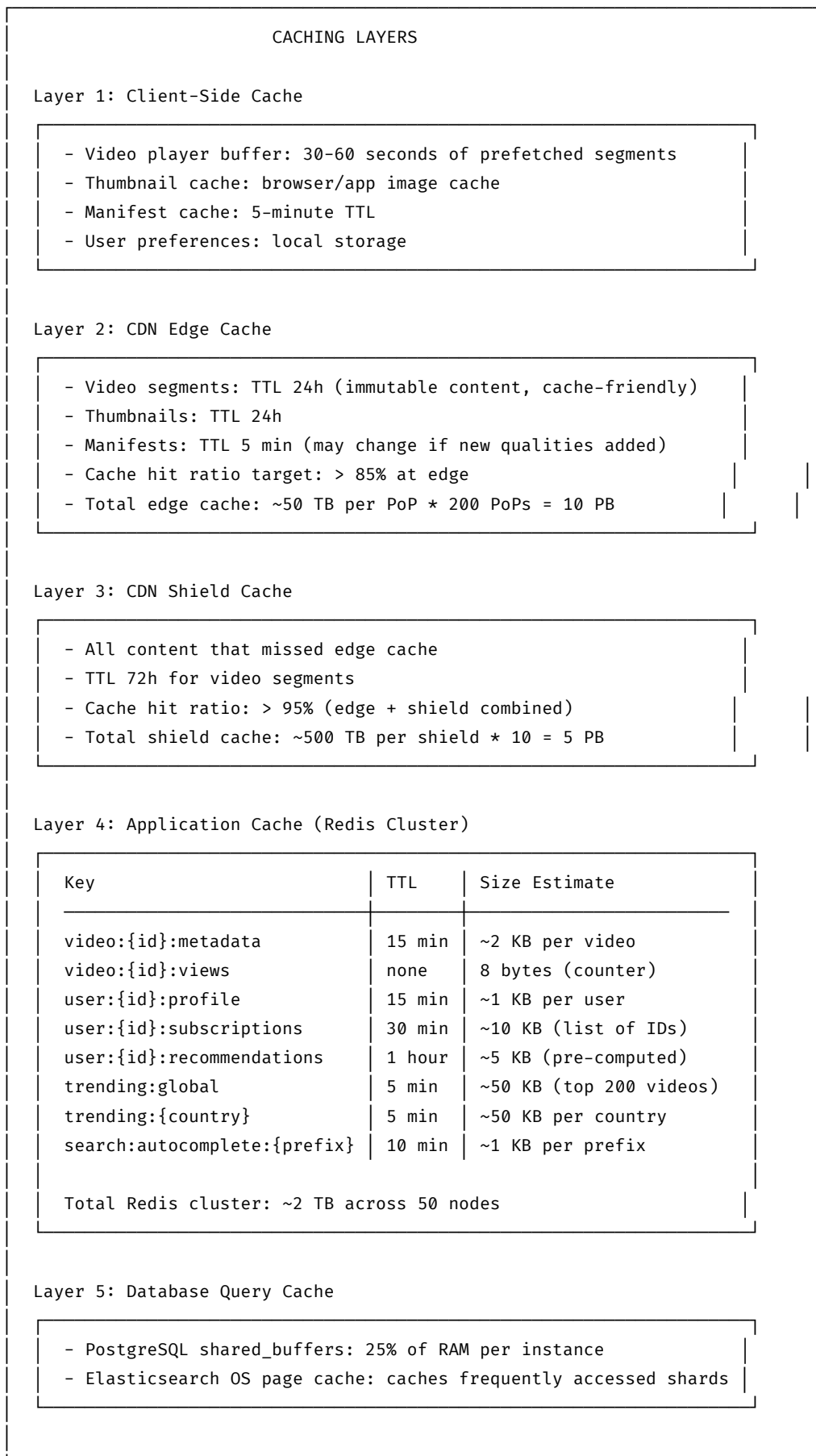
### 5.9.2 Cross-Shard Query Handling

Some queries span multiple shards: - **“All videos by creator X”**: Creator-to-videos mapping stored in a secondary index (denormalized table sharded by creator\_id). Not a cross-shard join. - **“Trending videos”**: Pre-computed by a batch/stream pipeline, stored as a sorted list in Redis. No cross-shard query needed. - **“Search”**: Elasticsearch handles its own sharding. A coordinating node scatters the query to all shards and gathers results.



## 5.10 10. Caching Strategy

### 5.10.1 Multi-Layer Cache Architecture



### 5.10.2 Cache Invalidation Strategy

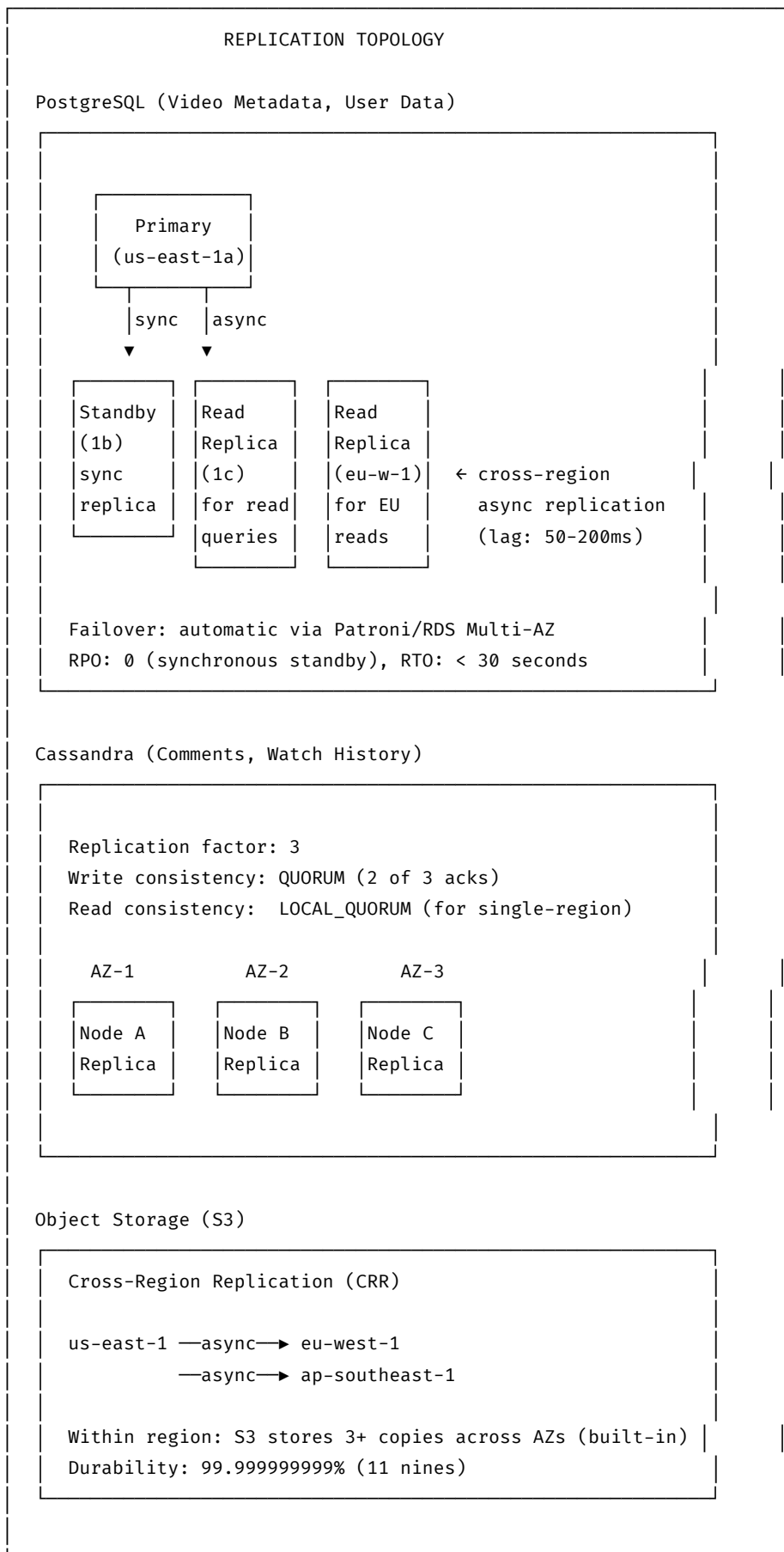
| Content Type    | Invalidation Strategy                                                                                                             |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Video segments  | Immutable (never change once transcoded).<br>Unique URL per version. No invalidation needed.<br>New transcode = new segment URLs. |
| Thumbnails      | Versioned URL: /thumbs/{video_id}/v2/default.jpg<br>Creator uploads new thumbnail → new version URL                               |
| Video metadata  | Event-driven: on metadata update, publish event<br>to Kafka → consumer invalidates Redis key<br>Short TTL (15 min) as safety net  |
| View counts     | No invalidation. Redis counter is source of truth.<br>Periodic flush to PostgreSQL (every 30 seconds)                             |
| Recommendations | Regenerated every hour per user.<br>Redis key overwritten (not invalidated).                                                      |
| Search index    | Elasticsearch near-real-time refresh (30s)<br>No cache invalidation; index is the cache.                                          |

## 5.11 11. Replication and Consistency

### 5.11.1 Consistency Requirements by Data Type

| CONSISTENCY MODEL       |                           |                                                                                                                  |
|-------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------|
| Data                    | Consistency               | Rationale                                                                                                        |
| Video upload status     | Strong                    | Creator must see upload status accurately. No lost uploads.                                                      |
| Video metadata (writes) | Strong (read-your-writes) | Title, description changes must be visible to creator immediately. Viewers can see stale data for a few seconds. |
| View counts             | Eventual                  | Approximate counts are fine. "1.2M views" vs "1,200,347" does not matter to viewers.                             |
| Like counts             | Eventual                  | Same as view counts. User's own like status: strong (read-your-writes via session)                               |
| Comments                | Eventual                  | Delay of a few seconds is acceptable. User sees own comment immediately (optimistic client-side rendering).      |
| Subscriptions           | Strong                    | User expects immediate effect of subscribe/unsubscribe.                                                          |
| Recommendations         | Eventual                  | Pre-computed, hourly refresh. Staleness is inherent.                                                             |
| Search index            | Eventual                  | Near-real-time (30s delay). New uploads searchable within a minute.                                              |

### 5.11.2 Replication Topology



---

## 5.12 12. Fault Tolerance and Failure Handling

### 5.12.1 Failure Scenarios and Mitigations

| FAILURE HANDLING MATRIX     |                                                                                                                                                                                   |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure                     | Mitigation                                                                                                                                                                        |
| CDN edge PoP goes down      | GeoDNS reroutes to next-nearest PoP.<br>User sees ~50ms latency increase, not outage.<br>Shield layer absorbs the miss traffic.                                                   |
| CDN origin shield fails     | Fallback to direct S3 access with signed URLs.<br>Higher latency but no data loss.<br>Multiple shield nodes with health checks.                                                   |
| Transcoding worker crashes  | Kafka retries: message stays in queue until another worker picks it up. Checkpointing allows resume from last completed segment.<br>Dead letter queue after 3 retries.            |
| Upload interrupted          | Resumable protocol: client queries upload status and resumes from last acknowledged chunk.<br>Incomplete uploads auto-cleaned after 24h.                                          |
| Database primary fails      | Automatic failover to synchronous standby.<br>RTO < 30s. Read replicas continue serving reads.<br>Write operations queue briefly, then resume.                                    |
| Redis cluster node fails    | Redis Sentinel/Cluster auto-failover.<br>View counts: in-memory batch flushed on recovery.<br>Worst case: lose a few seconds of count increments.                                 |
| Elasticsearch node fails    | Replica shards serve reads during primary shard recovery. Cluster rebalances automatically.<br>Temporary search result staleness acceptable.                                      |
| Recommendation service down | Serve cached recommendations from Redis.<br>Fallback to trending/popular videos (no persona).<br>Graceful degradation, not failure.                                               |
| Entire region outage        | DNS failover to healthy region.<br>Cross-region S3 replication ensures data available.<br>Viewers served from CDN cache during transition.<br>Uploads rerouted to healthy region. |
| Viral video traffic spike   | CDN auto-caches under load. Shield absorbs burst.<br>Rate limiting on non-critical APIs (comments).<br>Video streaming itself has highest priority.                               |

### 5.12.2 Degraded Mode Strategy

When the system is under extreme load, we apply progressive degradation:

|                     |                                                                                                                                                                   |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Level 0 (Normal):   | All features operational                                                                                                                                          |
| Level 1 (Elevated): | Disable recommendations, serve trending instead<br>Reduce search result freshness (5 min cache)                                                                   |
| Level 2 (High):     | Disable comments loading on initial page load<br>Reduce video quality options (drop 240p, 360p)<br>Rate limit uploads to 50% capacity                             |
| Level 3 (Critical): | Serve only cached content from CDN<br>Disable uploads entirely<br>Static home page with top 100 trending videos<br>All writes rejected except critical auth flows |

---

## 5.13 13. Scalability



### 5.13.1 Horizontal Scaling Approach

#### SCALABILITY DIMENSIONS

##### COMPUTE SCALING

| Component           | Scaling Trigger              | Max Scale   |
|---------------------|------------------------------|-------------|
| API Gateway         | Requests/sec > 80% capacity  | 200 nodes   |
| Upload Service      | Active uploads > threshold   | 100 nodes   |
| Streaming Service   | Concurrent streams           | 500 nodes   |
| Transcoding Workers | Queue depth > 1000 jobs      | 20K workers |
| Search Service      | Query latency p99 > 200ms    | 100 nodes   |
| Comment Service     | Write throughput > threshold | 50 nodes    |
| Recommendation Svc  | Request latency p99 > 100ms  | 200 nodes   |

Autoscaler: Kubernetes HPA + custom metrics from Prometheus

Scale-up time: < 2 minutes (warm pool of pre-provisioned nodes)

Scale-down: gradual (10% reduction every 5 minutes, with cooldown)

##### STORAGE SCALING

S3: Effectively unlimited. No manual scaling needed.

PostgreSQL: Add read replicas for read scaling.

Reshard for write scaling (rare, planned event).

Cassandra: Add nodes to ring. Data rebalances automatically.

Redis: Add shards to Redis Cluster for memory scaling.

Elasticsearch: Add data nodes, increase shard count at reindex.

##### NETWORK / CDN SCALING

CDN capacity scales with number of edge PoPs.

For viral events: CDN providers (Akamai, CloudFront) have burst capacity built in. No action needed from our side.

Multi-CDN strategy for extreme scale:

Primary CDN handles 70% of traffic

Secondary CDN handles 30% as overflow + failover

DNS-based traffic splitting with health checks

### 5.13.2 Handling Viral Videos

#### VIRAL VIDEO SCALING TIMELINE

T+0 min: Video uploaded, transcoded, available

T+5 min: Views spike to 10x normal for a new video  
→ Anomaly detector flags as "potentially viral"

T+10 min: Views at 50x normal  
→ Promote to "Hot" tier  
→ Pre-push all segments to ALL edge PoPs (~200)  
→ Warm CDN shield caches globally

T+30 min: Views at 500x normal, trending globally  
→ CDN cache hit ratio at 99%+  
→ Origin sees minimal traffic (< 1%)  
→ API servers auto-scaled for metadata requests  
→ Comment service rate-limited to prevent overload

T+2 hr: Peak traffic. Millions of concurrent viewers.  
→ System stable: CDN absorbs 99%+ of bandwidth  
→ View counter batching: aggregate in Redis,  
flush to DB every 30 seconds instead of per-view

T+24 hr: Traffic declining. Begin cache demotion.  
→ Remove from some edge PoPs, keep in shields

T+7 days: Traffic normalized. Standard caching policy resumes.

## 5.14 14. Monitoring and Observability

### 5.14.1 Key Metrics Dashboard

| OBSERVABILITY FRAMEWORK                                   |            |                      |
|-----------------------------------------------------------|------------|----------------------|
| VIEWER EXPERIENCE METRICS (most important -- user-facing) |            |                      |
| Metric                                                    | Target     | Alert Threshold      |
| Time to First Frame (TTFF)                                | < 2s p50   | > 3s p50 for 5 min   |
|                                                           | < 4s p99   | > 5s p99 for 5 min   |
| Rebuffering ratio                                         | < 0.5%     | > 1% for 10 min      |
| Video start failure rate                                  | < 0.1%     | > 0.5% for 5 min     |
| Average bitrate served                                    | Monitor    | Drop > 20% in 10 min |
| Quality switches/min                                      | < 2        | > 5 for 10 min       |
| Playback error rate                                       | < 0.05%    | > 0.2% for 5 min     |
| INFRASTRUCTURE METRICS                                    |            |                      |
| CDN cache hit ratio                                       | > 95%      | < 90% for 15 min     |
| CDN edge latency (p99)                                    | < 50ms     | > 100ms for 10 min   |
| Origin 5xx rate                                           | < 0.01%    | > 0.1% for 5 min     |
| API latency (p99)                                         | < 200ms    | > 500ms for 5 min    |
| Kafka consumer lag                                        | < 1000 msg | > 10000 for 10 min   |
| DB connection pool usage                                  | < 70%      | > 90% for 5 min      |
| Redis memory usage                                        | < 80%      | > 90%                |

## PIPELINE METRICS

|                           |          |                     |
|---------------------------|----------|---------------------|
| Upload success rate       | > 99.5%  | < 98% for 30 min    |
| Transcoding queue depth   | < 5000   | > 20000 for 15 min  |
| Transcoding time (1h vid) | < 30 min | > 60 min for 30 min |
| Transcoding failure rate  | < 0.5%   | > 2% for 15 min     |
| Content moderation time   | < 10 min | > 30 min for 30 min |

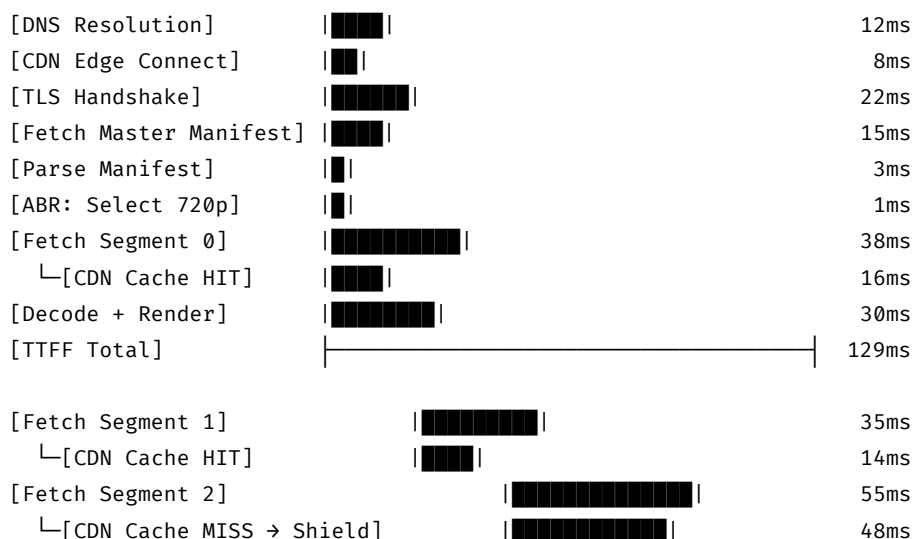
## BUSINESS METRICS

|                           |         |                         |
|---------------------------|---------|-------------------------|
| Daily active viewers      | Monitor | Drop > 10% day-over-day |
| Average watch time/user   | Monitor | Drop > 15%              |
| Upload volume             | Monitor | Drop > 30%              |
| Search click-through rate | Monitor | Drop > 20%              |
| Recommendation CTR        | Monitor | Drop > 15%              |

Tech stack: Prometheus + Grafana (infra), custom telemetry SDK  
in video player (viewer experience), ELK for logs, PagerDuty for alerts.

### 5.14.2 Distributed Tracing for Video Playback

Trace: viewer\_playback\_session\_abc123



## 5.15 15. Trade-offs and Design Decisions

### 5.15.1 Decision 1: Pre-Transcode All Resolutions vs On-Demand Transcoding

#### OPTION A: Pre-Transcode (Our Choice)

- + Instant playback: no transcoding delay when viewer requests quality
- + CDN-friendly: static files, highly cacheable
- + Simpler streaming service: just serve files
- Higher storage cost: 5-7x per video (all resolutions)
- Wasted compute: 80% of videos get < 100 views; many resolutions are never watched

#### OPTION B: On-Demand Transcoding

- + Lower storage: only store raw + most-requested resolutions
- + Less wasted compute for unpopular content
- Playback delay on first request for a resolution
- Complex real-time transcoding infrastructure
- CDN caching less effective (dynamic content)

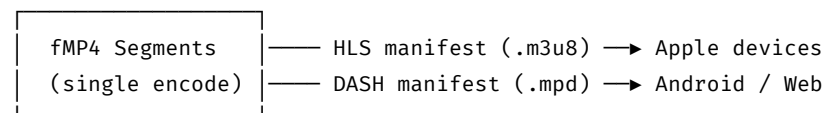
#### HYBRID APPROACH (Netflix-style):

- Popular content (top 20%): pre-transcode all resolutions
- Long-tail (bottom 80%): pre-transcode only 360p + 720p
- On-demand transcode other resolutions if requested
- Background job promotes videos to full pre-transcode once popular

### 5.15.2 Decision 2: HLS vs DASH vs Both (CMAF)

#### Our Choice: CMAF (Common Media Application Format)

Generate fMP4 segments once → serve with both HLS and DASH manifests



#### Benefits:

- Single set of segments reduces storage by ~40% vs separate formats
- Universal device support
- Simplified transcoding pipeline
- Industry convergence direction

#### Trade-off:

- Slightly more complex manifest generation
- Some older devices may not support fMP4 in HLS (fallback to .ts)

### 5.15.3 Decision 3: Push vs Pull CDN Strategy

PUSH (Proactive): Upload segments to CDN before anyone requests them

PULL (Reactive): CDN fetches from origin on first cache miss

Our Choice: HYBRID

| Content Tier | Strategy        | Rationale                                                                       |
|--------------|-----------------|---------------------------------------------------------------------------------|
| Hot (0.1%)   | PUSH            | Guaranteed instant start everywhere. Worth the cost.                            |
| Warm (5%)    | PUSH to shields | Cover major markets. Edge pulls from shield.                                    |
| Cool (20%)   | PULL            | CDN fetches on first view. First viewer gets slower start; subsequent are fast. |
| Cold (74.9%) | PULL            | May not even be cached. Direct S3 serving is fine for very low view counts.     |

### 5.15.4 Decision 4: Monolith vs Microservices

Our Choice: Microservices (for a platform at this scale)

Justification:

- Upload, transcoding, streaming, search, and recommendations have vastly different scaling characteristics
- Transcoding is GPU-heavy; streaming is bandwidth-heavy; search is CPU+memory heavy. Different hardware requirements.
- Independent deployability: update recommendation algorithm without touching video streaming
- Team autonomy: separate teams own separate services

Service boundaries:

```

Upload Service → Video Processing Pipeline (async)
Streaming Service (stateless, horizontally scalable)
Metadata Service (CRUD + caching)
Search Service (Elasticsearch wrapper)
Recommendation Service (ML model serving)
User Service (auth, profiles)
Social Service (comments, likes, subscriptions)
Notification Service (async, fan-out)

```

Trade-off:

- Increased operational complexity (service mesh, distributed tracing)
- Network overhead for inter-service communication
- Data consistency across services requires careful design

### 5.15.5 Decision 5: Segment Duration

|                       |    |                       |
|-----------------------|----|-----------------------|
| Shorter segments (2s) | vs | Longer segments (10s) |
|-----------------------|----|-----------------------|

- |                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ Faster quality switches</li> <li>+ Lower latency for live</li> <li>+ Finer-grained ABR control</li> <li>- More HTTP requests</li> <li>- Higher CDN overhead</li> <li>- Worse compression ratio</li> </ul> | <ul style="list-style-type: none"> <li>+ Fewer HTTP requests</li> <li>+ Better compression</li> <li>+ Lower CDN overhead</li> <li>- Slower quality switches</li> <li>- Higher live latency</li> <li>- Coarser ABR control</li> </ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Our choice: 6 seconds

Industry standard compromise. Apple recommends 6s for HLS.

For low-latency live: use 2s segments with chunked transfer encoding.

## 5.16 16. Interview Deep-Dive Questions

### 5.16.1 Q1: How do you handle viral videos with sudden traffic spikes?

**Answer:** Viral videos are handled through a multi-layered defense:

1. **CDN absorbs the burst.** Since video segments are static, immutable files, CDN caching is extremely effective. Once a segment is cached at an edge PoP, all subsequent requests from that region are served from cache without touching origin. A viral video with 10M views might only result in ~200 origin fetches (one per edge PoP for each segment).
  2. **Anomaly detection promotes content.** A real-time monitoring pipeline tracks view velocity (views/minute) per video. When a video exceeds 10x its baseline, we proactively push its segments to all edge PoPs globally (instead of waiting for pull-through caching). This reduces first-viewer latency in new regions.
  3. **View counting uses batching.** Under viral load, per-view database writes would overwhelm the database. Instead, view counts are atomically incremented in Redis ( $O(1)$  per view), and a background job flushes the aggregated count to PostgreSQL every 30 seconds. The displayed count is approximate and that is acceptable – “1.2M views” vs “1,234,567 views” does not matter to viewers.
  4. **Non-critical services are rate-limited.** Comments, likes, and other engagement features are rate-limited to protect the core streaming path. Viewers can always watch the video even if commenting is temporarily degraded.
- 

### 5.16.2 Q2: How does adaptive bitrate streaming work end-to-end?

**Answer:** ABR streaming works through a tight feedback loop between the server (which provides multiple quality options) and the client player (which chooses):

**Server side:** The transcoding pipeline generates the same video at multiple quality levels (240p through 4K), each split into small segments (6 seconds). A master manifest file lists all available quality levels with their bitrates and resolution. Each quality level has its own sub-manifest listing all segment URLs.

**Client side:** The video player uses an ABR algorithm that continuously monitors: (a) measured download throughput (exponentially weighted moving average of recent segment downloads), (b) current buffer level (how many seconds of video are buffered ahead), and (c) device capabilities.

The algorithm picks the highest quality whose bitrate is safely below the measured throughput (with a 20% safety margin). If buffer drops dangerously low (< 5 seconds), it aggressively drops quality. If buffer is healthy (> 30 seconds), it can cautiously try a higher quality.

Since each segment is an independent HTTP request, the client can switch quality at every segment boundary (every 6 seconds) without interruption.

---

### 5.16.3 Q3: How do you detect and handle copyrighted content?

**Answer:** Copyright detection uses a two-phase approach:

**Phase 1: Audio/Video Fingerprinting (Content ID)** - During transcoding, we extract perceptual fingerprints from both the audio track and video frames. Audio fingerprinting uses spectral analysis (similar to Shazam) to create a compact representation of the audio. Video fingerprinting uses

---



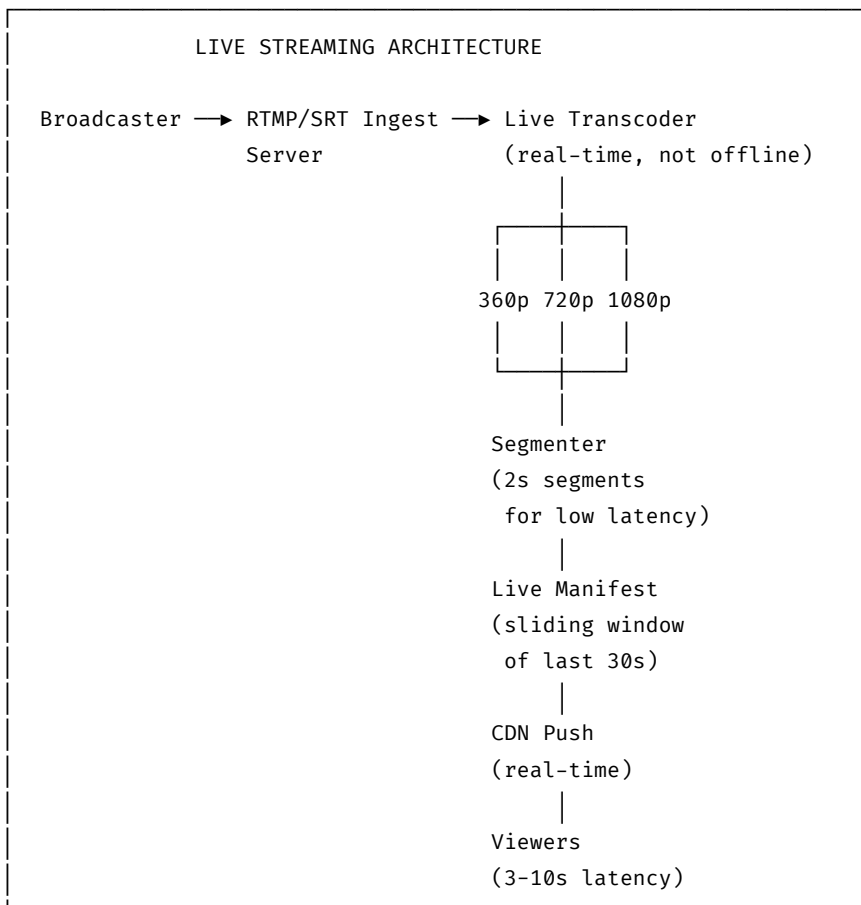
perceptual hashing of keyframes. - These fingerprints are compared against a reference database of copyrighted content provided by rights holders. The reference database contains fingerprints of millions of songs, movies, and TV shows. - Matching is fuzzy – it detects copyrighted content even if the video has been re-encoded, cropped, sped up, or has overlaid commentary.

**Phase 2: Policy Application** - If a match is found, the rights holder's policy determines the action: - **Block:** Video is prevented from publishing. - **Monetize:** Video is published but ad revenue goes to rights holder. - **Track:** Video is published normally; rights holder just wants analytics. - Creators can dispute claims through an appeal process.

**Technical implementation:** - Fingerprint extraction runs as a DAG step in the transcoding pipeline - Reference database is stored in a specialized similarity search index (approximate nearest neighbor search) - Matching runs in parallel with transcoding to avoid adding latency - New reference fingerprints from rights holders are ingested via a batch pipeline and retroactively scanned against existing content

#### 5.16.4 Q4: How would you design live streaming on top of this architecture?

**Answer:** Live streaming reuses much of the VOD infrastructure but with key differences for low latency:



**Key differences from VOD:** - **Ingest protocol:** RTMP or SRT from broadcaster software (OBS, Streamlabs) to our ingest servers. These are stateful, long-lived connections. - **Real-time transcoding:** Must keep up with the live feed. Uses dedicated GPU instances, not batch processing. One transcoder per stream. - **Shorter segments:** 2-second segments (vs 6s for VOD) for lower latency. Trade-off: more HTTP requests, worse compression. - **Sliding window manifest:** The HLS manifest

only contains the last ~30 seconds of segments (sliding window), not the entire video. - **CDN push, not pull:** Segments are pushed to CDN as soon as they are generated, rather than waiting for viewer requests. - **DVR / rewind:** Store recent segments (last 2-4 hours) for DVR functionality. After stream ends, hand off to standard VOD pipeline for archival transcoding.

---

### 5.16.5 Q5: How do you optimize for mobile vs desktop clients?

**Answer:** Mobile and desktop have fundamentally different constraints:

- **Screen size:** Mobile: 5-7 inches; Desktop: 13-32 inches
- **Max useful res.:** Mobile: 720p (most), 1080p (flagship); Desktop: 1080p-4K
- **Network:** Mobile: Variable (4G/5G/WiFi); Desktop: Stable broadband
- **Battery:** Mobile: Limited, decode costs power; Desktop: Not a concern
- **Data caps:** Mobile: Common (2-50 GB/month); Desktop: Rare

**Optimizations:** - **Resolution capping:** Mobile app does not request > 1080p even on WiFi. No point sending 4K to a 6-inch screen. - **Codec selection:** Prefer H.265/HEVC on iOS (hardware decode support). Prefer VP9/AV1 on Android (better software decode). These save 40-50% bandwidth vs H.264. - **Lower initial quality:** Mobile starts at 360p and ramps up (vs 720p on desktop) to minimize time-to-first-frame on cellular. - **Prefetch strategy:** On mobile WiFi, aggressively prefetch next 30s. On cellular, only prefetch next 10s to conserve data. - **Offline download:** Mobile app supports downloading videos at chosen quality for offline viewing. Desktop typically does not. - **Thumbnail quality:** Serve smaller thumbnails (320px) on mobile vs full-size (1280px) on desktop to reduce page load time.

---

### 5.16.6 Q6: How do you handle view counting at scale without losing accuracy?

**Answer:** View counting at our scale (46K views/sec average, 140K peak) requires careful architecture:

**Not a simple database increment.** A naive `UPDATE videos SET views = views + 1` would cause massive lock contention on hot rows (viral videos could get thousands of increments per second on the same row).

**Our approach (multi-tier counting):**

1. **Client-side deduplication:** The video player only reports a “view” after 30 seconds of watch time (prevents accidental/bot views). A session token prevents duplicate reports from the same session.
  2. **Redis atomic counter:** View events hit a Redis cluster where we do `INCR video:{id}:views`. Redis handles 100K+ increments/sec per node easily. This is the “real-time” count shown to users.
  3. **Kafka event stream:** Every view event is also published to Kafka for durable recording. This is the source of truth for analytics.
  4. **Periodic DB flush:** A background consumer reads from Redis every 30 seconds and batch-updates PostgreSQL. This coalesces thousands of increments into a single UPDATE per video.
  5. **Anti-fraud pipeline:** A separate Kafka consumer runs view validation (IP rate limiting, bot detection, engagement scoring) and may subtract fraudulent views in a daily batch job.
-

### 5.16.7 Q7: What is your content moderation strategy?

**Answer:** Content moderation operates at multiple levels:

1. **Pre-publish automated scan** (within the transcoding pipeline):
    - Nudity/violence detection (computer vision model, ~95% accuracy)
    - Audio transcription + toxicity classification
    - Copyright fingerprint matching (Content ID)
    - Known-bad content hash matching (CSAM database via PhotoDNA)
  2. **Post-publish monitoring:**
    - User reports (flagging system)
    - Continued ML scanning on reported content at higher sensitivity
    - Trending content gets priority review
  3. **Human review:**
    - Content flagged by ML with confidence 60-95% goes to human reviewers
    - Content flagged with confidence > 95% is auto-removed (pending appeal)
    - Content with confidence < 60% is published but monitored
    - Human review team for appeals and edge cases
  4. **Actions:**
    - Age-restrict, demonetize, hide from recommendations, remove, or ban
- 

### 5.16.8 Q8: How do you design the subscription feed (timeline)?

**Answer:** When a creator publishes a video, all subscribers should see it in their feed. This is a classic fan-out problem:

**Hybrid fan-out approach:** - **Small creators (< 10K subscribers):** Fan-out on write. When creator publishes, push the video\_id into each subscriber's feed (stored in Redis sorted set by publish\_time). This makes reads  $O(1)$  – just fetch the sorted set. - **Large creators (> 10K subscribers):** Fan-out on read. Store the video in the creator's publish list. When a subscriber opens their feed, pull from all subscribed creators and merge/rank. Cache the result in Redis for 5 minutes. - **Mega creators (> 1M subscribers):** Same as large, but with additional optimizations – most subscribers are inactive on any given day, so fan-out on write would waste 90%+ of the work.

---

### 5.16.9 Q9: How do you ensure smooth playback across different network conditions?

**Answer:** Beyond ABR (covered in Q2), several techniques ensure smoothness:

- **Buffer management:** The player maintains a 30-60 second buffer when bandwidth allows. This absorbs short network hiccups without visible impact.
  - **Segment prefetching:** While playing segment N, we start downloading segment N+1 and N+2 in parallel.
  - **Quality floor:** Even in worst conditions, we serve at least 240p rather than buffering. Low quality is better than no video.
  - **Server-side bandwidth estimation:** CDN can analyze TCP throughput and include hints in HTTP headers, giving the client better initial estimates.
  - **Multi-CDN failover:** If one CDN PoP is slow, the client can failover to an alternative CDN mid-stream.
  - **Persistent connections:** HTTP/2 multiplexing avoids per-segment TCP handshake overhead.
-

### 5.16.10 Q10: How do you handle video search at scale?

**Answer:** Video search is powered by Elasticsearch with custom ranking:

**Indexing pipeline:** - On video publish, metadata (title, description, tags) is indexed in Elasticsearch via a Kafka consumer. - Auto-generated captions (speech-to-text) are indexed for spoken content search. - Refresh interval: 30 seconds (newly uploaded video searchable within a minute).

**Ranking signals:** - Text relevance (BM25 on title, description, tags, captions) - Recency boost (newer videos ranked higher) - Popularity (view count, engagement rate) - Creator authority (subscriber count, historical performance) - Personalization (user's watch history, preferred categories)

**Performance:** - 50 Elasticsearch shards, 1 replica each = 100 shard copies across cluster - Query latency: < 100ms p50, < 200ms p99 - Autocomplete: trie-based suggestions with Redis cache (< 50ms)

### 5.16.11 Q11: What is the data retention and storage tiering strategy?

**Answer:** Not all data has equal access patterns. We use storage tiering:

| Tier            | Storage Class        | Content                     | Cost   |
|-----------------|----------------------|-----------------------------|--------|
| Hot (0-30 days) | S3 Standard          | All new videos              | \$\$\$ |
| Warm (30-180d)  | S3 Infrequent Access | Videos with declining views | \$\$   |
| Cold (180d+)    | S3 Glacier Instant   | Rarely viewed archive       | \$     |
| Deep (2yr+)     | S3 Glacier Deep      | Raw uploads (legal hold)    | \$0.1  |

- Processed segments stay in Hot/Warm tiers (since CDN caches the popular ones).
- Raw uploads are moved to deep archive after transcoding is confirmed successful. Kept for re-transcoding needs (new codec support) and legal compliance.
- Lifecycle policies automate tier transitions based on access patterns.

### 5.16.12 Q12: How do you handle DRM (Digital Rights Management)?

**Answer:** DRM is essential for premium/paid content (Netflix model):

- **Widevine** (Google): Android, Chrome, smart TVs
- **FairPlay** (Apple): iOS, Safari, Apple TV
- **PlayReady** (Microsoft): Edge, Xbox, Windows apps

Each DRM system requires separate encryption of the same content. With CMAF, we use Common Encryption (CENC) so the same encrypted segments work with both Widevine and PlayReady. FairPlay requires its own encryption (SAMPLE-AES).

**License server flow:** 1. Client requests video manifest 2. Manifest includes DRM initialization data 3. Client sends license request to DRM license server 4. License server validates entitlement (does user have subscription?) 5. Returns decryption key with usage rules (expiry, device limits) 6. Client decrypts and plays segments in hardware-protected pipeline

**5.16.13 Q13: How do you handle multi-language and subtitles?**

**Answer:** - **Subtitle formats:** WebVTT (streaming) and SRT (upload). Stored alongside video segments in S3. - **Auto-generated captions:** Speech-to-text ML model runs as a transcoding DAG step. Supports 50+ languages. - **Community contributions:** Viewers can submit subtitle corrections. - **Manifest integration:** Subtitle tracks listed in HLS/DASH manifest as separate streams. Client selects based on user preference. - **Storage:** Subtitles are tiny (< 100 KB per language per video) so all language variants are pre-pushed to CDN.

---

**5.16.14 Q14: What is your approach to A/B testing the video player and recommendation algorithm?**

**Answer:** - **Player A/B testing:** Different ABR algorithms, buffer sizes, or initial quality selection. Metrics: TTFF, rebuffering ratio, average quality served. Use consistent hashing on user\_id to ensure stable assignment. - **Recommendation A/B testing:** Different ranking models or candidate generation strategies. Metrics: watch time, click-through rate, user retention. Careful about novelty effects – run tests for at least 2 weeks. - **Gradual rollout:** New algorithms start at 1% of traffic, monitored for regressions, then scaled to 5%, 20%, 50%, 100%. - **Holdback group:** Always maintain a small control group (1%) on the previous algorithm for continuous comparison.

---

**5.16.15 Q15: How would you estimate the total infrastructure cost?**

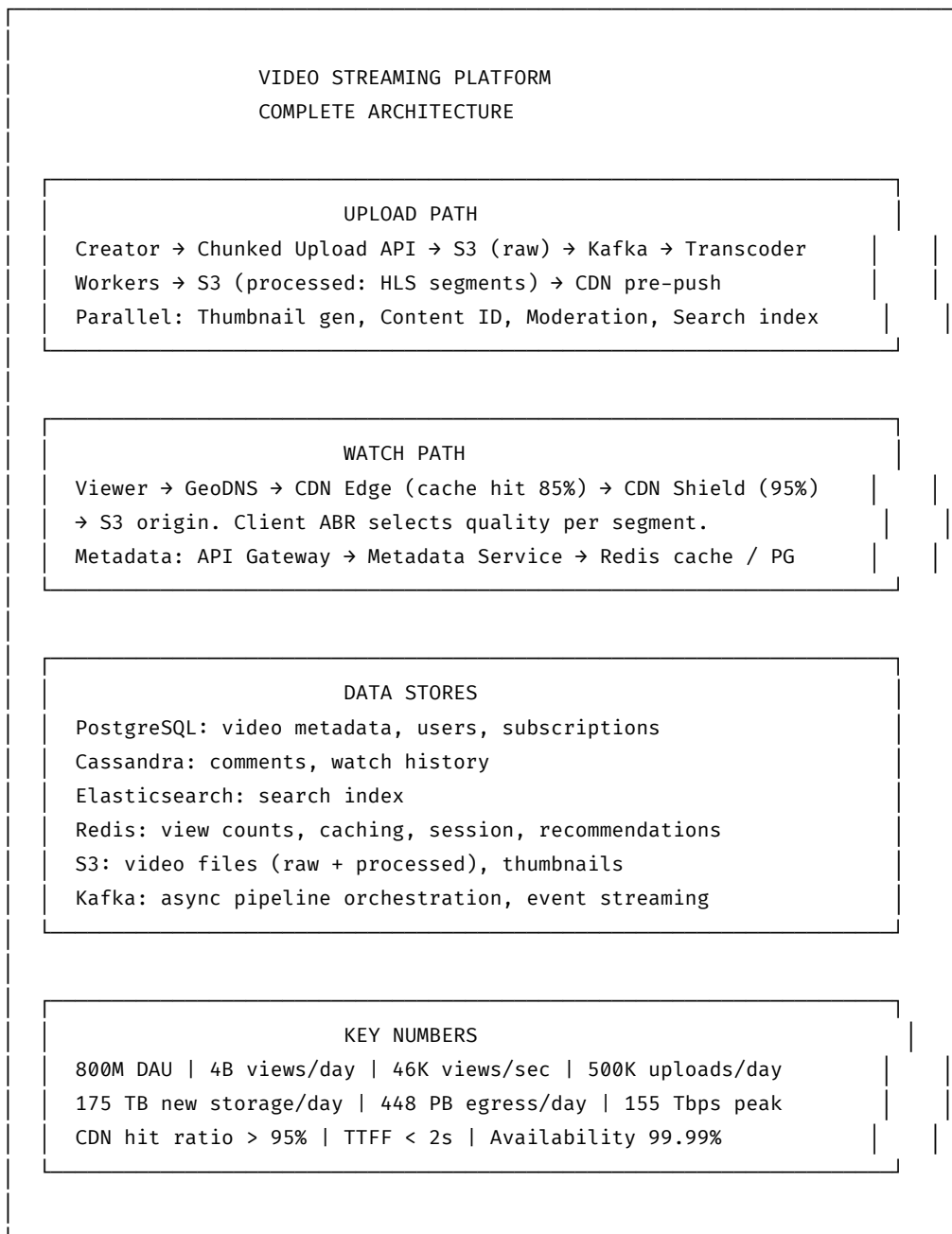
**Answer:** Rough order-of-magnitude for a platform at YouTube scale:

| Component                | Monthly Estimate                          |
|--------------------------|-------------------------------------------|
| CDN bandwidth (448 PB)   | \$500M - \$800M (at volume discounts)     |
| Object storage (64 PB)   | \$1.5M - \$2M                             |
| Compute (API + services) | \$10M - \$20M                             |
| Transcoding compute      | \$30M - \$50M (heavy spot instance usage) |
| Databases                | \$5M - \$10M                              |
| Redis / caching          | \$2M - \$5M                               |
| Elasticsearch            | \$3M - \$5M                               |
| ML / Recommendations     | \$10M - \$20M                             |
| Total                    | ~\$560M - \$910M / month                  |

CDN bandwidth dominates cost (70-80%+ of total). This is why Netflix and Google built their own CDN (Open Connect, Google Global Cache) – at their scale, owning the CDN infrastructure is vastly cheaper than paying a third party. Netflix places its own servers (Open Connect Appliances) directly inside ISP networks, eliminating transit costs entirely.

---

## 5.17 Appendix: Quick Reference Architecture



## Chapter 6

# Design a Notification System

**Difficulty:** Medium-Hard | **Companies:** Amazon, Google, Meta, Apple, Netflix, Uber

For the low-level class design, see [LLD: Notification System](#)

---

### 6.1 Table of Contents

1. Problem Statement and Scope
  2. Functional Requirements
  3. Non-Functional Requirements
  4. Back-of-Envelope Estimation
  5. API Design
  6. Data Model and Database Selection
  7. High-Level Architecture
  8. Deep Dive: Core Components
  9. Data Partitioning and Sharding
  10. Caching Strategy
  11. Replication and Consistency
  12. Fault Tolerance and Failure Handling
  13. Scalability
  14. Monitoring and Observability
  15. Trade-offs and Design Decisions
  16. Interview Deep-Dive Questions
- 

### 6.2 1. Problem Statement and Scope

Design a scalable, highly available notification system that delivers notifications across multiple channels – push notifications (iOS/Android), SMS, email, and in-app – with support for prioritization, rate limiting, user preference management, template-based rendering, deduplication, and guaranteed delivery.

The system must handle billions of notifications per day across a global user base, integrating with third-party providers (APNs, FCM, Twilio, SES) while providing consistent delivery tracking and analytics.

### 6.2.1 Scope Boundaries

| SCOPE DEFINITION               |                                                       |
|--------------------------------|-------------------------------------------------------|
| IN SCOPE                       | OUT OF SCOPE                                          |
| Push notifications (APNs, FCM) | Content moderation / filtering                        |
| Email delivery (SES, SendGrid) | Marketing campaign management UI                      |
| SMS delivery (Twilio, SNS)     | Email/SMS content authoring tools                     |
| In-app notifications           | User authentication / authorization                   |
| User preferences               | Billing for SMS/email usage                           |
| Template management            | A/B testing platform (referenced but not built)       |
| Delivery tracking              | Rich media processing (image resize, video transcode) |
| Priority-based routing         | Regulatory compliance engine (GDPR/CAN-SPAM details)  |
| Rate limiting                  |                                                       |
| Scheduled notifications        |                                                       |
| Batch/bulk sending             |                                                       |
| Deduplication                  |                                                       |
| Retry and DLQ handling         |                                                       |

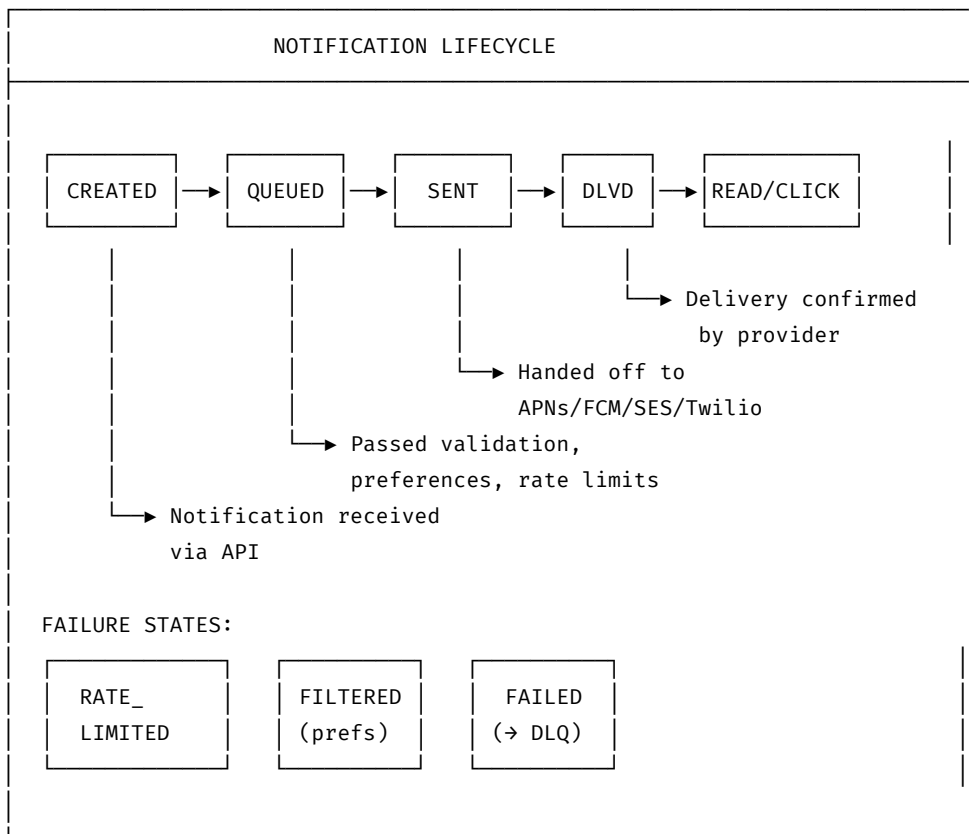
## 6.3 2. Functional Requirements

### 6.3.1 Core Features

- **F1 - Multi-channel delivery:** Push (iOS/Android), SMS, Email, In-App
- **F2 - User preferences:** Opt-in/out per channel, quiet hours, frequency caps
- **F3 - Template management:** Parameterized templates with personalization
- **F4 - Delivery tracking:** Status: created, queued, sent, delivered, read, clicked
- **F5 - Priority levels:** Urgent (P0), High (P1), Normal (P2), Low (P3)
- **F6 - Rate limiting:** Per-user, per-channel, per-notification-type limits
- **F7 - Batch notifications:** Bulk send to segments (millions of users)
- **F8 - Scheduled delivery:** Send at a future time, timezone-aware
- **F9 - Digest/aggregation:** Group related notifications into a single digest
- **F10 - Deduplication:** Prevent duplicate notifications within a time window



### 6.3.2 Notification Lifecycle



### 6.3.3 Channel-Specific Features

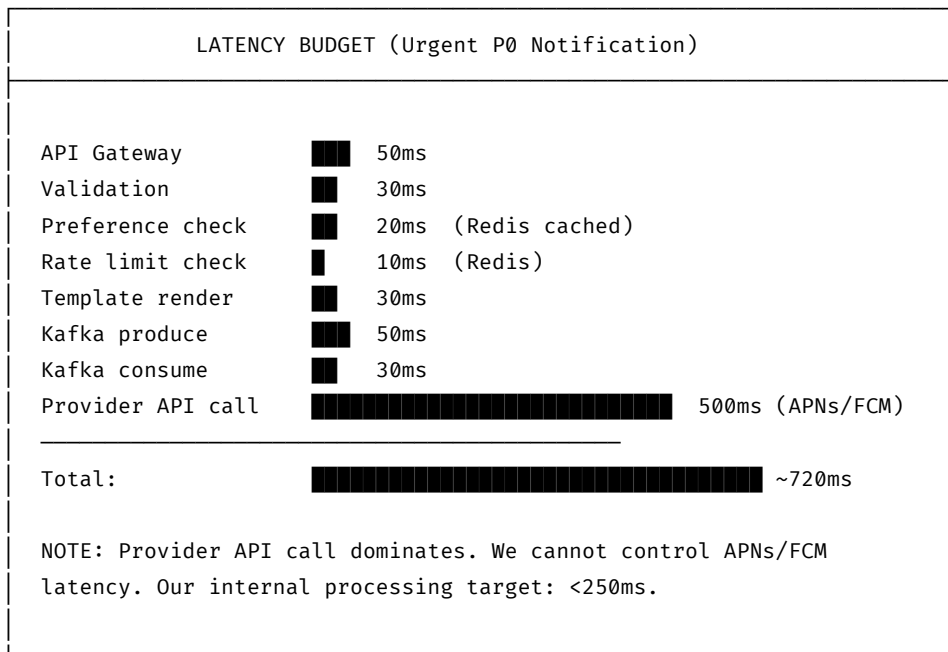
| CHANNEL CAPABILITIES |                   |                |                 |                     |
|----------------------|-------------------|----------------|-----------------|---------------------|
| Feature              | Push              | Email          | SMS             | In-App              |
| Rich text            | Limited           | Yes            | No              | Yes                 |
| Images               | Small             | Yes            | No              | Yes                 |
| Actions              | 2-3 btns          | Links          | Reply           | Buttons/Links       |
| Delivery confirm     | Provider callback | Bounce/receipt | Carrier receipt | Instant (WebSocket) |
| Cost                 | Free              | ~\$0.001       | ~\$0.01         | Free                |
| Latency              | <1s               | 1-30s          | 1-5s            | <100ms              |
| Read rate            | ~60%              | ~20%           | ~95%            | ~40%                |
| Offline delivery     | Stored by APNs    | Inbox          | Carrier retry   | Stored server-side  |

## 6.4 3. Non-Functional Requirements

- **Delivery latency:** <1s urgent, <5s high, <30s normal; Rationale: User expectation by priority
- **Availability:** 99.99% (52 min downtime/year); Rationale: Notifications are business-critical
- **Throughput:** 1B notifications/day, 50K/sec peak; Rationale: Scale for large user base

- **Delivery guarantee:** At-least-once; Rationale: Missed notification > duplicate
- **Deduplication:** Within 5-minute window; Rationale: Prevent spam from retries
- **Data retention:** 90 days hot, 1 year cold; Rationale: Analytics and audit trail
- **Global distribution:** Multi-region deployment; Rationale: Low latency worldwide
- **Provider failover:** <30s switchover; Rationale: No single provider dependency

### 6.4.1 Latency Budget Breakdown



## 6.5 4. Back-of-Envelope Estimation

### 6.5.1 Traffic Estimates

| TRAFFIC ESTIMATION    |                                                   |
|-----------------------|---------------------------------------------------|
| Total notifications:  | 1,000,000,000 / day                               |
| Average rate:         | 1B / 86400 $\approx$ 11,574 / sec                 |
| Peak rate (4x avg):   | $\sim$ 50,000 / sec                               |
| Burst rate (10x avg): | $\sim$ 120,000 / sec (flash sales, breaking news) |
| CHANNEL BREAKDOWN:    |                                                   |
| Push (40%):           | 400M/day = 4,630/sec avg                          |
| Email (30%):          | 300M/day = 3,472/sec avg                          |
| In-App (20%):         | 200M/day = 2,315/sec avg                          |
| SMS (10%):            | 100M/day = 1,157/sec avg                          |
| PRIORITY BREAKDOWN:   |                                                   |
| Urgent (P0):          | 1% = 10M/day = $\sim$ 115/sec                     |
| High (P1):            | 9% = 90M/day = $\sim$ 1,040/sec                   |
| Normal (P2):          | 70% = 700M/day = $\sim$ 8,100/sec                 |
| Low (P3):             | 20% = 200M/day = $\sim$ 2,315/sec                 |

## 6.5.2 Storage Estimates

| STORAGE ESTIMATION           |                                            |
|------------------------------|--------------------------------------------|
| NOTIFICATION RECORD:         |                                            |
| notification_id:             | 16 bytes (UUID)                            |
| user_id:                     | 16 bytes                                   |
| channel:                     | 4 bytes (enum)                             |
| priority:                    | 4 bytes                                    |
| template_id:                 | 16 bytes                                   |
| rendered_content:            | 200 bytes (avg)                            |
| metadata/payload:            | 150 bytes                                  |
| status:                      | 4 bytes                                    |
| timestamps:                  | 40 bytes (5×8)                             |
| provider_response:           | 50 bytes                                   |
| <hr/>                        |                                            |
| Total per record: ~500 bytes |                                            |
| DAILY STORAGE:               |                                            |
| Notifications:               | 1B * 500B = 500 GB/day                     |
| Delivery logs:               | 1B * 200B = 200 GB/day (status updates)    |
| Analytics:                   | 1B * 100B = 100 GB/day                     |
| <hr/>                        |                                            |
| Total daily:                 | ~800 GB/day                                |
| 90-day hot storage:          | ~72 TB                                     |
| 1-year cold storage:         | ~292 TB                                    |
| ANCILLARY STORAGE:           |                                            |
| User preferences:            | 500M users * 1KB = 500 GB (MySQL)          |
| Device tokens:               | 500M users * 200B = 100 GB (Redis + MySQL) |
| Templates:                   | 100K * 5KB = 500 MB (MySQL)                |

## 6.5.3 Bandwidth Estimates

|                         |                           |              |
|-------------------------|---------------------------|--------------|
| Ingress (API requests): | 50K/sec * 1KB avg payload | = 50 MB/sec  |
| Egress (to providers):  | 50K/sec * 2KB avg payload | = 100 MB/sec |
| Internal (Kafka):       | 50K/sec * 1.5KB           | = 75 MB/sec  |

## 6.6 5. API Design

### 6.6.1 5.1 Send Notification

```
POST /v1/notifications/send
Authorization: Bearer <api-key>
X-Idempotency-Key: <uuid>
Content-Type: application/json
```

```
{
 "recipients": [
 {
 "user_id": "user_12345",
 "channels": ["push", "email"], // optional override
 "data": { // per-recipient personalization
 "first_name": "Ajay",
 "order_id": "ORD-98765"
 }
 }
],
 "template_id": "order_shipped_v2",
 "channel": "push", // default channel
 "priority": "high", // urgent | high | normal | low
 "category": "transactional", // transactional | marketing | system
 "schedule_time": "2026-04-10T09:00:00Z", // optional, null = immediate
 "ttl_seconds": 86400, // notification expiry
 "collapse_key": "order_update_98765", // replaces prior with same key
 "data": { // global template data
 "company_name": "Acme Corp"
 },
 "options": {
 "respect_quiet_hours": true,
 "allow_digest": true,
 "track_clicks": true
 }
}
```

Response 202 Accepted:

```
{
 "request_id": "req_abc123",
 "notifications": [
 {
 "notification_id": "notif_xyz789",
 "user_id": "user_12345",
 "status": "queued",
 "estimated_delivery": "2026-04-10T09:00:00Z"
 }
]
}
```

### 6.6.2 5.2 Get In-App Notifications

GET /v1/notifications/users/{user\_id}?status=unread&limit=20&cursor=abc123

Authorization: Bearer <token>

Response 200:

```
{
 "notifications": [
 {
 "notification_id": "notif_xyz789",
 "title": "Your order has shipped!",
 "body": "Order ORD-98765 is on its way ... ",
 "type": "order_update",
 "priority": "high",
 "created_at": "2026-04-09T14:30:00Z",
 "read_at": null,
 "action_url": "/orders/ORD-98765",
 "image_url": "https://cdn.example.com/icons/shipping.png",
 "metadata": { "order_id": "ORD-98765" }
 }
],
 "cursor": "def456",
 "unread_count": 12,
 "has_more": true
}
```

### 6.6.3 5.3 Update User Preferences

PUT /v1/users/{user\_id}/preferences

Authorization: Bearer <token>

```
{
 "channels": {
 "push": {
 "enabled": true,
 "categories": {
 "transactional": true,
 "marketing": false,
 "social": true
 }
 },
 "email": {
 "enabled": true,
 "categories": {
 "transactional": true,
 "marketing": true, // subscribed to marketing emails
 "social": false
 },
 "digest": {
 "enabled": true,
 "frequency": "daily", // daily | weekly
 "time": "09:00",
 "timezone": "America/New_York"
 }
 },
 "sms": { "enabled": false },
 "in_app": { "enabled": true }
 },
 "quiet_hours": {
 "enabled": true,
 "start": "22:00",
 "end": "08:00",
 "timezone": "America/New_York",
 "override_for_urgent": true // P0 bypasses quiet hours
 },
 "frequency_cap": {
 "max_per_day": 50,
 "max_push_per_hour": 5
 }
}
```

## 6.6.4 5.4 Template Management

POST /v1/templates

Authorization: Bearer <admin-api-key>

```
{
 "template_id": "order_shipped_v2",
 "name": "Order Shipped Notification",
 "channels": {
 "push": {
 "title": "Your order is on its way! {{emoji}}",
 "body": "Hi {{first_name}}, order {{order_id}} has shipped.",
 "action": "OPEN_ORDER",
 "image": "{{product_image_url}}"
 },
 "email": {
 "subject": "Order {{order_id}} has shipped",
 "html_template": "<html>...</html>",
 "text_template": "Hi {{first_name}}, your order... "
 },
 "sms": {
 "body": "Hi {{first_name}}, order {{order_id}} shipped. Track: {{tracking_url}}"
 },
 "in_app": {
 "title": "Order Shipped",
 "body": "Your order {{order_id}} is on its way!",
 "icon": "shipping",
 "action_url": "/orders/{{order_id}}"
 }
 },
 "category": "transactional",
 "variables": ["first_name", "order_id", "tracking_url", "product_image_url"],
 "version": 2,
 "active": true
}
```



### 6.6.5 5.5 Delivery Status Webhook

POST /v1/webhooks/delivery-status (registered callback URL)

```
{
 "notification_id": "notif_xyz789",
 "user_id": "user_12345",
 "channel": "push",
 "status": "delivered", // delivered | bounced | failed | clicked
 "provider": "fcm",
 "provider_message_id": "fcm_msg_abc",
 "timestamp": "2026-04-09T14:30:05Z",
 "metadata": {
 "device_type": "android",
 "device_token": "abc ... xyz"
 }
}
```

---

## 6.7 6. Data Model and Database Selection

### 6.7.1 Database Selection Rationale

| DATABASE SELECTION    |                                |                                                                   |
|-----------------------|--------------------------------|-------------------------------------------------------------------|
| Data Type             | Database                       | Rationale                                                         |
| Notification records  | Cassandra                      | Write-heavy, time-series, horizontal scaling, TTL support         |
| User preferences      | MySQL (Aurora)                 | Read-heavy, relational, strong consistency for preference updates |
| Templates             | MySQL (Aurora)                 | Structured, versioned, low volume, needs ACID                     |
| Delivery status logs  | Cassandra                      | Append-only, high write throughput, time-based partitioning       |
| In-app notifications  | Redis (hot) + Cassandra (cold) | Fast reads for unread count; Cassandra for historical feed        |
| Device tokens         | Redis (cache) + MySQL (source) | Frequent lookups, needs persistence                               |
| Rate limit counters   | Redis                          | Atomic counters, TTL, in-memory speed                             |
| Dedup keys            | Redis                          | Fast lookup, auto-expire with TTL                                 |
| Analytics / Reporting | ClickHouse or BigQuery         | Columnar, fast aggregates on billions of rows                     |

## 6.7.2 Schema Design

### 6.7.2.1 Cassandra: Notification Records

```
CREATE TABLE notifications (
 user_id UUID,
 created_date DATE, -- partition by day for TTL cleanup
 notification_id TIMEUUID, -- clustering, sorted by time
 channel TEXT, -- push | email | sms | in_app
 priority INT,
 template_id TEXT,
 title TEXT,
 body TEXT,
 status TEXT, -- created | queued | sent | delivered | read
 category TEXT,
 metadata MAP<TEXT, TEXT>,
 created_at TIMESTAMP,
 sent_at TIMESTAMP,
 delivered_at TIMESTAMP,
 read_at TIMESTAMP,
 PRIMARY KEY ((user_id, created_date), notification_id)
) WITH CLUSTERING ORDER BY (notification_id DESC)
AND default_time_to_live = 7776000; -- 90 days
```

### 6.7.2.2 Cassandra: Delivery Logs

```
CREATE TABLE delivery_logs (
 notification_id TIMEUUID,
 attempt_num INT,
 channel TEXT,
 provider TEXT, -- apns | fcm | ses | twilio
 status TEXT, -- sent | delivered | bounced | failed
 provider_msg_id TEXT,
 error_code TEXT,
 error_message TEXT,
 latency_ms INT,
 timestamp TIMESTAMP,
 PRIMARY KEY ((notification_id), attempt_num)
) WITH default_time_to_live = 7776000;
```

### 6.7.2.3 MySQL: User Preferences

```
CREATE TABLE user_preferences (
 user_id VARCHAR(36) PRIMARY KEY,
 push_enabled BOOLEAN DEFAULT TRUE,
 email_enabled BOOLEAN DEFAULT TRUE,
 sms_enabled BOOLEAN DEFAULT FALSE,
 in_app_enabled BOOLEAN DEFAULT TRUE,
 quiet_hours_enabled BOOLEAN DEFAULT FALSE,
 quiet_hours_start TIME,
 quiet_hours_end TIME,
 timezone VARCHAR(50) DEFAULT 'UTC',
 max_per_day INT DEFAULT 100,
 max_push_per_hour INT DEFAULT 10,
 digest_enabled BOOLEAN DEFAULT FALSE,
 digest_frequency ENUM('daily', 'weekly') DEFAULT 'daily',
 digest_time TIME DEFAULT '09:00:00',
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
 INDEX idx_updated_at (updated_at)
);
```

```
CREATE TABLE user_channel_category_preferences (
 user_id VARCHAR(36),
 channel ENUM('push', 'email', 'sms', 'in_app'),
 category VARCHAR(50), -- transactional, marketing, social, system
 enabled BOOLEAN DEFAULT TRUE,
 PRIMARY KEY (user_id, channel, category),
 FOREIGN KEY (user_id) REFERENCES user_preferences(user_id)
);
```

### 6.7.2.4 MySQL: Device Tokens

```
CREATE TABLE device_tokens (
 device_token_id VARCHAR(36) PRIMARY KEY,
 user_id VARCHAR(36) NOT NULL,
 platform ENUM('ios', 'android', 'web') NOT NULL,
 token VARCHAR(512) NOT NULL,
 app_version VARCHAR(20),
 is_active BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 last_used_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 INDEX idx_user_id (user_id),
 UNIQUE INDEX idx_token (token)
);
```

### 6.7.2.5 MySQL: Templates

```
CREATE TABLE notification_templates (
 template_id VARCHAR(100),
 version INT,
 name VARCHAR(255),
 category VARCHAR(50),
 push_title TEXT,
 push_body TEXT,
 email_subject TEXT,
 email_html MEDIUMTEXT,
 email_text TEXT,
 sms_body TEXT,
 in_app_title TEXT,
 in_app_body TEXT,
 in_app_icon VARCHAR(100),
 variables JSON, -- list of required variables
 is_active BOOLEAN DEFAULT TRUE,
 created_by VARCHAR(36),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
 PRIMARY KEY (template_id, version)
);
```

### 6.7.2.6 Redis: In-App Notification Feed

Key: in\_app:{user\_id} (Sorted Set)  
 Score: timestamp (epoch ms)  
 Value: notification\_id

Key: in\_app:unread:{user\_id} (Counter)  
 Value: integer count

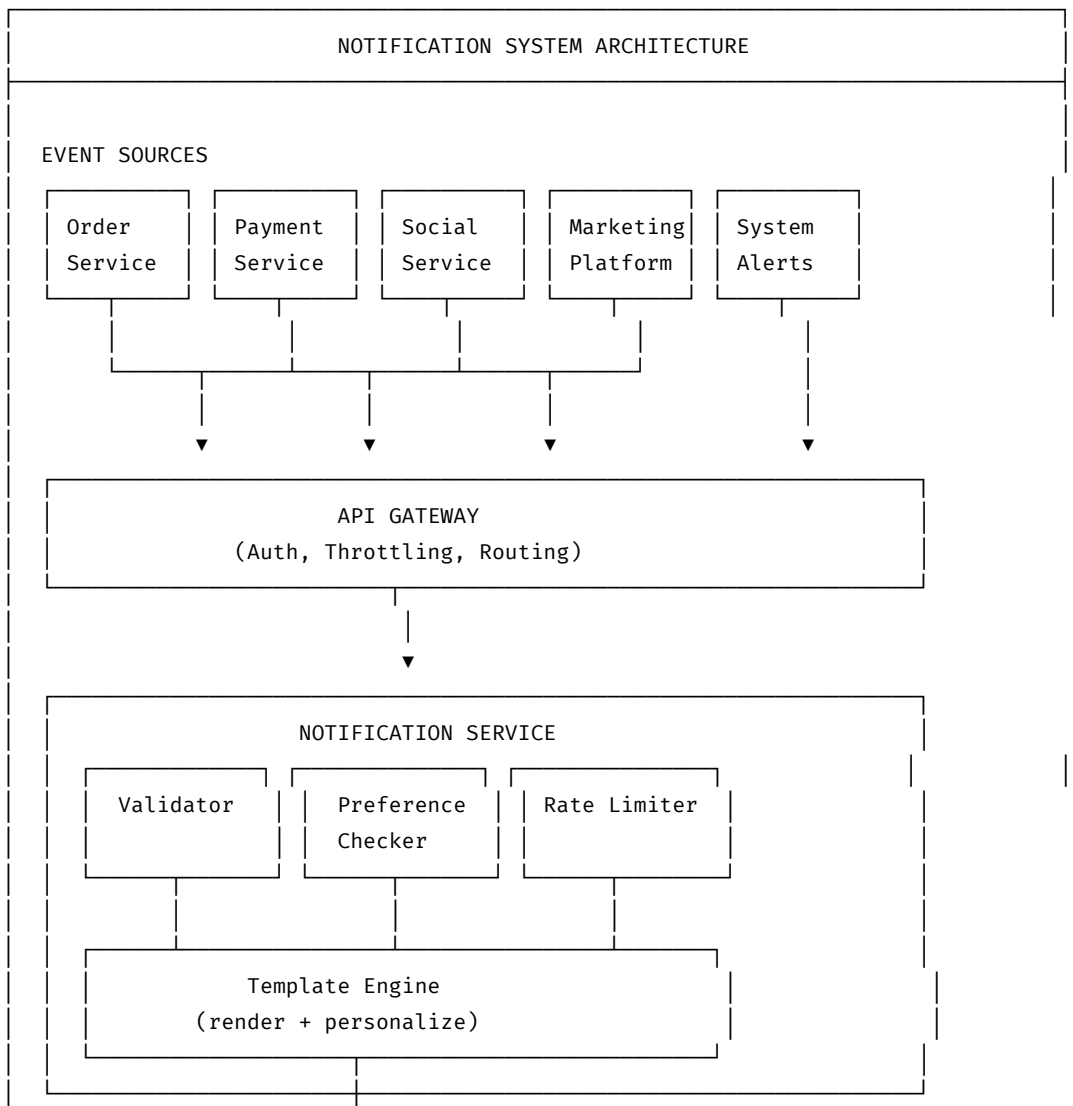
Key: in\_app:detail:{notif\_id} (Hash)  
 Fields: title, body, type, action\_url, image\_url, read, created\_at

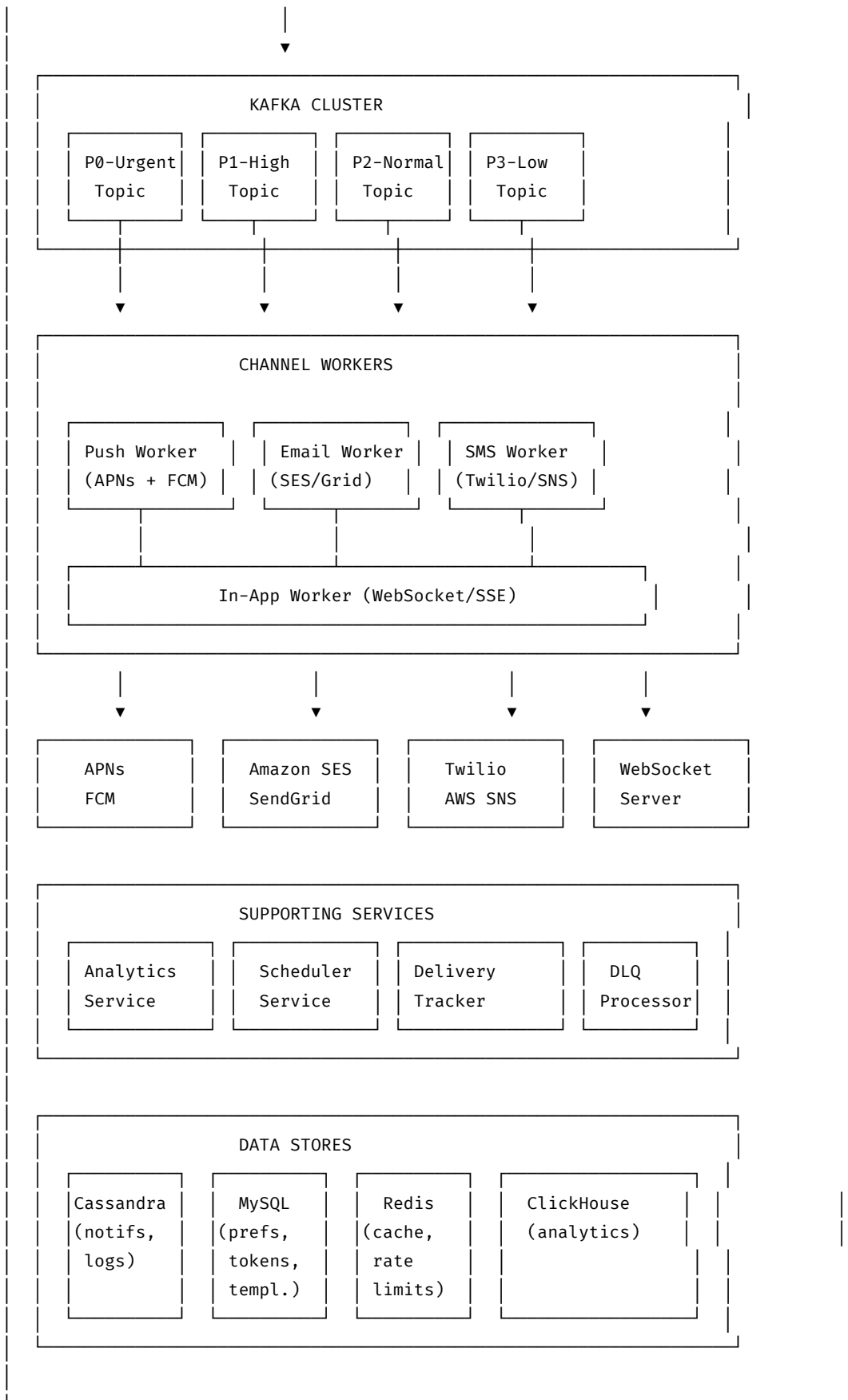
TTL: 30 days

---

## 6.8 7. High-Level Architecture

### 6.8.1 Overall System Architecture

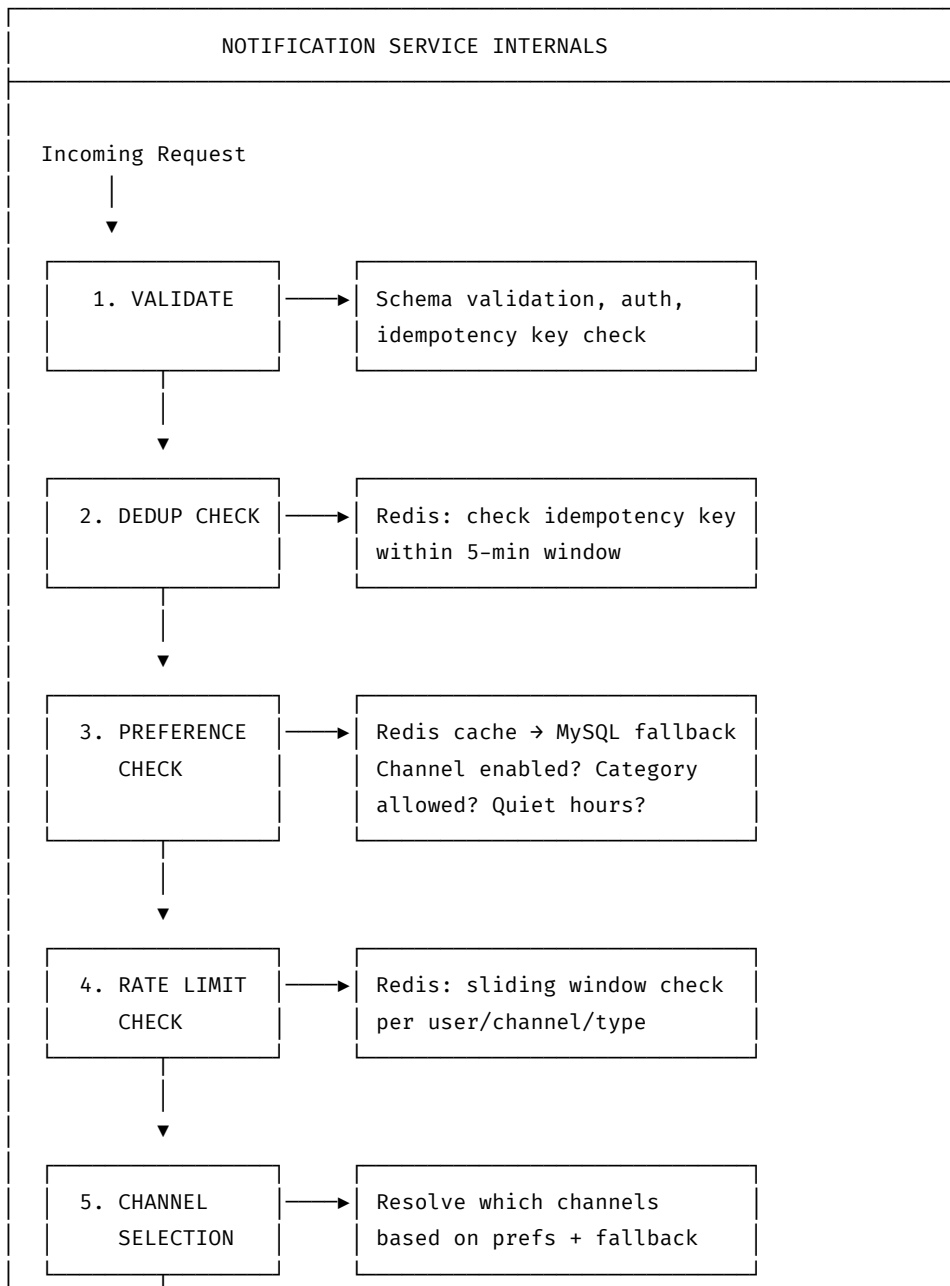




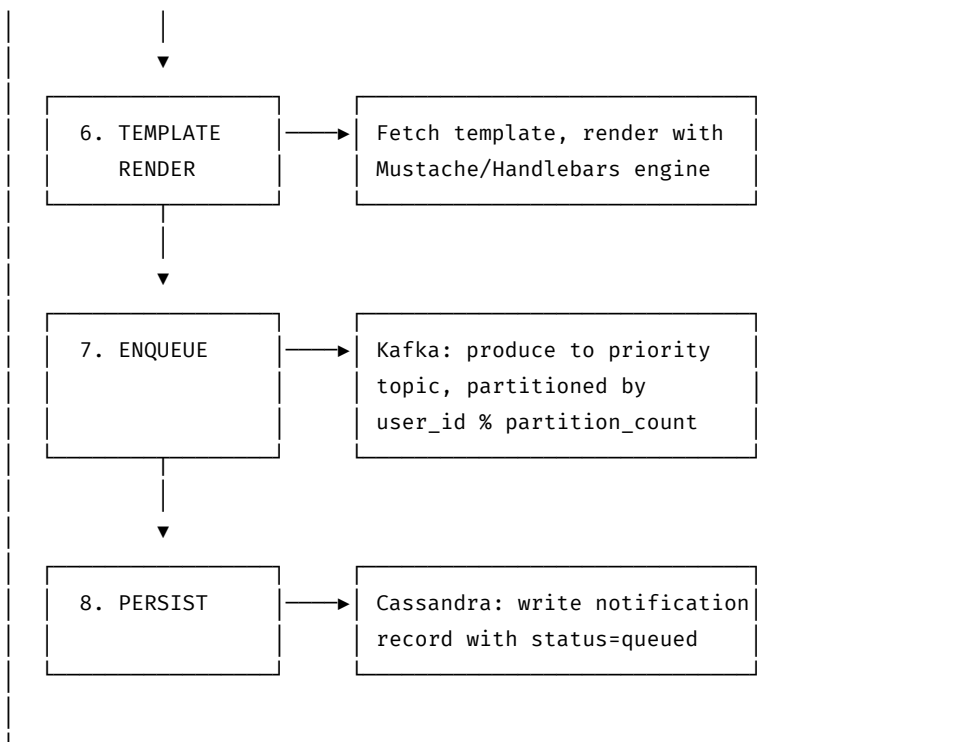
## 6.8.2 7.1 Component Breakdown

### 6.8.2.1 Notification Service (Core Orchestrator)

The central service that receives notification requests and orchestrates the processing pipeline. It is stateless and horizontally scalable.

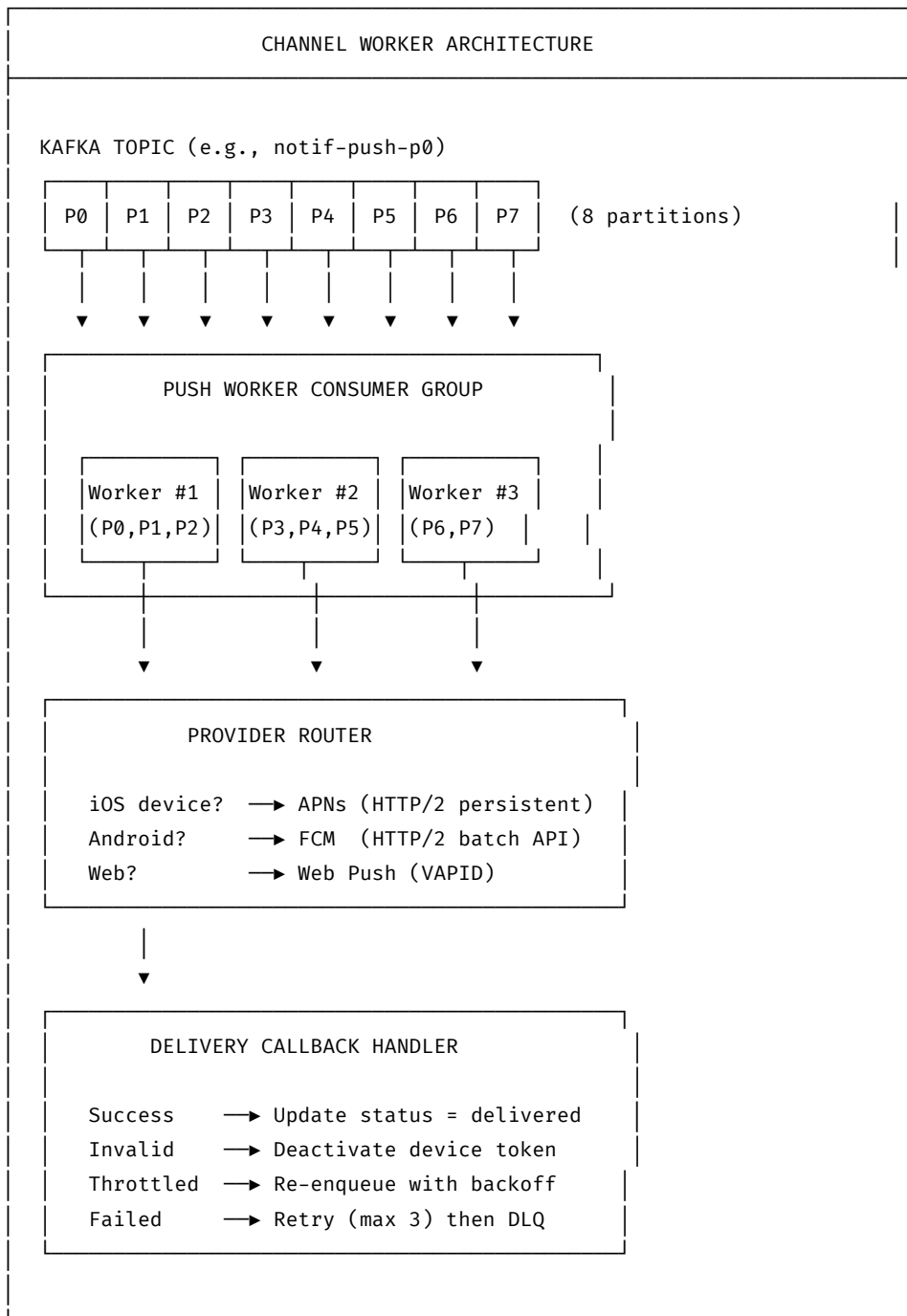






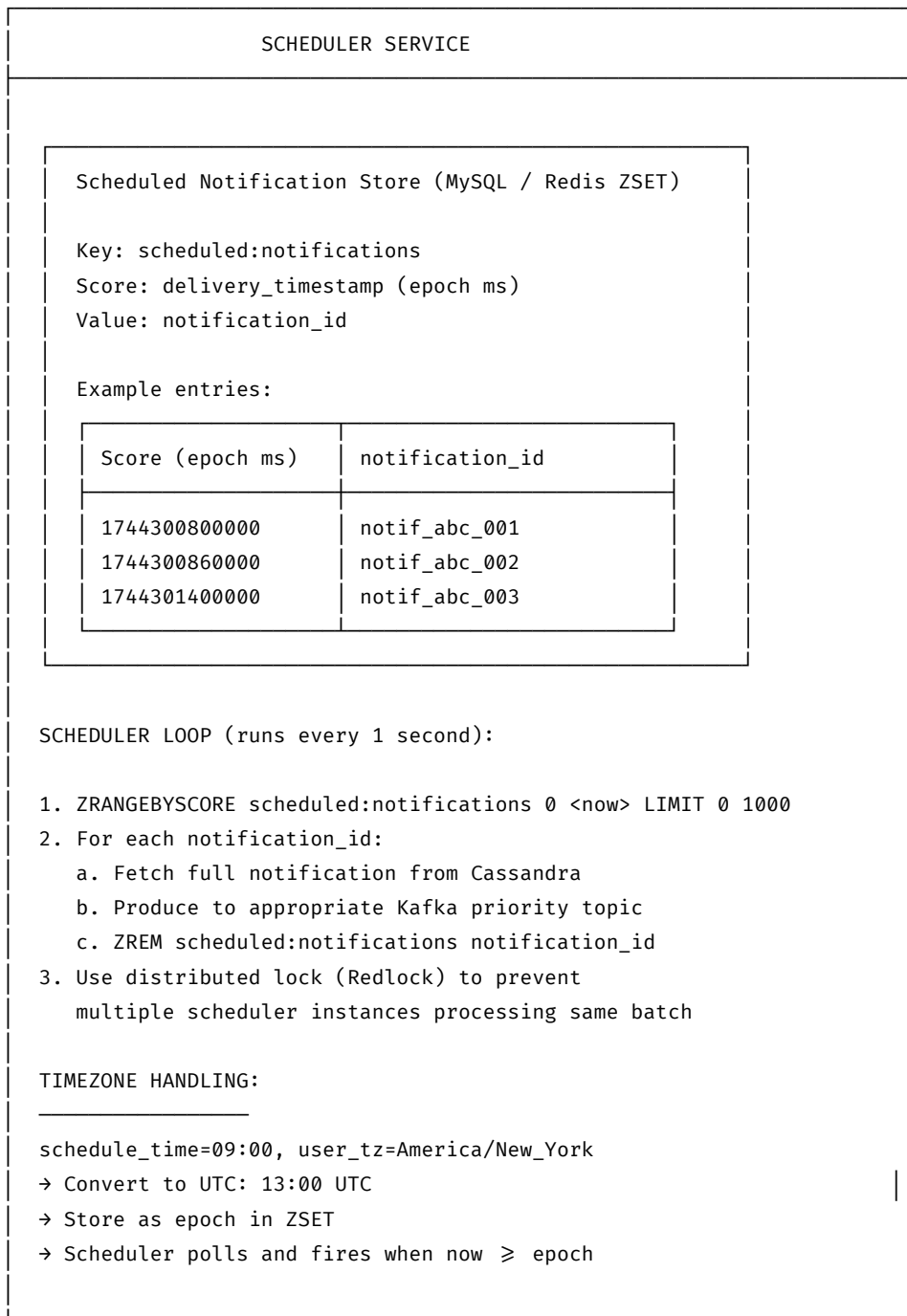
### 6.8.2.2 Channel Workers

Each channel has its own fleet of worker instances that consume from Kafka topics and deliver to the respective third-party provider.



### 6.8.2.3 Scheduler Service

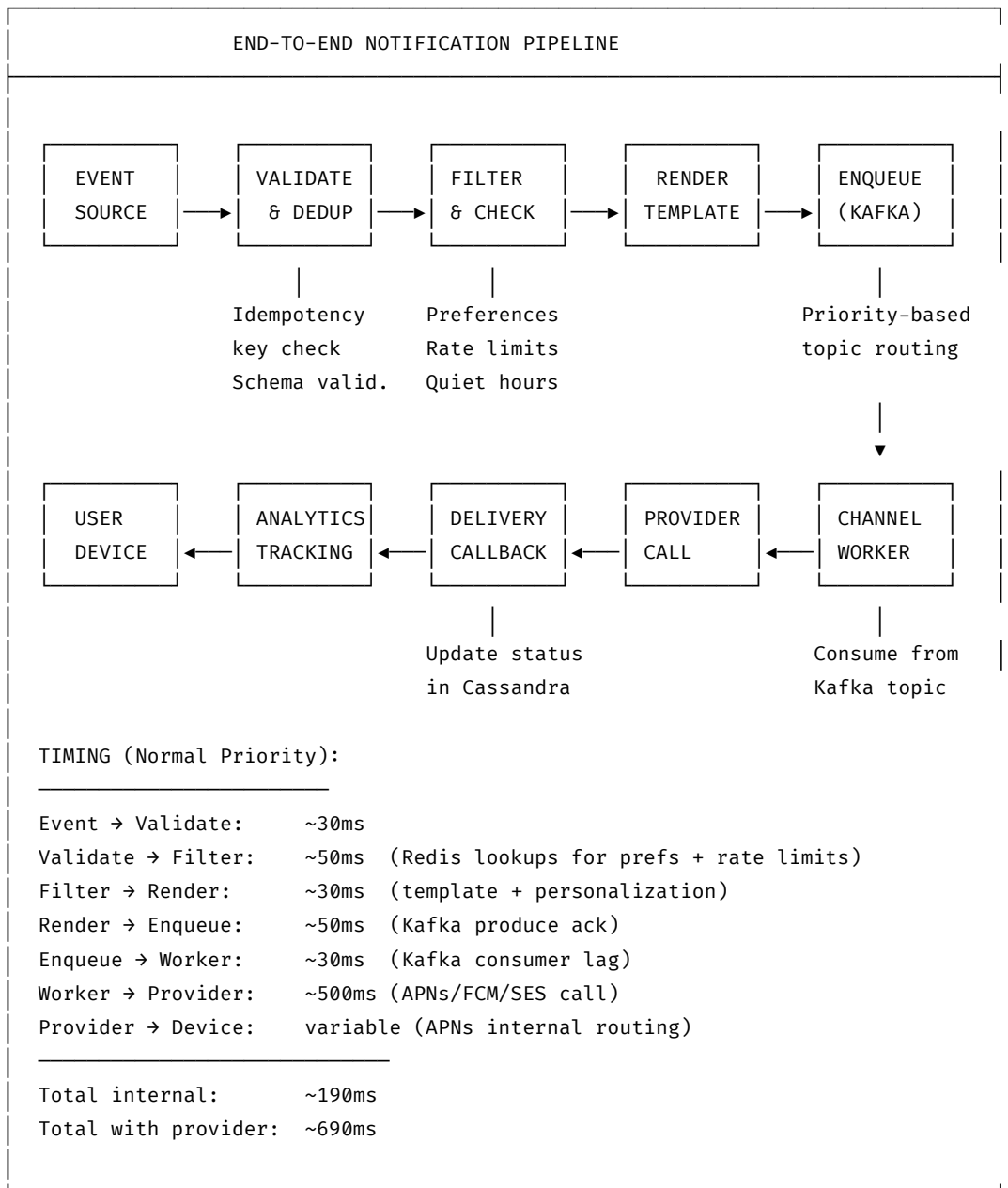
Handles deferred and timezone-aware notification delivery.



## 6.9 8. Deep Dive: Core Components

### 6.9.1 8.1 Notification Processing Pipeline

The end-to-end flow from event source to user device:



### 6.9.1.1 Pipeline Step Details

**Step 1: Validation and Deduplication** - Verify API key and permissions - Schema validation (required fields, valid channel, valid priority) - Check idempotency key in Redis: `SETNX dedup:{idempotency_key} 1 EX 300` - If key exists, return cached response (deduplicated)

**Step 2: Preference and Rate Limit Check** - Fetch user preferences from Redis (cache hit ~95%) or MySQL (cache miss) - Check if channel is enabled for user - Check if notification category is enabled for user on this channel - Check quiet hours: convert current time to user timezone, compare with quiet window - If in quiet hours and priority != P0, hold until quiet hours end - Rate limit check: `INCR rate:{user_id}:{channel}:{window}` with TTL - Exceeds limit? Drop notification with status=RATE\_LIMITED

**Step 3: Channel Selection and Template Rendering** - If sender specified channels, intersect with user-enabled channels - If no channels specified, use all user-enabled channels (fan-out) - Fetch template from local cache (TTL 5 minutes) or MySQL - Render template with user-specific data

using Mustache/Handlebars engine - Generate channel-specific payloads (different formats for push vs. email vs. SMS)

**Step 4: Kafka Enqueue** - Select Kafka topic by priority: `notif-{channel}-p{priority}` - Partition key: `user_id` (ensures ordering per user) - Message includes: `notification_id`, rendered content, device tokens, metadata - Async produce with `acks=all` for durability

## 6.9.2 8.2 Priority Queue and Ordering

### 6.9.2.1 Kafka Topic Design

| KAFKA TOPIC TOPOLOGY               |                                |                         |
|------------------------------------|--------------------------------|-------------------------|
| PRIORITY TOPICS (per channel):     |                                |                         |
| <code>notif-push-p0-urgent</code>  | (4 partitions, replication=3)  |                         |
| <code>notif-push-p1-high</code>    | (8 partitions, replication=3)  |                         |
| <code>notif-push-p2-normal</code>  | (32 partitions, replication=3) |                         |
| <code>notif-push-p3-low</code>     | (16 partitions, replication=3) |                         |
| <code>notif-email-p0-urgent</code> | (4 partitions, replication=3)  |                         |
| <code>notif-email-p1-high</code>   | (8 partitions, replication=3)  |                         |
| <code>notif-email-p2-normal</code> | (32 partitions, replication=3) |                         |
| <code>notif-email-p3-low</code>    | (16 partitions, replication=3) |                         |
| <code>notif-sms-p0-urgent</code>   | (4 partitions, replication=3)  |                         |
| <code>notif-sms-p1-high</code>     | (4 partitions, replication=3)  |                         |
| <code>notif-sms-p2-normal</code>   | (16 partitions, replication=3) |                         |
| <code>notif-sms-p3-low</code>      | (8 partitions, replication=3)  |                         |
| <code>notif-inapp-p0-urgent</code> | (4 partitions, replication=3)  |                         |
| <code>notif-inapp-p2-normal</code> | (16 partitions, replication=3) |                         |
| SPECIAL TOPICS:                    |                                |                         |
| <code>notif-dlq</code>             | (8 partitions)                 | Dead letter queue       |
| <code>notif-delivery-events</code> | (16 partitions)                | Delivery status updates |
| <code>notif-analytics</code>       | (8 partitions)                 | Analytics events        |

### 6.9.2.2 Consumer Group Allocation

| CONSUMER GROUP ALLOCATION STRATEGY                                                                                                                     |           |           |            |                |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------|------------|----------------|
| PRIORITY                                                                                                                                               | CONSUMERS | POLL RATE | BATCH SIZE | RATIONALE      |
| P0 (Urgent)                                                                                                                                            | 20        | 50ms      | 10         | Min latency    |
| P1 (High)                                                                                                                                              | 40        | 100ms     | 50         | Fast, batched  |
| P2 (Normal)                                                                                                                                            | 100       | 500ms     | 200        | Throughput     |
| P3 (Low)                                                                                                                                               | 30        | 1000ms    | 500        | Cost-efficient |
| STARVATION PREVENTION:                                                                                                                                 |           |           |            |                |
| Each consumer thread uses weighted fair queuing:                                                                                                       |           |           |            |                |
| <ul style="list-style-type: none"><li>- P0 gets 40% of processing capacity</li><li>- P1 gets 30%</li><li>- P2 gets 20%</li><li>- P3 gets 10%</li></ul> |           |           |            |                |
| When P0 queue is empty, its capacity shifts to lower priorities.<br>This ensures P3 notifications still get processed even under load.                 |           |           |            |                |
| CONSUMER ORDERING GUARANTEE:                                                                                                                           |           |           |            |                |
| Partition key = user_id                                                                                                                                |           |           |            |                |
| → All notifications for a user land on same partition                                                                                                  |           |           |            |                |
| → Single consumer processes them in order                                                                                                              |           |           |            |                |
| → Prevents "order shipped" arriving before "order placed"                                                                                              |           |           |            |                |

### 6.9.3 8.3 Push Notification Delivery

#### 6.9.3.1 APNs (Apple Push Notification Service)

##### APNs INTEGRATION

###### Push Worker

1. Get device token from cache/DB
2. Build APNs payload:

```
{
 "aps": {
 "alert": {
 "title": "Order Shipped",
 "body": "Your order is ... ",
 "loc-key": "ORDER_SHIP"
 },
 "badge": 5,
 "sound": "default",
 "category": "ORDER_UPDATE",
 "mutable-content": 1
 },
 "order_id": "ORD-98765"
}
```
3. Send via HTTP/2 persistent conn  
POST /3/device/{device\_token}  
Headers:

```
authorization: bearer <jwt>
apns-topic: com.app.bundle
apns-priority: 10 (immediate)
apns-collapse-id: order_98765
apns-expiration: <epoch>
```

## APNs RESPONSE HANDLING:

|                     |                                    |
|---------------------|------------------------------------|
| 200 OK              | → Status = sent, await feedback    |
| 400 BadRequest      | → Log error, don't retry           |
| 403 Forbidden       | → Certificate issue, alert ops     |
| 404 NotFound        | → Invalid token, deactivate device |
| 410 Gone            | → Token expired, remove from DB    |
| 429 TooManyRequests | → Back off, retry after delay      |
| 500/503             | → Retry with exponential backoff   |

## DEVICE TOKEN MANAGEMENT:

- Tokens stored in MySQL, cached in Redis (TTL 24h)
- On 410 Gone: DELETE from device\_tokens table
- On app uninstall: APNs feedback service reports invalid tokens (polled every 15 min)
- On token refresh: client sends new token via API, old token invalidated
- Multi-device: user may have 1-5 iOS devices, send to all active tokens



### 6.9.3.2 FCM (Firebase Cloud Messaging)

#### FCM INTEGRATION

##### FCM PAYLOAD:

```
{
 "message": {
 "token": "device_token_here",
 "notification": {
 "title": "Order Shipped",
 "body": "Your order ORD-98765 is on its way!"
 },
 "data": {
 "order_id": "ORD-98765",
 "click_action": "OPEN_ORDER"
 },
 "android": {
 "priority": "high",
 "notification": {
 "channel_id": "order_updates",
 "icon": "ic_shipping"
 },
 "ttl": "86400s"
 }
 }
}
```

##### BATCH SENDING:

FCM supports batch API: up to 500 messages per request  
POST <https://fcm.googleapis.com/batch>

Worker batches messages for efficiency:

- Accumulate up to 500 msgs or 100ms window
- Send as single HTTP request
- Parse individual responses from batch
- Handle per-token errors independently

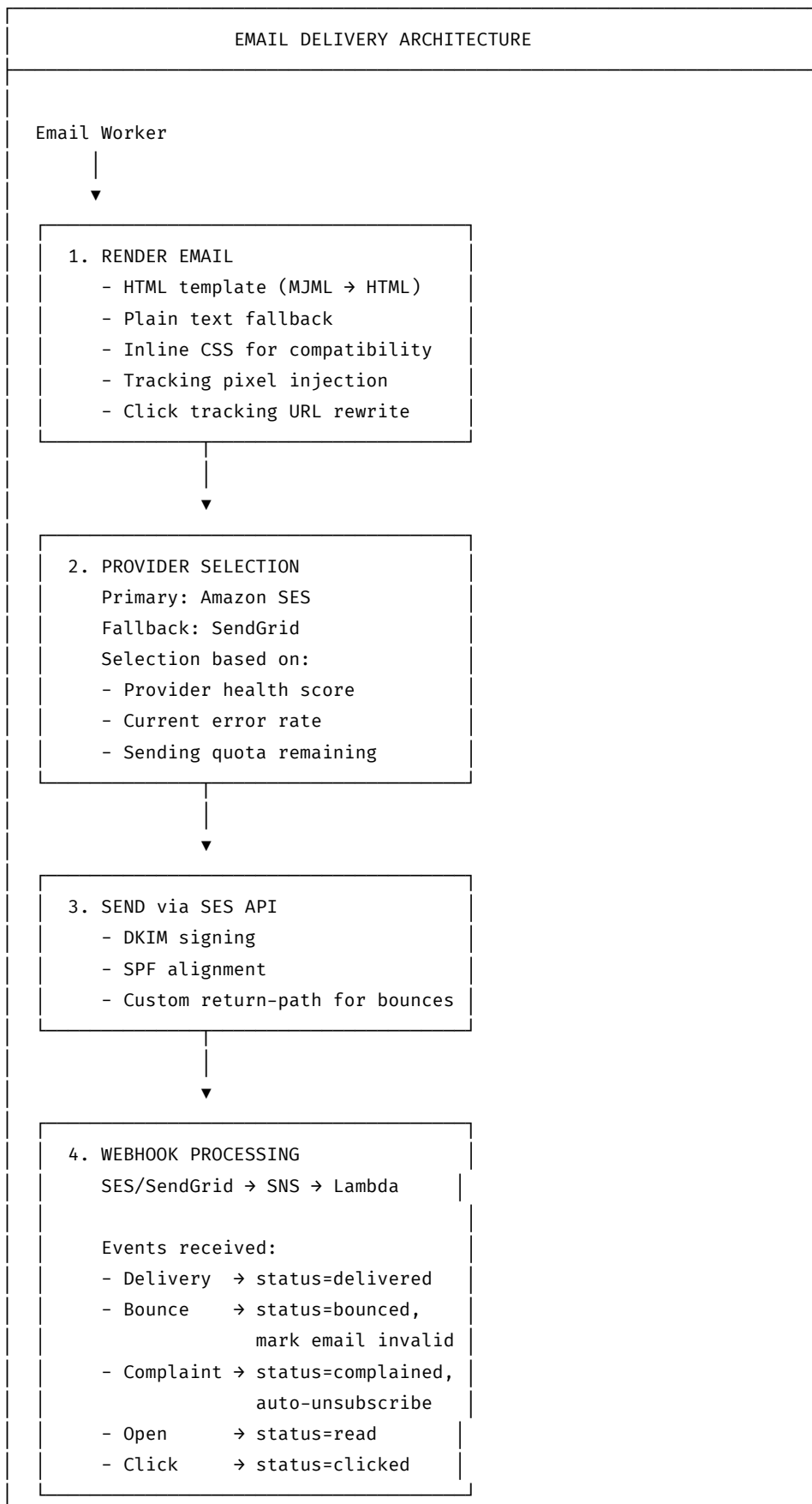
##### FCM ERROR HANDLING:

|                    |                                  |
|--------------------|----------------------------------|
| UNREGISTERED       | → Remove device token            |
| INVALID_ARGUMENT   | → Log, don't retry               |
| SENDER_ID_MISMATCH | → Config error, alert ops        |
| QUOTA_EXCEEDED     | → Back off, retry in 60s         |
| UNAVAILABLE        | → Retry with exponential backoff |
| INTERNAL           | → Retry with exponential backoff |

### 6.9.3.3 APNs vs FCM Comparison

| APNs vs FCM COMPARISON |                                       |                             |
|------------------------|---------------------------------------|-----------------------------|
| Feature                | APNs                                  | FCM                         |
| Protocol               | HTTP/2                                | HTTP/1.1 or HTTP/2          |
| Auth                   | JWT or Certificate                    | OAuth 2.0 (service account) |
| Max payload            | 4 KB                                  | 4 KB (data+notif combined)  |
| Batch send             | No (one-by-one)                       | Yes (up to 500)             |
| Topic messaging        | Yes (subscribe)                       | Yes (subscribe to topics)   |
| Delivery confirm       | Feedback service                      | Delivery receipts (limited) |
| Rate limit             | Per device token                      | Per project (quota)         |
| Token format           | Hex string (64ch)                     | Opaque string (~150ch)      |
| Silent push            | content-available:1                   | data-only message           |
| Priority levels        | 5 (immediately) /<br>10 (power-aware) | high / normal               |

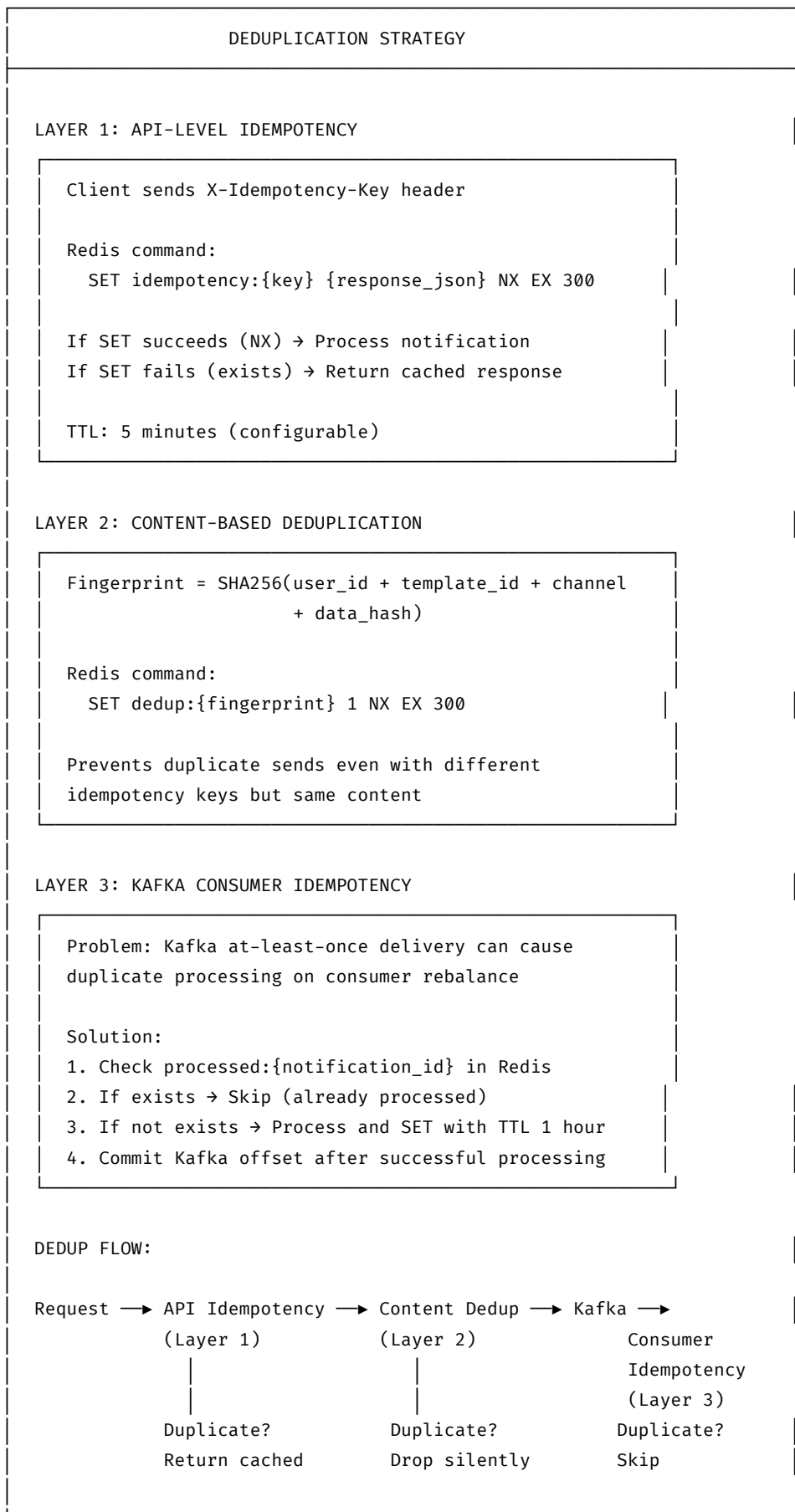
### 6.9.4 8.4 Email Delivery



## BOUNCE MANAGEMENT:

|                                                     |                                                |
|-----------------------------------------------------|------------------------------------------------|
| Hard bounce (invalid email)                         | → Permanently suppress                         |
| Soft bounce (inbox full)                            | → Retry 3x over 24h                            |
| Complaint (marked as spam)                          | → Auto-unsubscribe,<br>add to suppression list |
| Suppression list checked BEFORE sending every email |                                                |

### 6.9.5 8.5 Deduplication and Idempotency



### 6.9.6 8.6 Rate Limiting

#### RATE LIMITING DESIGN

##### MULTI-LEVEL RATE LIMITING:

##### Level 1: API Gateway

```
Per API key: 10,000 req/min (token bucket)
Global: 100,000 req/min
Burst: 2x sustained rate for 10 seconds
```

##### Level 2: Per User

```
Max notifications per user per day: 50 (default)
Max push per user per hour: 5
Max SMS per user per day: 3
Max email per user per day: 10
```

Implementation: Redis sliding window

EVAL "

```
 local key = KEYS[1]
 local window = ARGV[1]
 local limit = ARGV[2]
 local now = ARGV[3]
```

```
 redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
 local count = redis.call('ZCARD', key)
 if count < tonumber(limit) then
 redis.call('ZADD', key, now, now .. math.random())
 redis.call('EXPIRE', key, window)
 return 1 -- allowed
 end
```

```
 return 0 -- rate limited
```

" 1 rate:{user}:{channel} {window\_sec} {limit} {now}

## Level 3: Per Provider

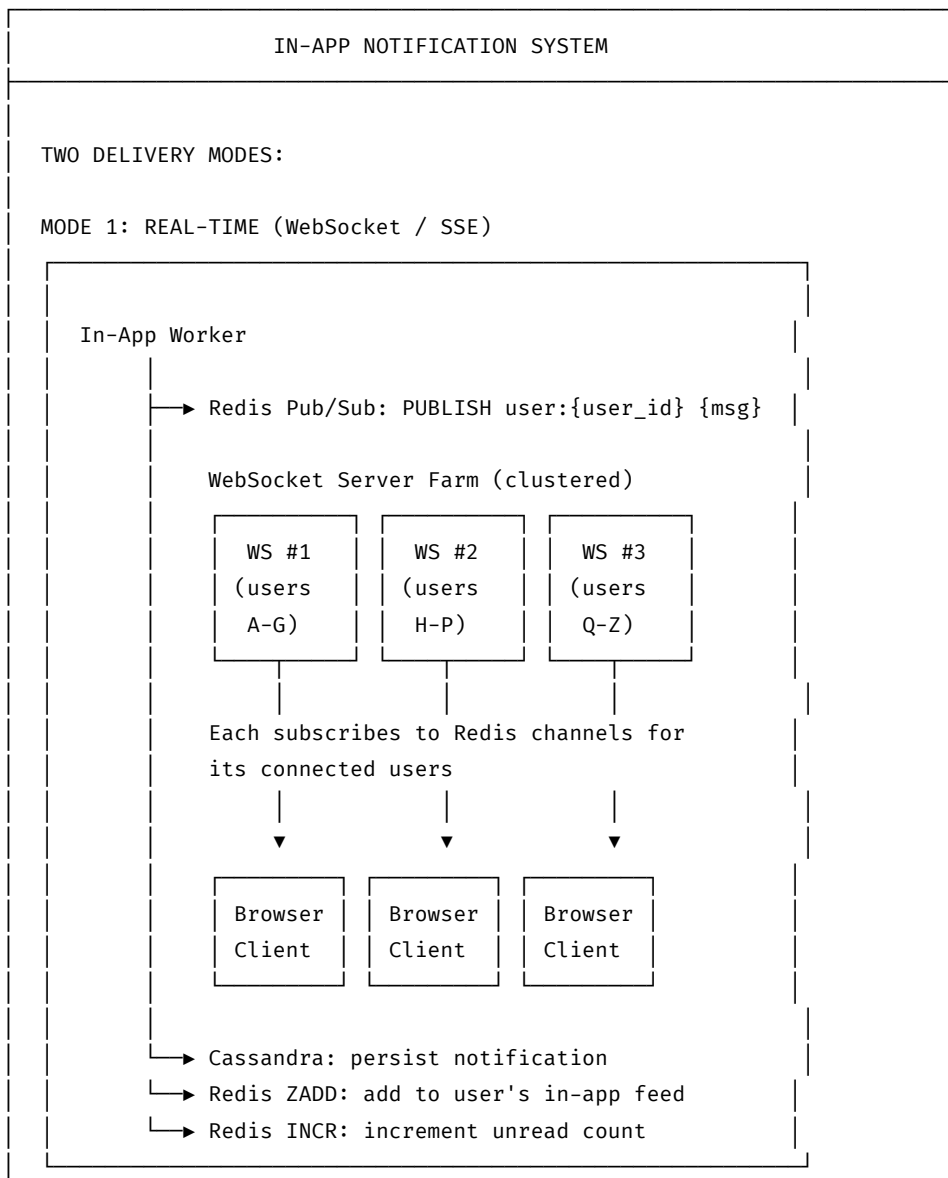
APNs: Respect per-device rate limits  
FCM: 600K msg/min per project  
SES: 200 emails/sec (adjustable)  
Twilio: Carrier-specific limits

Implementation: Token bucket at worker level  
If approaching limit → slow down consumer poll rate

## RATE LIMIT EXCEEDED BEHAVIOR:

Option A: Drop silently (for low-priority marketing)  
Option B: Queue for later (within TTL window)  
Option C: Aggregate into digest (if digest enabled)  
Option D: Return 429 to caller (API-level)

### 6.9.7 8.7 In-App Notification Delivery





## MODE 2: PULL (REST API polling)

Client polls GET /v1/notifications/users/{id}  
every 30-60 seconds when WebSocket unavailable

Response served from Redis (hot data, last 30 days)  
Falls back to Cassandra for older notifications

## UNREAD COUNT MANAGEMENT:

On new notification:

INCR in\_app:unread:{user\_id}

On mark-as-read:

DECR in\_app:unread:{user\_id}

HSET in\_app:detail:{notif\_id} read true

On mark-all-read:

SET in\_app:unread:{user\_id} 0

Batch update read flag in Cassandra (async)

## 6.9.8 8.8 Digest and Aggregation

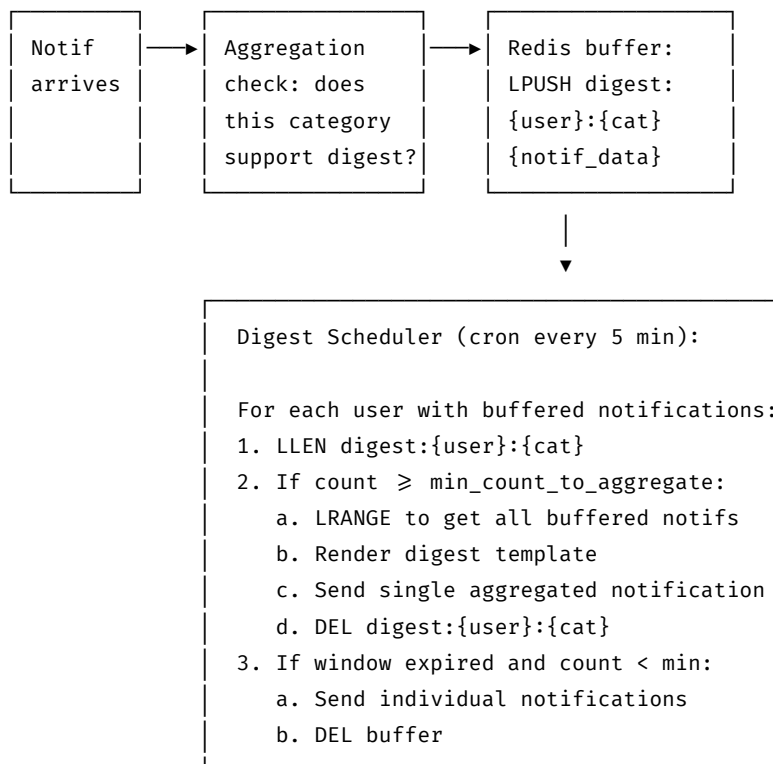
### DIGEST / AGGREGATION ENGINE

USE CASE: User receives 20 "new follower" notifications in 1 hour.  
Instead of 20 separate pushes, aggregate into:  
"You have 20 new followers including @alice and @bob"

#### AGGREGATION RULES:

```
Rule: {
 "category": "social.follow",
 "aggregate_window": "1h",
 "min_count_to_aggregate": 3,
 "digest_template": "new_followers_digest",
 "channels": ["push", "email"]
}
```

#### IMPLEMENTATION:



## DAILY EMAIL DIGEST:

- For users with `digest_frequency=daily`:
1. Collect all notifications from past 24h
  2. Group by category
  3. Render daily digest email template
  4. Send at user's preferred `digest_time` in their TZ
  5. Mark individual notifications as "digested"

## 6.10 9. Data Partitioning and Sharding

### 6.10.1 Partitioning Strategy

## PARTITIONING STRATEGY

## CASSANDRA: NOTIFICATION RECORDS

Partition key: `(user_id, created_date)`  
Clustering key: `notification_id (TIMEUUID DESC)`

WHY `user_id` + `date`?

- `user_id` alone → partitions grow unbounded
- Adding `date` → bounded partition size
- Typical user: 5-20 notifs/day → small partitions
- Power users: 100-200/day → still manageable

## Access patterns:

- ✓ Get user's notifications for today (single partition)
- ✓ Get user's recent notifications (few partitions)
- ☐ Get all notifications by template (full scan)  
→ Use analytics DB (ClickHouse) for this

## CASSANDRA: DELIVERY LOGS

Partition key: `notification_id`  
Clustering key: `attempt_num`

Typically 1-3 rows per notification (1 + retries)  
TTL: 90 days (auto-cleanup)

## MYSQL: USER PREFERENCES (Aurora)

500M users → shard by user\_id hash  
16 shards → ~31M rows per shard

Shard routing:  $\text{shard\_id} = \text{hash}(\text{user\_id}) \% 16$

Each shard: Aurora cluster (1 writer + 2 readers)  
Read replicas handle preference lookups  
Writer handles preference updates

## KAFKA: PARTITION STRATEGY

Partition key: user\_id

## Benefits:

- Per-user ordering guaranteed within a partition
- Even distribution (user IDs are UUIDs)

## Partition count per topic:

- P0: 4 (low volume, fast processing)
- P1: 8
- P2: 32 (highest volume)
- P3: 16

### 6.10.2 Hot Partition Mitigation

#### HOT PARTITION SCENARIOS

PROBLEM: Bulk notification to 10M users lands on all Kafka partitions at once, but some partitions may be temporarily slower.

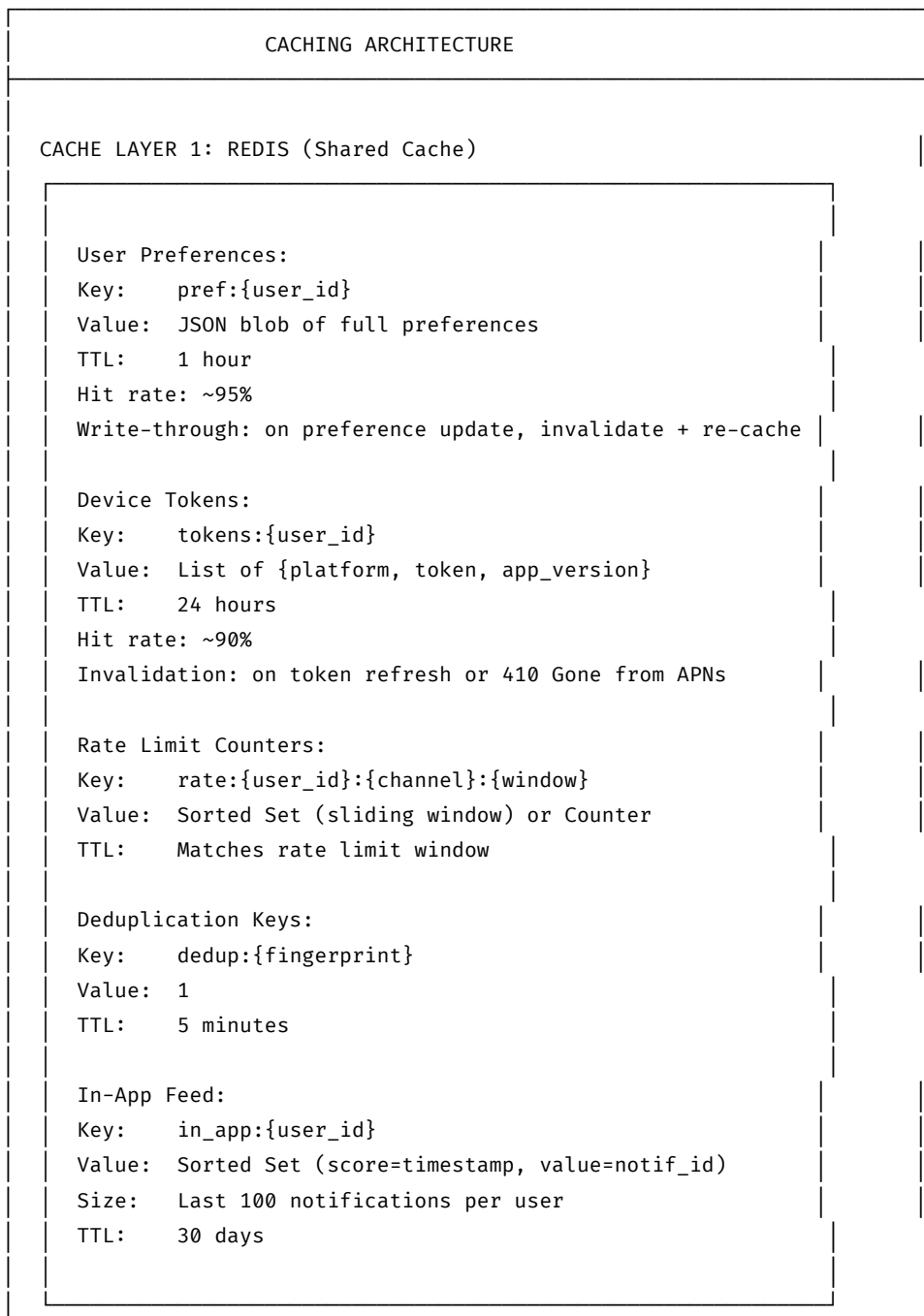
##### SOLUTION 1: Bulk Request Throttling

- Bulk API accepts segment\_id instead of user list
- Background job resolves segment to user list
- Produces to Kafka at controlled rate (10K/sec)
- Prevents overwhelming any single partition

##### SOLUTION 2: Salted Partition Keys

- For broadcast/bulk: key = user\_id + random\_salt
- Spreads load across all partitions
- Trade-off: loses per-user ordering for bulk sends
- Acceptable because bulk sends don't need ordering

## 6.11 10. Caching Strategy



## CACHE LAYER 2: LOCAL (In-Process Cache)

Templates:  
Storage: Caffeine / Guava Cache (JVM heap)  
TTL: 5 minutes  
Max size: 10,000 templates  
Hit rate: ~99% (templates change rarely)  
Invalidation: TTL-based + listen to template update events

Provider Health Scores:  
Storage: Local map, updated by circuit breaker  
TTL: 10 seconds  
Used for provider selection (SES vs SendGrid, etc.)

## CACHE SIZING:

Redis cluster for notification system:

|                |                                  |          |
|----------------|----------------------------------|----------|
| Preferences:   | $500M * 1KB * 20\% \text{ hot}$  | = 100 GB |
| Device tokens: | $500M * 200B * 15\% \text{ hot}$ | = 15 GB  |
| Rate counters: | $100M \text{ active} * 200B$     | = 20 GB  |
| Dedup keys:    | $5M \text{ active} * 100B$       | = 0.5 GB |
| In-app feeds:  | $100M \text{ active} * 10KB$     | = 1 TB   |

---

Total Redis: ~1.14 TB

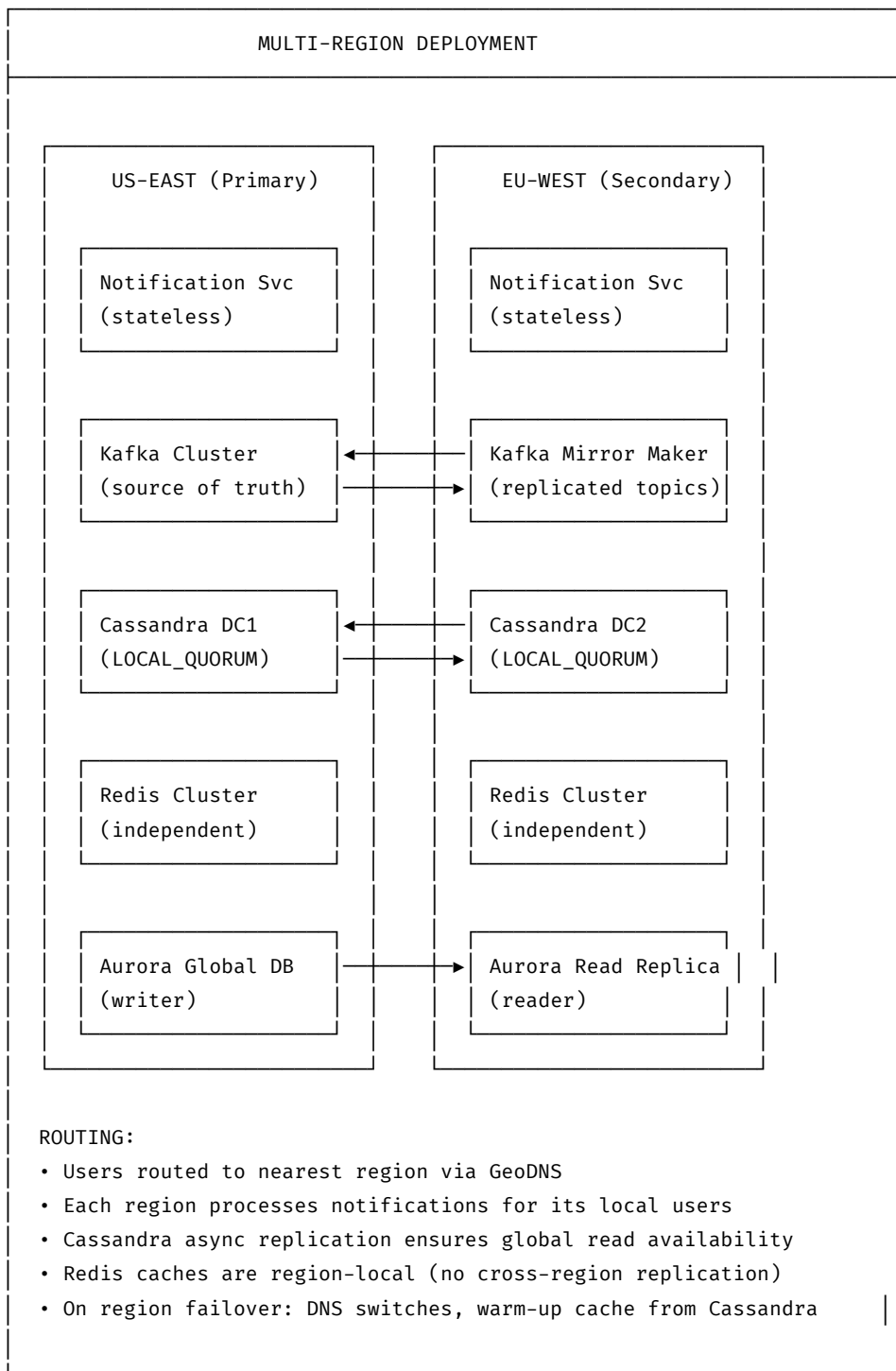
Redis Cluster: 24 nodes \* 64GB each = 1.5 TB capacity  
(with replication factor 2)

## 6.12 11. Replication and Consistency

| CONSISTENCY MODEL                   |                          |                                                                                                                                                 |
|-------------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| COMPONENT                           | CONSISTENCY              | RATIONALE                                                                                                                                       |
| Notification records<br>(Cassandra) | Eventual<br>(QUORUM)     | Write-heavy; seconds-level lag OK. CL=QUORUM for writes ensures durability.                                                                     |
| Delivery status<br>(Cassandra)      | Eventual<br>(ONE)        | Append-only logs, no conflict CL=ONE for writes (speed), CL=ONE for reads.                                                                      |
| User preferences<br>(MySQL Aurora)  | Eventual<br>(seconds)    | Strong consistency not needed. Read from replica; brief lag after update is acceptable. Worst case: one notification sent with old preferences. |
| Templates<br>(MySQL Aurora)         | Eventual<br>(minutes)    | Cached locally with 5-min TTL. Template update takes up to 5 min to propagate.                                                                  |
| Rate limit counters<br>(Redis)      | Best-effort              | Redis cluster; slightly overcounting is acceptable (user gets rate limited slightly early, better than not at all).                             |
| In-app feed<br>(Redis + Cassandra)  | Eventual<br>(sub-second) | New notifications appear within 1s. Read-your-writes via Redis primary.                                                                         |
| Dedup keys<br>(Redis)               | Best-effort              | False negative (duplicate sent) is rare; false positive (notif dropped) never happens because SETNX is atomic.                                  |



### 6.12.1 Multi-Region Replication



## 6.13 12. Fault Tolerance and Failure Handling

### 6.13.1 Failure Scenarios and Mitigations

| FAILURE HANDLING MATRIX                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>SCENARIO 1: THIRD-PARTY PROVIDER DOWN (APNs/FCM/SES/Twilio)</p> <div><p>Detection:</p><p>Circuit breaker monitors error rate (threshold: 50%)<br/>5xx errors, timeouts, connection refused</p><p>Response:</p><ol style="list-style-type: none"><li>1. Circuit breaker trips → OPEN state</li><li>2. Switch to fallback provider:<br/>Push: APNs down → queue for retry (no fallback)<br/>Email: SES down → switch to SendGrid<br/>SMS: Twilio down → switch to AWS SNS</li><li>3. Retry with exponential backoff:<br/>Attempt 1: immediate<br/>Attempt 2: 5 seconds<br/>Attempt 3: 30 seconds<br/>Attempt 4: 5 minutes<br/>Attempt 5: 30 minutes</li><li>4. After max retries → DLQ for manual review</li><li>5. Half-open probe every 30 seconds to check recovery</li></ol></div> |
| <p>SCENARIO 2: KAFKA BROKER FAILURE</p> <div><p>Detection: Producer timeout, consumer disconnection</p><p>Response:</p><ul style="list-style-type: none"><li>• Kafka replication (RF=3): 1 broker down = no impact</li><li>• Producer: acks=all, retries=3, retry.backoff.ms=100</li><li>• Consumer: auto rebalance assigns orphaned partitions</li><li>• If 2+ brokers down: temporary write to local disk queue, drain when Kafka recovers</li></ul></div>                                                                                                                                                                                                                                                                                                                            |

## SCENARIO 3: REDIS CLUSTER FAILURE

Impact: Preferences, rate limits, dedup unavailable

Response:

- Rate limiting: FAIL OPEN (allow all, risk over-send)
- Preferences: Fall back to MySQL directly
- Dedup: Fall back to Cassandra lookup (slower)
- In-app feed: Serve from Cassandra (higher latency)
- Redis Cluster auto-failover: replica promotion <30s

## SCENARIO 4: NOTIFICATION SERVICE CRASH

Detection: Health check failure, load balancer removes

Response:

- Stateless service → other instances handle traffic
- API Gateway retries to healthy instances
- Auto-scaling replaces crashed instance
- In-flight requests: client retries with idempotency key prevents duplicates

## SCENARIO 5: CASSANDRA NODE FAILURE

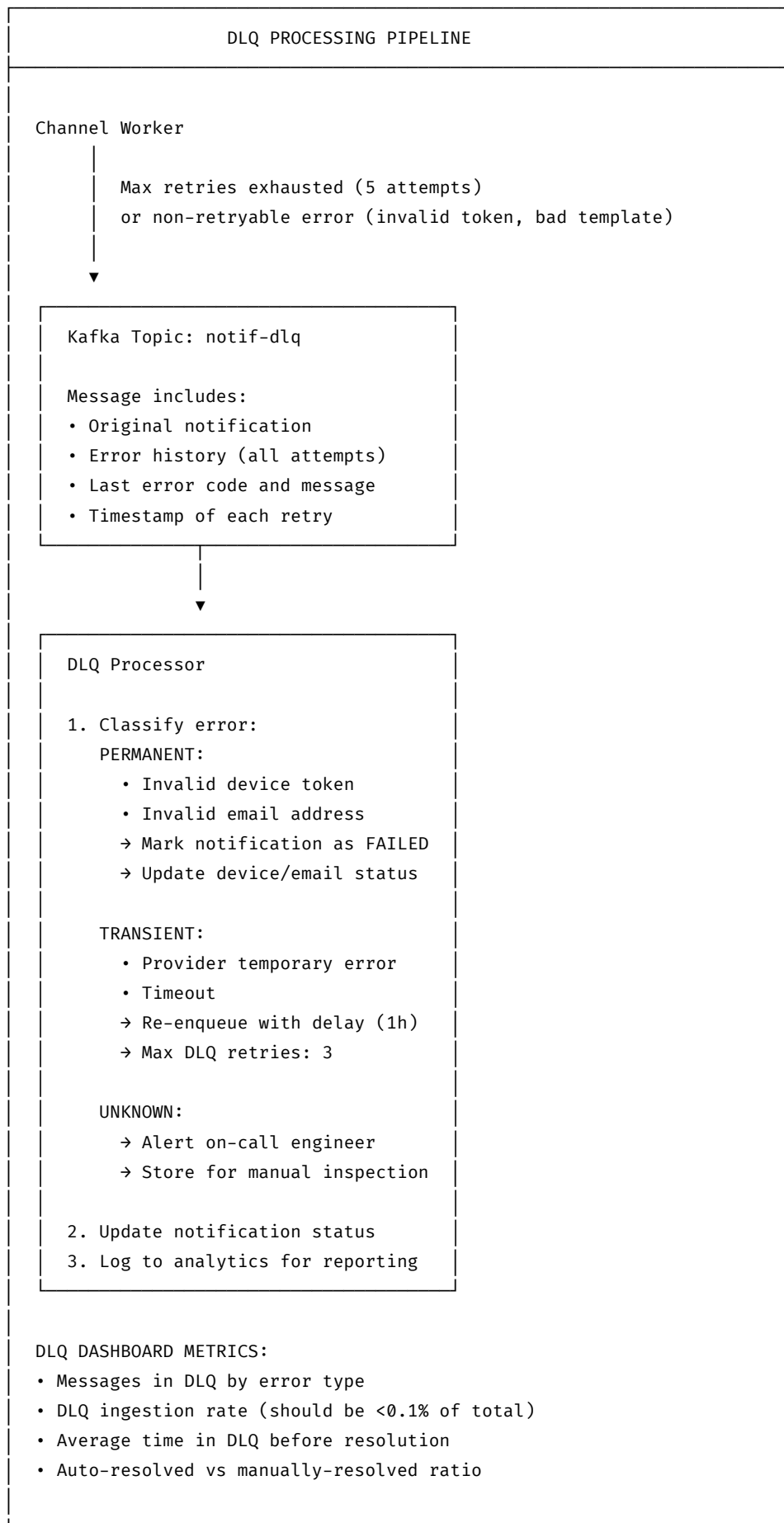
RF=3 with CL=QUORUM:

- 1 node down → reads/writes still succeed (2 of 3)
- 2 nodes down → writes fail, reads may succeed (1 of 3)

Response:

- Hinted handoff catches up when node recovers
- Anti-entropy repair runs daily
- If prolonged failure: replace node, stream data

### 6.13.2 Dead Letter Queue (DLQ) Processing



## 6.14 13. Scalability

### 6.14.1 Scaling Strategy

| SCALABILITY ARCHITECTURE                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                       |                  |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------|-----------|-------------------|---------|-------------|-----------------|----------|----------------------|-----------------|----------|--------------|---------------------------------------|------------------|---------------|----------------------------------|-------------|-------------|------------|-------------|----------------|----------------------|------------|---------------|----------------------|----------|-----------|-------------------|-----------|-------|-------------------------|----------|
| HORIZONTAL SCALING POINTS:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                       |                  |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| <table><tr><th>Component</th><th>Scaling Mechanism</th><th>Trigger</th></tr><tr><td>API Gateway</td><td>Auto-scale pods</td><td>CPU &gt;60%</td></tr><tr><td>Notification Service</td><td>Auto-scale pods</td><td>CPU &gt;60%</td></tr><tr><td>Push Workers</td><td>Auto-scale + Kafka partition increase</td><td>Consumer lag &gt;1K</td></tr><tr><td>Email Workers</td><td>Auto-scale (+ SES rate increase)</td><td>Queue depth</td></tr><tr><td>SMS Workers</td><td>Auto-scale</td><td>Queue depth</td></tr><tr><td>In-App Workers</td><td>Auto-scale + WS pods</td><td>Conn count</td></tr><tr><td>Kafka Cluster</td><td>Add brokers + rebal.</td><td>Disk/CPU</td></tr><tr><td>Cassandra</td><td>Add nodes to ring</td><td>Disk &gt;70%</td></tr><tr><td>Redis</td><td>Add shards (resharding)</td><td>Mem &gt;80%</td></tr></table> |                                       |                  | Component | Scaling Mechanism | Trigger | API Gateway | Auto-scale pods | CPU >60% | Notification Service | Auto-scale pods | CPU >60% | Push Workers | Auto-scale + Kafka partition increase | Consumer lag >1K | Email Workers | Auto-scale (+ SES rate increase) | Queue depth | SMS Workers | Auto-scale | Queue depth | In-App Workers | Auto-scale + WS pods | Conn count | Kafka Cluster | Add brokers + rebal. | Disk/CPU | Cassandra | Add nodes to ring | Disk >70% | Redis | Add shards (resharding) | Mem >80% |
| Component                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Scaling Mechanism                     | Trigger          |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| API Gateway                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Auto-scale pods                       | CPU >60%         |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Notification Service                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | Auto-scale pods                       | CPU >60%         |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Push Workers                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Auto-scale + Kafka partition increase | Consumer lag >1K |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Email Workers                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Auto-scale (+ SES rate increase)      | Queue depth      |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| SMS Workers                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Auto-scale                            | Queue depth      |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| In-App Workers                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Auto-scale + WS pods                  | Conn count       |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Kafka Cluster                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Add brokers + rebal.                  | Disk/CPU         |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Cassandra                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Add nodes to ring                     | Disk >70%        |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| Redis                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Add shards (resharding)               | Mem >80%         |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| KAFKA SCALING:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                       |                  |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |
| <p>Current: 32 partitions for P2-Normal push topic<br/>At 50K/sec: each partition handles ~1.5K msg/sec</p> <p>To scale to 200K/sec (4x):</p> <ol style="list-style-type: none"><li>1. Increase partitions: 32 → 128</li><li>2. Add more consumer instances: 32 → 128</li><li>3. Add Kafka brokers: 6 → 12</li></ol> <p>NOTE: Increasing partitions is one-way in Kafka.<br/>Plan partition count for 2x expected peak.</p>                                                                                                                                                                                                                                                                                                                                                                                                               |                                       |                  |           |                   |         |             |                 |          |                      |                 |          |              |                                       |                  |               |                                  |             |             |            |             |                |                      |            |               |                      |          |           |                   |           |       |                         |          |

**WORKER SCALING:**

Each channel worker type scales independently:

Normal load (12K/sec total):

Push workers: 20 instances  
Email workers: 15 instances  
SMS workers: 5 instances  
In-app workers: 10 instances

Peak load (50K/sec total):

Push workers: 80 instances  
Email workers: 60 instances  
SMS workers: 20 instances  
In-app workers: 30 instances

Auto-scaling trigger: Kafka consumer lag > 1000 msgs

Scale-down delay: 10 minutes after lag < 100

**HANDLING NOTIFICATION STORMS (Flash Sale, Breaking News):**

1. Pre-warm workers before scheduled events
2. Bulk sends are split into micro-batches  
(10K recipients per batch, 100ms delay between)
3. Priority queue ensures P0/P1 are not starved
4. Auto-scaler kicks in within 2 minutes
5. Provider rate limits enforced to avoid throttling
6. Kafka buffers absorb temporary spikes

## 6.15 14. Monitoring and Observability

### 6.15.1 Key Metrics Dashboard

| MONITORING DASHBOARD                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>DELIVERY METRICS:</p> <ul style="list-style-type: none"><li>• <code>delivery_success_rate</code> (by channel, by priority)<br/>Target: &gt;99.5% for push, &gt;99% for email, &gt;98% for SMS</li><li>• <code>delivery_latency_p50</code> / <code>p95</code> / <code>p99</code> (by channel)<br/>Target: <code>p99</code> &lt; 2s for push, <code>p99</code> &lt; 30s for email</li><li>• <code>notifications_sent_total</code> (counter, by channel)</li><li>• <code>notifications_failed_total</code> (counter, by error type)</li><li>• <code>notifications_rate_limited_total</code> (counter)</li><li>• <code>notifications_deduplicated_total</code> (counter)</li></ul> |
| <p>QUEUE METRICS:</p> <ul style="list-style-type: none"><li>• <code>kafka_consumer_lag</code> (by topic, by consumer group)<br/>ALERT: if lag &gt; 10K for P0 topic<br/>ALERT: if lag &gt; 100K for P2 topic</li><li>• <code>kafka_produce_rate</code> (msg/sec, by topic)</li><li>• <code>kafka_consume_rate</code> (msg/sec, by consumer group)</li><li>• <code>dlq_depth</code> (messages in dead letter queue)<br/>ALERT: if &gt; 1000 messages</li></ul>                                                                                                                                                                                                                     |

## PROVIDER METRICS:

- provider\_error\_rate (by provider: APNs/FCM/SES/etc)  
ALERT: if > 5% error rate
- provider\_latency\_p95 (by provider)
- provider\_throttle\_count (429 responses)
- circuit\_breaker\_state (OPEN/CLOSED/HALF\_OPEN)
- invalid\_token\_rate (indicates stale device tokens)

## USER ENGAGEMENT METRICS:

- notification\_open\_rate (by channel, by category)
- notification\_click\_rate (for actionable notifications)
- opt\_out\_rate (by channel, by category)  
ALERT: if opt-out rate spikes >2x normal
- unsubscribe\_rate (email)

## INFRASTRUCTURE METRICS:

- Redis memory\_usage / hit\_rate / eviction\_count
- Cassandra read\_latency / write\_latency / compactions
- Worker CPU / memory / instance\_count
- WebSocket active\_connections / messages\_per\_sec

## ALERTING RULES:

## P1 (Page on-call):

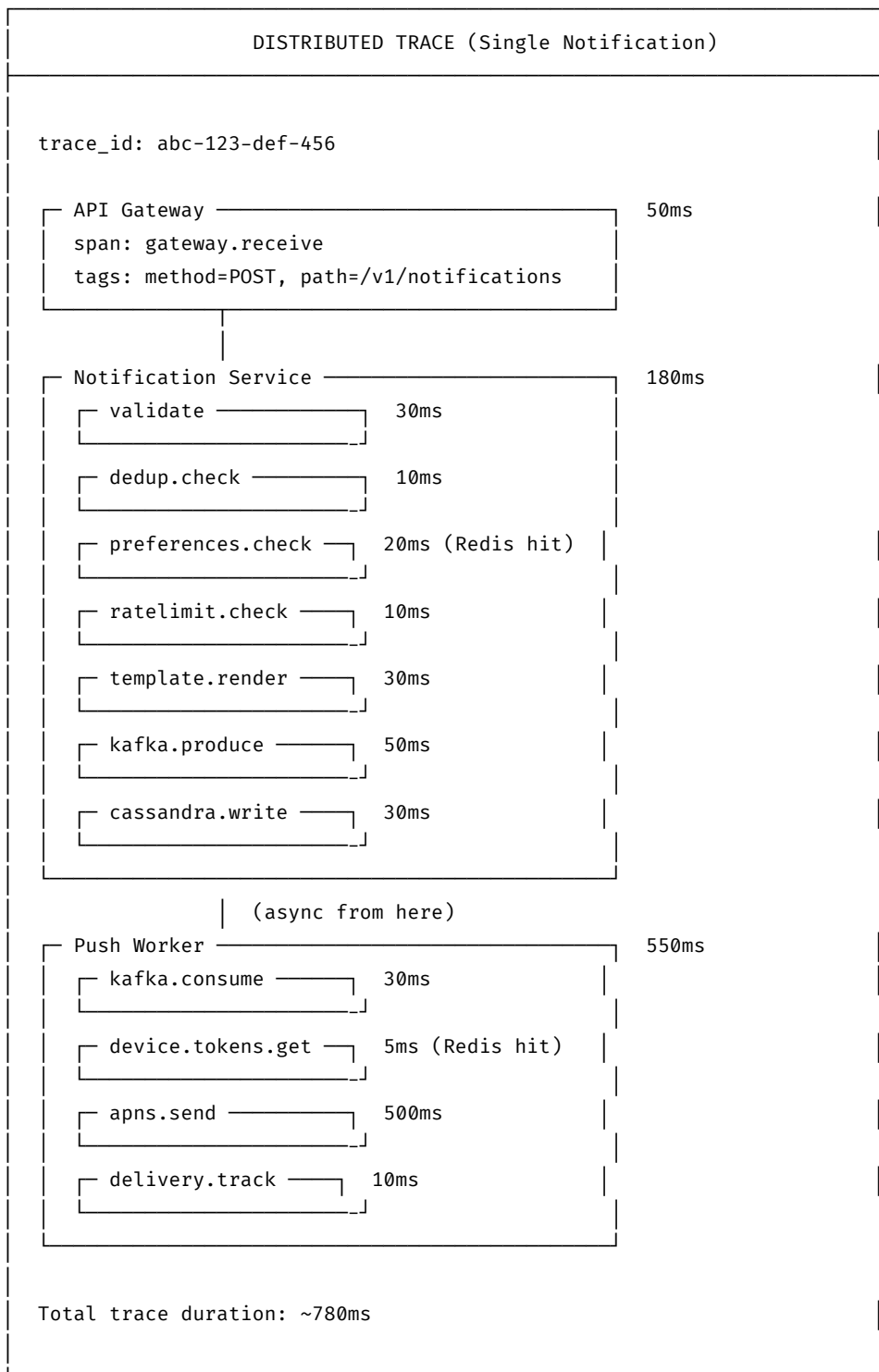
- Delivery success rate < 95% for 5 min
- P0 consumer lag > 10K for 2 min
- All providers for a channel in OPEN state
- DLQ depth > 10K

## P2 (Slack alert):

- Delivery success rate < 99% for 10 min
- P2 consumer lag > 50K for 5 min
- Single provider error rate > 10% for 5 min
- Opt-out rate 2x above weekly average



### 6.15.2 Distributed Tracing



## 6.16 15. Trade-offs and Design Decisions

### 6.16.1 Decision Matrix

| KEY DESIGN DECISIONS                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DECISION 1: Push vs Pull for In-App Notifications                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <div><div>Option A: WebSocket (Push)<ul style="list-style-type: none"><li>✓ Real-time delivery (&lt;100ms)</li><li>✓ Reduces server load (no polling)</li><li>⊠ Connection management complexity</li><li>⊠ Sticky sessions or Redis pub/sub needed</li></ul></div><div>Option B: Long Polling (Pull)<ul style="list-style-type: none"><li>✓ Simpler infrastructure</li><li>✓ Works through all proxies/firewalls</li><li>⊠ Higher latency (seconds)</li><li>⊠ More server load from frequent polls</li></ul></div><div>CHOSEN: Hybrid<ul style="list-style-type: none"><li>WebSocket as primary (real-time for connected users)</li><li>REST polling as fallback (for environments where WebSocket is blocked)</li></ul></div></div> |

## DECISION 2: At-Least-Once vs Exactly-Once Delivery

## Option A: At-Least-Once

- ✓ Simpler implementation
- ✓ Higher delivery guarantee
- ⊠ Possible duplicates (mitigated by dedup layers)

## Option B: Exactly-Once

- ✓ No duplicates
- ⊠ Significantly more complex (2PC or idempotent consumers with transactional outbox)
- ⊠ Higher latency
- ⊠ Not achievable end-to-end (providers don't guarantee exactly-once)

CHOSEN: At-Least-Once with multi-layer deduplication

Rationale: Exactly-once is impossible with external providers. 3-layer dedup makes duplicates extremely rare (<0.001%).

## DECISION 3: Single Queue vs Per-Channel Queues

## Option A: Single Queue (all channels in one topic)

- ✓ Simpler topology
- ⊠ Slow channel (email) blocks fast channel (push)
- ⊠ Cannot scale channels independently

## Option B: Per-Channel + Per-Priority Topics

- ✓ Independent scaling per channel
- ✓ Slow channel doesn't affect others
- ✓ Different consumer configurations per priority
- ⊠ More Kafka topics to manage (16 total)
- ⊠ More operational complexity

CHOSEN: Per-Channel + Per-Priority

Rationale: Isolation is critical for availability.  
A Twilio outage should not affect push delivery.

## DECISION 4: Template Rendering Location

Option A: Render in Notification Service (before queue)

- ✓ Rendered content stored in Kafka (debuggable)
- ✓ Template version locked at send time
- ⊠ Larger Kafka messages (rendered content > template)

Option B: Render in Channel Worker (after queue)

- ✓ Smaller Kafka messages
- ⊠ Template may change between enqueue and delivery
- ⊠ Channel workers need template engine + cache

CHOSEN: Render before queue (Option A)

Rationale: Template version consistency is important.  
Message size increase is modest (~500B vs ~200B).

## DECISION 5: Vendor Lock-in vs Abstraction

CHOSEN: Provider abstraction layer

Each channel has a provider interface:

```
interface PushProvider {
 send(token, payload): DeliveryResult
 isHealthy(): boolean
}
```

Implementations: APNsProvider, FCMProvider

Benefits:

- Switch providers without code changes
- A/B test providers for deliverability
- Automatic failover between providers
- Gradual migration to new provider

## DECISION 6: Cassandra vs DynamoDB for Notification Storage

## Cassandra:

- ✓ No vendor lock-in
- ✓ Predictable performance at scale
- ✓ Built-in TTL for automatic data expiry
- ✓ Multi-DC replication built-in
- ⊠ Operational overhead (manage cluster)

## DynamoDB:

- ✓ Fully managed
- ✓ Auto-scaling
- ⊠ Cost at 1B writes/day can be significant
- ⊠ TTL deletes are eventual (up to 48h lag)
- ⊠ AWS-only

CHOSEN: Cassandra (for cost control at scale and multi-cloud flexibility)

## 6.17 16. Interview Deep-Dive Questions

### 6.17.1 Q1: How do you handle notification storms (e.g., everyone gets notified at once)?

**A:** Notification storms (flash sales, breaking news, system-wide announcements) can generate millions of notifications simultaneously. We handle this through several mechanisms:

1. **Bulk API with micro-batching:** The bulk send API does not produce all messages to Kafka at once. Instead, it creates a background job that produces in micro-batches (10K messages per batch with 100ms delay between batches), spreading the load over minutes rather than seconds.
2. **Pre-warming:** For planned events (scheduled sale), we pre-scale workers 30 minutes before. The scheduler service exposes an API for capacity planning.
3. **Priority isolation:** Storm notifications are typically P2 or P3. The separate Kafka topics ensure P0 urgent notifications (password reset, security alerts) are unaffected.
4. **Provider rate limiting:** Workers enforce provider-specific rate limits (SES: 200/sec, FCM: 600K/min) using token bucket algorithms, preventing provider throttling.
5. **Kafka as buffer:** With 32 partitions and replication, Kafka can buffer millions of messages while workers process at sustainable rates.

### 6.17.2 Q2: How do you prevent duplicate notifications?

**A:** Three-layer deduplication:

- **Layer 1 (API):** Idempotency key in Redis with 5-min TTL. Same request ID returns cached response.
- **Layer 2 (Content):** SHA256 fingerprint of (user\_id + template + channel + data). Prevents duplicate sends from different API calls with same content.
- **Layer 3 (Consumer):** Kafka consumer checks `processed:{notification_id}` in Redis before processing. Handles at-least-once delivery duplication from consumer rebalance.

Combined, these reduce duplicate delivery to <0.001% of total notifications.

---

### 6.17.3 Q3: How do you handle timezone-aware scheduling?

**A:** When a notification is scheduled for “9:00 AM user’s local time”:

1. Fetch user’s timezone from preferences (e.g., “America/New\_York”).
2. Convert “9:00 AM America/New\_York” to UTC epoch timestamp.
3. Store in Redis ZSET (sorted set) with score = UTC epoch.
4. Scheduler polls ZRANGEBYSCORE every 1 second for due notifications.
5. Handle DST transitions: use IANA timezone database, which accounts for DST shifts (e.g., 2:00 AM might not exist during spring-forward).

For bulk sends across timezones: the system fans out into multiple scheduled entries, one per time-zone cohort. For example, “send at 9 AM local” for global users creates ~24 batches, one per major timezone offset.

---

### 6.17.4 Q4: What happens when APNs/FCM is down?

**A:** Unlike email and SMS, push notifications have no alternative provider – APNs is the only way to reach iOS devices, FCM for Android. Our strategy:

1. **Circuit breaker trips** after 50% error rate over 30 seconds.
  2. **Messages stay in Kafka** (retention: 7 days). Workers pause consumption.
  3. **Circuit breaker half-open probe** every 30 seconds sends a test push.
  4. **When provider recovers**, circuit closes, workers resume, backlog drains.
  5. **TTL enforcement:** Notifications older than their TTL are discarded (a “flash sale” notification 6 hours late is worse than no notification).
  6. **Channel fallback:** If push fails and notification is high priority, fallback to in-app + email based on user preferences.
- 

### 6.17.5 Q5: How do you A/B test notification content?

**A:** Template versioning supports A/B testing:

1. Template has multiple variants: `order_shipped_v2_A`, `order_shipped_v2_B`.
  2. Notification Service hashes `user_id` to deterministically assign variant (consistent hashing ensures same user always sees same variant).
  3. Variant ID is stored in the notification record.
  4. Analytics pipeline correlates variant with engagement metrics (open rate, click rate, conversion).
  5. After statistical significance reached, winning variant becomes the default.
-

### 6.17.6 Q6: How do you ensure ordering of related notifications?

**A:** Kafka partition key is `user_id`, which guarantees all notifications for a user land on the same partition and are consumed in order by a single consumer. This ensures “Order Placed” arrives before “Order Shipped.”

For cross-channel ordering (push before email for same event), we use sequence numbers in the notification metadata and a small delay (2 seconds) between channel dispatches.

---

### 6.17.7 Q7: How do you handle user preference updates propagating to in-flight notifications?

**A:** We use a “snapshot at send time” approach:

1. When the Notification Service processes a request, it reads current preferences.
  2. The preference snapshot (channel enabled, quiet hours, etc.) is included in the Kafka message.
  3. Channel workers use the snapshotted preferences, not a fresh lookup.
  4. This means a preference change made after the notification is queued but before delivery will not affect that notification.
  5. Trade-off: A user who unsubscribes may receive 1-2 more notifications that were already in the pipeline. This is acceptable and common in the industry.
- 

### 6.17.8 Q8: How do you handle multi-device push delivery?

**A:** A single user may have multiple devices (phone, tablet, watch):

1. Device token table stores all active tokens per user, with platform and last-used timestamp.
  2. When sending a push notification, the worker fetches ALL active device tokens.
  3. Sends to each device independently (separate APNs/FCM calls).
  4. Each delivery is tracked independently (device A delivered, device B failed).
  5. Collapse key ensures only the latest notification shows if multiple arrive while device was offline.
  6. Stale token cleanup: if a device token returns 410 Gone, it is deactivated. Additionally, devices not used for 90 days are proactively cleaned up.
- 

### 6.17.9 Q9: How would you add a new notification channel (e.g., WhatsApp)?

**A:** The provider abstraction makes this straightforward:

1. Create new Kafka topics: `notif-whatsapp-p{0-3}`.
2. Implement `WhatsAppProvider` conforming to the `Channel` interface.
3. Create WhatsApp worker fleet that consumes from the new topics.
4. Add `whatsapp` to the channel enum in templates and preferences.
5. Add WhatsApp-specific fields to the template table.
6. Deploy worker fleet, update API schema, publish new template format.

No changes needed to the Notification Service core pipeline – it is channel-agnostic. Estimated effort: 1-2 weeks for a senior engineer.

---

### 6.17.10 Q10: How do you handle cost optimization for SMS?

**A:** SMS is the most expensive channel (~\$0.01 per message). Cost controls:

1. **SMS opt-in required:** Users must explicitly enable SMS (default: off).
  2. **Rate limits:** Max 3 SMS per user per day (configurable).
  3. **Priority gating:** Only P0 and P1 notifications sent via SMS.
  4. **Smart routing:** Use local carrier gateways for domestic SMS (cheaper than international routes via Twilio).
  5. **Message length optimization:** Keep SMS under 160 characters to avoid multi-part messages (each part billed separately).
  6. **Fallback:** If SMS budget exhausted, fall back to push + email.
  7. **Cost monitoring:** Real-time SMS spend tracking with daily budget alerts.
- 

### 6.17.11 Q11: What is the retry strategy, and how do you prevent retry storms?

**A:** Retry strategy uses exponential backoff with jitter:

```
delay = min(base_delay * 2^attempt + random_jitter, max_delay)
```

```
Attempt 1: 0s (immediate)
Attempt 2: 5s + jitter (0-2s)
Attempt 3: 30s + jitter (0-10s)
Attempt 4: 5 min + jitter (0-30s)
Attempt 5: 30 min + jitter (0-60s)
Max attempts: 5, then DLQ
```

**Retry storm prevention:** - Jitter spreads retry attempts across time, preventing synchronized retries.  
- Per-provider circuit breaker stops retries when provider is confirmed down. - Retry count is tracked per notification; DLQ after max attempts. - Global retry rate limiter caps total retry throughput at 10% of normal throughput.

---

### 6.17.12 Q12: How do you handle notification preferences at scale (500M users)?

**A:** Preferences are read-heavy (read on every notification, written rarely):

1. **MySQL Aurora** as source of truth, sharded by `user_id` (16 shards).
  2. **Redis cache** with 1-hour TTL, ~95% hit rate.
  3. **Write-through caching:** On preference update, write to MySQL, then invalidate Redis cache. Next read populates cache from MySQL.
  4. **Bulk preference queries:** For segment-based sends, pre-fetch preferences in batch (`WHERE user_id IN (...)`) rather than one-by-one.
  5. **Default preferences:** New users get sensible defaults (push + email enabled, SMS disabled). Only store deviations from default to reduce storage.
- 

### 6.17.13 Q13: How do you ensure compliance with regulations (CAN-SPAM, GDPR)?

**A:** Regulatory compliance is enforced at multiple levels:

1. **CAN-SPAM (email):**
    - Every marketing email includes unsubscribe link.
-



- Unsubscribe requests processed within 10 minutes (regulatory requirement: 10 days).
  - Physical mailing address in email footer.
  - Accurate From/Subject lines.
2. **GDPR (EU users):**
    - Explicit consent recorded before sending marketing notifications.
    - Right to erasure: API to delete all notification history for a user.
    - Data retention: automatic purge after 90 days (hot) / 1 year (cold).
    - Data portability: export user's notification history as JSON.
  3. **TCPA (SMS in US):**
    - Prior express written consent for marketing SMS.
    - Easy opt-out (reply STOP).
    - Time-of-day restrictions (no SMS before 8 AM or after 9 PM local).
- 

#### 6.17.14 Q14: How do you design the collapse/replace notification feature?

**A:** Collapse key groups related notifications so only the latest is shown:

1. Sender includes `collapse_key` (e.g., "order\_update\_98765").
2. **For push:** APNs `apns-collapse-id` and FCM `collapse_key` natively replace prior notifications with the same key on the device.
3. **For in-app:** Before inserting new notification into Redis ZSET, check if a notification with the same `collapse_key` exists. If yes, update it in-place rather than adding a new entry.
4. **For email:** Collapse key is less useful (emails can't be replaced). Instead, use a dedup window: if another email with the same `collapse_key` was sent in the last 5 minutes, skip this one.

Use case: Ride-sharing app sends "Driver is 5 min away," then "Driver is 2 min away." With collapse, only the latest ETA shows on the user's lock screen.

---

#### 6.17.15 Q15: How would you migrate from a monolithic notification system to this architecture?

**A:** Phased migration strategy:

1. **Phase 1 - Dual Write (2 weeks):**
  - Deploy new system alongside monolith.
  - Monolith sends to both old path and new system (shadow mode).
  - Compare delivery results, fix discrepancies.
  - New system does NOT deliver to users yet.
2. **Phase 2 - Canary (2 weeks):**
  - Route 1% of traffic to new system (by `user_id` hash).
  - Monitor delivery rates, latency, error rates.
  - Gradually increase: 5% → 10% → 25% → 50%.
3. **Phase 3 - Full Migration (1 week):**
  - Route 100% to new system.
  - Keep monolith on standby for 1 week.
  - Monitor for regressions.
4. **Phase 4 - Decommission (1 week):**
  - Shut down monolith notification path.
  - Archive monolith notification data.

Key risk mitigation: Feature flags control routing percentage. Instant rollback by flipping flag to 0%.

---

## 6.18 Summary: System at a Glance

| NOTIFICATION SYSTEM SUMMARY                                                                                                                                                                                                                                                                                                                                           |                                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| SCALE:                                                                                                                                                                                                                                                                                                                                                                | 1B notifications/day, 50K/sec peak                   |
| CHANNELS:                                                                                                                                                                                                                                                                                                                                                             | Push (APNs/FCM), Email (SES), SMS (Twilio), In-App   |
| LATENCY:                                                                                                                                                                                                                                                                                                                                                              | <1s urgent, <30s normal (internal <250ms)            |
| AVAILABILITY:                                                                                                                                                                                                                                                                                                                                                         | 99.99% (multi-region, stateless services)            |
| DELIVERY:                                                                                                                                                                                                                                                                                                                                                             | At-least-once with 3-layer deduplication             |
| QUEUE:                                                                                                                                                                                                                                                                                                                                                                | Kafka (per-channel, per-priority topics)             |
| STORAGE:                                                                                                                                                                                                                                                                                                                                                              | Cassandra (notifs), MySQL (prefs), Redis (cache)     |
| SCALING:                                                                                                                                                                                                                                                                                                                                                              | Independent channel workers, Kafka partition scaling |
| FAULT TOL.:                                                                                                                                                                                                                                                                                                                                                           | Circuit breakers, DLQ, provider failover             |
| KEY PATTERNS:                                                                                                                                                                                                                                                                                                                                                         |                                                      |
| <ul style="list-style-type: none"><li>• Fan-out on write (multi-channel delivery)</li><li>• Priority queues (topic-per-priority)</li><li>• Provider abstraction (swap/failover providers)</li><li>• Idempotent processing (3-layer dedup)</li><li>• Circuit breaker (provider health management)</li><li>• Event sourcing (notification lifecycle tracking)</li></ul> |                                                      |

## Chapter 7

# Design a Web Search Engine (Google Search)

**Difficulty:** Hard | **Category:** Information Retrieval, Distributed Systems

**Companies:** Google, Microsoft (Bing), Amazon (A9), Apple (Spotlight/Siri), Yandex, Baidu, DuckDuckGo

---

### 7.1 1. Problem Statement and Scope

Design a web-scale search engine that continuously crawls billions of web pages, builds and maintains a searchable index, and returns highly relevant ranked results for arbitrary user queries with sub-second latency.

#### 7.1.1 In Scope

- **Web Crawling:** Discover, fetch, and store web pages at scale
- **Indexing:** Build and maintain an inverted index over crawled content
- **Query Processing:** Parse, understand, and execute user queries
- **Ranking:** Return results ordered by relevance (BM25 + PageRank + ML)
- **Autocomplete / Query Suggestions:** Real-time prefix-based suggestions
- **Spell Correction:** “Did you mean...” functionality
- **Snippet Generation:** Extract and highlight relevant text fragments
- **Pagination:** Serve results across multiple pages
- **Knowledge Panels:** Structured information cards for entities
- **Image and Video Search:** Multimedia result types

#### 7.1.2 Out of Scope

- Ads auction system (separate design)
  - Google Maps / local search integration details
  - Full NLP / conversational AI (e.g., featured snippets via LLMs)
  - Browser or client-side implementation
  - Content moderation / safe search filtering (mentioned briefly)
-

## 7.2 2. Functional Requirements

- **F1 - Web Crawling:** Continuously discover and fetch web pages respecting robots.txt
  - **F2 - Content Parsing:** Extract text, links, metadata, and structured data from HTML
  - **F3 - Index Building:** Build and maintain an inverted index for full-text search
  - **F4 - Query Processing:** Parse user queries, handle multi-word, phrases, operators
  - **F5 - Ranked Results:** Return top-K results ordered by relevance
  - **F6 - Autocomplete:** Suggest query completions as the user types
  - **F7 - Spell Correction:** Detect and correct misspelled queries
  - **F8 - Snippet Generation:** Show relevant text excerpts with query terms highlighted
  - **F9 - Pagination:** Return paginated results (10 per page, up to ~30 pages)
  - **F10 - Image/Video Search:** Index and retrieve multimedia content
  - **F11 - Knowledge Panels:** Display structured entity information
  - **F12 - Freshness:** Prioritize re-crawling time-sensitive content (news)
- 

## 7.3 3. Non-Functional Requirements

- **NF1 - Query Latency:** p50 < 200ms, p99 < 500ms end-to-end
  - **NF2 - Availability:** 99.99% uptime (< 52 min downtime/year)
  - **NF3 - Index Size:** 100B+ web pages indexed
  - **NF4 - Crawl Freshness:** News: < 1 hour, Popular: < 1 day, Long-tail: < 30 days
  - **NF5 - Throughput:** 115K+ queries per second globally
  - **NF6 - Cost Efficiency:** < \$0.001 per query served
  - **NF7 - Scalability:** Horizontal scaling for crawl, index, and query tiers
  - **NF8 - Fault Tolerance:** No single point of failure; graceful degradation
  - **NF9 - Consistency:** Index eventually consistent (stale by minutes, not hours)
  - **NF10 - Crawl Politeness:** Respect robots.txt, rate-limit per domain
- 

## 7.4 4. Back-of-the-Envelope Estimation

### 7.4.1 4.1 Query Traffic

|                    |                                                  |
|--------------------|--------------------------------------------------|
| Daily queries:     | 10 billion                                       |
| Seconds per day:   | 86,400                                           |
| Average QPS:       | $10B / 86,400 = \sim 115,740$ QPS                |
| Peak QPS (3x avg): | $\sim 350,000$ QPS                               |
| Queries per year:  | $\sim 3.65$ trillion                             |
|                    |                                                  |
| Avg query length:  | $\sim 3.5$ words = $\sim 25$ bytes               |
| Query traffic:     | $115K * 25$ bytes = $\sim 2.9$ MB/s (negligible) |

### 7.4.2 4.2 Web Scale

Total web pages: ~100 billion (indexed)  
 Avg raw HTML size: ~100 KB (compressed)  
 Avg extracted text size: ~10 KB per page  
 Total raw content: 100B \* 100KB = 10 PB (compressed HTML)  
 Total extracted text: 100B \* 10KB = 1 PB

Pages discovered but not indexed: ~500 billion (deep web, duplicates)  
 Unique domains: ~350 million

### 7.4.3 4.3 Inverted Index Size

Unique terms: ~10 billion (after stemming, normalization)  
 Avg posting list length: ~1,000 documents per term (power-law distribution)  
 Avg posting entry: ~12 bytes (doc\_id: 5B, tf: 2B, positions: 5B)

Posting lists total: 10B terms \* 1K docs \* 12 bytes = ~120 TB  
 Term dictionary: 10B terms \* 20 bytes avg = ~200 GB  
 Total inverted index: ~150-200 TB (with compression ~50-80 TB)

### 7.4.4 4.4 Crawler Throughput

Pages to re-crawl per day: ~1 billion (priority-based)  
 Avg page fetch time: ~500ms (including DNS + download)  
 Pages per crawler worker: ~170K/day (with parallelism per worker)  
 Crawler workers needed: 1B / 170K = ~6,000 workers  
 Bandwidth: 1B \* 100KB / 86400 = ~1.2 TB/s download

### 7.4.5 4.5 Storage Summary

| Component       | Size        | Storage Type                  |
|-----------------|-------------|-------------------------------|
| Raw HTML store  | 10 PB       | Distributed FS (GFS/Colossus) |
| Extracted text  | 1 PB        | Column store (BigTable)       |
| Inverted index  | 80 TB       | Custom in-memory + SSD        |
| PageRank scores | 800 GB      | Pre-computed, in-memory       |
| URL frontier    | 50 GB       | Distributed priority queue    |
| Link graph      | 2 TB        | Graph store                   |
| Query logs      | 500 TB/year | Append-only log store         |

### 7.4.6 4.6 Serving Infrastructure

Index shards: ~10,000 shards (each ~8 TB of index)  
 Replicas per shard: 3 (for availability + load distribution)  
 Total index servers: ~30,000 machines  
 Query frontend servers: ~5,000 (behind load balancers)  
 Data centers: ~20 globally (serve from nearest)

## 7.5 5. API Design

### 7.5.1 5.1 Web Search

GET /api/v1/search

Query Parameters:

|           |            |                                              |
|-----------|------------|----------------------------------------------|
| q         | (required) | Search query string, max 2048 chars          |
| page      | (optional) | Page number, default 1, max 30               |
| count     | (optional) | Results per page, default 10, max 50         |
| lang      | (optional) | Language filter (ISO 639-1), e.g., "en"      |
| region    | (optional) | Region filter (ISO 3166-1), e.g., "US"       |
| safe      | (optional) | Safe search: "off", "moderate", "strict"     |
| type      | (optional) | Result type: "web", "image", "video", "news" |
| freshness | (optional) | Time filter: "day", "week", "month", "year"  |

Response (200 OK):

```
{
 "query": "distributed systems consensus",
 "corrected_query": null,
 "total_results": 48200000,
 "search_time_ms": 187,
 "results": [
 {
 "rank": 1,
 "url": "https://example.com/raft-consensus",
 "title": "Understanding Raft Consensus Algorithm",
 "snippet": "... a distributed consensus algorithm ...",
 "domain": "example.com",
 "last_crawled": "2026-04-08T14:22:00Z",
 "content_type": "web",
 "sitelinks": [...]
 },
 ...
],
 "knowledge_panel": { ... },
 "related_searches": ["paxos vs raft", "distributed consensus explained"],
 "pagination": {
 "current_page": 1,
 "total_pages": 30,
 "next": "/api/v1/search?q=distributed+systems+consensus&page=2"
 }
}
```

### 7.5.2 5.2 Autocomplete / Suggestions

GET /api/v1/suggest

Query Parameters:

|       |            |                                  |
|-------|------------|----------------------------------|
| q     | (required) | Partial query prefix             |
| lang  | (optional) | Language, default "en"           |
| count | (optional) | Number of suggestions, default 8 |

Response (200 OK):

```
{
 "prefix": "how to des",
 "suggestions": [
 { "text": "how to design a search engine", "score": 0.95 },
 { "text": "how to design a database", "score": 0.89 },
 { "text": "how to design distributed systems", "score": 0.84 },
 ...
]
}
```

Latency requirement: < 50ms (must feel instant while typing)

### 7.5.3 5.3 Spell Correction

GET /api/v1/spell

Query Parameters:

|   |            |                |
|---|------------|----------------|
| q | (required) | Query to check |
|---|------------|----------------|

Response (200 OK):

```
{
 "original": "distribted systms",
 "corrected": "distributed systems",
 "confidence": 0.97,
 "alternatives": [
 "distributed systems",
 "distributed streams"
]
}
```

---

## 7.6 6. Data Model and Database Selection

7.6.1 6.1 Core Data Structures

7.6.1.1 Inverted Index Entry

Term Dictionary Entry:

|                                       |
|---------------------------------------|
| term: "consensus"                     |
| document_frequency: 2,340,000         |
| posting_list_offset: 0x7FA3B200       |
| posting_list_length: 28,080,000 bytes |
| collection_frequency: 5,120,000       |

Posting List (for term "consensus"):

| doc_id  | term_freq | positions                        | field_flags                   |
|---------|-----------|----------------------------------|-------------------------------|
| 1003842 | 7         | [12, 45, 89, 134, 201, 267, 312] | title=1<br>body=1<br>anchor=0 |
| 1003901 | 3         | [5, 78, 156]                     | title=1<br>body=1<br>anchor=1 |
| 1004217 | 1         | [42]                             | title=0<br>body=1<br>anchor=0 |

Sorted by doc\_id for efficient intersection and delta encoding.

7.6.1.2 Document Metadata Store

Document Record:

|                 |                                 |
|-----------------|---------------------------------|
| doc_id:         | uint64                          |
| url:            | string (max 2048)               |
| canonical_url:  | string                          |
| title:          | string (max 512)                |
| content_hash:   | uint64 (SimHash for near-dedup) |
| language:       | enum (ISO 639-1)                |
| last_crawled:   | timestamp                       |
| last_modified:  | timestamp                       |
| content_length: | uint32                          |
| pagerank:       | float32                         |
| spam_score:     | float32                         |
| domain_id:      | uint32                          |
| outlink_count:  | uint16                          |
| inlink_count:   | uint32                          |



### 7.6.1.3 URL Frontier Entry

URL Frontier Record:

```
url: string
priority: float32 (based on PageRank)
domain: string (for politeness)
last_crawled: timestamp
crawl_interval: duration (adaptive)
retry_count: uint8
status: enum (PENDING, IN_PROGRESS,
 DONE, FAILED)
```

## 7.6.2 6.2 Database Selection

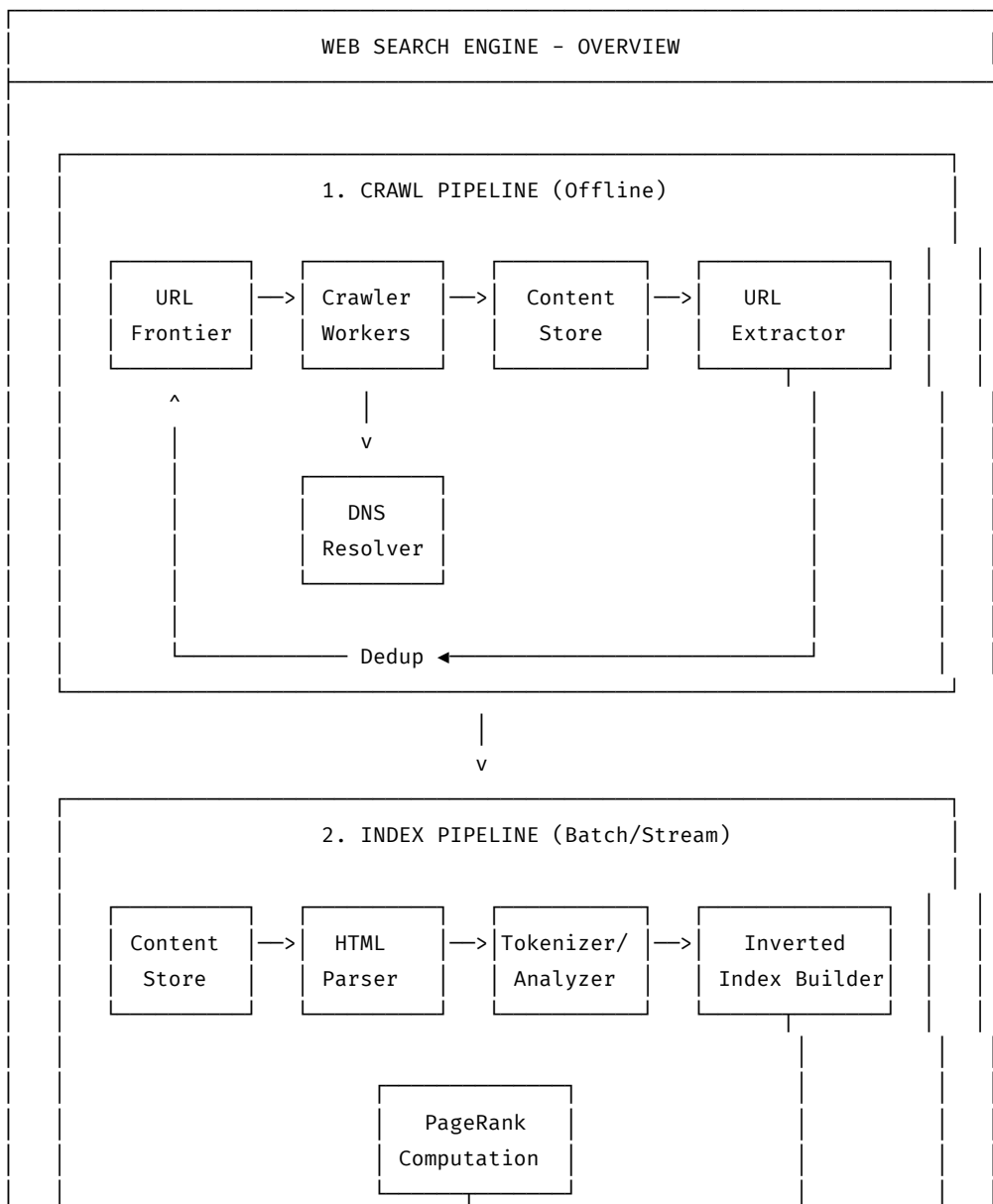
| Data                  | Storage System                                                                     |
|-----------------------|------------------------------------------------------------------------------------|
| Inverted Index        | Custom distributed store (SSTable-like)<br>In-memory term dict + SSD posting lists |
| Raw HTML / Content    | Distributed FS (GFS / Colossus / HDFS)                                             |
| Document Metadata     | BigTable / HBase (wide-column store)                                               |
| URL Frontier          | Distributed priority queue<br>(Redis sorted sets + persistent backing)             |
| Link Graph            | Graph store (custom adjacency lists)                                               |
| PageRank Scores       | Pre-computed, stored in doc metadata                                               |
| Query Logs            | Append-only distributed log (Kafka)                                                |
| Autocomplete Trie     | In-memory trie / prefix tree<br>(replicated across query servers)                  |
| Spell Correction Dict | In-memory (BK-tree / SymSpell)                                                     |

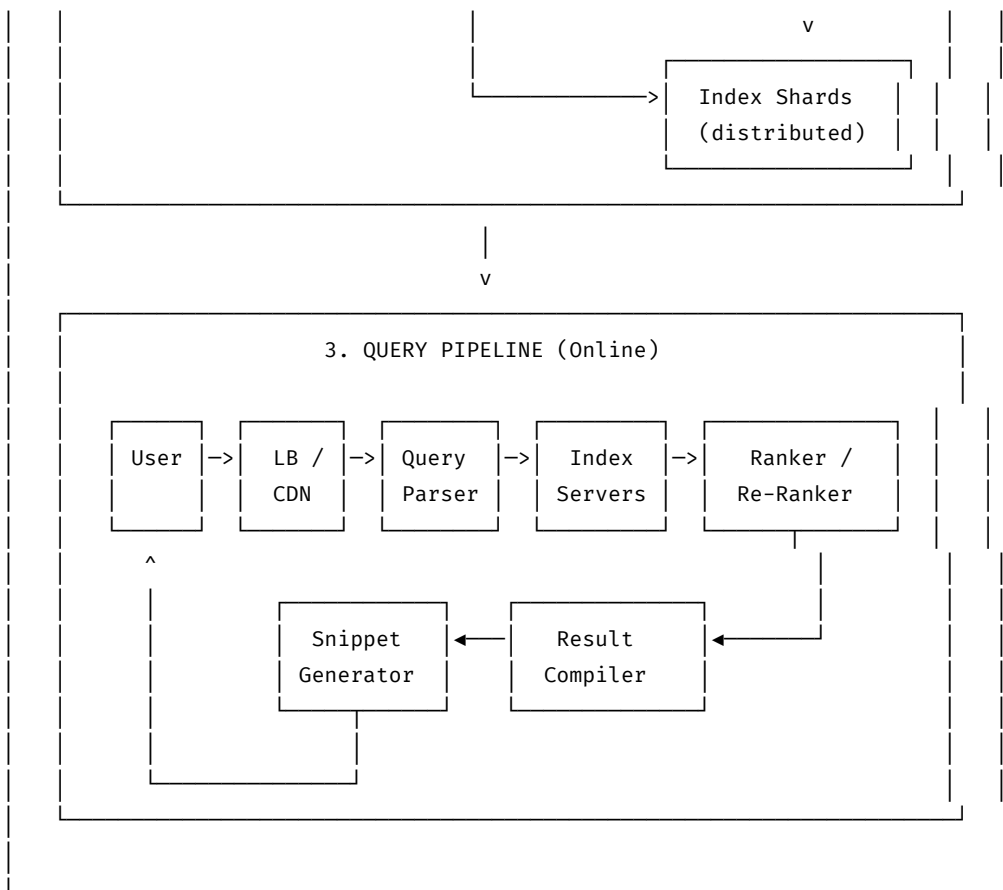
### 7.6.3 6.3 Why Not a Traditional RDBMS?

- **Scale:** 100B documents with 10B unique terms cannot fit in a single relational database
- **Access patterns:** Search requires posting list lookups (sequential scan of sorted lists), not row-by-row queries
- **Custom compression:** Inverted indexes use domain-specific compression (delta encoding, variable-byte) that relational stores don't support
- **Latency:** We need sub-millisecond posting list lookups, which requires memory-mapped custom data structures

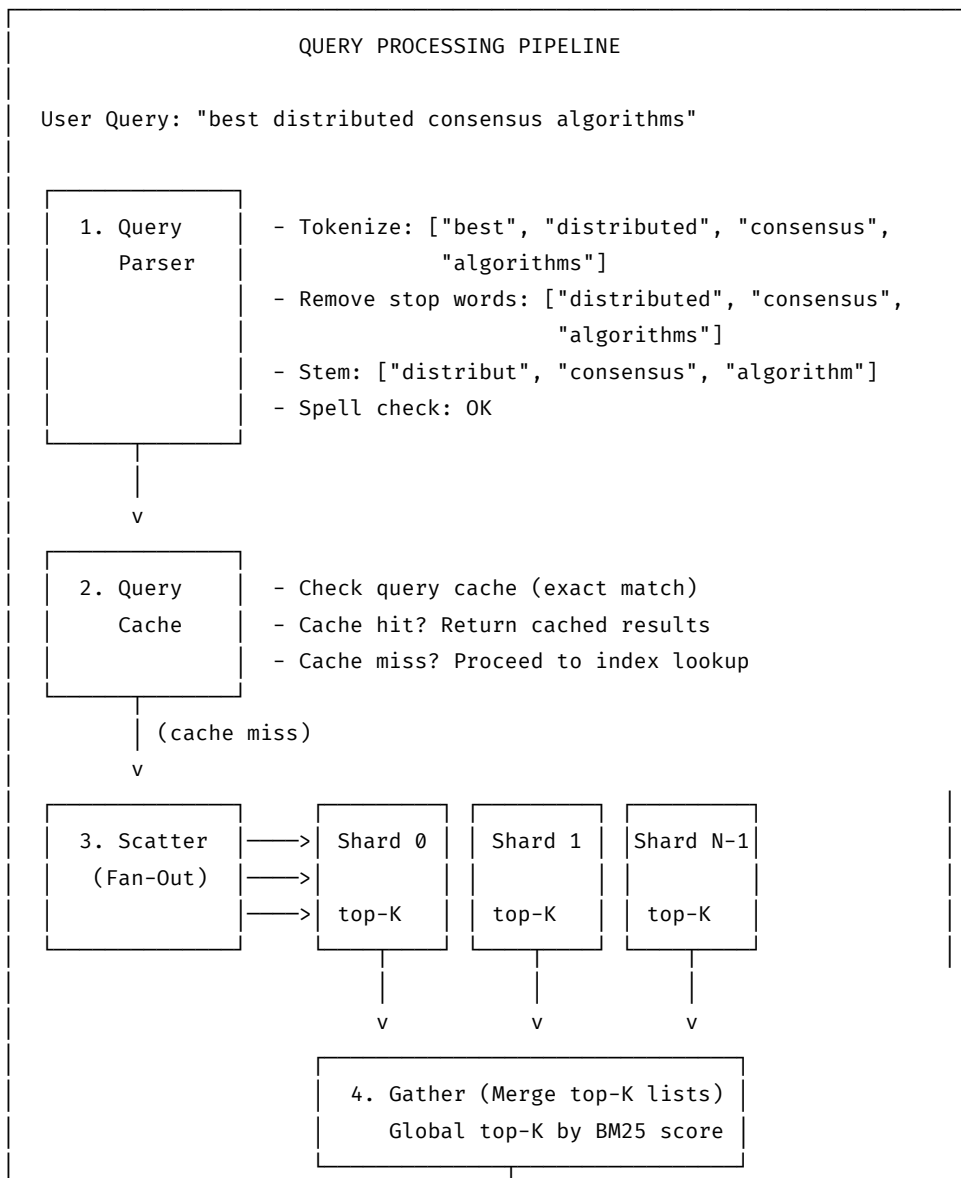
## 7.7 7. High-Level Architecture

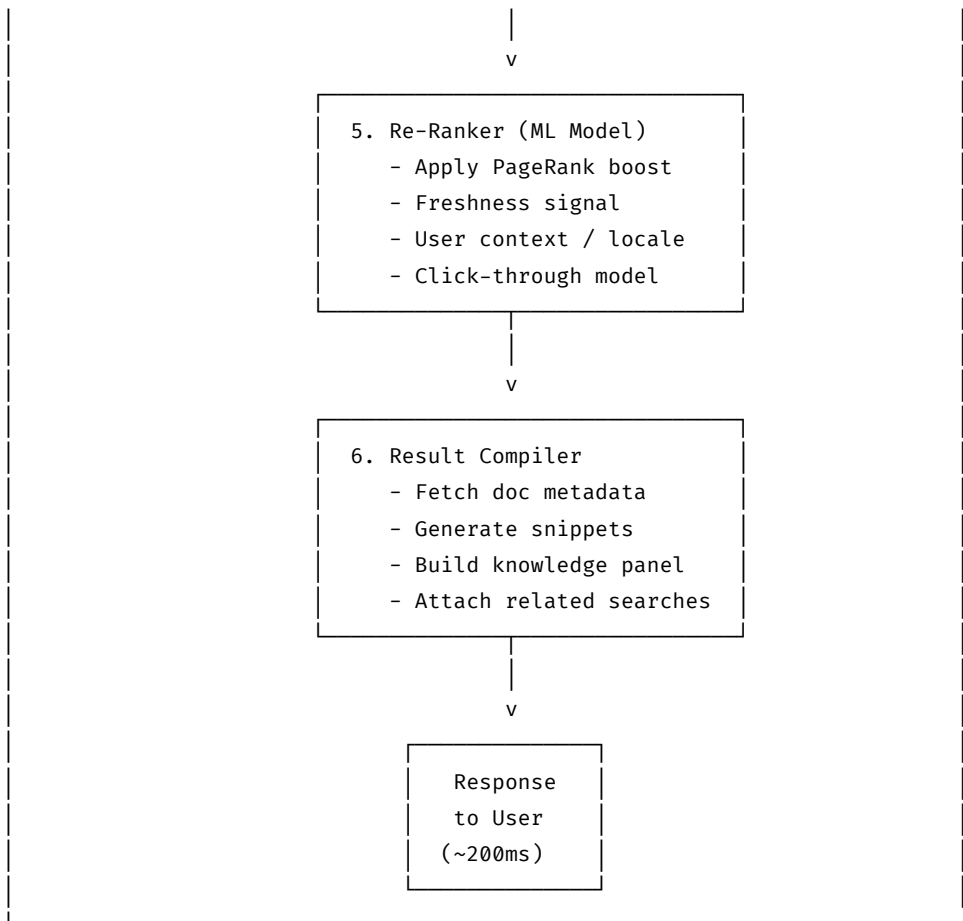
### 7.7.1 7.1 System Overview - Three Major Pipelines





### 7.7.2 7.2 Query Pipeline - Detailed Flow





### 7.7.3 7.3 Component Breakdown

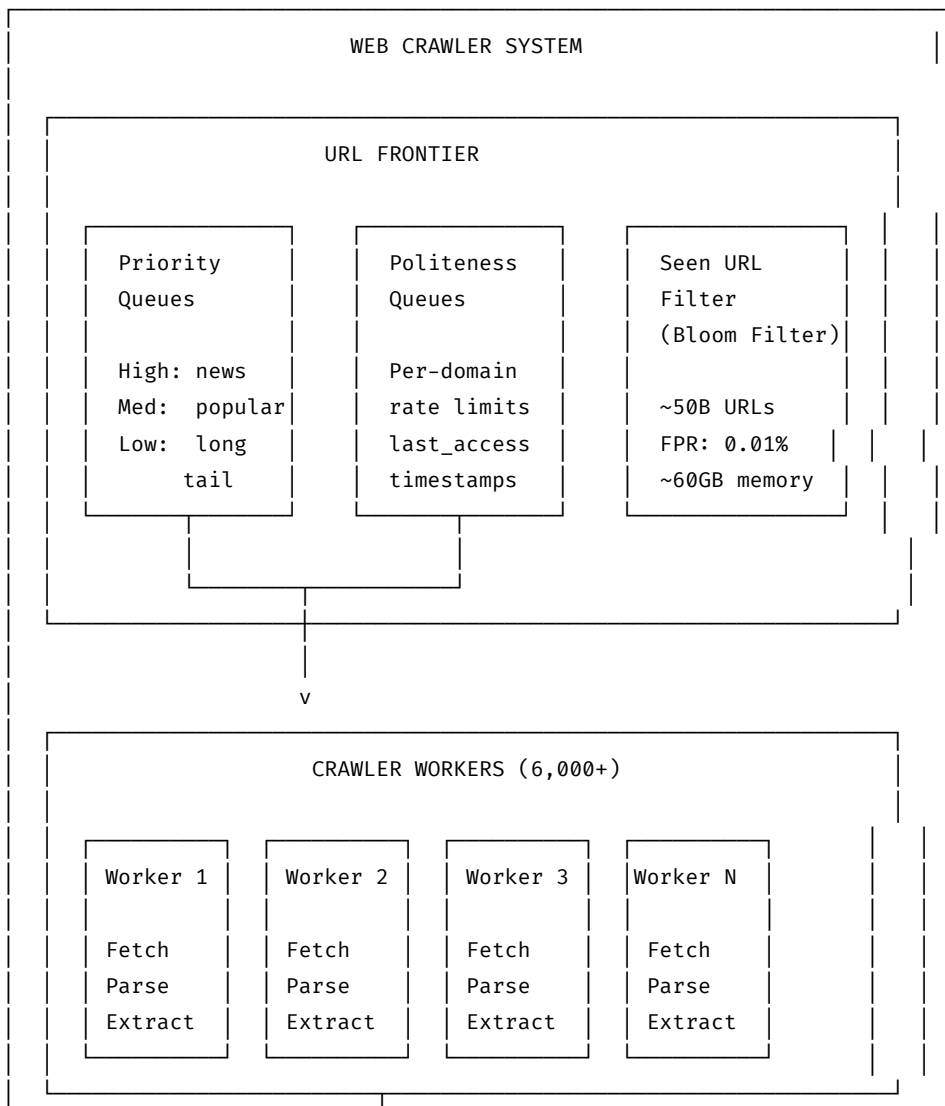
- **Load Balancer / CDN:** Route queries to nearest DC, cache static assets; Scale: Global, 20+ DCs
- **Query Parser:** Tokenize, normalize, spell-check, classify intent; Scale: Stateless, ~5K instances
- **Query Cache:** Cache frequent query results (LRU); Scale: ~100 TB distributed cache
- **Index Servers:** Store index shards, execute BM25 retrieval; Scale: ~30K machines
- **Ranker / Re-ranker:** ML-based relevance scoring; Scale: Co-located with index or separate tier
- **Snippet Generator:** Extract relevant text from stored documents; Scale: Stateless workers
- **Result Compiler:** Assemble final response (results + panels + suggestions); Scale: Stateless workers
- **Autocomplete Service:** Trie-based prefix matching on query logs; Scale: In-memory, replicated
- **Spell Correction:** Edit-distance computation, language models; Scale: In-memory, replicated

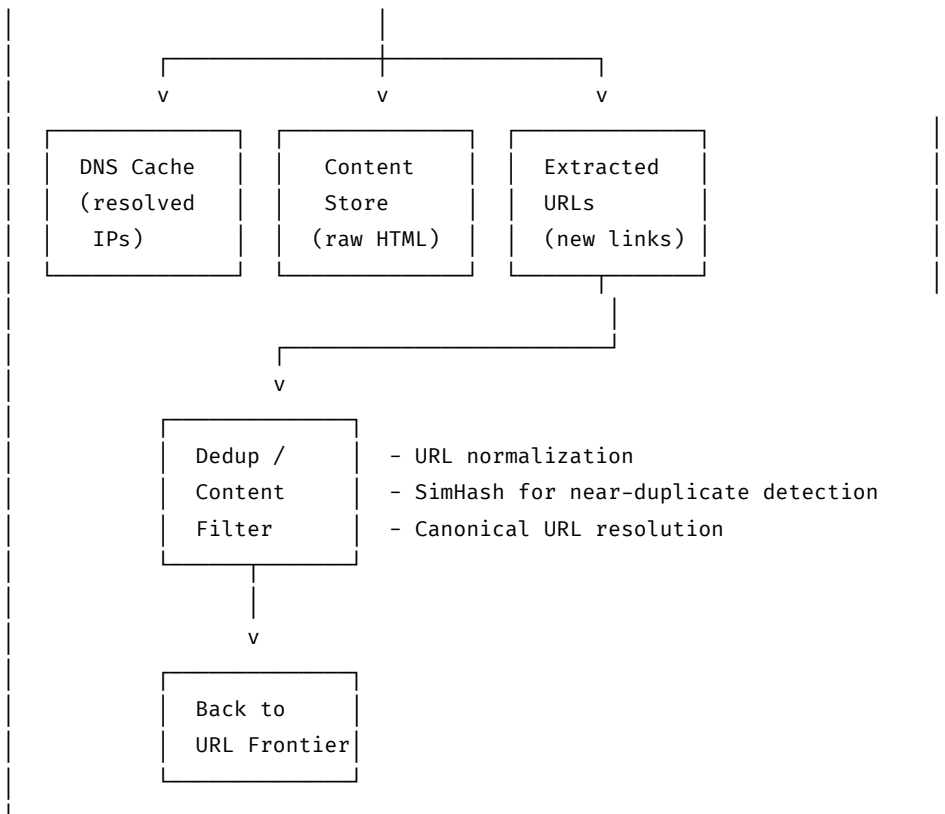
## 7.8 8. Deep Dive: Core Components

### 7.8.1 8.1 Web Crawler

The web crawler is the data ingestion layer of the search engine. It continuously discovers and fetches web pages, feeding content into the indexing pipeline.

### 7.8.1.1 8.1.1 Crawler Architecture



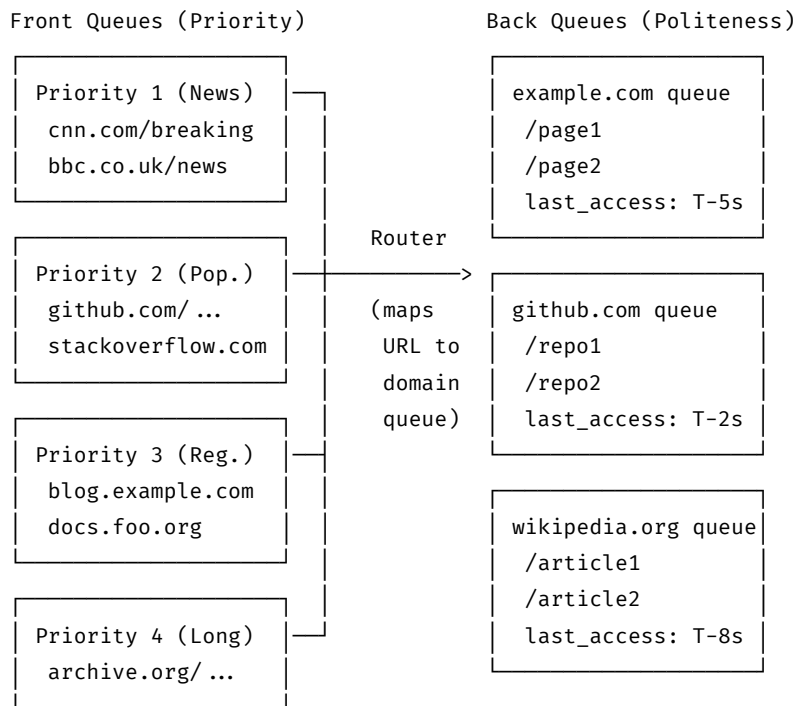


### 7.8.1.2 8.1.2 URL Frontier Design

The URL frontier is the most critical data structure in the crawler. It determines **what** to crawl and **when** to crawl it.

#### Two-level queue architecture:

1. **Front queues (Priority):** Multiple queues ordered by priority
  - Priority 1: Breaking news sites, frequently updated pages
  - Priority 2: Popular sites (top 1M by PageRank)
  - Priority 3: Regular content
  - Priority 4: Deep web, rarely changing pages
2. **Back queues (Politeness):** One queue per domain
  - Ensures at most one outstanding request per domain
  - Enforces minimum delay between requests (typically 1-10 seconds)
  - Respects `Crawl-delay` directive from robots.txt



### 7.8.1.3 8.1.3 Politeness and Ethics

Robots.txt Handling:

1. Fetch and cache robots.txt for each domain (TTL: 24 hours)
2. Parse Allow/Disallow directives
3. Respect Crawl-delay (default: 1 second between requests)
4. Honor noindex, nofollow meta tags

Rate Limiting per Domain:

- Max 1 concurrent request per domain
- Minimum 1-second gap between requests to same domain
- Back off exponentially on 429 / 503 responses
- Track domain health score; reduce crawl rate for slow domains

Ethical Crawling:

- Identify crawler via User-Agent header
- Provide contact info for webmasters
- Support robots.txt "Sitemap:" directive
- Handle HTTP 304 Not Modified (If-Modified-Since)

### 7.8.1.4 8.1.4 Handling Dynamic Content

Modern web pages rely heavily on JavaScript. A search engine must render JavaScript to see the full page content.



Approach: Two-Phase Crawling

Phase 1 - Static HTML Fetch:

- Fast, low-cost HTTP GET
- Extracts links, basic text, metadata
- Sufficient for ~60% of web pages

Phase 2 - JavaScript Rendering:

- Headless browser (Chromium-based)
- Wait for page load + dynamic content
- Much more expensive (~10x cost of static fetch)
- Used for pages detected as JS-heavy

Decision Logic:

```
if page has <script> tags AND extracted text is minimal:
 queue for Phase 2 rendering
else:
 use Phase 1 content
```

### 7.8.1.5 8.1.5 Duplicate and Near-Duplicate Detection

URL-level Dedup:

- Normalize URLs (lowercase host, remove fragments, sort params)
- Bloom filter with 50B entries, 0.01% FPR
- Memory: ~60 GB for 50B URLs

Content-level Dedup (SimHash / MinHash):

- Compute SimHash (64-bit fingerprint) of page content
- Two documents are near-duplicates if Hamming distance  $\leq 3$
- Store fingerprints in a lookup table
- O(1) lookup for exact duplicates
- For near-duplicates: use multi-probe approach (flip k bits)

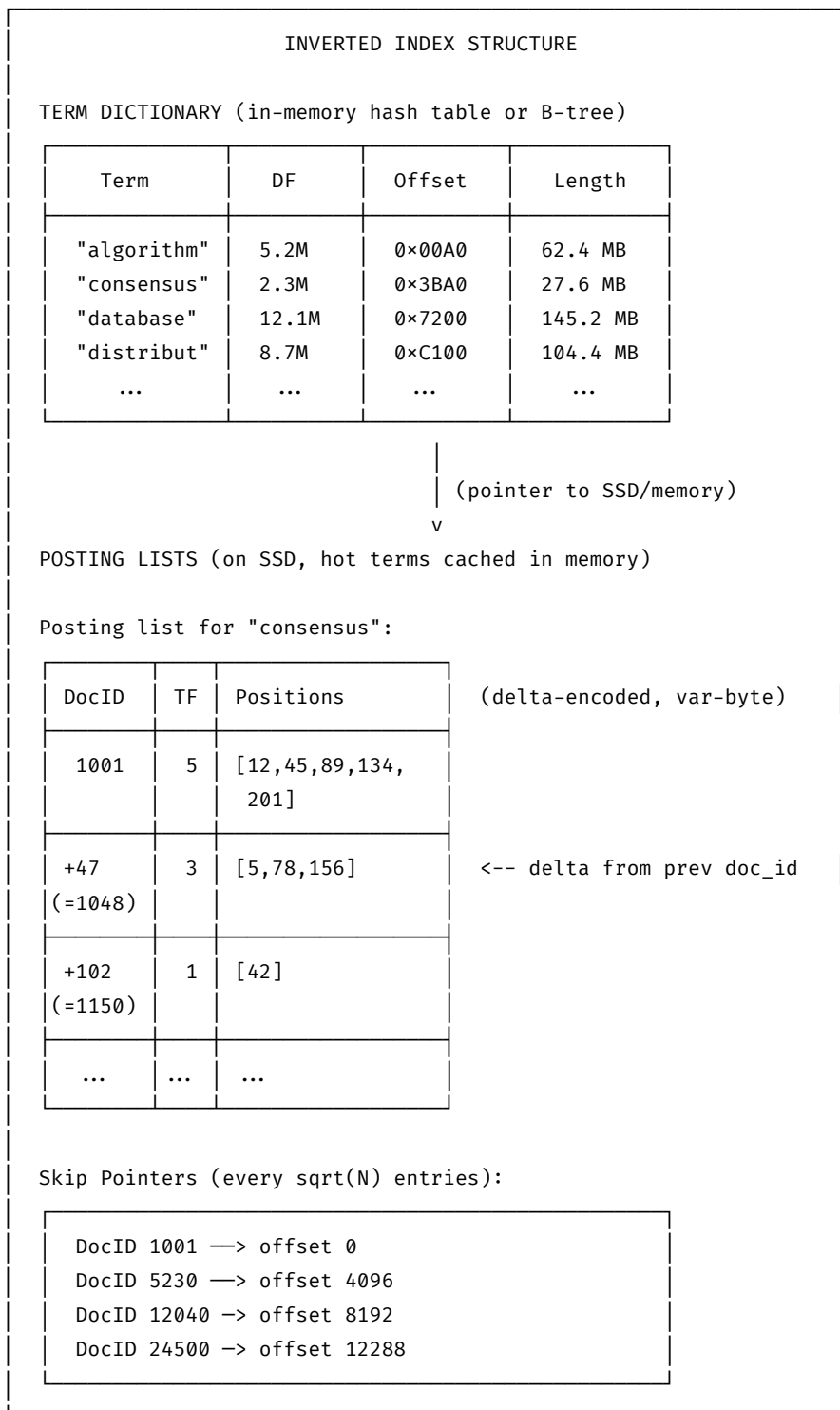
SimHash Algorithm:

1. Tokenize document into shingles (3-grams)
  2. Hash each shingle to 64-bit value
  3. For each bit position: sum weights (+1 for 1, -1 for 0)
  4. Final hash: 1 if sum > 0, else 0 for each position
  5. Compare: Hamming distance = number of differing bits
- 

## 7.8.2 8.2 Inverted Index

The inverted index is the heart of the search engine. It maps every term to the list of documents containing that term, enabling efficient full-text search.

### 7.8.2.1 8.2.1 Inverted Index Structure



### 7.8.2.2 8.2.2 Index Compression

Compression is critical because the raw inverted index is ~150 TB. Good compression reduces this to ~50-80 TB while maintaining fast decompression.

## Technique 1: Delta Encoding (for doc\_ids)

Original: [1001, 1048, 1150, 1302, 1510, 1998]

Deltas: [1001, 47, 102, 152, 208, 488]

Observation: Deltas are much smaller numbers, fewer bits needed

## Technique 2: Variable-Byte (VByte) Encoding

- Use 7 bits per byte for data, 1 bit as continuation flag
- Small numbers (< 128) = 1 byte
- Medium numbers (< 16384) = 2 bytes
- Large numbers (< 2M) = 3 bytes

Example: encoding delta 47

47 < 128, so: [0|0101111] = 1 byte

Example: encoding delta 488

488 ≥ 128, so: [1|0000011] [0|1101000] = 2 bytes

## Technique 3: PForDelta (Patched Frame-of-Reference)

- Encode blocks of 128 doc\_id deltas at a time
- Find the minimum bits needed for most values (e.g., 90th percentile)
- Encode most values with that bit width
- "Patch" exceptions (outliers) with a separate list
- Very SIMD-friendly for fast decompression

## Compression Ratios:

| Technique      | Ratio   | Decode Speed |
|----------------|---------|--------------|
| Uncompressed   | 1.0x    | Baseline     |
| VByte          | 3-4x    | ~1 GB/s      |
| PForDelta      | 5-8x    | ~2 GB/s      |
| Simple-9/16    | 4-6x    | ~1.5 GB/s    |
| Roaring Bitmap | 10-100x | ~3 GB/s      |

**7.8.2.3 8.2.3 Index Construction (MapReduce)**

## INDEX CONSTRUCTION PIPELINE

## Phase 1: MAP

Input: (doc\_id, raw\_content) pairs

For each document:

1. Parse HTML, extract text
2. Tokenize: split into terms
3. Normalize: lowercase, remove accents
4. Stem: "running" -> "run", "algorithms" -> "algorithm"
5. Emit: (term, doc\_id, term\_frequency, [positions])

Example:

Doc 1001: "Raft is a consensus algorithm"

Emits:

```
("raft", 1001, 1, [0])
("consensus", 1001, 1, [3])
("algorithm", 1001, 1, [4])
```

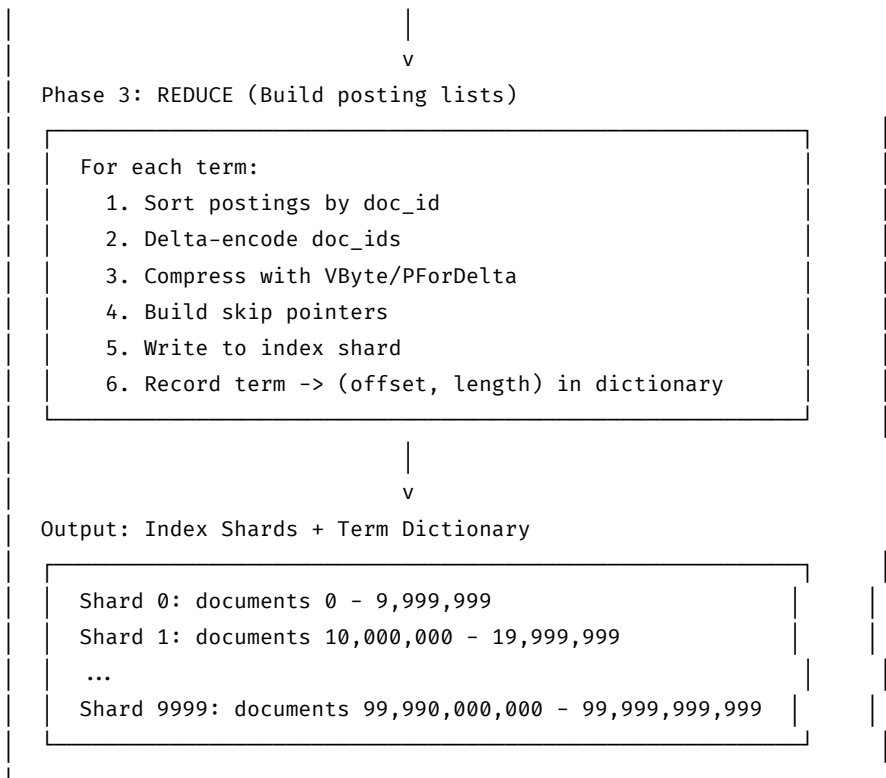
|  
v

## Phase 2: SHUFFLE (Group by term)

Sort and group all (term, doc\_id, tf, positions)  
by term, then by doc\_id within each term

```
"algorithm" -> [(1001, 1, [4]), (1048, 3, [1,5,12]),
 (1150, 2, [0,7]), ...]
```

```
"consensus" -> [(1001, 1, [3]), (1048, 2, [0,6]), ...]
```



### 7.8.2.4 8.2.4 Index Partitioning Strategies

Strategy 1: Document-Partitioned (Used by Google, most search engines)

Each shard contains ALL terms for a SUBSET of documents.

|                                |                                |                                |
|--------------------------------|--------------------------------|--------------------------------|
| Shard 0                        | Shard 1                        | Shard 2                        |
| Docs 0-10M                     | Docs 10M-20M                   | Docs 20M-30M                   |
| All terms<br>for these<br>docs | All terms<br>for these<br>docs | All terms<br>for these<br>docs |

Query execution: Scatter query to ALL shards, gather top-K from each

Pros:

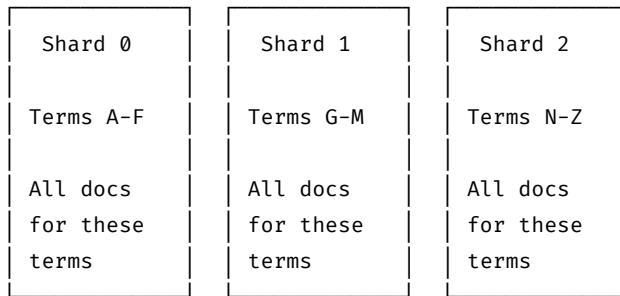
- + Each shard is self-contained (can compute local BM25 scores)
- + Easy to add/remove documents (affect only one shard)
- + Shard failure only loses a fraction of documents
- + Load naturally balanced for most queries

Cons:

- Every query must hit ALL shards (high fan-out)
- N shards = N network round trips per query

Strategy 2: Term-Partitioned

Each shard contains ALL documents for a SUBSET of terms.



Query execution: Route each query term to its shard, intersect results

Pros:

- + Single-term queries hit only ONE shard
- + Less fan-out for simple queries

Cons:

- Multi-term queries require cross-shard intersection
- Hot terms (e.g., "the") create hot shards
- Adding documents requires updating EVERY shard
- Harder to compute document-level scores (BM25)

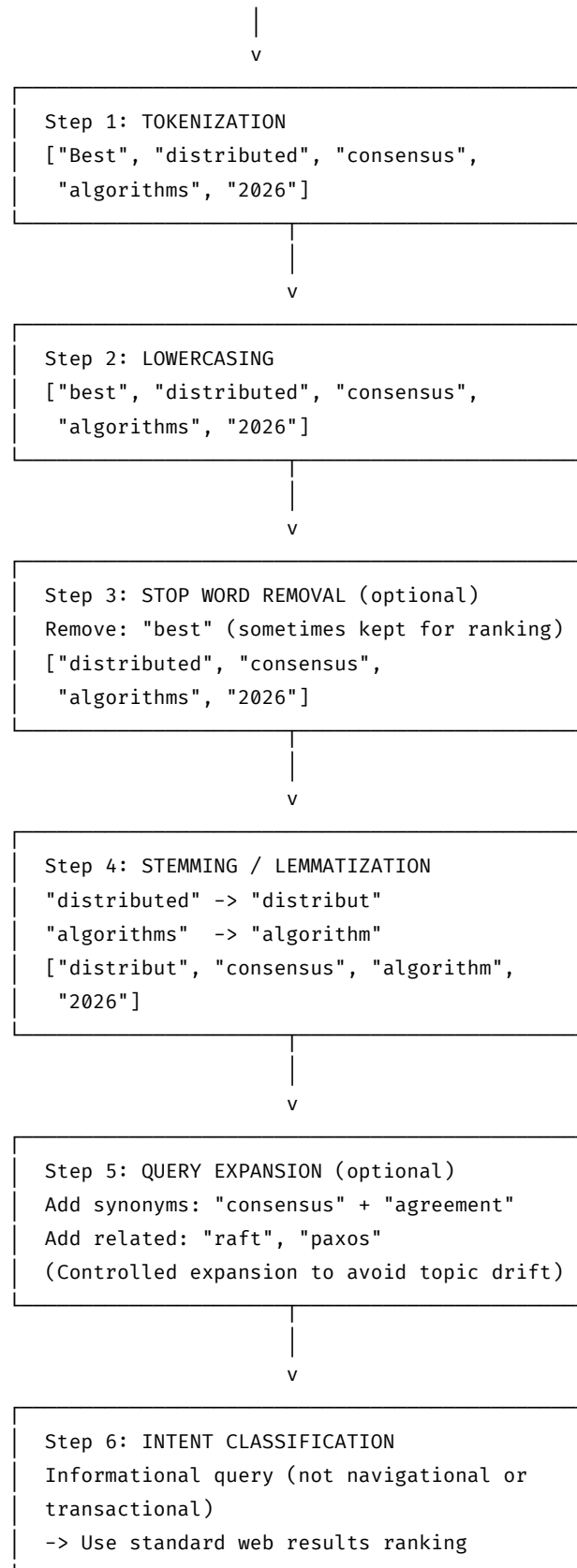
Verdict: Document-partitioned wins for web search at scale.

---

### 7.8.3 8.3 Query Processing and Ranking

#### 7.8.3.1 8.3.1 Query Processing Pipeline

Input: "Best distributed consensus algorithms 2026"





**7.8.3.2 8.3.2 Scoring: From Boolean to BM25**

Evolution of Scoring Models:

1. Boolean Retrieval (1960s)

Query: "distributed AND consensus"

Returns: All documents containing BOTH terms (unranked)

Problem: No ranking, returns too many or too few results

2. TF-IDF (1970s)

$\text{score}(q, d) = \text{SUM over terms } t \text{ in } q:$

$\text{tf}(t, d) * \text{idf}(t)$

where:

$\text{tf}(t, d)$  = frequency of term  $t$  in document  $d$

$\text{idf}(t) = \log(N / \text{df}(t))$

= inverse document frequency (rare terms score higher)

Problem: No term frequency saturation (repeating a term

100 times shouldn't be 100x better than once)

3. BM25 (1990s, Okapi BM25) - INDUSTRY STANDARD

$\text{score}(q, d) = \text{SUM over terms } t \text{ in } q:$

$\text{idf}(t) * (\text{tf}(t, d) * (k_1 + 1)) / (\text{tf}(t, d) + k_1 * (1 - b + b * |d| / \text{avgdl}))$

where:

$k_1 = 1.2$  (term frequency saturation parameter)

$b = 0.75$  (document length normalization parameter)

$|d|$  = document length

$\text{avgdl}$  = average document length in collection

Key properties:

- tf saturation: diminishing returns for high term frequency
- Length normalization: longer documents don't get unfair advantage
- IDF weighting: rare terms are more important

Example Calculation:

Query: "consensus algorithm"

Document d (length 500 words, avgdl = 1000):

$tf("consensus", d) = 5$

$tf("algorithm", d) = 3$

$df("consensus") = 2.3M$  out of 100B docs

$df("algorithm") = 5.2M$  out of 100B docs

$idf("consensus") = \log((100B - 2.3M + 0.5) / (2.3M + 0.5)) = 10.67$

$idf("algorithm") = \log((100B - 5.2M + 0.5) / (5.2M + 0.5)) = 9.85$

BM25("consensus"):

$10.67 * (5 * 2.2) / (5 + 1.2 * (1 - 0.75 + 0.75 * 500/1000))$

$= 10.67 * 11 / (5 + 1.2 * 0.625)$

$= 10.67 * 11 / 5.75$

$= 20.40$

BM25("algorithm"):

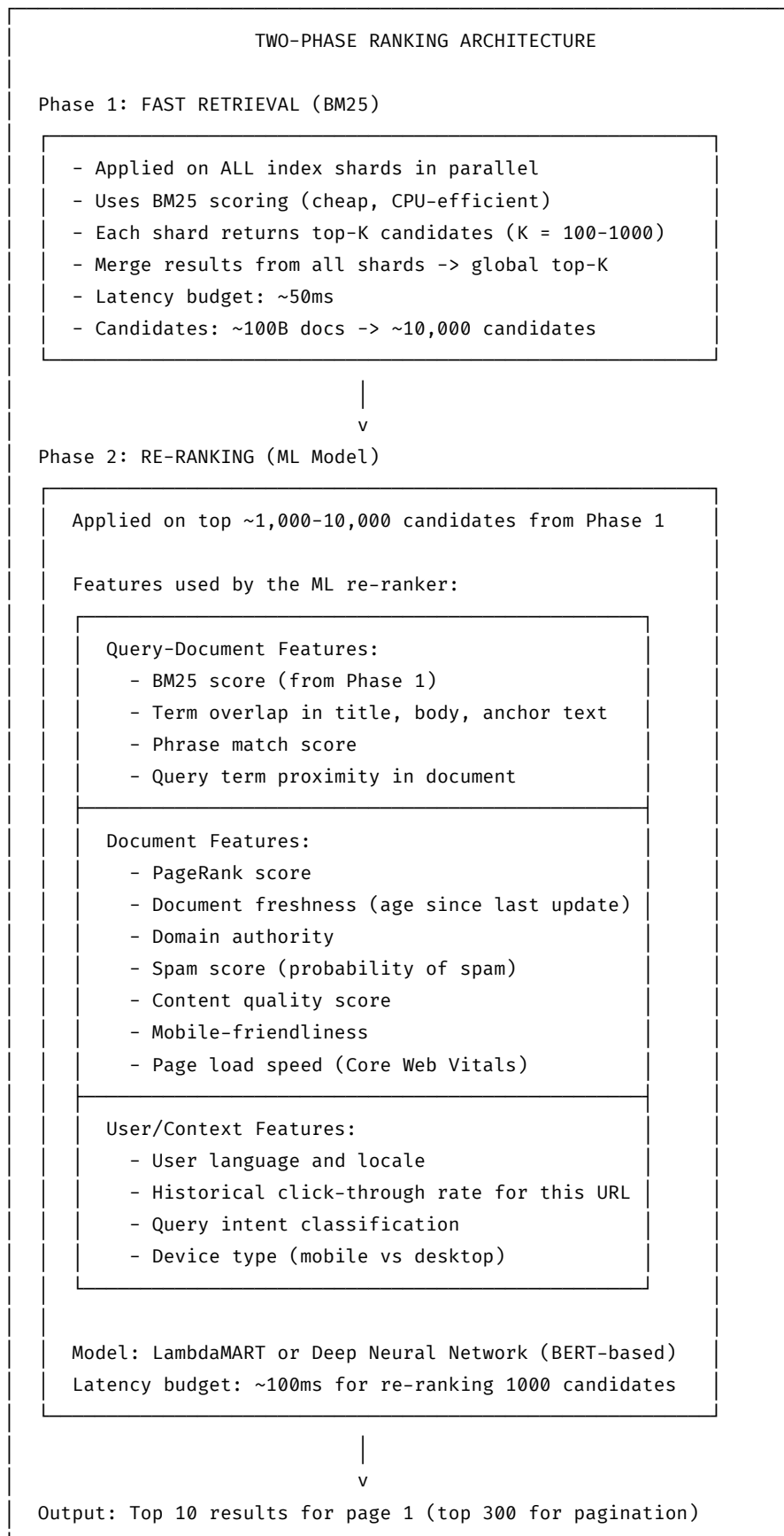
$9.85 * (3 * 2.2) / (3 + 1.2 * 0.625)$

$= 9.85 * 6.6 / 3.75$

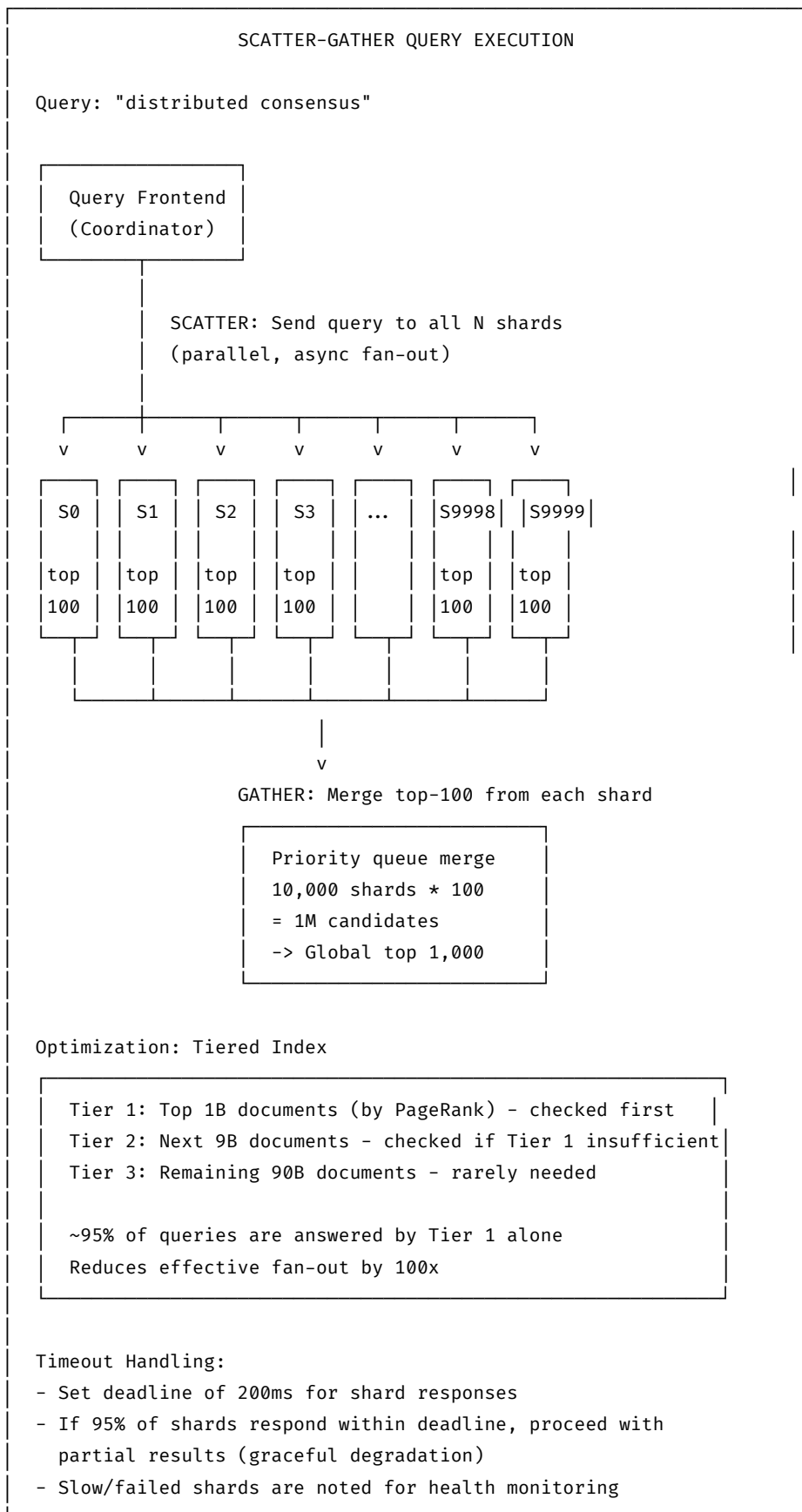
$= 17.34$

Total BM25 score =  $20.40 + 17.34 = 37.74$

### 7.8.3.3 8.3.3 Two-Phase Ranking



### 7.8.3.4 8.3.4 Scatter-Gather Execution



### 7.8.4 8.4 PageRank Algorithm

PageRank is a link-analysis algorithm that assigns an importance score to every web page based on the link structure of the web.

#### 7.8.4.1 8.4.1 Core Concept

Intuition: A page is important if it is linked to by other important pages.

Formula:

$$PR(A) = (1 - d) / N + d * \text{SUM over pages B linking to A:} \\ PR(B) / L(B)$$

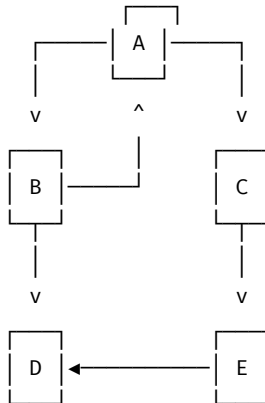
where:

$d$  = damping factor (typically 0.85)

$N$  = total number of pages

$L(B)$  = number of outgoing links from page B

Example Web Graph:



Iteration 0 (initial):  $PR(A)=PR(B)=PR(C)=PR(D)=PR(E) = 0.20$

Iteration 1:

$$PR(A) = 0.15/5 + 0.85 * PR(B)/2 = 0.03 + 0.85 * 0.10 = 0.115$$

$$PR(B) = 0.15/5 + 0.85 * PR(A)/2 = 0.03 + 0.85 * 0.10 = 0.115$$

$$PR(C) = 0.15/5 + 0.85 * PR(A)/2 = 0.03 + 0.85 * 0.10 = 0.115$$

$$PR(D) = 0.15/5 + 0.85 * (PR(B)/2 + PR(E)/1) = 0.03 + 0.85*(0.10+0.20) = 0.285$$

$$PR(E) = 0.15/5 + 0.85 * PR(C)/1 = 0.03 + 0.85 * 0.20 = 0.200$$

... converges after ~50-100 iterations

### 7.8.4.2 8.4.2 Handling Edge Cases

Problem 1: Dead Ends (Dangling Nodes)

Pages with no outgoing links absorb PageRank and never distribute it.

Solution: Redistribute dangling node PageRank equally to all pages.

For each dangling node  $D$ :

$PR_{redistributed} = PR(D) / N$  (added to every page)

Problem 2: Spider Traps

A group of pages that only link to each other, accumulating all PageRank.

Solution: The damping factor  $d = 0.85$  ensures that with probability  $(1 - d) = 0.15$ , a random surfer jumps to a random page, breaking out of the trap.

Problem 3: Scale (100B pages)

The PageRank vector has 100B entries. A single iteration requires reading the entire link graph.

Solution: MapReduce implementation (see below).

### 7.8.4.3 8.4.3 PageRank at Scale with MapReduce

#### PAGERANK COMPUTATION VIA MAPREDUCE

Input: Link graph G and current PR vector

Per Iteration:

MAP Phase:

```
For each page P with outlinks [L1, L2, ..., Lk]:
 contribution = PR(P) / k
 For each Li:
 emit (Li, contribution)
 emit (P, outlink_list) // preserve graph structure
```

REDUCE Phase:

```
For each page P:
 new_PR(P) = (1-d)/N + d * SUM(all contributions to P)
 emit (P, new_PR(P), outlink_list)
```

Repeat for 50-100 iterations until convergence  
( $|PR_{new} - PR_{old}| < \epsilon$  for all pages)

At Google's Scale:

- Link graph: ~2 TB (100B pages, avg 20 outlinks each)
- PR vector: ~800 GB (100B \* 8 bytes)
- Each iteration: ~3 TB read + write
- 50 iterations: ~150 TB of I/O
- Time per iteration: ~30 minutes on 10K machines
- Total: ~25 hours for full recomputation
- Schedule: recomputed weekly or incrementally updated

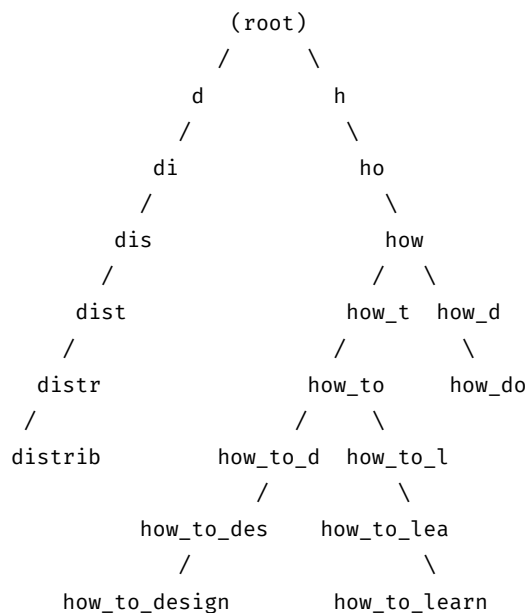
## 7.8.5 8.5 Autocomplete and Query Suggestions

### AUTOCOMPLETE SYSTEM

Data Source: Query logs (billions of past queries + frequencies)

Data Structure: Trie with Top-K Caching

Example Trie:



Each node stores: top-8 completions by popularity

Node "how\_to\_des":

```

top_completions = [
 ("how to design a database", freq: 50K/month),
 ("how to design a search engine", freq: 12K/month),
 ("how to design distributed systems", freq: 8K/month),
 ...
]

```

Size: ~50M unique query prefixes \* ~200 bytes = ~10 GB

Fits entirely in memory on each query server

Updates: Rebuild trie every few hours from query log aggregation

Personalization: Boost suggestions based on user history (optional)

Latency: < 10ms (in-memory trie lookup)



### 7.8.6 8.6 Spell Correction

#### SPELL CORRECTION SYSTEM

Approach 1: Edit Distance (Norvig's method)

- Generate all words within edit distance 1-2 of query term
- Check which candidates exist in the dictionary
- Rank by:  $P(\text{correction} \mid \text{misspelling}) \sim P(\text{misspelling} \mid \text{correction}) * P(\text{correction})$
- $P(\text{correction})$  from query log frequencies

Approach 2: SymSpell (faster, pre-computed)

- Pre-compute all delete-only edits for dictionary words
- At query time: generate deletes of query, look up in table
- O(1) lookup instead of generating all edit combinations
- 1M dictionary words, max edit distance 2:  
Pre-computed entries: ~25M (fits in ~2 GB)

Approach 3: Noisy Channel Model (production systems)

- Train on (misspelling, correction) pairs from query logs
- Error model:  $P(\text{"teh"} \mid \text{"the"})$  from observed typo patterns
- Language model:  $P(\text{"the cat"})$  from n-gram frequencies
- Combined:  $\text{argmax}_c P(c) * P(\text{query} \mid c)$

Example:

Input: "distribted systms"

Candidates for "distribted":

"distributed" (edit dist 1, freq: very high) -> winner  
"distribute" (edit dist 2, freq: high)

Candidates for "systms":

"systems" (edit dist 1, freq: very high) -> winner  
"stems" (edit dist 2, freq: medium)

Output: "distributed systems" (confidence: 0.97)

Context-Aware Correction:

"distributed systems" is far more likely as a phrase than "distribute stems", so the bigram/trigram language model heavily favors the correct interpretation.

### 7.8.7 8.7 Snippet Generation

#### SNIPPET GENERATION

Goal: Extract 1-2 sentence fragment from the document that best matches the query, with query terms highlighted in <b> tags.

Algorithm:

1. Retrieve stored document text (from content store)
2. Split into sentences
3. Score each sentence:
  - Count of query terms present
  - Proximity of query terms to each other
  - Position in document (earlier = slightly better)
  - Term density (query terms / total words in sentence)
4. Select top 1-2 sentences
5. Truncate to ~160 characters
6. Wrap query terms in <b>...</b> tags

Example:

Query: "distributed consensus algorithms"

Document: (long article about Raft)

Best sentence: "Raft is a distributed consensus algorithm designed to be more understandable than Paxos."

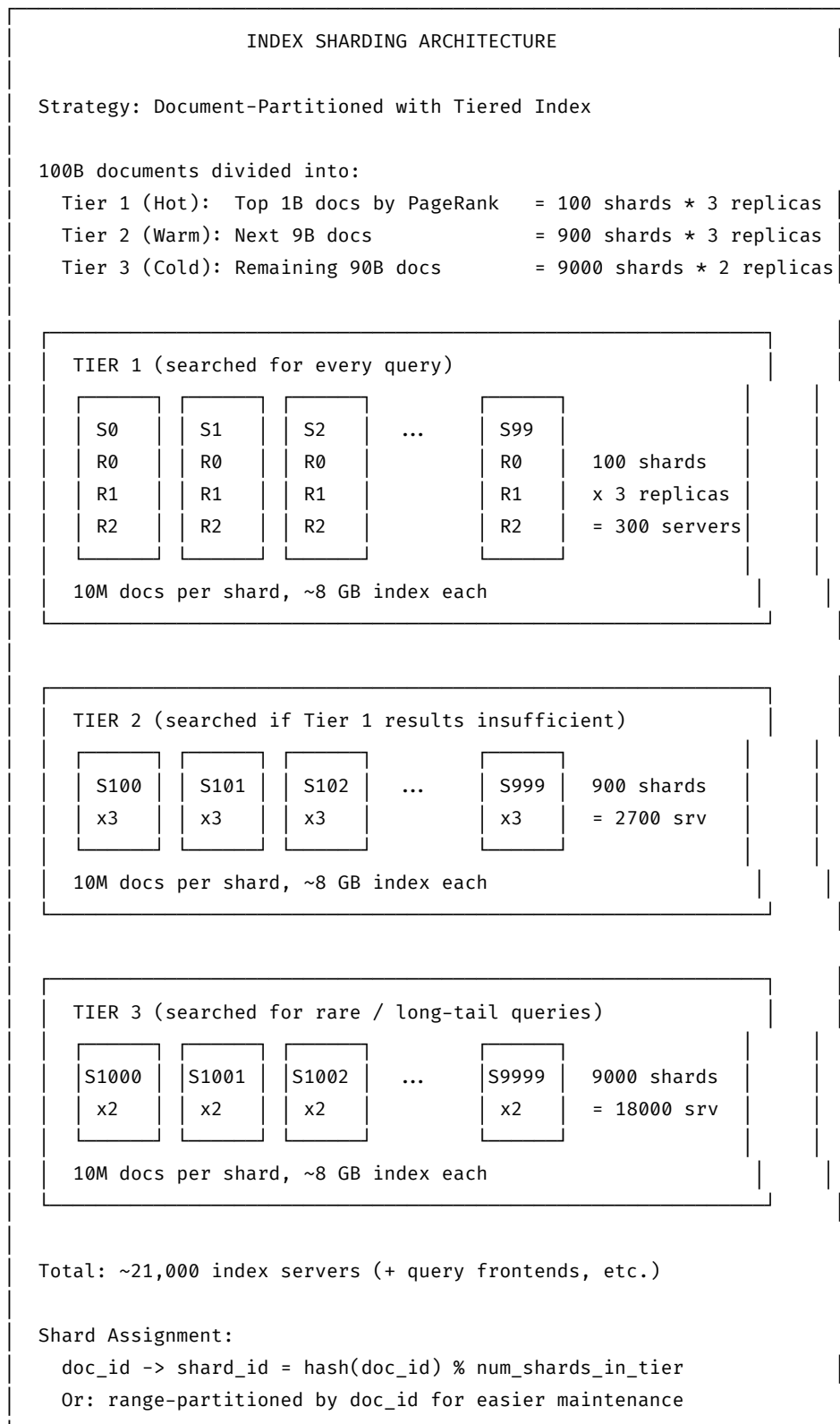
Snippet: "Raft is a <b>distributed consensus algorithm</b> designed to be more understandable than Paxos."

Performance:

- Must generate snippets for top-10 results per query
- Latency budget: ~20ms (done in parallel with re-ranking)
- Pre-stored sentence boundaries in document metadata help

## 7.9 9. Data Partitioning and Sharding

### 7.9.1 9.1 Index Sharding Strategy



## 7.9.2 9.2 Crawler Data Partitioning

URL Frontier Partitioning:

- Partition by domain hash: URLs for the same domain go to the same partition (required for per-domain rate limiting)
- 1000 frontier partitions, each handling ~350K domains

Content Store Partitioning:

- Partition by content hash (for dedup efficiency)
- Or by URL hash (for retrieval efficiency)
- Replicated 3x across data centers

## 7.10 10. Caching Strategy

### CACHING LAYERS

Layer 1: CDN / Edge Cache

- Cache static assets (CSS, JS, images for SERP UI)
- Geographic distribution for low latency
- TTL: 1 hour for static, no caching for search results

Layer 2: Query Result Cache

Key:     normalized\_query + lang + region + page\_number  
 Value:   serialized search results (top-K + snippets)  
 TTL:     5-60 minutes (shorter for news-related queries)

Cache hit rate: ~30-40% (query distribution follows Zipf)  
 Size: ~50 TB distributed across query frontend servers  
 Eviction: LRU with frequency-based admission (TinyLFU)

Impact: 30% cache hit = 30% fewer scatter-gather operations  
 = massive cost savings

Layer 3: Posting List Cache (per Index Server)

Key:     term\_id  
 Value:   decompressed posting list (or hot prefix of it)

Hot terms like "the", "how", "what" are always cached  
 Top 1M terms cover ~80% of query traffic  
 Size: ~500 GB per index server (in RAM)  
 Eviction: LFU (frequency-based, terms don't change often)

**Layer 4: Document Metadata Cache**

Key: doc\_id  
Value: title, URL, snippet-ready text, PageRank score

Cache top 100M most-accessed documents  
Size: ~1 TB (100M \* 10KB per doc metadata)  
Used during result compilation and snippet generation

**Layer 5: DNS Cache (for Crawler)**

Key: domain  
Value: IP address(es)  
TTL: follows DNS TTL (typically 5-60 minutes)  
Size: 350M domains \* 50 bytes = ~17 GB  
Avoids DNS lookup bottleneck during crawling

**Cache Invalidation:**

- Query result cache: TTL-based (no explicit invalidation)
- Posting list cache: Refreshed when index shards are rebuilt
- Document cache: Invalidated on re-crawl

## 7.11 11. Replication and Consistency

### 7.11.1 11.1 Index Replication

#### INDEX REPLICATION MODEL

Consistency Model: Eventually Consistent (acceptable for search)

Index Rebuild Process:

1. Index pipeline builds new index shards (batch, ~hours)
2. New shard replicas are copied to serving machines
3. Atomic swap: old shard -> new shard (via symlink or load balancer reroute)
4. Old shard kept for rollback (24-hour retention)

During swap, some replicas serve old index, some new:  
User may see slightly different results for ~minutes.  
This is acceptable for search.

Replication Topology:

Data Center A

Shard 0, R0  
Shard 1, R0  
...

Data Center B

Shard 0, R1  
Shard 1, R1  
...

Data Center C

Shard 0, R2  
Shard 1, R2  
...

Each DC has a full copy of the index for local query serving  
Cross-DC replication is async (index built centrally,  
distributed to all DCs)

No strong consistency needed because:

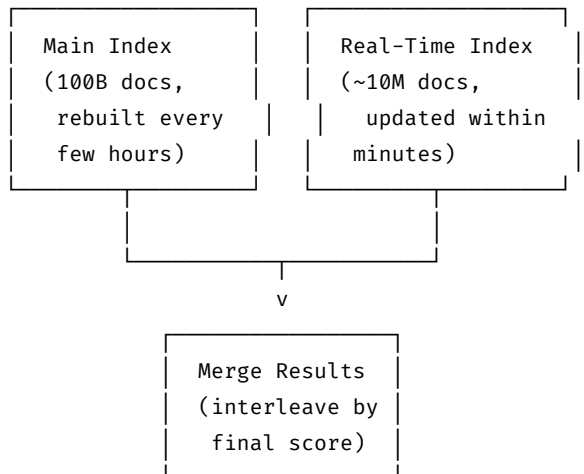
- Users don't notice minor ranking differences between DCs
- Index is rebuilt periodically (not real-time updated)
- Freshness is handled by a separate real-time index overlay

### 7.11.2 11.2 Real-Time Index for Freshness

#### REAL-TIME INDEX OVERLAY

Problem: The main index is rebuilt every few hours. Breaking news needs to appear within minutes.

Solution: Small in-memory real-time index that is merged with main index results at query time.



#### Real-Time Index Properties:

- Size: ~10M recently crawled/updated pages
- Storage: Entirely in-memory (small enough)
- Update latency: < 5 minutes from crawl to searchable
- Merged into main index on next full rebuild
- Covers: breaking news, trending topics, live events

## 7.12 12. Fault Tolerance and Failure Handling

| FAILURE SCENARIOS AND HANDLING |                                                                                                                                                                                                                                                                       |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure Mode                   | Handling Strategy                                                                                                                                                                                                                                                     |
| Index server crash             | <ul style="list-style-type: none"> <li>- Route queries to replica of same shard</li> <li>- 3 replicas per shard: survives 2 failures</li> <li>- Health checker detects within 10 seconds</li> <li>- Auto-restart on same or new machine</li> </ul>                    |
| Slow index shard response      | <ul style="list-style-type: none"> <li>- Query coordinator sets 200ms deadline</li> <li>- If shard misses deadline, use results from responded shards (partial results)</li> <li>- User sees slightly fewer results (ok)</li> </ul>                                   |
| Crawler worker failure         | <ul style="list-style-type: none"> <li>- URLs in-progress are re-queued after timeout (5 minutes)</li> <li>- Idempotent: re-crawling a URL is safe</li> <li>- Worker auto-replaced by orchestrator</li> </ul>                                                         |
| URL frontier node failure      | <ul style="list-style-type: none"> <li>- Persistent backing store (not just in-memory)</li> <li>- Frontier partitioned; loss of one partition = temporary gap in crawling</li> <li>- Rebuild from checkpoint + re-discovery</li> </ul>                                |
| Data center outage             | <ul style="list-style-type: none"> <li>- DNS-based failover to next nearest DC</li> <li>- Each DC has full index replica</li> <li>- Users redirected within ~30 seconds</li> <li>- Capacity planning: each DC can handle 1.5x normal load (N+1 redundancy)</li> </ul> |
| Index build pipeline fails     | <ul style="list-style-type: none"> <li>- Continue serving old index</li> <li>- Alert, retry build</li> <li>- Stale index is better than no index</li> <li>- Canary: deploy new index to 1% of traffic first, check quality metrics</li> </ul>                         |
| Query cache failure            | <ul style="list-style-type: none"> <li>- Cache is a performance optimization, not a correctness requirement</li> <li>- Cache miss -&gt; query goes to index</li> <li>- Higher latency, but correct results</li> <li>- Auto-repopulate as queries flow in</li> </ul>   |



Graceful Degradation Priority:

1. Always return SOME results (even if partial)
2. Prefer fast + slightly incomplete over slow + complete
3. Show cached results if live pipeline is degraded
4. Disable expensive features first (knowledge panels, ML re-rank)
5. Last resort: static "service unavailable" with retry

## 7.13 13. Scalability

### 7.13.1 13.1 Scaling Each Pipeline

#### SCALABILITY STRATEGIES

##### CRAWL PIPELINE SCALING

Bottleneck: Network I/O and DNS resolution

Horizontal: Add more crawler workers

- Each worker handles ~170K pages/day
- 1B pages/day -> ~6,000 workers
- 10B pages/day -> ~60,000 workers

Vertical: More concurrent connections per worker

- Async I/O (epoll/kqueue) for 1000s of connections
- Connection pooling per domain

DNS: Run local DNS caching resolvers

- Pre-resolve domains before crawl
- Cache 350M domain resolutions

##### INDEX PIPELINE SCALING

Bottleneck: Sorting and merging posting lists

Horizontal: Partition MapReduce across thousands of workers

- Map: embarrassingly parallel (one doc = one map task)
- Shuffle: partition by term hash
- Reduce: one reducer per index shard

Incremental: Don't rebuild full index every time

- Merge new/updated documents into existing index
- Full rebuild weekly; incremental updates every hour

#### QUERY PIPELINE SCALING

Bottleneck: Fan-out to index shards, ML re-ranking

Read Replicas: More replicas per shard = more query capacity

- 3 replicas -> 3x throughput per shard
- During peak: spin up additional replicas

Caching: 30-40% cache hit rate offloads index servers

Tiered Index: Only search Tier 1 for most queries

- 100 shards instead of 10,000 for 95% of queries

Geographic: 20+ data centers, serve from nearest

- User in Tokyo -> Tokyo DC (RTT < 10ms)
- User in Berlin -> Frankfurt DC (RTT < 20ms)

### 7.13.2 13.2 Scaling Numbers

Scale Milestones:

1M pages indexed (startup search)

- 1 index server, no sharding needed
- Single-machine crawler
- In-memory inverted index (~1 GB)

1B pages (medium-scale, e.g., DuckDuckGo)

- 100 index shards, 300 servers
- 500 crawler workers
- Index size: ~1 TB

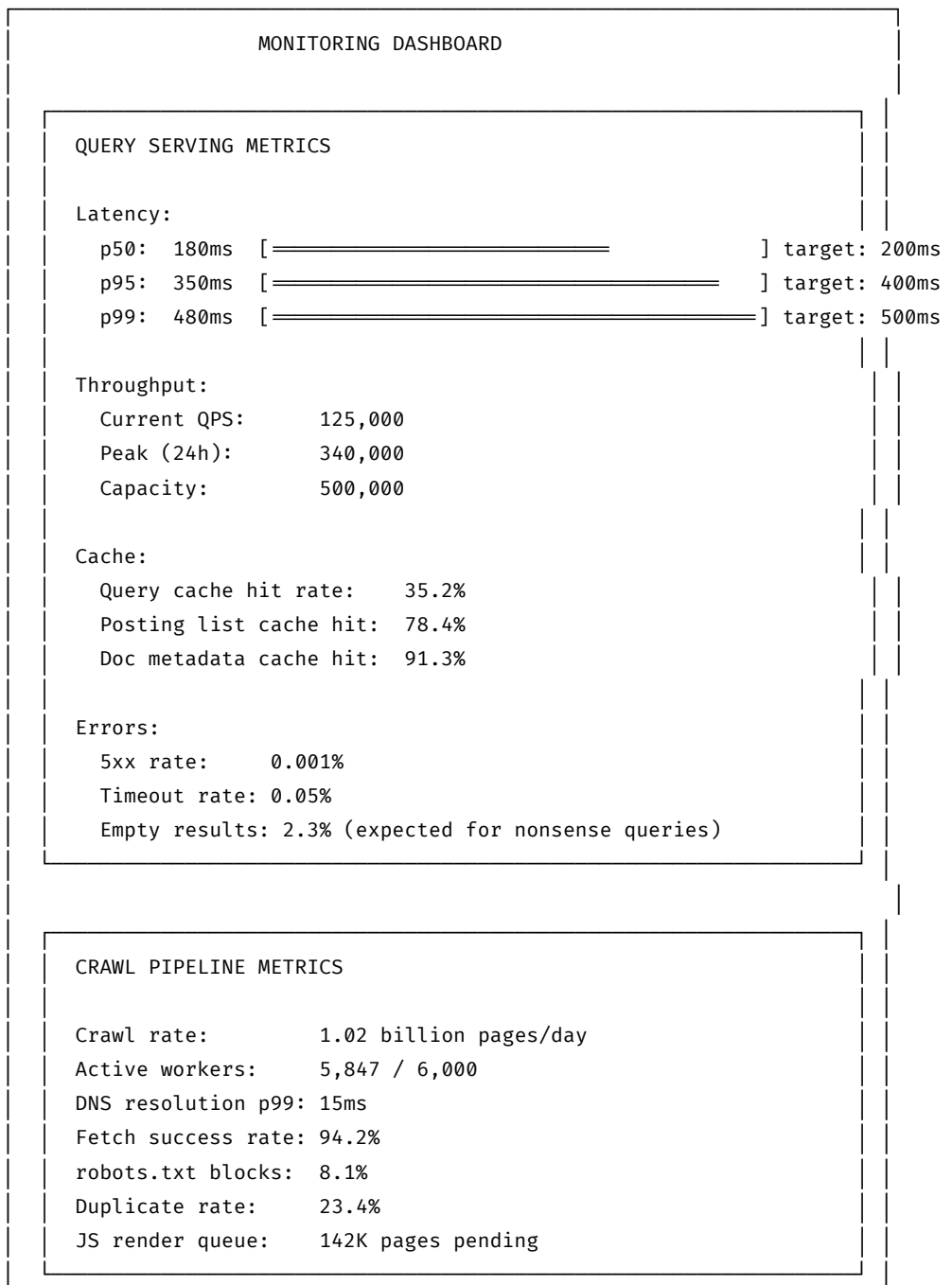
100B pages (Google-scale)

- 10,000 index shards, 30,000 servers
- 6,000+ crawler workers
- Index size: ~80 TB (compressed)
- 20+ data centers globally
- 115K QPS sustained

Growth Handling:

- Web grows ~10% per year
- Add index shards proportionally
- Re-partition annually or when shards exceed target size
- Linear cost scaling: 2x pages = ~2x infrastructure

## 7.14 14. Monitoring and Observability



**INDEX HEALTH METRICS**

## Index freshness:

Median document age: 3.2 days  
News document age: 47 minutes  
Real-time index size: 8.7M documents

Index size: 82.4 TB (compressed)  
Total documents: 101.2 billion  
Unique terms: 9.8 billion  
Last full rebuild: 6 hours ago  
Shard health: 9,997 / 10,000 healthy

**QUALITY METRICS**

Click-through rate (CTR) on result #1: 31.2%  
Mean Reciprocal Rank (MRR): 0.68  
NDCG@10: 0.74  
Query abandonment rate: 12.1%  
Spell correction trigger rate: 8.3%  
Autocomplete acceptance rate: 42.7%

## Alerting Thresholds:

CRITICAL: p99 latency > 1s, error rate > 0.1%, shard down > 5  
WARNING: p99 latency > 500ms, cache hit < 25%, crawl rate < 800M  
INFO: Index rebuild started/completed, new shard deployed

## 7.15 15. Trade-offs and Design Decisions

### 7.15.1 15.1 Key Design Decisions

| Decision            | Option A (Chosen)                                                                                       | Option B                                                                                              |
|---------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Index Partitioning  | Document-partitioned<br>+ Uniform load<br>+ Easy doc updates<br>- High fan-out                          | Term-partitioned<br>+ Lower fan-out for single-term query<br>- Hot term shards<br>- Cross-shard joins |
| First-Pass Ranking  | BM25<br>+ TF saturation<br>+ Length norm<br>+ Proven at scale                                           | TF-IDF<br>+ Simpler<br>- No saturation<br>- Length bias                                               |
| Re-Ranking Model    | LambdaMART / GBDT<br>+ Fast inference<br>+ Interpretable<br>- Feature engineering (Trend: DNNs winning) | Deep Neural Net<br>+ Better accuracy<br>- Slower inference<br>- Black box<br>+ End-to-end             |
| Crawl Strategy      | BFS with priority<br>+ Broad coverage<br>+ Finds important pages first                                  | DFS<br>+ Deep site coverage<br>- Misses breadth<br>- Can get trapped                                  |
| Index Update        | Periodic rebuild + real-time overlay<br>+ Simpler, cheaper<br>+ Optimized layout<br>- Slight staleness  | Real-time indexing (fully incremental)<br>+ Always fresh<br>- Complex<br>- Index fragmentation        |
| Content Store       | Distributed FS<br>+ Low latency reads<br>+ High throughput<br>- Operational cost                        | Object Store (S3)<br>+ Cheaper storage<br>+ Simpler ops<br>- Higher latency                           |
| Posting Compression | PForDelta<br>+ Better compression<br>+ SIMD-friendly<br>- More complex<br>- Block-based                 | VByte<br>+ Simpler code<br>+ Decent compression<br>- Slower for dense posting lists                   |

### 7.15.2 15.2 Freshness vs Completeness Trade-off

The fundamental tension: we cannot crawl 100B pages in real-time.

Solution: Priority-based crawl scheduling

| Page Category          | Re-crawl Interval  | % of Crawl |
|------------------------|--------------------|------------|
| Breaking news sites    | Every 5-15 minutes | 5%         |
| Major news portals     | Every 1-2 hours    | 10%        |
| Popular sites (top 1M) | Every 1-3 days     | 25%        |
| Regular content        | Every 1-4 weeks    | 40%        |
| Long-tail / archive    | Every 1-6 months   | 20%        |

Adaptive re-crawl: Track how often a page changes.

If change\_rate is high -> increase crawl frequency

If page hasn't changed in 3 crawls -> decrease frequency

### 7.15.3 15.3 Relevance vs Speed Trade-off

More accurate ranking requires more computation:

Fast path (~50ms):

BM25 only, Tier 1 index, top-100 results

Quality: Good for popular queries

Standard path (~200ms):

BM25 + ML re-ranking, Tier 1+2, top-1000 candidates

Quality: Good for most queries

Deep path (~500ms):

BM25 + ML re-ranking, all tiers, query expansion

Quality: Best, for rare/ambiguous queries

Decision: Route based on query characteristics

- Common queries (seen before) -> fast path + cache
- Normal queries -> standard path
- Rare / long queries -> deep path

## 7.16 16. Interview Deep-Dive Questions

### 7.16.1 Q1: How does PageRank work and how do you compute it at scale?

**Answer:** PageRank models a “random surfer” who follows links randomly. The probability of the surfer landing on a page is its PageRank. The formula is iterative:  $PR(A) = (1-d)/N + d * \sum (PR(B)/L(B))$  for all pages B linking to A, where  $d=0.85$  is the damping factor.

At scale (100B pages), we use MapReduce. Each mapper reads a page and its outlinks, distributing the page's PageRank equally among its outlinks. Each reducer sums contributions for a target page

and applies the damping formula. This requires ~50-100 iterations to converge, with each iteration processing ~3 TB of data. On 10K machines, one iteration takes ~30 minutes, so full computation takes ~25 hours. We recompute weekly and store results in the document metadata.

Dead ends (pages with no outlinks) are handled by redistributing their PageRank mass equally across all pages. Spider traps are handled by the damping factor, which ensures 15% probability of jumping to a random page.

---

### 7.16.2 Q2: How do you handle duplicate or near-duplicate content?

**Answer:** We use a multi-layered approach:

1. **URL-level dedup:** Normalize URLs (lowercase, remove fragments, sort parameters) and check against a Bloom filter (50B entries, 0.01% FPR, ~60 GB memory).
2. **Content-level exact dedup:** Compute a hash (e.g., MD5 or SHA-256) of the extracted text. Store hashes in a lookup table. Identical hashes = duplicate.
3. **Near-duplicate detection (SimHash):** Compute a 64-bit SimHash fingerprint. Two documents with Hamming distance  $\leq 3$  are near-duplicates. This catches pages that differ only in headers/footers/ads.
4. **Canonical URL resolution:** Respect the `<link rel="canonical">` tag, which tells us the preferred URL for duplicate content.

When duplicates are found, we keep only the canonical (highest PageRank) version in the index and merge link signals from all copies.

---

### 7.16.3 Q3: How do you keep the index fresh?

**Answer:** We use a two-tier approach:

1. **Main index:** Rebuilt every few hours via batch MapReduce pipeline. Contains 100B documents. Optimized for query performance (sorted, compressed, skip pointers).
2. **Real-time index overlay:** A small in-memory index (~10M documents) that is updated within minutes of crawl. When a query is processed, results from both indexes are merged by the query coordinator.

The crawler uses adaptive scheduling: pages that change frequently (detected via HTTP Last-Modified or content hash comparison) are crawled more often. News sites are crawled every 5-15 minutes, while static content may only be crawled monthly.

We also support push-based indexing via sitemaps (sitemap.xml) and the IndexNow protocol, where publishers notify us of changes.

---

### 7.16.4 Q4: How do you handle multi-word queries and phrase matching?

**Answer:** Multi-word queries require intersecting posting lists:

1. **AND semantics:** For “distributed consensus”, fetch posting lists for both terms and intersect them (documents containing both). We use skip pointers for efficient intersection of sorted posting lists.

2. **Phrase matching:** For “distributed consensus” in quotes, we need the terms to appear adjacent. We store term positions in the posting list. After intersecting doc\_ids, we check if  $\text{position}(\text{consensus}) = \text{position}(\text{distributed}) + 1$ .
  3. **Proximity scoring:** Even without quotes, terms appearing closer together in a document score higher. The position data enables this.
  4. **Optimization:** We process the rarest term first (smallest posting list) to minimize the number of documents we need to check for the other terms.
- 

#### 7.16.5 Q5: How do you prevent search result manipulation (SEO spam)?

**Answer:** Multiple defensive layers:

1. **Link spam detection:** Detect link farms (clusters of low-quality sites linking to each other). Use graph analysis to identify unnatural link patterns. Discount links from known spam domains.
  2. **Content quality signals:** ML models trained on human-rated examples to score content quality. Features include: text-to-HTML ratio, keyword stuffing density, grammar quality, E-E-A-T signals (experience, expertise, authoritativeness, trustworthiness).
  3. **Cloaking detection:** Crawl pages from different IPs and User-Agents. If the content served to the crawler differs from what users see, penalize the page.
  4. **Click signal analysis:** If users consistently bounce back quickly from a result (pogo-sticking), it's a signal of low quality. Incorporate dwell time and click-through patterns into ranking.
  5. **Manual actions:** A team of quality raters reviews flagged sites. Severe spam gets manually demoted or removed from the index.
- 

#### 7.16.6 Q6: How do you scale the scatter-gather query architecture?

**Answer:** Key strategies:

1. **Tiered index:** Only search the top-tier (1B most important pages = 100 shards) for most queries. This reduces fan-out from 10,000 to 100 shards. 95% of queries are satisfied by Tier 1 alone.
  2. **Replication:** Each shard has 3+ replicas. Query coordinator randomly picks a replica, distributing load. During peak, temporarily add more replicas.
  3. **Caching:** 30-40% of queries are cache hits, completely bypassing the scatter-gather. Posting list caching on index servers reduces SSD reads.
  4. **Deadline propagation:** Each query has a 200ms deadline. If a shard is slow, its results are excluded. The coordinator returns results from the shards that responded in time.
  5. **Request hedging:** For tail latency, send the same request to 2 replicas and use whichever responds first. This converts p99 latency to roughly  $p99^2$ .
- 

#### 7.16.7 Q7: How would you design the autocomplete system?

**Answer:** The autocomplete system needs < 50ms latency (feels instant).

**Data:** Aggregate query logs to find the most popular queries and their frequencies. Update every few hours.



**Data structure:** Build a trie where each node stores the top-8 completions by popularity. Size: ~10 GB, fits entirely in memory.

**Lookup:** User types “how to des” -> traverse trie to “how\_to\_des” node -> return pre-computed top-8 completions.

**Personalization:** Boost suggestions from user’s recent search history (stored in session/cookie). Mix personal suggestions with global popular ones.

**Freshness:** For trending topics, merge a small “trending trie” (updated every 15 minutes from real-time query stream) with the main trie.

**Sharding:** Shard by first 2 characters of query prefix. “ho” queries go to one server, “di” to another. Each shard is replicated.

---

### 7.16.8 Q8: How do you handle different languages and internationalization?

**Answer:** Each language requires its own processing pipeline:

1. **Language detection:** Classify document language using n-gram models (e.g., CLD2). Multi-language pages are indexed under the primary language.
  2. **Language-specific tokenization:** Chinese/Japanese need word segmentation (no spaces). Arabic/Hebrew are right-to-left. German has compound words that need compounding.
  3. **Language-specific stemming:** English uses Porter stemmer, other languages use language-specific stemmers (e.g., Snowball stemmers).
  4. **Index per language:** Maintain separate index shards per language or language group. Query routing uses the user’s language preference.
  5. **Cross-language search:** Optionally translate queries or use multilingual embeddings to find results in other languages.
- 

### 7.16.9 Q9: What is the role of the link graph in ranking?

**Answer:** The link graph serves multiple purposes beyond PageRank:

1. **PageRank:** Global authority score based on incoming link count and quality.
2. **Anchor text:** The text of links pointing to a page describes what the page is about. “Click here for Raft paper” tells us the target is about Raft.
3. **Domain authority:** Aggregate PageRank of all pages on a domain.
4. **Spam detection:** Unnatural link patterns indicate manipulation.
5. **Topic-sensitive PageRank:** Compute PageRank biased toward specific topic categories, giving different authority scores for different query intents.

The link graph is stored as an adjacency list (source -> [targets]) and reversed adjacency list (target -> [sources]). Total size: ~2 TB for 100B pages with ~20 average outlinks each.

---

### 7.16.10 Q10: How do you handle query intent classification?

**Answer:** Queries are classified into three main intents:

1. **Navigational:** User wants a specific site. E.g., “facebook login” -> return facebook.com as top result. Detected by brand name matching and high click-through on a single result.

2. **Informational:** User wants to learn something. E.g., “how does TCP work” -> return educational articles, knowledge panels. Most common type (~60%).
3. **Transactional:** User wants to do something. E.g., “buy laptop” -> return shopping results, product listings.

Classification uses an ML model (gradient-boosted trees or neural network) trained on labeled query-intent pairs. Features include: query length, presence of action verbs, entity detection, and historical click patterns.

Intent affects ranking: navigational queries boost the target domain, informational queries boost authoritative educational content, and transactional queries boost product pages with high ratings.

---

### 7.16.11 Q11: How do you build knowledge panels?

**Answer:** Knowledge panels display structured information for entity queries (e.g., “Albert Einstein”, “Python programming language”).

1. **Entity recognition:** Identify if the query matches a known entity in a knowledge base (e.g., Wikidata, Freebase). Use entity linking to disambiguate (e.g., “Python” -> programming language vs. snake).
2. **Structured data:** Extract facts from the knowledge base (birth date, occupation, related entities). Also ingest schema.org structured data from web pages.
3. **Panel assembly:** Template-based rendering: person template shows photo, birth date, occupation; company template shows CEO, stock price, headquarters.
4. **Freshness:** Some facts (stock price, weather) need real-time feeds. Others (birth date) are static and cached.

---

### 7.16.12 Q12: How do you handle image and video search?

**Answer:** Multimedia search adds two main subsystems:

1. **Image indexing:** Extract images from crawled pages. Index by: surrounding text, alt text, filename, page title. Also compute visual features using CNNs (ResNet/CLIP) for visual similarity search. Store thumbnails in a CDN.
2. **Video indexing:** Crawl video platforms (YouTube, Vimeo). Index by: title, description, captions/transcripts (ASR-generated), tags. Extract key frames for visual matching.
3. **Blended results:** For queries with multimedia intent (e.g., “cat photos”), the result compiler inserts image/video carousels into web search results.
4. **Reverse image search:** Given an image, find visually similar images. Uses perceptual hashing (pHash) for exact matches and deep embeddings (CLIP) for semantic similarity. Index embeddings in an approximate nearest neighbor (ANN) structure like HNSW or ScaNN.

---

### 7.16.13 Q13: What is the index serving data path for a single query?

**Answer:** Detailed timing breakdown for a query “distributed consensus”:

| Time (ms) | Action |
|-----------|--------|
|-----------|--------|

---

|         |                                                                                                  |
|---------|--------------------------------------------------------------------------------------------------|
| 0       | User sends query                                                                                 |
| 5       | DNS + TCP + TLS to nearest DC                                                                    |
| 10      | Load balancer routes to query frontend                                                           |
| 15      | Query parser: tokenize, stem, spell-check                                                        |
| 20      | Check query result cache -> MISS                                                                 |
| 25      | Scatter: send to 100 Tier-1 index shards                                                         |
| 30-80   | Each shard: lookup posting lists, intersect, compute BM25<br>Return local top-100 to coordinator |
| 85      | Gather: merge 100 * 100 = 10K candidates -> global top-1000                                      |
| 90-140  | ML re-ranker scores top-1000 candidates                                                          |
| 145     | Select top-10 for page 1                                                                         |
| 150-170 | Parallel: fetch doc metadata + generate snippets                                                 |
| 175     | Build knowledge panel (if entity detected)                                                       |
| 180     | Assemble final response JSON                                                                     |
| 185     | Send response to user                                                                            |
| 190     | User receives results                                                                            |

Total: ~190ms end-to-end (well within 200ms p50 target).

---

#### 7.16.14 Q14: How do you handle index compaction and garbage collection?

**Answer:** Over time, the index accumulates deleted/updated documents that waste space and slow queries.

1. **Tombstones:** When a document is removed or updated, mark the old doc\_id as deleted (bitmap of deleted doc\_ids per shard). During query processing, filter out deleted doc\_ids from results.
  2. **Periodic compaction:** During full index rebuild (every few hours), deleted documents are physically removed. Posting lists are re-sorted and re-compressed without gaps.
  3. **Segment merging:** Similar to Lucene's merge policy. Small index segments (from recent updates) are periodically merged into larger segments for better query performance (fewer segments to search).
  4. **Cost:** Compaction is done during off-peak hours and uses separate I/O bandwidth from query serving to avoid latency impact.
- 

#### 7.16.15 Q15: How would you design the system differently for a vertical search engine (e.g., e-commerce)?

**Answer:** Key differences for e-commerce search (like Amazon's A9):

1. **Structured data:** Products have structured attributes (price, brand, category, ratings). Use faceted search (filter by price range, brand, etc.) in addition to full-text search.
  2. **Ranking signals:** Replace PageRank with conversion rate, sales velocity, seller reputation, profit margin. Freshness matters less; availability and pricing matter more.
  3. **Real-time inventory:** Unlike web search, product availability changes in real-time. The index must support near-real-time updates (seconds, not hours).
-

4. **Smaller scale:** Millions of products (not billions of pages). Single-tier index fits on fewer machines. Can afford more expensive per-document processing.
  5. **Personalization:** E-commerce search is heavily personalized. User purchase history, browsing behavior, and demographic features significantly affect ranking. Web search is less personalized.
  6. **Merchandising:** Business rules override pure relevance ranking (e.g., boost sponsored products, new arrivals, clearance items). Web search tries to avoid this.
-

## 7.17 17. Summary: Key Takeaways for Interviews

### INTERVIEW CHEAT SHEET

#### Three Pipelines:

1. Crawl: URL Frontier -> Crawlers -> Content Store
2. Index: Content -> Parse -> Tokenize -> Inverted Index
3. Query: Parse -> Scatter-Gather -> Rank -> Compile Results

#### Key Numbers:

- 100B pages, 10B unique terms, 80 TB compressed index
- 115K QPS, < 200ms p50, < 500ms p99
- 10,000 index shards, 30,000 index servers
- 1B pages crawled per day, 6,000 crawler workers

#### Core Data Structures:

- Inverted index with posting lists (delta-encoded, compressed)
- Trie for autocomplete
- Bloom filter for URL dedup
- SimHash for content dedup
- Priority queue for URL frontier

#### Key Algorithms:

- BM25 for first-pass ranking
- PageRank for authority scoring
- LambdaMART/DNN for re-ranking
- MapReduce for index construction and PageRank computation

#### Architecture Patterns:

- Document-partitioned index (not term-partitioned)
- Scatter-gather query execution
- Tiered index (hot/warm/cold)
- Real-time index overlay for freshness
- Two-phase ranking (fast retrieval + ML re-ranking)

#### Stand-Out Points for Senior Engineers:

- Discuss index compression trade-offs (VByte vs PForDelta)
- Explain BM25 formula and why it beats TF-IDF
- Describe tiered index optimization (95% queries on Tier 1)
- Mention request hedging for tail latency
- Discuss freshness vs completeness trade-off
- Explain how real-time overlay index enables news search

## 7.18 References and Further Reading

- Brin, S. & Page, L. (1998). "The Anatomy of a Large-Scale Hypertextual Web Search Engine" (original Google paper)

- Robertson, S. & Zaragoza, H. (2009). “The Probabilistic Relevance Framework: BM25 and Beyond”
- Dean, J. & Ghemawat, S. (2004). “MapReduce: Simplified Data Processing on Large Clusters”
- Ghemawat, S. et al. (2003). “The Google File System”
- Chang, F. et al. (2006). “Bigtable: A Distributed Storage System for Structured Data”
- Manning, C. et al. (2008). “Introduction to Information Retrieval” (Stanford NLP textbook)
- Zobel, J. & Moffat, A. (2006). “Inverted Files for Text Search Engines” (ACM Computing Surveys)
- Barroso, L. et al. (2013). “The Datacenter as a Computer” (warehouse-scale computing)

## Chapter 8

# Design a Distributed Cache (Redis / Memcached)

### 8.1 Table of Contents

1. Problem Statement
  2. Functional Requirements
  3. Non-Functional Requirements
  4. Back-of-Envelope Estimation
  5. API Design
  6. Data Model
  7. High-Level Architecture
  8. Deep Dive
    - 8.1 Hash Slot Architecture
    - 8.2 Eviction Policies
    - 8.3 Replication
    - 8.4 Cache Patterns
  9. Data Partitioning
  10. Caching Strategy
  11. Replication and Consistency
  12. Fault Tolerance
  13. Scalability
  14. Monitoring
  15. Trade-offs
  16. Interview Questions
- 

### 8.2 1. Problem Statement

Modern web applications rely heavily on databases to store and retrieve data. As traffic scales, databases become bottlenecks – disk I/O, query parsing, and index lookups all add latency. A distributed cache sits between the application and the database, storing frequently accessed data in memory to deliver sub-millisecond responses while shielding the database from excessive load.

**Core challenge:** Design a distributed, in-memory key-value store that provides sub-millisecond reads, supports millions of operations per second, handles node failures gracefully, and scales horizontally across a cluster of machines.

**Real-world systems:** Redis, Memcached, Amazon ElastiCache, Azure Cache for Redis, Google Memorystore.

**Why this matters:** - Database queries typically take 5-50ms; cache hits return in <1ms (50-100x speedup). - A single cache node can serve 100K+ operations/sec vs ~5K queries/sec for a database. - Caching reduces database load by 80-95% for read-heavy workloads. - Without caching, scaling to millions of users requires proportionally more database capacity, which is expensive and complex.

---

## 8.3 2. Functional Requirements

### 8.3.1 Core Operations

- **GET:** Retrieve value by key
- **SET:** Store key-value pair with optional TTL
- **DELETE:** Remove key-value pair
- **EXISTS:** Check if key exists without fetching value
- **EXPIRE:** Set or update TTL on existing key
- **TTL:** Query remaining time-to-live for a key
- **INCR/DECR:** Atomic increment/decrement of integer values
- **MGET/MSET:** Batch get/set for multiple keys

### 8.3.2 Eviction Policies

- **LRU (Least Recently Used):** Evict keys that have not been accessed recently.
- **LFU (Least Frequently Used):** Evict keys with the lowest access frequency.
- **Random:** Evict a randomly selected key.
- **TTL-based:** Evict keys closest to expiration.
- **No eviction:** Return errors when memory is full (useful for critical data).

### 8.3.3 Data Structures (Redis-style)

- **String:** Operations: GET, SET, INCR, APPEND; Use Case: Counters, simple caching
- **Hash:** Operations: HGET, HSET, HMGET, HGETALL; Use Case: Object storage, user profiles
- **List:** Operations: LPUSH, RPUSH, LPOP, RPOP, LRANGE; Use Case: Message queues, activity feeds
- **Set:** Operations: SADD, SREM, SMEMBERS, SINTER; Use Case: Tags, unique visitors
- **Sorted Set:** Operations: ZADD, ZRANGE, ZRANGEBYSCORE; Use Case: Leaderboards, rate limiters
- **HyperLogLog:** Operations: PFADD, PFCOUNT, PFMERGE; Use Case: Cardinality estimation
- **Bitmap:** Operations: SETBIT, GETBIT, BITCOUNT; Use Case: Feature flags, daily active users
- **Stream:** Operations: XADD, XREAD, XRANGE; Use Case: Event sourcing, message streams

### 8.3.4 Pub/Sub

- **SUBSCRIBE/PUBLISH:** Real-time message broadcasting to channels.
- **Pattern subscribe:** Subscribe using glob-style patterns (`news.*`).
- **Key-space notifications:** Get notified when keys are modified or expired.



### 8.3.5 Atomic Operations

- **Transactions (MULTI/EXEC):** Group commands for atomic execution.
- **Lua scripting:** Execute server-side scripts atomically.
- **Optimistic locking (WATCH):** Detect concurrent modifications.

### 8.3.6 Persistence Options

- **RDB:** Mechanism: Point-in-time snapshots to disk; Pros: Compact, fast recovery; Cons: Data loss between snapshots
- **AOF:** Mechanism: Append every write command to log; Pros: Minimal data loss; Cons: Larger files, slower restart
- **Hybrid:** Mechanism: RDB snapshot + AOF tail; Pros: Best of both worlds; Cons: More complex management
- **None:** Mechanism: Pure in-memory, no persistence; Pros: Maximum performance; Cons: All data lost on restart

### 8.3.7 Cluster Mode

- **Automatic sharding:** Data distributed across multiple nodes.
  - **Hash slots:** 16384 slots mapped to nodes for deterministic routing.
  - **Online resharding:** Migrate slots between nodes without downtime.
  - **Automatic failover:** Promote replicas when masters fail.
- 

## 8.4 3. Non-Functional Requirements

- **Latency (p50):** < 0.5 ms for GET/SET operations
  - **Latency (p99):** < 1 ms for GET/SET operations
  - **Throughput:** 100,000+ operations/sec per node
  - **Availability:** 99.99% uptime (< 52.6 minutes downtime/year)
  - **Durability:** Configurable: from none to fsync-every-write
  - **Data size per key:** Up to 512 MB (Redis), typically 1 KB - 1 MB
  - **Max keys per node:** Hundreds of millions (limited by RAM)
  - **Cluster size:** Up to 1000 nodes
  - **Failover time:** < 15 seconds for automatic failover
  - **Network:** Support for 10 Gbps+ network interfaces
  - **Replication lag:** < 1 ms under normal conditions
  - **Memory efficiency:** < 20% overhead vs raw data size
- 

## 8.5 4. Back-of-Envelope Estimation

### 8.5.1 Traffic Estimation

Total operations/sec: 10,000,000 (10M ops/sec)  
Read:Write ratio: 80:20  
Read operations/sec: 8,000,000  
Write operations/sec: 2,000,000  
  
Peak multiplier: 3x  
Peak operations/sec: 30,000,000

### 8.5.2 Storage Estimation

Average key size: 64 bytes  
Average value size: 1 KB (1,024 bytes)  
Total entry size: ~1.1 KB (key + value + metadata)  
  
Total unique keys: 10 billion (10B)  
Raw data: 10B x 1.1 KB = 11 TB  
Memory overhead (20%): 11 TB x 1.2 = ~13.2 TB  
Per-node memory: 128 GB RAM  
Nodes needed (data): 13.2 TB / 128 GB = ~104 nodes  
Rounded up: 110 nodes (with headroom)

### 8.5.3 Node Calculation

Per-node throughput: 100,000 ops/sec  
Total ops/sec needed: 10,000,000  
Nodes needed (throughput): 10M / 100K = 100 nodes  
Peak nodes needed: 30M / 100K = 300 nodes

Decision: 110 nodes for storage, ~300 for peak throughput  
-> Use 110 primary nodes + 220 replicas (2 replicas each)  
-> Replicas absorb read traffic, handling peak load  
-> Total cluster: 330 nodes

### 8.5.4 Network Estimation

Per operation bandwidth: 1.1 KB (avg)  
Total bandwidth: 10M x 1.1 KB = 11 GB/sec = 88 Gbps  
Per node (110 nodes): 88 Gbps / 110 = ~0.8 Gbps per node  
With replication (2x): ~1.6 Gbps per node for writes  
Network interface needed: 10 Gbps (sufficient)

### 8.5.5 Memory Breakdown Per Node (128 GB)

User data: 100 GB (78%)  
Hash table overhead: 10 GB (8%)  
Eviction metadata (LRU): 3 GB (2%)  
Replication buffers: 5 GB (4%)  
Client output buffers: 5 GB (4%)  
OS and process overhead: 5 GB (4%)  
Total: 128 GB (100%)

---

## 8.6 5. API Design

### 8.6.1 Data Operations

```
String operations
SET key value [EX seconds] [PX milliseconds] [NX|XX]
GET key
DEL key [key ...]
MSET key value [key value ...]
MGET key [key ...]
INCR key
INCRBY key increment
DECR key
DECRBY key decrement
APPEND key value
SETNX key value # SET if Not eXists (atomic)

Expiration
EXPIRE key seconds
PEXPIRE key milliseconds
TTL key # Remaining TTL in seconds
PTTL key # Remaining TTL in milliseconds
PERSIST key # Remove TTL

Hash operations
HSET key field value [field value ...]
HGET key field
HMGET key field [field ...]
HGETALL key
HDEL key field [field ...]
HINCRBY key field increment

List operations
LPUSH key element [element ...]
RPUSH key element [element ...]
LPOP key [count]
RPOP key [count]
LRANGE key start stop
LLEN key
BLPOP key [key ...] timeout # Blocking pop
```

```

Set operations
SADD key member [member ...]
SREM key member [member ...]
SMEMBERS key
SISMEMBER key member
SINTER key [key ...]
SUNION key [key ...]
SCARD key

Sorted Set operations
ZADD key [NX|XX] [GT|LT] score member [score member ...]
ZRANGE key min max [BYSCORE|BYLEX] [REV] [LIMIT offset count]
ZRANGEBYSCORE key min max [WITHSCORES] [LIMIT offset count]
ZRANK key member
ZSCORE key member
ZREM key member [member ...]
ZCARD key

Stream operations
XADD key [MAXLEN|MINID] ID field value [field value ...]
XREAD [COUNT count] [BLOCK milliseconds] STREAMS key [key ...] ID [ID ...]
XRANGE key start end [COUNT count]
XLEN key

```

## 8.6.2 Cluster Management

```

CLUSTER INFO # Show cluster state
CLUSTER NODES # List all nodes in cluster
CLUSTER MEET ip port # Add node to cluster
CLUSTER ADDSLOTS slot [slot ...]
CLUSTER DELSLOTS slot [slot ...]
CLUSTER SETSLOT slot MIGRATING node-id
CLUSTER SETSLOT slot IMPORTING node-id
CLUSTER SETSLOT slot NODE node-id
CLUSTER REPLICATE node-id # Make current node replica of given master
CLUSTER FAILOVER [FORCE|TAKEOVER]
CLUSTER RESET [HARD|SOFT]
CLUSTER SLOTS # Slot-to-node mapping
CLUSTER KEYSLOT key # Which slot a key maps to

```

## 8.6.3 Pub/Sub

```

SUBSCRIBE channel [channel ...]
UNSUBSCRIBE [channel [channel ...]]
PUBLISH channel message
PSUBSCRIBE pattern [pattern ...] # Pattern subscribe
PUNSUBSCRIBE [pattern [pattern ...]]

```

## 8.6.4 Transactions and Scripting

```
MULTI # Start transaction
EXEC # Execute transaction
DISCARD # Abort transaction
WATCH key [key ...] # Optimistic lock

EVAL script numkeys [key ...] [arg ...] # Run Lua script
EVALSHA sha1 numkeys [key ...] [arg ...] # Run cached script
SCRIPT LOAD script
SCRIPT EXISTS sha1 [sha1 ...]
```

## 8.6.5 Administrative

```
INFO [section] # Server info and stats
CONFIG GET parameter
CONFIG SET parameter value
DBSIZE # Number of keys
FLUSHDB [ASYNC] # Delete all keys in current DB
SLOWLOG GET [count] # Slow query log
CLIENT LIST # Connected clients
MEMORY USAGE key # Memory used by key
DEBUG SLEEP seconds # (development only)
```

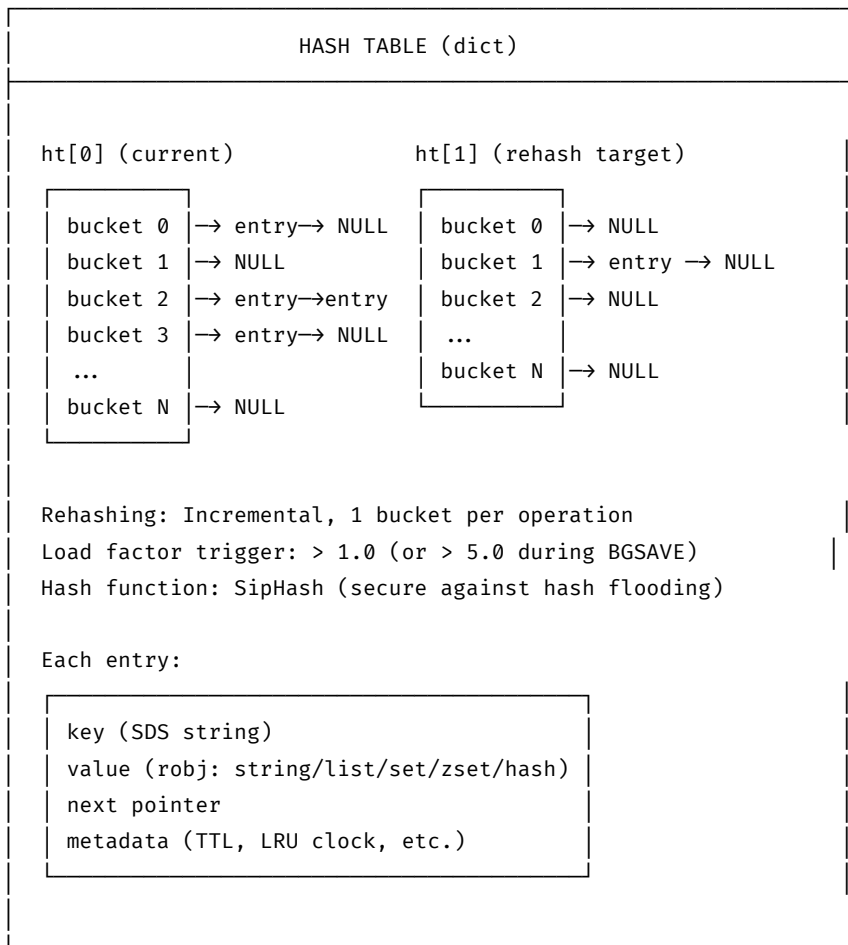
---

## 8.7 6. Data Model

### 8.7.1 Core Data Structures (Internal Implementation)

#### 8.7.1.1 Hash Table (Primary Index)

The main key-space is a hash table using chained hashing with incremental rehashing.



### 8.7.1.2 Doubly Linked List (LRU Eviction)

Redis uses an approximate LRU instead of a true LRU linked list to save memory. Each key stores a 24-bit LRU clock (last access time in seconds, wrapping ~194 days).

## APPROXIMATE LRU MECHANISM

Each key stores: `lru_clock` (24 bits)

When eviction is needed:

1. Sample N random keys (default N=5)
2. Compute idle time = `current_clock` - `key.lru_clock`
3. Evict the key with the highest idle time

Eviction pool (optimization in Redis 3.0+):

|    |    |    |    |    |    |    |    |    |     |     |     |     |     |     |     |
|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|
| k1 | k2 | k3 | k4 | k5 | k6 | k7 | k8 | k9 | k10 | k11 | k12 | k13 | k14 | k15 | k16 |
|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|

^

| least idle

most idle |

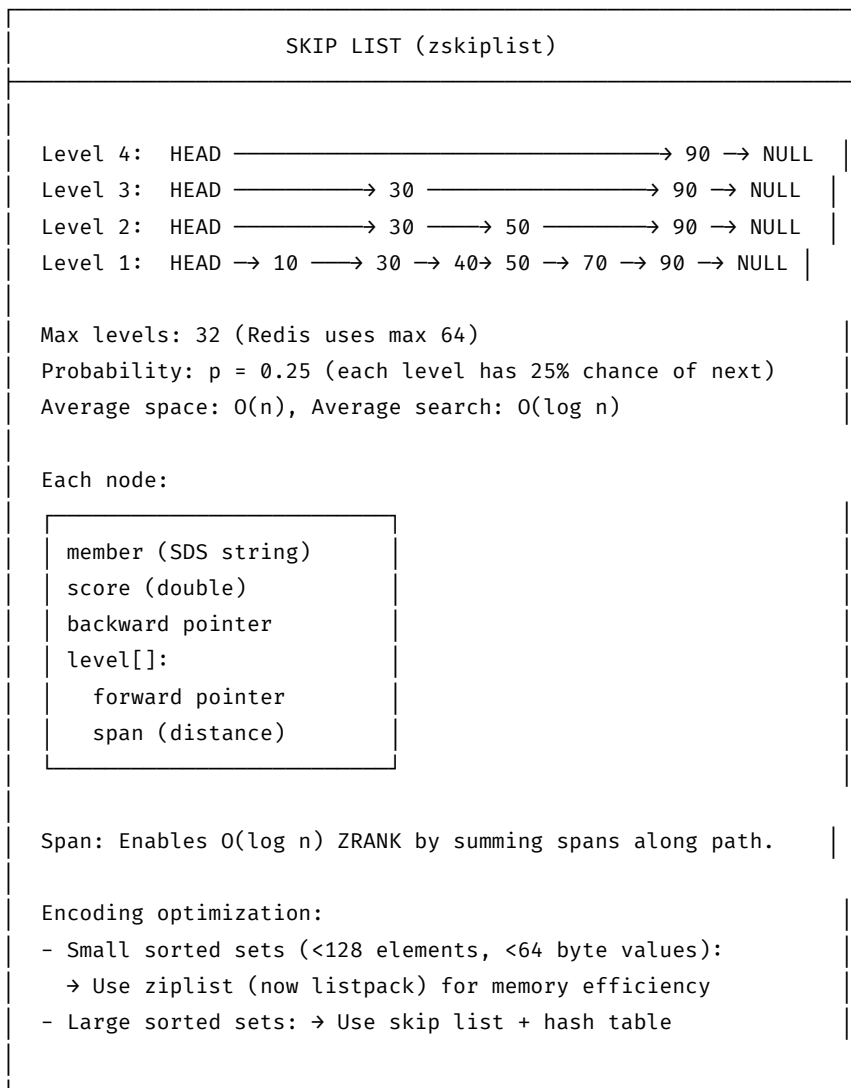
Sorted by idle time; best eviction candidate at the right.

Pool size: 16 entries; filled across multiple eviction cycles.

True LRU vs Approximate LRU:

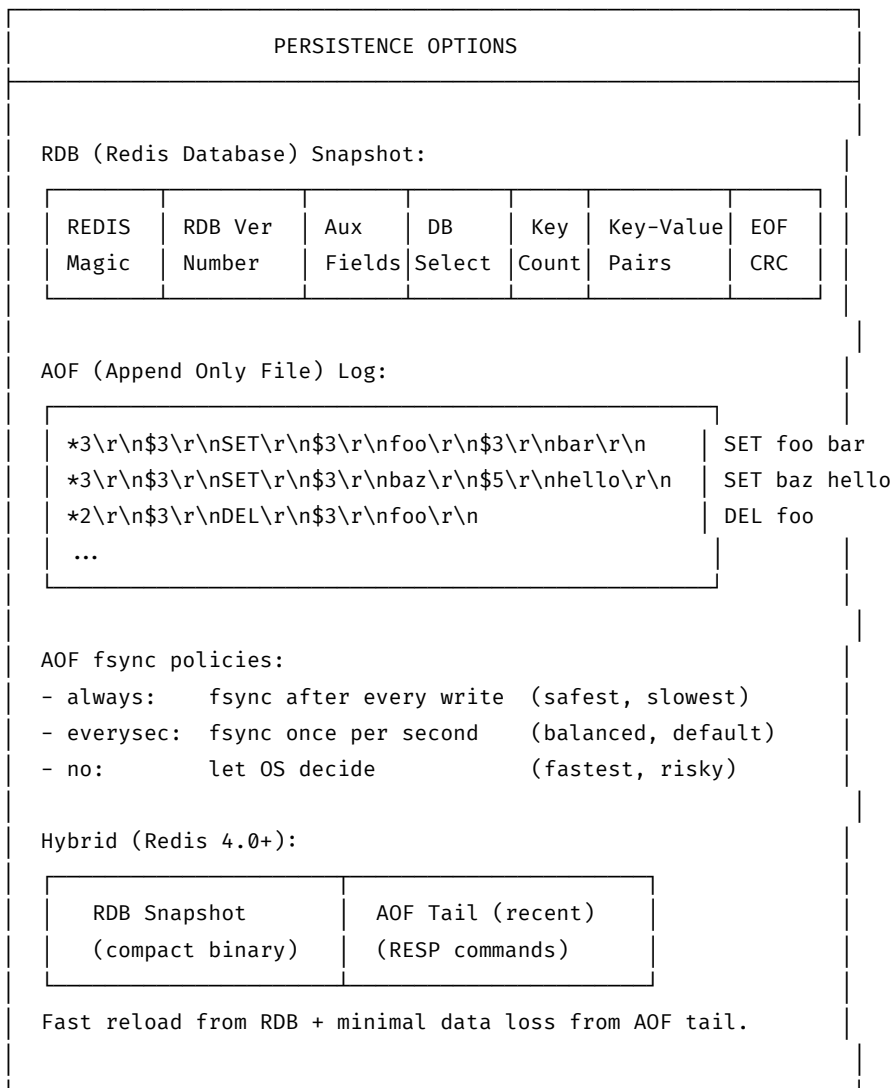
- True LRU:  $O(1)$  eviction but needs doubly-linked list  
+ hash map = ~80 bytes overhead per key
- Approx LRU: 24 bits (3 bytes) per key, very close to true  
LRU with sample size  $\geq 10$

### 8.7.1.3 Skip List (Sorted Sets)

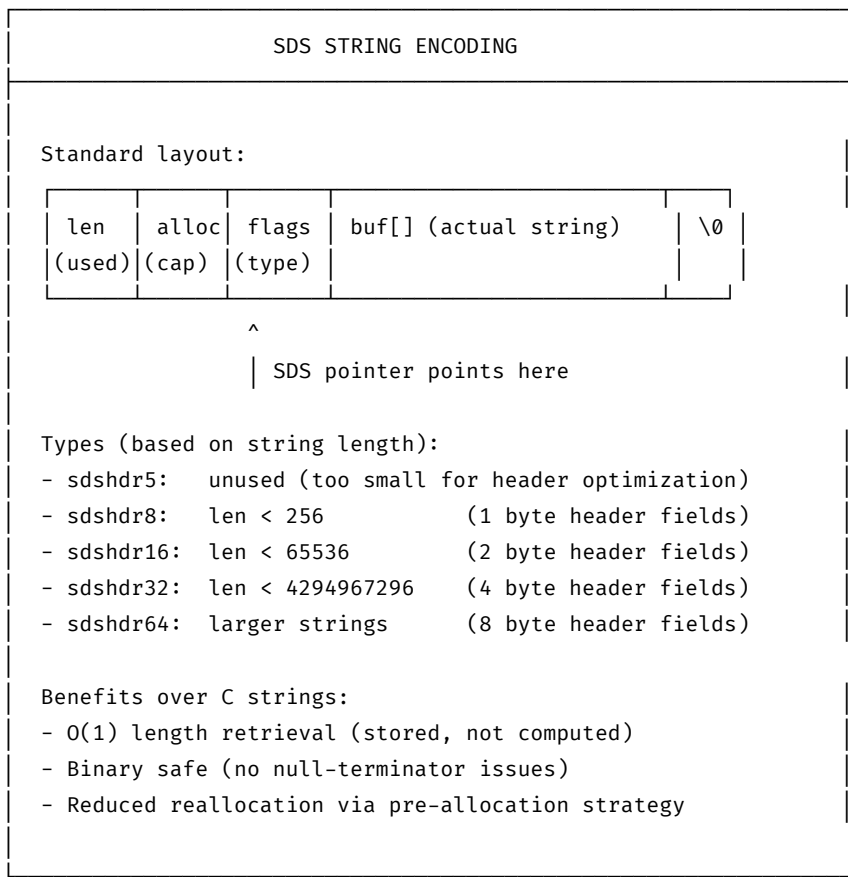




### 8.7.1.4 Persistence Data Layout

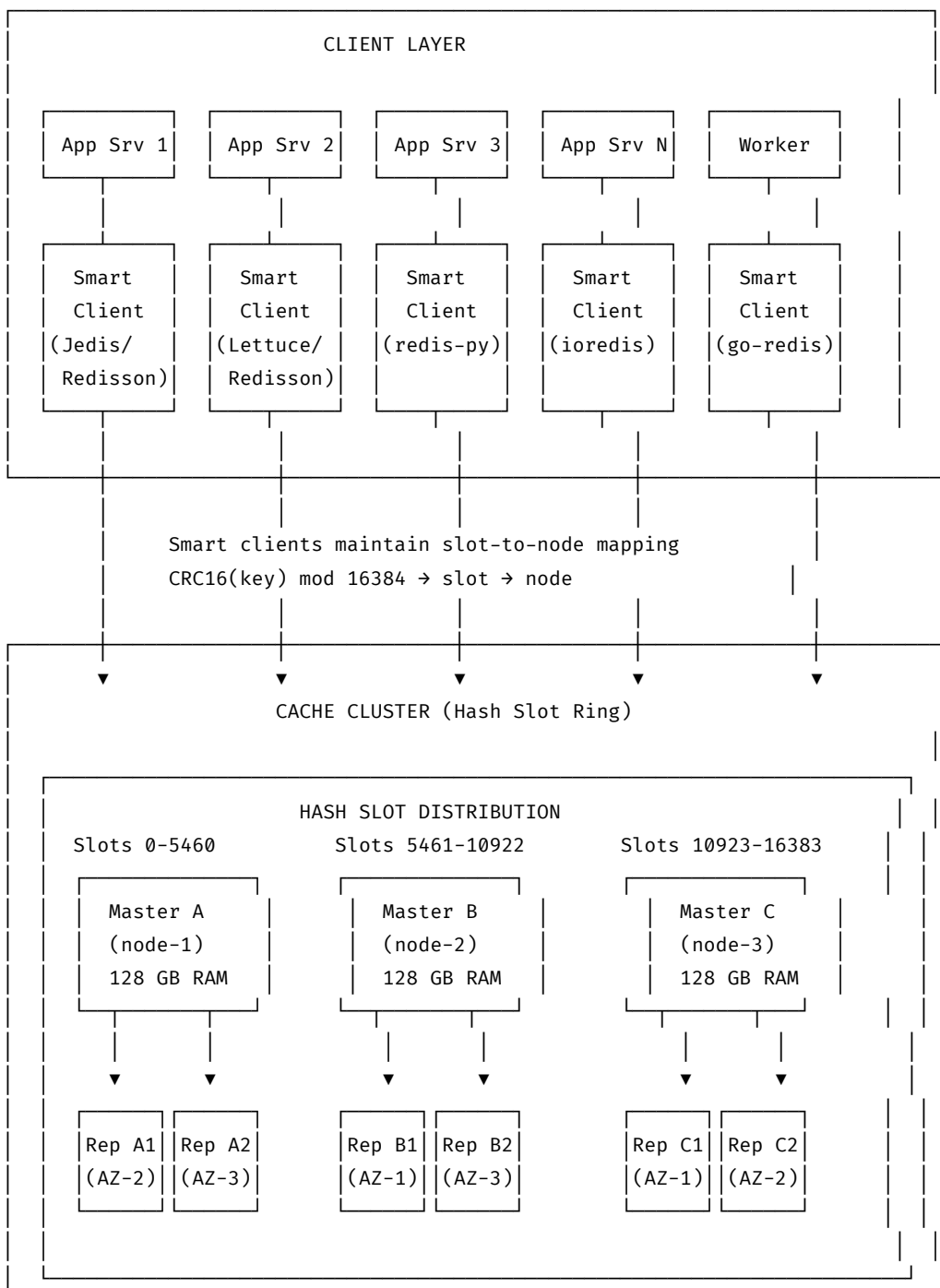


### 8.7.1.5 SDS (Simple Dynamic String)



## 8.8 7. High-Level Architecture

### 8.8.1 System Overview



GOSSIP PROTOCOL (Cluster Bus)

Port: data\_port + 10000 (e.g., 6379 → 16379)

Protocol: Binary, custom Redis Cluster Bus

Messages: PING, PONG, MEET, FAIL, PUBLISH

Frequency: Every node pings 1 random node/sec

Failure detection: pfail → fail (quorum based)

Node A ←—PING/PONG—→ Node B

Node B ←—PING/PONG—→ Node C

Node C ←—PING/PONG—→ Node A

## 8.8.2 Request Flow

### REQUEST LIFECYCLE

1. Client sends: SET user:1001 '{"name":"Alice"}' EX 3600

2. Smart Client:

slot = CRC16("user:1001") mod 16384 = 7438

node = slot\_map[7438] → Master B (node-2)

3. Connection to Master B (multiplexed TCP):

Client —RESP3—→ Master B

4. Master B processes command:

a. Parse RESP protocol

b. Look up command table → SET handler

c. Check memory limit (maxmemory)

d. If over limit → run eviction policy

e. Insert into hash table

f. Set TTL in expires dict

g. Append to AOF buffer (if enabled)

h. Propagate to replicas (replication stream)

i. Return "+OK\r\n" to client

5. If MOVED (wrong node):

Master B → "-MOVED 7438 192.168.1.2:6379"

Client updates slot map

Client retries to correct node

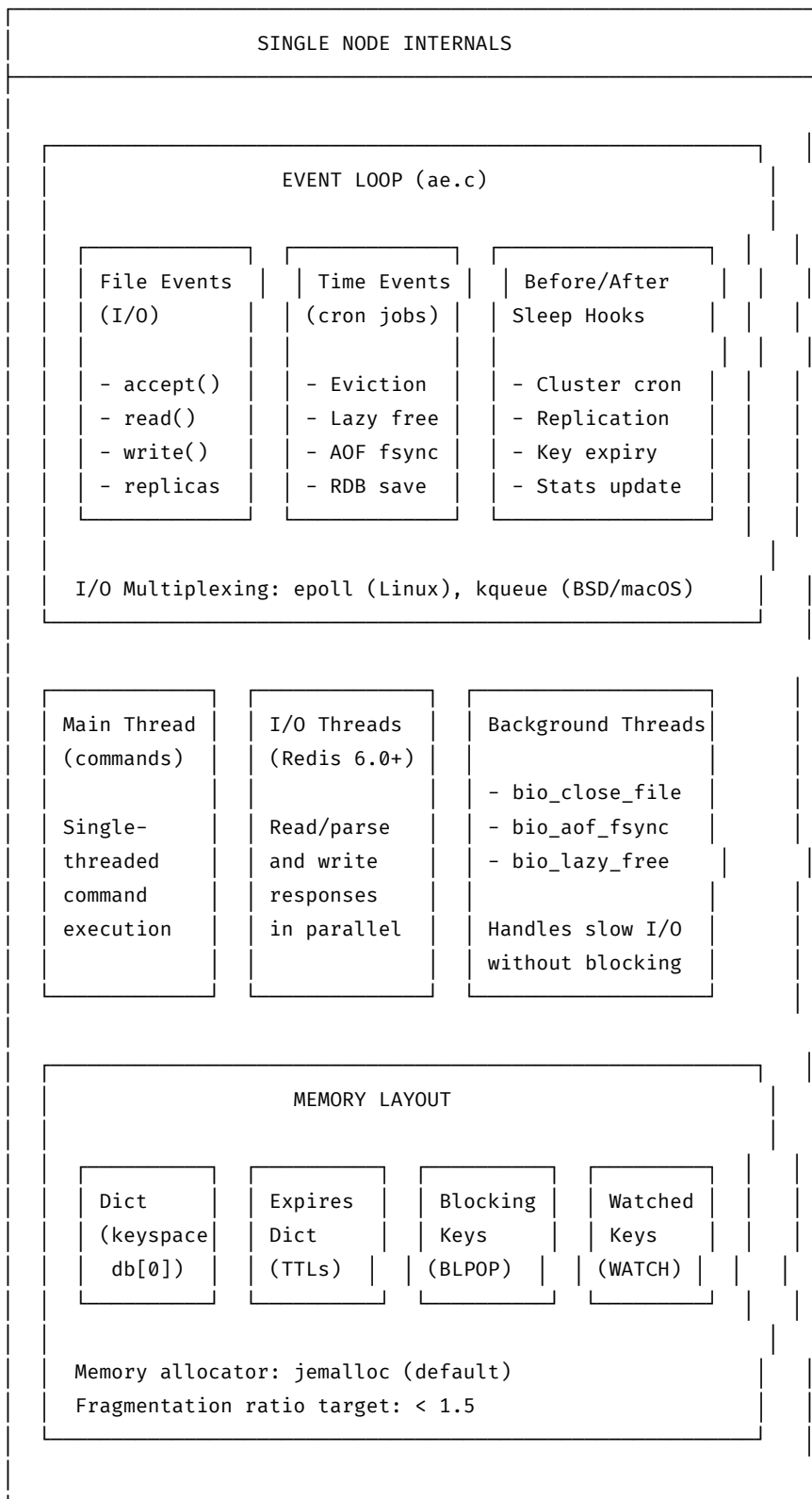
6. If ASK (slot migrating):

Master B → "-ASK 7438 192.168.1.3:6379"

Client sends ASKING + command to target node

(does NOT update slot map permanently)

### 8.8.3 Single Node Internal Architecture



## 8.9 8. Deep Dive

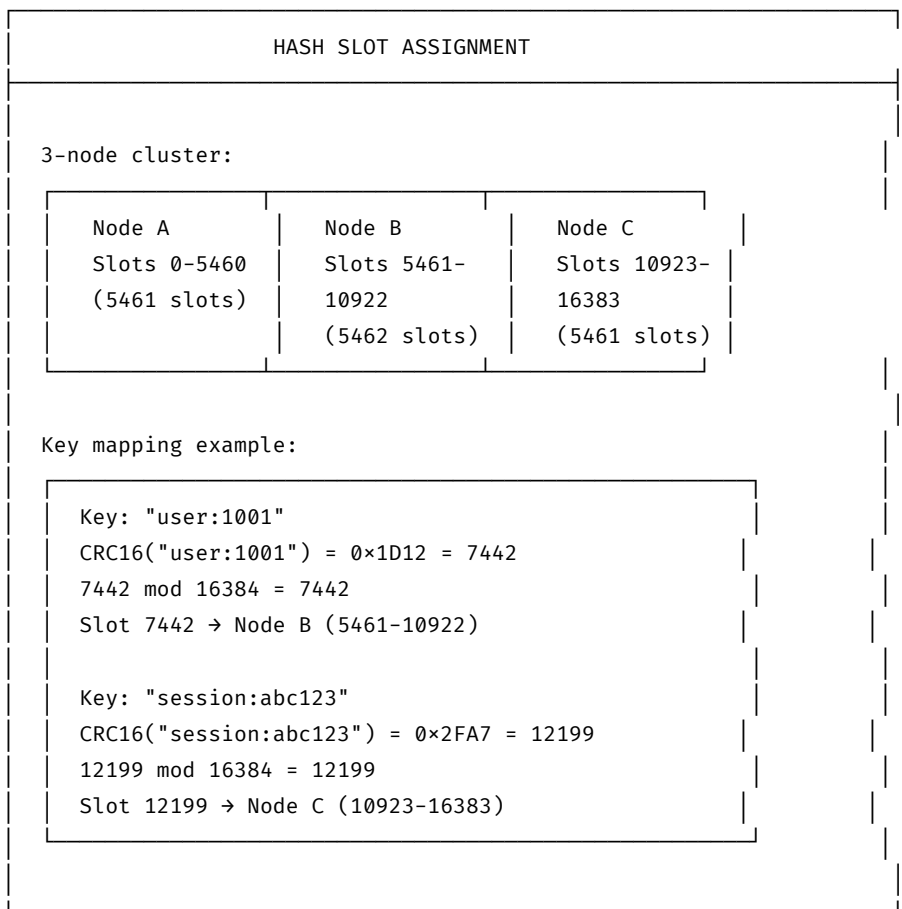
### 8.9.1 8.1 Hash Slot Architecture

Redis Cluster divides the keyspace into **16384 hash slots**. Each key is mapped to a slot using  $\text{CRC16}(\text{key}) \bmod 16384$ . Slots are then assigned to master nodes.

#### 8.9.1.1 Why 16384 Slots?

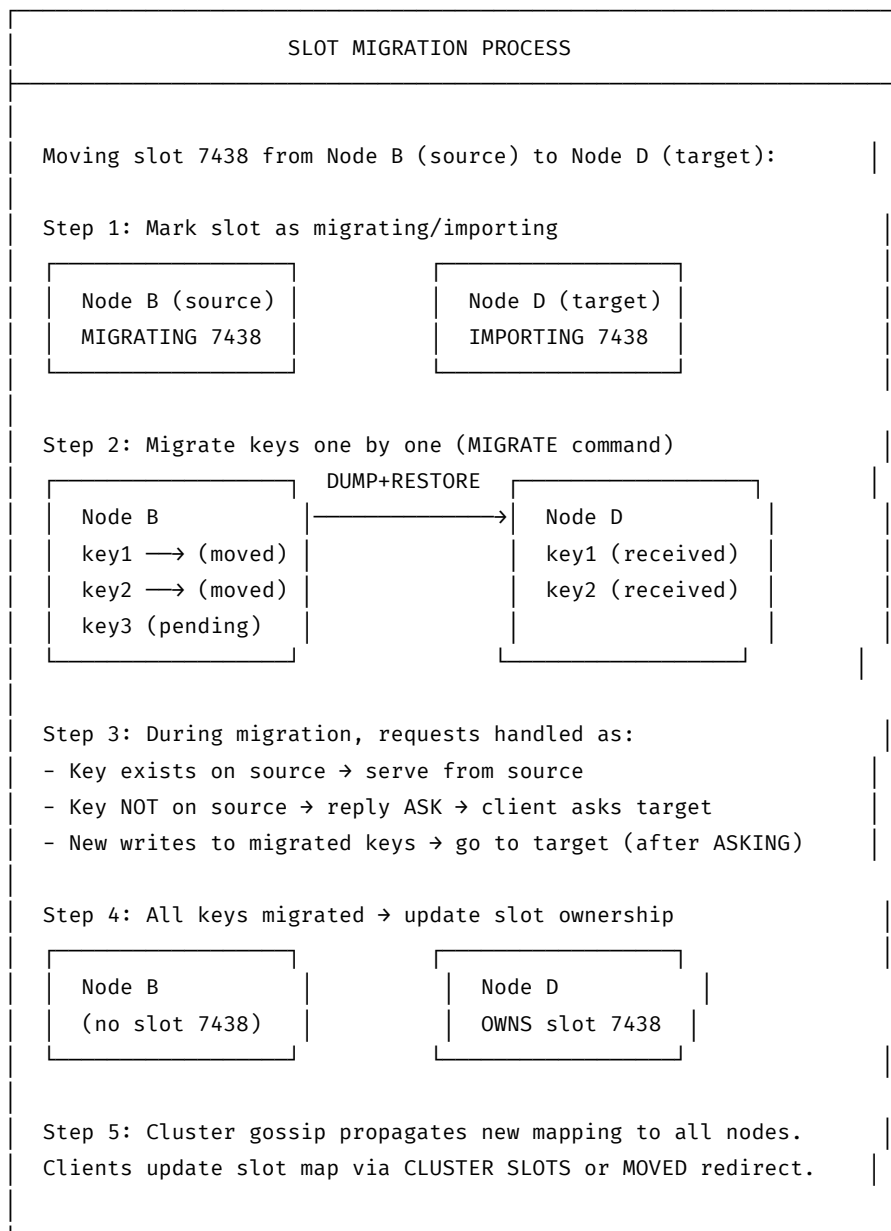
- Gossip messages include a bitmap of all slots (16384 bits = 2 KB per message).
- 65536 slots would require 8 KB per gossip message – too much bandwidth.
- 16384 is enough for up to ~1000 nodes (16 slots per node minimum).
- Power of 2 enables fast modulo via bitwise AND.

#### 8.9.1.2 Slot Distribution



#### 8.9.1.3 Slot Migration (Online Resharding)

When adding a node or rebalancing, slots are migrated live without downtime:



## 8.9.2 8.2 Eviction Policies

### 8.9.2.1 Approximate LRU

Redis does NOT maintain a true LRU list. Instead, it uses a sampling-based approach:



```
Configuration: maxmemory-policy allkeys-lru
 maxmemory-samples 5 (default, can increase to 10)
```

1. Memory usage exceeds maxmemory
2. Randomly sample 5 keys from keyspace
3. Compute `idle_time = now - lru_clock` for each
4. Insert into eviction pool (sorted by `idle_time`)
5. Evict key with highest `idle_time` from pool
6. Repeat until enough memory is freed

| Sample Size | Accuracy | Notes                      |
|-------------|----------|----------------------------|
| 5 (default) | ~90%     | Good enough for most cases |
| 10          | ~97%     | Near-perfect LRU behavior  |
| 20          | ~99%     | Diminishing returns        |

|     |     |     |     |     |     |     |      |
|-----|-----|-----|-----|-----|-----|-----|------|
| k12 | k45 | k78 | k3  | k91 | k22 | ... | k67  |
| 2s  | 5s  | 12s | 30s | 45s | 60s |     | 300s |

- volatile-lru: LRU among keys WITH expiry set
- allkeys-lru: LRU among ALL keys
- volatile-random: Random among keys with expiry
- allkeys-random: Random among all keys
- volatile-ttl: Evict keys closest to expiring
- noeviction: Return OOM error on writes

Redis 4.0+ introduced LFU, which tracks access frequency with a logarithmic counter and applies time-based decay to prevent stale popular keys from never being evicted.

## LFU WITH DECAY ALGORITHM

Each key stores 24 bits of LFU data:

|                                     |
|-------------------------------------|
| 16 bits: last decrement time (min)  |
| 8 bits: logarithmic counter (0-255) |

Logarithmic counter increment:

```
counter = current_counter
p = 1.0 / (counter * lfu_log_factor + 1)
if random() < p:
 counter = min(counter + 1, 255)
```

With lfu\_log\_factor=10 (default):

| Accesses | Counter | Note      |
|----------|---------|-----------|
| 1        | 1       |           |
| 10       | 2       |           |
| 100      | 8       |           |
| 1,000    | 18      |           |
| 10,000   | 38      |           |
| 100,000  | 107     |           |
| 1M+      | 255     | Saturated |

Decay mechanism (halving over time):

```
elapsed_minutes = now - last_decrement_time
decay = elapsed_minutes / lfu_decay_time
counter = max(counter - decay, 0)
```

With `lfu_decay_time=1` (default):

Counter loses 1 for every minute without access.

A key with counter=100 unused for 100 min → counter=0

Example: Flash sale item

T=0: Massive traffic → counter = 200

T=30min: Sale ends, no access → counter = 170

T=100min: → counter = 100

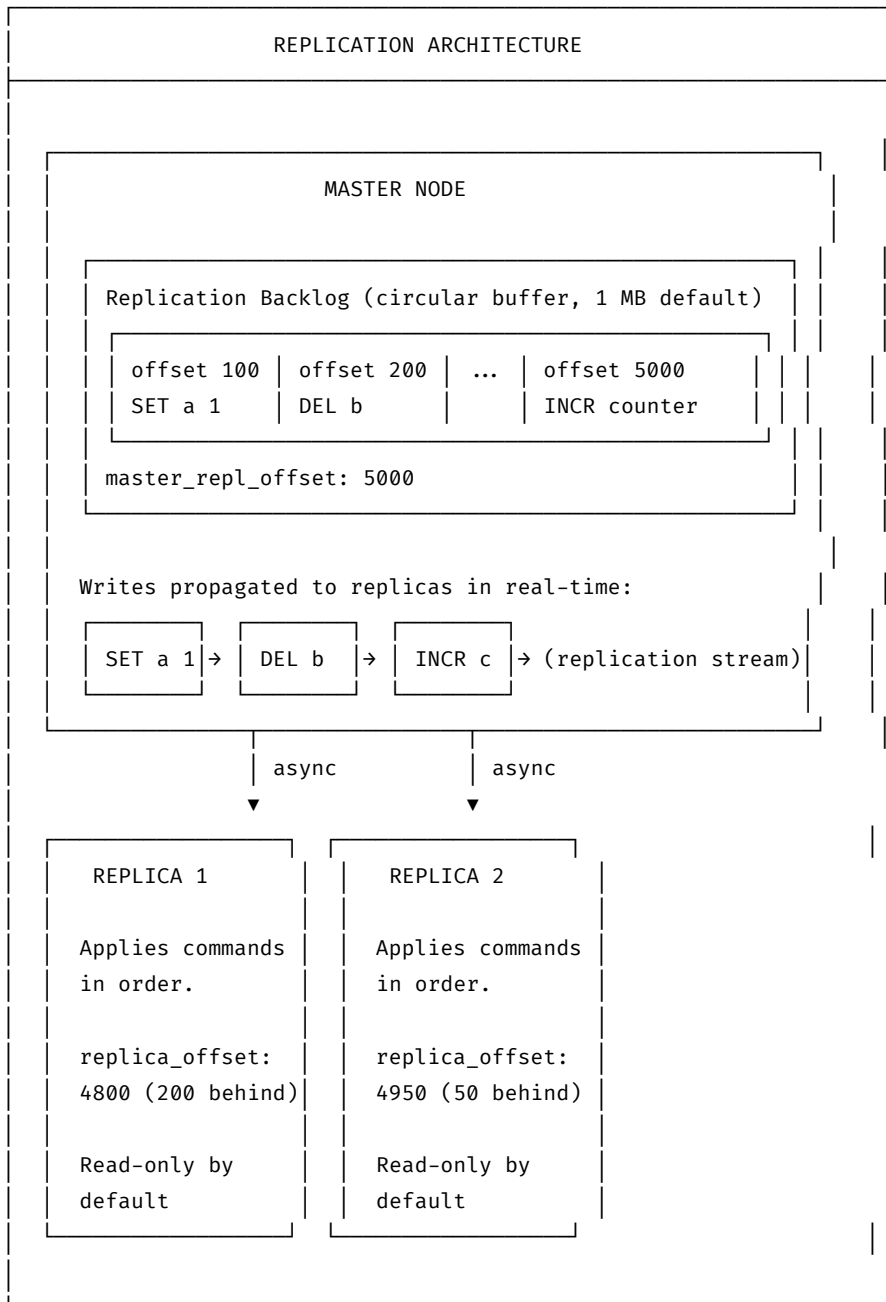
T=200min: → counter = 0 → eligible for eviction

When to use LFU over LRU:

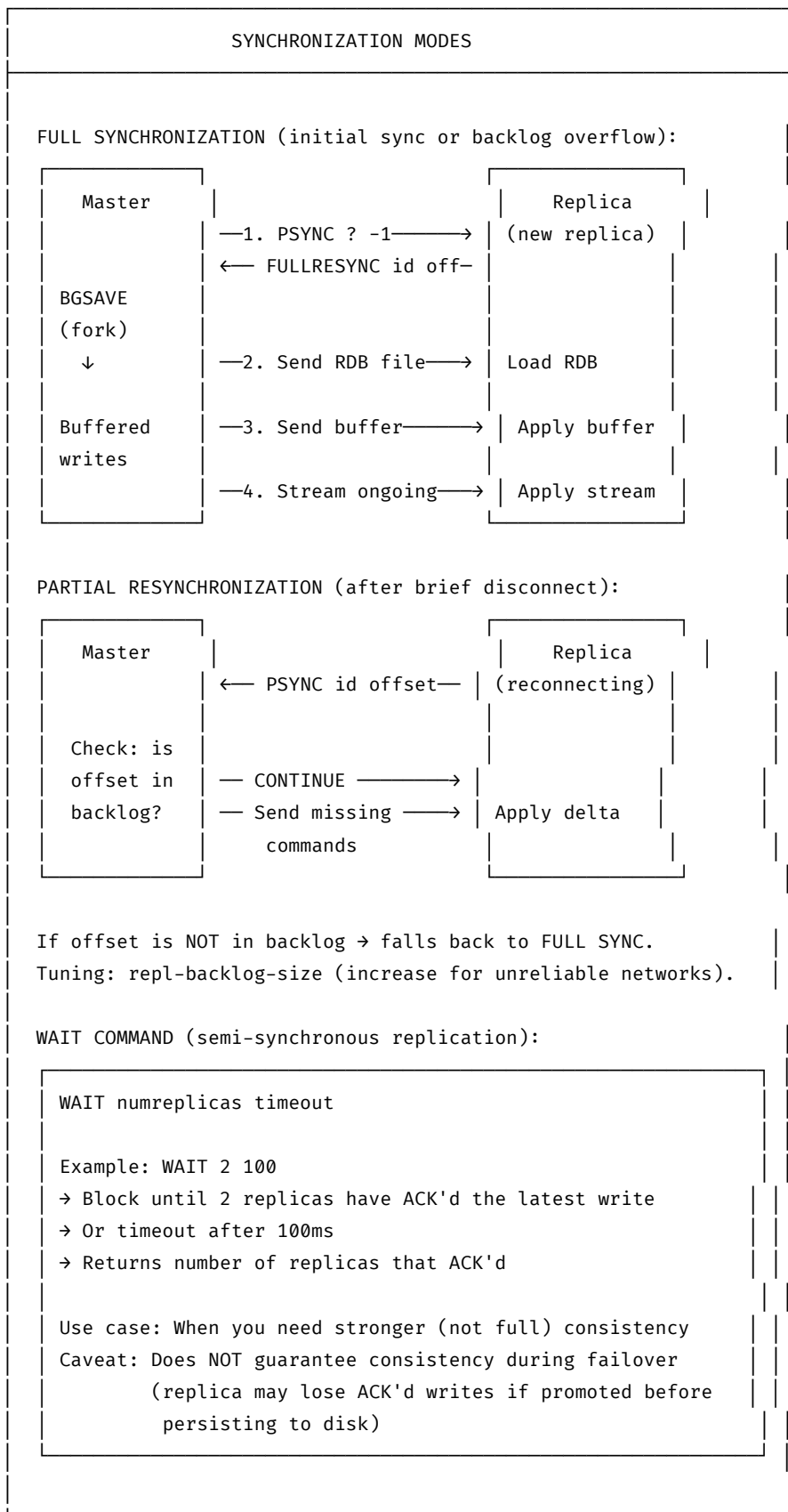
- Workloads with varying key popularity over time
- When recently accessed  $\neq$  frequently accessed
- Scan-resistant: a full key scan won't pollute the cache

### 8.9.3 8.3 Replication

#### 8.9.3.1 Master-Replica Architecture



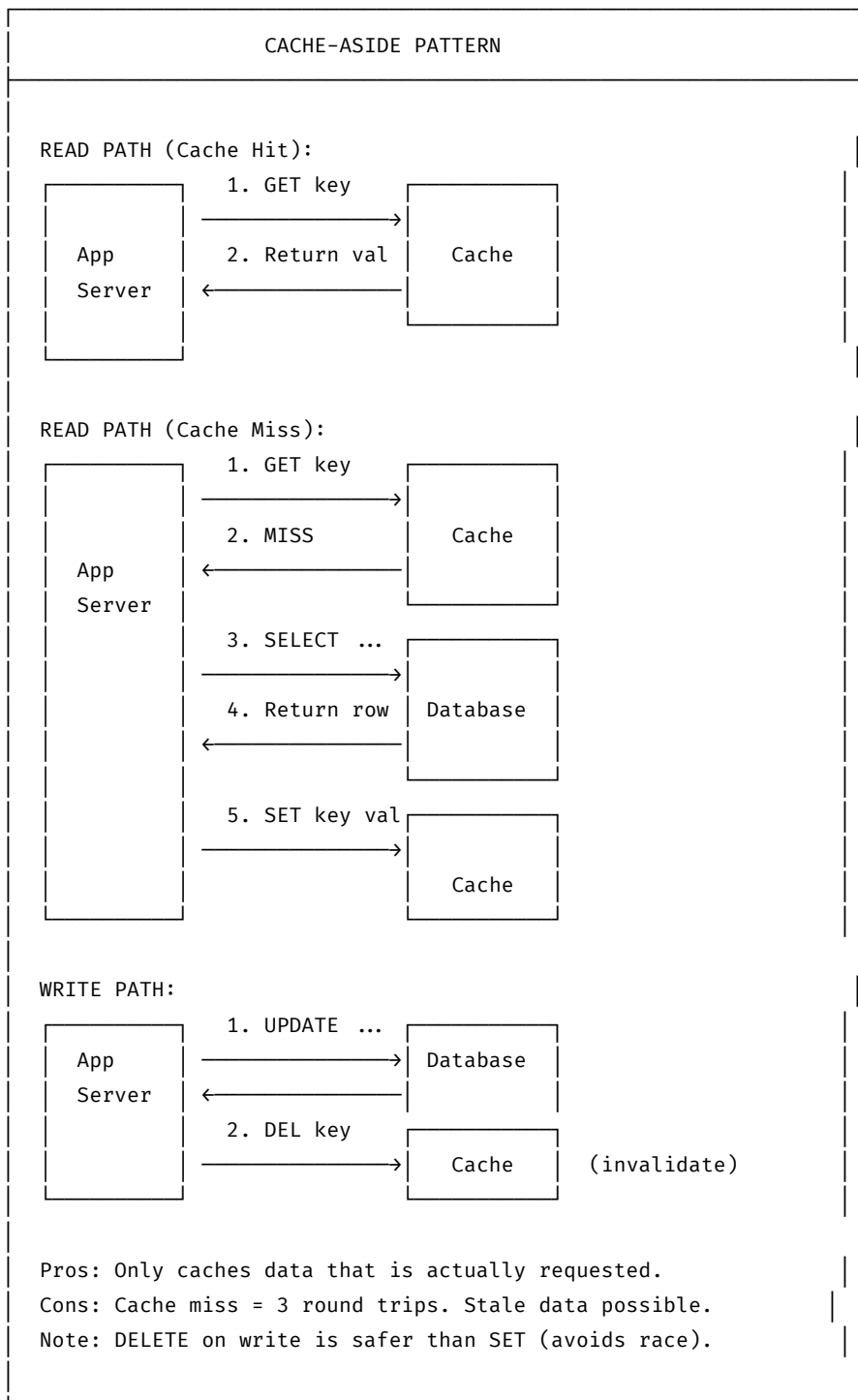
### 8.9.3.2 Synchronization Modes



## 8.9.4 8.4 Cache Patterns

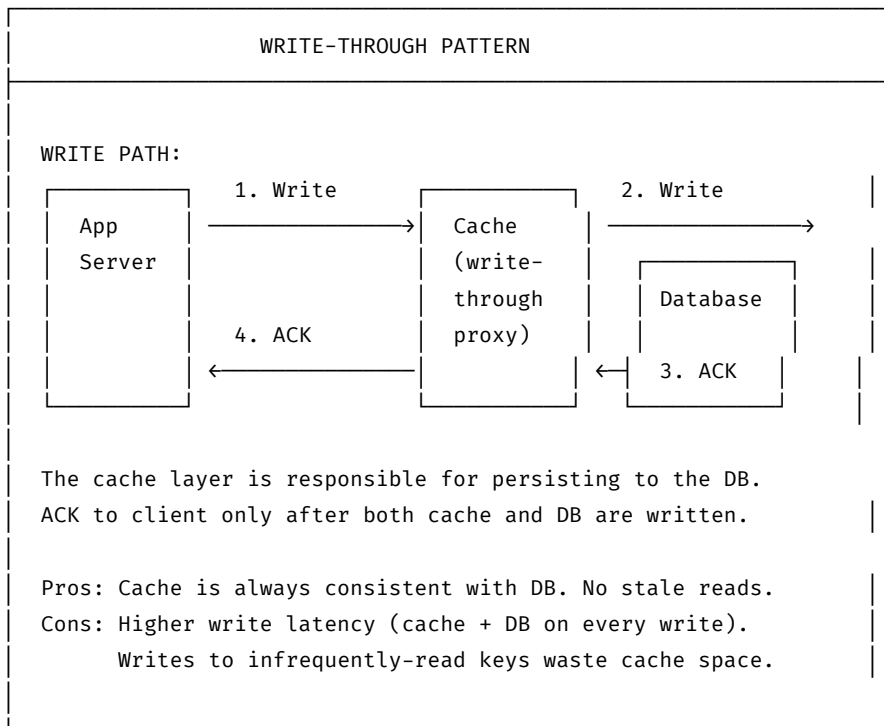
### 8.9.4.1 Cache-Aside (Lazy Loading)

The application manages the cache explicitly. Most common pattern.



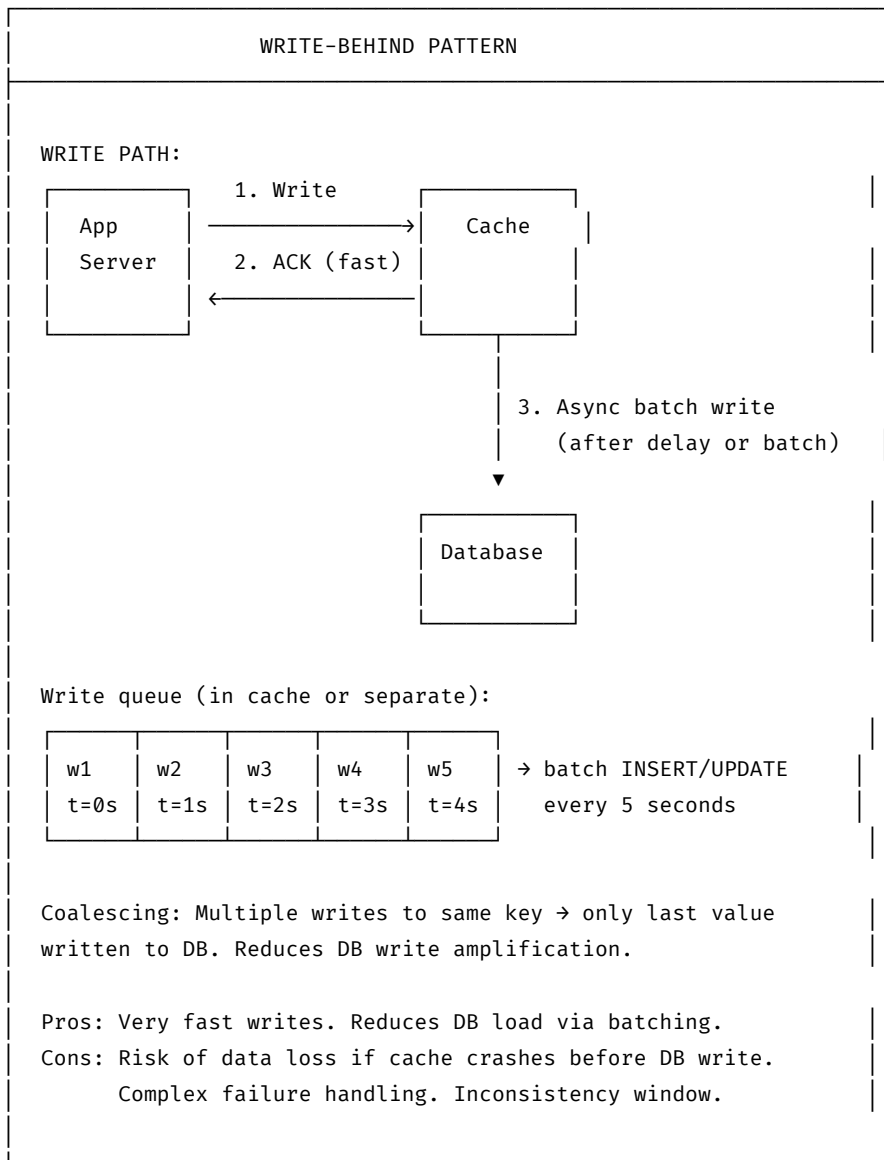
### 8.9.4.2 Write-Through

Every write goes to both cache and database synchronously.



#### 8.9.4.3 Write-Behind (Write-Back)

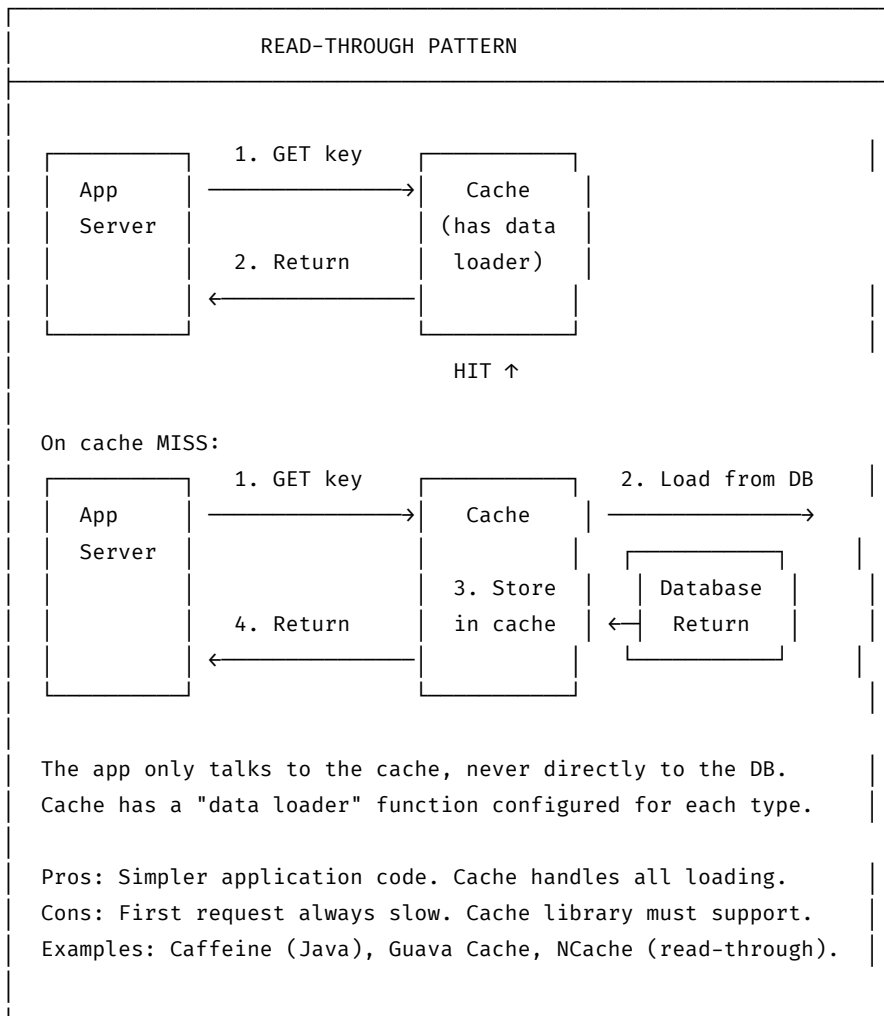
Writes go to cache immediately; database is updated asynchronously.



#### 8.9.4.4 Read-Through

Cache is responsible for loading data from the database on misses.

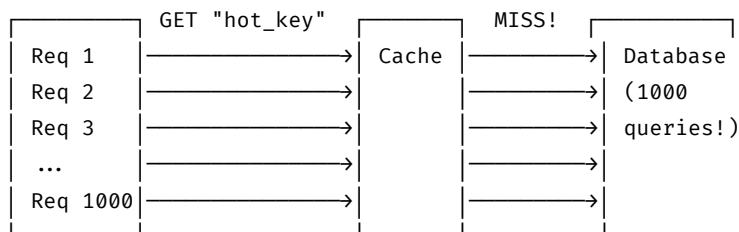




### 8.9.4.5 Cache Stampede Prevention

#### CACHE STAMPEDE PREVENTION

Problem: Hot key expires → 1000 concurrent requests → 1000 simultaneous database queries for the same data.



Solution 1: Locking (Mutex)

1. Cache miss → try SETNX "lock:hot\_key" "1" EX 5
2. If lock acquired:
  - Load from DB, SET in cache, release lock
3. If lock NOT acquired:
  - Sleep 50ms, retry GET from cache
  - If still miss after retries, fall through to DB

Solution 2: Probabilistic Early Expiration (PER)

On each cache hit, with probability that increases as TTL approaches 0:

$$\text{should\_refresh} = (\text{now} - \text{fetch\_time}) * \text{beta} \geq \text{TTL}$$

where  $\text{beta} = \text{random}(0, 1) * \log(\text{random}())$

Refresh in background before actual expiry. Only one thread statistically wins the "lottery" to refresh.

Solution 3: Background refresh

- Set TTL to 2x the desired refresh interval
- Background job refreshes at 1x interval
- Cache is never truly empty for hot keys
- Trade-off: may cache data nobody reads

## 8.10 9. Data Partitioning

### 8.10.1 Hash Slot Partitioning (Redis Cluster)

#### PARTITIONING STRATEGIES

Strategy 1: HASH SLOTS (Redis Cluster - recommended)

```
slot = CRC16(key) mod 16384
```

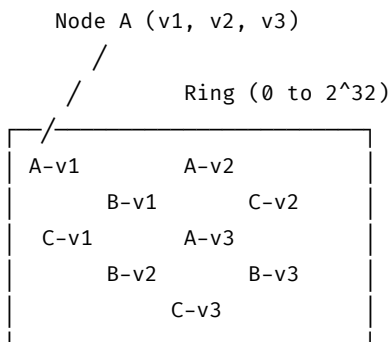
Advantages:

- Deterministic: any client can compute the target node
- Fine-grained: 16384 slots can be individually migrated
- No single point of failure (no external coordinator)
- Online resharding: move slots without downtime

Disadvantages:

- Multi-key operations only work within same slot
- 16384 slots limits practical cluster to ~1000 nodes

## Strategy 2: CONSISTENT HASHING (Memcached / client-side)



Each physical node has multiple virtual nodes (vnodes) on the ring for better distribution.

Key hashed  $\rightarrow$  walk clockwise  $\rightarrow$  first node = owner.  
Adding/removing a node only affects adjacent keys.

## Advantages:

- Minimal key movement when adding/removing nodes
- No centralized slot map needed

## Disadvantages:

- Harder to rebalance precisely
- Virtual nodes add complexity
- No native support in Redis Cluster

## HASH TAGS (forcing keys to same slot):

If key contains {...}, only the text inside {} is hashed.

## Examples:

"user:{1001}:profile"  $\rightarrow$  CRC16("1001")  $\rightarrow$  slot X  
 "user:{1001}:sessions"  $\rightarrow$  CRC16("1001")  $\rightarrow$  slot X  
 "user:{1001}:cart"  $\rightarrow$  CRC16("1001")  $\rightarrow$  slot X

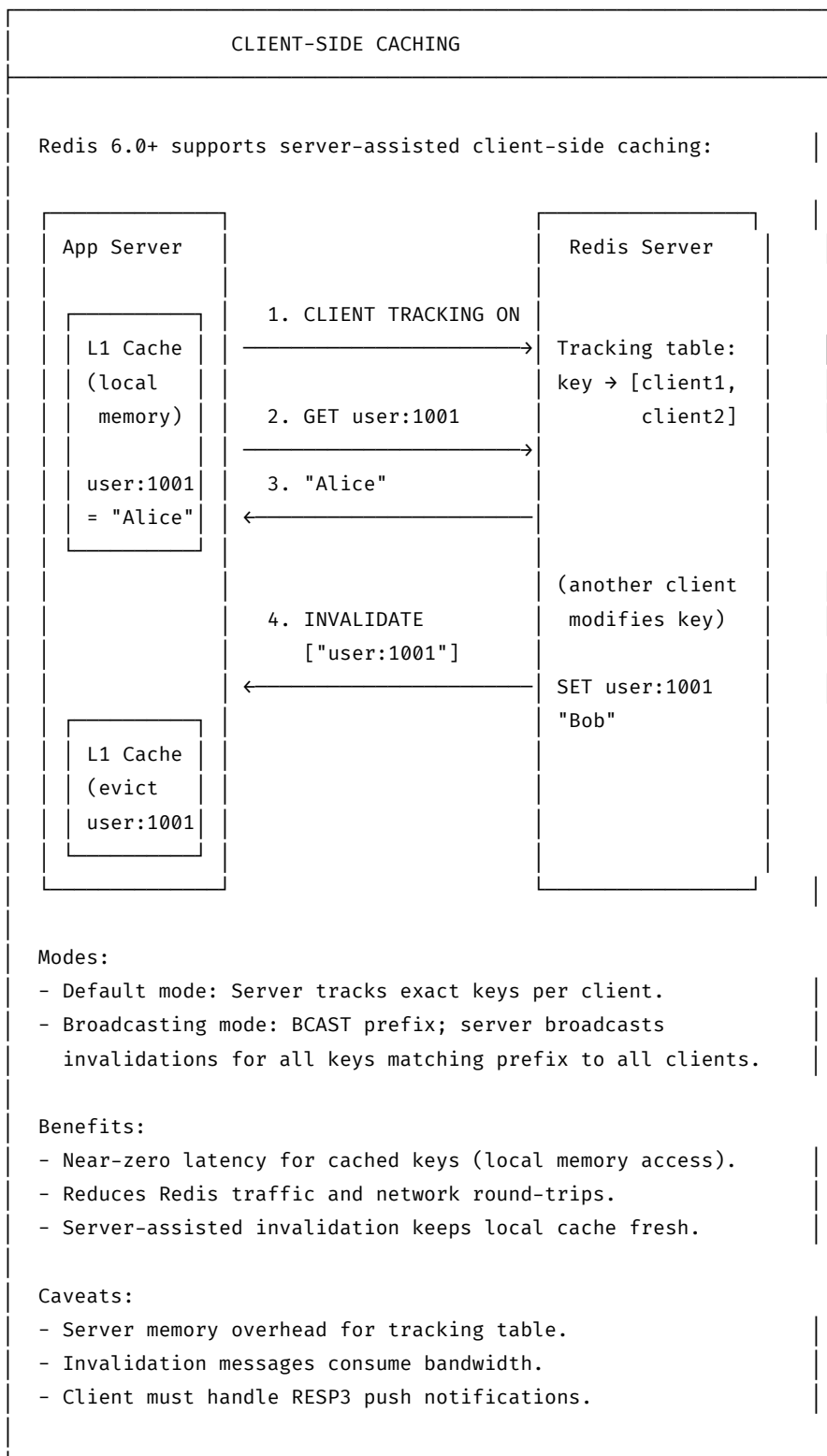
All three keys land on the same slot  $\rightarrow$  same node.

Enables multi-key operations (MGET, Lua scripts, pipeline).

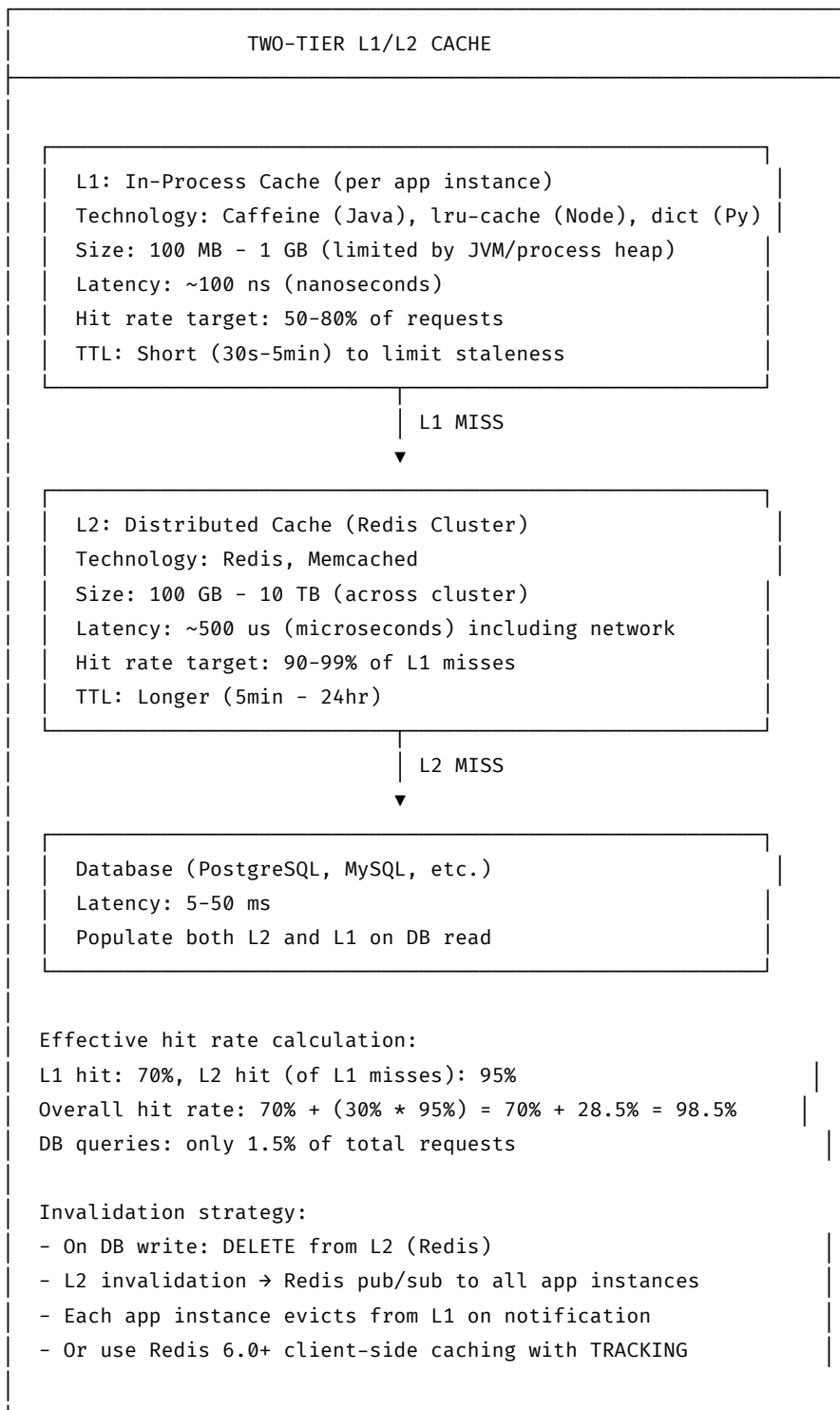
Caveat: Overuse creates hot spots if one hash tag has disproportionately many keys.

## 8.11 10. Caching Strategy

### 8.11.1 Client-Side Caching (Redis 6.0+ Tracking)



### 8.11.2 Two-Tier L1/L2 Caching



## 8.12 11. Replication and Consistency

### 8.12.1 Asynchronous Replication (Default)

#### CONSISTENCY MODEL

Redis provides EVENTUAL CONSISTENCY by default.

Timeline of a write:

```
T=0ms: Client sends SET key value to Master
T=0.1ms: Master writes to memory, returns OK to client
T=0.3ms: Master streams command to Replica 1
T=0.5ms: Replica 1 applies command
T=0.8ms: Master streams command to Replica 2
T=1.0ms: Replica 2 applies command

Window of inconsistency: 0.1ms - 1.0ms (during this window
a read from a replica may return stale data)
```

Read-your-writes guarantee:

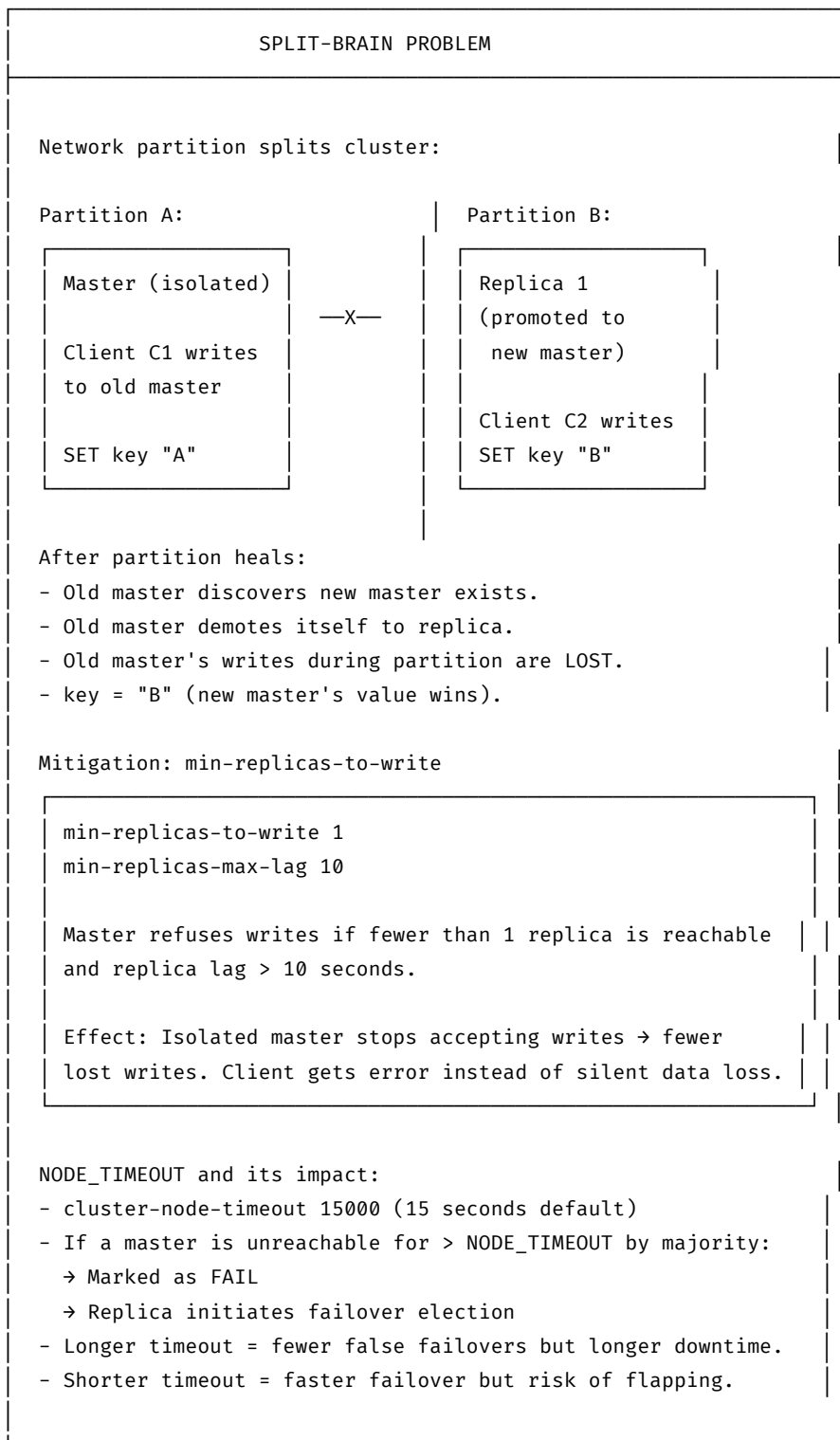
- Read from same master → always consistent
- Read from replica → may be stale
- Use WAIT for stronger guarantees (still not linearizable)

WAIT for stronger consistency:

```
SET user:1001 "Alice"
WAIT 1 100 # Wait for at least 1 replica, timeout 100ms
→ Returns 1 (one replica confirmed)

Note: WAIT blocks the client but not the server.
Other clients can still execute commands during WAIT.
```

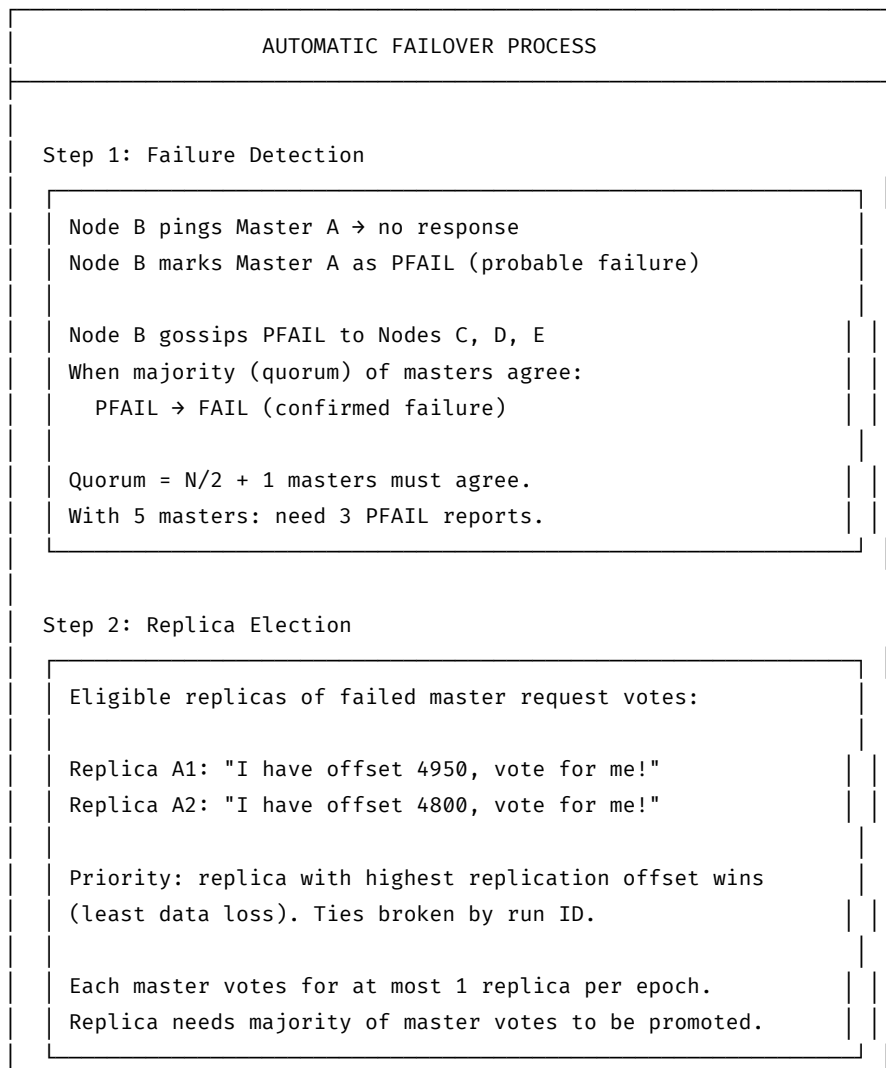
### 8.12.2 Split-Brain Scenario



## 8.13 12. Fault Tolerance



### 8.13.1 Automatic Failover



## Step 3: Promotion

Winning replica (A1):

1. Stops replicating from old master
2. Promotes itself: REPLICAOF NO ONE
3. Claims old master's hash slots
4. Broadcasts new config epoch via gossip
5. Starts accepting writes

Other nodes:

- Update slot mapping to point to new master
- Redirect clients via MOVED responses
- Remaining replicas replicate from new master

## Step 4: Old Master Recovery

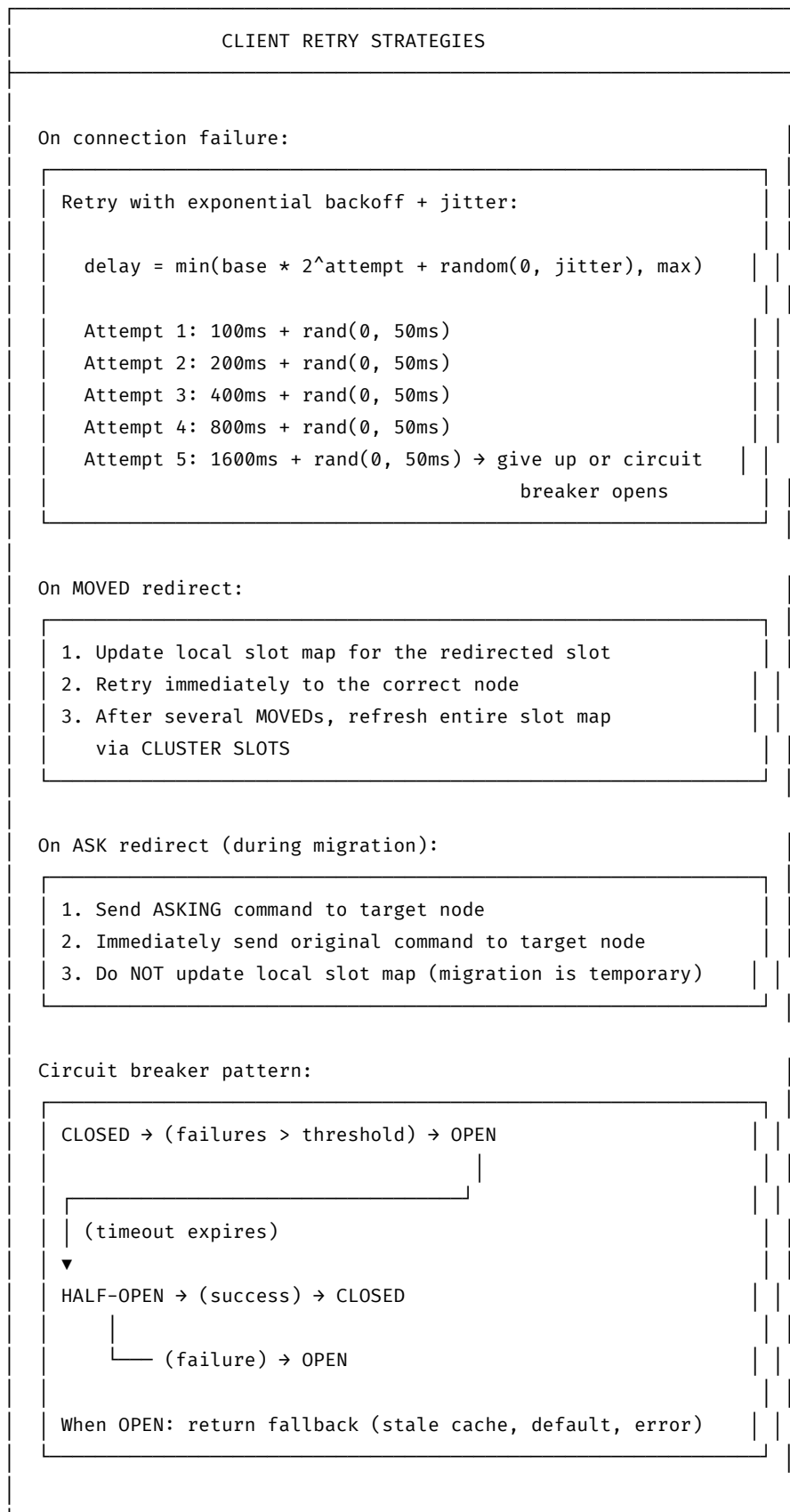
When old Master A comes back online:

- Discovers it has been replaced (lower config epoch)
- Automatically becomes a replica of new Master A1
- Syncs missing data from new master
- Cluster returns to desired replica count

Total failover time:

- Detection: cluster-node-timeout (default 15s)
- Election: 1-2 seconds
- Promotion: < 1 second
- Total: ~15-20 seconds (adjustable)

### 8.13.2 Retry and Recovery Strategies



## 8.14 13. Scalability

### 8.14.1 Horizontal Scaling (Adding Nodes + Migrating Slots)

HORIZONTAL SCALING

Before: 3 masters, uneven load on Node B

|            |             |            |
|------------|-------------|------------|
| Node A     | Node B      | Node C     |
| 5461 slots | 5462 slots  | 5461 slots |
| CPU: 40%   | CPU: 85%    | CPU: 45%   |
| MEM: 50 GB | MEM: 110 GB | MEM: 55 GB |

After: Add Node D, rebalance

|            |            |            |            |
|------------|------------|------------|------------|
| Node A     | Node B     | Node C     | Node D     |
| 4096 slots | 4096 slots | 4096 slots | 4096 slots |
| CPU: 30%   | CPU: 55%   | CPU: 35%   | CPU: 40%   |
| MEM: 40 GB | MEM: 70 GB | MEM: 42 GB | MEM: 50 GB |

Process:

1. redis-cli --cluster add-node new\_host:6379 existing:6379

2. redis-cli --cluster rebalance existing:6379

3. Tool calculates optimal slot distribution

4. Migrates slots in background (MIGRATE with pipeline)

5. Updates slot ownership via gossip

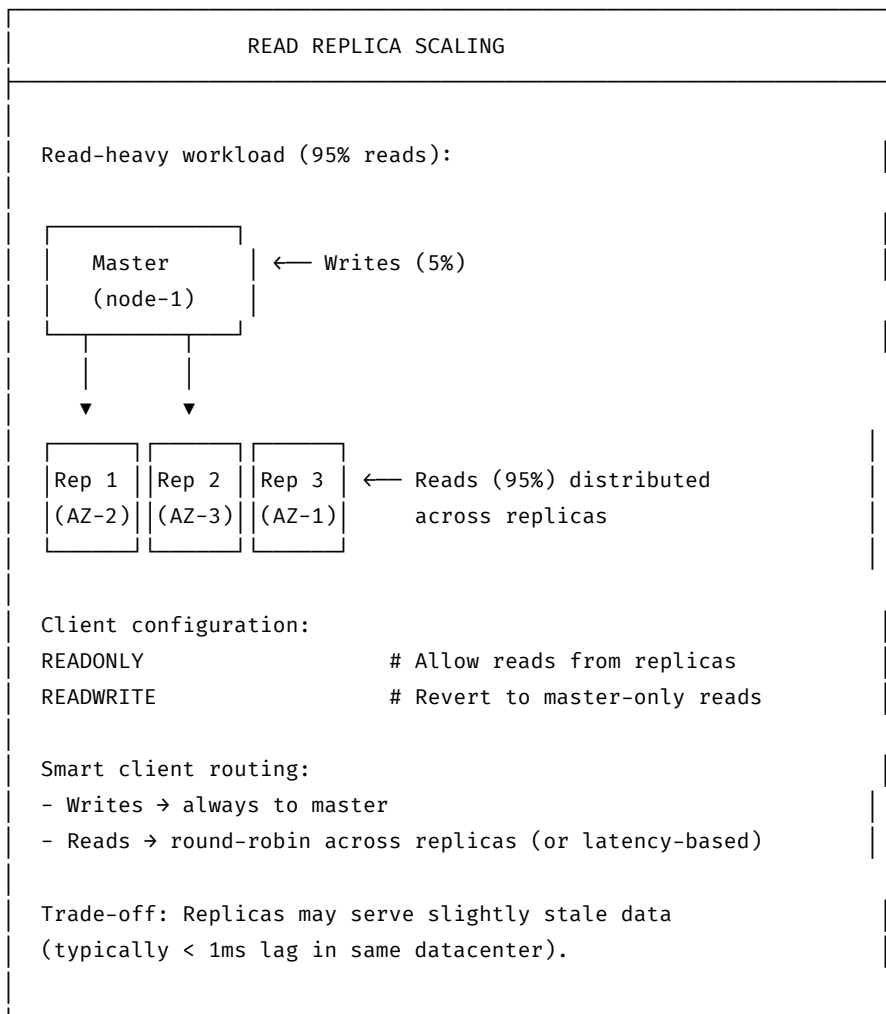
Migration speed:

- ~1000-5000 keys/sec per migration stream

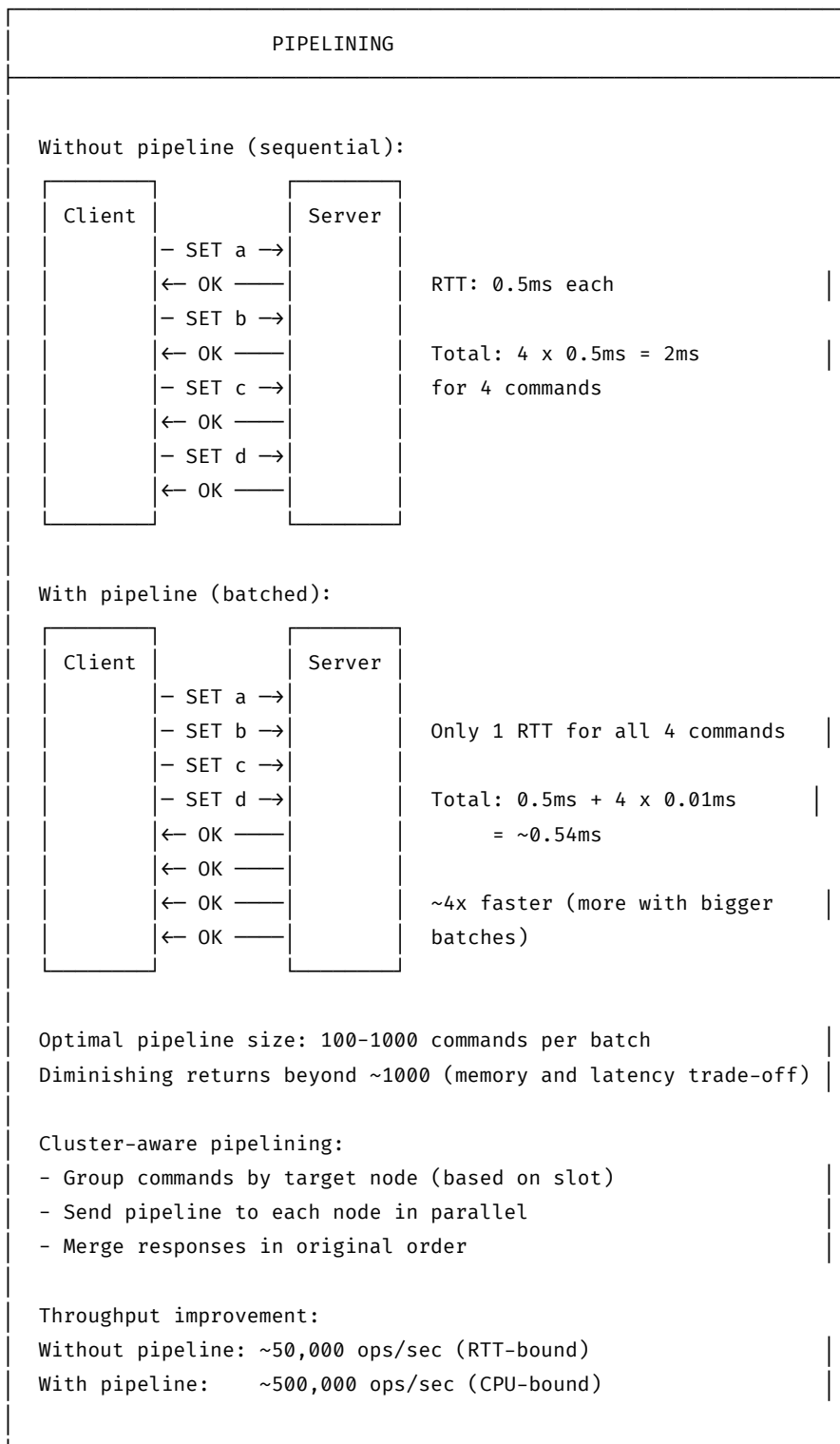
- Parallelizable across slots

- Can be throttled to reduce impact on live traffic

### 8.14.2 Read Replicas for Read Scaling



### 8.14.3 Pipelining for Throughput



## 8.15 14. Monitoring

### 8.15.1 Key Metrics

| MONITORING METRICS  |                                       |             |
|---------------------|---------------------------------------|-------------|
| CACHE EFFECTIVENESS |                                       |             |
| Metric              | Formula / Command                     | Target      |
| Hit Ratio           | <code>hits / (hits + misses)</code>   | > 95%       |
| Miss Ratio          | <code>misses / (hits + misses)</code> | < 5%        |
| Eviction Rate       | <code>evicted_keys/sec</code>         | Low/stable  |
| Expired Keys Rate   | <code>expired_keys/sec</code>         | Stable      |
| Keys Count          | <code>DBSIZE / INFO keyspace</code>   | Predictable |
| PERFORMANCE         |                                       |             |
| Metric              | Source                                | Target      |
| Ops/sec             | <code>INFO stats</code>               | > 100K      |
| Latency (p50)       | <code>redis-cli --latency</code>      | < 0.5ms     |
| Latency (p99)       | <code>LATENCY HISTORY</code>          | < 1ms       |
| Slow queries        | <code>SLOWLOG GET</code>              | 0           |
| Connected clients   | <code>INFO clients</code>             | < max       |
| Blocked clients     | <code>INFO clients</code>             | ~0          |
| Command rate        | <code>INFO commandstats</code>        | -           |

## MEMORY

| Metric               | Source                  | Target      |
|----------------------|-------------------------|-------------|
| Used memory          | INFO memory             | < maxmemory |
| Memory fragmentation | mem_fragmentation_ratio | 1.0 - 1.5   |
| RSS (Resident Set)   | used_memory_rss         | < physical  |
| Peak memory          | used_memory_peak        | < maxmemory |
| Eviction count       | evicted_keys            | Low         |
| Object encoding      | OBJECT ENCODING key     | Optimal     |

## REPLICATION

| Metric              | Source                       | Target      |
|---------------------|------------------------------|-------------|
| Replication offset  | INFO replication             | Close match |
| Replication lag     | master_offset - slave_offset | < 1 second  |
| Connected replicas  | connected_slaves             | = expected  |
| Sync status         | master_sync_in_progress      | 0 (idle)    |
| Repl backlog active | repl_backlog_active          | 1           |
| Repl backlog size   | repl_backlog_size            | Adequate    |

## CLUSTER

| Metric             | Source                   | Target     |
|--------------------|--------------------------|------------|
| Cluster state      | CLUSTER INFO             | ok         |
| Slots assigned     | cluster_slots_assigned   | 16384      |
| Slots OK           | cluster_slots_ok         | 16384      |
| Known nodes        | cluster_known_nodes      | = expected |
| Messages sent/recv | cluster_stats_messages_* | Stable     |

## Alerting Thresholds:

- Hit ratio < 90%: WARNING (investigate cold keys or eviction)
- Memory > 85% maxmemory: WARNING
- Memory > 95% maxmemory: CRITICAL
- Eviction rate spike: WARNING (may cause latency)
- Replication lag > 5 sec: WARNING
- Replication lag > 30 sec: CRITICAL
- Connected replicas < expected: CRITICAL
- Cluster state  $\neq$  ok: CRITICAL
- Fragmentation ratio > 1.5: WARNING (run MEMORY PURGE)
- p99 latency > 5ms: WARNING (check slow log)



## 8.16 15. Trade-offs

### 8.16.1 Redis vs Memcached

| REDIS vs MEMCACHED |                                                                       |                                                  |
|--------------------|-----------------------------------------------------------------------|--------------------------------------------------|
| Feature            | Redis                                                                 | Memcached                                        |
| Data structures    | Strings, Hashes, Lists, Sets, Sorted Sets, Streams, HyperLogLog, etc. | Strings only                                     |
| Threading model    | Single-threaded (I/O threads in 6.0+)                                 | Multi-threaded (better multi-core utilization)   |
| Persistence        | RDB, AOF, Hybrid                                                      | None (pure cache)                                |
| Replication        | Master-replica                                                        | None (built-in)                                  |
| Clustering         | Redis Cluster (hash slots)                                            | Client-side consistent hashing                   |
| Max value size     | 512 MB                                                                | 1 MB (default)                                   |
| Eviction           | LRU, LFU, Random, TTL, noeviction                                     | LRU only                                         |
| Pub/Sub            | Yes                                                                   | No                                               |
| Lua scripting      | Yes                                                                   | No                                               |
| Transactions       | MULTI/EXEC                                                            | CAS (compare-and-swap)                           |
| Memory efficiency  | Higher overhead per key                                               | Slab allocator, more efficient for uniform sizes |
| Ecosystem          | Rich (Sentinel, Cluster, Modules)                                     | Simple, focused                                  |

When to choose Redis:

- Need rich data structures beyond simple key-value
- Need persistence (even if cache, for faster restart)
- Need pub/sub, streams, or Lua scripting
- Need built-in replication and clustering
- Need atomic operations on complex data types

When to choose Memcached:

- Simple key-value caching with known access patterns
- Need multi-threaded performance on single node
- Memory efficiency is critical with uniform value sizes
- Already have external service discovery / consistent hashing
- Operational simplicity (fewer knobs to tune)

### 8.16.2 Persistence vs Pure In-Memory

#### PERSISTENCE TRADE-OFFS

|              | No Persistence                                      | With Persistence                                     |
|--------------|-----------------------------------------------------|------------------------------------------------------|
| Performance  | Highest (no disk I/O)                               | Slight overhead (fork, fsync)                        |
| Restart time | Cold start (empty)<br>Must repopulate from database | Warm start (load from RDB/AOF)<br>Seconds to minutes |
| Data safety  | All lost on crash                                   | Configurable loss window (0 to 1 sec)                |
| Memory usage | 100% for data                                       | ~2x during BGSAVE (fork COW overhead)                |
| Use case     | True cache (rebuildable from source)                | Session store, job queues, primary data store        |

RDB fork() considerations:

- fork() creates a child process with copy-on-write pages
- During BGSAVE, writes cause page copies (COW overhead)
- Rule: reserve 50% more RAM than data size for BGSAVE
- On 128 GB node with 100 GB data: need ~150 GB virtual memory
- Alternative: AOF with no-rewrite avoids fork entirely

### 8.16.3 Single-Threaded vs Multi-Threaded

#### SINGLE-THREADED vs MULTI-THREADED

Redis (single-threaded command execution):

Advantages:

- No locks, no race conditions, no context switching
- Predictable latency (no lock contention spikes)
- Simpler code, fewer bugs
- Atomicity for free (each command runs to completion)

Disadvantages:

- Single core bottleneck (~100K-250K ops/sec)
- Slow commands (KEYS \*, large SORT) block everything
- Cannot leverage multiple CPU cores for computation

Mitigation (Redis 6.0+):

- I/O threads: read requests and write responses in parallel across threads
- Command execution remains single-threaded
- Result: ~2x throughput improvement

Scale-out: Run multiple Redis instances per server (1 per CPU core) instead of multi-threading

Memcached (multi-threaded):

Advantages:

- Leverages all CPU cores on a single machine
- Higher throughput on a single instance

Disadvantages:

- Global cache lock (slab lock per class)
- Lock contention under high concurrency
- More complex code, harder to extend

Emerging: Redis 8+ / KeyDB / Dragonfly

- Multi-threaded shared-nothing architectures
- Each thread owns a subset of the keyspace
- No cross-thread locking needed
- Dragonfly claims ~25x Redis throughput on single node

## 8.17 16. Interview Questions

### 8.17.1 Q1: How does Redis achieve sub-millisecond latency?

**Answer:**

1. **In-memory storage:** All data resides in RAM. Memory access is ~100ns vs ~10ms for disk.
  2. **Single-threaded execution:** No lock contention, no context switching. Each command runs atomically without synchronization overhead.
  3. **Efficient data structures:** Hash tables with  $O(1)$  lookups, skip lists with  $O(\log n)$  for sorted operations, pre-allocated SDS strings to avoid repeated allocations.
  4. **I/O multiplexing:** Uses epoll/kqueue to handle thousands of connections in a single thread without blocking.
  5. **RESP protocol:** Simple, binary-safe protocol with minimal parsing overhead.
  6. **Zero-copy where possible:** Shared objects for small integers (0-9999), avoiding memory allocation for common values.
  7. **Pipelining:** Amortizes network RTT across multiple commands, keeping the CPU fed.
- 

### 8.17.2 Q2: How does Redis Cluster handle resharding without downtime?

**Answer:**

1. **Mark slots:** Source node marks slot as MIGRATING, target marks it as IMPORTING.
  2. **Key-by-key migration:** Each key is atomically moved using DUMP (serialize) on source and RESTORE (deserialize) on target, then deleted from source.
  3. **During migration:**
    - If the key exists on source: serve normally from source.
    - If the key does NOT exist on source: reply with `-ASK target_node`. Client sends `ASKING +` command to the target.
    - New keys for the migrating slot can be created on either node (source if not yet migrated, target after `ASKING`).
  4. **Completion:** After all keys are moved, slot ownership is updated in the cluster config. All nodes learn via gossip. Clients get `MOVED` responses and update slot maps.
  5. **No data loss:** Each key is atomically present on exactly one node at any time.
- 

### 8.17.3 Q3: Explain the differences between cache avalanche, cache stampede, and cache penetration.

**Answer:**

- **Avalanche:** Large number of keys expire simultaneously, overwhelming the database; Cause: TTLs set to same value (e.g., midnight expiry); Solution: Add random jitter to TTL:  $TTL = base + random(0, spread)$
- **Stampede:** Single hot key expires, many concurrent requests hit DB; Cause: Popular key expiration; Solution: Mutex lock (SETNX), probabilistic early refresh, background pre-warming
- **Penetration:** Requests for keys that will NEVER exist in cache or DB; Cause: Attacker queries non-existent IDs; Solution: Bloom filter to check existence, cache null results with short TTL, input validation

**Avalanche** is about volume (many keys at once), **stampede** is about concurrency (many requests for one key), and **penetration** is about non-existent data bypassing cache.

---

#### 8.17.4 Q4: How can Redis be single-threaded yet handle 100K+ ops/sec?

**Answer:**

The bottleneck for most network services is NOT CPU but network I/O and system calls. Redis operates entirely in memory, so each command takes only a few microseconds of CPU time:

- GET: Hash lookup = ~0.5 microseconds
- SET: Hash insert = ~1 microsecond
- At 1 microsecond per command, a single core can handle ~1M commands/second theoretically

In practice, network I/O and protocol parsing reduce this to ~100K-250K ops/sec. Redis 6.0+ introduces I/O threads to parallelize read/write I/O while keeping command execution single-threaded, pushing throughput to ~500K+ ops/sec.

The real comparison: a database doing 1 command at 5ms (disk + parse + index) gets 200 ops/sec per core. Redis at 1 microsecond gets 1,000,000 ops/sec per core. The 5000x difference is purely because of in-memory access vs disk access.

---

#### 8.17.5 Q5: How do you handle hot keys in a distributed cache?

**Answer:**

A “hot key” is a key receiving disproportionately high traffic (e.g., trending topic, flash sale item), creating a bottleneck on the single node that owns the key’s slot.

**Solutions:**

1. **Read replicas with READONLY:** Spread read traffic across replicas. For a key on a master with 3 replicas, you get 4x read capacity.
  2. **Local (L1) caching:** Cache hot keys in application memory (Caffeine, in-process HashMap with short TTL). Reduces Redis traffic by 50-80%.
  3. **Key replication / fan-out:** Create N copies of the key with suffix: `hot_key:1`, `hot_key:2`, ..., `hot_key:N`. Client randomly selects one. Distributes load across N different slots/nodes.
  4. **Client-side caching (Redis 6.0+):** Use `CLIENT TRACKING` to cache the key locally with server-assisted invalidation.
  5. **Rate limiting + coalescing:** When cache misses, use a single-flight pattern – only one goroutine/thread fetches from DB, others wait for the result.
- 

#### 8.17.6 Q6: Explain Redis persistence options and when to use each.

**Answer:**

- **No persistence:** Pure cache. Data is expendable and can be rebuilt from source of truth. Use when: session caching, computed results, CDN edge cache.
- **RDB only:** Point-in-time snapshots every N seconds (after M changes). Compact binary format, fast loading. Data loss = changes since last snapshot (up to minutes). Use when: Backup/disaster recovery, cache warm-up after restart.
- **AOF only (everysec):** Logs every write command, fsync every second. At most 1 second of data loss. Larger files, slower restart (must replay all commands). Use when: Session stores, leaderboards, moderate durability needs.

- **AOF (always):** fsync after every write. Zero data loss. Significant performance hit. Use when: Financial data, critical counters (but consider a real database instead).
  - **Hybrid (RDB + AOF):** RDB for fast loading, AOF tail for recent changes. Best of both. Use when: Production systems needing both fast restart and minimal data loss. This is the recommended default for Redis 4.0+.
- 

### 8.17.7 Q7: How does the Redis eviction pool work?

#### Answer:

Before Redis 3.0, eviction sampled N random keys and evicted the one with the highest idle time. This was repeated every time memory was needed. The problem: each eviction cycle was independent; a good eviction candidate found in one cycle but not the worst might be lost.

Redis 3.0+ introduced the **eviction pool** (size 16):

1. When eviction is needed, sample `maxmemory-samples` random keys (default 5).
2. For each sampled key, compute its idle time (LRU) or frequency (LFU).
3. Insert the key into the eviction pool, which is sorted by eviction priority.
4. The pool accumulates candidates across multiple eviction cycles.
5. Evict the key with the highest priority (most idle / least frequent) from the pool.
6. This “best effort” accumulation across cycles closely approximates true LRU/LFU.

With pool + sample size of 10, Redis achieves ~97% accuracy compared to true LRU, while using only 24 bits per key instead of ~80 bytes for a linked list entry.

---

### 8.17.8 Q8: What happens during a Redis Cluster failover?

#### Answer:

1. **Detection:** Each node pings random peers every second. If a node doesn't respond within `cluster-node-timeout` (default 15s), the pinging node marks it as PFAIL.
2. **Gossip:** PFAIL state is gossiped. When a majority of masters agree a node is PFAIL, it's promoted to FAIL (hard failure).
3. **Election:** Replicas of the failed master wait a random delay (based on replication offset – the most up-to-date replica waits the shortest). Each replica requests votes from all masters. A replica needs a majority of master votes.
4. **Promotion:** The winning replica executes `REPLICAOF NO ONE`, claims the failed master's slots, increments the config epoch, and broadcasts the new configuration.
5. **Convergence:** All nodes update their slot map. Clients receive `MOVED` redirections and update their cached slot map.
6. **Recovery:** When the old master comes back, it detects it's been replaced (lower config epoch) and automatically becomes a replica of the new master.

Total time: ~15-20 seconds (dominated by `cluster-node-timeout`).

---

### 8.17.9 Q9: How would you implement a distributed rate limiter using Redis?

#### Answer:

**Sliding window with sorted set:**

---

```
-- Lua script for atomic rate limiting
local key = KEYS[1]
local window = tonumber(ARGV[1]) -- window size in seconds
local limit = tonumber(ARGV[2]) -- max requests
local now = tonumber(ARGV[3]) -- current timestamp (ms)

-- Remove entries outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window * 1000)

-- Count requests in current window
local count = redis.call('ZCARD', key)

if count < limit then
 redis.call('ZADD', key, now, now .. ':' .. math.random())
 redis.call('EXPIRE', key, window)
 return 1 -- allowed
else
 return 0 -- rate limited
end
```

#### Token bucket with INCR + TTL:

```
-- Simple fixed-window rate limiter
local key = "ratelimit:" .. KEYS[1]
local limit = tonumber(ARGV[1])
local window = tonumber(ARGV[2])

local current = redis.call('INCR', key)
if current == 1 then
 redis.call('EXPIRE', key, window)
end
if current > limit then
 return 0 -- rate limited
end
return 1 -- allowed
```

The Lua script approach is preferred because the entire operation is atomic on the server, avoiding race conditions that would occur with multiple round trips.

---

#### 8.17.10 Q10: How do you handle cache warming after a Redis restart?

##### Answer:

1. **RDB/AOF persistence:** If enabled, Redis reloads data from disk on restart. This is the simplest approach – data survives restart automatically.
2. **Pre-warming script:** On startup, query the database for the top N most accessed keys and populate the cache before serving traffic. Use pipelining for speed.
3. **Gradual warm-up behind load balancer:** Add the restarted node to the cluster but route only a fraction of traffic to it initially. As its cache fills, increase the traffic share. This prevents cache miss storms.
4. **RDB transfer from replica:** Before restarting a master, take an RDB snapshot from a replica. After restart, load that RDB. Avoids cold-start entirely.

5. **Shadow traffic:** Replay production read traffic to the new node without serving responses. The cache fills with real access patterns.
  6. **Avoid thundering herd:** Add random jitter to TTLs when pre-warming to prevent all keys from expiring at the same time later.
- 

#### 8.17.11 Q11: What is the difference between MOVED and ASK redirections?

**Answer:**

- **Meaning:** MOVED: “Slot X permanently lives on Node Y”; ASK: “Slot X is being migrated; try Node Y for this key”
- **Client action:** MOVED: Update slot map permanently, retry to Node Y; ASK: Send ASKING + command to Node Y, do NOT update slot map
- **When seen:** MOVED: After resharding is complete, or client has stale map; ASK: During active slot migration
- **Frequency:** MOVED: Should be rare after initial discovery; ASK: Temporary, only during migration

**ASK flow:** Client sends command to source node. Source node doesn't have the key (already migrated). Source replies `-ASK slot target`. Client sends `ASKING` to target (tells target “I know this slot isn't fully mine yet, but serve this key anyway”), then sends the command. This is a one-time redirect – the next command for the same slot goes back to source first.

---

#### 8.17.12 Q12: How would you design cache invalidation for a microservices architecture?

**Answer:**

1. **Event-driven invalidation:** When a service updates its database, it publishes an event (via Kafka, RabbitMQ, or Redis Streams). Consuming services that cache that data subscribe and invalidate/update their cache entries.
  2. **CDC (Change Data Capture):** Use Debezium or similar to capture database changes and stream them to a cache invalidation service. Guarantees no missed invalidations (even if the application crashes after DB write but before publishing event).
  3. **TTL-based eventual consistency:** Set short TTLs (30s-5min) on all cache entries. Accept that data may be stale for up to TTL duration. Simplest approach when slight staleness is acceptable.
  4. **Write-through with cache service:** All writes go through a centralized cache service that updates both cache and database. Ensures consistency but adds latency and a single point of failure.
  5. **Version-based invalidation:** Each cache entry includes a version number. On read, check if the cached version matches the current version (stored in a lightweight version store). Stale versions trigger a cache refresh.
- 

#### 8.17.13 Q13: Explain Redis Cluster's gossip protocol.

**Answer:**



Redis Cluster uses a gossip protocol for decentralized cluster state management:

1. **Cluster bus:** Each node opens a second port (data port + 10000, e.g., 16379) for node-to-node binary communication.
  2. **PING/PONG:** Every second, each node sends a PING to a randomly selected node. The PING contains:
    - The sender's node ID, IP, port, cluster bus port
    - The sender's current config epoch
    - A bitmap of slots the sender is responsible for
    - Flags (master/replica, PFAIL/FAIL status)
    - Information about a few other random nodes (gossip section)
  3. **Failure detection:** If a node doesn't respond to PING within `cluster-node-timeout`, it's marked PFAIL. The PFAIL state is gossiped. When >50% of masters mark a node as PFAIL, it's promoted to FAIL (a definitive failure that triggers failover).
  4. **Configuration propagation:** When a node's configuration changes (e.g., new slot assignment), it increments its config epoch. During gossip, nodes compare epochs – higher epoch wins. This ensures convergence to the latest configuration.
  5. **Scalability:**  $O(N)$  messages per node per second ( $N$  = cluster size). For a 100-node cluster, each node sends/receives ~100 gossip messages/second – manageable bandwidth.
- 

#### 8.17.14 Q14: How do you prevent cache penetration attacks?

**Answer:**

Cache penetration occurs when attackers query keys that don't exist in cache OR database, bypassing the cache entirely and hitting the database with every request.

**Solutions:**

1. **Bloom filter:** Maintain a Bloom filter of all valid keys. Before querying cache/DB, check the filter. If the key is definitely not in the set, return 404 immediately. False positive rate of ~1% is acceptable (those few requests hit DB and find nothing).
  2. **Cache null results:** When a DB query returns no result, cache a null/empty value with a short TTL (30-60 seconds). Subsequent requests for the same non-existent key get the cached null without hitting DB. Caveat: if millions of unique non-existent keys are queried, this can fill up cache memory.
  3. **Input validation:** Validate key format before querying. If your user IDs are UUIDs, reject anything that's not a valid UUID. If IDs are sequential integers, reject negative or impossibly large values.
  4. **Rate limiting:** Limit requests per IP/user. Attackers generating unique non-existent keys are throttled before they can overwhelm the database.
  5. **Request coalescing:** Use singleflight/dedup to ensure only one DB query is in-flight for a given key, even if 1000 concurrent requests arrive for it.
- 

#### 8.17.15 Q15: Design a leaderboard system using Redis sorted sets.

**Answer:**

```
Add/update player score
ZADD leaderboard 1500 "player:alice"
ZADD leaderboard 2300 "player:bob"
ZADD leaderboard 1800 "player:charlie"

Increment score atomically
ZINCRBY leaderboard 100 "player:alice" # Alice now has 1600

Get top 10 players (descending)
ZREVRANGE leaderboard 0 9 WITHSCORES
Returns: bob(2300), charlie(1800), alice(1600)

Get player's rank (0-indexed, descending)
ZREVRANK leaderboard "player:alice" # Returns 2 (3rd place)

Get player's score
ZSCORE leaderboard "player:alice" # Returns 1600

Get players ranked 50-60 (pagination)
ZREVRANGE leaderboard 49 59 WITHSCORES

Get players with scores between 1000-2000
ZRANGEBYSCORE leaderboard 1000 2000 WITHSCORES

Count players with score > 1000
ZCOUNT leaderboard 1000 +inf

Remove inactive player
ZREM leaderboard "player:charlie"
```

**At scale (millions of players):** - Single sorted set handles millions of members efficiently ( $O(\log N)$  operations). - For sharding: use hash tags `{game:1}:leaderboard` to keep related data on same node. - For time-based leaderboards (daily/weekly): use separate keys per period, delete old ones. - For real-time updates: Redis handles ~100K ZINCRBY/sec per node, sufficient for most games. - For very large leaderboards with approximate ranking: use HyperLogLog for count and approximate percentile-based ranking.

---

## 8.18 Summary

A distributed cache is a critical infrastructure component that sits between applications and databases, dramatically reducing latency and database load. The key design decisions are:

- **Architecture:** Hash slot partitioning (16384 slots) with master-replica topology
- **Eviction:** Approximate LRU with eviction pool (or LFU for frequency-based workloads)
- **Persistence:** Hybrid RDB+AOF for production; none for pure cache
- **Replication:** Async replication with WAIT for critical writes
- **Cache pattern:** Cache-aside with stampede prevention (mutex or early refresh)
- **Scaling reads:** Read replicas + client-side caching (L1/L2)
- **Scaling writes:** Add masters + rebalance slots
- **Fault tolerance:** Automatic failover with gossip-based failure detection
- **Monitoring:** Hit ratio, eviction rate, memory utilization, replication lag

The system achieves sub-millisecond latency through in-memory storage, single-threaded execution (avoiding lock contention), and efficient I/O multiplexing. It scales horizontally by distributing hash slots across nodes and vertically through read replicas and pipelining.

## Chapter 9

# Design an Object Storage Service (Amazon S3)

**Difficulty:** Hard | **Companies:** Amazon, Google, Microsoft, Alibaba, MinIO

---

### 9.1 1. Problem Statement and Scope

#### 9.1.1 Problem

Design a highly durable, scalable object storage service—akin to Amazon S3—that stores and retrieves arbitrary-sized objects (files, images, videos, backups, logs, ML datasets) with **99.999999999% (11 nines) durability**. The service must handle petabyte-scale storage, serve millions of requests per second, and provide strong read-after-write consistency while remaining cost-efficient through tiered storage.

#### 9.1.2 Scope

**In Scope:** - Object CRUD operations (PUT, GET, DELETE, HEAD) - Bucket management (create, list, delete, configure) - Versioning (object-level version history) - Multipart upload for large objects (up to 5 TB) - Access control: ACLs, bucket policies, IAM integration - Pre-signed URLs for temporary access - Lifecycle policies (transition between storage tiers, expiration) - Object metadata and tagging - Listing with prefix filtering and delimiter support (virtual directory hierarchy) - Cross-region replication - Storage classes (hot, warm, cold, archive)

**Out of Scope:** - File-system semantics (POSIX compliance, rename atomicity) - Real-time streaming or append operations - SQL-style querying over object contents (S3 Select is a separate deep-dive) - CDN edge caching design (assume integration with existing CDN) - Billing and metering subsystem internals

---

### 9.2 2. Functional Requirements

- **FR-1 - PUT Object:** Upload objects from 1 byte to 5 TB. Support streaming uploads.
- **FR-2 - GET Object:** Download full object or byte-range requests.
- **FR-3 - DELETE Object:** Mark object for deletion; reclaim space asynchronously.

- **FR-4 - HEAD Object:** Retrieve metadata without downloading the object body.
  - **FR-5 - Bucket Management:** Create, list, delete buckets. Globally unique bucket names.
  - **FR-6 - Object Listing:** List objects in a bucket with prefix, delimiter, marker-based pagination.
  - **FR-7 - Versioning:** Enable per-bucket versioning; each PUT creates a new version.
  - **FR-8 - Multipart Upload:** Initiate, upload parts (5 MB - 5 GB each), complete/abort.
  - **FR-9 - Access Control:** Bucket policies (JSON), object ACLs, IAM policy evaluation.
  - **FR-10 - Pre-signed URLs:** Time-limited signed URLs for upload or download.
  - **FR-11 - Lifecycle Policies:** Transition objects between storage classes; auto-expire objects.
  - **FR-12 - Object Metadata:** System metadata (size, ETag, content-type) + user-defined metadata.
  - **FR-13 - Cross-Region Replication:** Async replication of objects to a target bucket in another region.
  - **FR-14 - Storage Classes:** Standard, Infrequent Access, Glacier (archive), Deep Archive.
- 

### 9.3 3. Non-Functional Requirements

- **NFR-1 - Durability:** 99.999999999% (11 nines) annual durability
- **NFR-2 - Availability:** 99.99% (four nines) for standard storage class
- **NFR-3 - Scale:** Support petabytes of storage, trillions of objects
- **NFR-4 - Throughput:** Sustain millions of requests/sec globally
- **NFR-5 - Latency:** p50 < 50ms for GET, p99 < 200ms for GET, p50 < 100ms for PUT
- **NFR-6 - Consistency:** Strong read-after-write consistency
- **NFR-7 - Cost Efficiency:** Erasure coding (1.5x overhead) over triple replication (3x)
- **NFR-8 - Security:** Encryption at rest (SSE-S3, SSE-KMS, SSE-C), encryption in transit (TLS)
- **NFR-9 - Compliance:** WORM (Write Once Read Many) support for regulatory compliance
- **NFR-10 - Operability:** Zero-downtime upgrades, automated failure recovery

#### 9.3.1 Durability Math: What Does 11 Nines Mean?

11 nines = 99.999999999% durability per year

Probability of losing an object in a year = 0.000000001% = 1e-11

If you store 10 billion objects:

Expected objects lost per year =  $10^{10} * 10^{-11} = 0.1$

=> Statistically, you lose LESS THAN 1 object per 10 billion per year.

For 10 PB of data (100 billion 100KB objects):

Expected loss =  $100 * 10^9 * 10^{-11} = 1$  object/year

---

### 9.4 4. Back-of-Envelope Estimation

### 9.4.1 Storage Scale

|                          |                                                         |
|--------------------------|---------------------------------------------------------|
| Total objects stored:    | 100 billion ( $10^{11}$ )                               |
| Average object size:     | 100 KB                                                  |
| Total raw data:          | $100 * 10^9 * 100 \text{ KB} = 10 \text{ PB}$           |
| Erasure coding overhead: | 1.5x                                                    |
| Total physical storage:  | $10 \text{ PB} * 1.5 = 15 \text{ PB}$                   |
| Metadata per object:     | ~1 KB (key, bucket, version, ETag, timestamps, ACL ref) |
| Total metadata:          | $100 * 10^9 * 1 \text{ KB} = 100 \text{ TB}$            |

### 9.4.2 Traffic Estimates

|                       |                                                                      |
|-----------------------|----------------------------------------------------------------------|
| Write (PUT) requests: | 1 million/sec                                                        |
| Read (GET) requests:  | 10 million/sec (10:1 read-write ratio)                               |
| DELETE requests:      | 100K/sec                                                             |
| Average PUT payload:  | 100 KB                                                               |
| Write throughput:     | $1\text{M} * 100 \text{ KB} = 100 \text{ GB/sec} = 800 \text{ Gbps}$ |
| Read throughput:      | $10\text{M} * 100 \text{ KB} = 1 \text{ TB/sec} = 8 \text{ Tbps}$    |
| Daily new data:       | $1\text{M/sec} * 100 \text{ KB} * 86400 = \sim 8.6 \text{ PB/day}$   |
| Annual growth:        | ~3 EB/year (before erasure coding)                                   |

### 9.4.3 Hardware Estimates

|                            |                                                                    |
|----------------------------|--------------------------------------------------------------------|
| Storage node capacity:     | 100 TB usable per node (12 x 16TB HDDs, ~80% utilization)          |
| Nodes for 15 PB:           | $15,000 \text{ TB} / 100 \text{ TB} = 150 \text{ nodes (initial)}$ |
| With growth headroom (2x): | 300 storage nodes                                                  |
| API servers:               |                                                                    |
| Each handles ~10K req/sec  |                                                                    |
| For 11M total req/sec:     | ~1,100 API servers                                                 |
| Metadata servers:          |                                                                    |
| 100 TB metadata, sharded   |                                                                    |
| With caching, ~50 servers  |                                                                    |

### 9.4.4 Erasure Coding vs Replication Cost

|                     | Replication (3x)                                  | Erasure Coding (8+4)      |
|---------------------|---------------------------------------------------|---------------------------|
| Storage overhead:   | 3.0x                                              | 1.5x                      |
| For 10 PB raw data: | 30 PB                                             | 15 PB                     |
| Fault tolerance:    | 2 failures                                        | 4 failures                |
| Repair cost:        | Copy full replica                                 | Reconstruct from k chunks |
| Read latency:       | Fast (any replica)                                | Slightly higher (decode)  |
| Write latency:      | Slightly lower                                    | Slightly higher (encode)  |
| Cost per PB/month:  | ~\$30K                                            | ~\$15K                    |
| Winner for scale:   | Erasure Coding (50% cheaper, more fault tolerant) |                           |

---

## 9.5 5. API Design

### 9.5.1 5.1 Object Operations

PUT /v1/{bucket}/{key}

Headers:

Content-Type: application/octet-stream

Content-Length: <size>

x-amz-storage-class: STANDARD | IA | GLACIER

x-amz-meta-{key}: {value} # user-defined metadata

x-amz-server-side-encryption: AES256 | aws:kms

Body: <object data>

Response: 200 OK

ETag: "d41d8cd98f00b204e9800998ecf8427e"

x-amz-version-id: "v3.2.1"

GET /v1/{bucket}/{key}

Headers:

Range: bytes=0-1023 # byte-range request (optional)

If-None-Match: "<etag>" # conditional GET

Query Params:

versionId=<version> # specific version

Response: 200 OK | 206 Partial Content | 304 Not Modified

Body: <object data>

DELETE /v1/{bucket}/{key}

Query Params:

versionId=<version> # delete specific version

Response: 204 No Content

x-amz-delete-marker: true # if versioning enabled

HEAD /v1/{bucket}/{key}

Response: 200 OK

Content-Length: 1048576

Content-Type: image/png

ETag: "d41d8cd98f00b204e9800998ecf8427e"

Last-Modified: Thu, 01 Jan 2026 00:00:00 GMT

x-amz-version-id: "v3.2.1"

## 9.5.2 5.2 Bucket Operations

PUT /v1/{bucket}

Headers:

x-amz-bucket-region: us-east-1

Body: <CreateBucketConfiguration>

Response: 200 OK

GET /v1/{bucket}?prefix=photos/2026/&delimiter=/&max-keys=1000&marker=<last-key>

Response: 200 OK

<ListBucketResult>

<Contents>

<Key>photos/2026/jan/photo1.jpg</Key>

<Size>2048576</Size>

<ETag>"abc123"</ETag>

<LastModified>2026-01-15T10:30:00Z</LastModified>

<StorageClass>STANDARD</StorageClass>

</Contents>

<CommonPrefixes>

<Prefix>photos/2026/feb/</Prefix>

</CommonPrefixes>

<IsTruncated>true</IsTruncated>

<NextMarker>photos/2026/jan/photo999.jpg</NextMarker>

</ListBucketResult>

DELETE /v1/{bucket}

Precondition: bucket must be empty

Response: 204 No Content



### 9.5.3 5.3 Multipart Upload

```
Step 1: Initiate
POST /v1/{bucket}/{key}?uploads
 Response: 200 OK
 <UploadId>abc123</UploadId>

Step 2: Upload Parts (parallelizable)
PUT /v1/{bucket}/{key}?partNumber=1&uploadId=abc123
 Body: <part data, 5MB - 5GB>
 Response: 200 OK
 ETag: "part1-etag"

PUT /v1/{bucket}/{key}?partNumber=2&uploadId=abc123
 Body: <part data>
 Response: 200 OK
 ETag: "part2-etag"

Step 3: Complete
POST /v1/{bucket}/{key}?uploadId=abc123
 Body:
 <CompleteMultipartUpload>
 <Part><PartNumber>1</PartNumber><ETag>"part1-etag"</ETag></Part>
 <Part><PartNumber>2</PartNumber><ETag>"part2-etag"</ETag></Part>
 </CompleteMultipartUpload>
 Response: 200 OK
 ETag: "final-composite-etag"

Abort (cleanup)
DELETE /v1/{bucket}/{key}?uploadId=abc123
 Response: 204 No Content
```

### 9.5.4 5.4 Pre-signed URL Generation (Server-side)

```
Generate pre-signed URL (internal API, not HTTP-facing):
GeneratePresignedURL(
 bucket: "my-bucket",
 key: "private/report.pdf",
 method: "GET",
 expiry: 3600, # seconds
 conditions: {content-type: "application/pdf"}
)

Returns:
https://s3.example.com/v1/my-bucket/private/report.pdf
?X-Amz-Algorithm=AWS4-HMAC-SHA256
&X-Amz-Credential=AKID/20260101/us-east-1/s3/aws4_request
&X-Amz-Date=20260101T000000Z
&X-Amz-Expires=3600
&X-Amz-Signature=<computed-signature>
```

## 9.6 6. Data Model and Database Selection

### 9.6.1 6.1 Data Categories and Storage Choices

| Data Category       | Storage Engine                                                | Rationale                                                                |
|---------------------|---------------------------------------------------------------|--------------------------------------------------------------------------|
| Object data (blobs) | Custom distributed store on bare metal / HDDs                 | Raw disk I/O, erasure coded, no filesystem overhead                      |
| Object metadata     | Distributed KV store (RocksDB + Raft) or CockroachDB / Vitess | High write throughput, range queries for listing, sharded by bucket hash |
| Bucket metadata     | MySQL (Vitess sharded) + Redis cache                          | Relatively small dataset, strong consistency needed                      |
| IAM / Policies      | MySQL + local cache                                           | Read-heavy, infrequent updates                                           |
| Placement map       | etcd / ZooKeeper                                              | Small, critical config data, strong consistency via Raft                 |

### 9.6.2 6.2 Object Metadata Schema

```
ObjectMetadata {
 bucket_id: uint64 # FK to bucket (hashed bucket name)
 object_key: string # full key path (e.g., "photos/2026/img.jpg")
 version_id: string # UUID, null if versioning disabled
 object_id: UUID # internal unique identifier
 size: uint64 # object size in bytes
 etag: string # MD5 or multipart composite hash
 content_type: string # MIME type
 storage_class: enum # STANDARD, IA, GLACIER, DEEP_ARCHIVE
 encryption: enum # NONE, SSE_S3, SSE_KMS, SSE_C
 kms_key_id: string # if SSE_KMS
 created_at: timestamp
 last_modified: timestamp
 delete_marker: bool # true if this is a delete marker (versioned)
 user_metadata: map<str, str> # x-amz-meta-* headers
 tags: map<str, str> # object tagging
 acl: bytes # serialized ACL
 data_locations: []ChunkLocation # pointers to erasure-coded chunks
}

ChunkLocation {
 chunk_index: uint8 # 0..k+m-1
 node_id: uint32 # storage node identifier
 disk_id: uint16 # disk within node
 offset: uint64 # byte offset on disk
 length: uint32 # chunk size
 checksum: uint32 # CRC32C of this chunk
}
```

### 9.6.3 6.3 Bucket Metadata Schema

```
BucketMetadata {
 bucket_id: uint64 # hash of bucket_name
 bucket_name: string # globally unique
 owner_id: uint64 # account ID
 region: string # primary region
 created_at: timestamp
 versioning: enum # DISABLED, ENABLED, SUSPENDED
 encryption_config: EncryptionConfig
 lifecycle_rules: []LifecycleRule
 cors_config: CORSConfig
 replication_config: ReplicationConfig
 policy: string # JSON bucket policy
 acl: bytes
 logging_config: LoggingConfig
 object_count: uint64 # approximate, updated async
 total_size: uint64 # approximate
}
```

### 9.6.4 6.4 Why Not a Filesystem for Object Data?

Traditional filesystem (ext4, XFS):

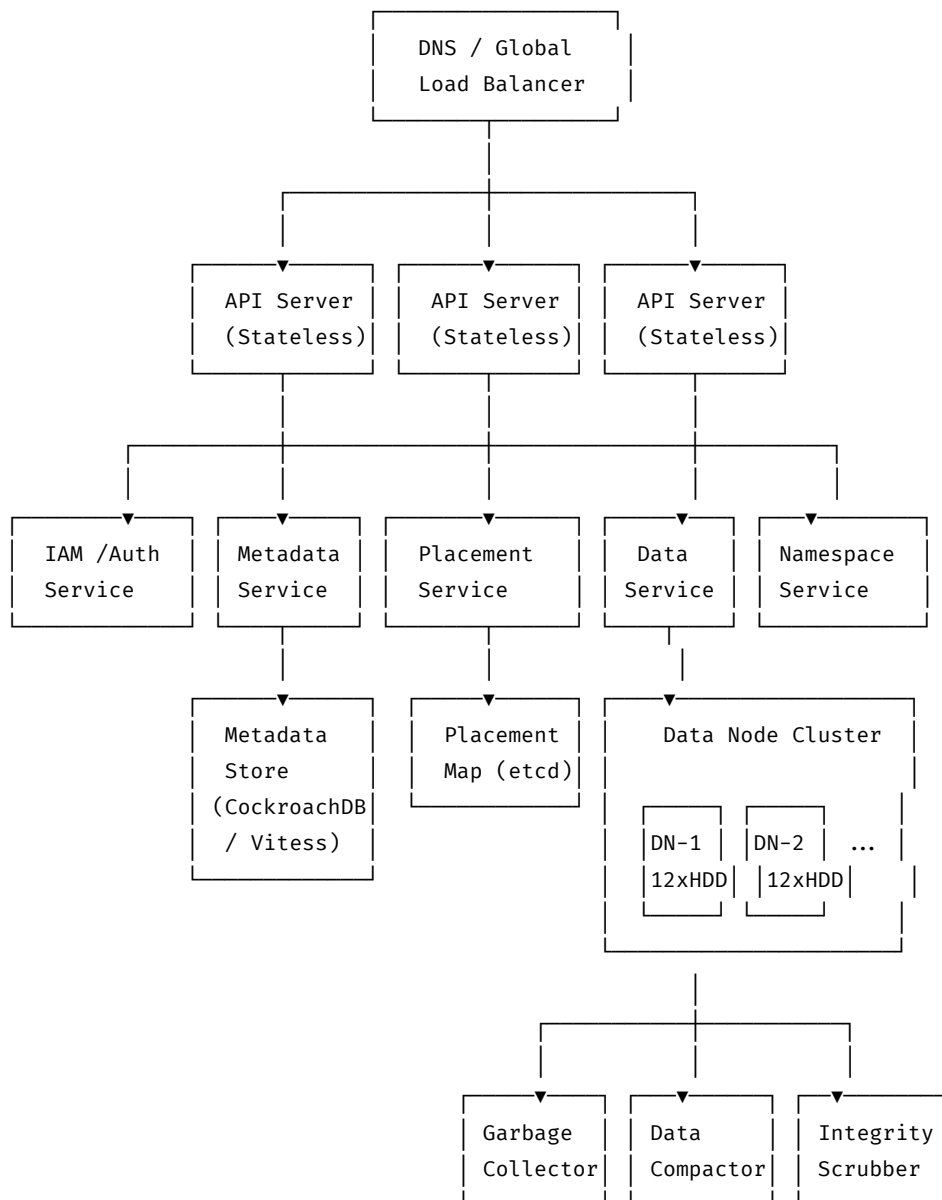
- Inode overhead per file: ~256 bytes + directory entry
- 1 billion small objects = massive inode table, slow lookups
- Journal overhead, metadata sync costs
- Not optimized for erasure coding chunk layout

Custom storage engine:

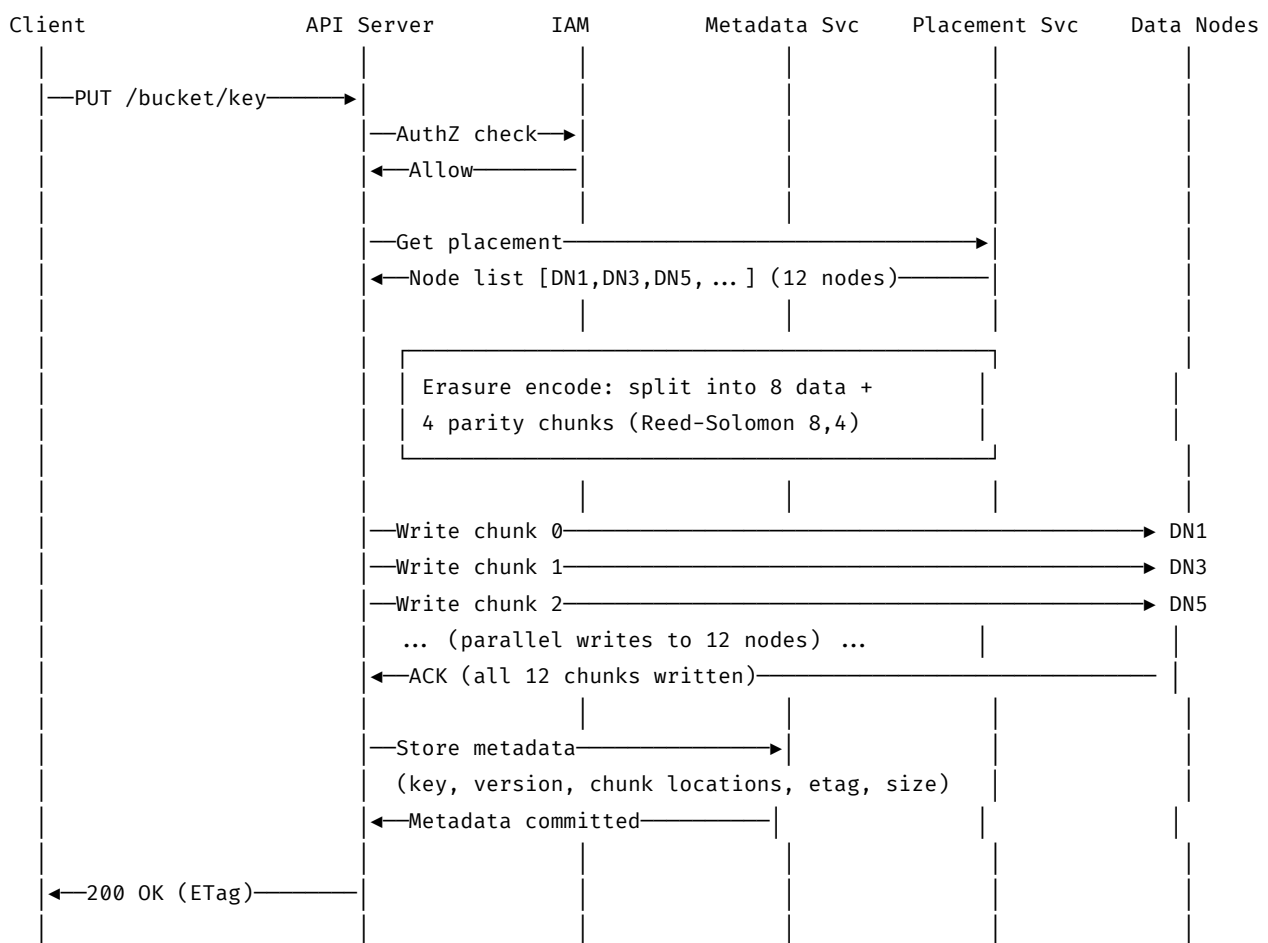
- Objects packed into large "extent files" (e.g., 1 GB segments)
  - Offset-based addressing within segments
  - No inode overhead; metadata is external (in metadata store)
  - Can align chunks for direct I/O (bypass page cache for large objects)
  - Better control over disk scheduling and repair I/O
- 

## 9.7 7. High-Level Architecture

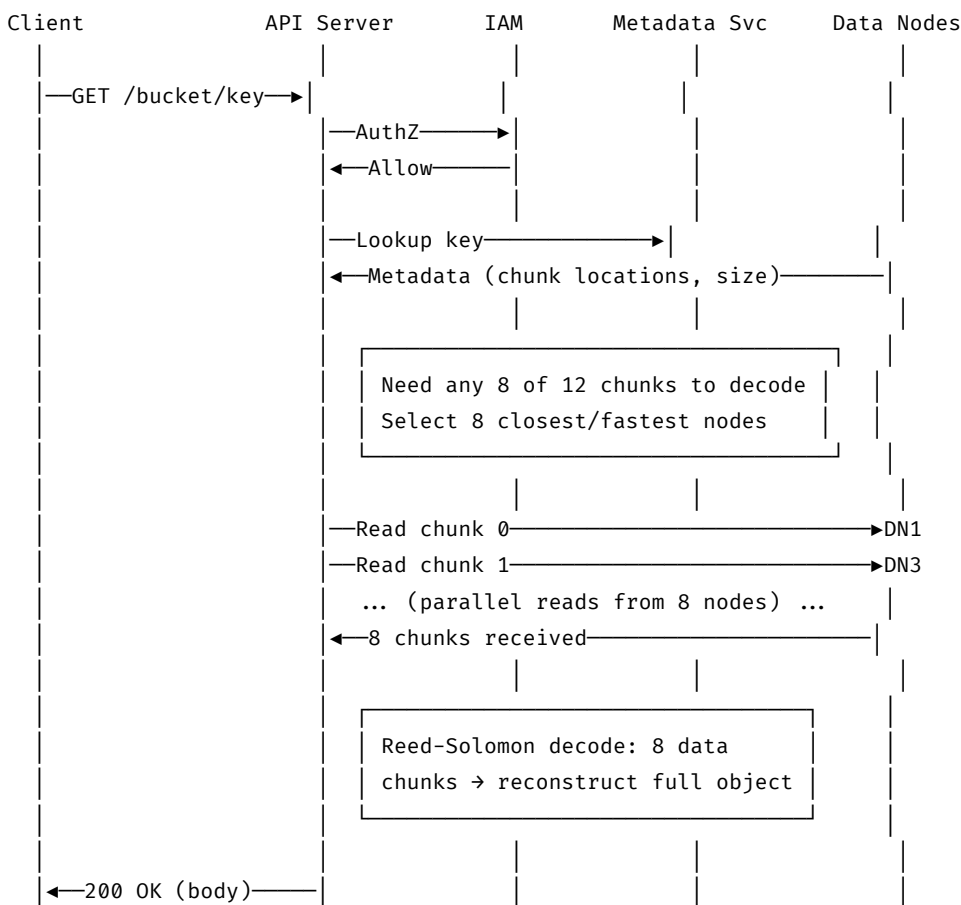
### 9.7.1 7.1 System Overview



### 9.7.2 7.2 Request Flow: PUT Object



### 9.7.3 7.3 Request Flow: GET Object



### 9.7.4 7.4 Component Breakdown

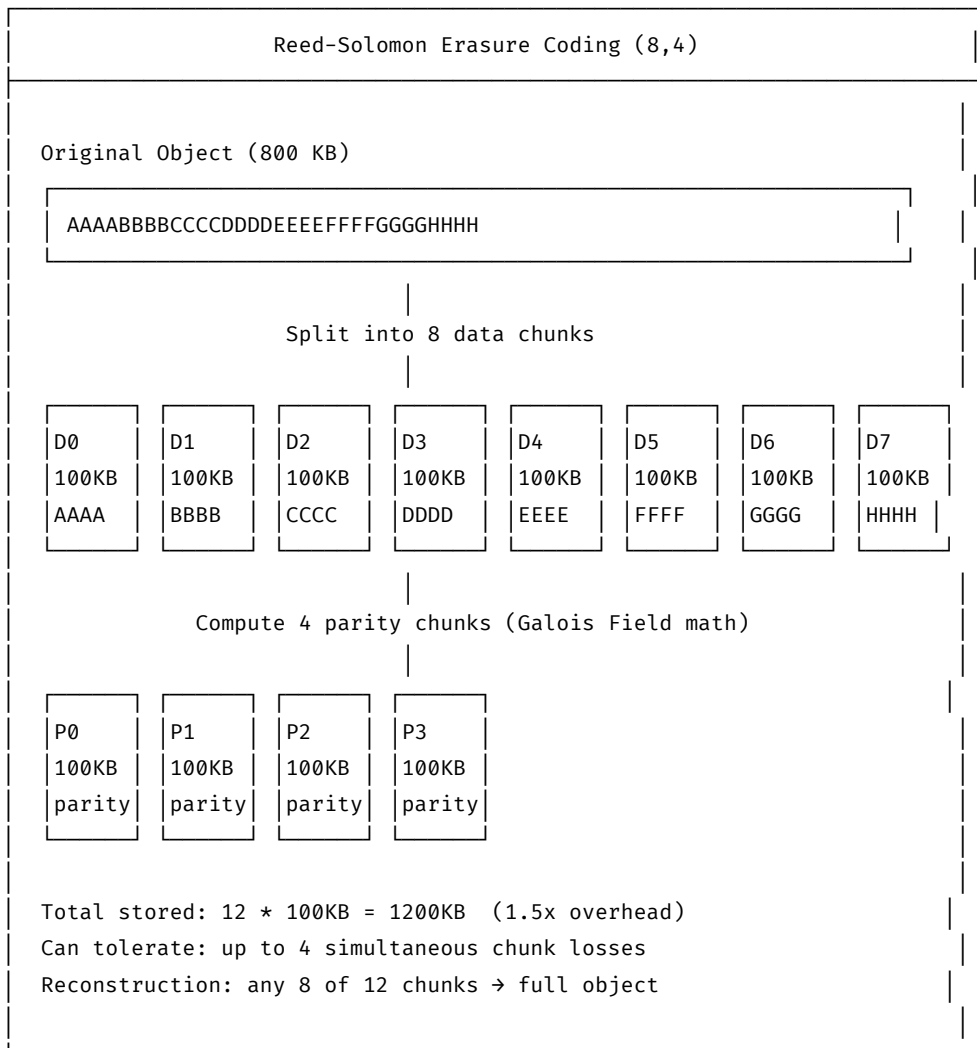
- **API Server:** Parse HTTP, auth routing, erasure encode/decode, stream data; Stateless?: Yes; Scale Strategy: Horizontal auto-scale
- **IAM / Auth Service:** Evaluate bucket policies, ACLs, IAM policies, signature verification; Stateless?: Yes; Scale Strategy: Cache policies locally
- **Metadata Service:** CRUD for object/bucket metadata, listing queries; Stateless?: Stateless tier; stateful store; Scale Strategy: Shard by bucket hash
- **Namespace Service:** Global bucket name uniqueness, bucket-to-region mapping; Stateless?: Stateless tier; backed by global DB; Scale Strategy: Small, replicated
- **Placement Service:** Map objects to data node sets using CRUSH-like algorithm; Stateless?: Yes (reads placement map); Scale Strategy: Replicate placement map
- **Data Service:** Route data reads/writes to correct data nodes; Stateless?: Yes; Scale Strategy: Horizontal
- **Data Node:** Store/retrieve erasure-coded chunks on raw disks; Stateless?: Stateful; Scale Strategy: Add nodes, rebalance
- **Garbage Collector:** Reclaim space from deleted objects, incomplete multipart uploads; Stateless?: Background daemon; Scale Strategy: Partition work by node
- **Data Compactor:** Compact fragmented extent files, reclaim gaps; Stateless?: Background daemon; Scale Strategy: Per-node
- **Integrity Scrubber:** Periodically verify chunk checksums, trigger repair; Stateless?: Background daemon; Scale Strategy: Per-node

## 9.8 8. Deep Dive: Core Components

### 9.8.1 8.1 Data Storage and Durability

#### 9.8.1.1 8.1.1 Erasure Coding: Reed-Solomon (8,4)

Erasure coding is the backbone of achieving 11 nines of durability at reasonable cost. Instead of storing 3 full copies (replication), we split data into  $k$  data chunks and compute  $m$  parity chunks. Any  $k$  of  $k+m$  chunks are sufficient to reconstruct the original data.



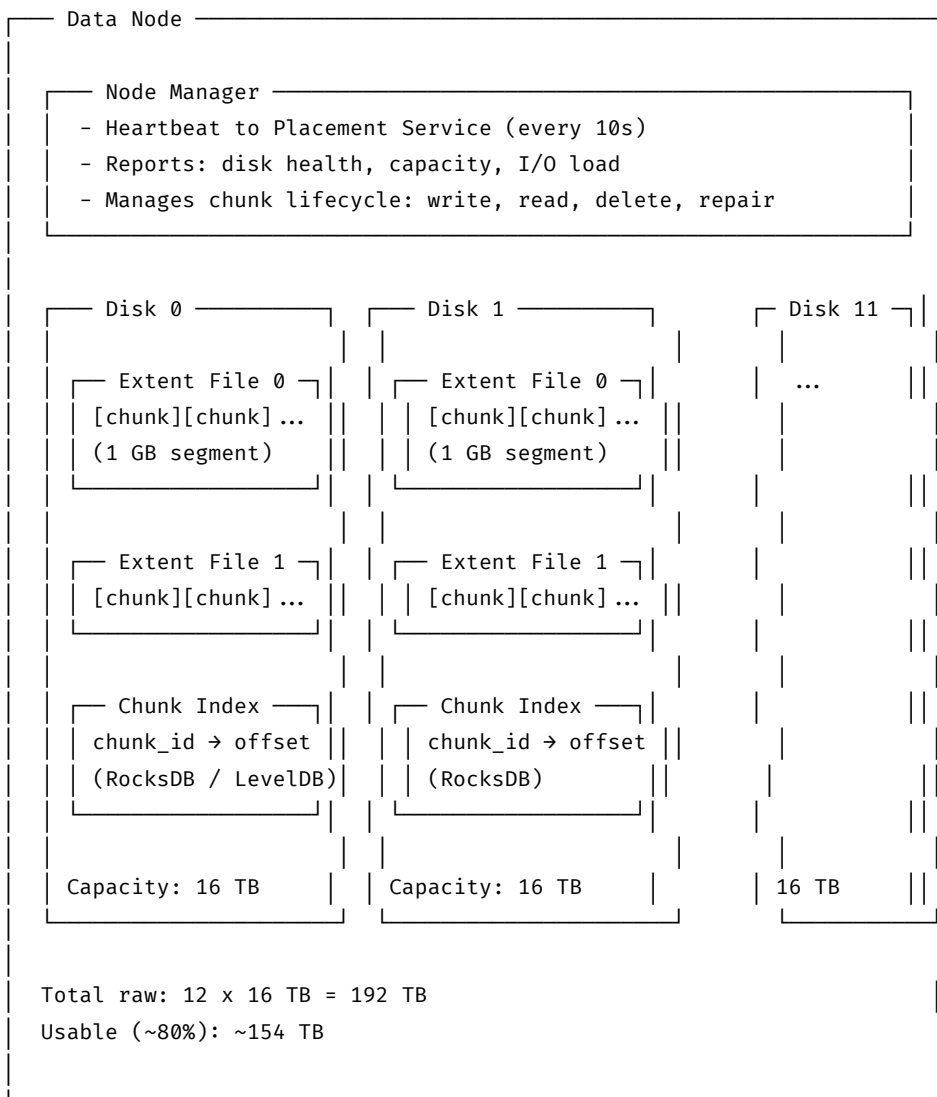
#### 9.8.1.2 8.1.2 Chunk Placement Across Failure Domains

Each of the 12 chunks must be placed on different failure domains to maximize durability:



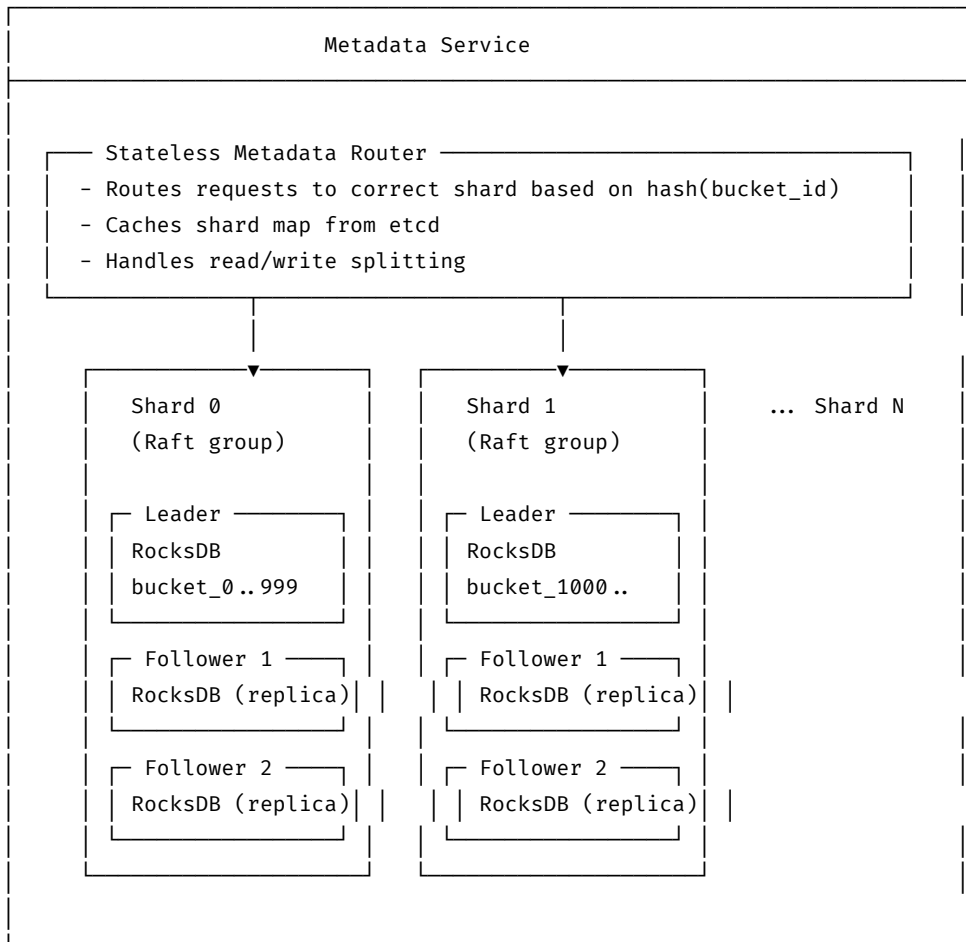


### 9.8.1.4 8.1.4 Data Node Internal Architecture



## 9.8.2 8.2 Metadata Service

### 9.8.2.1 8.2.1 Architecture



### 9.8.2.2 8.2.2 Key Design for Efficient Listing

Objects within a bucket are stored with a composite key that enables efficient prefix listing:

Key format in RocksDB:

```
[bucket_id (8 bytes)][object_key (variable)][version_id (16 bytes)]
```

Example entries (sorted lexicographically):

```
[bucket_42]["photos/2025/dec/img001.jpg"][v1] → metadata
[bucket_42]["photos/2025/dec/img002.jpg"][v1] → metadata
[bucket_42]["photos/2026/jan/img001.jpg"][v1] → metadata
[bucket_42]["photos/2026/jan/img001.jpg"][v2] → metadata (new version)
[bucket_42]["photos/2026/jan/img002.jpg"][v1] → metadata
[bucket_42]["videos/clip1.mp4"][v1] → metadata
```

List with prefix="photos/2026/" and delimiter="/":

```
→ Seek to [bucket_42]["photos/2026/"]
→ Scan until key no longer starts with "photos/2026/"
→ Group by delimiter: CommonPrefixes = ["photos/2026/jan/"]
→ Return grouped result
```

### 9.8.2.3 8.2.3 Metadata Versioning

Versioning States per Bucket:

DISABLED (default):

PUT key → overwrite single metadata entry

DELETE key → remove metadata entry (data GC'd later)

ENABLED:

PUT key → insert new version with generated version\_id

DELETE key → insert "delete marker" version

GET key → return latest non-delete-marker version

GET key?versionId=X → return specific version

SUSPENDED:

PUT key → overwrite the "null" version only

Existing versions preserved

|                                                        |                               |
|--------------------------------------------------------|-------------------------------|
| Key: "report.pdf", Versioning: ENABLED                 |                               |
| Version v3 (latest)                                    | — DELETE MARKER               |
| Version v2                                             | — 2.1 MB, modified 2026-03-15 |
| Version v1                                             | — 1.8 MB, modified 2026-01-10 |
| GET report.pdf → 404 (delete marker)                   |                               |
| GET report.pdf?versionId=v2 → 200 OK, returns v2       |                               |
| DELETE report.pdf?versionId=v3 → removes delete marker |                               |
| GET report.pdf → 200 OK, returns v2 (now latest)       |                               |

## 9.8.3 8.3 Data Placement and Rebalancing

### 9.8.3.1 8.3.1 CRUSH-Inspired Placement Algorithm

Instead of maintaining a massive lookup table mapping each object to its nodes, we use a deterministic algorithm (inspired by Ceph's CRUSH) that computes placement from the object ID and a small cluster map.

## Placement Algorithm

Input: object\_id (UUID), cluster\_map, replication\_config

Output: ordered list of 12 node IDs (one per chunk)

Step 1: Compute placement group (PG)

pg\_id = hash(object\_id) % num\_placement\_groups

(num\_placement\_groups = 4096, tuned per cluster size)

Step 2: CRUSH algorithm maps PG → 12 nodes

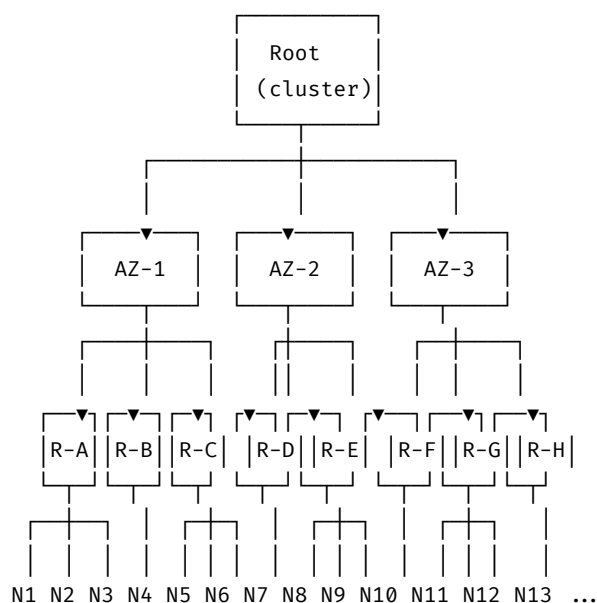
- Start at root of cluster topology tree
- For each chunk i (0..11):
  - Descend tree: root → AZ → rack → node
  - At each level, select bucket using hash(pg\_id, i, level)
  - Enforce failure domain separation:
    - chunks spread across ≥ 3 AZs, ≥ 6 racks
  - Skip nodes that are down or over-capacity

Step 3: Return [node\_1, node\_2, ..., node\_12]

Properties:

- Deterministic: same input always produces same output
- Minimal data movement when nodes added/removed
- No central lookup table needed
- $O(\log N)$  computation per object

### 9.8.3.2 8.3.2 Cluster Topology Tree



### 9.8.3.3 8.3.3 Rebalancing on Node Add/Remove

Before: 3 nodes, 9 PGs

| Node-A    | Node-B    | Node-C    |
|-----------|-----------|-----------|
| PG: 0,1,2 | PG: 3,4,5 | PG: 6,7,8 |
| (3 PGs)   | (3 PGs)   | (3 PGs)   |

After: 4 nodes, 9 PGs (add Node-D)

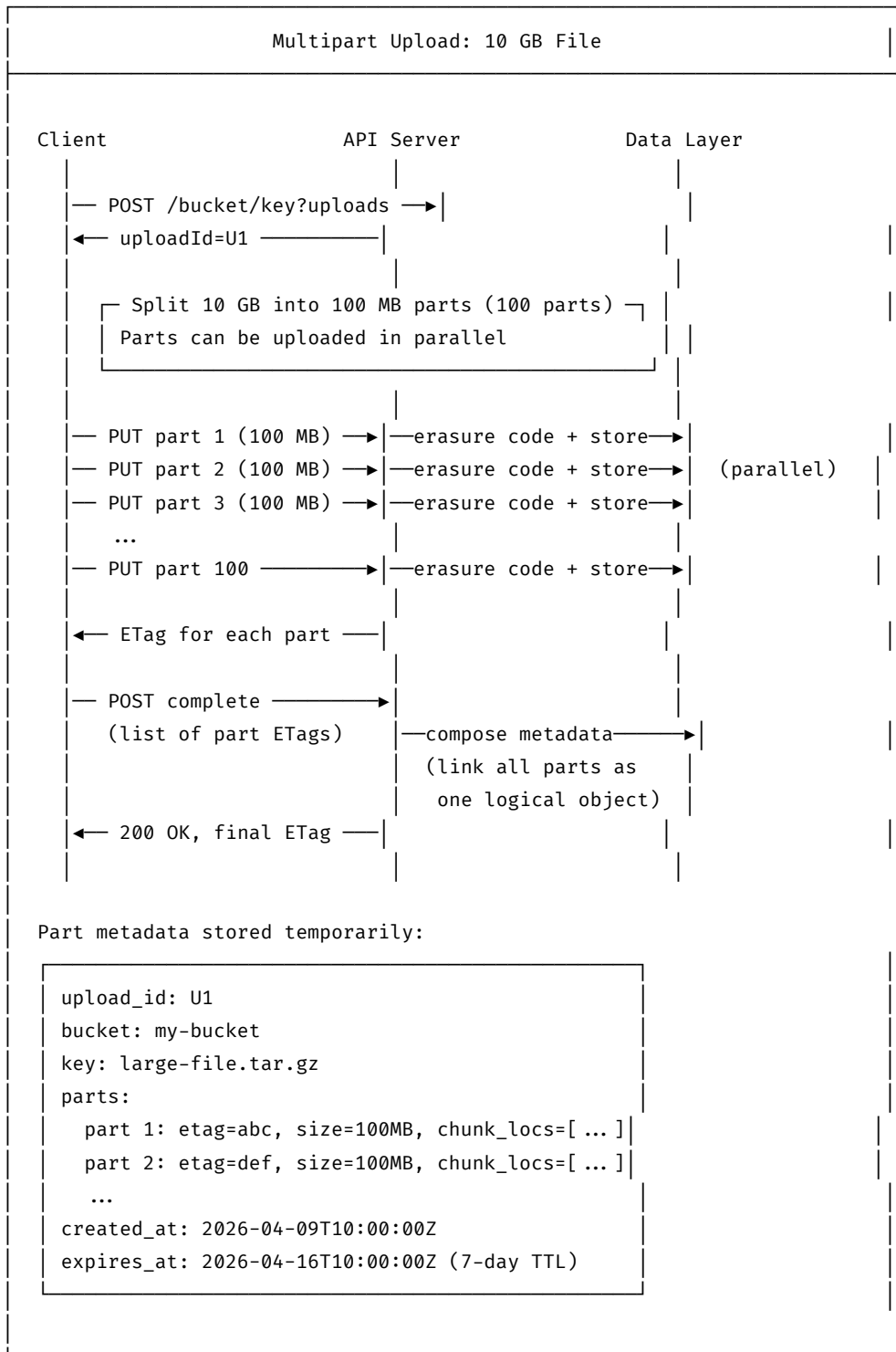
| Node-A  | Node-B  | Node-C  | Node-D    |
|---------|---------|---------|-----------|
| PG: 0,1 | PG: 3,4 | PG: 6,7 | PG: 2,5,8 |
| (2 PGs) | (2 PGs) | (2 PGs) | (3 PGs)   |

Key properties:

- Only  $\sim 1/N$  of data moves when adding Nth node (minimal disruption)
- Rebalancing is background, throttled to avoid impacting live traffic
- Placement map version is incremented; old and new maps coexist during transition
- Reads try new placement first, fall back to old placement during migration

## 9.8.4 8.4 Multipart Upload

### 9.8.4.1 8.4.1 Flow



### 9.8.4.2 8.4.2 Failure Handling in Multipart Upload

Scenario 1: Part upload fails mid-transfer

- Client retries that specific part (idempotent by part number)
- Server discards incomplete chunk data
- Same part number can be re-uploaded any number of times

Scenario 2: Client crashes, upload never completed

- Incomplete upload metadata has TTL (default 7 days)
- Lifecycle policy: `AbortIncompleteMultipartUpload` (configurable days)
- GC daemon scans for expired uploads, deletes orphaned chunks

Scenario 3: Complete request fails after partial metadata commit

- Retry the complete request (idempotent with same uploadId + parts)
- If parts list differs, reject with error

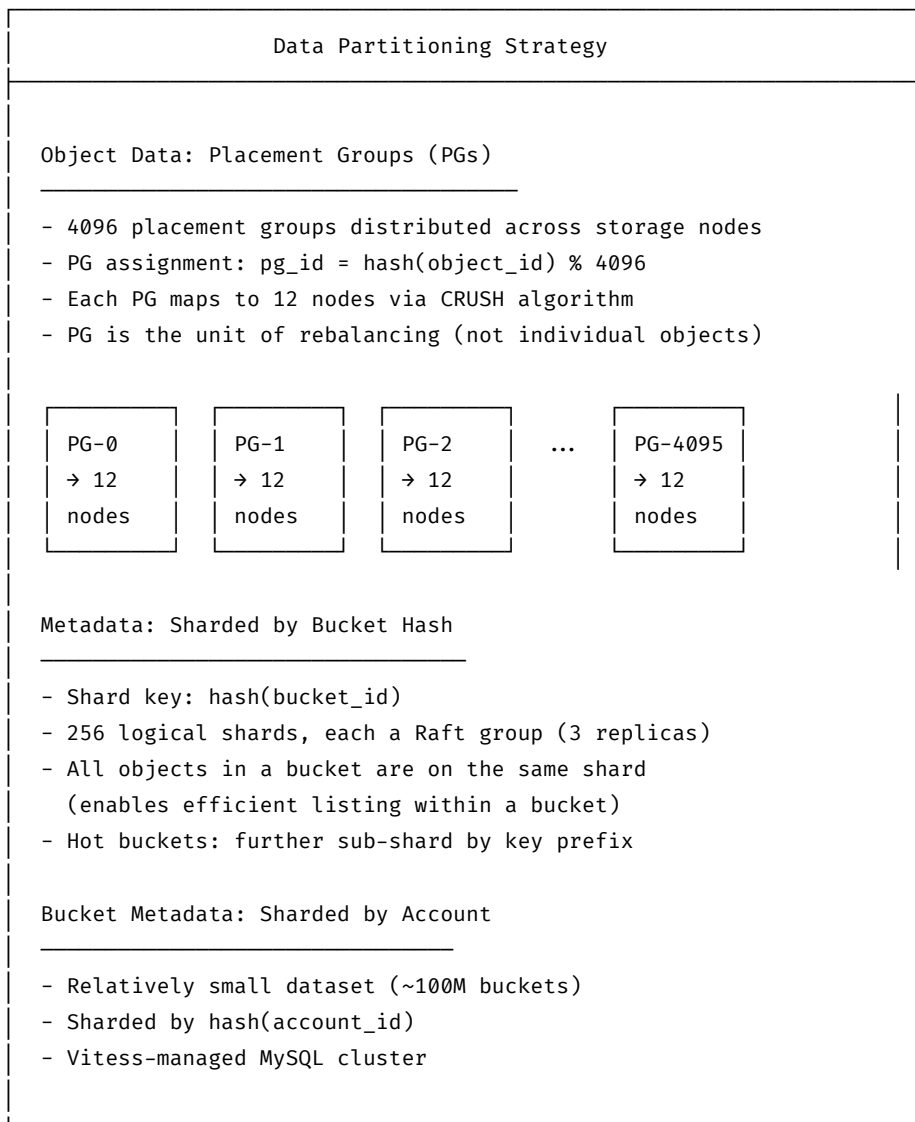
Scenario 4: Part uploaded to a node that crashes

- Part data is erasure-coded across 12 nodes
  - If < 5 nodes for that part fail, data is recoverable
  - Scrubber detects missing chunks, triggers repair from parity
- 

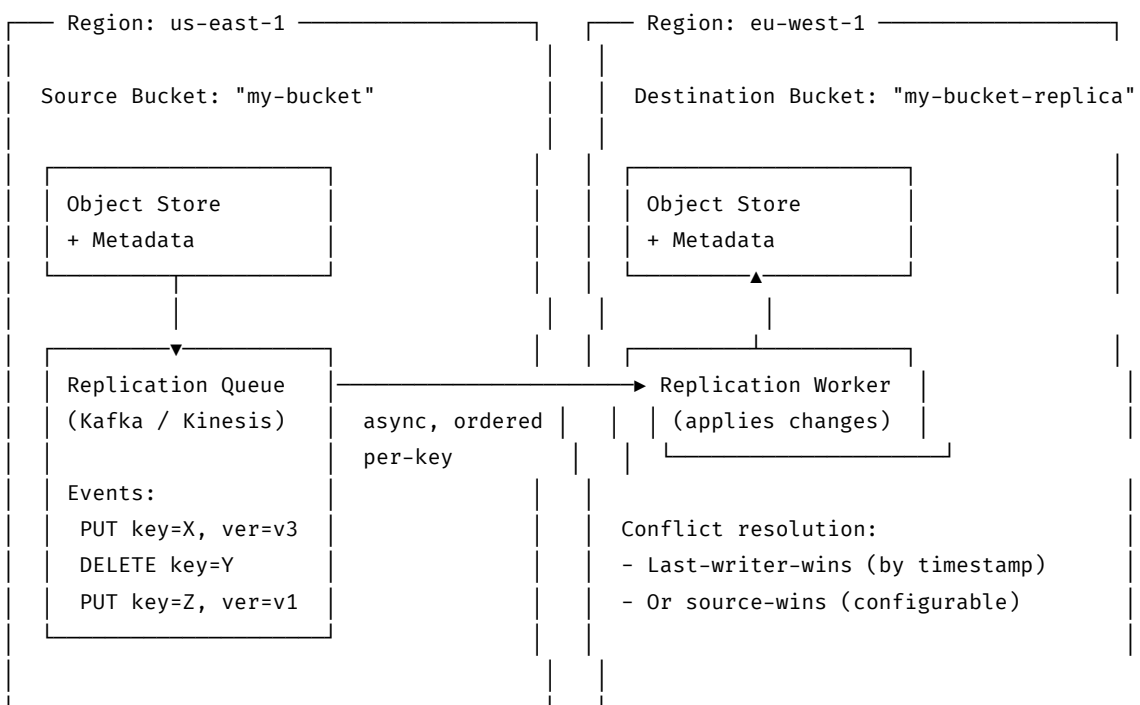
## 9.9 9. Data Partitioning and Sharding



### 9.9.1 9.1 Data Layer Partitioning



### 9.9.2 9.2 Cross-Region Replication Architecture

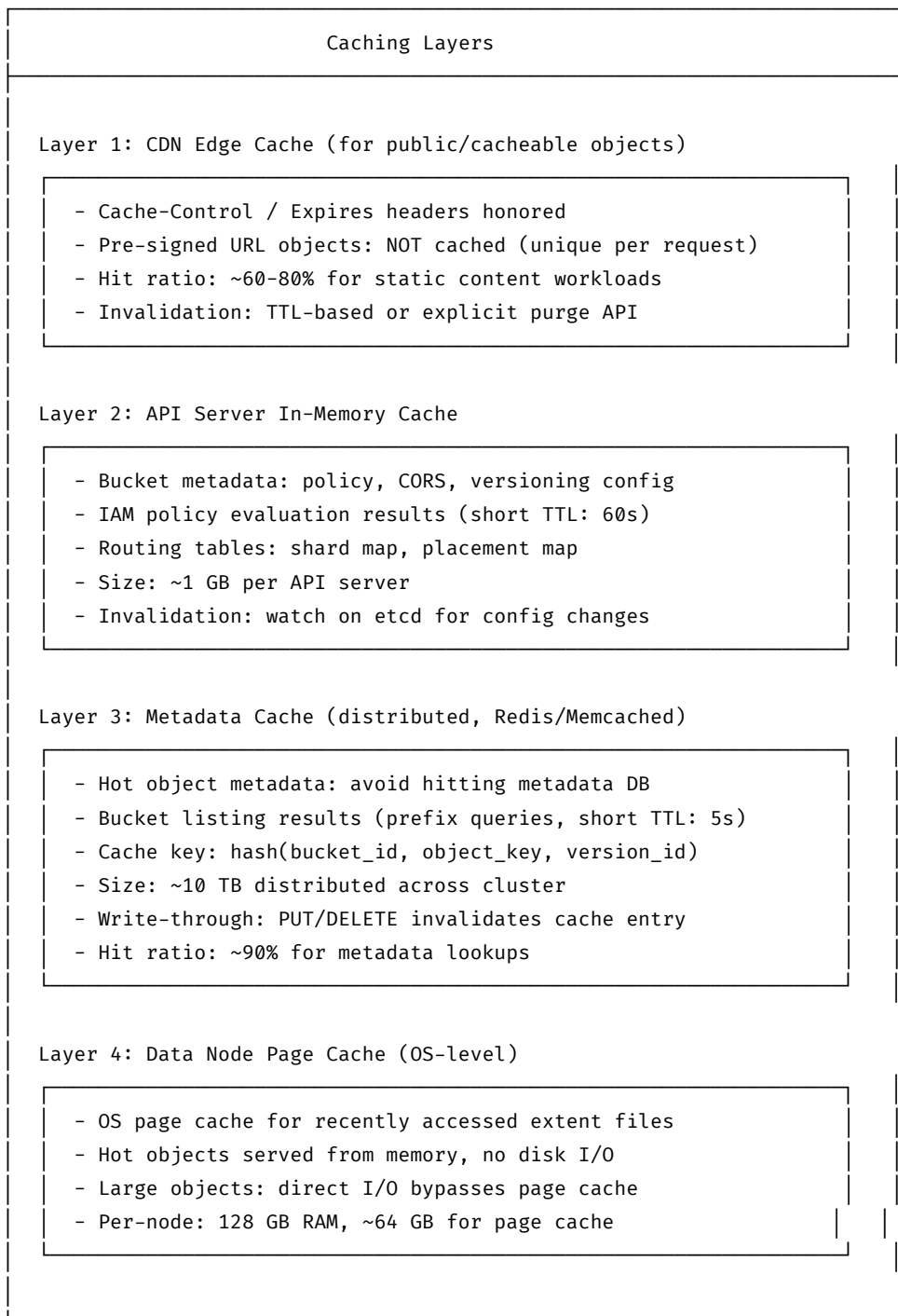


Replication metrics:

- Replication lag: p50 < 15 min, p99 < 1 hour
- Bandwidth: throttled to avoid impacting primary traffic
- Filtering: replicate only objects matching prefix/tag rules

## 9.10 10. Caching Strategy

### 9.10.1 10.1 Multi-Layer Caching Architecture



### 9.10.2 10.2 Cache Invalidation Strategy

Write path (PUT/DELETE):

1. Write to metadata store (source of truth)
2. Invalidate metadata cache entry (delete from Redis)
3. Subsequent reads go to metadata store (miss), then populate cache

Strong consistency requirement:

- No stale reads after a successful PUT
  - Solution: metadata cache uses write-through invalidation
  - Read path: check cache → if miss, read from metadata store leader
  - For listing: short TTL (5s) since listings are eventually consistent for performance (configurable)
- 

## 9.11 11. Replication and Consistency

### 9.11.1 11.1 Strong Read-After-Write Consistency

Amazon S3 achieved strong read-after-write consistency in December 2020. Here is how this can be designed.

### Strong Read-After-Write Consistency Protocol

Approach: "Witness" protocol on metadata

PUT object flow:

1. Write data chunks to data nodes
2. Write metadata to metadata store (Raft-based, linearizable)
3. Metadata write commits only when Raft leader + majority ACK
4. Return success to client only after metadata commit

GET object flow:

1. Read metadata from Raft leader (linearizable read)
2. Leader confirms it is still leader (lease check / read index)
3. Return latest committed version

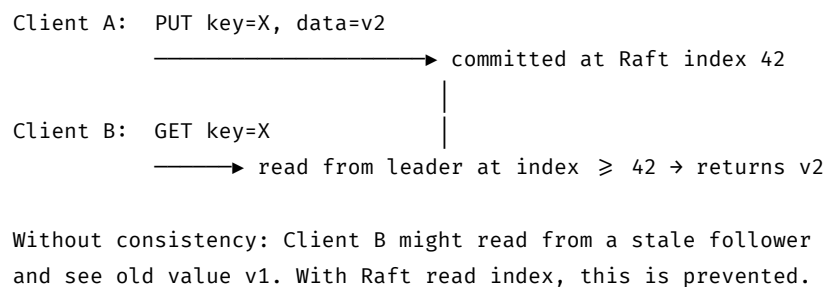
Why this guarantees read-after-write:

- PUT returns only after metadata is committed in Raft
- GET always reads from leader with confirmed leadership
- Therefore, any GET after a successful PUT sees the new version

Performance optimization:

- Follower reads with "read index" (avoid overloading leader)
- Follower asks leader for current commit index
- Waits until follower's applied index  $\geq$  leader's commit index
- Then serves the read locally

Timeline:

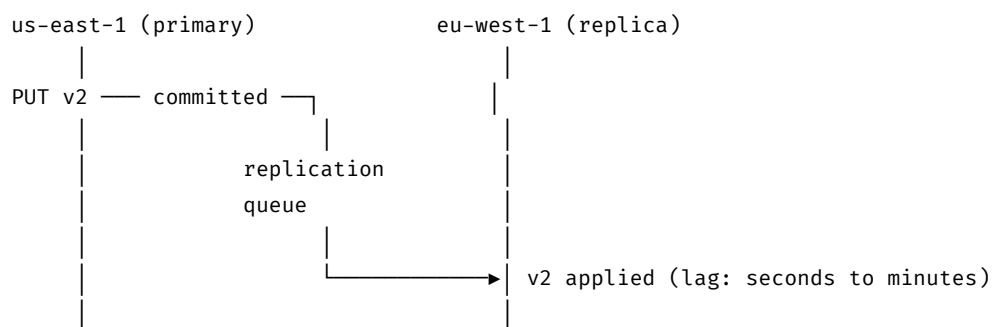


### 9.11.2 11.2 Consistency Levels by Operation

| Operation            | Consistency Level | Implementation           |
|----------------------|-------------------|--------------------------|
| PUT (new object)     | Strong R-A-W      | Raft commit before ACK   |
| PUT (overwrite)      | Strong R-A-W      | Raft commit before ACK   |
| DELETE               | Strong R-A-W      | Delete marker via Raft   |
| GET (by key)         | Strong            | Leader read / read index |
| HEAD (by key)        | Strong            | Leader read / read index |
| LIST (prefix)        | Strong            | Leader read / read index |
| Cross-region GET     | Eventual          | Async replication lag    |
| Bucket config update | Strong            | Raft on bucket metadata  |

### 9.11.3 11.3 Cross-Region Consistency

Cross-region replication is always asynchronous (eventual consistency):



During lag window:

- GET in us-east-1 → v2 (strong)
- GET in eu-west-1 → v1 (stale, eventually v2)

For applications needing strong cross-region consistency:

- Route all reads/writes to primary region
- Or use multi-region Raft (at significant latency cost)

## 9.12 12. Fault Tolerance and Failure Handling

**9.12.1 12.1 Failure Modes and Recovery**

| Failure Mode                  | Detection                                               | Recovery                                                                                          |
|-------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Single disk failure           | SMART monitoring, I/O errors, CRC mismatch              | Reconstruct chunks from remaining 11 chunks on other disks (Reed-Solomon)                         |
| Node failure                  | Heartbeat timeout (30s), health checks                  | Mark node down in placement map; reads route to other 11 nodes; repair copies chunks to new nodes |
| Rack failure (power/network)  | Multiple node failures in same rack                     | 4 chunks max per rack; remaining 8+ chunks on other racks suffice                                 |
| AZ failure                    | Network partition or DC-wide outage                     | Chunks spread across 3 AZs; losing 1 AZ = losing ~4 chunks; 8 remaining suffice                   |
| Region failure                | All AZs down                                            | Cross-region replication; failover to replica region (RPO: replication lag)                       |
| Metadata node failure         | Raft leader election timeout                            | Automatic failover to Raft follower; new leader elected in < 10 seconds                           |
| Bit rot / silent corruption   | Background scrubber checksums every chunk every 30 days | CRC mismatch detected; chunk reconstructed from parity and rewritten                              |
| Network partition between AZs | Split-brain detection via Raft consensus                | Raft requires majority; minority partition becomes read-only or unavailable                       |

### 9.12.2 12.2 Data Scrubbing and Integrity

#### Background Integrity Scrubber

Purpose: Detect and repair silent data corruption (bit rot)

Process (per data node):

1. Iterate through all chunks on each disk
2. For each chunk:
  - a. Read chunk data
  - b. Compute CRC32C checksum
  - c. Compare with stored checksum (in chunk index)
  - d. If mismatch:
    - Mark chunk as corrupt
    - Notify repair service
    - Repair service reads k healthy chunks from peers
    - Reed-Solomon decode to reconstruct corrupt chunk
    - Write repaired chunk, update checksum
3. Throttle I/O to 10% of disk bandwidth (low priority)
4. Complete full scan every 30 days

Scrubbing rate per node:

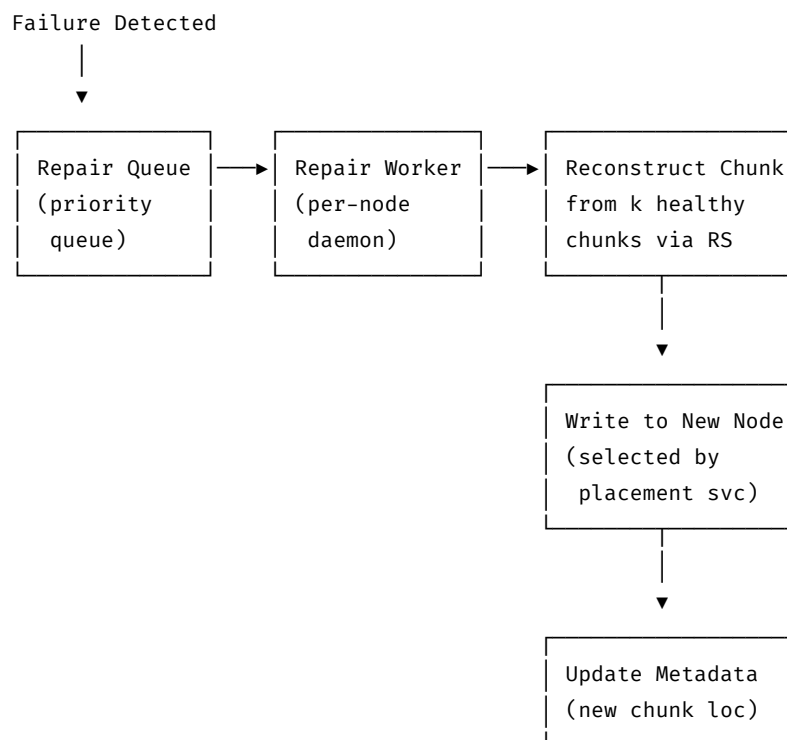
- 154 TB usable / 30 days = ~5.1 TB/day = ~60 MB/s continuous
- At 10% throttle: ~6 MB/s background I/O

Metrics tracked:

- Chunks scanned per day
- Corrupt chunks detected (should be near zero)
- Repair success rate
- Repair latency (time from detection to fixed)



### 9.12.3 12.3 Automatic Repair Pipeline



#### Priority levels:

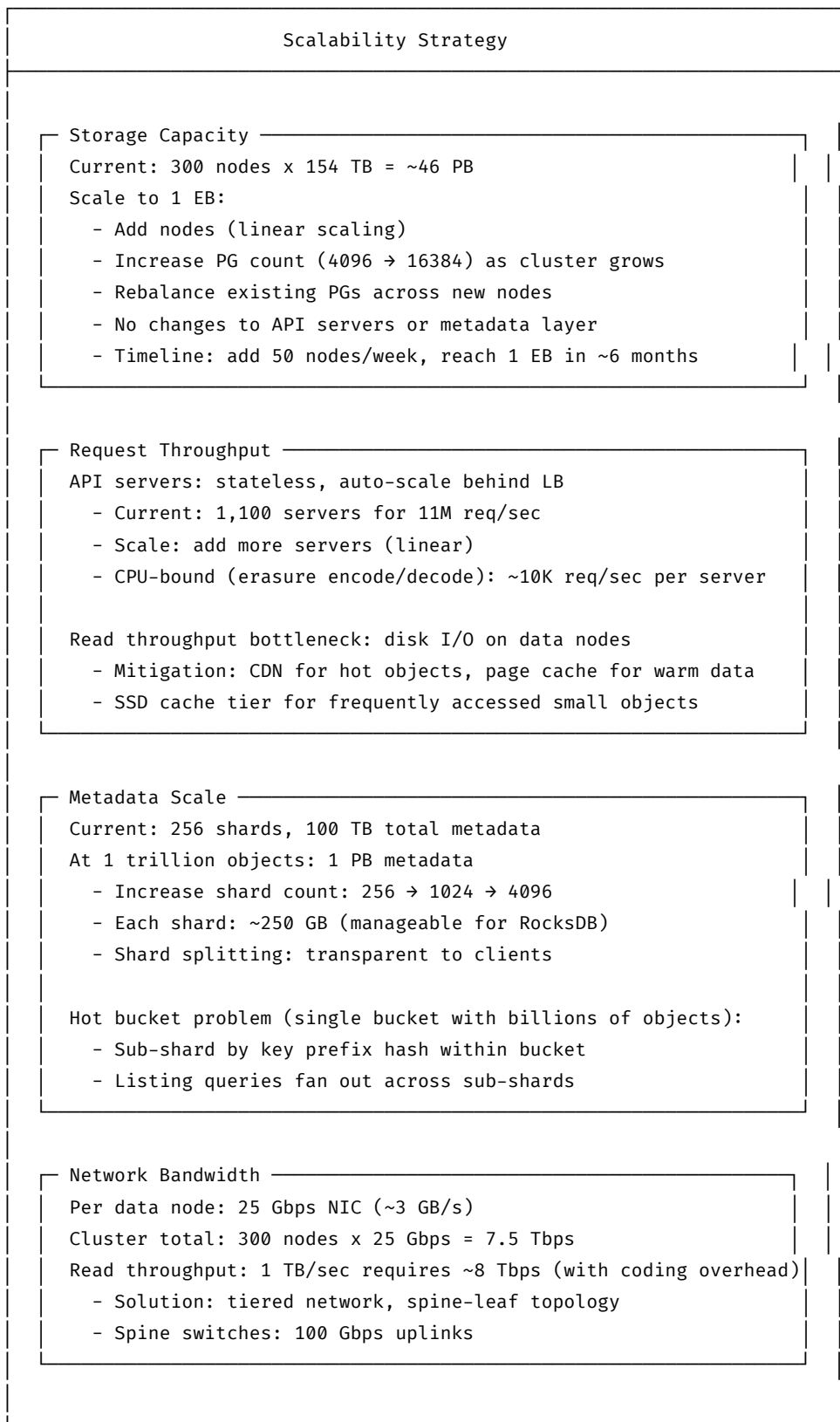
- CRITICAL: Object has  $\leq k$  healthy chunks (at risk of data loss)
- HIGH: Object has  $k+1$  healthy chunks
- MEDIUM: Object has  $k+2$  healthy chunks
- LOW: Object has  $k+3$  healthy chunks (still fully redundant)

#### Repair throughput:

- Single disk failure (16 TB): ~4 hours at 1 GB/s network per node
- Node failure (192 TB): ~8 hours with parallel repair from 100+ peers
- Goal: repair before a second correlated failure

## 9.13 13. Scalability

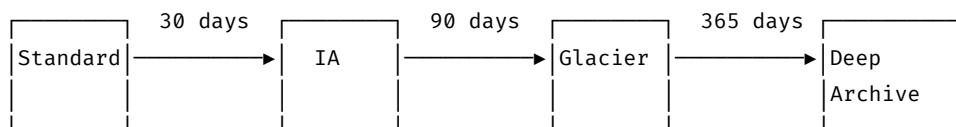
### 9.13.1 13.1 Scaling Dimensions



### 9.13.2 13.2 Storage Class Tiering

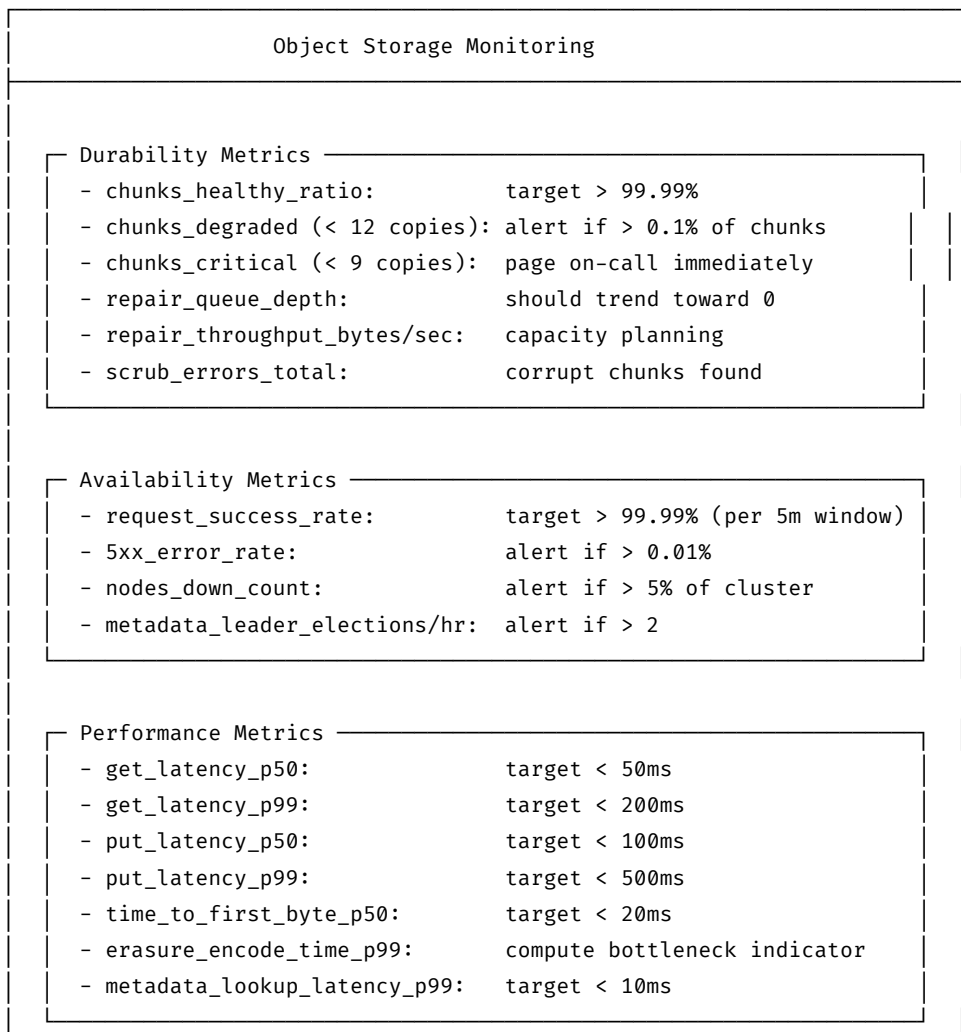
| Storage Classes and Tiering |                 |                |                     |                    |                       |
|-----------------------------|-----------------|----------------|---------------------|--------------------|-----------------------|
| Class                       | Media           | Erasure Scheme | Latency (GET p50)   | Cost/GB /month     | Durability            |
| Standard                    | HDD+SSD cache   | 8+4 (3AZ)      | <50ms               | \$0.023            | 11 nines              |
| Infrequent Access (IA)      | HDD             | 8+4 (3AZ)      | <50ms               | \$0.0125 +retriev. | 11 nines              |
| One-Zone IA                 | HDD             | 8+4 (1AZ)      | <50ms               | \$0.010            | 11 nines*<br>*same AZ |
| Glacier (Archive)           | HDD (dense)     | 12+6 (cold)    | 1-5 min (restore)   | \$0.004            | 11 nines              |
| Deep Archive                | Tape + cold HDD | 12+6 (offline) | 12-48 hrs (restore) | \$0.00099          | 11 nines              |

Lifecycle Transition Example:



## 9.14 14. Monitoring and Observability

### 9.14.1 14.1 Key Metrics Dashboard



**Capacity Metrics**

- total\_storage\_used\_bytes: capacity planning
- storage\_utilization\_percent: alert if > 80%
- objects\_total: trend tracking
- daily\_ingest\_bytes: growth rate
- gc\_reclaimed\_bytes/day: space reclamation rate
- disk\_utilization per node: hotspot detection

**Operational Metrics**

- rebalance\_progress\_percent: during node add/remove
- multipart\_uploads\_incomplete: GC candidate count
- cross\_region\_replication\_lag: SLA tracking
- api\_server\_cpu\_utilization: auto-scale trigger (> 70%)

**Alerting Tiers:**

|              |                                                     |
|--------------|-----------------------------------------------------|
| P0 (PAGE)    | chunks_critical > 0, 5xx_rate > 1%, node_down > 20% |
| P1 (URGENT)  | durability < 99.9999%, repair_queue > 1M            |
| P2 (WARNING) | storage_util > 80%, replication_lag > 1h            |
| P3 (INFO)    | scrub_errors > 0, daily_ingest exceeds forecast     |

**9.14.2 14.2 Distributed Tracing**

Trace: PUT /v1/my-bucket/large-file.bin (multipart, 1 GB)

|                                        |                   |
|----------------------------------------|-------------------|
| Span 1: api_server.handle_put          | [0ms — 1200ms]    |
| Span 1.1: auth.evaluate_policy         | [2ms — 15ms]      |
| Span 1.2: namespace.resolve_bucket     | [16ms — 22ms]     |
| Span 1.3: placement.get_nodes          | [23ms — 35ms]     |
| Span 1.4: data.eraser_encode           | [36ms — 180ms]    |
| Span 1.5: data.write_chunks (parallel) | [181ms — 1100ms]  |
| Span 1.5.1: write_chunk_to_dn1         | [181ms — 650ms]   |
| Span 1.5.2: write_chunk_to_dn3         | [181ms — 800ms]   |
| Span 1.5.3: write_chunk_to_dn5         | [181ms — 950ms]   |
| ... (12 parallel writes)               |                   |
| Span 1.5.12: write_chunk_to_dn23       | [181ms — 1100ms]  |
| Span 1.6: metadata.store_object        | [1101ms — 1180ms] |
| Span 1.7: response.send                | [1181ms — 1200ms] |

Attributes: bucket=my-bucket, key=large-file.bin, size=1073741824,  
storage\_class=STANDARD, erasure\_scheme=8+4, chunks=12,  
slowest\_node=dn23 (919ms)

## 9.15 15. Trade-offs and Design Decisions

### 9.15.1 15.1 Key Trade-offs

#### Trade-off 1: Erasure Coding vs Replication

We chose: Erasure Coding (Reed-Solomon 8,4)

Pros:

- + 50% less storage cost (1.5x vs 3x)
- + Higher fault tolerance (4 vs 2 failures)
- + Better durability mathematically

Cons:

- Higher CPU for encode/decode
- Higher read latency (need 8 chunks vs 1 replica)
- More complex repair logic
- Small objects: overhead disproportionate

Mitigation for cons:

- SIMD-optimized erasure coding libraries (ISA-L): ~10 GB/s per core
- Read from closest 8 nodes, latency dominated by slowest
- Small objects (<1 MB): batch into larger extent files
- Hot objects: replicate to SSD cache tier for fast reads

#### Trade-off 2: Strong vs Eventual Consistency

We chose: Strong read-after-write (within region)

Pros:

- + Simpler application logic
- + No stale reads after writes
- + Eliminates class of subtle bugs
- + Matches S3's actual behavior

Cons:

- Higher metadata read latency (must go to leader or use read index)
- Cannot cache metadata aggressively
- Cross-region: still eventual (acceptable)

Why it works at scale:

- Metadata is small; Raft overhead is microseconds per operation
- Read index optimization avoids overloading leader
- 90%+ of reads served from metadata cache (cache invalidation ensures consistency)

#### Trade-off 3: Flat Namespace vs Hierarchical

We chose: Flat namespace (keys are opaque strings)

Pros:

- + Simpler metadata model (just KV store)
- + No rename overhead (rename = copy+delete in hierarchical FS)
- + Infinite nesting depth
- + No directory inode overhead

Cons:

- "Directories" are simulated with prefix + delimiter
- LIST with delimiter has  $O(n)$  scan cost within prefix range
- No atomic "directory rename"

S3 uses this approach:

"photos/2026/jan/img.jpg" is just a key, not a path

#### Trade-off 4: Small Object Handling Strategy

Problem: Erasure coding 8+4 on a 1 KB object creates 12 x 125-byte chunks. The per-chunk metadata overhead dominates.

Solution: Object packing

- Small objects (< 256 KB) are batched into "packed extents"
- A packed extent is ~1 MB of aggregated small objects
- The packed extent is erasure-coded as one unit
- Metadata stores: extent\_id + offset within extent

Trade-off:

- + Amortizes erasure coding overhead
- + Reduces chunk count dramatically for small-object workloads
- DELETE of packed object: mark tombstone, space reclaimed on compaction (delayed reclamation)
- Read amplification: must read 8 chunks even for tiny object (mitigated by extent-level caching)

#### Trade-off 5: Centralized vs Decentralized Placement

We chose: CRUSH-like algorithmic placement (decentralized)

Centralized (lookup table):

- Need massive table: 100B objects x 12 locations = 1.2 trillion entries
- Bottleneck at lookup service
- Hot spot for all operations
- Complex to keep consistent

Algorithmic (CRUSH):

- $O(\log N)$  computation
- Cluster map is tiny (KB)
- Any node can compute placement independently
- Minimal data movement on cluster changes
- Harder to implement perfectly balanced load

## 9.16 16. Interview Deep-Dive Questions

### 9.16.1 Q1: How do you achieve 11 nines of durability?

**Answer:** We combine multiple strategies. First, **erasure coding (Reed-Solomon 8,4)**: each object is split into 8 data chunks and 4 parity chunks, stored across 12 different nodes in at least 3 availability zones. The object survives up to 4 simultaneous chunk losses. Second, **failure domain separation**: chunks are spread across different disks, nodes, racks, and AZs, so correlated failures (rack power loss, AZ outage) don't take out enough chunks. Third, **background data scrubbing**: every chunk is checksummed and verified every 30 days, catching silent bit rot before it accumulates. Fourth, **rapid repair**: when a disk or node fails, the repair pipeline reconstructs affected chunks within hours, well before a second failure is likely. The mathematical probability of losing an object under these conditions is approximately  $10^{-23}$  per year, far exceeding the 11-nines target.



### 9.16.2 Q2: How does erasure coding work, and why prefer it over replication?

**Answer:** Erasure coding treats data as a polynomial over a Galois field ( $GF(2^8)$ ). The object is split into  $k=8$  data chunks. We evaluate this polynomial at  $m=4$  additional points to produce 4 parity chunks. Any 8 of the 12 total chunks can reconstruct the original data via polynomial interpolation.

Compared to triple replication: - **Cost:** 1.5x storage overhead vs 3x—50% cheaper at petabyte scale (savings of ~\$15K/PB/month). - **Fault tolerance:** Survives 4 failures vs 2. - **Durability:** Higher mathematical durability due to more independent failure events needed. - **Downsides:** Reads require fetching 8 chunks and decoding (vs reading any 1 replica). Writes require encoding. Both are mitigated by SIMD-optimized libraries (Intel ISA-L achieves ~10 GB/s encode throughput per core) and caching hot objects in replicated form.

---

### 9.16.3 Q3: How do you handle consistency for read-after-write?

**Answer:** We use Raft-based consensus for the metadata layer. Every PUT commits metadata to a Raft group (leader + majority of followers). The PUT returns success only after Raft commit. For reads, we use one of two approaches: (1) always read from the Raft leader, which is guaranteed to have the latest committed state, or (2) use the “read index” optimization where a follower asks the leader for the current commit index, waits until its local state catches up, then serves the read locally. This guarantees that any GET issued after a successful PUT returns the new version. For cache invalidation, we use write-through: every PUT/DELETE invalidates the corresponding cache entry, so subsequent reads either hit the fresh cache or go to the metadata store.

---

### 9.16.4 Q4: How do you implement storage classes (hot/warm/cold/archive)?

**Answer:** Storage classes differ in three dimensions: media type, erasure coding scheme, and access latency. **Standard** uses HDD with SSD caching, 8+4 across 3 AZs, sub-50ms latency. **Infrequent Access** uses the same scheme but charges retrieval fees, encouraging users to self-select. **Glacier** uses dense HDD (high capacity, slower seek), 12+6 coding for cheaper storage, and requires an explicit restore request (data is “thawed” from cold pools to a temporary staging area, available in minutes). **Deep Archive** may use tape or offline HDD arrays, with restore times of 12-48 hours. Lifecycle policies automate transitions: metadata is updated with the new storage class, and background workers physically migrate data between storage tiers, re-encoding with the target erasure scheme.

---

### 9.16.5 Q5: How does multipart upload handle failures mid-upload?

**Answer:** Each part is uploaded independently and erasure-coded immediately upon receipt. The server stores part metadata (part number, ETag, chunk locations) in a temporary upload record keyed by the `uploadId`. If a part fails, the client simply retries that specific part number (idempotent—same part number overwrites the previous attempt). If the client crashes and never calls “complete,” the incomplete upload is subject to a TTL (default 7 days) or a lifecycle rule (`AbortIncompleteMultipartUpload`). The garbage collector periodically scans for expired incomplete uploads, deletes their orphaned chunks, and removes the upload metadata. The “complete” call itself is idempotent: calling it again with the same part list returns the same result.

---

### 9.16.6 Q6: How do you handle the “hot bucket” problem (one bucket with billions of objects)?

**Answer:** If all objects in a bucket map to a single metadata shard, that shard becomes a bottleneck. We address this by **sub-sharding hot buckets**: when a shard’s load exceeds a threshold, we split it by key prefix hash. For example, a bucket with key prefix “a-m” goes to shard A, “n-z” to shard B. Listing queries fan out across sub-shards and merge results (similar to scatter-gather). The sub-shard boundaries are stored in the shard routing table (etcd). This is transparent to the client. Additionally, rate limiting per bucket prevents a single tenant from overwhelming the system.

---

### 9.16.7 Q7: How do you design the garbage collection system?

**Answer:** GC operates in several layers. (1) **Deleted objects**: a DELETE marks the object with a tombstone in metadata (or inserts a delete marker if versioned). A background GC worker periodically scans for tombstoned metadata older than a grace period (e.g., 24 hours to allow for replication and rollback), then issues chunk deletion commands to data nodes. (2) **Orphaned chunks**: if metadata commits but some chunk writes fail, the chunks that were written become orphans. The scrubber detects chunks not referenced by any metadata entry and queues them for deletion. (3) **Incomplete multipart uploads**: as described in Q5, TTL-based cleanup. (4) **Compaction**: when packed extents have many deleted small objects, the compactor rewrites live objects into new extents, releasing the old ones. GC is always throttled to avoid impacting live traffic.

---

### 9.16.8 Q8: How do you handle encryption at rest?

**Answer:** We support three modes. **SSE-S3**: the service manages keys; each object is encrypted with a unique data encryption key (DEK), and the DEK is encrypted with a master key (envelope encryption). DEKs are stored alongside object metadata. Master keys are rotated periodically; re-encryption is lazy (re-encrypt DEK, not re-encrypt data). **SSE-KMS**: the customer provides a KMS key ID. The service calls KMS to generate a DEK for each object. The encrypted DEK is stored in metadata. On GET, the service calls KMS to decrypt the DEK, then decrypts the object. **SSE-C**: the customer provides the encryption key in the request header. The service encrypts/decrypts in memory, never stores the key. Encryption happens before erasure coding: the plaintext is encrypted, then the ciphertext is erasure-coded and stored.

---

### 9.16.9 Q9: How does listing with prefix and delimiter work efficiently?

**Answer:** Objects in each metadata shard are stored in a RocksDB instance with the key format [bucket\_id][object\_key][version\_id], sorted lexicographically. A prefix query translates to a RocksDB range scan: seek to [bucket\_id][prefix] and iterate until the key no longer starts with the prefix. The delimiter (typically “/”) is applied in-memory: keys between two delimiters are grouped into “common prefixes” (virtual directories). Pagination uses a “marker” (the last key returned), and the next request seeks to just after the marker. For hot buckets sub-sharded across multiple metadata nodes, the API server sends the listing query to all relevant sub-shards in parallel, merges the sorted results, and applies the max-keys limit. This is efficient because RocksDB’s LSM-tree structure provides fast range scans.

---

#### 9.16.10 Q10: What happens when you need to add a new availability zone?

**Answer:** Adding an AZ is a multi-step operation. (1) Deploy new data nodes in the new AZ and register them in the cluster topology tree (under the new AZ node). (2) Update the CRUSH map to include the new AZ. (3) The placement algorithm now considers the new AZ for new objects. (4) For existing objects, a background rebalancer gradually moves a subset of chunks to the new AZ to improve distribution. Because CRUSH is deterministic, changing the map causes only a fraction of placement groups to shift—roughly proportional to the new AZ’s weight relative to total cluster weight. (5) During the transition, the system maintains dual placement maps (old and new); reads try the new placement first, fall back to old. (6) Once migration completes, the old map is retired. The entire process is online with no downtime.

---

#### 9.16.11 Q11: How do you implement pre-signed URLs securely?

**Answer:** A pre-signed URL embeds the access credentials and a cryptographic signature directly in the URL query parameters. The server generates it by: (1) constructing a “string to sign” that includes the HTTP method, bucket, key, expiration timestamp, and any conditions (e.g., content-type); (2) computing an HMAC-SHA256 signature using the user’s secret key (or a service-managed signing key); (3) encoding the signature and credential info into query params. When a request arrives with a pre-signed URL, the API server: (a) checks the expiration—reject if expired; (b) reconstructs the string to sign from the request; (c) looks up the signing key and verifies the signature; (d) evaluates IAM/bucket policies to ensure the signer had permission at signing time. The URL is usable by anyone (no auth header needed), but only for the specified method, key, and time window.

---

#### 9.16.12 Q12: How do you handle the “small file problem” in an erasure-coded system?

**Answer:** For objects smaller than 256 KB, erasure coding individually is wasteful: a 1 KB object would produce 12 chunks of ~125 bytes each, but each chunk has metadata overhead (checksum, location, index entry) of ~100 bytes. Our solution is **object packing**: small objects are buffered in memory on the API server and aggregated into a “packed extent” of ~1 MB. The packed extent is then erasure-coded as a single unit. Each small object’s metadata stores the extent ID and its offset/length within the extent. Reads require fetching the extent (8 chunks) and extracting the relevant bytes. We mitigate read amplification by caching entire extents (they tend to have temporal locality—objects uploaded together are often accessed together). Deletes mark the object as tombstoned within the extent; space is reclaimed when the compactor rewrites the extent without the deleted objects.

---

#### 9.16.13 Q13: How does the system handle a correlated failure (e.g., software bug that corrupts all replicas)?

**Answer:** This is the nightmare scenario that replication and erasure coding within a single software stack cannot protect against. Defenses: (1) **Immutable writes**: once written, chunk data is never modified in place, only appended or deleted. (2) **End-to-end checksums**: the client computes a checksum (Content-MD5) that is verified at every stage—API server, data node write, data node read, API server response. (3) **Diverse storage codepaths**: repair and scrubbing code runs a separate checksum verification path from the write path, reducing the chance of a shared bug. (4) **Cross-region replication**: regions may run different software versions, so a bug in one region’s stack does not affect the replica. (5) **Object versioning**: even if the latest version is corrupted, previous

---

versions (stored as independent data) may be intact. (6) **Periodic offline validation**: sample objects are periodically downloaded and verified by an independent system.

---

#### 9.16.14 Q14: How do you design rate limiting for a multi-tenant object storage service?

**Answer:** Rate limiting operates at multiple levels. (1) **Per-account**: each account has a default request rate limit (e.g., 3,500 PUT/sec and 5,500 GET/sec per prefix). Exceeding this returns HTTP 503 SlowDown. (2) **Per-bucket**: configurable limits via bucket policies. (3) **Global**: the API server fleet has a total capacity; if aggregate load approaches capacity, adaptive throttling kicks in (prioritize existing connections, reject new ones). Implementation uses a **token bucket** algorithm with distributed state: each API server maintains a local token bucket and periodically syncs with a centralized rate limiter (Redis-based). For burst tolerance, the token bucket allows short bursts up to 2x the sustained rate. Clients receive `Retry-After` headers to implement exponential backoff.

---

#### 9.16.15 Q15: How would you design the system to support WORM (Write Once Read Many) compliance?

**Answer:** WORM compliance (for regulatory requirements like SEC 17a-4 or GDPR) requires that objects cannot be modified or deleted for a specified retention period. Implementation: (1) **Object Lock** at the bucket level: once enabled, every object version gets a retention configuration (retain-until date + legal hold flag). (2) **Retention enforcement**: the metadata service checks retention before allowing DELETE. If retention is active, DELETE is rejected with HTTP 403. (3) **Governance mode**: allows users with special IAM permissions to bypass retention (for correcting mistakes). **Compliance mode**: even root/admin cannot delete before retention expires—the system enforces this at the storage layer, not just IAM. (4) **Legal hold**: independently locks an object (regardless of retention date) until explicitly removed by an authorized user. (5) **Clock trust**: the system uses NTP-synced clocks with skew detection; retention expiry checks use the cluster's consensus-agreed time to prevent clock manipulation.

---

## 9.17 17. References and Further Reading

- **Amazon S3 Design**: Dynamo paper (DeCandia et al., 2007) influenced S3's metadata layer. S3 strong consistency announcement (Dec 2020) describes the witness protocol.
- **Erasure Coding**: Reed-Solomon codes, Intel ISA-L library for SIMD-optimized implementations.
- **Ceph CRUSH**: Weil et al., "CRUSH: Controlled, Scalable, Decentralized Placement of Replicated Data" (SC'06). The placement algorithm described here is inspired by CRUSH.
- **Facebook f4**: Muralidhar et al., "f4: Facebook's Warm BLOB Storage System" (OSDI 2014). Demonstrates erasure coding at scale for warm storage.
- **Windows Azure Storage**: Calder et al., "Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency" (SOSP 2011).
- **MinIO**: Open-source S3-compatible object storage. Good reference implementation for understanding erasure coding and distributed object storage.
- **Google Colossus**: Successor to GFS, uses Reed-Solomon erasure coding for storage efficiency.

## 9.18 18. Summary Cheat Sheet

| Object Storage (S3) - Quick Reference                                                                                                                                                                                                                                                                                                            |                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Durability:                                                                                                                                                                                                                                                                                                                                      | 11 nines (99.999999999%) via erasure coding 8+4        |
| Availability:                                                                                                                                                                                                                                                                                                                                    | 99.99% via multi-AZ, auto-failover                     |
| Consistency:                                                                                                                                                                                                                                                                                                                                     | Strong read-after-write (Raft-based metadata)          |
| Scale:                                                                                                                                                                                                                                                                                                                                           | Petabytes data, trillions of objects, millions req/sec |
| Data path:                                                                                                                                                                                                                                                                                                                                       | Client → API Server → erasure encode → 12 data nodes   |
| Metadata path:                                                                                                                                                                                                                                                                                                                                   | API Server → Metadata Store (Raft, sharded by bucket)  |
| Placement:                                                                                                                                                                                                                                                                                                                                       | CRUSH algorithm (deterministic, no lookup table)       |
| Key components:                                                                                                                                                                                                                                                                                                                                  |                                                        |
| API Server:                                                                                                                                                                                                                                                                                                                                      | stateless, erasure encode/decode                       |
| Metadata Service:                                                                                                                                                                                                                                                                                                                                | Raft groups, RocksDB, sharded by bucket                |
| Data Nodes:                                                                                                                                                                                                                                                                                                                                      | raw disk, extent files, chunk index                    |
| Placement Svc:                                                                                                                                                                                                                                                                                                                                   | CRUSH algorithm + cluster topology map                 |
| GC / Scrubber:                                                                                                                                                                                                                                                                                                                                   | background integrity + space reclamation               |
| Storage cost:                                                                                                                                                                                                                                                                                                                                    | 1.5x raw data (vs 3x for replication) = 50% savings    |
| Storage tiers:                                                                                                                                                                                                                                                                                                                                   | Standard → IA → Glacier → Deep Archive                 |
| Interview signals:                                                                                                                                                                                                                                                                                                                               |                                                        |
| <ul style="list-style-type: none"><li>- Understand erasure coding trade-offs (cost, latency, complexity)</li><li>- Explain 11-nines durability math</li><li>- Articulate strong consistency via Raft + read index</li><li>- Discuss failure domains and correlated failure mitigation</li><li>- Know multipart upload lifecycle and GC</li></ul> |                                                        |

## Chapter 10

# Design a Ride-Sharing Service (Uber / Lyft)

**Difficulty:** Hard | **Companies:** Uber, Lyft, Google, Amazon, Grab, Ola, DiDi, Bolt

---

### 10.1 1. Problem Statement and Scope

Design a **ride-sharing platform** that matches riders with nearby drivers in real time, supports live location tracking, dynamic (surge) pricing, fare estimation, payments, and end-to-end ride lifecycle management – from requesting a ride to completing payment.

#### 10.1.1 In Scope

- **Ride Request & Matching:** Rider requests ride, system finds and dispatches best driver
- **Real-Time Location:** Driver location tracked every 3-4 seconds
- **Dynamic Pricing:** Surge pricing based on supply-demand ratio per geo-cell
- **Fare Estimation & Payment:** Upfront fare estimate, metered billing, payment processing
- **Ride Lifecycle:** Request -> Match -> Pickup -> In-Progress -> Complete -> Pay
- **Rating & Feedback:** Bidirectional ratings after ride completion
- **Multiple Ride Types:** UberX, UberXL, UberBlack, Pool (shared rides)

#### 10.1.2 Out of Scope

- Autonomous vehicles / self-driving integration
  - Food delivery (Uber Eats) or package delivery
  - Driver background checks and detailed onboarding workflows
  - Detailed fraud detection ML pipelines (mentioned at high level only)
  - Multi-modal transport (bikes, scooters, public transit)
- 

### 10.2 2. Functional Requirements

#### 10.2.1 2.1 Rider Functions

- **R1 - Request Ride:** Specify pickup, dropoff, ride type; receive fare estimate

- **R2 - See Nearby Drivers:** View available drivers on the map within radius
- **R3 - Fare Estimation:** Get upfront price based on distance, time, surge multiplier
- **R4 - Real-Time Tracking:** Track assigned driver en route and during trip
- **R5 - Ride History:** View past rides, receipts, routes
- **R6 - Payment:** Pay via credit card, wallet, or cash; split fare
- **R7 - Rating:** Rate driver (1-5 stars) and leave optional comment
- **R8 - Cancel Ride:** Cancel before or after matching with applicable fee
- **R9 - Ride Pooling:** Share ride with other riders heading in similar direction
- **R10 - Schedule Ride:** Book ride in advance for a future time

### 10.2.2 2.2 Driver Functions

- **D1 - Toggle Availability:** Go online/offline to accept ride requests
- **D2 - Accept/Decline Ride:** Receive ride request with pickup info; accept or decline/timeout
- **D3 - Navigation:** Turn-by-turn directions to pickup and dropoff
- **D4 - Start/End Trip:** Mark pickup complete and trip end for billing
- **D5 - Earnings Tracking:** View daily/weekly earnings, trip details, surge bonuses
- **D6 - Rating:** Rate rider after trip completion
- **D7 - Location Broadcast:** App continuously sends GPS coordinates while online

### 10.2.3 2.3 Admin / Platform Functions

- **A1 - Driver Onboarding:** Approve new driver registrations, document verification
  - **A2 - Surge Management:** Configure surge pricing rules, caps, and geo-cell granularity
  - **A3 - Analytics Dashboard:** Real-time metrics: rides, wait times, supply-demand heatmaps
  - **A4 - Dispute Resolution:** Handle payment disputes, ride complaints
  - **A5 - Geo-Fencing:** Define service areas, airports, restricted zones
- 

## 10.3 3. Non-Functional Requirements

- **Matching Latency:** < 3 seconds from ride request to driver notification
  - **Location Update Rate:** Every 3-4 seconds per active driver
  - **Availability:** 99.99% uptime (< 52.6 min downtime/year)
  - **Concurrent Rides:** Support 1M+ concurrent active rides
  - **Concurrent Drivers:** Support 5M+ online drivers sending location updates
  - **Global Coverage:** Multi-region deployment across continents
  - **Data Durability:** Zero loss of trip and payment records
  - **Consistency:** Strong consistency for trip state and payments
  - **Scalability:** Horizontal scaling; handle 10x traffic spikes (NYE, events)
  - **Security:** Encrypt PII, PCI-DSS compliance for payments, end-to-end TLS
- 

## 10.4 4. Back-of-Envelope Estimation



### 10.4.1 4.1 Traffic Estimates

Daily Active Riders: 30M  
 Daily Active Drivers: 5M  
 Daily Rides: 20M  
 Average Ride Duration: 15 minutes

Peak concurrent rides:  $20M / (24 * 4) = \sim 833K$ , round to 1M with peak factor  
 Concurrent online drivers: 5M (sending location updates)

### 10.4.2 4.2 Location Update Throughput

Online drivers: 5,000,000  
 Location update interval: 4 seconds  
 Updates per second:  $5,000,000 / 4 = 1,250,000$  updates/sec

Each update payload:  $\sim 60$  bytes (driver\_id, lat, lng, heading, speed, timestamp)  
 Location ingestion bandwidth:  $1.25M * 60 B = 75 MB/sec = 600 Mbps$

Location updates per day:  $1.25M * 86,400 = \sim 108$  billion

### 10.4.3 4.3 Storage Estimates

Trip Data:  
 - Per trip record:  $\sim 1 KB$  (IDs, locations, timestamps, fare, status)  
 - Daily:  $20M * 1 KB = 20 GB/day$   
 - Yearly:  $20 GB * 365 = \sim 7.3 TB/year$

Location History (for analytics, not real-time):  
 - Per update: 60 bytes  
 - Daily:  $108B * 60 B = \sim 6.5 TB/day$  (hot: Redis, cold: object store)  
 - Retention: 7 days hot =  $\sim 45 TB$ , then archive to S3/GCS

Ride Route (polyline):  
 - Per ride:  $\sim 5 KB$  (encoded polyline of GPS points during trip)  
 - Daily:  $20M * 5 KB = 100 GB/day$

User Profiles:  
 - 100M riders + 10M drivers = 110M profiles \* 2 KB =  $\sim 220 GB$  (fits one DB)

### 10.4.4 4.4 Bandwidth Estimates

Rider tracking updates:  $1M \text{ concurrent rides} * 1 \text{ update}/3 \text{ sec} * 100 B = \sim 33 MB/sec$   
 Driver location ingestion: 75 MB/sec (calculated above)  
 API requests (ride ops):  $\sim 50K$  ride-related API calls/sec at peak

Total inbound bandwidth:  $\sim 110 MB/sec = \sim 880 Mbps$   
 Total outbound bandwidth:  $\sim 150 MB/sec = \sim 1.2 Gbps$  (includes map tiles, tracking)



## 10.5 5. API Design

### 10.5.1 5.1 Ride APIs

POST /v1/rides/request

```
Body: {
 rider_id: "uuid",
 pickup: { lat: 37.7749, lng: -122.4194, address: "123 Market St" },
 dropoff: { lat: 37.7849, lng: -122.4094, address: "456 Mission St" },
 ride_type: "UBER_X" | "UBER_XL" | "UBER_BLACK" | "POOL",
 payment_method_id: "pm_xxx",
 scheduled_at: null | "2026-04-09T18:00:00Z" // null = now
}
Response: 201 {
 ride_id: "ride_xxx",
 status: "MATCHING",
 estimated_fare: { min: 12.50, max: 16.00, currency: "USD", surge: 1.2 },
 estimated_pickup_time: "3 min"
}
```

GET /v1/rides/estimate

Params: pickup\_lat, pickup\_lng, dropoff\_lat, dropoff\_lng, ride\_type

```
Response: 200 {
 estimates: [
 { ride_type: "UBER_X", fare: { min: 12, max: 16 }, eta: "3 min", surge: 1.0 },
 { ride_type: "UBER_XL", fare: { min: 18, max: 24 }, eta: "5 min", surge: 1.0 },
 { ride_type: "UBER_BLACK", fare: { min: 30, max: 40 }, eta: "7 min", surge: 1.5 }
]
}
```

POST /v1/rides/{ride\_id}/accept

```
Body: { driver_id: "uuid" }
Response: 200 { status: "ACCEPTED", pickup_eta: "4 min" }
```

POST /v1/rides/{ride\_id}/cancel

```
Body: { cancelled_by: "RIDER" | "DRIVER", reason: "string" }
Response: 200 { cancellation_fee: 5.00 }
```

```
POST /v1/rides/{ride_id}/start
 Body: { driver_id: "uuid", odometer: 45230.5 }
 Response: 200 { status: "IN_PROGRESS", started_at: "timestamp" }

POST /v1/rides/{ride_id}/complete
 Body: { driver_id: "uuid", odometer: 45238.2 }
 Response: 200 {
 status: "COMPLETED",
 fare: { base: 2.50, distance: 8.40, time: 3.20, surge: 1.50, total: 18.60 },
 payment_status: "CHARGED"
 }

GET /v1/rides/{ride_id}/track
 Response: 200 {
 driver_location: { lat: 37.775, lng: -122.418 },
 eta_to_pickup: "2 min" | null,
 eta_to_dropoff: "12 min" | null,
 route_polyline: "encoded_polyline_string"
 }
```

### 10.5.2 5.2 Driver Location API (High Frequency)

```
PUT /v1/drivers/{driver_id}/location
 Body: {
 lat: 37.7749,
 lng: -122.4194,
 heading: 270.0, // degrees
 speed: 35.5, // km/h
 accuracy: 5.0, // meters
 timestamp: 1712678400000 // epoch ms
 }
 Response: 200 { ack: true }
```

#### Notes:

- Called every 3-4 seconds by driver app
- Use UDP or persistent WebSocket for reduced overhead
- Batching: driver app can batch 3-5 updates and send every 12-15 sec

### 10.5.3 5.3 Driver Availability API

```
PUT /v1/drivers/{driver_id}/availability
 Body: { status: "ONLINE" | "OFFLINE", vehicle_type: "SEDAN" }
 Response: 200 { status: "ONLINE" }

GET /v1/drivers/nearby
 Params: lat, lng, radius_km=5, vehicle_type=SEDAN
 Response: 200 {
 drivers: [
 { driver_id: "uuid", lat: 37.776, lng: -122.419, eta: "3 min", rating: 4.8 },
 ...
]
 }
```

### 10.5.4 5.4 Rating API

```
POST /v1/rides/{ride_id}/rate
 Body: {
 rated_by: "RIDER" | "DRIVER",
 rating: 5, // 1-5
 comment: "Great ride!",
 tags: ["clean_car", "good_navigation"]
 }
 Response: 201 { success: true }
```

---

## 10.6 6. Data Model and Database Selection

### 10.6.1 6.1 Core Entities and Schema

| users         |                                          |
|---------------|------------------------------------------|
| user_id       | UUID PK                                  |
| type          | ENUM('RIDER','DRIVER')                   |
| name          | VARCHAR(100)                             |
| email         | VARCHAR(255) UNIQUE                      |
| phone         | VARCHAR(20) UNIQUE                       |
| password_hash | VARCHAR(255)                             |
| rating_avg    | DECIMAL(3,2)                             |
| rating_count  | INT                                      |
| status        | ENUM('ACTIVE','SUSPENDED','DEACTIVATED') |
| created_at    | TIMESTAMP                                |
| updated_at    | TIMESTAMP                                |

| drivers           |                                    |
|-------------------|------------------------------------|
| driver_id         | UUID PK FK(users.user_id)          |
| license_number    | VARCHAR(50)                        |
| vehicle_type      | ENUM('SEDAN','SUV','LUXURY','VAN') |
| vehicle_make      | VARCHAR(50)                        |
| vehicle_model     | VARCHAR(50)                        |
| vehicle_year      | INT                                |
| vehicle_plate     | VARCHAR(20)                        |
| is_online         | BOOLEAN                            |
| current_ride_id   | UUID NULLABLE                      |
| city_id           | UUID FK(cities.city_id)            |
| acceptance_rate   | DECIMAL(5,2)                       |
| cancellation_rate | DECIMAL(5,2)                       |

| trips            |                                                                                          |                            |
|------------------|------------------------------------------------------------------------------------------|----------------------------|
| trip_id          | UUID                                                                                     | PK                         |
| rider_id         | UUID                                                                                     | FK(users.user_id)          |
| driver_id        | UUID                                                                                     | FK(users.user_id) NULLABLE |
| ride_type        | ENUM('UBER_X','UBER_XL','UBER_BLACK','POOL')                                             |                            |
| status           | ENUM('REQUESTED','MATCHING','ACCEPTED','ARRIVING','IN_PROGRESS','COMPLETED','CANCELLED') |                            |
| pickup_lat       | DECIMAL(10,7)                                                                            |                            |
| pickup_lng       | DECIMAL(10,7)                                                                            |                            |
| pickup_address   | VARCHAR(500)                                                                             |                            |
| dropoff_lat      | DECIMAL(10,7)                                                                            |                            |
| dropoff_lng      | DECIMAL(10,7)                                                                            |                            |
| dropoff_address  | VARCHAR(500)                                                                             |                            |
| fare_estimate    | JSONB                                                                                    |                            |
| fare_actual      | JSONB                                                                                    |                            |
| surge_multiplier | DECIMAL(3,2)                                                                             |                            |
| distance_km      | DECIMAL(8,2)                                                                             |                            |
| duration_min     | DECIMAL(8,2)                                                                             |                            |
| requested_at     | TIMESTAMP                                                                                |                            |
| accepted_at      | TIMESTAMP                                                                                |                            |
| pickup_at        | TIMESTAMP                                                                                |                            |
| dropoff_at       | TIMESTAMP                                                                                |                            |
| cancelled_at     | TIMESTAMP                                                                                |                            |
| cancelled_by     | ENUM('RIDER','DRIVER','SYSTEM')                                                          |                            |
| city_id          | UUID                                                                                     |                            |
| payment_id       | UUID                                                                                     | FK(payments.payment_id)    |
| route_polyline   | TEXT                                                                                     |                            |

| payments         |                                                                 |                   |
|------------------|-----------------------------------------------------------------|-------------------|
| payment_id       | UUID                                                            | PK                |
| trip_id          | UUID                                                            | FK(trips.trip_id) |
| rider_id         | UUID                                                            | FK(users.user_id) |
| driver_id        | UUID                                                            | FK(users.user_id) |
| amount           | DECIMAL(10,2)                                                   |                   |
| currency         | VARCHAR(3)                                                      |                   |
| payment_method   | ENUM('CARD', 'WALLET', 'CASH')                                  |                   |
| status           | ENUM('PENDING', 'AUTHORIZED', 'CAPTURED', 'REFUNDED', 'FAILED') |                   |
| stripe_charge_id | VARCHAR(100)                                                    |                   |
| platform_fee     | DECIMAL(10,2)                                                   |                   |
| driver_payout    | DECIMAL(10,2)                                                   |                   |
| created_at       | TIMESTAMP                                                       |                   |
| updated_at       | TIMESTAMP                                                       |                   |

| ratings    |           |                   |
|------------|-----------|-------------------|
| rating_id  | UUID      | PK                |
| trip_id    | UUID      | FK(trips.trip_id) |
| rated_by   | UUID      | FK(users.user_id) |
| rated_user | UUID      | FK(users.user_id) |
| score      | INT       | CHECK(1..5)       |
| comment    | TEXT      |                   |
| tags       | TEXT[]    |                   |
| created_at | TIMESTAMP |                   |

### 10.6.2 6.2 Database Selection per Data Type

| Data Type                    | Database               | Rationale                                                        |
|------------------------------|------------------------|------------------------------------------------------------------|
| User Profiles                | PostgreSQL             | Relational, ACID, moderate size                                  |
| Driver Profiles              | PostgreSQL             | Joins with users, transactional                                  |
| Active Trips                 | PostgreSQL             | Strong consistency for state machine                             |
| Trip History                 | Cassandra              | High write volume, time-series, partition by city + date         |
| Driver Locations (real-time) | Redis (GeoSet)         | In-memory, GEOADD/GEORADIUS, ultra low latency, 1.25M writes/sec |
| Location History (archival)  | Apache Kafka + S3/HDFS | Stream to S3/HDFS for analytics                                  |
| Payments                     | PostgreSQL             | ACID required, audit trail                                       |
| Surge Pricing Config         | Redis                  | Fast reads per geo-cell, TTL-based                               |
| Notifications                | Redis Pub/Sub          | Real-time push to driver/rider                                   |
| Search/Analytics             | Elasticsearch          | Full-text search, aggregations                                   |

### 10.6.3 6.3 Geospatial Index Design

Approach: Redis GeoHash + Application-Level QuadTree Hybrid

Redis GeoSet for Real-Time Queries:

```
GEOADD drivers:online:sedan <lng> <lat> <driver_id>
```

```
GEORADIUS drivers:online:sedan <lng> <lat> 5 km WITHCOORD WITHDIST COUNT 20 ASC
```

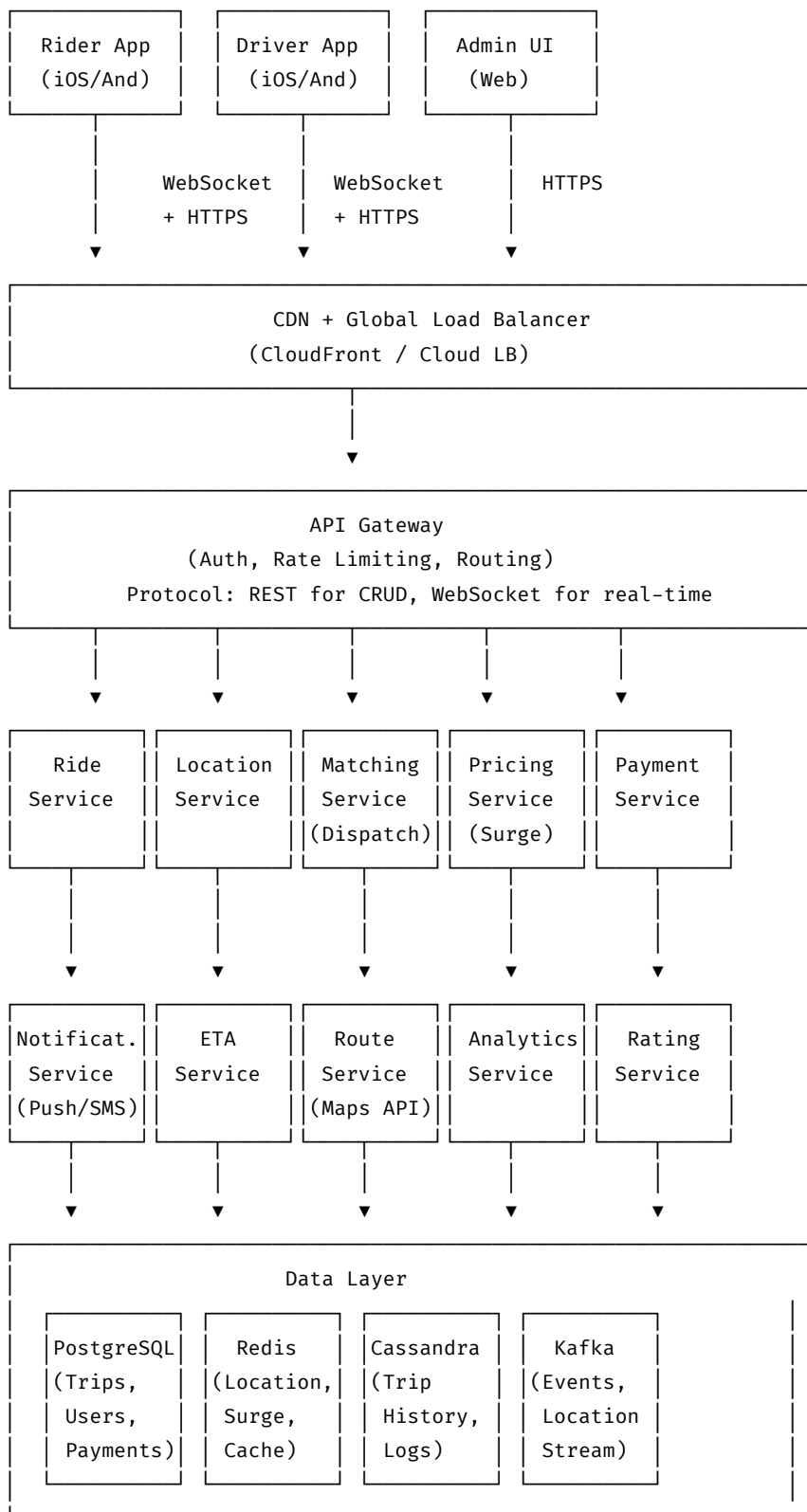
- Separate GeoSets per vehicle type: drivers:online:sedan, drivers:online:suv
- Separate GeoSets per city for scalability: drivers:nyc:sedan, drivers:sf:sedan
- $O(N+\log(M))$  where  $N$  = elements returned,  $M$  = elements in set
- Redis cluster with ~16 shards handles 1.25M updates/sec

S2 Geometry (Google-style) for Advanced Queries:

- Earth divided into hierarchical cells at multiple levels
  - Level 12 cell ~ 3.3 km<sup>2</sup> (good for surge pricing zones)
  - Level 14 cell ~ 0.2 km<sup>2</sup> (good for driver proximity)
  - Cell IDs are 64-bit integers - fast indexing and range queries
- 

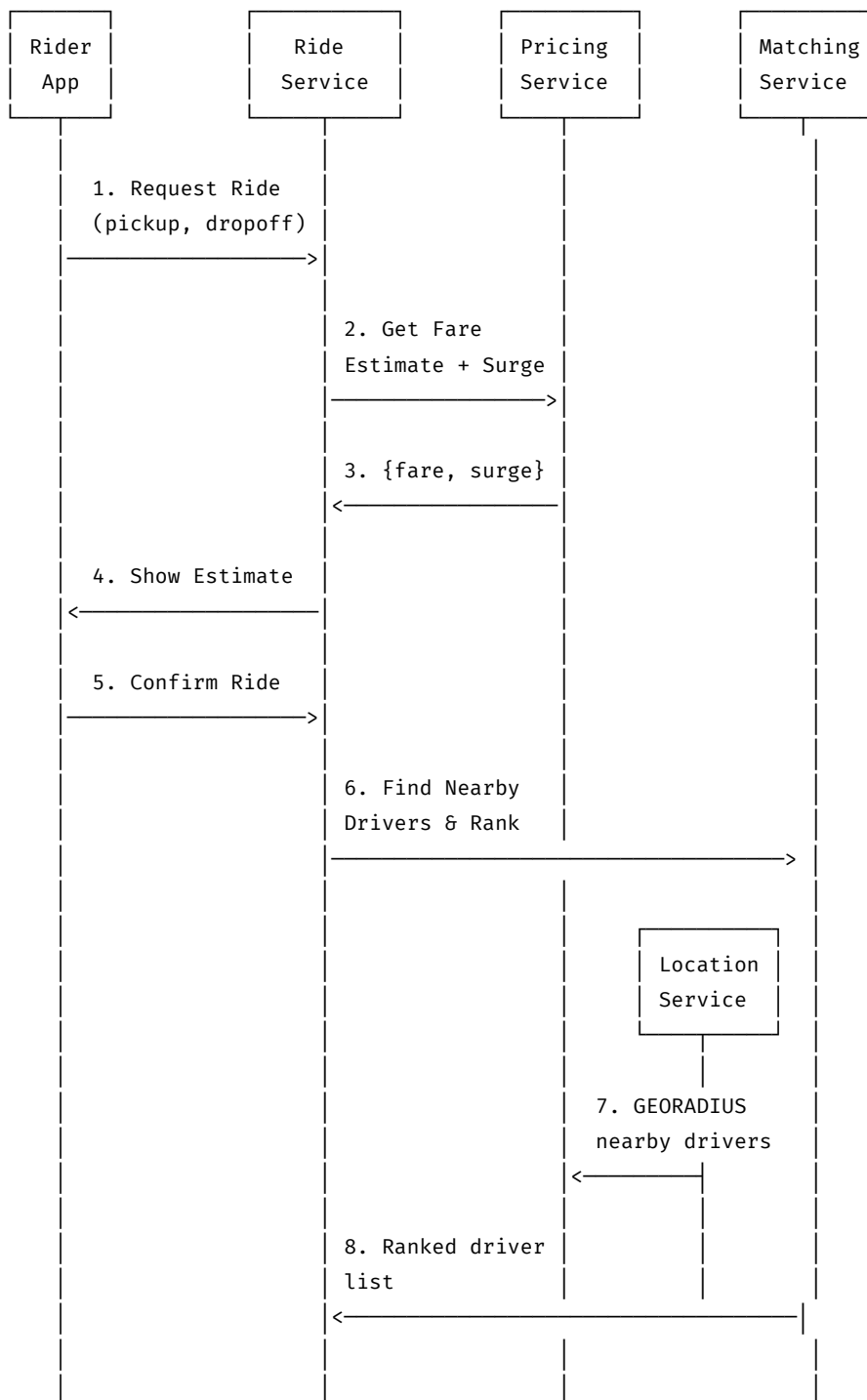
## 10.7 7. High-Level Architecture

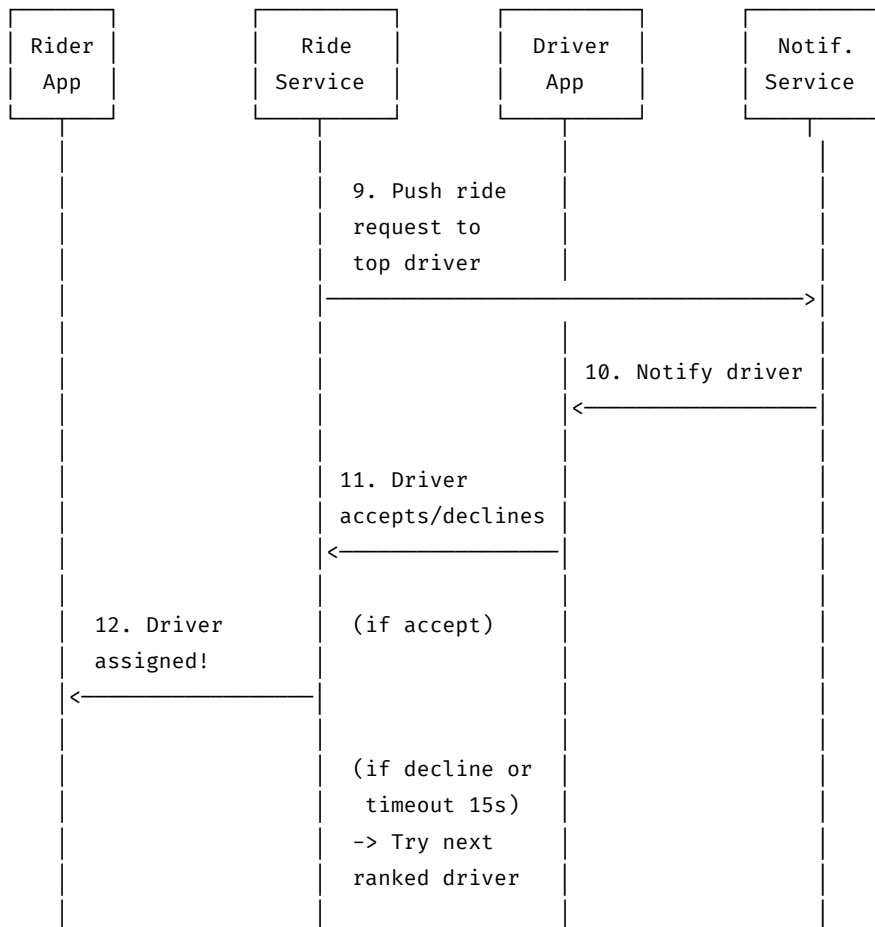
### 10.7.1 7.1 System Architecture Overview





### 10.7.2 7.2 Ride Request and Dispatch Flow





### 10.7.3 7.3 Component Breakdown

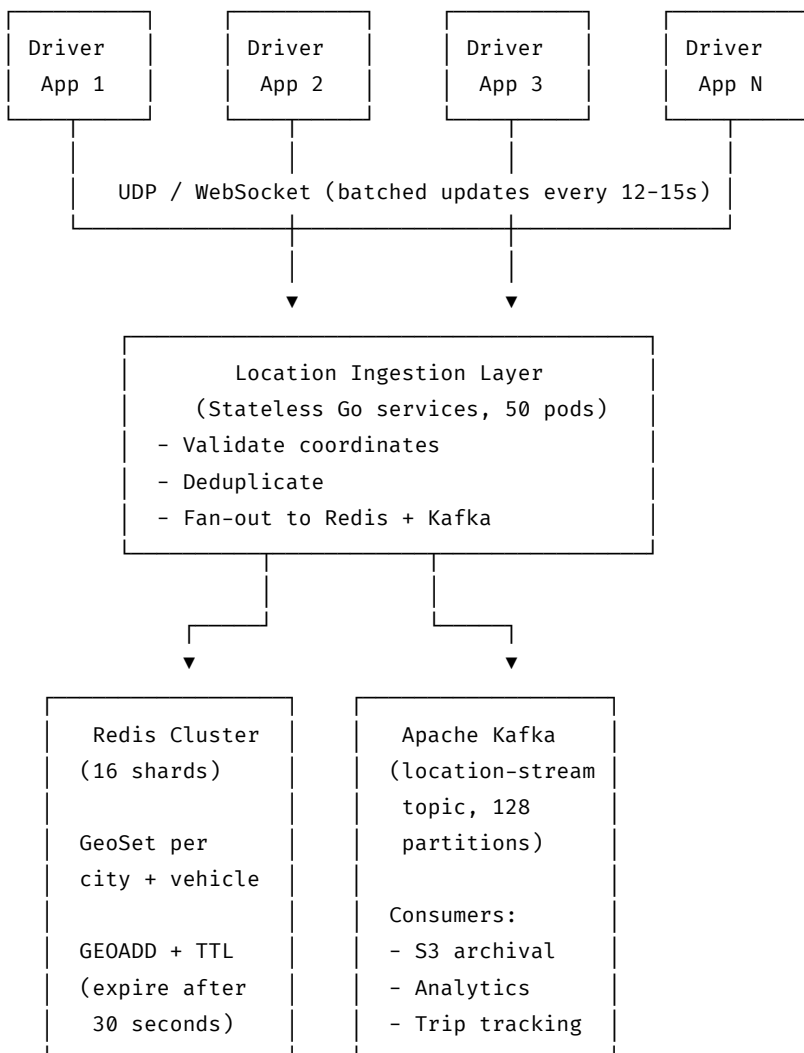
- **Ride Service:** Trip lifecycle (CRUD), state machine; Tech Stack: Go/Java, PostgreSQL
- **Location Service:** Ingest driver GPS, geospatial queries; Tech Stack: Go, Redis GeoSet, Kafka
- **Matching Service:** Find best driver, dispatch, handle retries; Tech Stack: Go/Java, in-memory ranking
- **Pricing Service:** Fare calculation, surge multiplier, estimates; Tech Stack: Python/Go, Redis
- **Payment Service:** Authorization, capture, refund, driver payout; Tech Stack: Java, PostgreSQL, Stripe
- **ETA Service:** Time estimate using road network + traffic; Tech Stack: Go, Maps API, ML models
- **Route Service:** Path computation, turn-by-turn directions; Tech Stack: Go, OSRM / Google Maps
- **Notification Svc:** Push notifications, SMS, in-app messages; Tech Stack: Go, FCM/APNs, Twilio
- **Rating Service:** Store ratings, update averages; Tech Stack: Go, PostgreSQL
- **Analytics Service:** Real-time dashboards, supply-demand heatmaps; Tech Stack: Spark, Flink, Elasticsearch

## 10.8 8. Deep Dive: Core Components

### 10.8.1 8.1 Location Service and Geospatial Indexing

The Location Service is the most write-heavy component in the system, handling **1.25 million location updates per second** from online drivers.

#### 10.8.1.1 Architecture



**10.8.1.2 GeoHash vs QuadTree vs S2 Geometry**

| Criteria      | GeoHash                                          | QuadTree                                         | S2 Geometry                                               |
|---------------|--------------------------------------------------|--------------------------------------------------|-----------------------------------------------------------|
| How it works  | Encode lat/lng into string, prefix = area        | Recursive subdivision of 2D space into quadrants | Project sphere onto cube, Hilbert curve cell IDs          |
| Precision     | Fixed levels (1-12 chars)                        | Adaptive (split/merge based on density)          | 30 levels, from 85 km <sup>2</sup> to < 1 cm <sup>2</sup> |
| Edge problem  | Adjacent cells may have very different prefixes  | None (tree traversal)                            | None (Hilbert curve ensures spatial locality)             |
| Query perf    | O(1) per cell lookup, need multi-cell for radius | O(log N) traversal                               | O(1) cell lookup, range scan for neighbors                |
| Redis support | Built-in GEOADD/GEORADIUS                        | Custom implementation                            | Custom implementation                                     |
| Used by       | Redis, many startups                             | Uber (H3 is hex variant)                         | Google Maps, Foursquare                                   |
| Best for      | Simple radius queries, quick MVP                 | Dynamic density handling                         | Global-scale, covering queries                            |

Recommendation: Use Redis GeoHash for real-time driver queries (simple, built-in, performant). Layer H3/S2 cells for surge pricing zones and analytics where uniform cell sizing matters.

### 10.8.1.3 Finding Drivers Within 5 km Radius

Step 1: Rider requests ride at (37.7749, -122.4194)

Step 2: Location Service executes:

```
GEORADIUS drivers:sf:sedan -122.4194 37.7749 5 km
 WITHCOORD WITHDIST COUNT 20 ASC
```

Step 3: Redis returns up to 20 nearest drivers within 5 km:

```
[
 { driver_id: "d1", dist: 0.8 km, lat: 37.776, lng: -122.420 },
 { driver_id: "d5", dist: 1.2 km, lat: 37.772, lng: -122.415 },
 { driver_id: "d3", dist: 2.1 km, lat: 37.780, lng: -122.425 },
 ...
]
```

Step 4: Filter out drivers who are:

- Currently on a trip (check drivers table: `current Ride_id`  $\neq$  NULL)
- Have low acceptance rate ( $< 60\%$ )
- Vehicle type mismatch

Step 5: For remaining candidates, compute ETA via road network

(GEORADIUS gives straight-line distance; actual ETA needs routing)

Step 6: Rank by: ETA (50%), rating (30%), acceptance\_rate (20%)

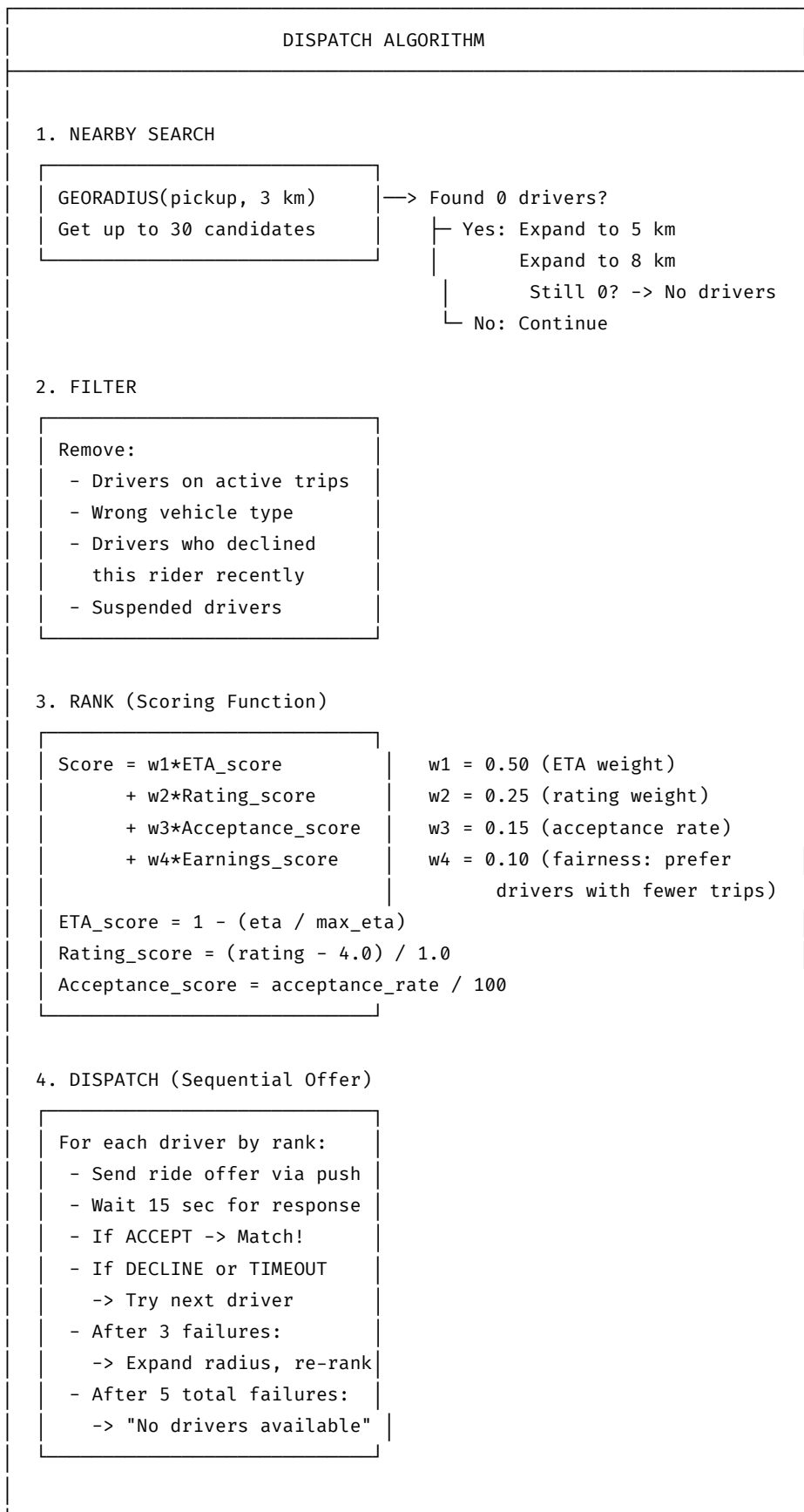
### 10.8.1.4 Handling 1.25M Location Updates/sec

Strategy: Batching + Sharding + Pipelining

1. Client-Side Batching:
  - Driver app collects GPS samples every 3-4 seconds
  - Batches 3-5 samples (12-15 seconds of data)
  - Sends batch in single request
  - Reduces network calls by 4x -> 312K requests/sec
2. Redis Sharding:
  - 16-shard Redis cluster
  - Shard key: city\_id (drivers in same city on same shard)
  - Per shard: ~78K updates/sec (well within Redis capacity)
3. Redis Pipelining:
  - Location ingestion service pipelines 100-500 GEOADD commands
  - Single round-trip for batch of updates
  - Redis single-threaded but handles 100K+ ops/sec per instance
4. Write-Behind to Kafka:
  - Fire-and-forget to Kafka for archival
  - Kafka handles 1M+ messages/sec easily
5. TTL-Based Expiry:
  - GEOADD does not support TTL natively
  - Use separate EXPIRE key per driver or background cleanup job
  - Drivers not updating for 30 seconds are removed from GeoSet

## 10.8.2 8.2 Matching / Dispatch Algorithm

The Matching Service is the brain of the ride-sharing platform. It finds the optimal driver for each ride request.

**10.8.2.1 Dispatch Flow**

### 10.8.2.2 Broadcast vs Sequential Dispatch

|                   | Sequential (Uber)                                | Broadcast (early Lyft)                                         |
|-------------------|--------------------------------------------------|----------------------------------------------------------------|
| How it works      | Offer to one driver at a time, wait for response | Offer to multiple drivers simultaneously, first to accept wins |
| Matching time     | Slower (15s per attempt worst case)              | Faster (parallel)                                              |
| Driver experience | Better (guaranteed if you accept)                | Worse (race condition, frustrating to lose)                    |
| Optimal match     | Better (can send to best-fit driver)             | Worse (closest driver may not be best fit)                     |
| Recommendation    | Use sequential with smart ranking                | Use broadcast only during extreme surge                        |

### 10.8.2.3 Supply-Demand Balancing

For each H3 cell (hexagonal geo-cell,  $\sim 1 \text{ km}^2$ ):

Supply = count of online, available drivers in cell

Demand = count of ride requests in cell in last 5 minutes

If Demand > Supply \* 1.5:

- > Trigger surge pricing in this cell
- > Send "high demand" notification to nearby drivers
- > Nudge drivers to reposition toward this cell

If Supply > Demand \* 3:

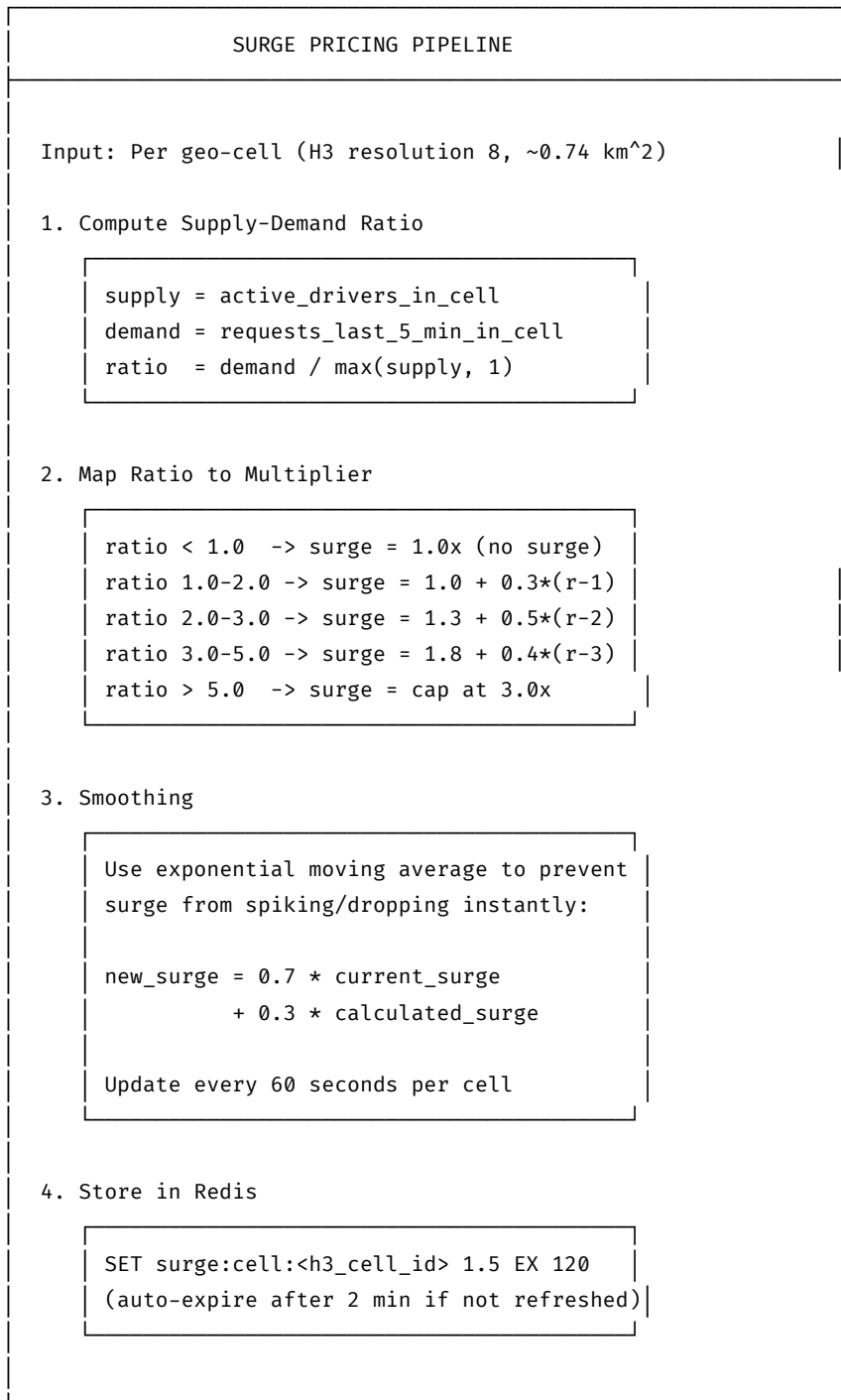
- > Reduce surge to 1.0x
- > Send "low demand" notification

Rebalancing: Show "demand heatmap" to drivers so they can voluntarily reposition to high-demand areas.



### 10.8.3 8.3 Dynamic Pricing (Surge)

#### 10.8.3.1 Surge Multiplier Calculation



### 10.8.3.2 Fare Calculation

```
fare = base_fare
 + (distance_km * per_km_rate)
 + (duration_min * per_min_rate)
 + booking_fee
 + tolls

surged_fare = fare * surge_multiplier

final_fare = max(surged_fare, minimum_fare)
```

Example (UberX in San Francisco):

```
base_fare = $2.55
per_km_rate = $1.15
per_min_rate = $0.22
booking_fee = $2.75
surge = 1.5x
```

Trip: 8.5 km, 18 min

fare = 2.55 + (8.5 \* 1.15) + (18 \* 0.22) + 2.75 = \$19.03

surged = \$19.03 \* 1.5 = \$28.55

### 10.8.3.3 Price Transparency and Capping

Rider Experience:

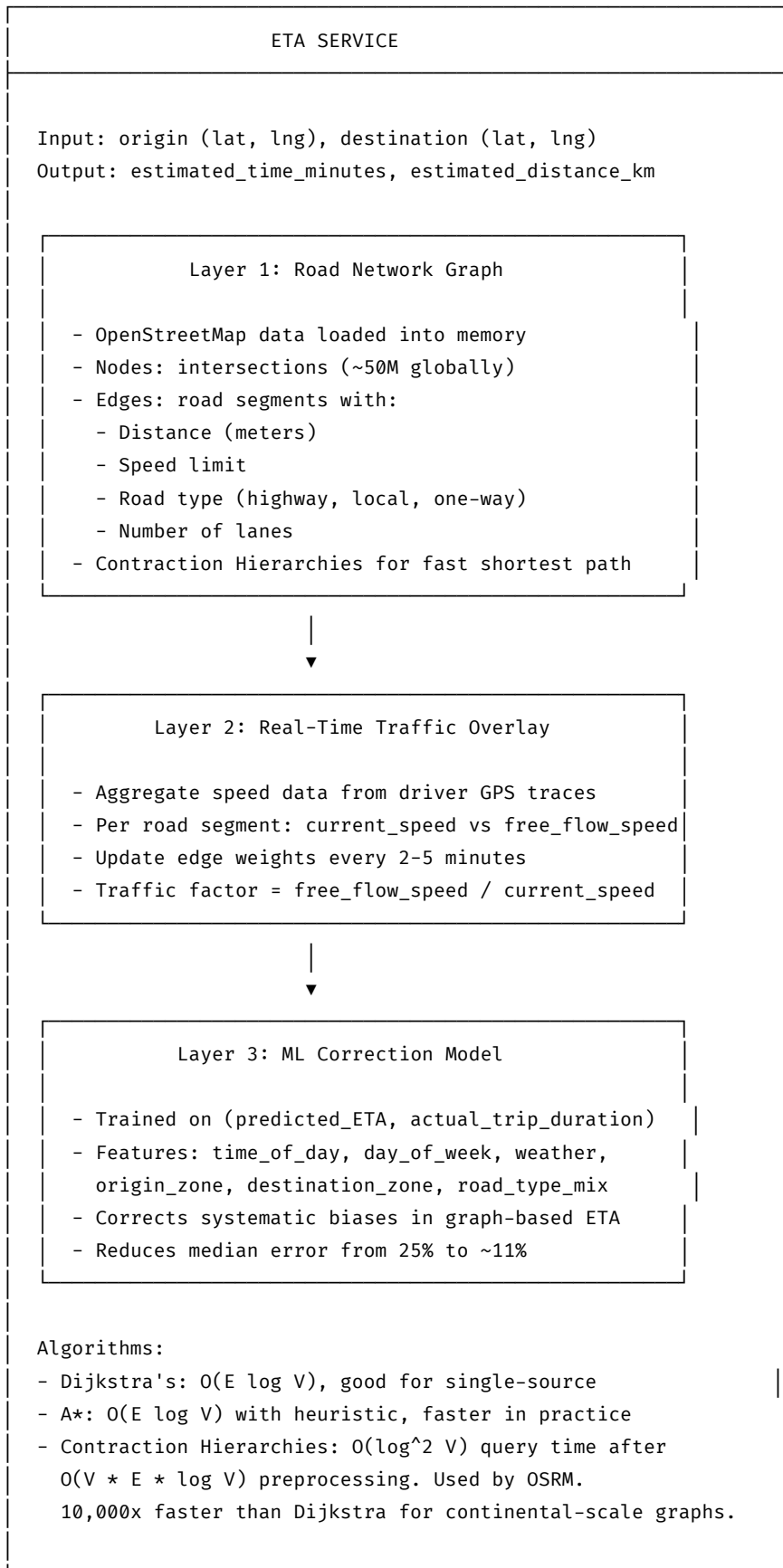
1. Before confirming ride, show:  
"Prices are higher due to increased demand"  
"Surge: 1.5x | Estimated fare: \$25-\$30"
2. Require explicit confirmation for surge > 2.0x:  
"Type the surge multiplier (2.3) to confirm"
3. Hard cap at 3.0x to prevent public backlash
4. Show fare range (min-max) not exact number for better UX

Business Rules:

- No surge during natural disasters (regulatory compliance)
- Gradual ramp-up: surge changes by at most 0.5x per minute
- Notify riders when surge drops: "Prices are back to normal"

## 10.8.4 8.4 ETA Calculation

### 10.8.4.1 ETA Service Architecture



### 10.8.4.2 ETA Caching

Popular Route Caching:

- Pre-compute ETAs for top 10K origin-destination pairs per city
- Cache in Redis with 5-minute TTL
- Key: eta:<origin\_h3\_cell>:<dest\_h3\_cell>:<time\_bucket>
- Covers ~40% of all ETA requests from cache

Cell-to-Cell ETA Matrix:

- For each city, maintain  $N \times N$  matrix where  $N$  = number of H3 cells
  - San Francisco: ~800 cells -> 640K entries
  - Update every 5 minutes using traffic data
  - Lookup:  $O(1)$  from matrix -> sub-millisecond
- 

## 10.9 9. Data Partitioning and Sharding

### 10.9.1 9.1 Sharding Strategy by Data Type

| SHARDING STRATEGY       |                      |                                                                                                                                                                                 |
|-------------------------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data Type               | Shard Key            | Rationale                                                                                                                                                                       |
| Driver Locations        | city_id (Redis)      | Drivers operate in one city; all location queries are city-scoped. GeoSet per city+vehicle type.<br><br>Key: drivers:<city>:<vehicle_type><br>Shards: ~200 (one per major city) |
| Trips (Active)          | city_id + date       | Queries mostly by city and date range. Active trips queried by city. Historical trips by date.<br><br>PostgreSQL: partition by city_id<br>Sub-partition by month for history    |
| Trip History (Archival) | city_id + year_month | Cassandra partition key: (city_id, year_month). Clustering key: trip_id<br>Even distribution across nodes.                                                                      |
| Users                   | user_id (hash)       | Hash-based sharding on user_id. Queries are always by specific user. Consistent hashing for rebalancing.                                                                        |
| Payments                | user_id (hash)       | Co-locate with user data for joins. Payment queries always include rider_id or driver_id.                                                                                       |
| Surge Pricing           | h3_cell_id           | Natural geographic distribution. Each cell's surge is independent. Fits entirely in Redis.                                                                                      |

### 10.9.2 9.2 Cross-Shard Query Handling

Problem: A rider in City A requests a ride near the boundary of City B.  
Drivers in City B are on a different shard.

Solution: Boundary Zone Overlap

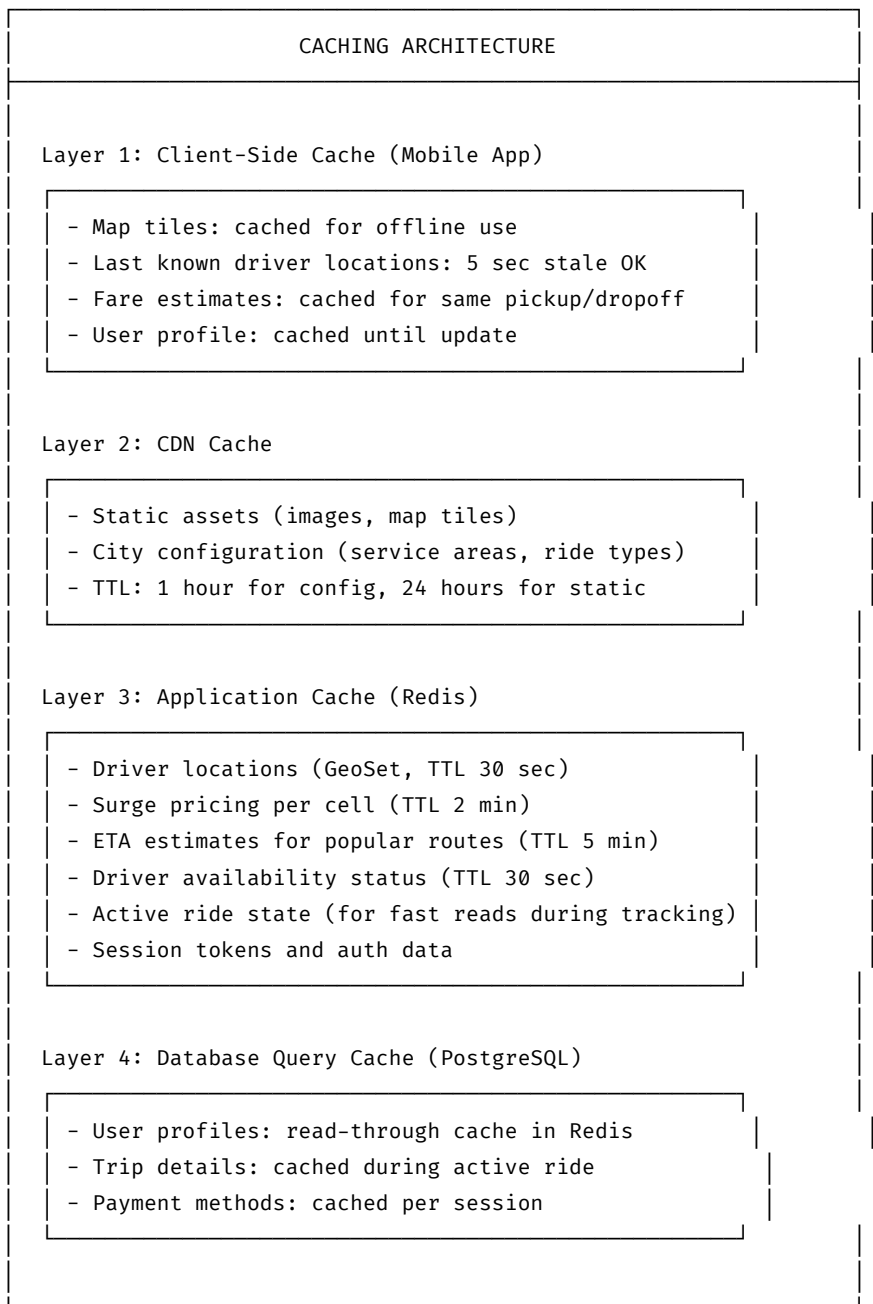
- Define overlap zones (5 km buffer) around city boundaries
- Drivers within 5 km of a boundary are indexed in BOTH city GeoSets
- Location Service writes to both: `drivers:cityA:sedan` AND `drivers:cityB:sedan`
- Matching Service queries only the rider's city shard
- Overlap zone doubles writes for ~5% of drivers but avoids cross-shard reads

Alternative: Global Driver Index

- Single global Redis GeoSet with ALL drivers (simpler but doesn't scale)
  - Only viable up to ~500K drivers, not at 5M scale
- 

## 10.10 10. Caching Strategy

### 10.10.1 10.1 Cache Layers



### 10.10.2 10.2 Cache Invalidation

#### Driver Location:

- Self-invalidating: new GEOADD overwrites old position
- TTL of 30 seconds: if driver stops sending updates, they disappear
- No explicit invalidation needed

#### Surge Pricing:

- Recomputed every 60 seconds by pricing pipeline
- Stored with TTL 120 seconds (2 min)
- If pipeline fails, surge naturally expires to 1.0x (safe default)

#### Active Ride State:

- Write-through cache: every trip state change updates Redis + PostgreSQL
- Redis key: ride:<ride\_id> -> JSON of current ride state
- TTL: 2 hours (rides rarely last longer)
- Explicitly deleted on ride completion

#### User Profiles:

- Cache-aside pattern
  - TTL: 15 minutes
  - Explicit invalidation on profile update via pub/sub
- 

## 10.11 11. Replication and Consistency

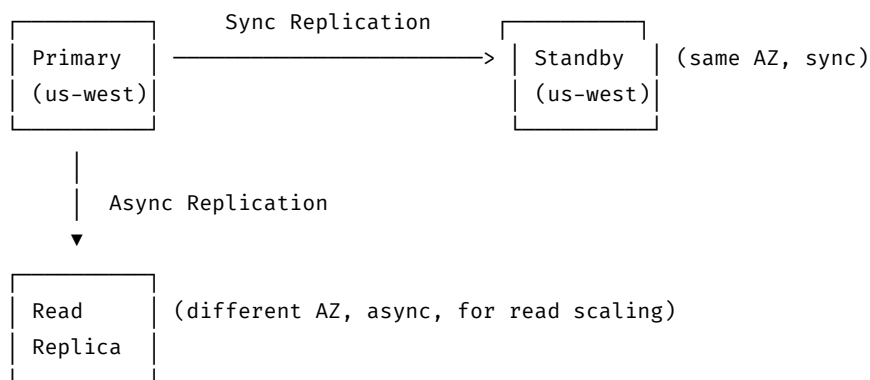


**10.11.1 11.1 Consistency Model per Data Type**

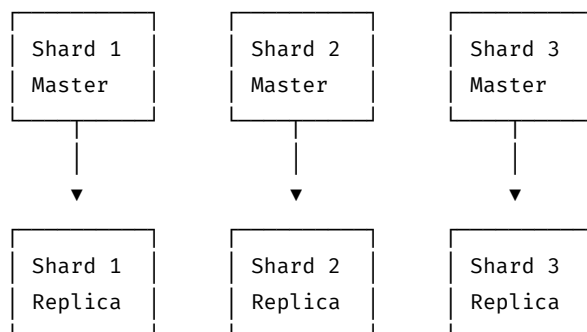
| Data Type                 | Consistency           | Justification                                                                                                                   |
|---------------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Driver Locations          | Eventually Consistent | Stale by 3-4 sec is acceptable. Driver moves; next update corrects. No need for strong consistency.                             |
| Trip State (status field) | Strongly Consistent   | State transitions must be atomic: MATCHING->ACCEPTED->IN_PROGRESS. Race conditions could cause double dispatch or lost rides.   |
| Payments                  | Strongly Consistent   | Financial data requires ACID. Double-charge is unacceptable. Use PostgreSQL with serializable isolation for payment operations. |
| Surge Pricing             | Eventually Consistent | 60-second staleness is fine. Surge changes gradually. All riders in same cell see same surge (read from same Redis key).        |
| User Profiles             | Read-Your-Writes      | User should see own updates immediately. Other users can see stale data briefly.                                                |
| Ratings                   | Eventually Consistent | Small delay in average update is acceptable. Batch recalculation.                                                               |

### 10.11.2 11.2 Replication Topology

PostgreSQL (Trips, Users, Payments):



Redis (Locations, Cache):

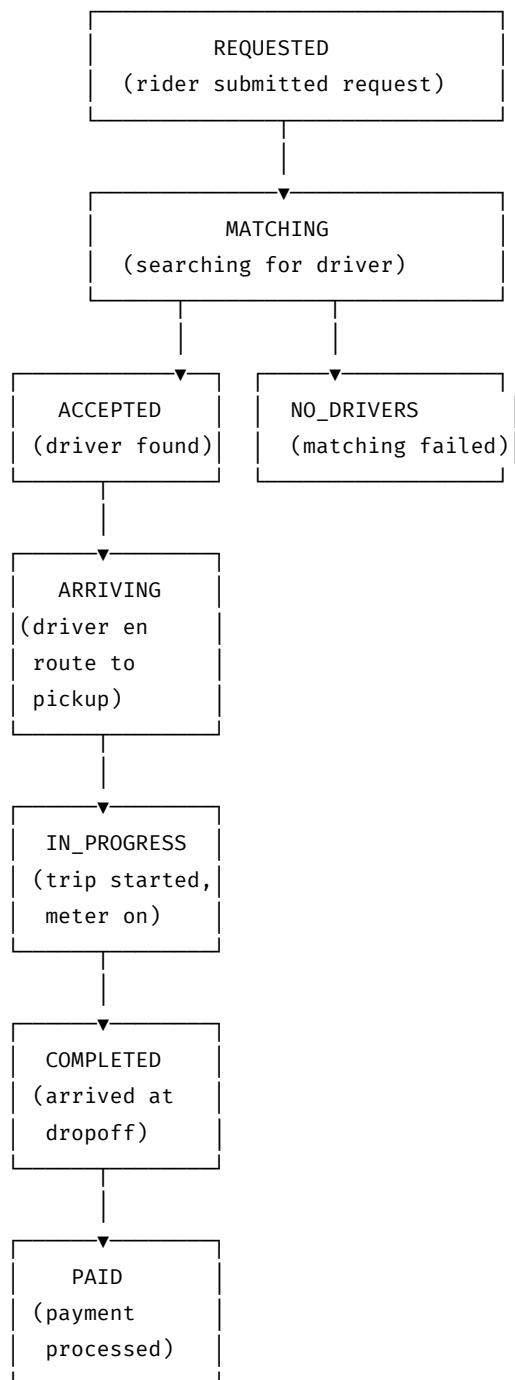


- Redis Cluster with automatic failover
- Replica serves read queries (GEORADIUS) to offload master
- Master handles writes (GEOADD)

Cassandra (Trip History):

- Replication Factor = 3
- Consistency Level: QUORUM for writes, ONE for reads
- Multi-datacenter replication for disaster recovery

### 10.11.3 11.3 Trip State Machine with Consistency



CANCELLED can occur from: REQUESTED, MATCHING, ACCEPTED, ARRIVING

State Transition Enforcement:

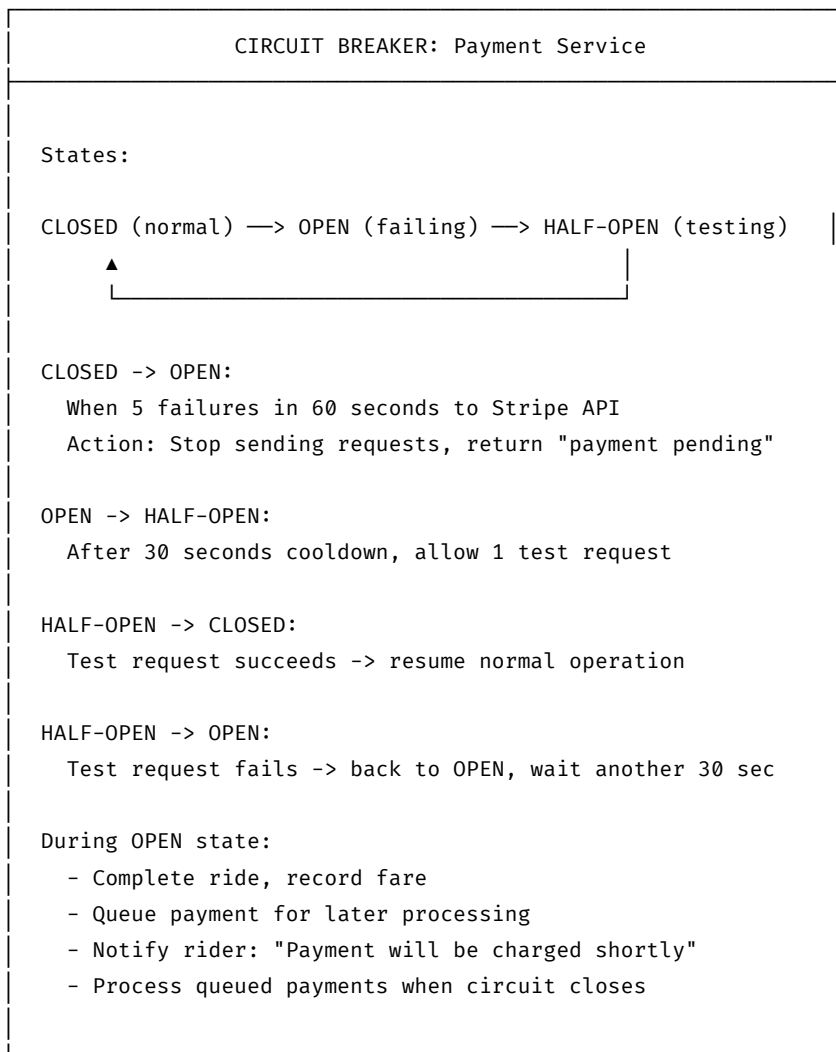
- Use PostgreSQL advisory locks or optimistic locking (version column)
- UPDATE trips SET status = 'ACCEPTED', version = version + 1  
WHERE trip\_id = ? AND status = 'MATCHING' AND version = ?
- If affected\_rows = 0 -> concurrent modification, retry or reject

## **10.12 12. Fault Tolerance and Failure Handling**

**10.12.1 12.1 Failure Scenarios and Mitigations**

| Failure Scenario                          | Mitigation                                                                                                                                                                                       |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Matching timeout<br>(no driver responds)  | Retry with expanded radius (3km -> 5km -> 8km). After 3 expansions, notify rider "No drivers available" with retry option.                                                                       |
| Driver app disconnects<br>mid-ride        | If no location update for 30 sec, mark driver as disconnected. Hold trip in IN_PROGRESS state. If > 5 min, notify rider, offer to cancel or wait. Use last known location for partial fare calc. |
| Payment processing<br>failure             | Pre-authorize card before matching. If capture fails at ride end: retry 3x with exponential backoff. Queue for manual review. Still pay driver from platform float.                              |
| Location Service down<br>(Redis failure)  | Redis cluster failover (< 15 sec). During outage, use last known positions (stale by seconds). Active rides use WebSocket direct connection as fallback.                                         |
| Ride Service crash                        | Stateless service; restart on any node. Trip state in PostgreSQL survives crash. Kubernetes restarts pod in < 10 sec.                                                                            |
| Double dispatch<br>(two drivers assigned) | Optimistic locking on trip.status. Only one UPDATE can succeed (MATCHING -> ACCEPTED). Losing driver gets "ride no longer available" response.                                                   |
| Kafka consumer lag                        | Location archival is async; lag is OK. Monitor consumer lag. Scale consumers. Real-time queries use Redis, not Kafka.                                                                            |
| Network partition<br>between regions      | City-level isolation: partition in one city doesn't affect other cities. Each city can operate independently.                                                                                    |
| Surge pricing service<br>down             | Default to 1.0x (no surge) if service is unavailable. Redis TTL ensures stale surge expires. Riders get normal pricing.                                                                          |
| GPS drift / inaccuracy                    | Kalman filter on driver app to smooth GPS readings. Reject updates with accuracy > 50m. Snap-to-road algorithm to place driver on nearest road.                                                  |

### 10.12.2 12.2 Circuit Breaker Pattern



## 10.13 13. Scalability

### 10.13.1 13.1 Scaling Strategy

#### SCALABILITY ARCHITECTURE

##### Dimension 1: Horizontal Service Scaling

All services are stateless (state in DB/Redis)  
Kubernetes HPA: auto-scale on CPU/request rate

| Service          | Baseline | Peak (NYE) |
|------------------|----------|------------|
| Ride Service     | 50 pods  | 500 pods   |
| Location Service | 100 pods | 300 pods   |
| Matching Service | 30 pods  | 200 pods   |
| Pricing Service  | 20 pods  | 100 pods   |
| Payment Service  | 30 pods  | 150 pods   |
| Notification Svc | 40 pods  | 200 pods   |

##### Dimension 2: Data Layer Scaling

###### PostgreSQL:

- Vertical scaling to 64-core, 512GB RAM
- Read replicas for read-heavy queries
- Partitioning by city\_id for trips table
- Connection pooling via PgBouncer (10K+ conns)

###### Redis:

- Cluster mode: 16 shards -> 64 shards
- Each shard: 64GB RAM, handles 100K ops/sec
- Total cluster: 1.6M+ ops/sec capacity

###### Cassandra:

- Add nodes for storage growth
- Rebalance with vnodes
- Target: 3-5 TB per node

###### Kafka:

- 128 partitions for location topic
- Add brokers for throughput
- Target: 2M messages/sec per cluster

### Dimension 3: Geographic Isolation

City-level isolation for blast radius containment

Each city cluster:

- Own Redis GeoSet shard
- Own PostgreSQL partition
- Own surge pricing computation
- Can operate independently during outages

Regional grouping:

US-West: SF, LA, Seattle, Portland

US-East: NYC, Boston, DC, Miami

Europe: London, Paris, Berlin, Amsterdam

APAC: Singapore, Mumbai, Sydney, Tokyo

### 10.13.2 13.2 Handling 10x Traffic Spikes (New Year's Eve)

Preparation (weeks before):

1. Pre-scale all services to 3x baseline capacity
2. Warm up Redis with additional shards
3. Pre-authorize Kafka topic partitions
4. Alert on-call teams across all time zones

During spike:

1. Auto-scaling kicks in based on request rate metrics
2. Surge pricing activates in high-demand cells
3. Enable "broadcast dispatch" mode (offer ride to 3 drivers simultaneously)
4. Relax matching criteria (accept drivers up to 10 km away)
5. Enable ride request queue: "All drivers busy, estimated wait: 8 min"

Graceful degradation order:

1. Disable ride history queries (non-critical)
2. Disable analytics pipeline (async, can catch up)
3. Simplify ETA to straight-line estimate (skip road network)
4. Increase location update interval to 8 seconds (halve write load)
5. Last resort: limit new ride requests by city (queue with ETA)

## 10.14 14. Monitoring and Observability



### 10.14.1 14.1 Key Metrics Dashboard

| RIDE-SHARING METRICS DASHBOARD |                           |
|--------------------------------|---------------------------|
| <b>MATCHING METRICS</b>        | <b>RIDE METRICS</b>       |
| Avg Match Time: 2.4 sec        | Active Rides: 847,293     |
| P99 Match Time: 8.1 sec        | Rides/min: 14,200         |
| Match Success %: 94.7%         | Completion Rate: 96.2%    |
| No Driver Rate: 3.1%           | Cancellation Rate: 3.8%   |
| Avg Attempts: 1.3              | Avg Ride Duration: 14 min |
| <b>LOCATION METRICS</b>        | <b>PRICING METRICS</b>    |
| Updates/sec: 1,247,000         | Avg Surge: 1.12x          |
| Update Lag P99: 120 ms         | % Rides Surged: 18.4%     |
| Stale Drivers: 0.3%            | Max Surge: 2.8x           |
| GeoQuery P99: 12 ms            | Avg Fare: \$18.50         |
| Redis Memory: 78% used         | Revenue/hour: \$2.1M      |
| <b>DRIVER METRICS</b>          | <b>SYSTEM HEALTH</b>      |
| Online Drivers: 4,821,000      | API P99 Latency: 180 ms   |
| Busy (on trip): 847,293        | Error Rate: 0.02%         |
| Available: 3,973,707           | CPU Util (avg): 42%       |
| Avg Wait (idle): 6.2 min       | Redis Hit Rate: 99.1%     |
| Driver Churn: 2.1%/month       | Kafka Consumer Lag: 1.2s  |

### 10.14.2 14.2 Alerting Rules

CRITICAL (Page immediately):

- Match success rate < 85% for 5 min
- Location update ingestion drops > 30% suddenly
- Payment failure rate > 5% for 2 min
- Active rides count drops > 20% in 5 min (data loss?)
- Redis cluster node down (failover should be automatic)
- API error rate > 1% for 3 min

WARNING (Slack notification):

- P99 matching time > 10 seconds
- Surge multiplier > 2.5x in any cell for > 30 min
- Driver churn rate > 3% in a week
- Kafka consumer lag > 60 seconds
- PostgreSQL replication lag > 5 seconds
- Redis memory usage > 85%

INFO (Dashboard only):

- New peak concurrent rides record
- Surge activated/deactivated per city
- Service auto-scaling events

### 10.14.3 14.3 Distributed Tracing

Trace: Ride Request to Completion

```
[Rider App] —POST /rides/request—>
 [API Gateway: 2ms] —>
 [Ride Service: 5ms] —>
 [Pricing Service: 8ms] (fare estimate + surge lookup)
 [Matching Service: 15ms] —>
 [Location Service: 3ms] (GEORADIUS query)
 [ETA Service: 25ms] (road network calculation)
 [Notification Service: 5ms] (push to driver)
```

Total E2E Latency for ride request: ~50ms server-side

Driver notification received: ~200ms (including push network)

Driver response (accept): 5-15 sec (human decision time)

Trace ID propagated through all services via X-Trace-ID header.

Stored in Jaeger/Zipkin for debugging.

## 10.15 15. Trade-offs and Design Decisions

**10.15.1 15.1 Key Trade-offs**

| DESIGN TRADE-OFFS |                                                                                                                                                                   |                                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Decision          | Option A                                                                                                                                                          | Option B                                                                                                        |
| Geospatial Index  | Redis GeoHash<br>+ Simple, built-in<br>+ Fast integration<br>- Edge case at cell boundaries<br>CHOSE: Redis for MVP<br>+ H3 for surge/ analytics                  | Custom QuadTree/H3<br>+ Adaptive density<br>+ Better for analytics<br>- Build & maintain<br>- Higher complexity |
| Dispatch Model    | Sequential (one-by-one offer)<br>+ Better match<br>+ Driver UX<br>- Slower (15s/try)<br>CHOSE: Sequential with fast timeout (10s) and broadcast fallback in surge | Broadcast (send to multiple drivers)<br>+ Faster matching<br>- Wastes driver attention                          |
| Location Protocol | HTTP REST (poll)<br>+ Simpler<br>+ Stateless<br>- HTTP overhead per update (headers)<br>CHOSE: WebSocket for active drivers, REST for infrequent calls            | WebSocket (push)<br>+ Lower overhead<br>+ Bidirectional<br>- Stateful connections<br>- Reconnection logic       |

|               |                                                                                                                                                                      |                                                                                                                                              |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Pricing Model | Post-trip metered (actual distance/ time)<br>+ Accurate billing<br>- Rider anxiety about final fare<br><br>CHOSE: Upfront with adjustment for major route deviations | Upfront fixed pricing (estimate at request time, locked in)<br>+ Price transparency<br>+ Rider confidence<br>- Platform absorbs routing risk |
| Trip Storage  | Single PostgreSQL with partitions<br>+ Simpler ops<br>+ SQL joins<br>- Scale ceiling<br>CHOSE: Dual-store (PG for active + Cassandra for cold)                       | PostgreSQL (active) + Cassandra (historical)<br>+ Scale reads for history queries<br>- Dual-write complexity                                 |
| ETA Approach  | Simple (straight-line * factor)<br>+ Fast, no external dependency<br>- Inaccurate in complex road nets<br>CHOSE: Full road network with fallback to simple method    | Full road network with traffic<br>+ Accurate<br>+ Trusted by users<br>- Expensive to compute<br>- Map data dependency                        |

### 10.15.2 15.2 Why Not a Single Database?

Why we split across PostgreSQL, Redis, and Cassandra:

PostgreSQL alone:

- Cannot handle 1.25M geo-writes/sec (write bottleneck)
- PostGIS is good for geo queries but not at this write volume
- Fine for trips (20M/day = ~230 writes/sec) and users

Redis alone:

- No ACID for trip state transitions (data loss risk)
- No durable storage (persistence is slow / lossy)
- Perfect for ephemeral location data with TTL

Cassandra alone:

- No multi-key transactions (trip + payment atomicity)
- Eventual consistency not suitable for payments
- Great for append-only trip history (time-series pattern)

Hybrid approach gives best of each:

PostgreSQL: Transactional data (trips, payments, users)

Redis: Real-time ephemeral data (locations, cache, surge)

Cassandra: High-volume historical data (trip logs, analytics)

Kafka: Event streaming (location stream, trip events)

---

## 10.16 16. Interview Deep-Dive Questions

### 10.16.1 Q1: How do you handle drivers at city boundaries?

**Answer:** We define overlap zones – 5 km buffer areas around city boundaries. Drivers within these zones are indexed in GeoSets for both adjacent cities. When the Location Service receives an update from a boundary driver, it writes to both city shards. This doubles writes for ~5% of drivers but eliminates cross-shard reads during matching. The rider's pickup location determines which city shard is queried. If a driver is 3 km away but in a different city shard, the overlap zone ensures they still appear in the query results.

For country boundaries (e.g., US-Mexico border), regulatory restrictions may prevent cross-border matching entirely, enforced via geo-fencing rules.

---

### 10.16.2 Q2: How do you prevent surge pricing abuse?

**Answer:** Several mechanisms:

1. **Smoothing:** Surge multiplier uses exponential moving average, changing by at most 0.3x per computation cycle (60 sec). Prevents artificial spikes.
  2. **Minimum duration:** Surge must persist for at least 3 computation cycles (3 min) before being displayed to riders. Brief demand spikes are absorbed.
  3. **Hard cap:** Maximum surge is 3.0x. Above that, we absorb the cost differential.
-

4. **Driver gaming detection:** If multiple drivers go offline simultaneously in the same cell (colluding to create artificial shortage), flag for review. Use historical patterns to detect anomalies.
  5. **Rider-side protection:** Riders must explicitly confirm rides above 2.0x surge. Show “prices are higher than normal” warning with clear multiplier.
  6. **Regulatory compliance:** Disable surge during declared emergencies (natural disasters, severe weather events). Configurable per city.
- 

### 10.16.3 Q3: How do you handle ride cancellation mid-trip?

**Answer:** Several scenarios:

**Rider cancels mid-trip:** - Trip status transitions to CANCELLED - Rider is charged a partial fare based on distance/time so far - Driver receives partial payment + cancellation compensation - Trip route is saved up to the cancellation point - If rider frequently cancels, their reliability score decreases

**Driver cancels mid-trip:** - Platform reassigns trip to another nearby driver if possible - Current driver penalized (affects acceptance rate, future dispatch priority) - Rider is not charged for the cancelled portion - If no replacement driver found, rider gets full refund + credit - Repeated driver cancellations trigger review/suspension

**System cancellation (driver disconnect):** - If driver's location updates stop for > 2 minutes during active trip - System attempts to contact driver via push notification + SMS - After 5 minutes, system auto-cancels and attempts reassignment - Rider notified with options: wait, cancel, or accept new driver - Fare calculated based on last known position

---

### 10.16.4 Q4: How would you design ride pooling/sharing?

**Answer:** Ride pooling (UberPool/Lyft Shared) adds significant complexity:

Pooling Matching Algorithm:

1. Rider A requests Pool from point X to point Y
2. Before matching a dedicated driver, check active Pool trips:
  - Find trips where adding this rider has  $< 40\%$  detour
  - $\text{Detour} = (\text{new\_total\_distance} - \text{original\_distance}) / \text{original\_distance}$
3. If compatible trip found:
  - Update route to include Rider A's pickup and dropoff
  - Notify existing riders of slight route change
  - Adjust fare: each rider pays  $\sim 60\%$  of solo fare
4. If no compatible trip: match with new driver, keep trip "open" for 2 minutes to accept additional riders

Route Optimization:

- Given N pickup/dropoff points, find optimal ordering
- This is a variant of the Traveling Salesman Problem
- For  $N \leq 4$  riders (8 points): brute force is feasible ( $8! = 40,320$ )
- Constraint: never pass a rider's dropoff before their pickup
- Use greedy insertion heuristic for real-time response

Data Model Addition:

- pool\_trip\_id: groups individual rider trips
- seat\_count: available seats (start at 3, decrement per rider)
- detour\_tolerance: configurable per rider (10-40%)

### 10.16.5 Q5: How do you ensure payment consistency?

**Answer:** We use a two-phase approach:

**Phase 1: Pre-Authorization (at ride request)** - When rider confirms ride, we authorize (hold) the estimated fare on their card - Stripe: `PaymentIntent.create(amount, capture_method='manual')` - If auth fails -> reject ride request immediately - Hold expires after 7 days if not captured

**Phase 2: Capture (at ride completion)** - When driver marks trip complete, calculate actual fare - If actual fare  $\leq$  authorized amount: capture actual amount - If actual fare  $>$  authorized amount (e.g., route change): capture original, charge difference separately - Use idempotency keys to prevent double-charging - All payment operations wrapped in PostgreSQL transaction with trip status update

BEGIN;

```
UPDATE trips SET status = 'COMPLETED', fare_actual = $28.50
WHERE trip_id = ? AND status = 'IN_PROGRESS';
INSERT INTO payments (trip_id, amount, status)
VALUES (?, 28.50, 'CAPTURED');
```

```
-- Call Stripe async, update payment status on callback
```

COMMIT;

For cash payments (common in emerging markets): trip completes, driver collects cash, platform debits driver's earnings balance for platform commission.

### 10.16.6 Q6: How does the system handle GPS inaccuracy?

**Answer:**

1. **Kalman Filter:** Driver app applies Kalman filtering to smooth raw GPS readings, reducing noise from urban canyons, tunnels, and buildings.
  2. **Snap-to-Road:** Server-side algorithm maps GPS coordinates to the nearest road segment using road network graph. Prevents drivers from appearing inside buildings or rivers.
  3. **Accuracy Threshold:** Reject location updates with accuracy > 50 meters. Wait for better GPS fix.
  4. **Speed Validation:** If implied speed between two updates > 200 km/h (impossible for car), discard the outlier update.
  5. **Interpolation:** During GPS blackouts (tunnels), interpolate position along the expected route using last known speed and heading.
- 

### 10.16.7 Q7: How do you handle a complete data center failure?

#### Answer:

- **Active-Active:** Deploy in 2+ regions per continent. US-West and US-East both serve US traffic, partitioned by city.
  - **City-level failover:** Each city is assigned to a primary region. If US-West fails, SF/LA/Seattle traffic fails over to US-East.
  - **DNS failover:** Route53/Cloud DNS health checks detect region failure within 30 seconds, update DNS to redirect traffic.
  - **Data sync:** PostgreSQL cross-region async replication. Accept ~1 second of data loss during failover (trips in flight get reconciled from driver app state on reconnection).
  - **Redis:** No cross-region replication for location data. Drivers reconnect and re-send current location to the new region's Redis. Recovery time: ~15 seconds as all drivers re-register.
  - **Stateless services:** All microservices are stateless; they start fresh in the failover region using the replicated databases.
- 

### 10.16.8 Q8: How do you calculate the optimal pickup point?

#### Answer:

The rider's GPS pin may be in a building, park, or wrong side of a highway. The pickup point optimization involves:

1. **Snap to nearest road:** Map the GPS coordinate to the closest drivable road.
  2. **Consider road directionality:** Place pickup on the correct side of a one-way street or divided highway.
  3. **Accessibility check:** Ensure the point is not in a no-stop zone (bus lane, highway ramp, fire lane).
  4. **Landmarks:** Cross-reference with known pickup spots (building entrances, designated rideshare zones at airports).
  5. **Historical data:** Use ML model trained on where drivers actually stopped for previous pickups at similar GPS coordinates.
-



**10.16.9 Q9: How does the system handle peak events (concerts, sports)?****Answer:**

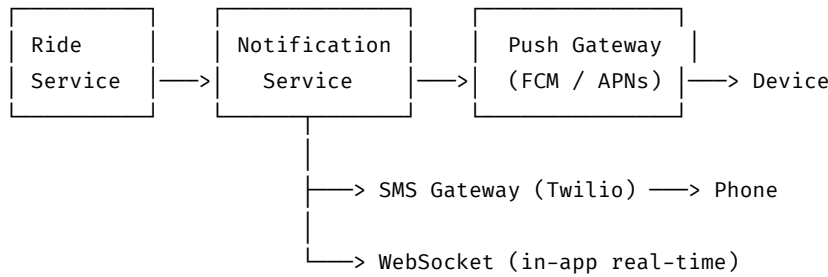
1. **Geo-fence the venue:** Define virtual boundary around the event location.
  2. **Pre-positioning:** 1-2 hours before event ends, send drivers “high demand expected” notifications with bonus incentives to position near venue.
  3. **Queuing system:** When event ends (mass demand spike), implement virtual queue: “You are #47 in line. Estimated wait: 12 min.”
  4. **Staged pickup zones:** Define specific pickup lanes (Zone A, B, C) to prevent congestion. Assign riders to zones based on destination direction.
  5. **Pre-surge:** Gradually increase surge 30 min before event ends (predicted demand), attracting drivers before the actual spike hits.
  6. **Capacity pre-scaling:** Auto-scale services 1 hour before event based on historical data for this venue.
- 

**10.16.10 Q10: How do you prevent fraud (fake rides, GPS spoofing)?****Answer:**

1. **GPS spoofing detection:** Compare reported GPS with cell tower triangulation and WiFi positioning. If GPS reports driver in Manhattan but cell tower says Brooklyn, flag the trip.
  2. **Trip anomaly detection:** ML model flags trips where:
    - Distance is 10x the straight-line distance (circular driving)
    - Trip speed is suspiciously consistent (fake GPS replay)
    - Pickup and dropoff are the same location
  3. **Collusion detection:** Flag driver-rider pairs who:
    - Ride together > 5 times in a week
    - Always rate each other 5 stars
    - Rides consistently happen during promo/bonus periods
  4. **Device fingerprinting:** Track device changes. If a driver account switches devices frequently or runs on emulators, flag for review.
  5. **Photo verification:** Periodically ask drivers to take a selfie; compare against registered photo using face recognition.
- 

**10.16.11 Q11: How do you design the notification system for ride updates?****Answer:**

Notification Flow:



Notification Types:

- RIDE\_MATCHED: "Your driver John is 3 min away" (push + in-app)
- DRIVER\_ARRIVING: "Your driver is arriving now" (push + in-app)
- TRIP\_STARTED: "Your trip has started" (in-app)
- TRIP\_COMPLETED: "Trip complete. \$18.50" (push + in-app + email)
- DRIVER\_RIDE\_REQUEST: "New ride request: 0.8 mi away" (push + sound)

Reliability:

- Primary: WebSocket for in-app users (instant)
- Fallback: Push notification if WebSocket disconnected
- Last resort: SMS for critical messages (ride matched, safety alerts)
- Each notification has a TTL; expired ride requests are not delivered

### 10.16.12 Q12: How do you handle international expansion?

Answer:

1. **Multi-region deployment:** Deploy service clusters per continent (US, EU, APAC, LATAM). Each region handles its own cities independently.
2. **Regulatory compliance:** Different cities have different rules:
  - License requirements (London requires specific driver licenses)
  - Pricing regulations (some cities cap surge)
  - Data residency (EU: GDPR requires data to stay in EU)
  - Cash payment support (India, Brazil)
3. **Currency handling:** Store all amounts in minor units (cents) with currency code. Convert at payment time using real-time exchange rates.
4. **Localization:** Ride types, UI strings, support channels localized per market. Right-to-left support for Arabic-speaking markets.
5. **Map provider flexibility:** Google Maps in most markets, local providers where required (Baidu Maps in China, Yandex in Russia).

### 10.16.13 Q13: How do you ensure driver and rider safety?

Answer:

1. **Real-time trip sharing:** Rider can share trip link with contacts showing live location. Emergency button sends location to local authorities.

2. **Trip recording:** Audio recording (opt-in) stored for dispute resolution. Video recording in some markets.
  3. **Route deviation alerts:** If driver deviates significantly from expected route, alert rider and safety team.
  4. **Identity verification:** Rider sees driver photo, name, license plate before pickup. Driver sees rider name and rating.
  5. **Trusted contacts:** Automated notification to rider's emergency contacts for late-night rides or when trip takes unexpectedly long.
  6. **RideCheck:** ML model monitors trip sensors (sudden stops, long unexpected pauses, unusual route). Proactively asks "Is everything OK?" to both parties.
- 

#### 10.16.14 Q14: How do you handle scheduled (future) rides?

##### Answer:

Scheduled Ride Flow:

1. Rider requests ride for future time T
2. System stores `scheduled_ride` record:  
`{rider_id, pickup, dropoff, scheduled_at: T, ride_type}`
3. No driver is matched at this point

At T - 15 minutes:

4. Scheduler (cron-like service) picks up the record
5. Creates a normal ride request
6. Matching Service finds and dispatches driver
7. Driver receives: "Scheduled pickup at 6:00 PM at 123 Market St"

Why not pre-match?

- Driver availability 15 min from now is unpredictable
- Driver may go offline, be on another trip, or relocate
- Pre-matching wastes driver time (idle waiting)
- 15-min window balances reliability with efficiency

Reliability:

- If first matching attempt fails, retry every 2 min
  - If no driver found by T - 5 min, notify rider
  - Offer to try on-demand or cancel with full refund
  - For premium service: guarantee with pre-matched dedicated drivers
- 

#### 10.16.15 Q15: How would you redesign the matching algorithm using ML?

##### Answer:

Move from rule-based scoring to ML-based matching:

**Training Data:** - Historical trips: (driver, rider, context) -> outcome - Outcome: acceptance probability, trip completion, rider rating

**Features:** - Driver: current location, heading, time online, trips today, rating, vehicle type - Rider: pickup/dropoff locations, ride history, rating, payment reliability - Context: time of day, day of week, weather, surge level, event proximity

**Model:** - Multi-objective optimization: - Minimize rider wait time (ETA) - Maximize match acceptance probability - Maximize trip completion probability - Maximize driver earnings fairness (distribute trips evenly) - Use gradient-boosted trees (XGBoost) for real-time inference (< 5ms) - Retrain daily on last 30 days of data

**A/B Testing:** - Run ML matching alongside rule-based matching in shadow mode - Compare: match rate, cancellation rate, rider satisfaction, driver satisfaction - Gradually shift traffic: 1% -> 5% -> 25% -> 100%

**Advanced: Batched Matching (Uber's approach):** - Instead of matching rides one-by-one, batch all pending requests in a 2-second window - Solve a global optimization: assign N riders to M drivers to minimize total wait time - This is a bipartite matching problem solvable with the Hungarian algorithm:  $O(N^3)$  - For N = 100 pending requests per batch in a city: feasible in < 50ms

---

## 10.17 17. System Evolution and Future Considerations

### 10.17.1 Phase 1: MVP (Months 1-3)

- Single city, one ride type (sedan)
- Redis GeoHash for driver locations
- PostgreSQL for everything else
- Basic sequential matching
- Fixed pricing (no surge)
- Stripe integration for payments

### 10.17.2 Phase 2: Scale (Months 4-8)

- Multi-city expansion with city-level sharding
- Surge pricing pipeline
- Multiple ride types (XL, Black)
- Cassandra for trip history archival
- ETA service with road network routing
- Driver/rider ratings

### 10.17.3 Phase 3: Optimization (Months 9-12)

- ML-based ETA prediction
- ML-based matching (batched optimization)
- Ride pooling
- Scheduled rides
- Advanced fraud detection
- International expansion framework

### 10.17.4 Phase 4: Platform (Year 2+)

- Multi-modal transport (bikes, scooters)
- Autonomous vehicle integration

- Food delivery (shared driver fleet)
- Enterprise solutions (business accounts)
- Real-time ML model serving at edge

## 10.18 18. Quick Reference Card

| RIDE-SHARING SYSTEM DESIGN CHEAT SHEET                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Scale Numbers:</p> <p>20M rides/day, 5M online drivers, 1.25M location updates/sec</p>                                                                                                                                                                                                                                                                                                                                           |
| <p>Key Components:</p> <p>Location Svc (Redis GeoHash) -&gt; Matching Svc -&gt; Ride Svc (PG)</p> <p>Pricing Svc (surge) -&gt; Payment Svc (Stripe) -&gt; Notification Svc</p>                                                                                                                                                                                                                                                      |
| <p>Critical Design Choices:</p> <ol style="list-style-type: none"><li>1. Redis GeoHash for real-time driver queries (not PostGIS)</li><li>2. Sequential dispatch with broadcast fallback</li><li>3. WebSocket for location streaming (not HTTP polling)</li><li>4. City-level sharding for blast radius containment</li><li>5. Upfront pricing with post-trip adjustment</li><li>6. Pre-authorize payment before matching</li></ol> |
| <p>Consistency Model:</p> <p>Eventually consistent: locations, surge, ratings</p> <p>Strongly consistent: trip state, payments</p>                                                                                                                                                                                                                                                                                                  |
| <p>Failure Modes to Discuss:</p> <p>Double dispatch -&gt; optimistic locking</p> <p>Payment failure -&gt; circuit breaker + queue</p> <p>DC failure -&gt; city-level failover</p> <p>GPS inaccuracy -&gt; Kalman filter + snap-to-road</p>                                                                                                                                                                                          |
| <p>Impress the Interviewer:</p> <ul style="list-style-type: none"><li>- Mention S2/H3 cells for surge zones (not just lat/lng grid)</li><li>- Contraction Hierarchies for fast ETA (not raw Dijkstra)</li><li>- Batched matching as global optimization (Hungarian algorithm)</li><li>- Exponential smoothing for surge (not instant spikes)</li></ul>                                                                              |

*Last updated: 2026-04-09*

## Chapter 11

# Design a Payment System (Stripe / PayPal)

**Difficulty:** Hard | **Companies:** Stripe, PayPal, Square, Amazon, Google, Visa

---

### 11.1 1. Problem Statement and Scope

Design a large-scale payment processing system similar to Stripe or PayPal that enables merchants to accept online payments from customers across the globe. The system must handle the full life-cycle of a payment – authorization, capture, settlement, refunds – while guaranteeing exactly-once processing, maintaining PCI DSS compliance, supporting multiple currencies, and performing financial reconciliation.

#### 11.1.1 In Scope

- **Payment processing:** Credit card, debit card, and bank transfer (ACH/SEPA).
- **Payment lifecycle:** Authorize, capture, settle, void, refund.
- **Merchant management:** Onboarding, configuration, dashboard, payout scheduling.
- **Multi-currency:** Accept payments in 135+ currencies, settle in merchant's local currency.
- **Recurring payments:** Subscriptions with billing cycles, retries on failure, dunning.
- **Webhooks and notifications:** Real-time event delivery to merchants.
- **Ledger and reconciliation:** Double-entry bookkeeping, daily bank reconciliation.
- **Compliance:** PCI DSS Level 1, SOC2, GDPR, PSD2 (Strong Customer Authentication).
- **Fraud detection:** Rule-based and ML-based scoring before authorization.
- **Reporting and analytics:** Payment success rates, revenue dashboards, dispute tracking.

#### 11.1.2 Out of Scope

- Physical point-of-sale (POS) terminal hardware.
  - Cryptocurrency payments.
  - Peer-to-peer (P2P) money transfers.
  - Full banking / neobank services (loans, savings accounts).
  - Issuing of payment cards.
-

## 11.2 2. Functional Requirements

- **FR-1 - Process payments:** Accept credit card, debit card, and bank transfer payments via API
  - **FR-2 - Authorization and capture:** Support separate auth/capture flows and combined auth+capture
  - **FR-3 - Refunds:** Full and partial refunds with automatic ledger reversal
  - **FR-4 - Payment status tracking:** Real-time status queries with deterministic state machine
  - **FR-5 - Merchant dashboard:** View transactions, revenue, chargebacks, payouts
  - **FR-6 - Multi-currency:** Accept 135+ currencies, auto-convert with live exchange rates
  - **FR-7 - Recurring payments:** Subscriptions with configurable billing intervals and retry logic
  - **FR-8 - Payment history:** Searchable, filterable history with export capability
  - **FR-9 - Webhooks:** Notify merchants of payment events with at-least-once delivery
  - **FR-10 - Merchant payouts:** Schedule and execute payouts to merchant bank accounts
  - **FR-11 - Dispute management:** Handle chargebacks, evidence submission, resolution tracking
  - **FR-12 - Payment method vault:** Securely store and tokenize payment methods for reuse
- 

## 11.3 3. Non-Functional Requirements

- **NFR-1 - Exactly-once processing:** No double charges under any failure scenario
  - **NFR-2 - Data consistency:** ACID for payment transactions, eventual for analytics
  - **NFR-3 - PCI DSS compliance:** Level 1 certification, no raw PAN storage in application tier
  - **NFR-4 - Latency:** < 500ms for payment API response (excluding PSP round-trip)
  - **NFR-5 - Availability:** 99.999% uptime (< 5.26 minutes downtime per year)
  - **NFR-6 - Audit trail:** Every state change logged immutably with actor, timestamp, reason
  - **NFR-7 - Fault tolerance:** Survive single data center failure with zero data loss
  - **NFR-8 - Idempotency:** Every mutating API is idempotent via client-supplied key
  - **NFR-9 - Throughput:** Handle 100 TPS peak with ability to scale to 1,000 TPS
  - **NFR-10 - Data durability:** Zero tolerance for lost payment records (RPO = 0)
  - **NFR-11 - Encryption:** TLS 1.3 in transit, AES-256 at rest, HSM for key management
  - **NFR-12 - Regulatory:** GDPR-compliant data handling, PSD2 SCA support
- 

## 11.4 4. Back-of-Envelope Estimation

### 11.4.1 Traffic Estimates

|                            |                                                         |
|----------------------------|---------------------------------------------------------|
| Daily transactions:        | 1,000,000                                               |
| Seconds per day:           | 86,400                                                  |
| Average TPS:               | $1,000,000 / 86,400 = \sim 12$ TPS                      |
| Peak TPS (10x average):    | $\sim 120$ TPS                                          |
| Black Friday peak (30x):   | $\sim 360$ TPS                                          |
| API calls per transaction: | $\sim 5$ (create, auth, capture, status check, webhook) |
| Total API calls/day:       | 5,000,000                                               |
| Peak API calls/second:     | $\sim 600$                                              |
| Webhook deliveries/day:    | 2,000,000 (2 events per transaction avg)                |

### 11.4.2 Storage Estimates

|                             |                                         |
|-----------------------------|-----------------------------------------|
| Payment record size:        | $\sim 2$ KB (order + metadata)          |
| Ledger entries per payment: | 2-4 entries x 500 bytes = $\sim 1.5$ KB |
| Audit log per payment:      | $\sim 3$ KB (multiple state changes)    |
| Total per payment:          | $\sim 6.5$ KB                           |

|                  |                                 |
|------------------|---------------------------------|
| Daily storage:   | $1M \times 6.5$ KB = 6.5 GB/day |
| Monthly storage: | $\sim 195$ GB/month             |
| Yearly storage:  | $\sim 2.4$ TB/year              |

|                                                   |  |
|---------------------------------------------------|--|
| Payment ledger (append-only):                     |  |
| - 1M transactions x 3 entries x 500B = 1.5 GB/day |  |
| - Yearly: $\sim 550$ GB/year                      |  |
| - 5-year retention: $\sim 2.75$ TB                |  |

|                                                       |  |
|-------------------------------------------------------|--|
| Encrypted card vault:                                 |  |
| - 10M unique cards x 1 KB = 10 GB (relatively static) |  |

### 11.4.3 Compute Estimates

|                            |                                                      |
|----------------------------|------------------------------------------------------|
| Payment service instances: | 10-20 (each handles $\sim 10$ -15 TPS with headroom) |
| Database connections:      | 200-400 (connection pooling)                         |
| Redis cache nodes:         | 3 (clustered, for idempotency + config)              |
| Worker nodes (async):      | 10-15 (settlement, reconciliation, webhooks)         |

### 11.4.4 Financial Estimates

|                            |                                                  |
|----------------------------|--------------------------------------------------|
| Average transaction value: | \$50                                             |
| Daily GMV:                 | \$50M                                            |
| Annual GMV:                | \$18.25B                                         |
| Revenue (2.9% + \$0.30):   | $\sim \$529M + \$109M = \sim \$638M/\text{year}$ |

|                         |                                                  |
|-------------------------|--------------------------------------------------|
| Fraud rate target:      | $< 0.1\%$ of GMV = $< \$18.25M/\text{year}$      |
| Chargeback rate target: | $< 0.5\%$ of transactions = $< 5,000/\text{day}$ |



## 11.5 5. API Design

### 11.5.1 5.1 Create Payment (Auth + Capture or Auth Only)

POST /v1/payments

Headers:

```
Authorization: Bearer {api_key}
Idempotency-Key: {uuid} # Required for all mutating operations
Content-Type: application/json
```

Request Body:

```
{
 "amount": 5000, # Amount in smallest currency unit (cents)
 "currency": "usd", # ISO 4217 currency code
 "payment_method": "pm_abc123", # Tokenized payment method ID
 "merchant_id": "merch_xyz", # Merchant identifier
 "capture": true, # false = auth-only, true = auth+capture
 "description": "Order #12345",
 "metadata": { # Arbitrary key-value pairs
 "order_id": "ord_12345",
 "customer_email": "user@example.com"
 },
 "return_url": "https://merchant.com/payment/complete",
 "statement_descriptor": "ACME CORP" # Appears on bank statement
}
```

Response (201 Created):

```
{
 "id": "pay_a1b2c3d4e5",
 "object": "payment",
 "amount": 5000,
 "currency": "usd",
 "status": "succeeded", # or "requires_action", "processing"
 "payment_method": "pm_abc123",
 "merchant_id": "merch_xyz",
 "capture": true,
 "created_at": "2026-04-09T10:30:00Z",
 "updated_at": "2026-04-09T10:30:01Z",
 "idempotency_key": "uuid-here",
 "metadata": { "order_id": "ord_12345" },
 "receipt_url": "https://payments.example.com/receipts/pay_a1b2c3d4e5"
}
```

### 11.5.2 5.2 Get Payment

GET /v1/payments/{payment\_id}

Headers:

Authorization: Bearer {api\_key}

Response (200 OK):

```
{
 "id": "pay_a1b2c3d4e5",
 "object": "payment",
 "amount": 5000,
 "amount_captured": 5000,
 "amount_refunded": 0,
 "currency": "usd",
 "status": "succeeded",
 "payment_method": "pm_abc123",
 "merchant_id": "merch_xyz",
 "psp_reference": "psp_ref_xyz", # Reference from downstream PSP
 "risk_score": 12, # 0-100, higher = riskier
 "created_at": "2026-04-09T10:30:00Z",
 "captured_at": "2026-04-09T10:30:01Z",
 "settled_at": null,
 "timeline": [# Full audit trail
 { "event": "created", "at": "2026-04-09T10:30:00Z" },
 { "event": "authorized", "at": "2026-04-09T10:30:00.5Z" },
 { "event": "captured", "at": "2026-04-09T10:30:01Z" }
]
}
```

### 11.5.3 5.3 Capture Payment (for Auth-only Flows)

POST /v1/payments/{payment\_id}/capture

Headers:

Authorization: Bearer {api\_key}

Idempotency-Key: {uuid}

Request Body:

```
{
 "amount": 4500 # Optional: partial capture (up to authorized amount)
}
```

Response (200 OK):

```
{
 "id": "pay_a1b2c3d4e5",
 "status": "captured",
 "amount": 5000,
 "amount_captured": 4500,
 "captured_at": "2026-04-09T12:00:00Z"
}
```

### 11.5.4 5.4 Refund Payment

POST /v1/payments/{payment\_id}/refund

Headers:

Authorization: Bearer {api\_key}  
Idempotency-Key: {uuid}

Request Body:

```
{
 "amount": 2000, # Partial refund; omit for full refund
 "reason": "customer_request", # customer_request | duplicate | fraudulent
 "metadata": {
 "support_ticket": "TKT-789"
 }
}
```

Response (201 Created):

```
{
 "id": "rfnd_f6g7h8i9",
 "object": "refund",
 "payment_id": "pay_a1b2c3d4e5",
 "amount": 2000,
 "currency": "usd",
 "status": "pending", # pending → succeeded | failed
 "reason": "customer_request",
 "created_at": "2026-04-09T14:00:00Z"
}
```

### 11.5.5 5.5 List Payments

GET /v1/payments?merchant\_id=merch\_xyz&status=succeeded&limit=50&starting\_after=pay\_xxx

Headers:

Authorization: Bearer {api\_key}

Response (200 OK):

```
{
 "object": "list",
 "data": [...],
 "has_more": true,
 "total_count": 15234
}
```

### 11.5.6 5.6 Create Payment Method (Tokenization)

POST /v1/payment\_methods

Headers:

Authorization: Bearer {publishable\_key} # Client-side publishable key

Request Body:

```
{
 "type": "card",
 "card": {
 "number": "4242424242424242", # Only handled in PCI-compliant vault
 "exp_month": 12,
 "exp_year": 2028,
 "cvc": "123"
 },
 "billing_details": {
 "name": "Jane Doe",
 "email": "jane@example.com"
 }
}
```

Response (201 Created):

```
{
 "id": "pm_abc123",
 "object": "payment_method",
 "type": "card",
 "card": {
 "brand": "visa",
 "last4": "4242",
 "exp_month": 12,
 "exp_year": 2028,
 "fingerprint": "fp_unique_hash" # For dedup across merchants
 }
}
```

### 11.5.7 5.7 Webhook Registration

POST /v1/webhooks/endpoints

Headers:

Authorization: Bearer {api\_key}

Request Body:

```
{
 "url": "https://merchant.com/webhooks/payments",
 "events": ["payment.succeeded", "payment.failed", "refund.created"],
 "secret": "whsec_..." # Auto-generated signing secret
}
```

### 11.5.8 5.8 Idempotency Key Handling

Idempotency Contract:

1. Client sends Idempotency-Key header with every mutating request.
  2. Server stores: (idempotency\_key, merchant\_id) -> (status\_code, response\_body, created\_at).
  3. If a duplicate key is received:
    - a. If original request is still processing: return 409 Conflict.
    - b. If original request completed: return the stored response (same status code + body).
    - c. If original request failed: allow retry (key is "consumable" only on success).
  4. Idempotency records expire after 24 hours.
  5. Keys are scoped per merchant (same key from different merchants = different requests).
- 

## 11.6 6. Data Model and Database Selection

### 11.6.1 6.1 Database Selection Rationale

- **Payment orders:** Database: PostgreSQL (primary); Rationale: ACID transactions, strong consistency, mature ecosystem
- **Ledger entries:** Database: PostgreSQL (separate cluster); Rationale: Double-entry bookkeeping requires ACID, append-only
- **Payment methods (vault):** Database: PostgreSQL + HSM; Rationale: Encrypted at column level, tokenized references
- **Merchant config:** Database: PostgreSQL + Redis cache; Rationale: Relatively static, frequently read
- **Idempotency keys:** Database: Redis (primary) + PostgreSQL (backup); Rationale: Fast lookup, 24h TTL, persistence for crash recovery
- **Webhook events:** Database: PostgreSQL + Kafka; Rationale: Ordered, durable event stream for reliable delivery
- **Analytics/reporting:** Database: ClickHouse or BigQuery; Rationale: Columnar, optimized for aggregation queries
- **Audit logs:** Database: Append-only PostgreSQL / S3; Rationale: Immutable, tamper-evident, long retention

## 11.6.2 6.2 Core Schema

### 11.6.2.1 payments (Primary payment order table)

```

CREATE TABLE payments (
 id VARCHAR(26) PRIMARY KEY, -- ULID (sortable, unique)
 merchant_id VARCHAR(26) NOT NULL, -- FK to merchants
 idempotency_key VARCHAR(64), -- Client-provided dedup key
 amount BIGINT NOT NULL, -- In smallest currency unit (cents)
 currency CHAR(3) NOT NULL, -- ISO 4217
 status VARCHAR(20) NOT NULL, -- CREATED, PROCESSING, AUTHORIZED,
 -- CAPTURED, SETTLED, FAILED,
 -- CANCELED, REFUNDED

 payment_method_id VARCHAR(26) NOT NULL, -- FK to payment_methods (token)
 capture_mode VARCHAR(10) NOT NULL, -- 'auto' or 'manual'
 amount_authorized BIGINT DEFAULT 0,
 amount_captured BIGINT DEFAULT 0,
 amount_refunded BIGINT DEFAULT 0,
 description TEXT,
 statement_descriptor VARCHAR(22), -- Max 22 chars for bank statement
 metadata JSONB, -- Merchant-defined key-value pairs
 risk_score SMALLINT, -- 0-100
 psp_id VARCHAR(26), -- Which PSP processed this
 psp_reference VARCHAR(128), -- PSP's transaction ID
 failure_code VARCHAR(50), -- decline_code if failed
 failure_message TEXT,
 ip_address INET, -- Customer IP for fraud detection
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 authorized_at TIMESTAMPTZ,
 captured_at TIMESTAMPTZ,
 settled_at TIMESTAMPTZ,
 version INTEGER NOT NULL DEFAULT 1, -- Optimistic locking

 CONSTRAINT uq_idempotency UNIQUE (merchant_id, idempotency_key),
 CONSTRAINT chk_amount CHECK (amount > 0),
 CONSTRAINT chk_currency CHECK (currency ~ '^[A-Z]{3}$')
);

CREATE INDEX idx_payments_merchant_created ON payments (merchant_id, created_at DESC);
CREATE INDEX idx_payments_status ON payments (status) WHERE status IN ('PROCESSING', 'AUTHORIZED');
CREATE INDEX idx_payments_psp_reference ON payments (psp_reference);

```

### 11.6.2.2 ledger\_entries (Double-entry bookkeeping)

```
CREATE TABLE ledger_entries (
 id BIGSERIAL PRIMARY KEY,
 payment_id VARCHAR(26) NOT NULL, -- FK to payments
 entry_type VARCHAR(20) NOT NULL, -- AUTHORIZATION, CAPTURE, REFUND,
 -- SETTLEMENT, FEE, PAYOUT
 account_id VARCHAR(26) NOT NULL, -- FK to ledger_accounts
 direction CHAR(1) NOT NULL, -- 'D' (debit) or 'C' (credit)
 amount BIGINT NOT NULL, -- Always positive
 currency CHAR(3) NOT NULL,
 balance_after BIGINT NOT NULL, -- Running balance after this entry
 counterpart_id BIGINT, -- The matching entry (debit<→credit)
 description TEXT,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT chk_direction CHECK (direction IN ('D', 'C')),
 CONSTRAINT chk_amount CHECK (amount > 0)
);

CREATE INDEX idx_ledger_payment ON ledger_entries (payment_id);
CREATE INDEX idx_ledger_account ON ledger_entries (account_id, created_at DESC);
```

### 11.6.2.3 ledger\_accounts

```
CREATE TABLE ledger_accounts (
 id VARCHAR(26) PRIMARY KEY,
 account_type VARCHAR(30) NOT NULL, -- MERCHANT_RECEIVABLE, MERCHANT_PAYABLE,
 -- PLATFORM_REVENUE, PROCESSING_FEE,
 -- SETTLEMENT_HOLDING, PSP_CLEARING
 owner_id VARCHAR(26), -- merchant_id or NULL for system accounts
 currency CHAR(3) NOT NULL,
 balance BIGINT NOT NULL DEFAULT 0, -- Current balance in smallest unit
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT uq_account UNIQUE (account_type, owner_id, currency)
);
```

### 11.6.2.4 payment\_methods (Tokenized vault)

```
CREATE TABLE payment_methods (
 id VARCHAR(26) PRIMARY KEY, -- Token (e.g., pm_abc123)
 customer_id VARCHAR(26), -- Optional customer association
 type VARCHAR(20) NOT NULL, -- card, bank_account, wallet
 -- Card fields (encrypted)
 card_brand VARCHAR(20), -- visa, mastercard, amex
 card_last4 CHAR(4), -- Last 4 digits
 card_exp_month SMALLINT,
 card_exp_year SMALLINT,
 card_fingerprint VARCHAR(64), -- Hash for dedup across merchants
 card_token_vault VARCHAR(256), -- Reference to HSM-stored token
 -- Bank account fields (encrypted)
 bank_routing VARCHAR(64) ENCRYPTED, -- Column-level encryption
 bank_account_last4 CHAR(4),
 -- Metadata
 billing_name VARCHAR(128),
 billing_email VARCHAR(256),
 billing_country CHAR(2), -- ISO 3166-1 alpha-2
 is_default BOOLEAN DEFAULT FALSE,
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

 CONSTRAINT chk_type CHECK (type IN ('card', 'bank_account', 'wallet'))
);
```

### 11.6.2.5 merchants

```
CREATE TABLE merchants (
 id VARCHAR(26) PRIMARY KEY,
 business_name VARCHAR(256) NOT NULL,
 country CHAR(2) NOT NULL,
 default_currency CHAR(3) NOT NULL,
 settlement_schedule VARCHAR(20) DEFAULT 'T+2', -- T+1, T+2, weekly, monthly
 fee_rate_bps INTEGER DEFAULT 290, -- 2.90% = 290 basis points
 fee_fixed_cents INTEGER DEFAULT 30, -- $0.30
 payout_account_id VARCHAR(26), -- Bank account for payouts
 risk_level VARCHAR(10) DEFAULT 'normal', -- low, normal, high, restricted
 webhook_url TEXT,
 webhook_secret VARCHAR(64),
 api_key_hash VARCHAR(128) NOT NULL, -- Hashed API key
 status VARCHAR(20) DEFAULT 'active', -- active, suspended, closed
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```



### 11.6.2.6 idempotency\_keys

```
CREATE TABLE idempotency_keys (
 key VARCHAR(64) NOT NULL,
 merchant_id VARCHAR(26) NOT NULL,
 request_path VARCHAR(256) NOT NULL,
 request_hash VARCHAR(64) NOT NULL, -- Hash of request body
 response_code SMALLINT,
 response_body JSONB,
 status VARCHAR(20) NOT NULL, -- 'processing' or 'complete'
 created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
 expires_at TIMESTAMPTZ NOT NULL, -- created_at + 24h

 PRIMARY KEY (merchant_id, key)
);
```

### 11.6.3 6.3 Double-Entry Ledger: How Entries Balance

Every financial movement creates at least two ledger entries that sum to zero.

Example: Customer pays \$50.00 to Merchant

Entry 1 (Debit): PSP\_CLEARING account +\$50.00 (money coming in from card network)  
 Entry 2 (Credit): MERCHANT\_RECEIVABLE account +\$50.00 (merchant is owed this money)

When platform fee is taken (\$1.75 = 2.9% + \$0.30):

Entry 3 (Debit): MERCHANT\_RECEIVABLE account -\$1.75 (reduce what we owe merchant)  
 Entry 4 (Credit): PLATFORM\_REVENUE account +\$1.75 (platform earns the fee)

At settlement (T+2), payout to merchant:

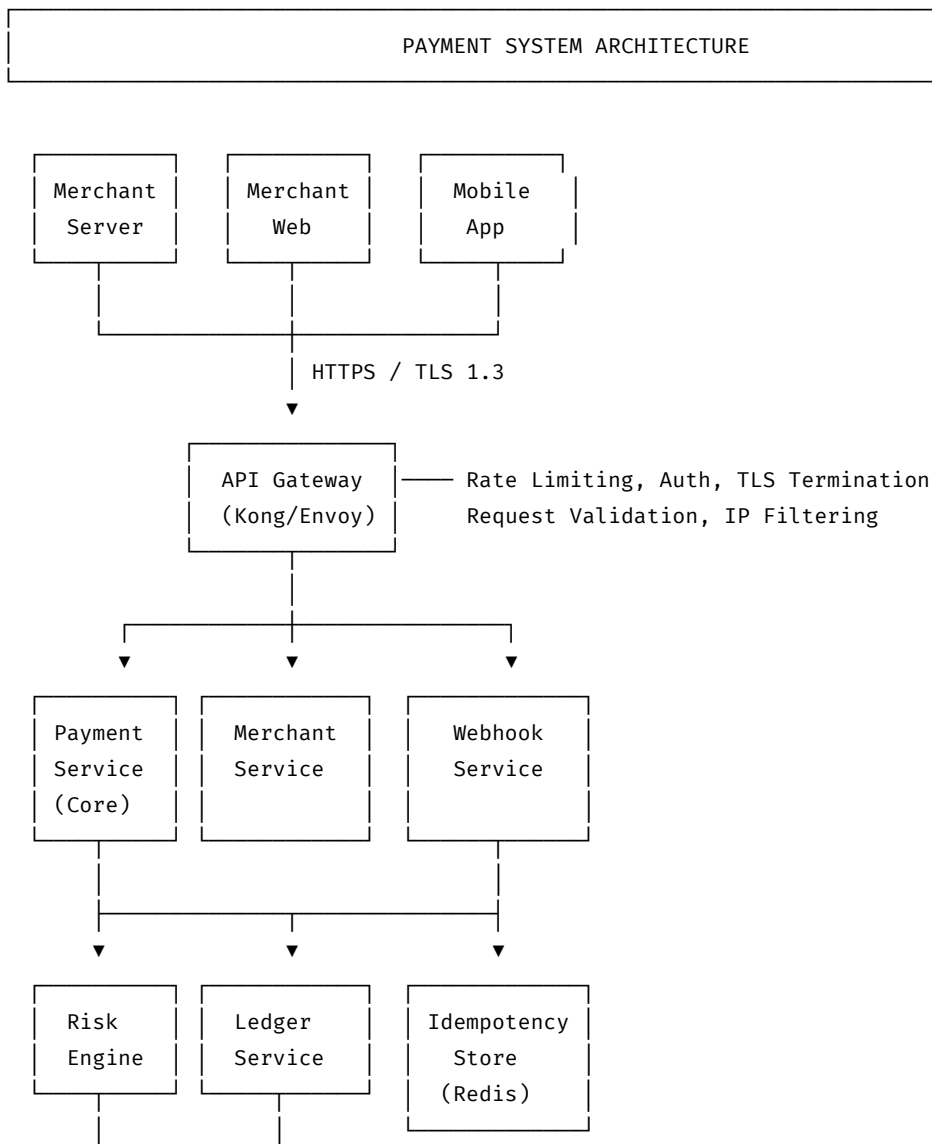
Entry 5 (Debit): MERCHANT\_RECEIVABLE account -\$48.25 (clear the receivable)  
 Entry 6 (Credit): MERCHANT\_PAYABLE account +\$48.25 (payout initiated)  
 Entry 7 (Debit): MERCHANT\_PAYABLE account -\$48.25 (payout sent)  
 Entry 8 (Credit): SETTLEMENT\_HOLDING account +\$48.25 (bank transfer in progress)

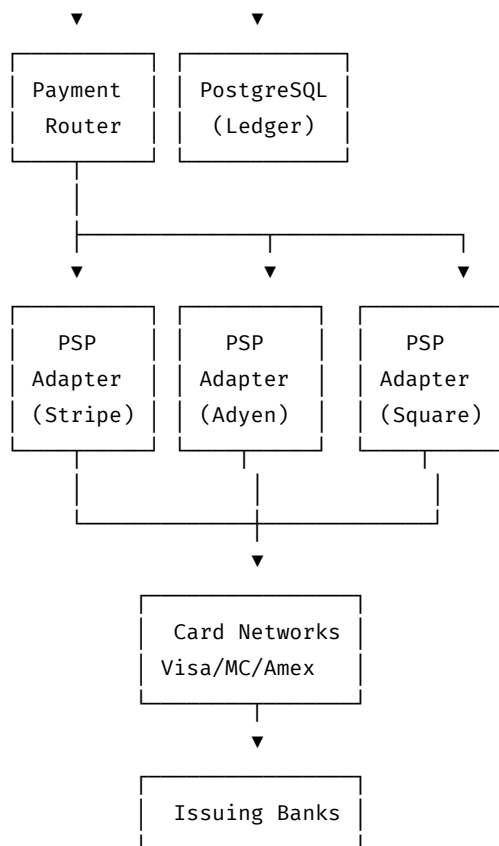
Invariant: SUM(debits) = SUM(credits) at all times

Every account's balance = SUM(credits) - SUM(debits) for that account

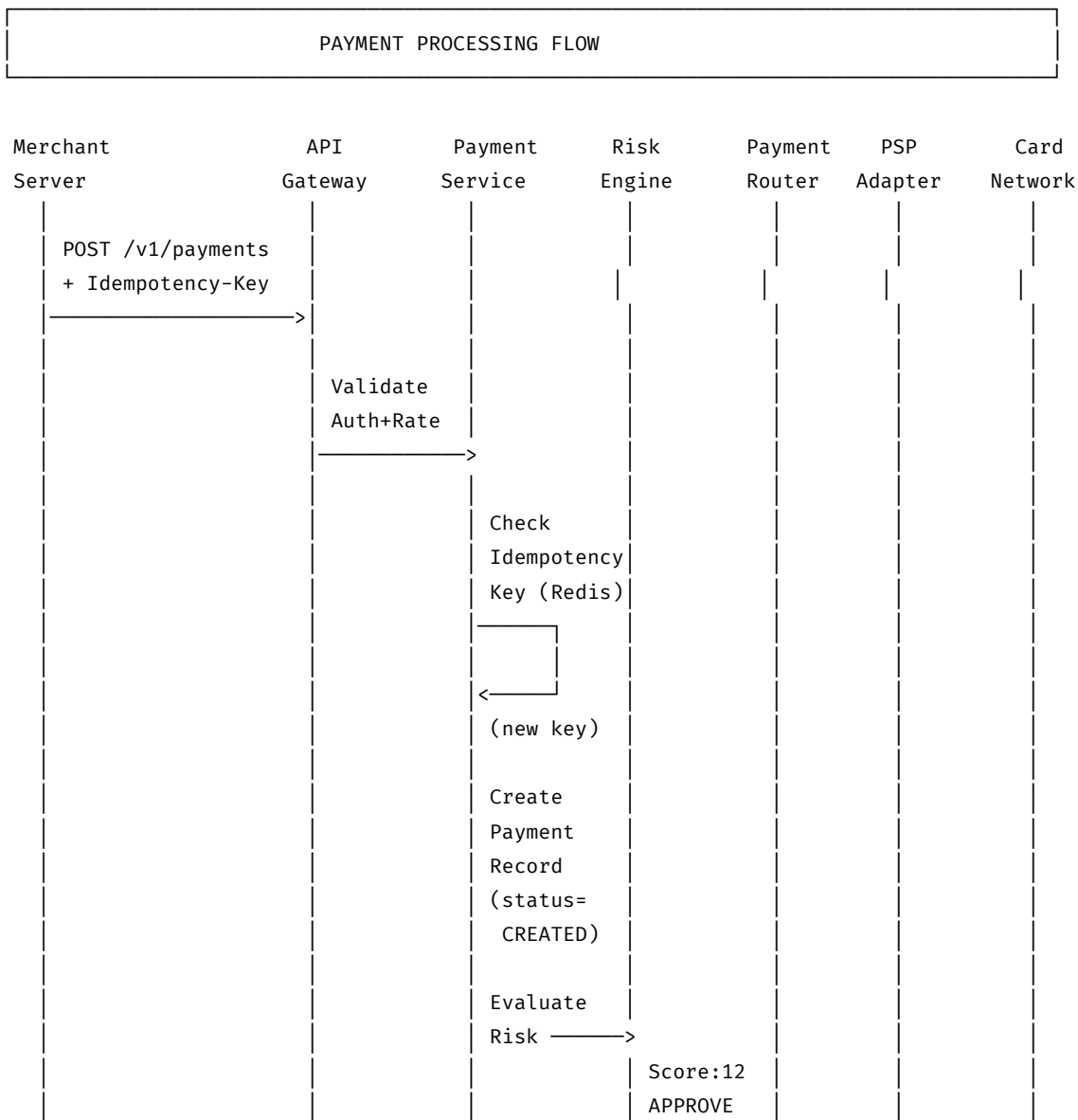
## 11.7 7. High-Level Architecture

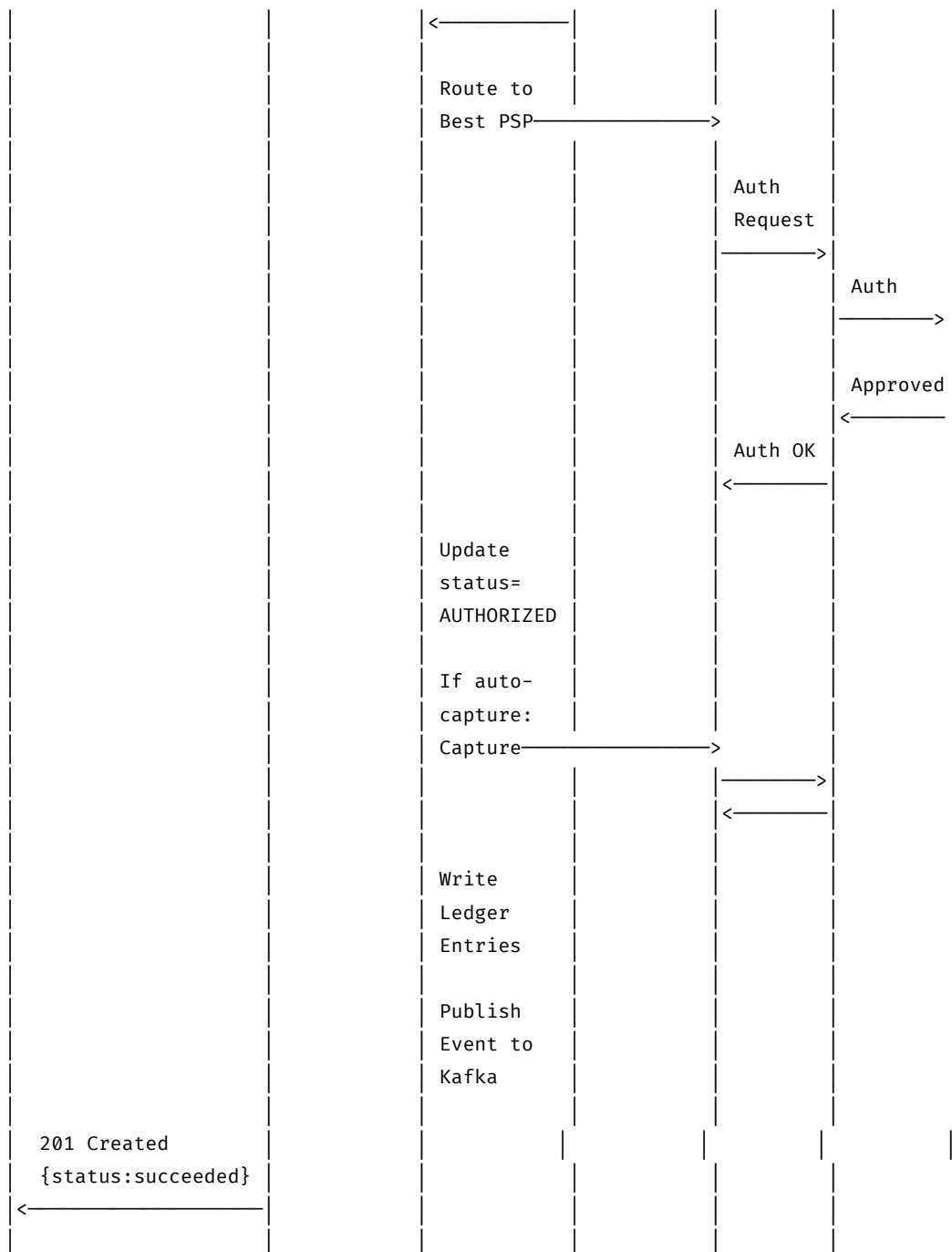
### 11.7.1 7.1 System Overview





11.7.2 7.2 Complete Payment Flow





### 11.7.3 7.3 Component Breakdown

#### 11.7.3.1 Payment Service (Core Orchestrator)

The central coordinator that manages the payment lifecycle. It is stateless and horizontally scalable. Each instance connects to PostgreSQL via PgBouncer connection pooling.

**Responsibilities:** - Accept and validate payment requests. - Check idempotency keys (Redis lookup, then DB fallback). - Orchestrate the authorization-capture flow. - Write payment records with optimistic concurrency (version column). - Publish events to Kafka for downstream consumers.

### 11.7.3.2 Risk Engine

Evaluates every payment before authorization. Produces a risk score (0-100) and a recommendation (APPROVE, REVIEW, DECLINE).

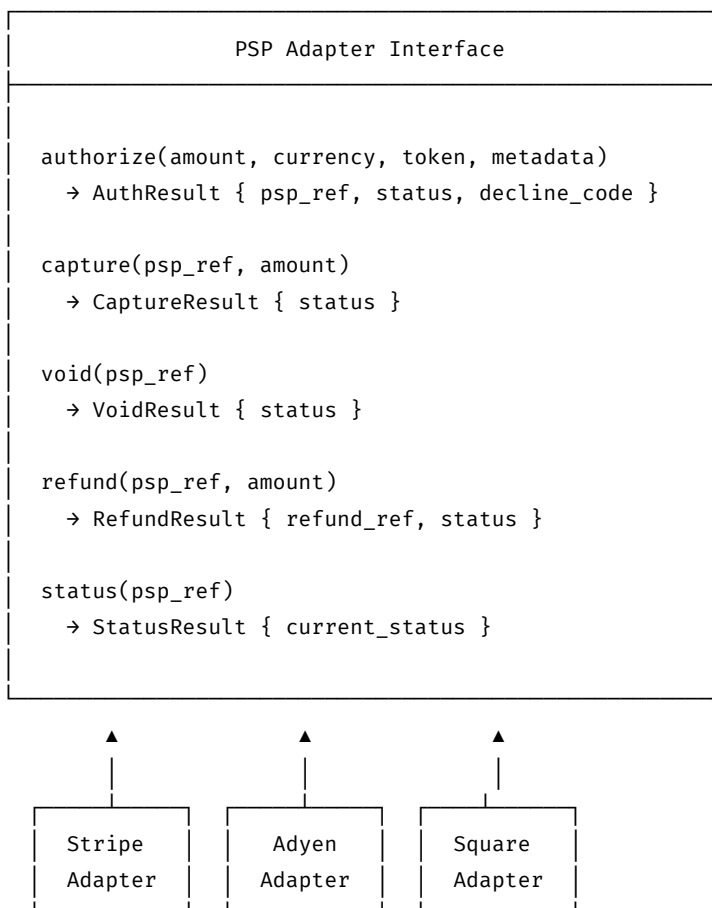
**Signals evaluated:** - Velocity checks: too many transactions from same card, IP, or device in short window. - Geolocation mismatch: card's issuing country vs. customer IP country. - BIN analysis: high-risk BIN ranges, prepaid cards. - Amount anomalies: unusually large transaction for this merchant category. - Device fingerprint: known fraudulent device signatures. - ML model: trained on historical chargeback data.

### 11.7.3.3 Payment Router

Selects the optimal PSP (Payment Service Provider) based on: - Card network and issuing country (local acquiring preferred). - PSP success rates by card type (tracked in real time). - PSP latency (route away from degraded providers). - Cost optimization (interchange++ vs. blended pricing). - Merchant-specific PSP preferences. - PSP health status (circuit breaker state).

### 11.7.3.4 PSP Adapter Layer

Abstracts the interface to each external PSP behind a uniform internal API.



### 11.7.3.5 Ledger Service

Maintains the double-entry bookkeeping system. Every financial movement produces balanced debit and credit entries. The ledger is append-only – entries are never modified or deleted. Corrections are made by posting reversal entries.

### 11.7.3.6 Webhook Service

Delivers payment events to merchant-configured HTTP endpoints.

**Delivery guarantees:** - At-least-once delivery (merchants must be idempotent). - Exponential back-off: retry at 1m, 5m, 30m, 2h, 8h, 24h (then dead-letter). - Signed payloads (HMAC-SHA256) for authenticity verification. - Event ordering per payment (not globally).

### 11.7.3.7 Reconciliation Service

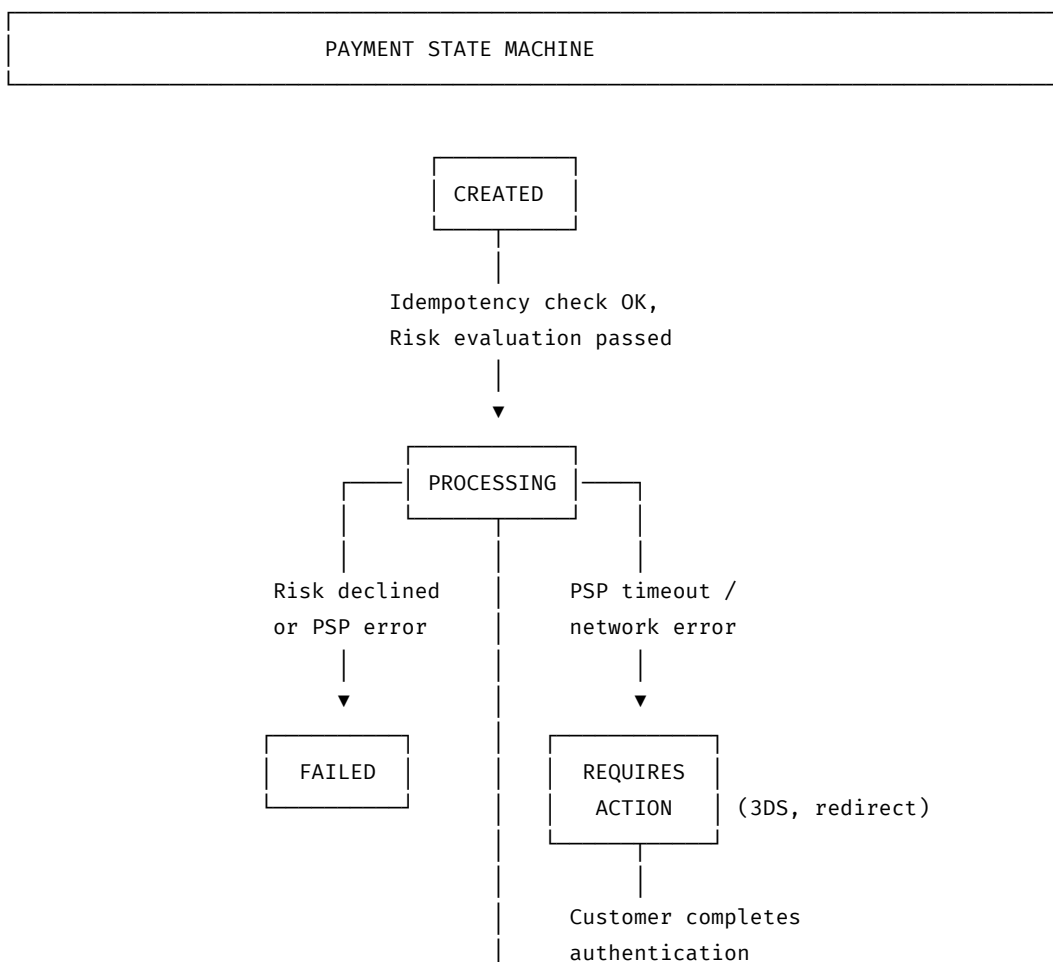
Runs daily batch jobs that compare internal ledger entries against PSP settlement reports and bank statements. Flags discrepancies for manual review.

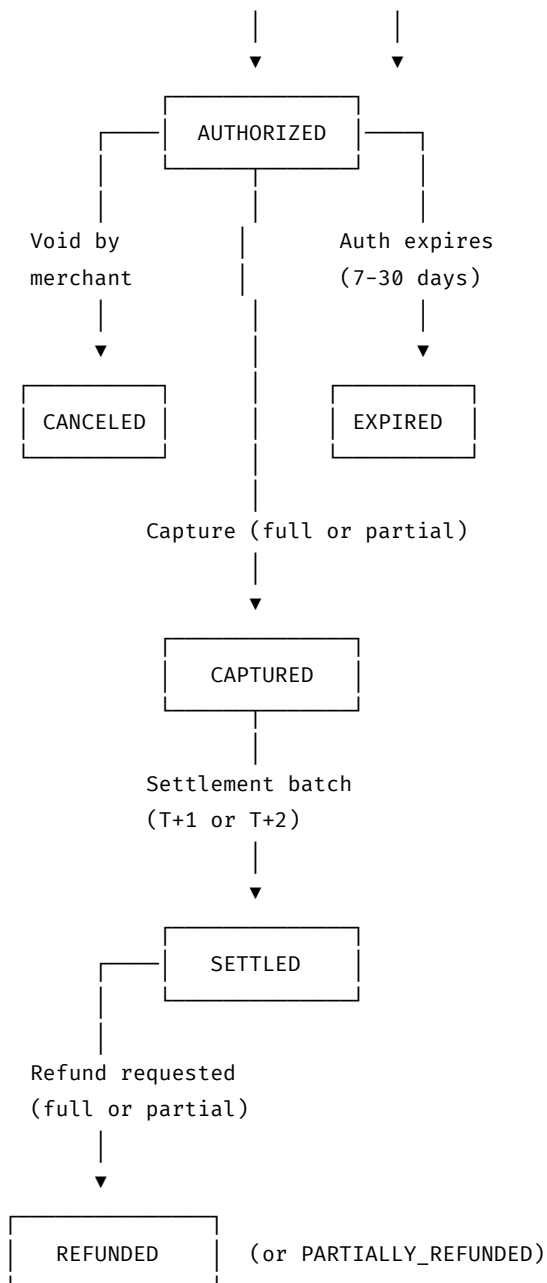
---

## 11.8 8. Deep Dive: Core Components

### 11.8.1 8.1 Payment Processing Flow and State Machine

#### 11.8.1.1 Payment States





### 11.8.1.2 Auth-Only vs. Auth+Capture Flow

**AUTH-ONLY FLOW** (e.g., hotel reservation):

1. Merchant creates payment with `capture=false`
2. Payment is **AUTHORIZED** (hold on customer's card)
3. Days later, merchant calls `POST /v1/payments/{id}/capture`
4. Payment moves to **CAPTURED**
5. Settled in next batch

**AUTH+CAPTURE FLOW** (e.g., e-commerce checkout):

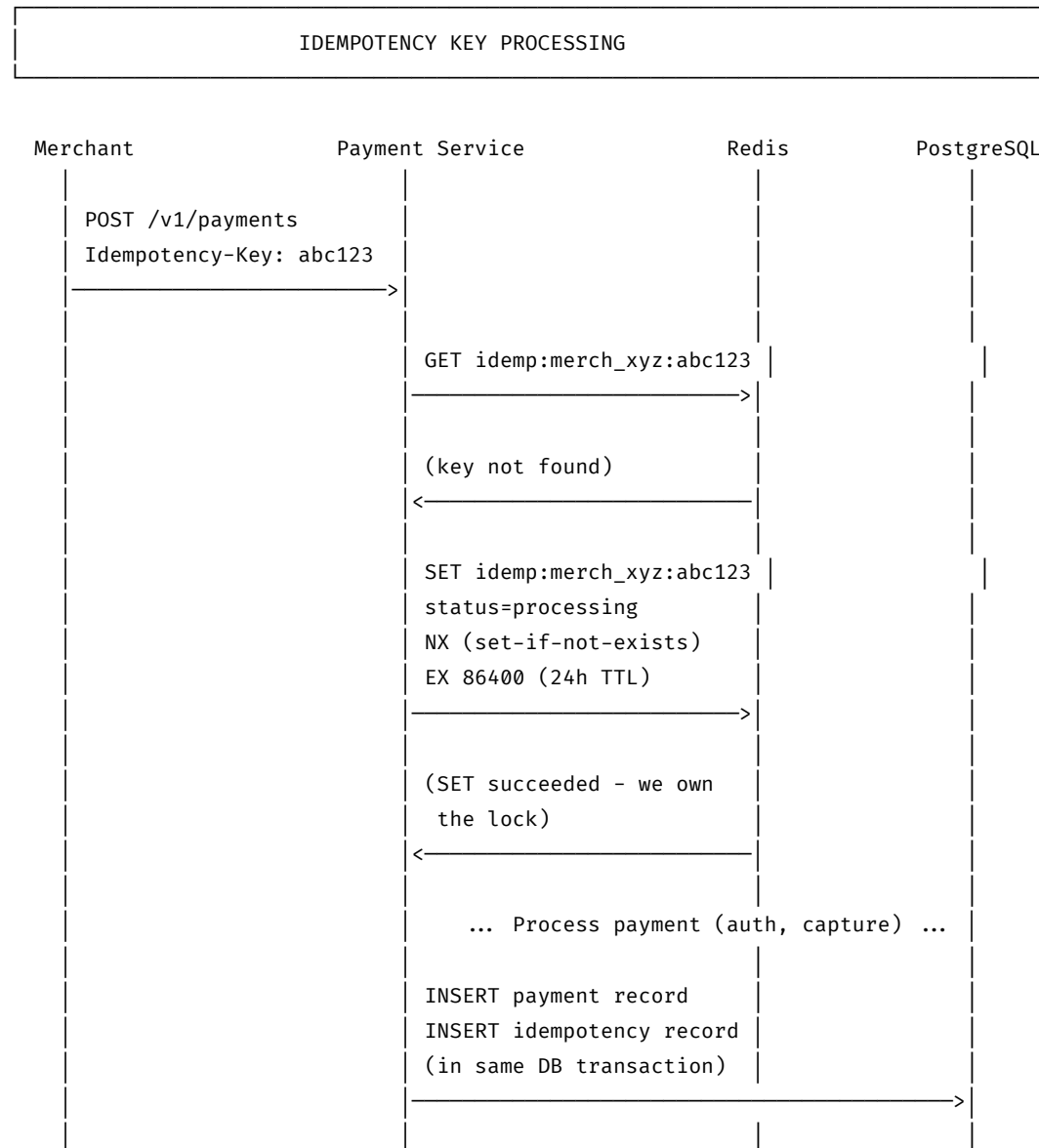
1. Merchant creates payment with `capture=true`
2. Authorization and capture happen atomically
3. Payment goes directly from **PROCESSING** -> **CAPTURED**
4. Settled in next batch

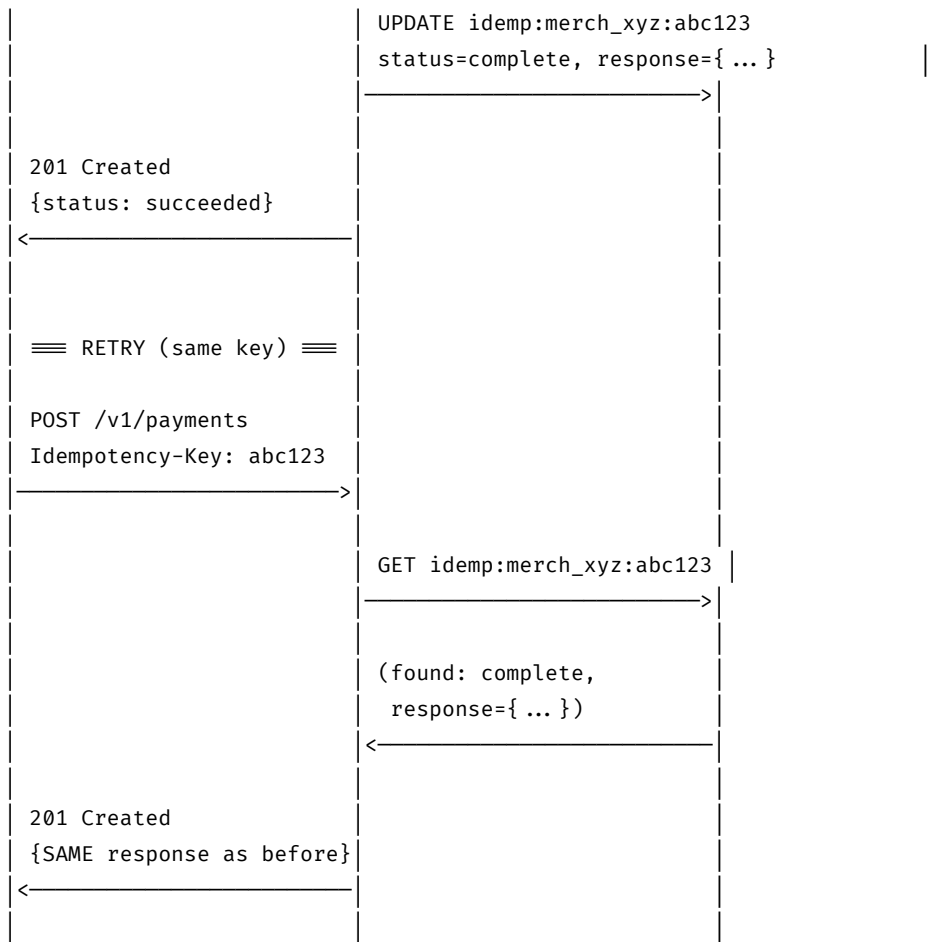


## 11.8.2 8.2 Idempotency and Exactly-Once Processing

This is the most critical design concern in a payment system. A customer must never be charged twice for the same transaction, even if the merchant retries due to a timeout or network error.

### 11.8.2.1 Idempotency Key Flow





### 11.8.2.2 Handling Edge Cases

CASE 1: Retry while original is still processing

- Redis returns status=processing
- Return 409 Conflict with Retry-After header
- Client retries after delay

CASE 2: Server crashes after PSP authorization but before DB write

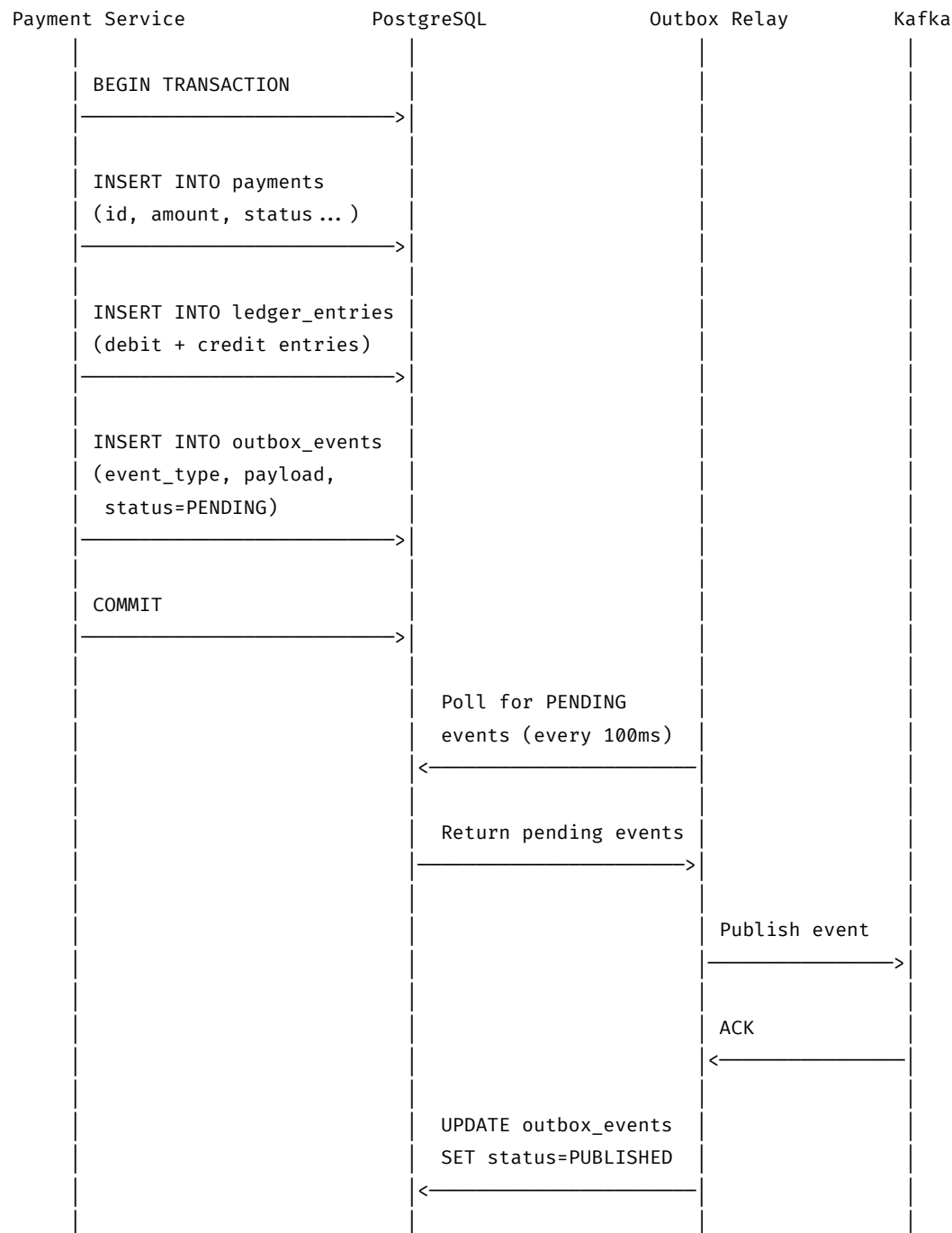
- Idempotency key left in "processing" state in Redis
- Recovery worker (runs every 30s) finds stale "processing" keys (>60s old)
- Queries PSP for transaction status using psp\_reference
- If PSP says "authorized": record the authorization, mark idempotency complete
- If PSP says "not found": expire the idempotency key, allow retry

CASE 3: Database write succeeds but Redis update fails

- Next retry reads Redis (still "processing") -> returns 409
- Recovery worker eventually detects the stale key
- Checks DB, finds completed payment, updates Redis
- OR: Client retries after 409 TTL, DB unique constraint prevents duplicate

### 11.8.2.3 Outbox Pattern for Reliable Event Publishing

#### TRANSACTIONAL OUTBOX PATTERN



Key insight: The payment record and the event are written in the SAME database transaction. If the transaction commits, the event is guaranteed to be published (eventually). If the transaction rolls back, no event is published. This avoids the dual-write problem where the DB write succeeds but the Kafka write fails (or vice versa).

### 11.8.3 8.3 Double-Entry Ledger

The ledger is the financial backbone of the payment system. Every monetary movement must create balanced entries following the fundamental equation:

**Assets = Liabilities + Equity** (always balanced)

11.8.3.1 Ledger Entry Flow for a \$50.00 Payment (2.9% + \$0.30 fee)

|                                |  |  |  |  |
|--------------------------------|--|--|--|--|
| DOUBLE-ENTRY LEDGER FLOW       |  |  |  |  |
| Payment: \$50.00   Fee: \$1.75 |  |  |  |  |

Step 1: AUTHORIZATION (holds are off-ledger, tracked in payments table)

- No ledger entries yet
- Payment status: AUTHORIZED
- Card issuer places \$50 hold on customer's card

Step 2: CAPTURE (money movement begins)

| Entry | Account             | Direction | Amount  | Description        |
|-------|---------------------|-----------|---------|--------------------|
| 1     | PSP_CLEARING        | DEBIT     | \$50.00 | Funds from Stripe  |
| 2     | MERCHANT_RECEIVABLE | CREDIT    | \$50.00 | Owed to merchant   |
|       |                     | Balance:  | \$0.00  | (entries sum to 0) |

Step 3: FEE DEDUCTION

| Entry | Account             | Direction | Amount | Description        |
|-------|---------------------|-----------|--------|--------------------|
| 3     | MERCHANT_RECEIVABLE | DEBIT     | \$1.75 | Platform fee       |
| 4     | PLATFORM_REVENUE    | CREDIT    | \$1.75 | Fee earned         |
|       |                     | Balance:  | \$0.00 | (entries sum to 0) |

Step 4: SETTLEMENT (payout to merchant)

| Entry | Account             | Direction | Amount  | Description        |
|-------|---------------------|-----------|---------|--------------------|
| 5     | MERCHANT_RECEIVABLE | DEBIT     | \$48.25 | Clear receivable   |
| 6     | BANK_SETTLEMENT     | CREDIT    | \$48.25 | Wire to merchant   |
|       |                     | Balance:  | \$0.00  | (entries sum to 0) |

Account Balances After Full Cycle:

| Account             | Balance  |                                             |
|---------------------|----------|---------------------------------------------|
| PSP_CLEARING        | -\$50.00 | (liability - we owe PSP this on net settle) |
| MERCHANT_RECEIVABLE | \$0.00   | (fully settled)                             |
| PLATFORM_REVENUE    | +\$1.75  | (our earnings)                              |
| BANK_SETTLEMENT     | +\$48.25 | (outgoing wire)                             |
| TOTAL               | \$0.00   | (always balanced)                           |

### 11.8.3.2 Refund Ledger Entries

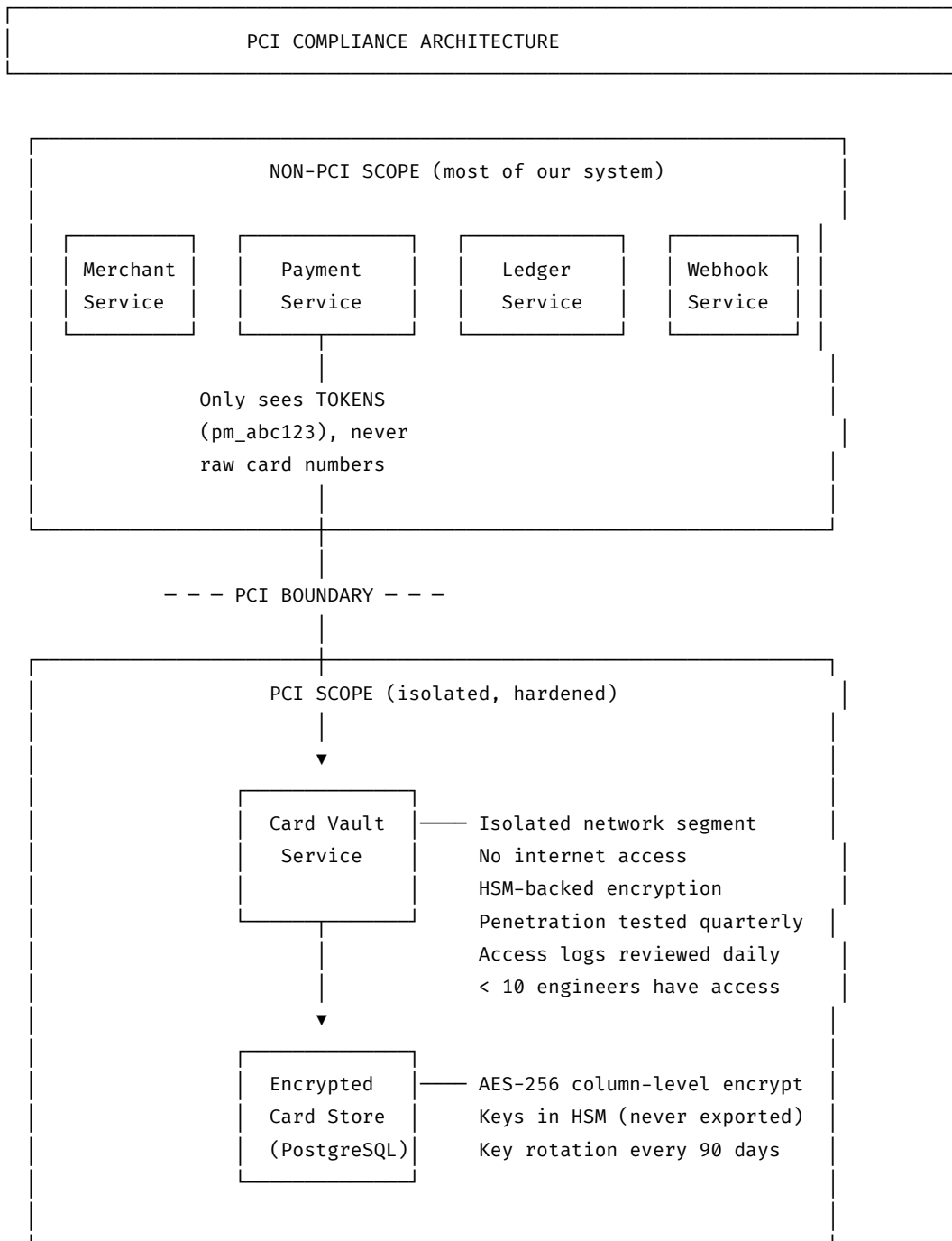
REFUND: Full refund of \$50.00 payment

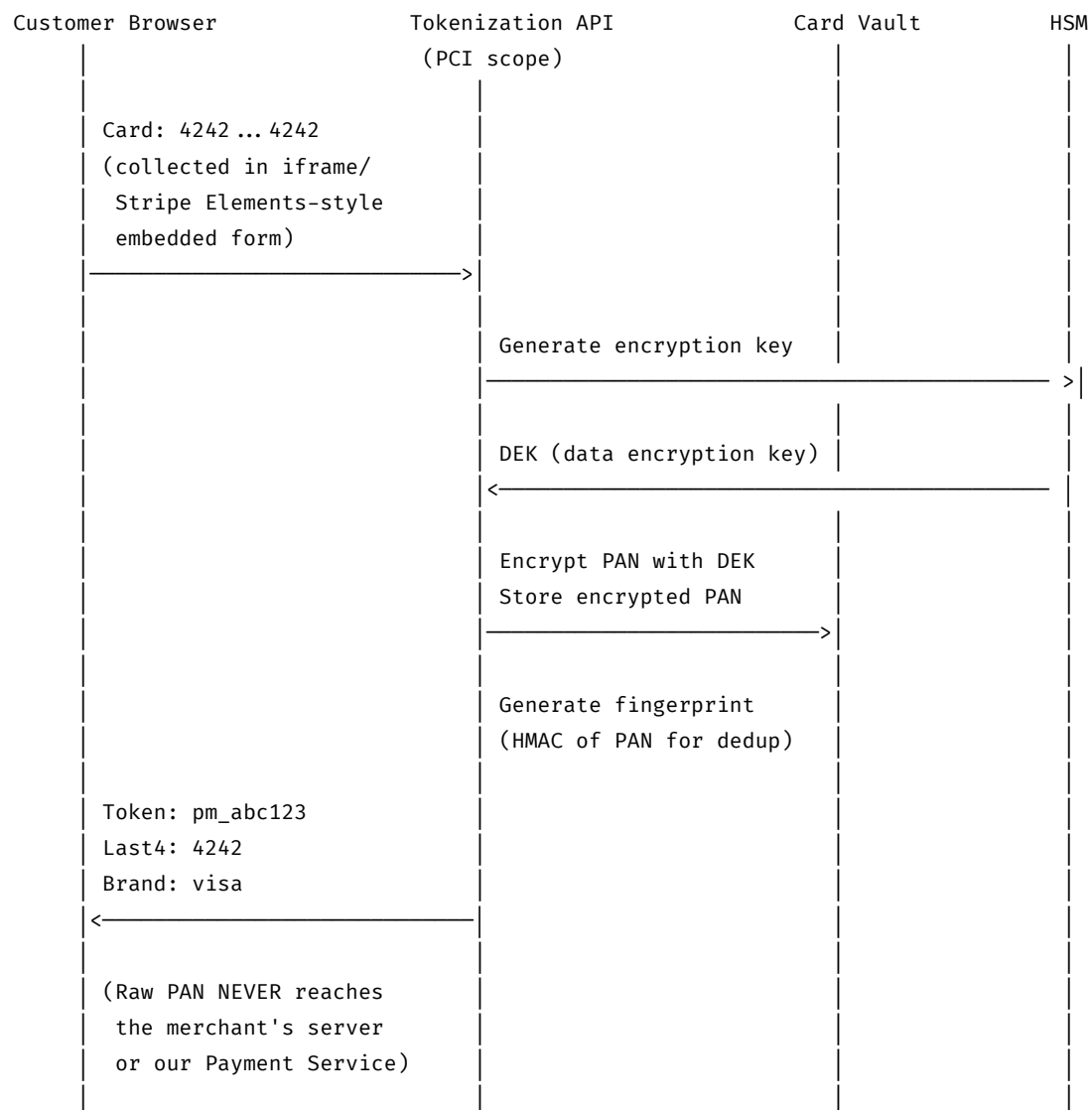
| Entry | Account             | Direction | Amount  | Description                          |
|-------|---------------------|-----------|---------|--------------------------------------|
| 1     | MERCHANT_RECEIVABLE | DEBIT     | \$48.25 | Charge back to merchant              |
| 2     | REFUND_CLEARING     | CREDIT    | \$48.25 | Refund to customer                   |
| 3     | PLATFORM_REVENUE    | DEBIT     | \$1.75  | Return our fee                       |
| 4     | REFUND_CLEARING     | CREDIT    | \$1.75  | (or keep fee per merchant agreement) |
|       |                     | Balance:  | \$0.00  | (entries sum to 0)                   |

## 11.8.4 8.4 Payment Security and PCI Compliance

### 11.8.4.1 PCI DSS Compliance Architecture

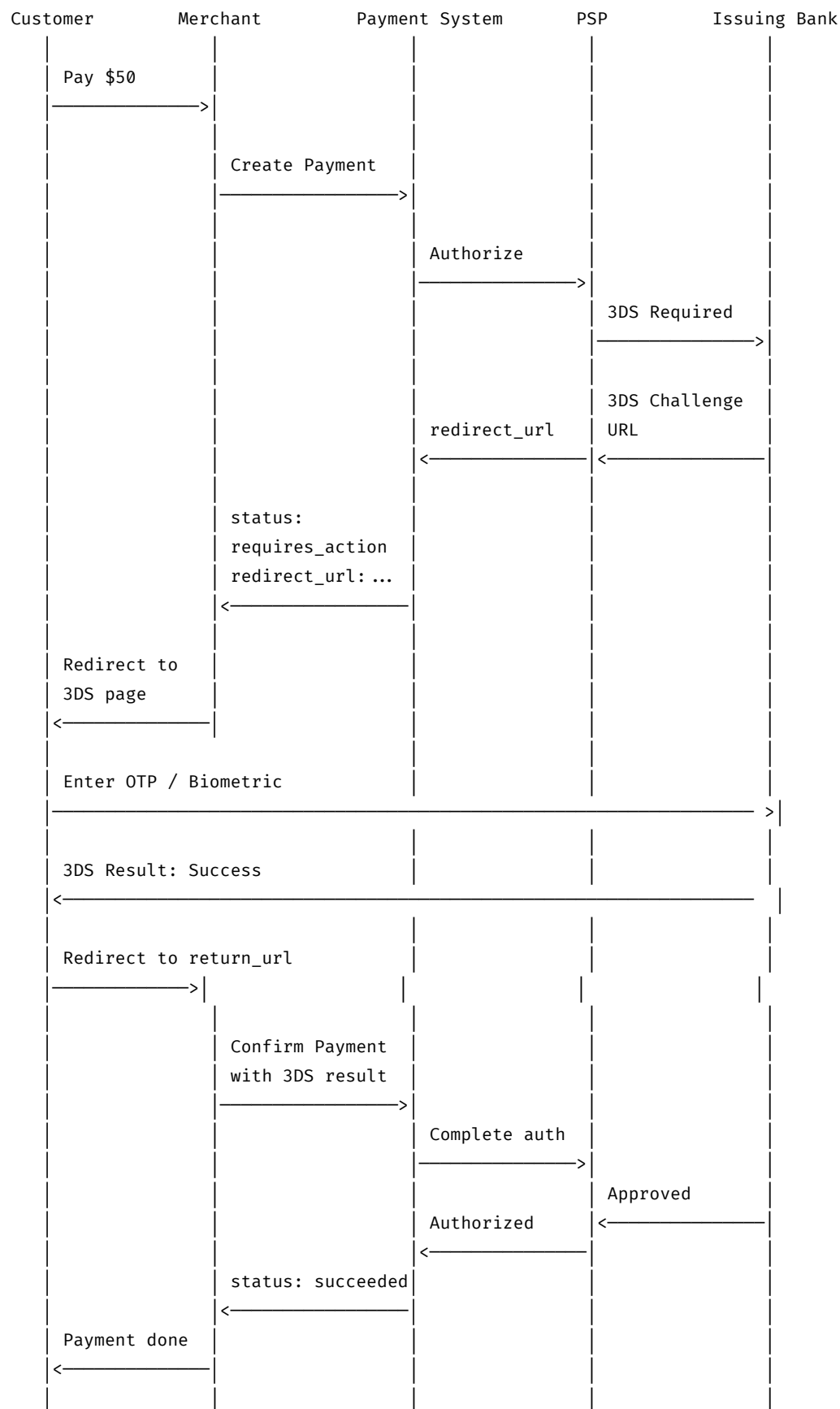
PCI DSS (Payment Card Industry Data Security Standard) Level 1 requires strict controls over how cardholder data is stored, processed, and transmitted. The key principle is to **minimize the scope** of PCI compliance by keeping raw card numbers (PANs) out of the main application.



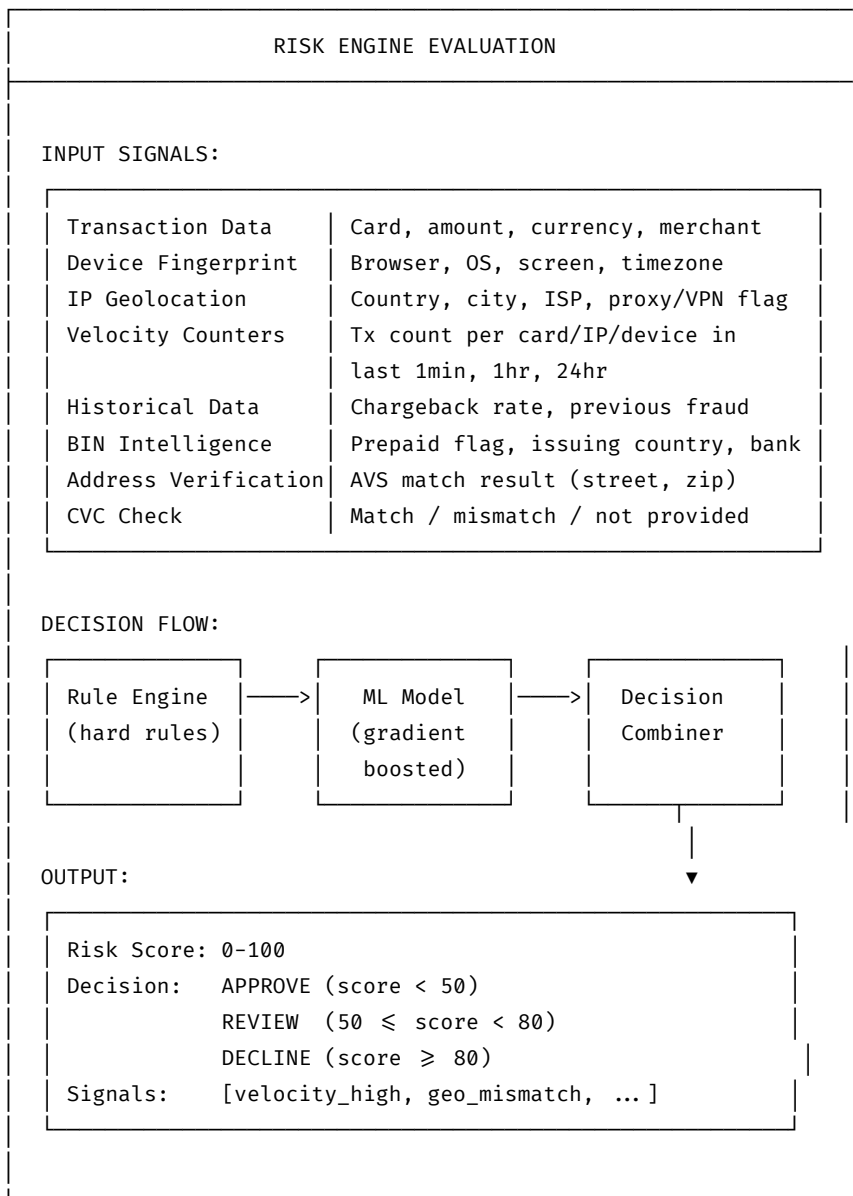
**11.8.4.2 Tokenization Flow****11.8.4.3 3D Secure (3DS) Authentication Flow**

3DS adds an extra authentication step where the card issuer verifies the cardholder (e.g., via SMS OTP or biometric). Required by PSD2 in Europe (Strong Customer Authentication).





#### 11.8.4.4 Fraud Detection Signals



## 11.9 9. Data Partitioning and Sharding

### 11.9.1 Sharding Strategy

As the system grows beyond what a single PostgreSQL instance can handle (~10K TPS for writes), we need a sharding strategy.

## SHARDING STRATEGY

## Payment Data

Shard Key: `merchant_id`

Rationale: Most queries are merchant-scoped  
(list my payments, my revenue, etc.)

| Shard 0   | Shard 1   | Shard 2   | Shard 3   |
|-----------|-----------|-----------|-----------|
| merchants | merchants | merchants | merchants |
| A-F       | G-L       | M-R       | S-Z       |
| payments  | payments  | payments  | payments  |
| refunds   | refunds   | refunds   | refunds   |

Shard routing: `hash(merchant_id) % num_shards`

Consistent hashing for minimal re-shuffling on scale-out

## Ledger Data

Shard Key: `account_id`

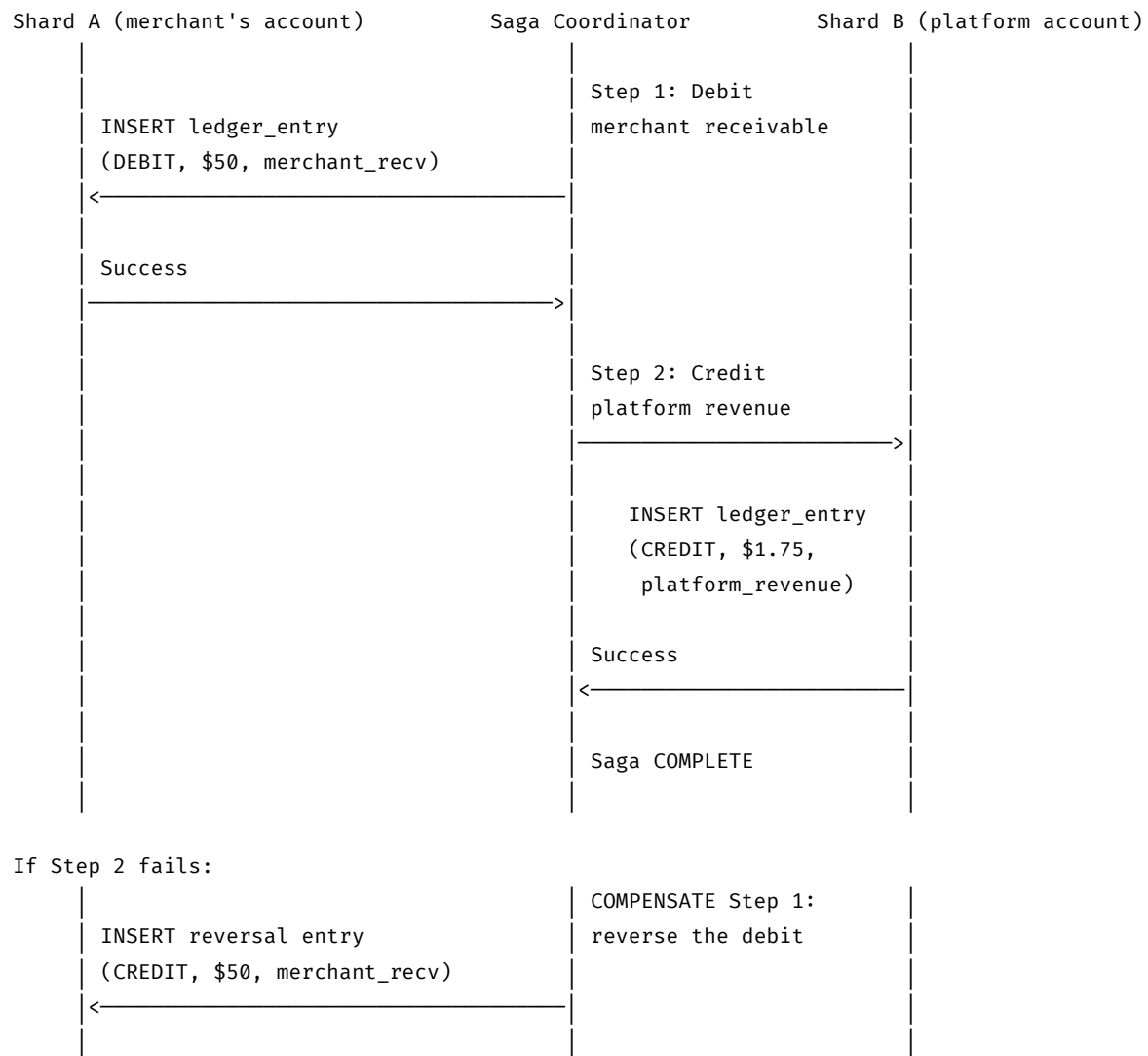
Rationale: Ledger queries are account-scoped  
(balance of account X, entries for account X)

Special handling:

- Cross-shard transactions (e.g., fee entry touches `MERCHANT_RECEIVABLE` on shard 2 and `PLATFORM_REVENUE` on shard 0) handled via saga pattern
- Platform accounts (`PLATFORM_REVENUE`, etc.) are on a dedicated "system shard" to avoid cross-shard writes

### 11.9.2 Cross-Shard Transactions via Saga

Scenario: Payment capture creates entries across two shards



## 11.10 10. Caching Strategy

### CACHING STRATEGY

CACHE (Redis Cluster - 3 nodes, 32GB each)

#### LAYER 1: Idempotency Keys

Key: idemp:{merchant\_id}:{idempotency\_key}

Value: {status, response\_code, response\_body}

TTL: 24 hours

Pattern: Write-through (written during payment processing)

Volume: ~1M keys/day, ~6M active at any time

#### LAYER 2: Merchant Configuration

Key: merchant:{merchant\_id}:config

Value: {fee\_rate, currency, psp\_preference, risk\_settings, ...}

TTL: 5 minutes (invalidated on config change via pub/sub)

Pattern: Cache-aside with event-driven invalidation

Volume: ~100K merchants, ~100KB each = ~10GB

#### LAYER 3: Exchange Rates

Key: fx:{from\_currency}:{to\_currency}

Value: {rate, fetched\_at}

TTL: 60 seconds (very short - rates fluctuate)

Pattern: Write-through, updated by FX rate polling service

Volume: ~18K pairs (135 currencies squared), negligible size

LAYER 4: PSP Health & Routing

Key:      psp:{psp\_id}:health

Value:    {success\_rate\_1m, avg\_latency\_1m, circuit\_state}

TTL:      10 seconds (near-real-time)

Pattern: Write-through, updated by health checker

LAYER 5: Velocity Counters (Fraud Detection)

Key:      velocity:{dimension}:{value}:{window}

          e.g., velocity:card\_fp:fp\_abc:1h

Value:    Counter (using Redis INCR)

TTL:      Matches window (1min, 1hr, 24hr)

Pattern: Write-through on every transaction

CRITICAL RULE: Payment status is NEVER cached.

Payment state must always be read from PostgreSQL to ensure consistency.

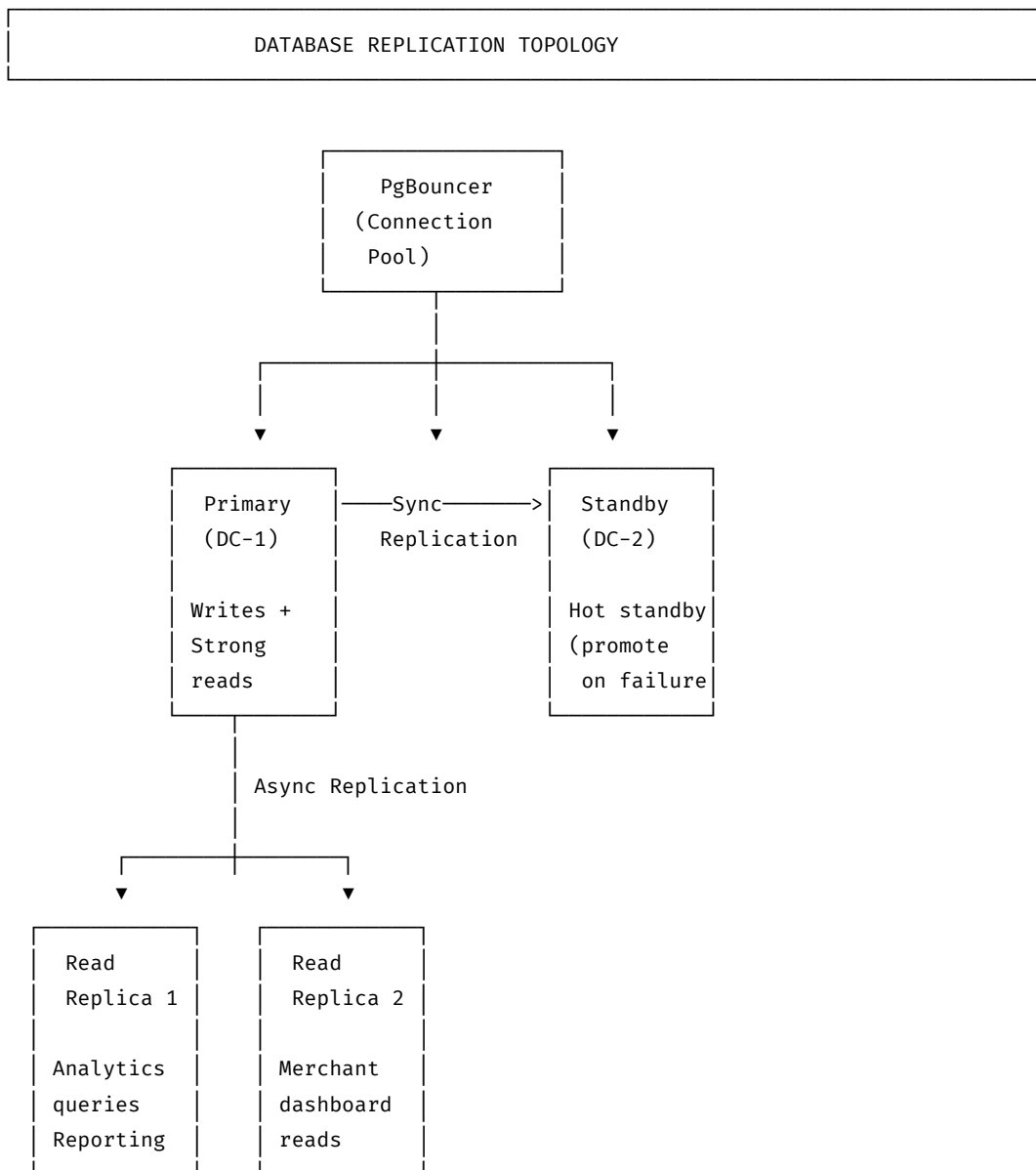
Serving stale payment status could cause double charges or missed refunds.

## 11.11 11. Replication and Consistency

### 11.11.1 Consistency Requirements by Data Type

| Data Type                | Consistency Model                             | Rationale                                       |
|--------------------------|-----------------------------------------------|-------------------------------------------------|
| Payment records          | Strong (synchronous replication)              | Cannot risk inconsistent payment state          |
| Ledger entries           | Strong (synchronous replication)              | Financial records must be perfectly consistent  |
| Idempotency keys         | Strong (Redis with WAIT + DB backup)          | Duplicate detection is critical for correctness |
| Merchant configuration   | Eventual (async replication, cache)           | Config changes can propagate in seconds         |
| Webhooks / notifications | Eventual (Kafka, at-least-once)               | At-least-once delivery is sufficient            |
| Analytics / reporting    | Eventual (async replication to read replicas) | Minutes of delay is acceptable for dashboards   |
| Audit logs               | Strong (synchronous append-only writes)       | Regulatory requirement for tamper-evidence      |

### 11.11.2 PostgreSQL Replication Topology



Failover: Automatic via Patroni / pg\_auto\_failover

RPO: 0 (synchronous replication to standby)

RTO: < 30 seconds (automatic failover with health checks)

### 11.11.3 Distributed Transaction: Saga Pattern

For operations that span multiple services (e.g., payment + ledger + webhook), we use the saga pattern instead of 2PC (two-phase commit).

**Why saga over 2PC:** - 2PC holds locks across services, creating coupling and latency. - 2PC has a blocking failure mode (coordinator crash leaves participants in doubt). - Saga allows each service to commit independently and compensate on failure. - Better suited for long-running transactions (e.g., payment + settlement).



PAYMENT SAGA (Happy Path)

## Step 1: Create Payment Record

- |— Service: Payment Service
- |— Action: INSERT into payments (status=CREATED)
- |— Compensate: UPDATE payments SET status=CANCELED

## Step 2: Risk Assessment

- |— Service: Risk Engine
- |— Action: Evaluate and score
- |— Compensate: No-op (stateless evaluation)

## Step 3: Authorize with PSP

- |— Service: PSP Adapter
- |— Action: Send authorization request to card network
- |— Compensate: Send void/reversal to PSP

## Step 4: Record Authorization

- |— Service: Payment Service
- |— Action: UPDATE payments SET status=AUTHORIZED
- |— Compensate: UPDATE payments SET status=AUTH\_REVERSED

## Step 5: Capture (if auto-capture)

- |— Service: PSP Adapter
- |— Action: Send capture request
- |— Compensate: Send refund to PSP

## Step 6: Write Ledger Entries

- |— Service: Ledger Service
- |— Action: INSERT balanced debit+credit entries
- |— Compensate: INSERT reversal entries

## Step 7: Publish Events

- |— Service: Event Bus (Kafka)
- |— Action: Publish payment.succeeded event
- |— Compensate: Publish payment.reversed event

If ANY step fails, compensating transactions run in REVERSE ORDER.

---

## 11.12 12. Fault Tolerance and Failure Handling

### 11.12.1 Payment Timeout Handling

#### TIMEOUT HANDLING DECISION TREE

Payment Service sends auth request to PSP

— PSP responds within 30s timeout? — YES — Process response normally

NO (timeout)



DO NOT BLINDLY RETRY (could cause double charge!)



Query PSP for transaction status

(using our idempotency/reference key)

— PSP says "authorized" — Record the authorization, proceed

— PSP says "declined" — Record the decline, mark FAILED

— PSP says "not found" — Safe to retry the authorization

— PSP also times out — Mark payment as UNKNOWN

Alert operations team

Reconciliation will catch it later

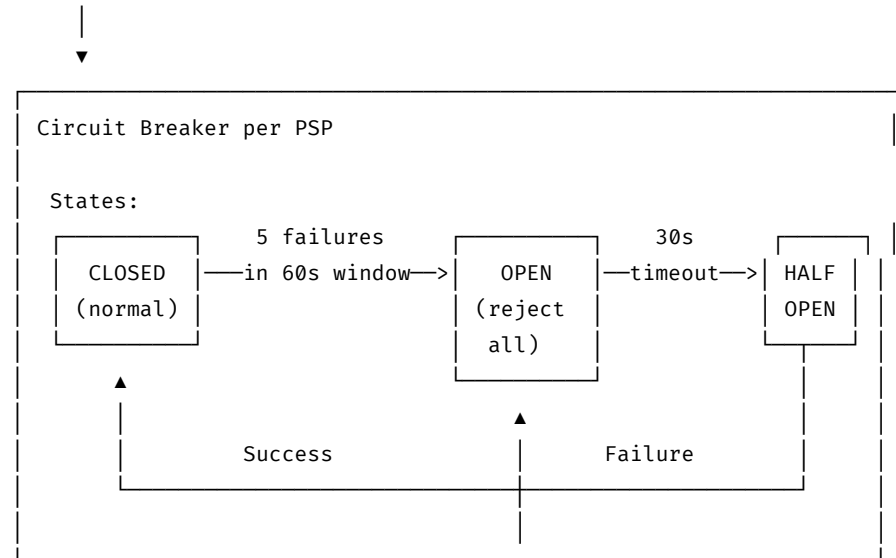
Return status "processing" to merchant

CRITICAL: The combination of idempotency key + status check ensures we NEVER send a duplicate authorization to the card network.

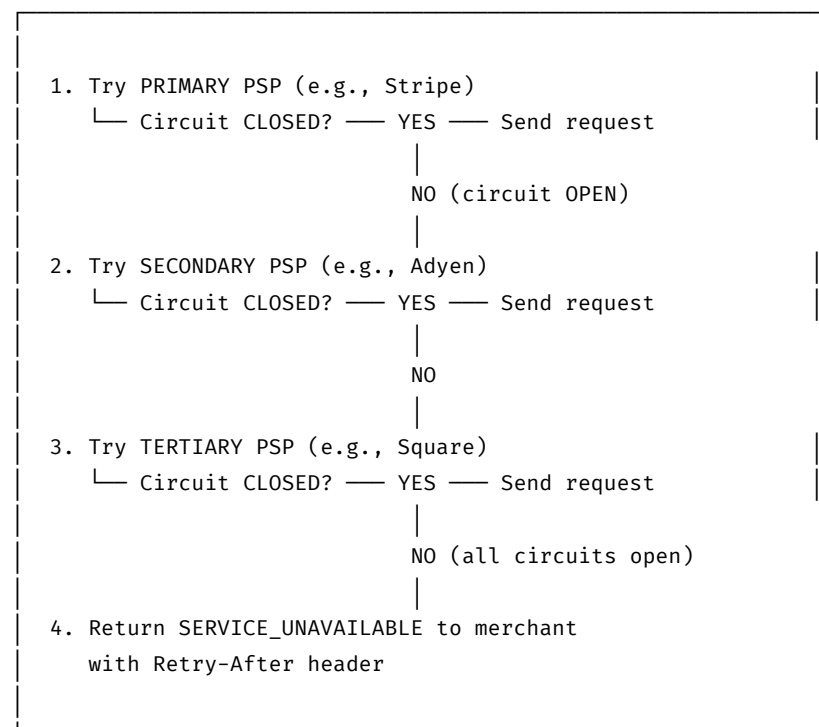
### 11.12.2 PSP Failover

#### PSP FAILOVER WITH CIRCUIT BREAKER

Payment Router



Failover Logic:

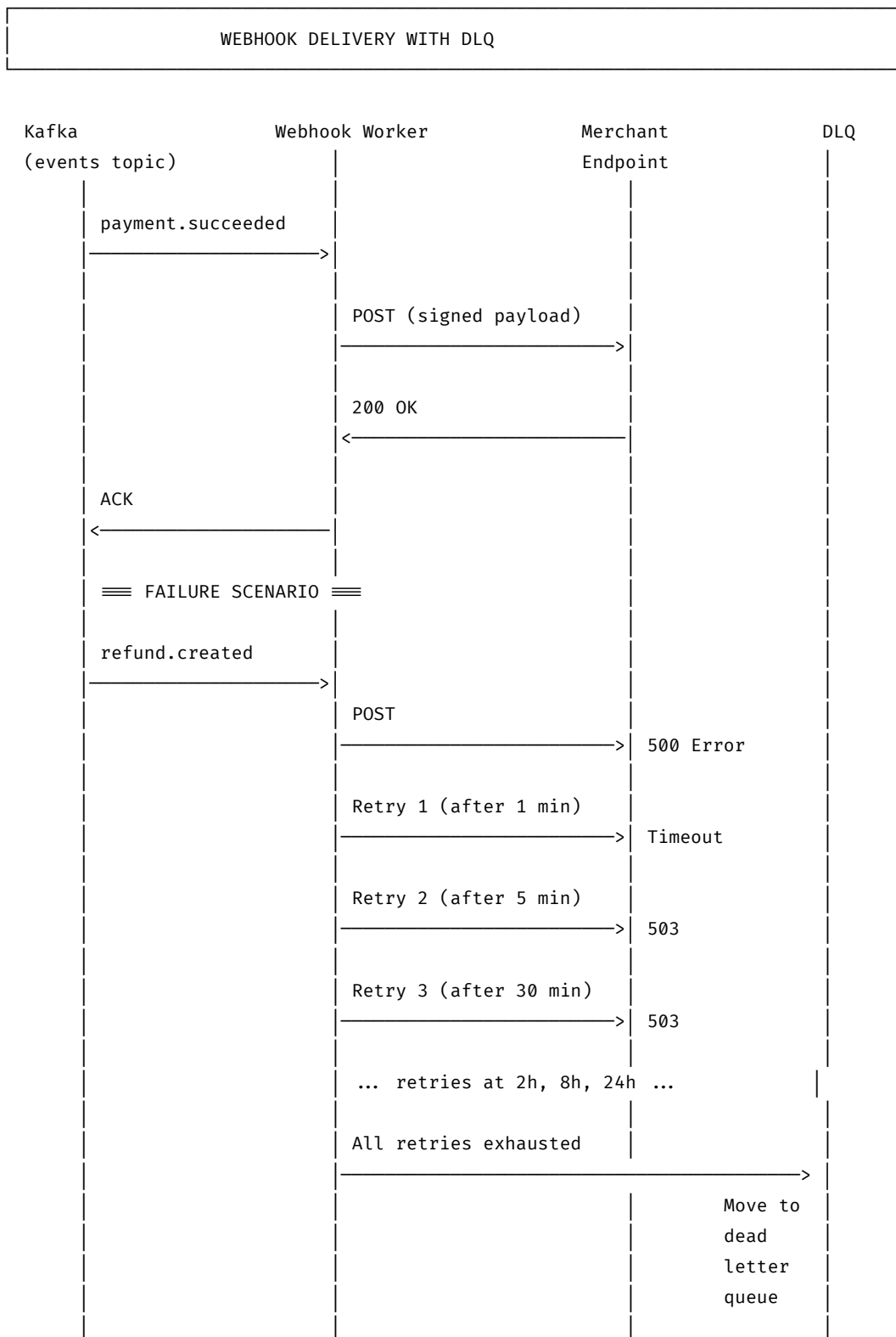


PSP Selection Scoring (when multiple are available):

Score = (success\_rate \* 0.5) + (1/latency\_ms \* 0.3) + (1/cost\_bps \* 0.2)

Highest score wins.

### 11.12.3 Dead Letter Queue for Webhooks



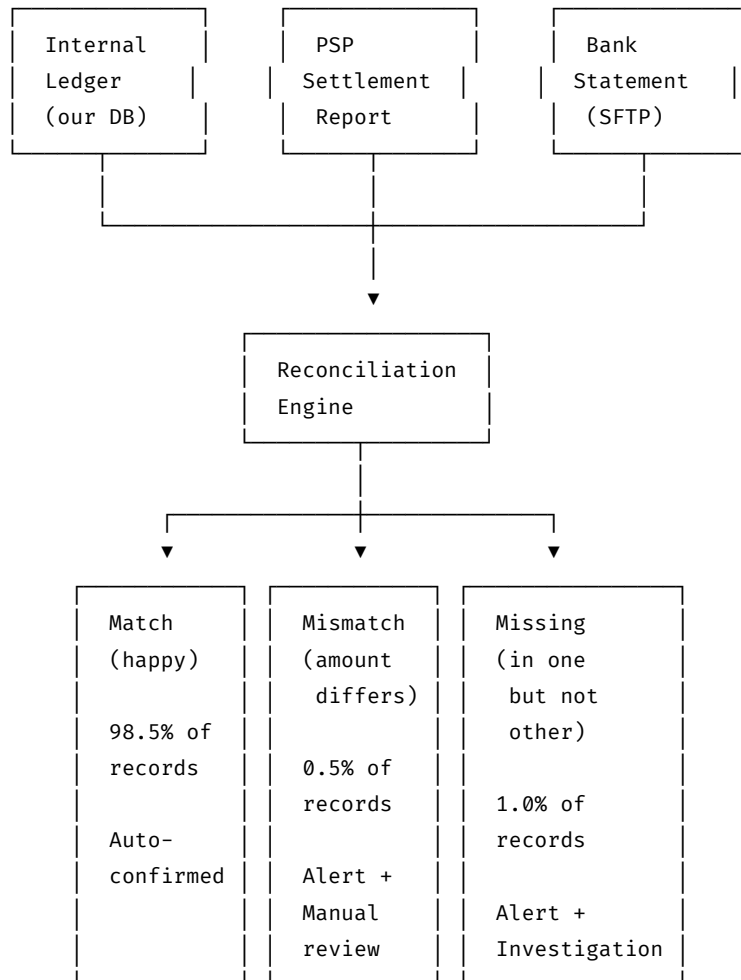
DLQ events are:

- Visible in merchant dashboard for manual review.
- Automatically retried when merchant re-enables webhook endpoint.
- Retained for 30 days, then archived to cold storage.
- Monitored: alert if DLQ size exceeds threshold.

### 11.12.4 Reconciliation Catches Discrepancies

#### DAILY RECONCILIATION PROCESS

Time: Every day at 02:00 UTC (during lowest traffic)



Common discrepancy types:

1. OUR\_ONLY: We recorded a successful payment, PSP has no record  
→ Check with PSP via API; possibly our status update was premature
2. PSP\_ONLY: PSP settled a payment we don't have  
→ Likely a crash-before-record scenario; create the record retroactively
3. AMOUNT\_DIFF: Our amount differs from PSP's settled amount  
→ Usually currency conversion rounding; flag if > \$0.01
4. STATUS\_DIFF: We say "captured", PSP says "refunded"  
→ Webhook was missed; sync our status

## 11.13 13. Scalability

### 11.13.1 Horizontal Scaling Strategy

#### SCALING DIMENSIONS

| Component            | Scaling Approach            | Current → 10x Target |
|----------------------|-----------------------------|----------------------|
| API Gateway          | Horizontal (stateless)      | 4 → 40 instances     |
| Payment Service      | Horizontal (stateless)      | 10 → 100 instances   |
| Risk Engine          | Horizontal (stateless)      | 6 → 60 instances     |
| PSP Adapters         | Horizontal (stateless)      | 8 → 80 instances     |
| PostgreSQL (primary) | Vertical + sharding         | 1 → 8 shards         |
| PostgreSQL (reads)   | Read replicas               | 2 → 16 replicas      |
| Redis Cluster        | Add nodes                   | 3 → 12 nodes         |
| Kafka Cluster        | Add partitions + brokers    | 6 → 24 brokers       |
| Webhook Workers      | Horizontal (stateless)      | 10 → 100 workers     |
| Reconciliation       | Partition by merchant range | 2 → 20 workers       |

### 11.13.2 Database Scaling Path

Stage 1: Single PostgreSQL (up to ~5K TPS writes)

- 96 vCPUs, 768GB RAM, NVMe SSD
- PgBouncer for connection pooling (10K connections → 200 DB connections)
- 2 async read replicas for queries

Stage 2: Functional partitioning (up to ~20K TPS aggregate)

- Separate DB clusters for: payments, ledger, card vault, analytics
- Each cluster: primary + sync standby + 2 async replicas
- Cross-cluster references via application-level joins

Stage 3: Horizontal sharding (up to ~100K TPS aggregate)

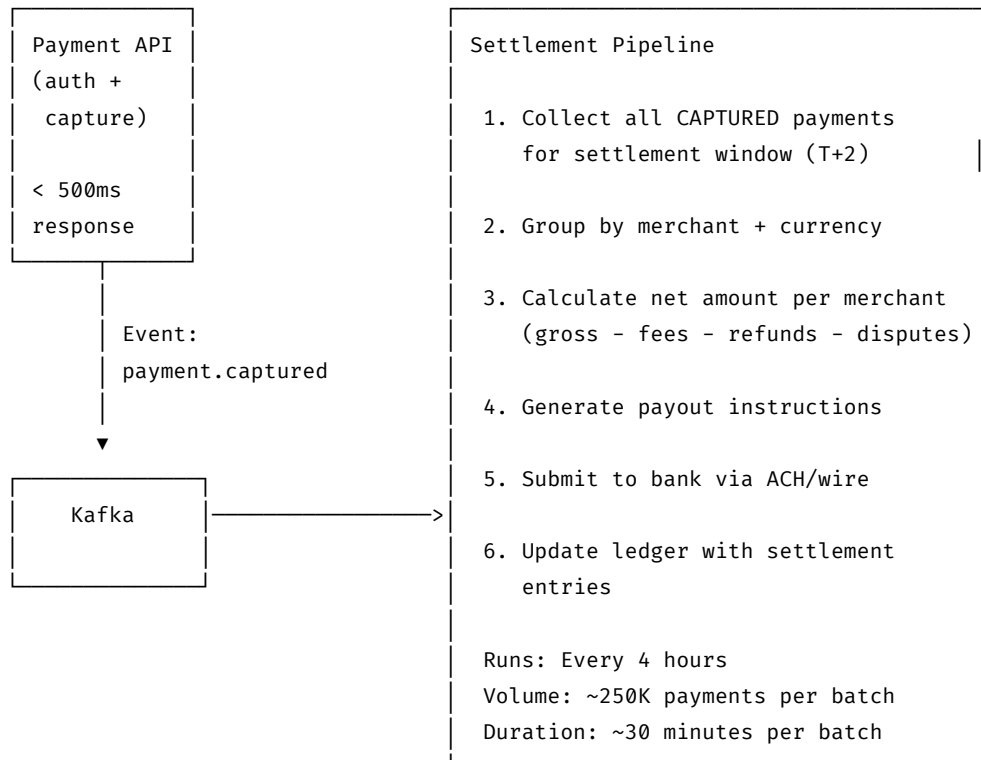
- Payment DB: 8 shards by consistent hash of merchant\_id
- Ledger DB: 8 shards by consistent hash of account\_id
- Shard routing via application-layer proxy (e.g., Vitess or custom)
- Cross-shard queries via scatter-gather (only for admin/analytics)

### 11.13.3 Async Settlement Processing

#### SETTLEMENT PIPELINE (ASYNC)

Real-time path:  
(latency-sensitive)

Async path:  
(batch, throughput-optimized)



## 11.14 14. Monitoring and Observability

### 11.14.1 Key Metrics Dashboard

| PAYMENT SYSTEM METRICS               |                   |                 |                      |
|--------------------------------------|-------------------|-----------------|----------------------|
| BUSINESS METRICS (Grafana Dashboard) |                   |                 |                      |
| Payment Success Rate                 | Target: > 96%     | Alert: < 93%    |                      |
| Authorization Rate                   | Target: > 85%     | Alert: < 80%    |                      |
| GMV (last 24h)                       | \$50M             | Alert: < \$30M  |                      |
| Refund Rate                          | Target: < 5%      | Alert: > 8%     |                      |
| Chargeback Rate                      | Target: < 0.5%    | Alert: > 0.75%  |                      |
| Fraud Detection Rate                 | Target: > 95%     | Alert: < 90%    |                      |
| SYSTEM METRICS                       |                   |                 |                      |
| API Latency (p50/p95/p99)            | 50/200/500ms      | Alert: p99>1s   |                      |
| PSP Latency (p50/p95/p99)            | 100/500/1500ms    | Alert: p99>3s   |                      |
| TPS (current)                        | 12 avg / 120 peak | Alert: > 200    |                      |
| Error Rate (5xx)                     | Target: < 0.01%   | Alert: > 0.1%   |                      |
| DB Connection Pool Usage             | Target: < 70%     | Alert: > 85%    |                      |
| Kafka Consumer Lag                   | Target: < 1000    | Alert: > 10000  |                      |
| Redis Memory Usage                   | Target: < 70%     | Alert: > 85%    |                      |
| RECONCILIATION METRICS               |                   |                 |                      |
| Match Rate                           | Target: > 99%     | Alert: < 98%    |                      |
| Unmatched Transactions               | Target: < 100     | Alert: > 500    |                      |
| Amount Discrepancy Total             | Target: < \$1000  | Alert: > \$5000 |                      |
| Reconciliation Duration              | Target: < 1hr     | Alert: > 3hr    |                      |
| PSP HEALTH (per PSP)                 |                   |                 |                      |
| PSP                                  | Success Rate      | Avg Latency     | Circuit State        |
| Stripe                               | 97.2%             | 180ms           | CLOSED (healthy)     |
| Adyen                                | 96.8%             | 210ms           | CLOSED (healthy)     |
| Square                               | 95.1%             | 250ms           | HALF_OPEN (degraded) |

### 11.14.2 Distributed Tracing

Every payment request gets a trace ID that follows the request across all services.



```

Trace: pay_a1b2c3d4e5
├── API Gateway (2ms)
│ └── Auth + rate limit check
├── Payment Service (15ms)
│ ├── Idempotency check (Redis) (1ms)
│ ├── Create payment record (DB) (3ms)
│ └── Risk evaluation (8ms)
├── PSP Adapter (180ms)
│ ├── Serialize request (1ms)
│ ├── HTTP to Stripe API (175ms) ← Bulk of latency is external
│ └── Parse response (2ms)
├── Ledger Service (5ms)
│ └── Write 2 ledger entries (DB) (4ms)
├── Event Publisher (2ms)
│ └── Write to Kafka (1ms)
└── Total: 204ms

```

Tracing tools: OpenTelemetry → Jaeger/Tempo for visualization

### 11.14.3 Alerting Strategy

SEVERITY LEVELS:

P0 (Page immediately - 24/7 on-call):

- Payment success rate drops below 90%
- All PSP circuits open simultaneously
- Database primary unreachable
- Reconciliation finds > \$10,000 unmatched

P1 (Page during business hours):

- Payment success rate below 93% for > 5 minutes
- Single PSP circuit open for > 10 minutes
- Kafka consumer lag > 50,000
- Webhook DLQ size > 10,000

P2 (Slack alert, next business day):

- Latency p99 > 1s for > 15 minutes
- DB connection pool > 85%
- Fraud rate > 0.08% for rolling 24h

P3 (Dashboard only):

- Non-critical service restart
- Cache hit rate drop
- Approaching storage thresholds

## 11.15 15. Trade-offs and Design Decisions

### 11.15.1 15.1 Synchronous vs. Asynchronous Payment Processing

| SYNC vs ASYNC PAYMENT PROCESSING                                                                                                                             |                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SYNCHRONOUS<br>(Our choice for auth)                                                                                                                         | ASYNCHRONOUS<br>(Our choice for settlement)                                                                                                                                                      |
| Merchant gets immediate result<br>Simpler error handling<br>Higher latency (blocks on PSP)<br>Scales to ~1000 TPS per node<br>PSP timeout = merchant timeout | Merchant polls or receives webhook<br>Complex state management<br>Lower perceived latency for merchant<br>Scales to ~10,000 TPS per node<br>PSP timeout handled internally                       |
| USED FOR: <ul style="list-style-type: none"><li>- Authorization</li><li>- Capture</li><li>- Refund initiation</li></ul>                                      | USED FOR: <ul style="list-style-type: none"><li>- Settlement/payout processing</li><li>- Webhook delivery</li><li>- Reconciliation</li><li>- Analytics/reporting</li><li>- Retry logic</li></ul> |

Decision: Authorization and capture are synchronous because merchants need immediate confirmation to show the customer a success/failure page. Settlement and reconciliation are asynchronous because they are batch-oriented and can tolerate minutes of delay.

### 11.15.2 15.2 Saga vs. Two-Phase Commit (2PC)

| SAGA<br>(Our choice)                                        | 2PC<br>(Not chosen)                                             |
|-------------------------------------------------------------|-----------------------------------------------------------------|
| Each service commits independently                          | All services commit or all rollback atomically                  |
| Compensating transactions on failure (eventual consistency) | Coordinator holds locks until all participants agree (blocking) |
| No distributed locks                                        | Distributed locks across services                               |
| Higher availability                                         | Lower availability (coordinator is SPOF)                        |
| More complex application logic (compensations)              | Simpler application logic but complex infrastructure            |
| Works across heterogeneous services (DB + PSP + Kafka)      | Requires all participants to support XA/2PC protocol            |

Decision: Saga wins because:

1. External PSPs do not support 2PC.
2. 2PC would require holding DB locks for 100-500ms (duration of PSP call), killing throughput.
3. Saga's eventual consistency is acceptable because reconciliation catches any inconsistencies within 24 hours.

### 11.15.3 15.3 Single PSP vs. Multi-PSP

| SINGLE PSP              | MULTI-PSP (Our choice)            |
|-------------------------|-----------------------------------|
| Simpler integration     | Complex adapter layer             |
| Single contract/pricing | Multiple contracts to manage      |
| No routing logic needed | Intelligent routing required      |
| Single point of failure | Failover to backup PSPs           |
| Vendor lock-in          | Negotiate better rates            |
| Limited geo coverage    | Optimal local acquiring worldwide |

Decision: Multi-PSP because:

1. No single PSP has 99.999% uptime; failover improves our availability.
2. Local acquiring (matching PSP to card's issuing country) improves authorization rates by 5-15%.
3. Competitive pricing leverage across PSPs.
4. Regulatory requirements (some countries require local acquirers).

### 11.15.4 15.4 Real-Time vs. Batch Settlement

| REAL-TIME SETTLEMENT                     | BATCH SETTLEMENT (Our choice, industry norm) |
|------------------------------------------|----------------------------------------------|
| Instant merchant payouts                 | T+1 or T+2 payouts                           |
| Higher operational cost                  | Lower banking fees (bulk wire)               |
| Complex netting logic                    | Simple net calculation per batch             |
| Real-time bank integration               | SFTP/batch file to bank                      |
| Higher fraud risk                        | Time to detect and reverse fraud             |
| Premium feature (Stripe Instant Payouts) | Standard for most merchants                  |

Decision: Batch settlement as default (T+2) because:

1. Industry standard; merchants expect it.
2. Netting across transactions reduces banking fees.
3. The 2-day window allows fraud detection and dispute handling before payout.
4. Offer "instant payout" as premium feature with additional fee (1.5%).

### 11.15.5 15.5 Additional Design Decisions

| Decision                | Choice              | Key Reason                                    |
|-------------------------|---------------------|-----------------------------------------------|
| Primary database        | PostgreSQL          | ACID, mature, JSON support                    |
| Message broker          | Kafka               | Durability, ordering, scale                   |
| ID generation           | ULID                | Sortable, no coordination                     |
| API style               | REST + webhooks     | Industry standard for payment APIs            |
| Currency representation | Smallest unit (int) | Avoids floating-point errors (\$50.00 = 5000) |
| Idempotency key storage | Redis + PostgreSQL  | Fast lookup + crash-safe                      |
| Secret management       | AWS KMS + HSM       | PCI requirement                               |
| Ledger mutability       | Append-only         | Audit trail, no data loss                     |

## 11.16 16. Interview Deep-Dive Questions

### 11.16.1 Q1: How do you prevent double charging?

**Answer:** Multiple layers of defense:

1. **Idempotency key:** Every payment request includes a client-generated unique key. The server stores (merchant\_id, idempotency\_key) → response in Redis (fast path) and PostgreSQL (durable path). Before processing, we check if this key was seen before. If yes, return the cached response.
2. **Database unique constraint:** UNIQUE (merchant\_id, idempotency\_key) on the payments table prevents duplicate records even if Redis is unavailable.

3. **Optimistic locking:** The `version` column on the payment record prevents concurrent state transitions. `UPDATE payments SET status='CAPTURED', version=version+1 WHERE id=? AND version=?` – if the version doesn't match, the update affects 0 rows.
  4. **PSP-level idempotency:** We forward our payment ID as the PSP's idempotency key. If we accidentally send the same auth request twice, the PSP returns the original result.
  5. **Reconciliation:** Even if all runtime safeguards fail, daily reconciliation catches any duplicate charges by comparing our records with PSP settlement reports.
- 

### 11.16.2 Q2: How do you handle a payment that succeeds at the bank but your system crashes before recording it?

**Answer:** This is the classic “crash after external side effect” problem.

1. **Before the PSP call:** We write the payment record with `status=PROCESSING` and store the idempotency key as `processing` in Redis. This is our “intent to charge.”
  2. **Recovery worker:** A background job runs every 30 seconds, scanning for payments stuck in `PROCESSING` for more than 60 seconds.
  3. **Status check:** The recovery worker queries the PSP using our payment ID / reference to determine the actual outcome:
    - If PSP says “authorized”: update our record to `AUTHORIZED`, proceed normally.
    - If PSP says “not found”: update to `FAILED`, release the idempotency key for retry.
    - If PSP is unreachable: keep in `PROCESSING`, retry the status check later.
  4. **Idempotency safety:** Even during recovery, the PSP's own idempotency ensures we don't accidentally authorize again when checking status.
  5. **Daily reconciliation:** As a final safety net, the reconciliation process compares every PSP-recorded transaction against our database. Any “PSP-only” transactions (succeeded at PSP but missing in our DB) are flagged and auto-corrected.
- 

### 11.16.3 Q3: How does reconciliation work?

**Answer:** Three-way reconciliation runs daily:

1. **Internal ledger vs. PSP settlement report:**
  - Download settlement files from each PSP (via SFTP or API).
  - Match each settlement line item to our internal payment record by PSP reference ID.
  - Flag: amount mismatches, missing records (either side), status conflicts.
2. **PSP settlement vs. bank statement:**
  - Download bank statements (MT940/CAMT.053 format via SFTP).
  - Match aggregate PSP settlement amounts to bank credit entries.
  - Verify net settlement = gross payments - refunds - chargebacks - PSP fees.
3. **Internal ledger balance check:**
  - Verify `SUM(debits) = SUM(credits)` across all ledger entries (must be zero).
  - Verify each account's computed balance matches the stored running balance.
  - Any non-zero sum indicates a bug in the ledger writing code.

Discrepancies are categorized by severity and routed to the appropriate team. Auto-correction is applied for known patterns (e.g., rounding differences < \$0.01). Manual review is required for amounts > \$1.

---

#### 11.16.4 Q4: How do you handle currency conversion?

##### Answer:

1. **Exchange rate sourcing:** We pull rates from multiple providers (e.g., European Central Bank, Open Exchange Rates) every 60 seconds and cache in Redis with 60-second TTL. We use the mid-market rate plus a configurable margin (typically 1-2%).
  2. **Rate locking:** When a payment is created, we lock the exchange rate at that moment and store it on the payment record (`exchange_rate`, `exchange_rate_locked_at`). This prevents rate fluctuation between authorization and settlement.
  3. **Presentation currency:** The customer sees the charge in their card's currency (decided by the card network via Dynamic Currency Conversion or Multi-Currency Pricing).
  4. **Settlement currency:** The merchant always receives payouts in their configured settlement currency. Conversion happens at capture time using the locked rate.
  5. **Ledger entries:** Multi-currency payments create entries in BOTH currencies:
    - Customer side: debit in customer's currency (e.g., EUR).
    - Merchant side: credit in merchant's currency (e.g., USD).
    - FX gain/loss account absorbs rounding differences.
  6. **Amounts always in smallest unit:** EUR 10.50 = 1050 cents. JPY 1000 = 1000 (JPY has no minor unit). We maintain a currency configuration table with decimal places per currency.
- 

#### 11.16.5 Q5: How do you design for PCI compliance without handling raw card numbers?

##### Answer:

1. **Client-side tokenization:** Card numbers are collected in an isolated iframe or embedded component (like Stripe Elements) that communicates directly with our PCI-scoped Card Vault service. The merchant's server never sees the raw PAN.
  2. **Network segmentation:** The Card Vault runs in an isolated VPC segment with no internet access (except to card networks). Only the tokenization API is exposed internally, and only to the Payment Service.
  3. **Column-level encryption:** Raw PANs are encrypted with AES-256 using Data Encryption Keys (DEKs) that are themselves encrypted by Key Encryption Keys (KEKs) stored in an HSM (Hardware Security Module). The HSM never exports keys in plaintext.
  4. **Tokenization:** After encryption, the Card Vault returns a token (e.g., `pm_abc123`) that represents the card. All other services reference the token, never the PAN.
  5. **Scope minimization:** By keeping raw PANs confined to the Card Vault (a small, heavily audited service), the PCI audit scope covers only ~5% of our infrastructure instead of 100%.
  6. **Key rotation:** Encryption keys rotate every 90 days. Old data is re-encrypted in background batch jobs during rotation.
-

### 11.16.6 Q6: How do you handle partial captures and partial refunds?

**Answer:**

Partial capture: A merchant authorizes \$100 but only captures \$75 (e.g., one item out of stock). - The payment record tracks `amount` (authorized), `amount_captured`, `amount_refunded`. - Capture request specifies the partial amount. - Remaining authorized amount (\$25) is automatically voided after capture. - Ledger entries reflect only the captured amount.

Partial refund: A merchant refunds \$30 out of a \$75 captured payment. - A new refund record is created linked to the payment. - `amount_refunded` on the payment is incremented. - Validation:  $\text{amount\_refunded} + \text{new\_refund\_amount} \leq \text{amount\_captured}$ . - Multiple partial refunds are allowed until the captured amount is fully refunded. - Each partial refund creates its own balanced ledger entries.

---

### 11.16.7 Q7: How does the webhook signing and verification work?

**Answer:**

1. **Signing:** Each webhook delivery is signed with the merchant's webhook secret using HMAC-SHA256.

```
signature = HMAC-SHA256(webhook_secret, timestamp + "." + payload_json)
```

2. **Replay protection:** The timestamp is included in the signature. Merchants should reject webhooks where  $\text{abs}(\text{current\_time} - \text{timestamp}) > 5 \text{ minutes}$ .

3. **Header format:**

```
Webhook-Signature: t=1712649600,v1=5d4b3 ... a2f1
```

4. **Merchant verification:** Merchant recalculates the HMAC using their stored secret and compares. If it matches, the webhook is authentic and untampered.
- 

### 11.16.8 Q8: How do you handle subscription/recurring payments?

**Answer:**

1. **Subscription object:** Stores plan, billing cycle (monthly, yearly), next billing date, payment method, retry policy.
  2. **Billing scheduler:** A cron job runs hourly, queries subscriptions where  $\text{next\_billing\_date} \leq \text{NOW}()$ , and creates payment requests.
  3. **Smart retries:** If a recurring payment fails:
    - Retry 1: 24 hours later (soft decline might be temporary insufficient funds).
    - Retry 2: 3 days later (wait for paycheck deposit).
    - Retry 3: 7 days later (final attempt).
    - After 3 failures: mark subscription as `past_due`, notify merchant via webhook.
  4. **Dunning:** Email notifications to customer on each failure, with a link to update their payment method.
  5. **Proration:** When a customer upgrades/downgrades mid-cycle, calculate the prorated amount based on remaining days in the current period.
-

### 11.16.9 Q9: How do you ensure exactly-once settlement (no double payouts to merchants)?

**Answer:**

1. **Settlement batch ID:** Each settlement run generates a unique batch ID. Payments are marked with `settlement_batch_id` when included. A payment can only appear in one batch (`UNIQUE INDEX ON settlement_batch_id, payment_id`).
  2. **State machine:** Payments must be in `CAPTURED` state to be included in settlement. After inclusion, they transition to `SETTLED`. The state transition is atomic.
  3. **Batch-level idempotency:** The bank payout instruction includes the batch ID. If the same batch is submitted twice (e.g., due to a retry), the bank deduplicates.
  4. **Settlement hold:** Payments are only eligible for settlement after a configurable hold period (default T+2). This window allows for fraud detection and disputes.
- 

### 11.16.10 Q10: How do you handle chargebacks/disputes?

**Answer:**

1. **Chargeback notification:** Card networks notify us via the PSP when a customer disputes a charge. This arrives as an asynchronous event.
  2. **Automatic hold:** The disputed amount is immediately debited from the merchant's balance and held in a `DISPUTE_HOLDING` ledger account.
  3. **Evidence submission:** The merchant is notified via webhook and can submit evidence (receipts, shipping info, communication logs) through our API within the response deadline (typically 7-21 days).
  4. **Resolution:** The card network makes the final decision:
    - Won: Held funds returned to merchant, chargeback reversed.
    - Lost: Held funds returned to customer, merchant absorbs the loss.
  5. **Monitoring:** Merchants exceeding chargeback thresholds (>1% of transactions) are flagged for review and may be restricted or suspended.
- 

### 11.16.11 Q11: How does the payment router decide which PSP to use?

**Answer:** The router uses a scoring algorithm considering:

1. **Authorization rate** (50% weight): Historical success rate for this card BIN + merchant category on each PSP. A PSP with 97% auth rate scores higher than one with 93%.
2. **Latency** (20% weight): Rolling p95 latency. Faster PSPs score higher.
3. **Cost** (15% weight): Interchange plus markup. Lower fees score higher.
4. **Local acquiring** (15% weight): Strong preference for PSPs that have a local acquiring relationship in the card's issuing country. Local acquirers see 5-15% higher auth rates.
5. **Circuit breaker override:** If a PSP's circuit is `OPEN`, it is excluded regardless of score. If `HALF-OPEN`, it receives a small fraction of traffic for testing.
6. **Merchant override:** Merchants can configure PSP preferences (e.g., "always use Stripe for USD transactions").



---

**11.16.12 Q12: How do you handle multi-region deployment for a payment system?****Answer:**

1. **Active-passive per region:** One region is primary for writes (e.g., us-east-1). A secondary region (eu-west-1) has a warm standby database with synchronous replication.
  2. **Why not active-active:** Payment processing requires strong consistency. Active-active with eventual consistency risks double charges. The complexity of conflict resolution for financial data is not worth the latency improvement.
  3. **Latency mitigation:** Even with a single write region, read-heavy operations (payment status, merchant dashboard) can be served from regional read replicas.
  4. **Regulatory compliance:** For EU/GDPR, some data must remain in EU. We handle this by routing EU merchant data to the EU database cluster while maintaining a global ledger view via async replication.
  5. **Failover:** If the primary region goes down, we promote the secondary region. RTO: < 60 seconds. RPO: 0 (synchronous replication).
- 

**11.16.13 Q13: How do you handle rate limiting for payment APIs?****Answer:**

Rate limiting is essential to prevent abuse, but we must be careful not to reject legitimate payment traffic.

1. **Per-merchant limits:** Default 100 requests/second per merchant, configurable. High-volume merchants (e.g., Amazon) get custom higher limits.
  2. **Token bucket algorithm:** Allows short bursts above the sustained rate. A merchant with 100 RPS limit can burst to 200 RPS for 5 seconds.
  3. **Different limits per endpoint:**
    - POST /v1/payments: 100/s (most critical, highest limit).
    - GET /v1/payments/{id}: 500/s (reads are cheaper).
    - POST /v1/refunds: 50/s (refunds are less frequent).
  4. **429 response:** Includes `Retry-After` header and `X-RateLimit-Remaining` for client-side throttling.
  5. **Global rate limiting:** Overall system limit (e.g., 10,000 TPS) with fair queuing to prevent one merchant from starving others.
- 

**11.16.14 Q14: What happens if Kafka goes down? Are payments lost?**

**Answer:** No. Kafka is used for async operations (webhooks, settlement, analytics), not for the critical payment path.

1. **Payment processing does not depend on Kafka:** Authorization and capture happen synchronously via the PSP. If Kafka is down, the payment still succeeds.
  2. **Outbox pattern:** Events are written to the PostgreSQL `outbox_events` table in the same transaction as the payment record. The outbox relay publishes to Kafka when it's available.
-

3. **If Kafka is down:** Events accumulate in the outbox table. When Kafka recovers, the relay publishes all pending events in order. This may cause delayed webhooks but no data loss.
  4. **Webhook impact:** Merchants receive delayed notifications but can always poll `GET /v1/payments/{id}` for real-time status.
  5. **Settlement impact:** Settlement batches may be delayed but will process correctly once Kafka recovers (events are idempotent).
- 

#### 11.16.15 Q15: How would you migrate from a monolithic payment system to this microservices architecture?

**Answer:** The Strangler Fig pattern:

1. **Phase 1 - Extract Card Vault** (Week 1-4):
  - Build the PCI-scoped Card Vault as a new service.
  - Migrate raw PANs from the monolith DB to the vault.
  - Update the monolith to use tokens instead of raw PANs.
  - Immediate PCI scope reduction benefit.
2. **Phase 2 - Extract Ledger** (Week 5-8):
  - Build the Ledger Service with double-entry bookkeeping.
  - Dual-write: monolith writes to both old tables and new ledger.
  - Verify consistency between old and new for 2 weeks.
  - Cut over reads to new ledger service.
3. **Phase 3 - Extract Payment Processing** (Week 9-16):
  - Build the Payment Service, Risk Engine, and Payment Router.
  - Use feature flags to route X% of traffic to the new system.
  - Gradually increase: 1% -> 5% -> 25% -> 50% -> 100%.
  - Monitor success rates, latency, and reconciliation at each step.
4. **Phase 4 - Decommission monolith** (Week 17-20):
  - Once 100% of traffic is on the new system and stable for 2 weeks.
  - Keep monolith in read-only mode for 30 days as rollback safety net.
  - Fully decommission after 30 days.

Key principle: Never do a big-bang migration for a payment system. Gradual cutover with dual-write verification at every stage. If anything looks wrong, immediately route back to the monolith.

---

## 11.17 Summary: Key Takeaways for Interview

### TOP INTERVIEW TALKING POINTS

#### 1. EXACTLY-ONCE PROCESSING

Idempotency key + DB unique constraint + PSP-level dedup + reconciliation

#### 2. DOUBLE-ENTRY LEDGER

Every transaction creates balanced entries. SUM always = 0.

Append-only. Corrections via reversal entries, never mutations.

#### 3. PCI COMPLIANCE

Minimize scope. Raw PANs only in Card Vault. Everything else uses tokens.

#### 4. FAILURE HANDLING

Never retry blindly. Always check PSP status first.

Recovery workers fix stuck payments. Reconciliation is the safety net.

#### 5. SAGA PATTERN

No 2PC across PSP boundary. Compensating transactions for rollback.

Eventual consistency acceptable with reconciliation backstop.

#### 6. MULTI-PSP ROUTING

Circuit breakers, success-rate-based routing, local acquiring preference.

#### 7. CONSISTENCY MODEL

Strong consistency for payment state (PostgreSQL synchronous replication).

Eventual consistency for analytics, webhooks, dashboards.

#### 8. CURRENCY HANDLING

Always smallest unit (cents). Never floating point. Lock rate at capture.

## Chapter 12

# Design a Ticket Booking System (Book-MyShow / Ticketmaster)

**Difficulty:** Hard | **Companies:** Amazon, Google, Microsoft, Uber, Swiggy

For the low-level class design, see [LLD: Movie Ticket Booking](#)

---

### 12.1 1. Problem Statement and Scope

Design a large-scale ticket booking system (similar to BookMyShow, Ticketmaster, or Fandango) that allows users to browse events – movies, concerts, sports, theatre – search by location, date, and genre, view real-time seat maps, select seats, make payments, and receive confirmations.

The system must handle **extreme concurrency** during flash sales (e.g. a BTS concert where 500K fans compete for 50K seats in seconds) while guaranteeing **zero double bookings** and maintaining sub-second response times for seat availability queries.

#### 12.1.1 In Scope

- Event/movie/show catalog management and discovery
- Real-time seat map with availability status
- Concurrent seat selection with temporary holds
- Payment processing with timeout and retry logic
- Booking confirmation, history, and cancellation
- Flash sale / high-demand event handling
- Notifications (email, SMS, push)
- Search and recommendations

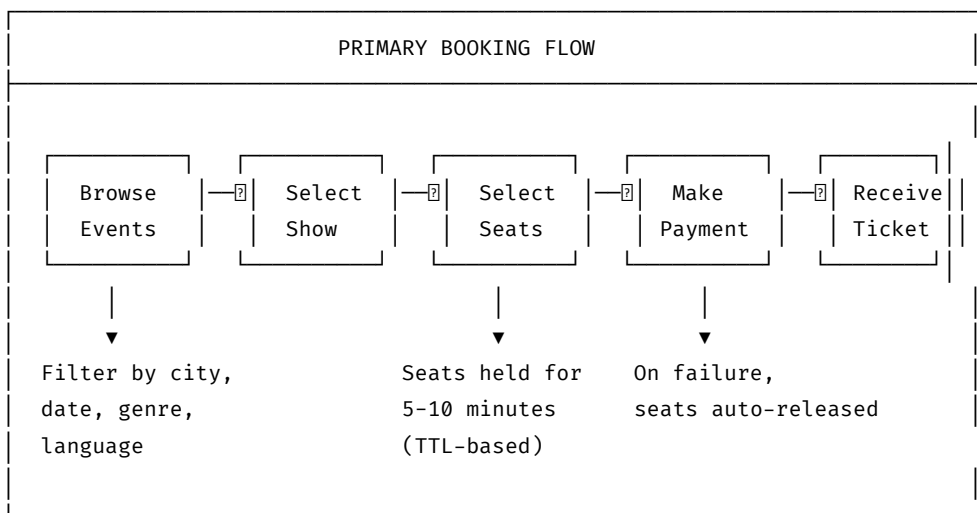
#### 12.1.2 Out of Scope

- Theater/venue onboarding and admin panel (CMS)
  - Content streaming (for virtual events)
  - Loyalty points / reward programs
  - Third-party affiliate and reseller integrations
  - Ticket transfer or secondary marketplace
-

## 12.2 2. Functional Requirements

- **FR-1 - Browse events/movies by city, date:** Priority: P0
- **FR-2 - Search with filters (genre, language):** Priority: P0
- **FR-3 - View seat map with real-time status:** Priority: P0
- **FR-4 - Select seats and hold temporarily:** Priority: P0
- **FR-5 - Process payment and confirm booking:** Priority: P0
- **FR-6 - Booking cancellation and refund:** Priority: P0
- **FR-7 - Booking history and e-ticket:** Priority: P0
- **FR-8 - Notifications (confirmation, remind):** Priority: P1
- **FR-9 - Reviews and ratings for events:** Priority: P1
- **FR-10 - Dynamic pricing for high-demand shows:** Priority: P2
- **FR-11 - Group booking (10+ seats together):** Priority: P2
- **FR-12 - Waiting list for sold-out events:** Priority: P2

### 12.2.1 User Journeys



## 12.3 3. Non-Functional Requirements

- **Availability:** 99.99% (52 min downtime/year)
- **Seat consistency:** Zero double bookings (strong consistency)
- **Booking flow latency:** End-to-end < 30 seconds (excluding payment gateway)
- **Seat map load time:** < 500ms at P99
- **Flash sale concurrency:** 500K concurrent users competing for 50K seats
- **Search latency:** < 200ms at P95
- **Notification delivery:** < 60 seconds after booking confirmation
- **Data durability:** Zero booking data loss
- **Eventual consistency:** Event metadata, reviews: < 5 seconds propagation
- **Horizontal scalability:** Scale to 10x load within 15 minutes (auto-scaling)

### 12.3.1 Consistency Requirements by Data Type

| Data Type                | Consistency Model    | Rationale            |
|--------------------------|----------------------|----------------------|
| Seat inventory / holds   | Strong consistency   | No double booking    |
| Booking records          | Strong consistency   | Financial accuracy   |
| Payment transactions     | Strong consistency   | Idempotency required |
| Event catalog / metadata | Eventual consistency | Reads >> Writes      |
| Search index             | Near-real-time (~2s) | Acceptable lag       |
| Reviews and ratings      | Eventual consistency | Non-critical path    |
| User sessions            | Session consistency  | Per-user state       |

## 12.4 4. Back-of-Envelope Estimation

### 12.4.1 Traffic Estimates

|                                   |                                              |
|-----------------------------------|----------------------------------------------|
| Daily active users (DAU):         | 50M                                          |
| Daily bookings:                   | 10M                                          |
| Avg seats per booking:            | 2.5                                          |
| Daily seat transactions:          | 25M                                          |
| Daily seat availability checks:   | 50M (5x bookings -- browsing without buying) |
| Peak QPS (normal):                | 10M / 86400 ≈ 115 bookings/sec               |
| Peak QPS (flash sale, 10s burst): | 500K users / 10s = 50K req/sec               |

### 12.4.2 Storage Estimates

| STORAGE ESTIMATION           |                                     |
|------------------------------|-------------------------------------|
| Booking record               | ~2 KB each                          |
| Daily booking storage        | 2 KB × 10M = 20 GB/day              |
| Annual booking storage       | 20 GB × 365 = 7.3 TB/year           |
| Event metadata               | ~5 KB each (includes images refs)   |
| Total events (active)        | 100K events × 5 KB = 500 MB         |
| Seat inventory (per show)    | ~100 bytes/seat                     |
| Active seat inventory        | 100K shows × 1K seats × 100B = 10GB |
| User profiles                | ~1 KB each                          |
| Total users                  | 200M × 1 KB = 200 GB                |
| Search index (Elasticsearch) | ~2 GB (event + show metadata)       |

### 12.4.3 Bandwidth Estimates

Seat map response: ~50 KB (1000 seats with status, coordinates, pricing)  
 Seat map peak bandwidth: 50 KB × 50K req/sec = 2.5 GB/sec (flash sale)  
 Event listing response: ~10 KB (paginated, 20 events)  
 Average API response: ~5 KB  
 Normal bandwidth: 5 KB × 5000 req/sec = 25 MB/sec

### 12.4.4 Redis Memory for Seat Holds

Active holds at any moment: ~500K seats (assuming 5-min hold, 10M daily bookings)  
 Per-hold Redis entry: ~200 bytes (key + metadata)  
 Redis memory for holds: 500K × 200B = 100 MB (trivial)  
 Flash sale spike: 50K holds × 200B = 10 MB additional

---

## 12.5 5. API Design

### 12.5.1 5.1 Event Discovery APIs

```
GET /v1/events
 ?city=mumbai
 &date=2026-04-15
 &category=movies|concerts|sports
 &genre=action
 &language=hindi
 &page=1
 &limit=20
```

Response 200:

```
{
 "events": [
 {
 "id": "evt_abc123",
 "title": "Avengers: Secret Wars",
 "category": "movie",
 "genre": ["action", "sci-fi"],
 "rating": 4.5,
 "languages": ["hindi", "english"],
 "thumbnail_url": "https://cdn.example.com/ ... ",
 "min_price": 150,
 "shows_available": 42
 }
],
 "pagination": { "page": 1, "total_pages": 12, "total_results": 235 }
}
```

```
GET /v1/events/{event_id}/shows
 ?city=mumbai
 &date=2026-04-15
```

Response 200:

```
{
 "shows": [
 {
 "show_id": "shw_xyz789",
 "event_id": "evt_abc123",
 "venue": { "id": "ven_001", "name": "PVR Phoenix", "city": "mumbai" },
 "screen": "Screen 3",
 "start_time": "2026-04-15T14:30:00+05:30",
 "end_time": "2026-04-15T17:15:00+05:30",
 "available_seats": 180,
 "total_seats": 250,
 "pricing": {
 "silver": 150,
 "gold": 250,
 "platinum": 400
 }
 }
]
}
```

### 12.5.2 5.2 Seat Map API

```
GET /v1/shows/{show_id}/seats
```

Response 200:

```
{
 "show_id": "shw_xyz789",
 "layout": {
 "rows": 15,
 "columns": 20,
 "screen_position": "top"
 },
 "sections": [
 {
 "type": "platinum",
 "price": 400,
 "seats": [
 { "id": "A1", "row": "A", "number": 1, "status": "available" },
 { "id": "A2", "row": "A", "number": 2, "status": "held" },
 { "id": "A3", "row": "A", "number": 3, "status": "booked" }
]
 }
],
 "last_updated": "2026-04-15T14:25:00Z",
 "cache_ttl_seconds": 5
}
```



### 12.5.3 5.3 Booking APIs

POST /v1/bookings/hold

Headers: Authorization: Bearer <token>  
Idempotency-Key: <uuid>

Body:

```
{
 "show_id": "shw_xyz789",
 "seat_ids": ["A1", "A4", "A5"],
 "user_id": "usr_456"
}
```

Response 200:

```
{
 "hold_token": "hld_abc123def456",
 "seats_held": ["A1", "A4", "A5"],
 "hold_expires_at": "2026-04-15T14:35:00Z",
 "ttl_seconds": 480,
 "total_amount": 1200.00,
 "breakdown": [
 { "seat": "A1", "type": "platinum", "price": 400 },
 { "seat": "A4", "type": "platinum", "price": 400 },
 { "seat": "A5", "type": "platinum", "price": 400 }
]
}
```

Response 409 (Conflict):

```
{
 "error": "SEATS_UNAVAILABLE",
 "unavailable_seats": ["A4"],
 "message": "Some selected seats are no longer available"
}
```

POST /v1/bookings/confirm

Headers: Authorization: Bearer <token>  
Idempotency-Key: <uuid>

Body:

```
{
 "hold_token": "hld_abc123def456",
 "payment_info": {
 "method": "upi",
 "upi_id": "user@upi",
 "amount": 1200.00
 }
}
```

Response 200:

```
{
 "booking_id": "bkg_789xyz",
 "status": "confirmed",
 "e_ticket_url": "https://tickets.example.com/bkg_789xyz",
 "qr_code_url": "https://cdn.example.com/qr/bkg_789xyz.png",
 "seats": ["A1", "A4", "A5"],
 "show_details": { ... },
 "payment_receipt_id": "pay_receipt_123"
}
```

DELETE /v1/bookings/{booking\_id}

Response 200:

```
{
 "booking_id": "bkg_789xyz",
 "status": "cancelled",
 "refund": {
 "amount": 1080.00,
 "deduction": 120.00,
 "reason": "cancellation_fee_10_percent",
 "refund_to": "original_payment_method",
 "estimated_completion": "2026-04-18T00:00:00Z"
 }
}
```

### 12.5.4 5.4 Search API

```
GET /v1/search
 ?q=avengers
 &city=mumbai
 &lat=19.076
 &lon=72.877
 &radius_km=10
 &category=movies
 &sort=relevance|date|rating|price_low
```

Response 200:

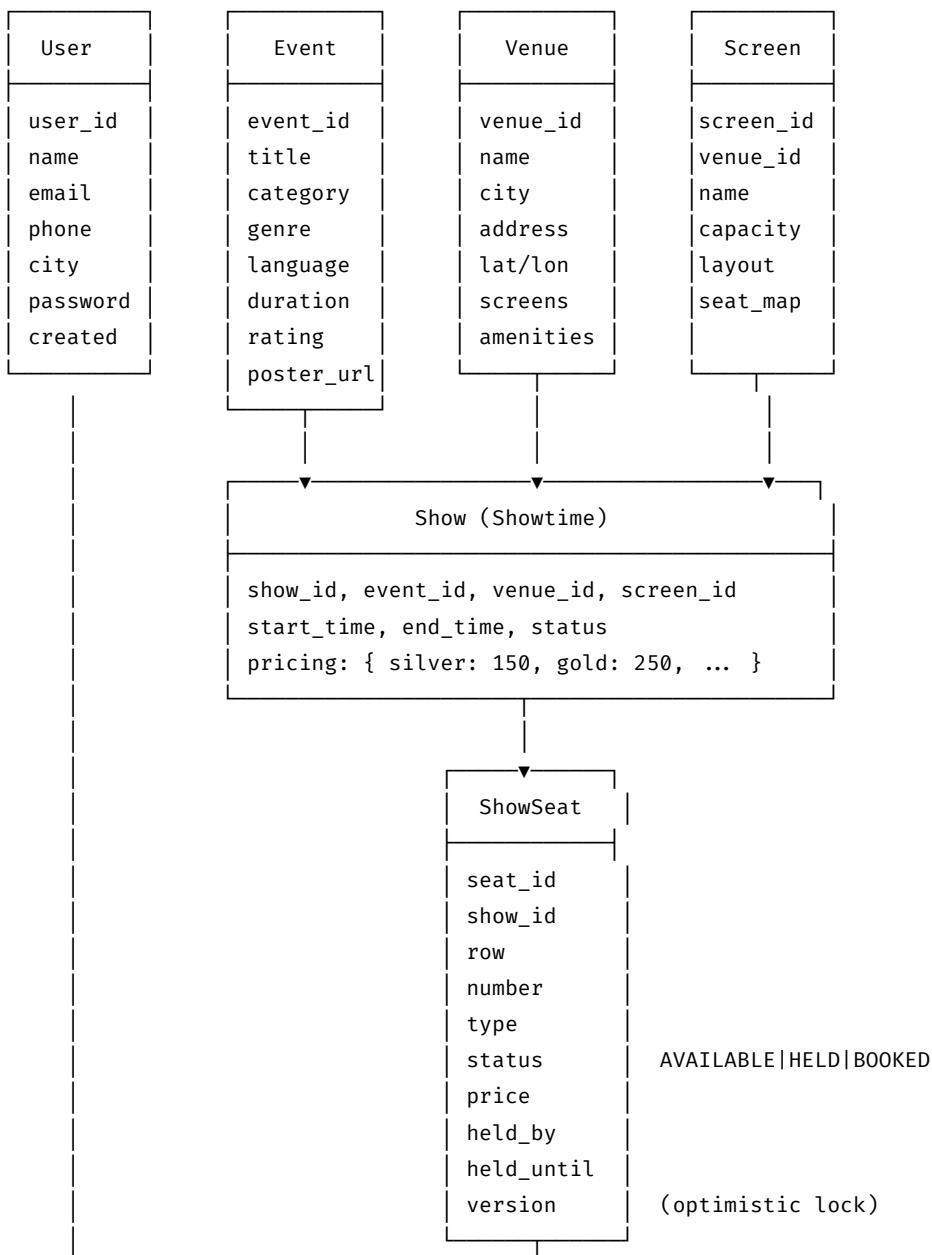
```
{
 "results": [...],
 "facets": {
 "genres": [{"name": "action", "count": 12}, ...],
 "languages": [{"name": "hindi", "count": 8}, ...],
 "price_ranges": [{"range": "100-200", "count": 15}, ...]
 }
}
```

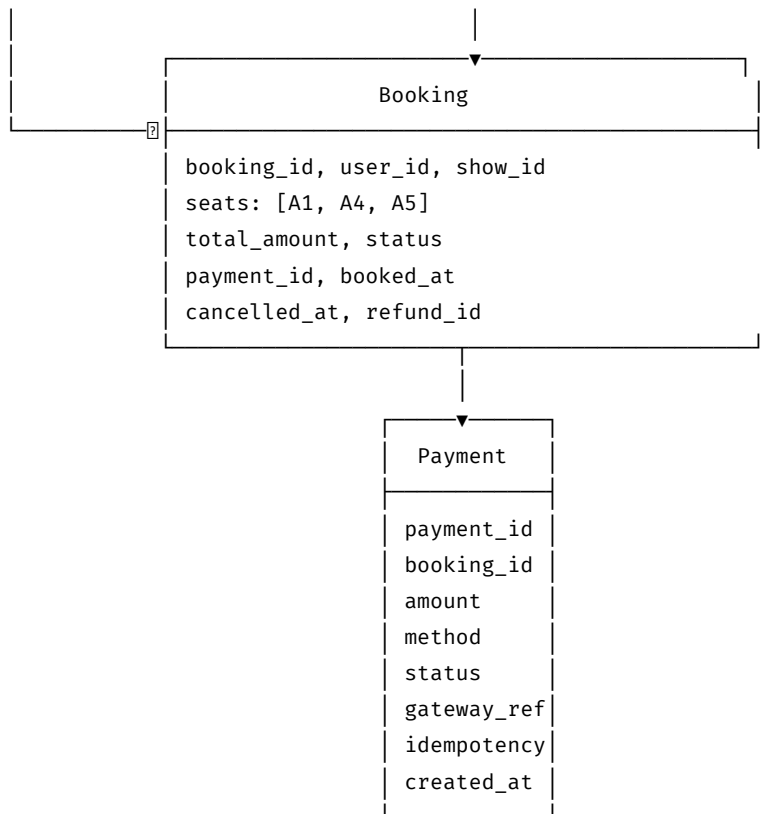
### 12.5.5 Rate Limiting Strategy

| API RATE LIMITS     |                  |                              |
|---------------------|------------------|------------------------------|
| Endpoint            | Rate Limit       | Notes                        |
| GET /events         | 100 req/min/user | Cacheable, lenient           |
| GET /shows/{}/seats | 30 req/min/user  | Higher during flash sale     |
| POST /bookings/hold | 5 req/min/user   | Prevent seat hoarding        |
| POST /bookings/conf | 3 req/min/user   | Tied to active hold          |
| GET /search         | 60 req/min/user  | Elasticsearch backed         |
| Flash sale queue    | 1 req/event/user | One entry per user per event |

## 12.6 6. Data Model and Database Selection

### 12.6.1 6.1 Entity-Relationship Overview

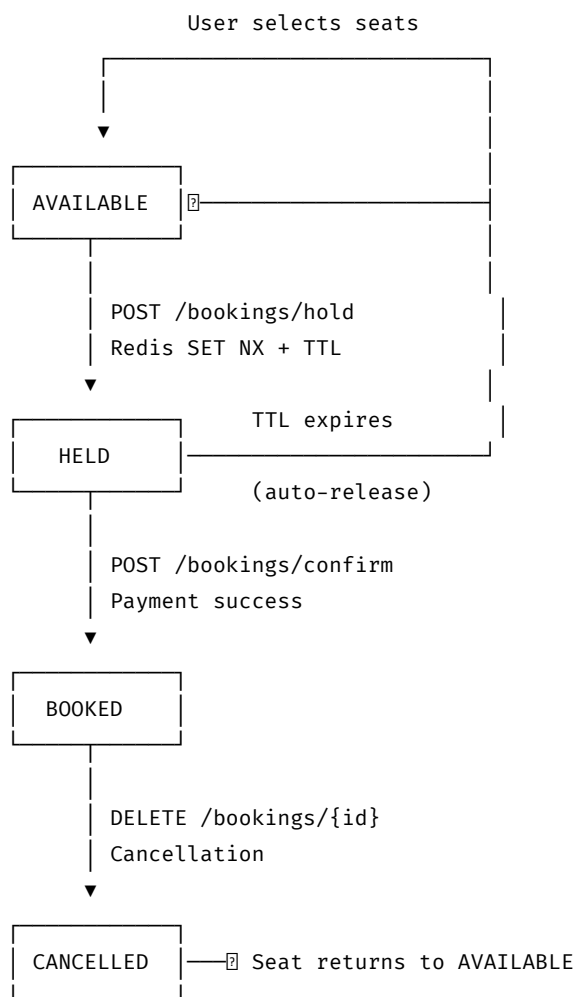




### 12.6.2 6.2 Database Selection

| Data                                             | Database                          | Rationale                                                          |
|--------------------------------------------------|-----------------------------------|--------------------------------------------------------------------|
| Users, Events, Venues, Shows, Bookings, Payments | MySQL (InnoDB)                    | ACID, relational joins, mature tooling, strong consistency         |
| Seat inventory (real-time holds)                 | Redis (primary) + MySQL (durable) | O(1) lookups, atomic SET NX, TTL-based auto-expiry, sub-ms latency |
| Search index                                     | Elasticsearch                     | Full-text search, faceted filters, geo queries                     |
| Session / cart state                             | Redis                             | Fast reads, TTL expiry                                             |
| Event images / assets                            | S3 + CDN                          | Blob storage, edge caching                                         |
| Analytics / event logs                           | ClickHouse / Apache Druid         | Column-oriented, aggregation queries                               |

### 12.6.3 6.3 Seat Status State Machine



### 12.6.4 6.4 Key Table Schemas

-- Show Seats with optimistic locking

```
CREATE TABLE show_seats (
 id BIGINT PRIMARY KEY AUTO_INCREMENT,
 show_id VARCHAR(36) NOT NULL,
 seat_id VARCHAR(10) NOT NULL, -- e.g., "A1", "B12"
 row_label CHAR(2) NOT NULL,
 seat_number SMALLINT NOT NULL,
 seat_type ENUM('silver','gold','platinum') NOT NULL,
 status ENUM('available','held','booked','cancelled') DEFAULT 'available',
 price DECIMAL(10,2) NOT NULL,
 held_by_user VARCHAR(36),
 held_until TIMESTAMP NULL,
 booked_by_user VARCHAR(36),
 booking_id VARCHAR(36),
 version INT DEFAULT 0, -- optimistic lock version

 UNIQUE INDEX idx_show_seat (show_id, seat_id),
 INDEX idx_show_status (show_id, status),
 INDEX idx_held_until (held_until) -- for cleanup of expired holds
) ENGINE=InnoDB;
```

-- Bookings

```
CREATE TABLE bookings (
 booking_id VARCHAR(36) PRIMARY KEY,
 user_id VARCHAR(36) NOT NULL,
 show_id VARCHAR(36) NOT NULL,
 seats JSON NOT NULL, -- ["A1", "A4", "A5"]
 seat_count SMALLINT NOT NULL,
 total_amount DECIMAL(10,2) NOT NULL,
 status ENUM('pending','confirmed','cancelled','refunded') NOT NULL,
 hold_token VARCHAR(64),
 payment_id VARCHAR(36),
 booked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 cancelled_at TIMESTAMP NULL,
```

```
INDEX idx_user (user_id),
INDEX idx_show (show_id),
INDEX idx_status_time (status, booked_at)
) ENGINE=InnoDB;

-- Payments with idempotency
CREATE TABLE payments (
 payment_id VARCHAR(36) PRIMARY KEY,
 booking_id VARCHAR(36) NOT NULL,
 user_id VARCHAR(36) NOT NULL,
 amount DECIMAL(10,2) NOT NULL,
 currency CHAR(3) DEFAULT 'INR',
 method ENUM('upi','card','netbanking','wallet') NOT NULL,
 status ENUM('initiated','processing','success','failed','refunded') NOT NULL,
 gateway_ref VARCHAR(128),
 idempotency_key VARCHAR(64) UNIQUE NOT NULL,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

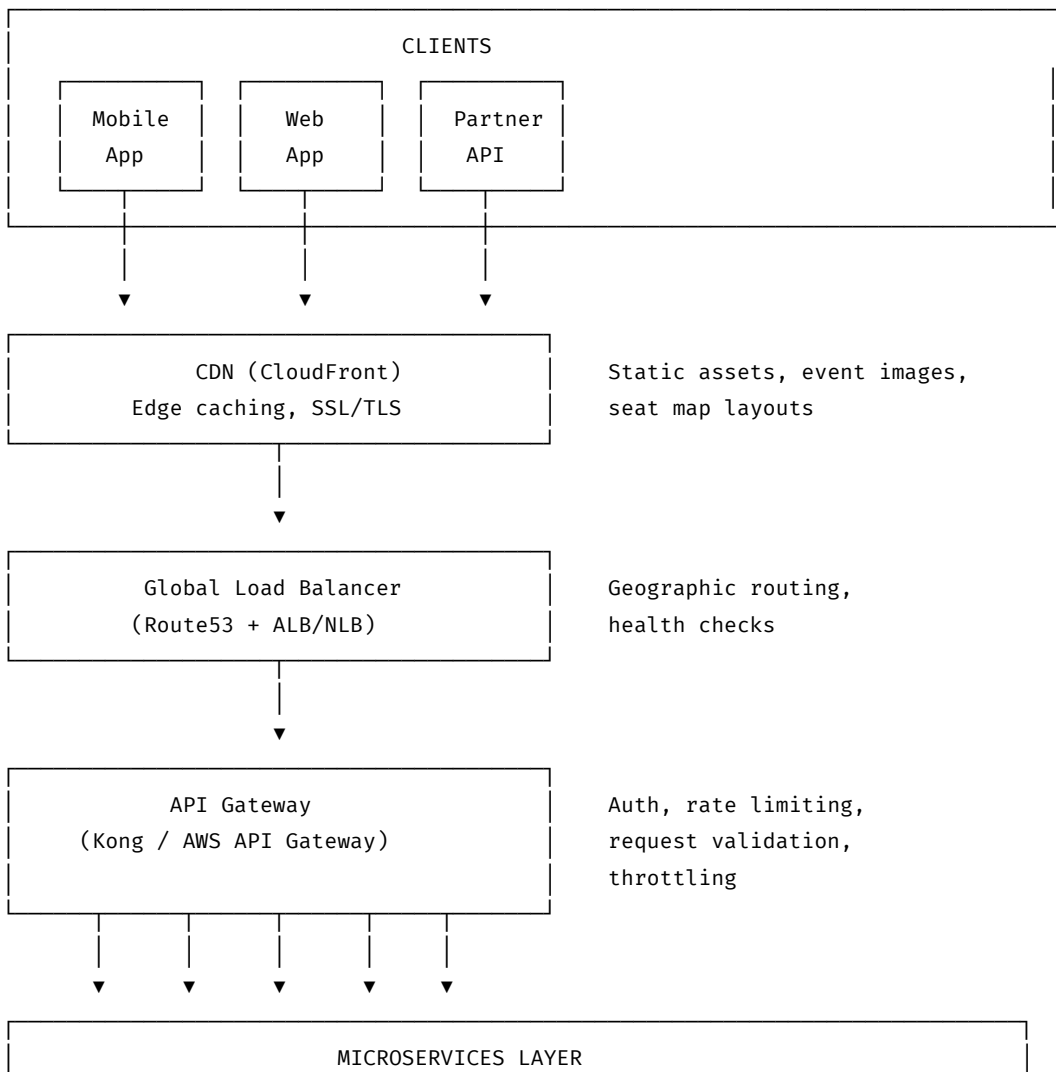
 INDEX idx_booking (booking_id),
 INDEX idx_idempotency (idempotency_key)
) ENGINE=InnoDB;
```

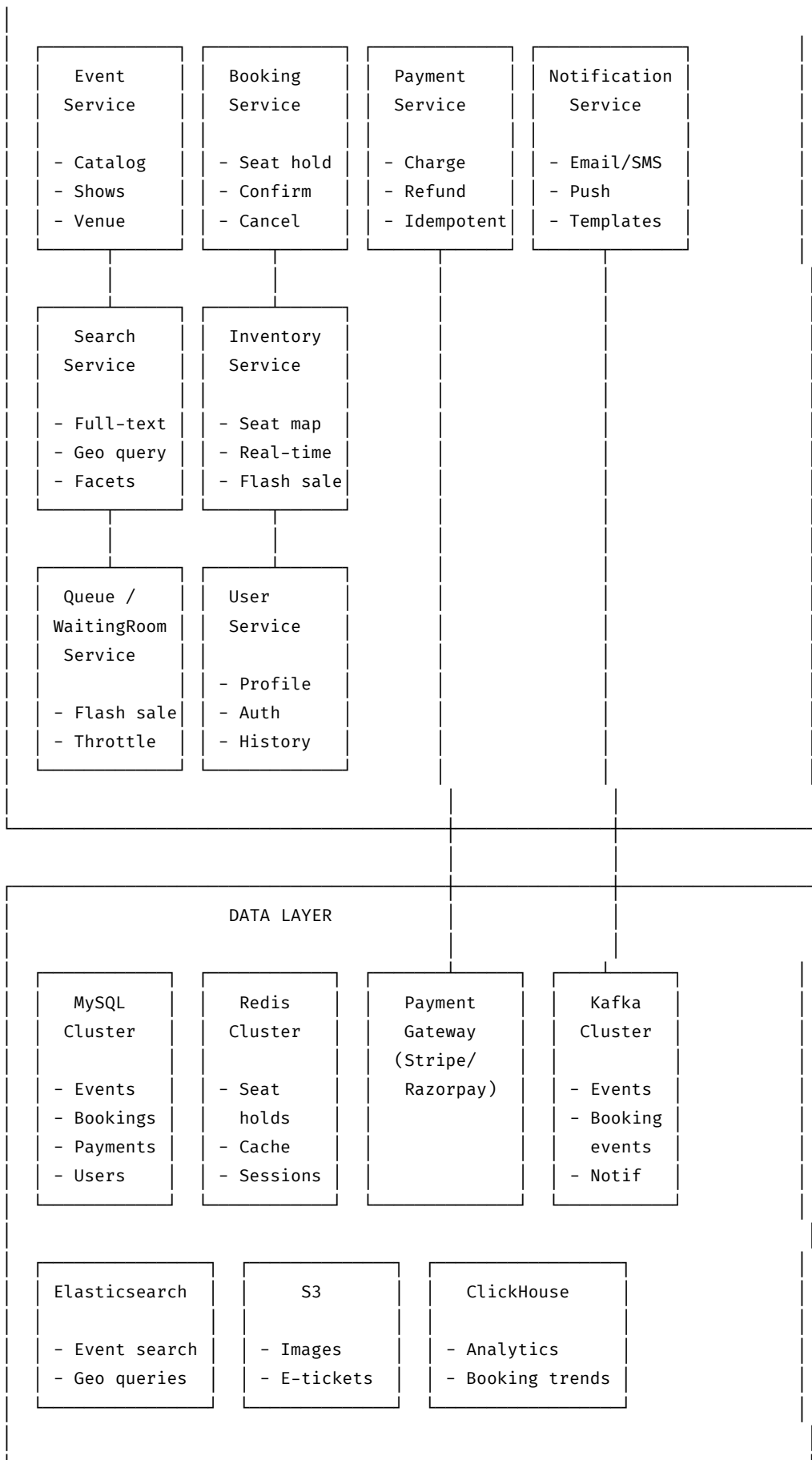
---

## 12.7 7. High-Level Architecture

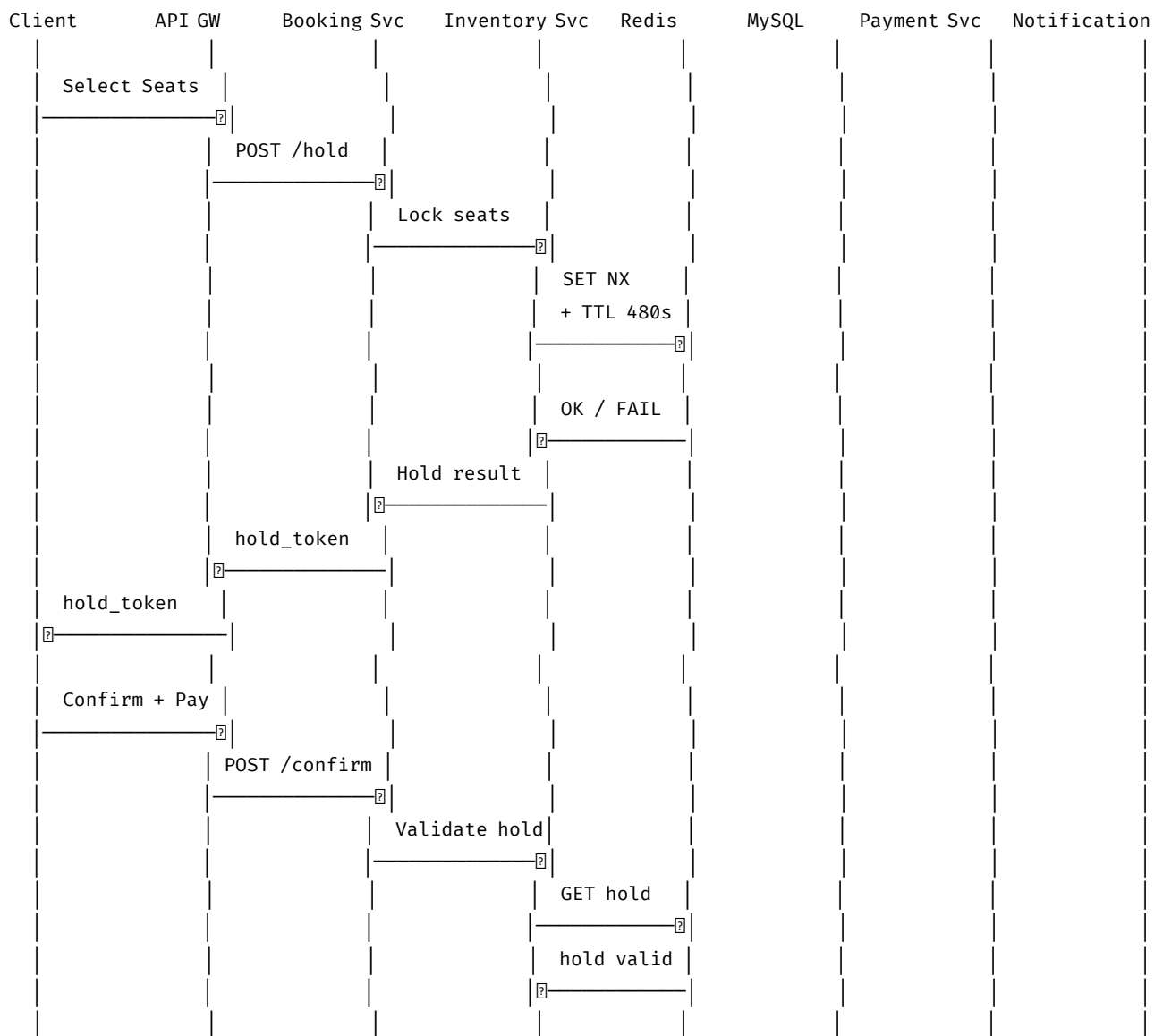


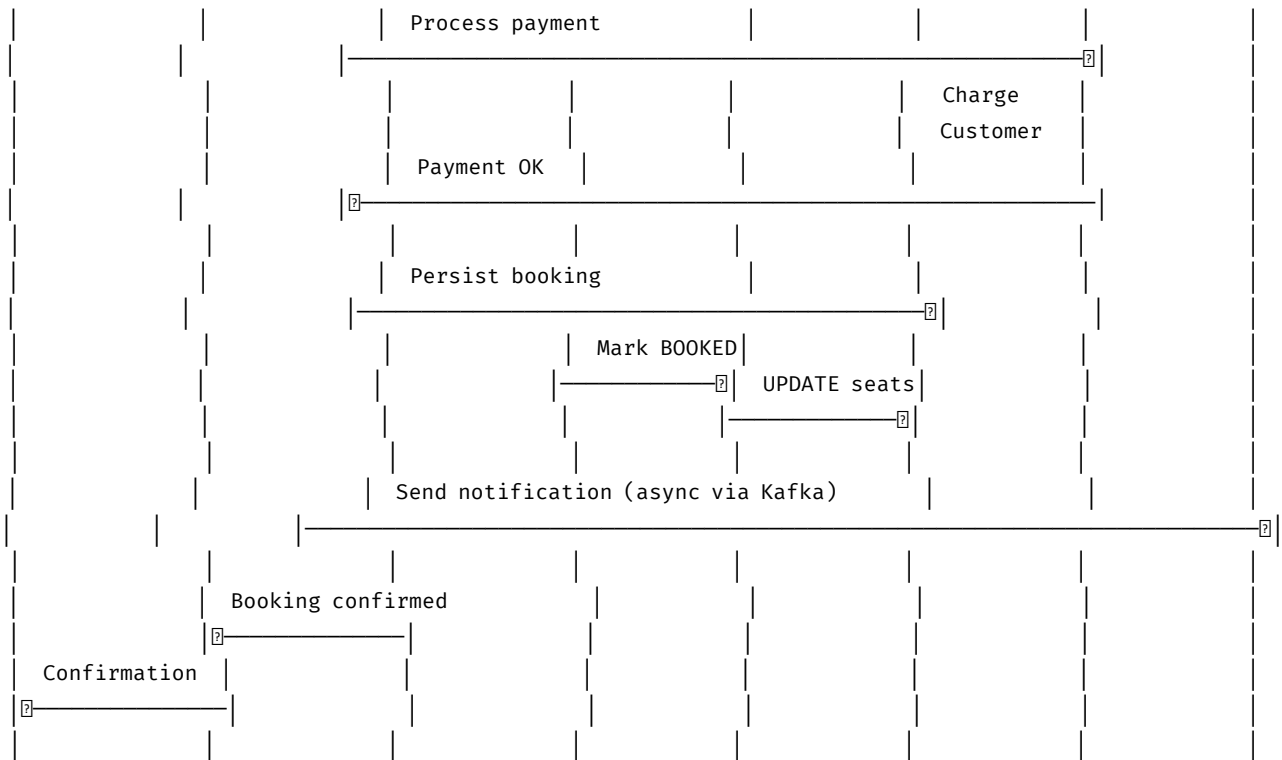
### 12.7.1 7.1 System Architecture Diagram





### 12.7.2 7.2 Booking Flow – Sequence Diagram





### 12.7.3 7.3 Component Breakdown

- **API Gateway:** Auth, rate limiting, request routing, SSL termination; Scaling Model: Horizontal, stateless
- **Event Service:** CRUD for events, venues, shows; schedule management; Scaling Model: Horizontal, read replicas
- **Booking Service:** Orchestrate hold → confirm → cancel flows; Scaling Model: Horizontal, stateless
- **Inventory Service:** Seat availability, holds, real-time updates; Scaling Model: Horizontal, Redis-backed
- **Payment Service:** Gateway integration, idempotent charges, refunds; Scaling Model: Horizontal, stateless
- **Search Service:** Full-text search, geo queries, facets, autocomplete; Scaling Model: Elastic-search cluster
- **Notification Svc:** Email, SMS, push via Kafka consumers; Scaling Model: Horizontal, async
- **User Service:** Profile, auth, booking history; Scaling Model: Horizontal, read replicas
- **Queue/WaitRoom:** Flash sale traffic management, fair ordering; Scaling Model: Horizontal, Redis-backed

## 12.8 8. Deep Dive: Core Components

### 12.8.1 8.1 Seat Inventory and Locking

This is the **single most critical component** of the system. Every seat must have exactly one owner – zero double bookings, even under extreme concurrency.

### 12.8.1.1 Locking Strategy Comparison

| Strategy                                    | Details                                                                                                                                                                                                                                 |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pessimistic Locking<br>(DB-level row locks) | <p>SELECT ... FOR UPDATE on seat rows</p> <p>Pros: Simple, strong guarantees</p> <p>Cons: Blocks concurrent readers, lock contention at scale, DB becomes bottleneck</p> <p>Use: Low-traffic events</p>                                 |
| Optimistic Locking<br>(Version-based)       | <p>UPDATE ... WHERE version = X; check affected rows</p> <p>Pros: No blocking, better throughput</p> <p>Cons: Retry storms under high contention</p> <p>Use: Medium-traffic events</p>                                                  |
| Distributed Lock<br>(Redis-based)           | <p>Redis SET NX with TTL</p> <p>Pros: Sub-ms latency, automatic TTL expiry, no DB load for holds, horizontally scalable</p> <p>Cons: Redis failure risks, need dual-write to DB</p> <p>Use: Flash sales, high-traffic events — BEST</p> |

### 12.8.1.2 Redis-Based Seat Hold Implementation

Key Pattern: seat\_hold:{show\_id}:{seat\_id}

Value: { "user\_id": "usr\_456", "hold\_token": "hld\_abc", "held\_at": "..." }

TTL: 480 seconds (8 minutes)

```
-- Hold a seat (atomic operation)
SET seat_hold:shw_xyz789:A1 '{"user_id":"usr_456","hold_token":"hld_abc"}' NX EX 480

-- Result: "OK" → seat successfully held
-- Result: (nil) → seat already held by another user, return 409

-- Release a seat (on timeout or cancellation)
DEL seat_hold:shw_xyz789:A1

-- Check seat availability for entire show (pipeline)
MGET seat_hold:shw_xyz789:A1 seat_hold:shw_xyz789:A2 ... seat_hold:shw_xyz789:A250
-- Returns: [nil, nil, "...", nil, "...", ...]
-- nil = available, non-nil = held/booked
```

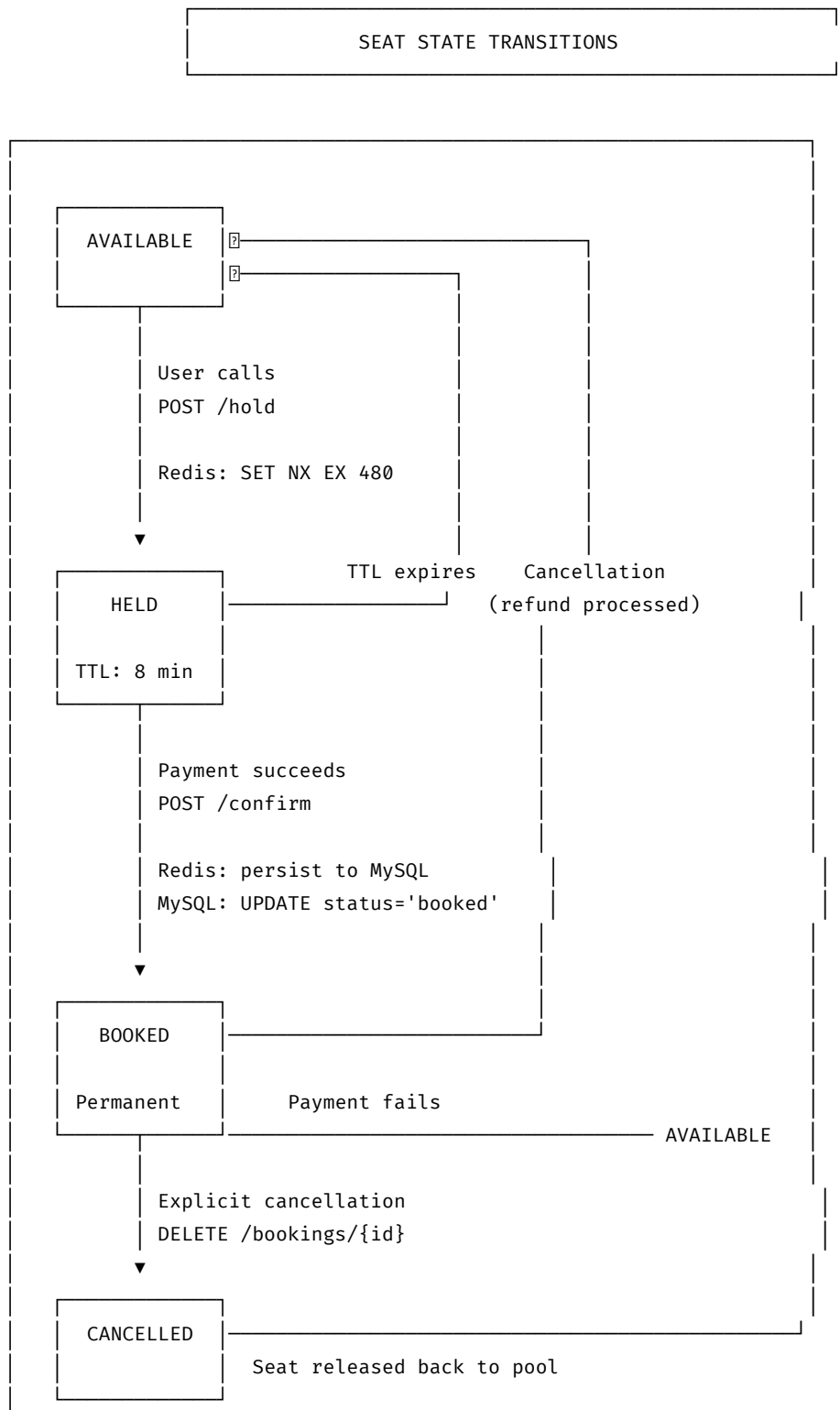
### 12.8.1.3 Multi-Seat Atomic Hold (Lua Script)

When a user selects multiple seats, all must be held atomically – either all succeed or none:

```
-- Redis Lua script for atomic multi-seat hold
-- KEYS: seat keys to hold
-- ARGV[1]: hold value (JSON), ARGV[2]: TTL in seconds

local held = {}
for i, key in ipairs(KEYS) do
 local result = redis.call('SET', key, ARGV[1], 'NX', 'EX', tonumber(ARGV[2]))
 if result then
 table.insert(held, key)
 else
 -- Rollback: release all seats held in this batch
 for _, held_key in ipairs(held) do
 redis.call('DEL', held_key)
 end
 -- Return the conflicting seat
 return { 'CONFLICT', key }
 end
end
return { 'OK', unpack(held) }
```

### 12.8.1.4 Seat State Machine (Detailed)



Transition Guards:

AVAILABLE → HELD: Only if Redis SET NX succeeds (atomic)  
 HELD → BOOKED: Only if hold\_token matches AND not expired  
 HELD → AVAILABLE: Automatic via Redis TTL expiry  
 BOOKED → CANCELLED: Only by hold owner OR admin  
 BOOKED → AVAILABLE: Via cancellation + refund flow

### 12.8.1.5 Race Condition Handling

**Scenario:** 1000 users click seat “A1” at the exact same instant.

User 1 — Redis SET seat\_hold:shw\_x:A1 NX EX 480 — "OK" ← WINS  
 User 2 — Redis SET seat\_hold:shw\_x:A1 NX EX 480 — (nil) ← 409 Conflict  
 User 3 — Redis SET seat\_hold:shw\_x:A1 NX EX 480 — (nil) ← 409 Conflict  
 ...  
 User 1000 — Redis SET seat\_hold:shw\_x:A1 NX EX 480 — (nil) ← 409 Conflict

Redis SET NX is **single-threaded and atomic** – only one writer wins. There is no race condition possible at the Redis level. The losing users receive an immediate 409 and can select alternative seats.

### 12.8.1.6 Dual-Write Consistency (Redis + MySQL)

#### DUAL-WRITE STRATEGY

Phase 1: HOLD (Redis is source of truth)

1. SET NX in Redis (atomic, with TTL)
2. Async: write hold record to MySQL (for audit)
3. If Redis fails: no hold, no MySQL write
4. If MySQL async fails: hold still valid in Redis, retry later

Phase 2: CONFIRM (MySQL becomes source of truth)

1. Validate hold in Redis (still active?)
2. Process payment
3. BEGIN TRANSACTION in MySQL:
  - UPDATE show\_seats SET status='booked' WHERE show\_id=? AND seat\_id IN (?) AND status='held'
  - INSERT INTO bookings ( ... )
  - INSERT INTO payments ( ... )
4. COMMIT
5. Update Redis: remove hold key, set permanent booked flag
6. If MySQL COMMIT fails: release Redis hold, refund payment

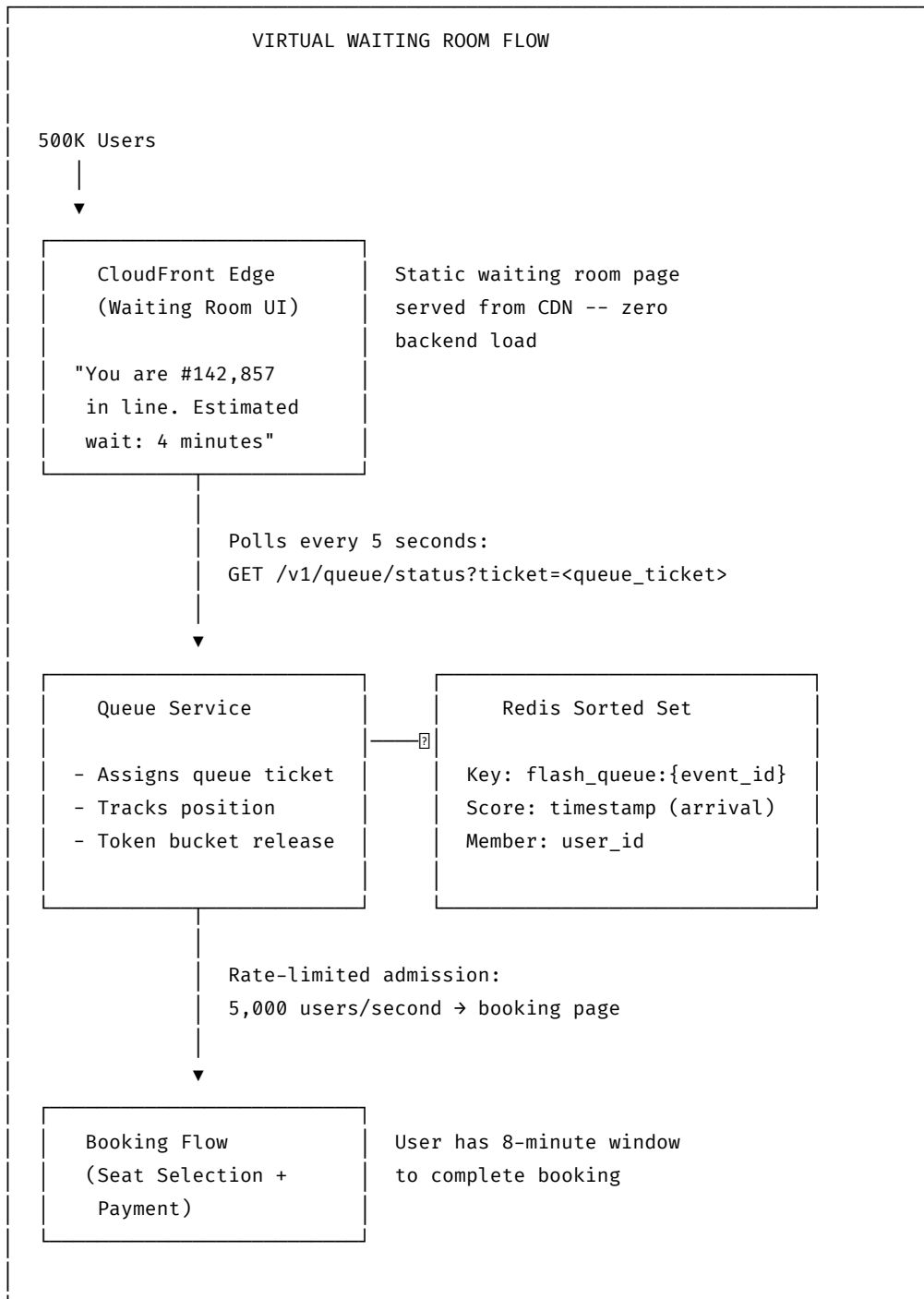
Invariant: After confirmation, MySQL is the permanent record.  
 Redis can be rebuilt from MySQL state on cold start.



## 12.8.2 8.2 Flash Sale / High-Demand Event Handling

Flash sales represent the hardest scaling challenge: 500K users competing for 50K seats in seconds. Naive first-come-first-served would overwhelm the system.

### 12.8.2.1 Virtual Waiting Room Architecture



### 12.8.2.2 Token Bucket for Controlled Admission

| TOKEN BUCKET ADMISSION CONTROL                                                                                                                                                                                                                                                      |                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Configuration for a 50K-seat event:                                                                                                                                                                                                                                                 |                                                    |
| Bucket capacity:                                                                                                                                                                                                                                                                    | 5,000 tokens                                       |
| Refill rate:                                                                                                                                                                                                                                                                        | 5,000 tokens/second                                |
| Expected fill time:                                                                                                                                                                                                                                                                 | 50K / 5K = 10 seconds                              |
| Buffer for drop-offs:                                                                                                                                                                                                                                                               | 2x admission = 100K users admitted                 |
| Timeline:                                                                                                                                                                                                                                                                           |                                                    |
| T+0s:                                                                                                                                                                                                                                                                               | Sale opens. 500K users land on waiting room page.  |
| T+0-1s:                                                                                                                                                                                                                                                                             | First 5,000 users admitted to booking page.        |
| T+1-2s:                                                                                                                                                                                                                                                                             | Next 5,000 admitted. First batch selecting seats.  |
| T+10s:                                                                                                                                                                                                                                                                              | 50,000 users admitted. ~30K seats likely held.     |
| T+20s:                                                                                                                                                                                                                                                                              | 100K total admitted. Most seats held or booked.    |
| T+30s:                                                                                                                                                                                                                                                                              | Remaining queue users see "event likely sold out." |
| T+8min:                                                                                                                                                                                                                                                                             | Expired holds release. Waitlisted users notified.  |
| Benefits:                                                                                                                                                                                                                                                                           |                                                    |
| <ul style="list-style-type: none"> <li>- Backend never sees more than ~5K concurrent booking requests</li> <li>- Predictable load regardless of demand</li> <li>- Fair ordering (FIFO by arrival time)</li> <li>- Graceful degradation (waiting room is static CDN page)</li> </ul> |                                                    |

### 12.8.2.3 Pre-Generated Booking Tokens

For ultra-high-demand events, pre-generate a limited number of booking tokens:

Before sale opens (T-10 minutes):

1. Generate exactly 60,000 booking tokens (for 50K seats + 20% buffer)
2. Store in Redis list: LPUSH flash\_tokens:{event\_id} token\_1 token\_2 ... token\_60000

During sale:

1. Queue service admits user → RPOP flash\_tokens:{event\_id}
2. If token received → proceed to seat selection
3. If (nil) → all tokens distributed, show "sold out" immediately

Advantage: Hard cap on concurrent booking flow users. Once tokens exhausted, the system immediately stops admitting new users.

### 12.8.2.4 Overselling Prevention

#### OVERSELLING PREVENTION LAYERS

##### Layer 1: Redis SET NX

Each seat can only have one holder. Atomic guarantee.

##### Layer 2: Token budget

Pre-generated tokens limit total concurrent booking attempts to (seats + buffer).

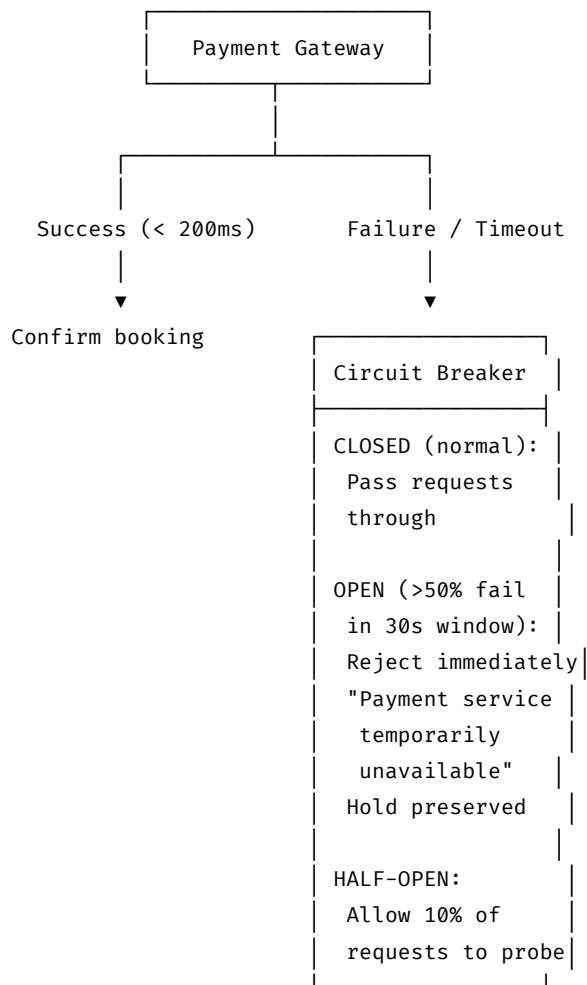
##### Layer 3: MySQL optimistic lock on confirm

```
UPDATE show_seats SET status='booked', version=version+1
WHERE seat_id=? AND status='held' AND version=?
If affected_rows=0 → seat was already booked, reject.
```

##### Layer 4: Total booked count check

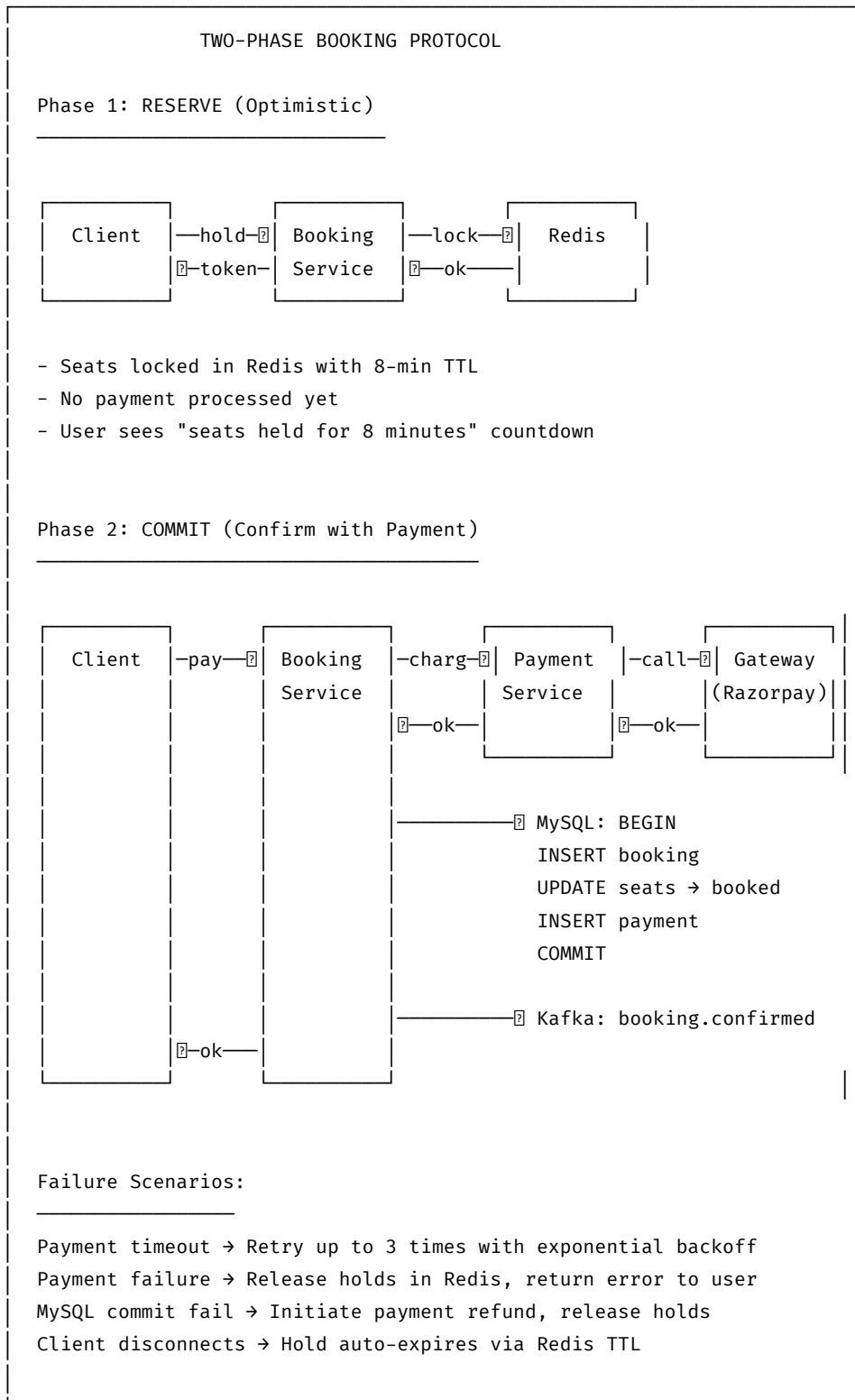
Before confirming, verify: `COUNT(booked) < total_capacity`  
Acts as a circuit breaker against any edge cases.

### 12.8.2.5 Circuit Breaker for Payment Failures



### 12.8.3 8.3 Payment Integration

#### 12.8.3.1 Two-Phase Booking Protocol



### 12.8.3.2 Payment Timeout Handling

#### PAYMENT TIMEOUT STRATEGY

Timeline for a single booking attempt:

T+0:00    Seats held (8-min TTL starts)  
T+0:05    User enters payment details  
T+0:15    Payment initiated → Gateway  
T+0:15    ... waiting for gateway response ...

Scenario A: Gateway responds in 3s

T+0:18    Payment SUCCESS → Confirm booking

Scenario B: Gateway timeout at 30s

T+0:45    Timeout → Check payment status via GET /payments/{id}  
T+0:46    Status: "processing" → Wait + retry check  
T+1:00    Status: "success" → Confirm booking  
          Status: "failed" → Release seats, show error  
          Status: "unknown" → Hold payment, retry in 30s

Scenario C: Repeated failures

T+2:00    3 attempts exhausted → Release seats  
T+2:01    Show user: "Payment failed. Seats released.  
          Please try again."

Scenario D: User abandons (closes app)

T+8:00    Redis TTL expires → Seats auto-released  
T+8:01    If payment was initiated:  
          → Async reconciliation job checks gateway  
          → If payment succeeded: auto-confirm booking  
          → If payment failed/absent: no action needed

### 12.8.3.3 Idempotent Payment Requests

POST /v1/payments/charge

Headers:

Idempotency-Key: idem\_a1b2c3d4e5f6 ← Client generates once per booking attempt

Server logic:

---

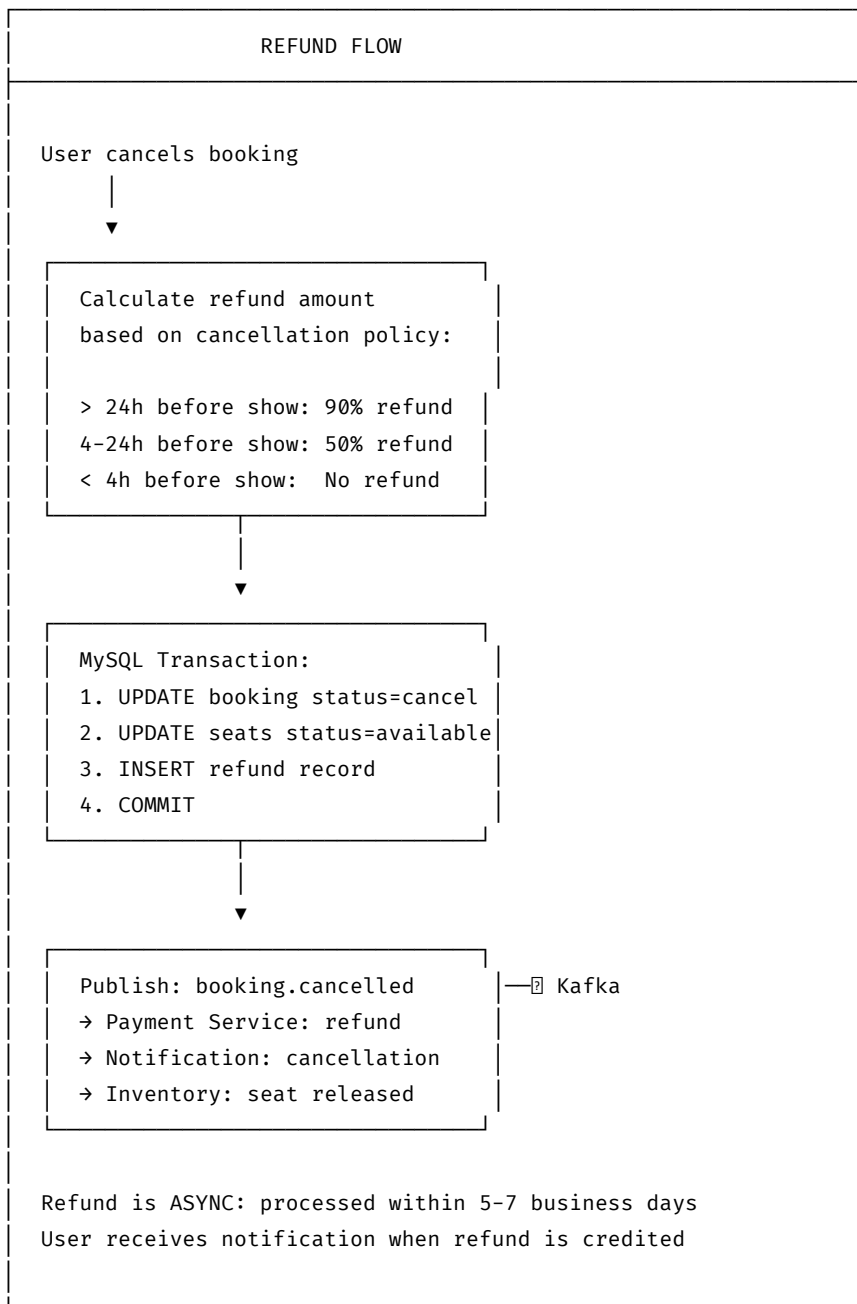
1. Check: SELECT \* FROM payments WHERE idempotency\_key = 'idem\_a1b2c3d4e5f6'
2. If found AND status = 'success': return cached response (no re-charge)
3. If found AND status = 'processing': return 202 (still processing)
4. If found AND status = 'failed': allow retry with NEW idempotency key
5. If not found: process payment, store with idempotency\_key

Why:

---

- Network retries may duplicate payment requests
- User may click "Pay" multiple times
- Gateway timeout doesn't mean payment failed
- Without idempotency: user charged 2x or 3x (catastrophic)

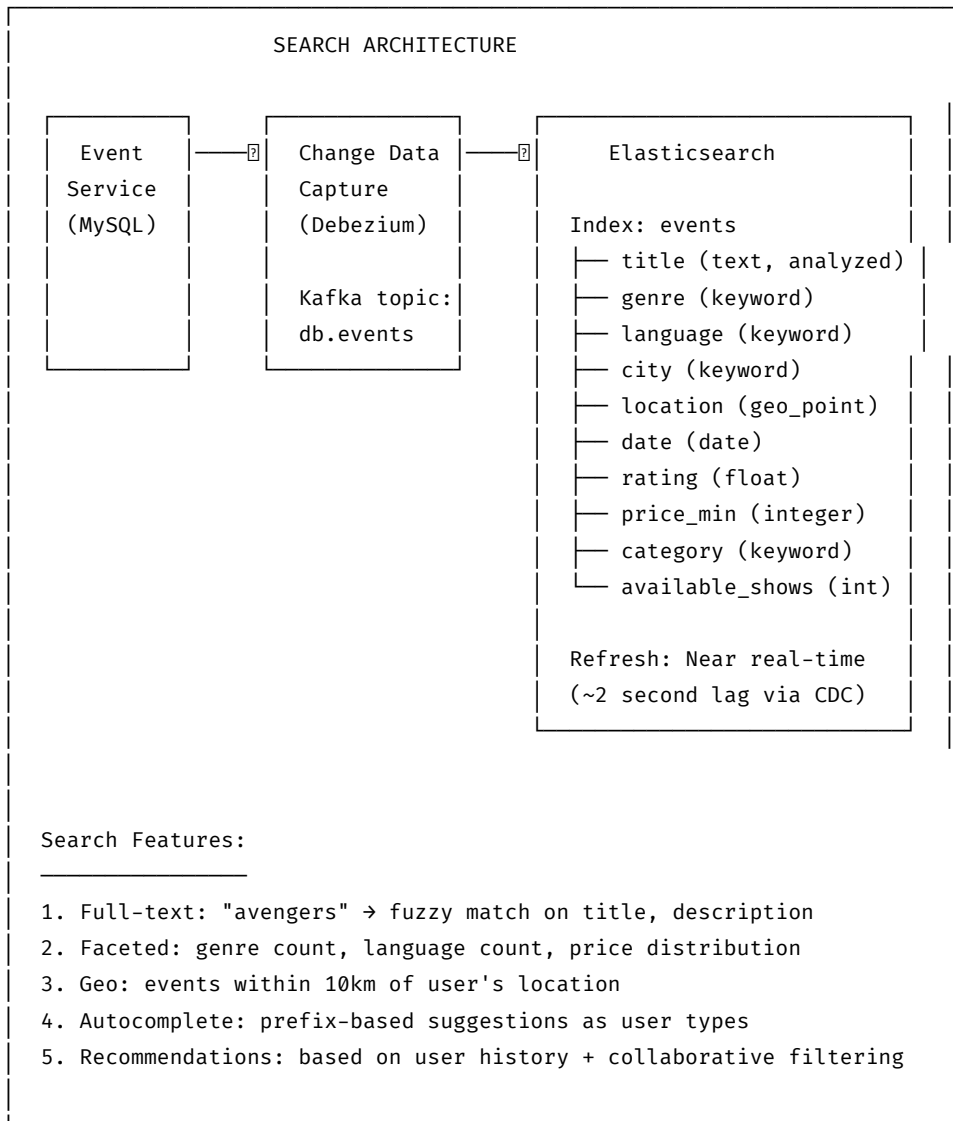
### 12.8.3.4 Refund Processing





## 12.8.4 8.4 Search and Discovery

### 12.8.4.1 Search Architecture



**12.8.4.2 Geospatial Query for Nearby Theaters**

```

{
 "query": {
 "bool": {
 "must": [
 { "match": { "title": "avengers" } },
 { "term": { "city": "mumbai" } }
],
 "filter": [
 {
 "geo_distance": {
 "distance": "10km",
 "location": { "lat": 19.076, "lon": 72.877 }
 }
 },
 {
 "range": {
 "date": { "gte": "2026-04-15", "lte": "2026-04-22" }
 }
 }
]
 }
 },
 "sort": [
 { "_score": "desc" },
 {
 "_geo_distance": {
 "location": { "lat": 19.076, "lon": 72.877 },
 "order": "asc"
 }
 }
],
 "aggs": {
 "genres": { "terms": { "field": "genre" } },
 "price_ranges": {
 "range": {
 "field": "price_min",
 "ranges": [
 { "to": 200 },
 { "from": 200, "to": 500 },
 { "from": 500 }
]
 }
 }
 }
}

```

### 12.8.4.3 Recommendation Engine

| RECOMMENDATION APPROACH                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Input signals:</p> <ul style="list-style-type: none"><li>- Booking history (genres, languages, price range)</li><li>- Browse history (events viewed but not booked)</li><li>- Location (city, preferred theaters)</li><li>- Time preferences (weekday vs weekend, afternoon vs evening)</li><li>- Similar users' behavior (collaborative filtering)</li></ul> <p>Implementation:</p> <p>Phase 1: Rule-based (trending in city + genre affinity)</p> <p>Phase 2: Collaborative filtering (users who booked X also Y)</p> <p>Phase 3: ML model (embeddings, real-time re-ranking)</p> <p>Served via:</p> <p>Pre-computed recommendations stored in Redis</p> <p>Updated every 6 hours by batch pipeline</p> <p>Real-time re-ranked based on current availability</p> |

## 12.9 9. Data Partitioning and Sharding

### 12.9.1 Sharding Strategy

| SHARDING STRATEGY         |                         |                                                                                                                  |
|---------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------|
| Entity                    | Shard Key               | Rationale                                                                                                        |
| Events / Shows            | city_id                 | Queries are almost always city-scoped<br>Keeps related data co-located<br>~50 shards for major cities            |
| Bookings                  | user_id<br>(hash-based) | User queries own booking history<br>Even distribution across shards<br>16 shards with consistent hashing         |
| Seat inventory<br>(MySQL) | show_id                 | All seats for a show on same shard<br>Enables single-shard transactions<br>Important for atomicity               |
| Payments                  | booking_id              | Co-located with booking data<br>Enables join without cross-shard                                                 |
| Redis seat holds          | show_id<br>(hash tag)   | All holds for a show on same Redis<br>node; enables Lua script atomicity<br>Key: {shw_xyz}:A1 (hash tag routing) |

### 12.9.2 Hot Partition Mitigation for Popular Events

Problem:

---

A BTS concert (show\_id: shw\_bts\_001) has 50K seats. During flash sale, ALL requests hit the SAME Redis shard (because all keys share show\_id).

Solutions:

---

#### 1. Dedicated Redis instance for flash sale events

|               |                                        |
|---------------|----------------------------------------|
| Normal events | → Redis Cluster (shared, 6 nodes)      |
| Flash events  | → Dedicated Redis (single node, 128GB) |
|               | Provisioned 1 hour before sale         |

#### 2. Seat-range partitioning within the event

|                                        |                                   |
|----------------------------------------|-----------------------------------|
| Seats A1-F20                           | → Redis shard 1 (Section: Front)  |
| Seats G1-L20                           | → Redis shard 2 (Section: Middle) |
| Seats M1-R20                           | → Redis shard 3 (Section: Back)   |
| Key pattern: {shw_bts_001:front}:A1    |                                   |
| Different hash tags → different shards |                                   |

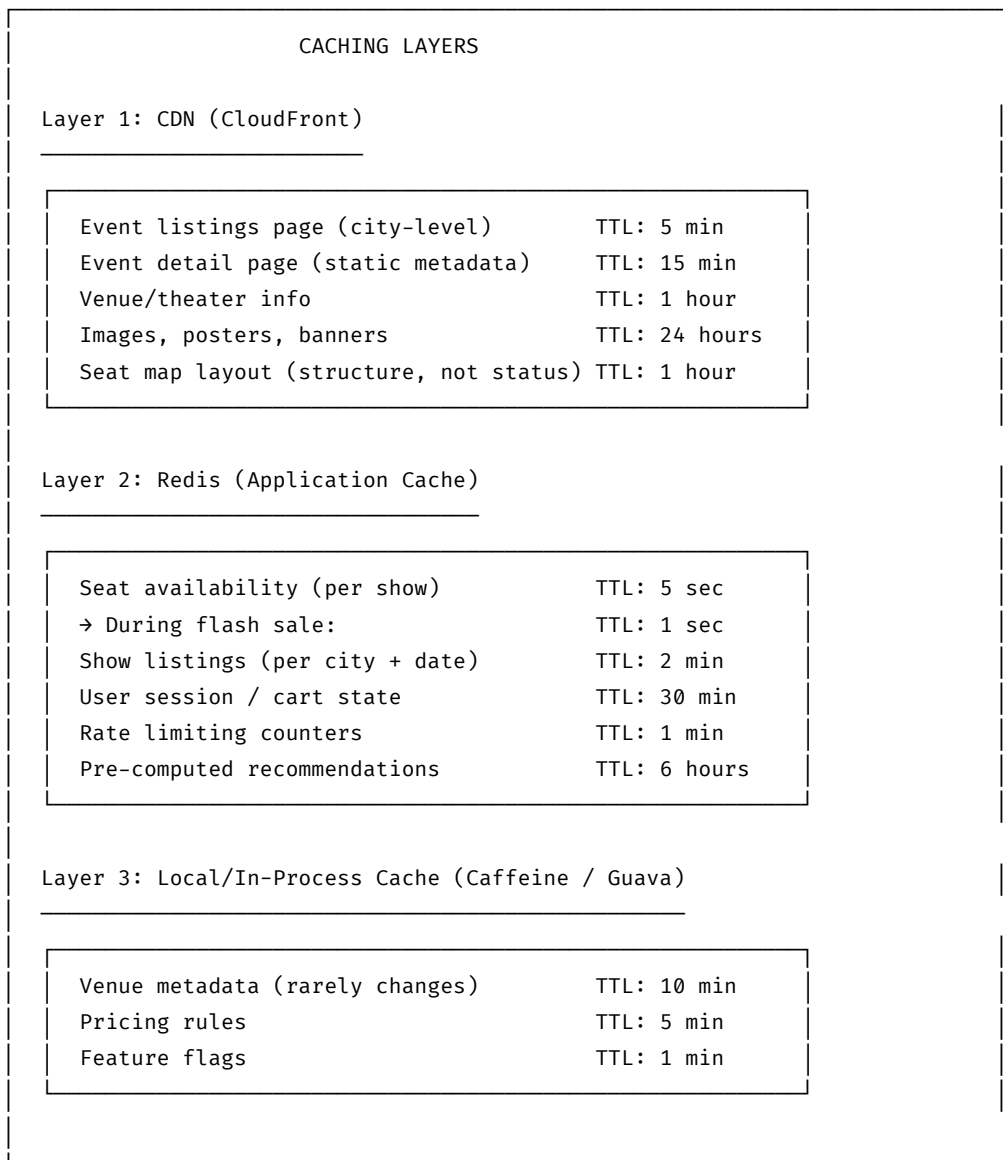
#### 3. Pre-partitioned queue with per-section admission

|                                                      |
|------------------------------------------------------|
| User selects preferred section BEFORE entering queue |
| Each section has independent queue + inventory       |
| Reduces per-shard contention by 3-5x                 |

---

## 12.10 10. Caching Strategy

### 12.10.1 Multi-Layer Cache Architecture

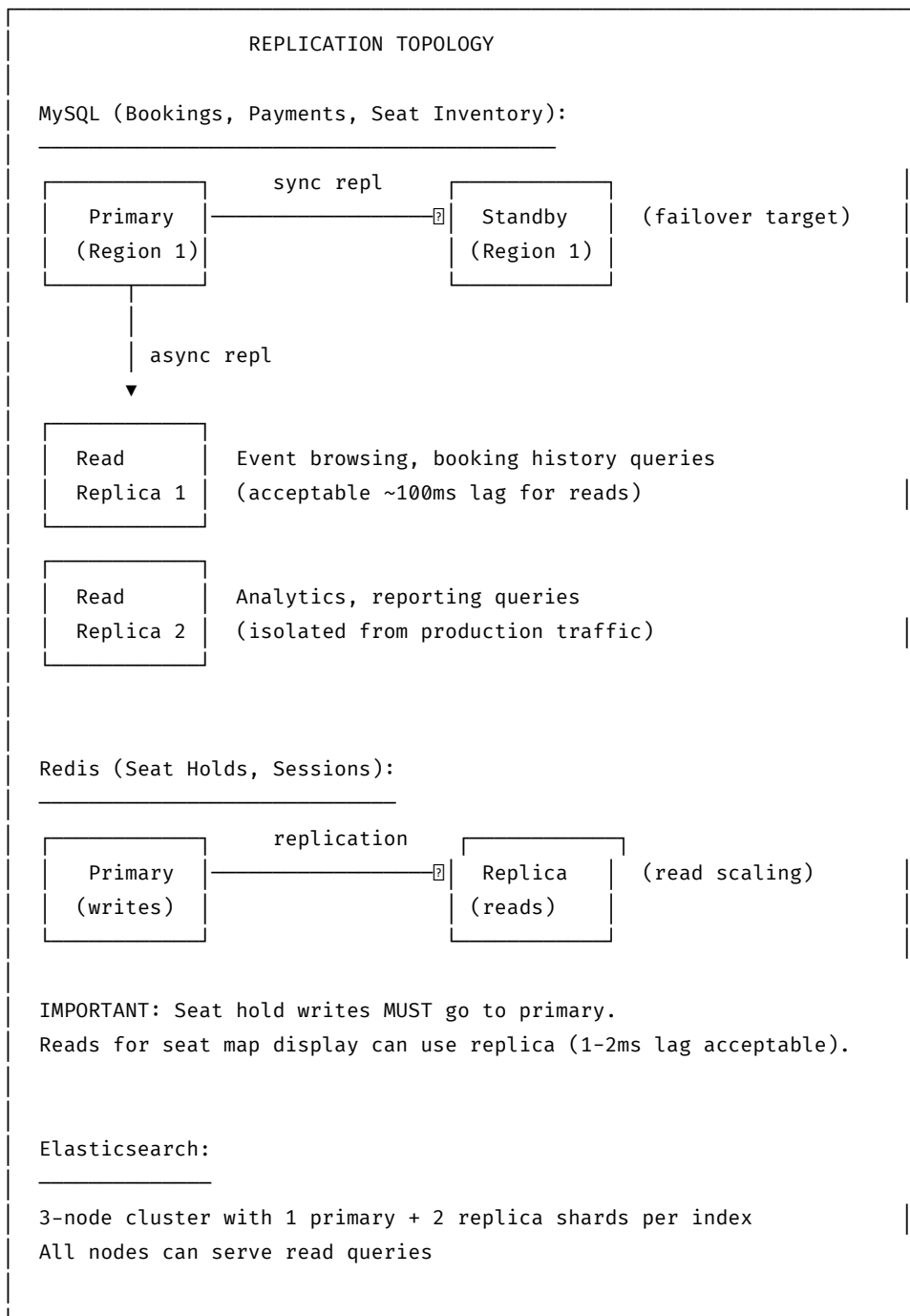


### 12.10.2 Cache Invalidation Strategy

| CACHE INVALIDATION                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Seat Availability (most critical):</p> <hr/> <ul style="list-style-type: none"> <li>- NOT cached in traditional sense; Redis IS the source of truth</li> <li>- Seat map API reads directly from Redis</li> <li>- No stale data risk (queries are always live)</li> <li>- For UI display: client-side polling every 5s (normal), every 1s (flash sale) via WebSocket/SSE</li> </ul> <p>Event Metadata (moderate staleness OK):</p> <hr/> <ul style="list-style-type: none"> <li>- Write-through: update MySQL → invalidate Redis → invalidate CDN</li> <li>- Event: Kafka topic "event.updated" → cache invalidation consumer</li> <li>- Max staleness: 5 seconds (Redis) + 5 minutes (CDN)</li> </ul> <p>Search Index:</p> <hr/> <ul style="list-style-type: none"> <li>- CDC (Debezium) captures MySQL changes → Kafka → ES consumer</li> <li>- Near-real-time: ~2 second lag</li> <li>- Bulk re-index weekly for consistency verification</li> </ul> |

## 12.11 11. Replication and Consistency

### 12.11.1 Replication Topology





**12.11.2 Consistency Guarantees by Operation**

| Operation            | Consistency     | Implementation                                   |
|----------------------|-----------------|--------------------------------------------------|
| Seat hold (SET NX)   | Linearizable    | Redis single-thread                              |
| Booking confirmation | Serializable    | MySQL transaction                                |
| Payment processing   | Serializable    | Idempotency key + txn                            |
| Seat map display     | Read-your-write | Read from Redis primary                          |
| Event browsing       | Eventual        | Read replicas + cache                            |
| Booking history      | Read-your-write | Route to primary after write, replicas otherwise |
| Search results       | Eventual (~2s)  | CDC pipeline lag                                 |
| Reviews              | Eventual (~5s)  | Async write + cache                              |

**12.12 12. Fault Tolerance and Failure Handling**

**12.12.1 Failure Modes and Recovery**

| FAILURE MODE ANALYSIS |                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Failure               | Impact and Recovery                                                                                                                                                                                                                                                                                                                                              |
| Redis primary down    | <p>Impact: Cannot hold seats (booking halted)</p> <p>Recovery:</p> <ul style="list-style-type: none"> <li>- Redis Sentinel promotes replica to primary (&lt; 30 seconds failover)</li> <li>- During failover: return 503 for hold requests</li> <li>- Existing holds preserved if replica has them</li> <li>- Rebuild holds from MySQL if data loss</li> </ul>   |
| MySQL primary down    | <p>Impact: Cannot confirm bookings</p> <p>Recovery:</p> <ul style="list-style-type: none"> <li>- Synchronous standby promoted (&lt; 60 seconds)</li> <li>- In-flight transactions rolled back</li> <li>- Held seats remain valid (Redis-based TTL)</li> <li>- Users can retry confirmation after failover</li> </ul>                                             |
| Payment gateway down  | <p>Impact: Cannot process payments</p> <p>Recovery:</p> <ul style="list-style-type: none"> <li>- Circuit breaker opens after 50% failure rate</li> <li>- Display "Payment temporarily unavailable"</li> <li>- Extend seat hold TTL by 5 minutes</li> <li>- Failover to secondary gateway if available</li> <li>- Users notified when gateway recovers</li> </ul> |

|                       |                                                                                                                                                                                                                                                                                                          |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Booking service crash | <p>Impact: In-flight bookings interrupted</p> <p>Recovery:</p> <ul style="list-style-type: none"><li>- Stateless: new instance picks up immediately</li><li>- Unconfirmed holds auto-expire via Redis TTL</li><li>- Saga-based rollback for partial confirmations</li></ul>                              |
| Kafka broker down     | <p>Impact: Notifications delayed</p> <p>Recovery:</p> <ul style="list-style-type: none"><li>- 3-broker cluster: tolerates 1 broker failure</li><li>- Notifications are async, non-blocking</li><li>- Bookings still succeed without notifications</li><li>- Messages replayed after recovery</li></ul>   |
| Elasticsearch down    | <p>Impact: Search unavailable</p> <p>Recovery:</p> <ul style="list-style-type: none"><li>- Fallback to MySQL direct queries (slower)</li><li>- Serve cached popular searches from Redis</li><li>- Homepage shows trending/popular events</li><li>- Browsing by city/date still works via MySQL</li></ul> |

### 12.12.2 Graceful Degradation Strategy

#### GRACEFUL DEGRADATION TIERS

**Tier 0 (Fully healthy):**

All features operational

**Tier 1 (Search degraded):**

Search → fallback to MySQL + cached results

Browsing still works via category/city navigation

Booking fully operational

**Tier 2 (Notifications degraded):**

Booking works; confirmation shown in UI

Email/SMS notifications queued for later delivery

User can check booking status in-app

**Tier 3 (Payment degraded):**

Browse and search work normally

Seat selection and hold work

Payment page shows "temporarily unavailable"

Hold TTL extended; user notified when payment is back

**Tier 4 (Booking degraded):**

Browse and search work (cached data)

Seat map shows "booking temporarily unavailable"

System serves read-only event information

### 12.12.3 Expired Hold Cleanup

#### HOLD EXPIRY AND CLEANUP

Primary mechanism: Redis TTL auto-expiry

```
SET seat_hold:shw_x:A1 ... EX 480
```

After 480 seconds: key auto-deleted, seat becomes available

No background job needed for Redis cleanup

MySQL cleanup (for audit trail):

Scheduled job every 1 minute:

```
UPDATE show_seats
```

```
SET status = 'available', held_by_user = NULL,
 held_until = NULL
```

```
WHERE status = 'held' AND held_until < NOW();
```

Redis keyspace notifications (optional):

Subscribe to expired key events:

```
CONFIG SET notify-keyspace-events Ex
```

```
SUBSCRIBE __keyevent@0__:expired
```

→ On expiry: update MySQL status + notify waiting users

Waiting list notification:

When held seat expires for a sold-out event:

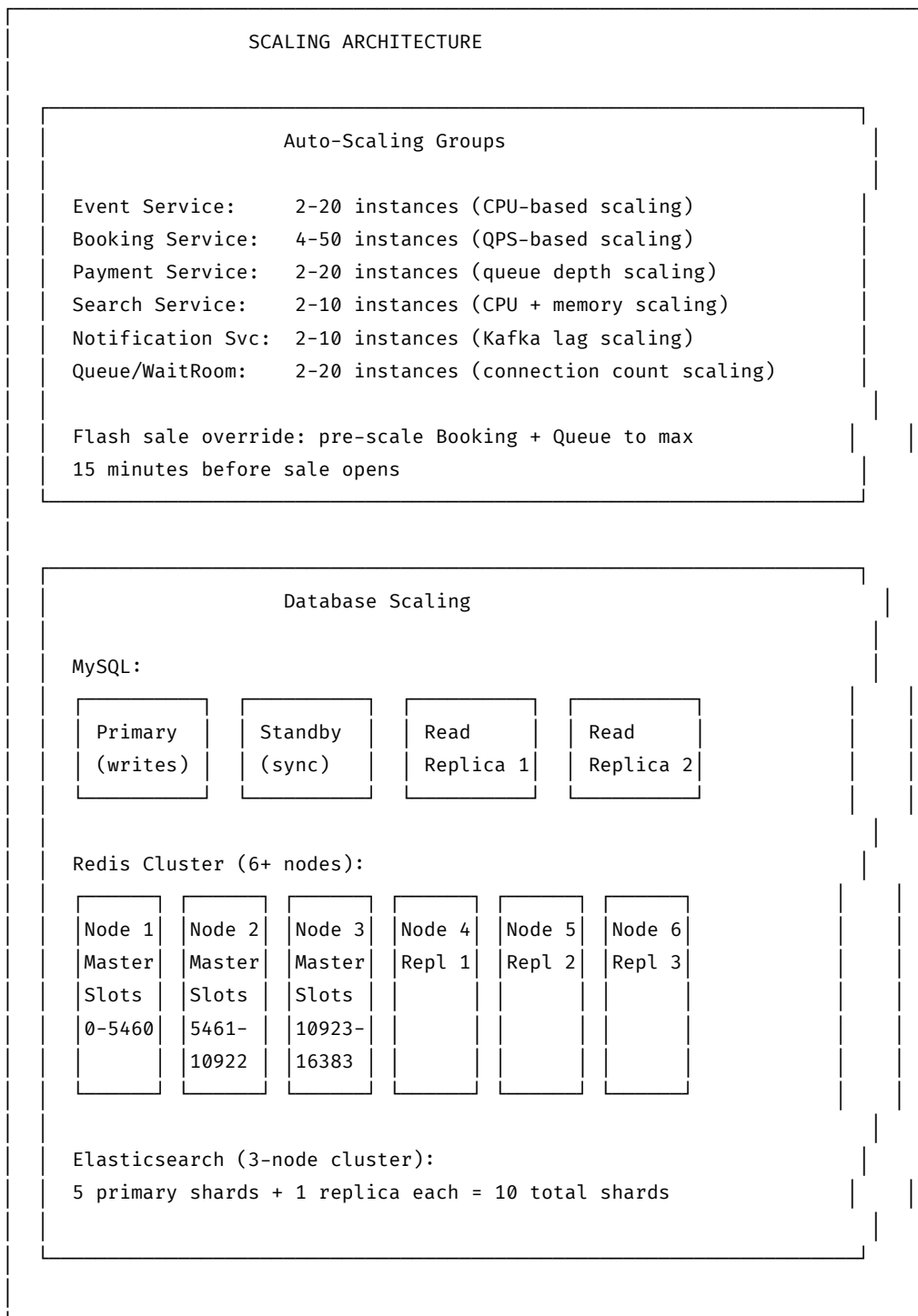
→ Check waiting list for this show

→ Send push notification: "A seat is now available!"

→ First user to hold wins

## 12.13 13. Scalability

### 12.13.1 Horizontal Scaling Architecture



### 12.13.2 Scaling Bottleneck Analysis

| Bottleneck                               | Symptom                                 | Solution                                                            |
|------------------------------------------|-----------------------------------------|---------------------------------------------------------------------|
| Single Redis shard for popular show      | High latency on hold requests, timeouts | Seat-range partitioning, dedicated instance for flash sale events   |
| MySQL write throughput during flash sale | Booking confirmation latency spikes     | Batch confirms, write buffering via queue, shard by show_id         |
| API Gateway under 500K concurrent conns  | 429 errors, connection drops            | Pre-scale, connection pooling, waiting room at CDN layer            |
| Elasticsearch under flash sale traffic   | Search latency > 1s                     | Warm cache, pre-compute popular queries, circuit breaker + fallback |
| Payment gateway rate limits hit          | Payment timeouts                        | Multiple gateways, circuit breaker, request queuing                 |

## 12.14 14. Monitoring and Observability

### 12.14.1 Key Metrics Dashboard

MONITORING DASHBOARD

BUSINESS METRICS (Grafana)

|                                                        |                |               |
|--------------------------------------------------------|----------------|---------------|
| Booking success rate                                   | Target: > 95%  | Alert: < 90%  |
| Booking completion rate<br>(hold → confirm conversion) | Target: > 70%  | Alert: < 50%  |
| Revenue per hour                                       | Baseline: ₹50L | Alert: < ₹20L |
| Seat hold expiry rate                                  | Target: < 30%  | Alert: > 50%  |
| Cancellation rate                                      | Target: < 10%  | Alert: > 20%  |
| Average booking value                                  | Track trend    |               |

SYSTEM METRICS (Prometheus + Grafana)

|                             |                             |             |
|-----------------------------|-----------------------------|-------------|
| API latency (P50/P95/P99)   | Target P99: < 500ms         |             |
| Seat hold latency (P99)     | Target: < 10ms              |             |
| Payment processing time     | Target P95: < 5s            |             |
| Redis operations/sec        | Capacity: 100K ops/sec/node |             |
| MySQL QPS (reads/writes)    | Alert: > 80% capacity       |             |
| Kafka consumer lag          | Alert: > 10K messages       |             |
| Error rate (5xx)            | Target: < 0.1%              | Alert: > 1% |
| Connection pool utilization | Alert: > 80%                |             |



#### FLASH SALE SPECIFIC

---

|                             |                               |
|-----------------------------|-------------------------------|
| Queue depth (waiting users) | Display on war-room dashboard |
| Queue wait time (P50/P95)   | Target P95: < 10 minutes      |
| Admission rate (users/sec)  | Target: 5K/sec                |
| Seats remaining             | Real-time countdown           |
| Hold/Confirm ratio          | Target: > 60%                 |
| Time to sell out            | Track for capacity planning   |

#### DISTRIBUTED TRACING (Jaeger / Zipkin)

---

Trace ID propagated across:

Client → Gateway → Booking Svc → Inventory Svc → Redis  
→ Payment Svc → Gateway (external)  
→ Notification Svc → Kafka → Email/SMS provider

SLO: End-to-end booking trace < 30 seconds

#### ALERTING (PagerDuty)

---

P1: Double booking detected, payment service down, Redis primary down

P2: Booking success rate < 90%, API P99 > 2s, seat hold expiry > 50%

P3: Search latency > 1s, notification delay > 5 min

### 12.14.2 Anomaly Detection

| ANOMALY DETECTION RULES                                                                                                                                                                                                     |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Rule 1: Double Booking Detection (CRITICAL)</p> <hr/>                                                                                                                                                                    |
| <p>Periodic check (every 30 seconds):</p> <pre>SELECT show_id, seat_id, COUNT(*) as cnt FROM bookings WHERE status = 'confirmed' GROUP BY show_id, seat_id HAVING cnt &gt; 1; If cnt &gt; 1: PAGE ON-CALL IMMEDIATELY</pre> |
| <p>Rule 2: Revenue Anomaly</p> <hr/>                                                                                                                                                                                        |
| <p>If hourly revenue drops below 40% of same-hour-last-week:<br/>Alert P2 with potential causes (payment down, traffic drop)</p>                                                                                            |
| <p>Rule 3: Bot Detection</p> <hr/>                                                                                                                                                                                          |
| <p>If single IP/user holds &gt; 20 seats across shows in 1 hour:<br/>Flag as potential bot, apply stricter rate limits</p>                                                                                                  |

## 12.15 15. Trade-offs and Design Decisions

### 12.15.1 Decision 1: Optimistic vs Pessimistic Locking

DECISION: Use Redis SET NX (distributed lock) as primary mechanism with MySQL optimistic lock as secondary safety net

Why not pessimistic (SELECT FOR UPDATE)?

- Row-level locks block ALL readers during hold period (8 min!)
- Under 50K concurrent requests, lock wait timeouts are guaranteed
- DB connection pool exhausted within seconds
- MySQL is designed for sub-second transactions, not 8-min locks

Why not pure optimistic (version-based)?

- Works well for low-medium contention
- Under flash sale (1000 users on same seat): 999 retries per round
- Retry storms amplify load instead of reducing it
- Still hits MySQL for every attempt

Why Redis SET NX + MySQL optimistic?

- SET NX: single-threaded, O(1), sub-ms, no retry needed
- Loser gets immediate (nil) → show alternative seats
- MySQL optimistic lock only during confirm (low contention)
- Best of both: speed + durability

### 12.15.2 Decision 2: Queue vs First-Come-First-Served

DECISION: Virtual waiting room with queue for flash sales  
Direct access for normal events

First-Come-First-Served (no queue):

- + Simpler architecture
- + Lower latency for first arrivals
- 500K simultaneous requests overwhelm backend
- Unfair: users with faster connections/bots win
- Thundering herd on seat selection page
- Poor UX: most users see errors or timeouts

Queue-based (chosen for flash sales):

- + Controlled backend load (5K/sec admission)
- + Fair: FIFO ordering by arrival time
- + Predictable wait time for users
- + Backend never overloaded
- + Waiting room page is static (CDN-served, zero backend cost)
- Extra component to build and maintain
- Slight delay for users (but they get a working experience)

Hybrid approach:

- Events with < 80% fill rate: direct access, no queue
- Events flagged as "high demand": queue enabled automatically
- Queue activation trigger: > 10x normal traffic in 60s window

### 12.15.3 Decision 3: Pre-Allocated vs Dynamic Seat Selection

DECISION: User-selected seats (primary) with auto-assign option

User-Selected (BookMyShow model):

- + Users see exact seat map and pick preferred seats
- + Higher satisfaction (users control their experience)
- + Premium seats can be priced higher (visible on map)
- More complex UI and more API calls
- Higher contention (everyone wants Row A center)
- Seat map must update in near-real-time

Auto-Assigned (Ticketmaster "best available"):

- + Simpler: user just picks count + section
- + Lower contention (system distributes demand)
- + Faster booking flow
- Less user control
- May assign split seats (A1 + C15 instead of A1 + A2)

Our hybrid:

- Default: user-selected with real-time seat map
- "Quick book" option: auto-assign best available N adjacent seats
- Flash sales: can default to auto-assign to reduce contention

### 12.15.4 Decision 4: Monolith vs Microservices

DECISION: Microservices with domain-based decomposition

Why not monolith?

- Booking service needs independent scaling during flash sales
- Payment service needs circuit breaker isolation
- Search service has different scaling characteristics (CPU-bound)
- Team ownership boundaries align with service boundaries
- Failure isolation: payment crash should not affect browsing

Service boundaries (Domain-Driven Design):

Event Bounded Context: Event, Venue, Show, Screen

Booking Bounded Context: Booking, Hold, SeatInventory

Payment Bounded Context: Payment, Refund

User Bounded Context: User, Auth, Preferences

Search Bounded Context: Index, Query, Recommendations

Inter-service communication:

Synchronous (gRPC): Booking → Inventory (hold seats)

Booking → Payment (charge)

Asynchronous (Kafka): Booking → Notification (confirm email)

Event → Search (index update)

Booking → Analytics (event stream)

### 12.15.5 Decision 5: Hold Duration

DECISION: 8-minute hold with dynamic adjustment

Too short (2 min):

- User can't enter payment details in time
- High hold expiry rate → frustrated users
- Especially bad for UPI/netbanking (OTP delays)

Too long (15 min):

- Seats locked unnecessarily by users who abandoned
- During flash sale: 50K seats held for 15 min blocks everyone
- Reduces effective inventory

Chosen: 8 minutes (default), dynamically adjusted:

- 
- Normal event: 8 minutes
  - High-demand event (>80% full): 5 minutes
  - Flash sale: 5 minutes with 1-min payment extension on request
  - Group booking (10+ seats): 12 minutes

## 12.16 16. Interview Deep-Dive Questions

### 12.16.1 Q1: How do you prevent double booking?

**Answer:** We use a multi-layer defense:

1. **Redis SET NX (primary guard):** The `SET seat_hold:{show_id}:{seat_id} NX EX 480` command is atomic and single-threaded. Only one caller can succeed for a given seat. All others receive `(nil)` immediately. This is the first and most critical barrier.
2. **MySQL optimistic locking (secondary guard):** During confirmation, we execute:
 

```
UPDATE show_seats SET status='booked', version=version+1
WHERE show_id=? AND seat_id=? AND status='held' AND version=?
```

If `affected_rows = 0`, the seat was already booked (version mismatch) – reject.
3. **Unique constraint:** `UNIQUE(show_id, seat_id, status='booked')` ensures the database itself rejects duplicate bookings even if application logic has a bug.
4. **Post-hoc validation:** Background job runs every 30 seconds checking for duplicate `(show_id, seat_id)` combinations with `status='confirmed'`. Any duplicates trigger P1 alert.

### 12.16.2 Q2: How do you handle 500K users trying to book 50K seats simultaneously?

**Answer:** The virtual waiting room is key:

1. **CDN-served waiting room:** Users land on a static HTML page served entirely by CloudFront. Zero backend load for the initial stampede.

2. **Queue assignment:** Users poll `GET /queue/status` which reads their position from a Redis sorted set (ZRANK). Server load: simple Redis reads at  $\sim 500K/5s = 100K$  reads/sec – well within Redis capacity.
  3. **Controlled admission:** Token bucket releases 5,000 users/second to the actual booking page. Backend sees max 5K concurrent booking requests – manageable.
  4. **Pre-generated tokens:** For extra safety, we pre-generate 60K tokens. Once exhausted, the system immediately shows “sold out” instead of processing more requests.
  5. **Pre-scaling:** 15 minutes before sale, we pre-scale all services to peak capacity. No auto-scaling delay during the critical window.
- 

### 12.16.3 Q3: What happens if payment fails after seats are held?

**Answer:** The seats remain held until either:

1. **User retries payment** (within hold TTL): The `hold_token` remains valid. User can retry with a different payment method. We generate a new `idempotency_key` for each retry to prevent double-charging.
  2. **Hold expires (TTL):** After 8 minutes, Redis auto-deletes the hold keys. The MySQL cleanup job marks seats as `available`. Seats return to the pool for other users.
  3. **Explicit release:** If the user clicks “Cancel” or navigates away, we proactively `DEL` the hold keys in Redis and update MySQL immediately, freeing seats faster.
  4. **Payment in limbo (timeout):** If the payment gateway timed out but actually processed the charge, a reconciliation job checks pending payments every 5 minutes. If the gateway confirms success, we auto-confirm the booking (if hold is still valid) or initiate a refund (if hold expired).
- 

### 12.16.4 Q4: How do you handle seat selection for group bookings (10+ seats)?

**Answer:**

1. **Atomic multi-seat hold:** We use the Redis Lua script that attempts to `SET NX` all requested seats atomically. If any single seat fails, all previously acquired seats in the batch are released. This ensures the group gets all seats or none.
  2. **Adjacent seat algorithm:** The system finds contiguous blocks of available seats:
    - Query available seats for the row, find consecutive sequences  $\geq$  group size
    - Prefer center-of-row positions (better view)
    - If no single row has enough, split across 2 adjacent rows
  3. **Extended hold:** Group bookings get 12-minute hold instead of 8 minutes, because payment coordination may take longer (e.g., one person paying for 15 friends).
  4. **Concurrency safety:** The Lua script runs atomically on a single Redis thread, so there is no risk of a partial group hold.
- 

### 12.16.5 Q5: How would you design dynamic pricing?

**Answer:**



**Dynamic Pricing Formula:**

$$\text{final\_price} = \text{base\_price} \times \text{demand\_multiplier} \times \text{time\_multiplier}$$
**demand\_multiplier:**

fill\_rate < 30%: 0.8 (discount to drive sales)  
 fill\_rate 30-70%: 1.0 (base price)  
 fill\_rate 70-90%: 1.3 (high demand premium)  
 fill\_rate > 90%: 1.5 (near sold-out premium)

**time\_multiplier:**

> 7 days before: 0.9 (early bird discount)  
 1-7 days before: 1.0 (standard)  
 < 24h before: 1.2 (last-minute surge)  
 < 4h before: 0.7 (fire sale for empty seats)

**Implementation:**

- Price computed at seat-map load time (not stored statically)
- Locked at hold time (price stored in hold\_token)
- Price honored during confirmation (no bait-and-switch)
- Pricing rules stored in config service with 5-min cache

### 12.16.6 Q6: How do you handle the thundering herd problem when a hold expires during a flash sale?

**Answer:** When a popular seat's hold expires during a flash sale, many users may be watching that seat on their seat map and try to grab it simultaneously.

1. **Server-side jitter:** When sending seat availability updates via WebSocket, each client receives the update with a random jitter of 0-500ms. This spreads the thundering herd over half a second.
2. **Client-side exponential backoff:** If a hold attempt returns 409, the client waits  $100\text{ms} * \text{random}(1, 2^{\text{attempt}})$  before retrying.
3. **Redis SET NX:** Even if 1000 users try simultaneously, Redis handles this with zero contention – exactly one wins, 999 get (nil) in microseconds. The thundering herd is a non-issue at the Redis level.

### 12.16.7 Q7: How do you ensure exactly-once payment processing?

**Answer:** Exactly-once semantics in payment through idempotency keys:

1. Client generates a UUID `idempotency_key` per booking attempt.
2. Server stores (`idempotency_key`, `status`, `response`) in the payments table.
3. On retry: if key exists with `status=success`, return the cached response without re-charging.
4. On retry: if key exists with `status=processing`, return 202 (try again later).
5. The payment gateway itself uses our `idempotency_key` in its API call, so even if we send the same request twice, the gateway charges only once.

6. **Critical:** the `INSERT payment` and `UPDATE booking status=confirmed` happen in the same MySQL transaction. Either both succeed or neither does.
- 

### 12.16.8 Q8: How do you handle multi-region deployment?

**Answer:**

- **Event catalog:** Replicated across regions via MySQL async replication. Users in Mumbai see Mumbai events; the data is region-local.
  - **Seat inventory:** Seats for a given show exist on a single Redis primary in the show's home region. All booking requests for that show route to the home region, even from remote users. This ensures strong consistency without cross-region coordination.
  - **User data:** Partitioned by home region but accessible cross-region with slight latency.
  - **Search:** Region-local Elasticsearch clusters indexed with region-relevant events only.
  - **Latency:** Users booking events in their own city see <50ms latency. Users booking events in another city (rare) see ~200ms additional latency due to cross-region routing.
- 

### 12.16.9 Q9: How do you handle partial failures in the booking confirmation flow?

**Answer:** We use a saga pattern with compensating transactions:

|                                                |                                                                                                                 |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Step 1: Validate hold                          | → Success → Continue                                                                                            |
|                                                | → Failure → Return error (hold expired)                                                                         |
| Step 2: Charge payment                         | → Success → Continue                                                                                            |
|                                                | → Failure → Release hold, return error                                                                          |
| Step 3: Persist booking<br>(MySQL transaction) | → Success → Continue                                                                                            |
|                                                | → Failure → Refund payment, release hold                                                                        |
| Step 4: Update seat status<br>(Redis → MySQL)  | → Success → Continue                                                                                            |
|                                                | → Failure → Reconciliation job fixes later<br>(booking is confirmed, seats<br>will be marked booked eventually) |
| Step 5: Send notification<br>(async via Kafka) | → Success → Done                                                                                                |
|                                                | → Failure → Retry via dead-letter queue<br>(booking is confirmed regardless)                                    |

Each step has a compensating action. Steps 4-5 are eventually consistent and non-blocking.

---

### 12.16.10 Q10: How would you handle a scenario where Redis loses data (e.g., server crash without persistence)?

**Answer:**

1. **Prevention:** Redis AOF persistence with `appendfsync everysec`. Maximum 1 second of data loss. For critical flash sales, use `appendfsync always` (slight performance hit).
  2. **Detection:** Monitoring alert when Redis restarts. Health check detects empty key space.
-

**3. Recovery:**

- Query MySQL for all seats with `status='held' and held_until > NOW()`
  - Re-create Redis hold keys with `TTL = held_until - NOW()`
  - Recovery time: ~5 seconds for a typical show (1000 seats)
4. **During recovery:** Hold requests return 503. The 8-minute hold window provides a buffer – users are unlikely to notice a 5-second recovery window.
  5. **Worst case (total Redis loss during flash sale):** Switch to MySQL pessimistic locking (`SELECT FOR UPDATE`) as fallback. Higher latency but correct behavior. Feature flag controls this failover.
- 

**12.16.11 Q11: How do you prevent bots and scalpers from hoarding tickets?****Answer:**

1. **CAPTCHA:** Required before entering the waiting room queue for high-demand events.
  2. **Rate limiting:** Max 5 hold requests per minute per user. Max 1 queue entry per event per user.
  3. **Device fingerprinting:** Detect multiple accounts from the same device/browser.
  4. **IP-based limits:** Max 10 bookings per IP per hour (accommodates office networks).
  5. **Purchase limits:** Max 6 tickets per user per event (policy-enforced).
  6. **Behavioral analysis:** ML model scores suspicious patterns (instant selection, no browsing, API-only access) and adds CAPTCHA challenges.
  7. **Phone verification:** Require verified phone number before booking high-demand events.
- 

**12.16.12 Q12: How do you design the seat map for real-time updates?****Answer:**

- **Initial load:** `GET /shows/{id}/seats` returns full seat map (~50KB for 1000 seats).
  - **Real-time updates:** WebSocket connection per user viewing a seat map:  
`ws://api.example.com/v1/shows/{show_id}/seats/live`
  - **Server broadcasts deltas:** When a seat status changes, the server publishes only the diff:  

```
{ "type": "seat_update", "seats": [{ "id": "A1", "status": "held" }] }
```
  - **Fan-out:** Redis Pub/Sub per `show_id`. Each WebSocket server subscribes to the show's channel and pushes updates to connected clients.
  - **Scaling concern:** If 500K users view the same seat map, fan-out is massive. Solution: batch updates every 1 second and send consolidated diffs. Clients don't need per-millisecond accuracy for display.
- 

**12.16.13 Q13: How do you handle timezone and scheduling for global events?****Answer:**

- All times stored in UTC in the database.
- Show records include a `timezone` field (e.g., `Asia/Kolkata`, `America/New_York`).
- API responses include both UTC and local time for display.

- Client converts to user's local timezone for display.
  - For global events (e.g., virtual concert), sale open time is in UTC; the client shows "Sale opens at 10 PM IST / 12:30 PM ET."
  - Scheduled jobs (hold cleanup, sale opening) run on UTC cron.
- 

#### 12.16.14 Q14: How do you handle inventory sync between the online system and box office (walk-in) sales?

##### Answer:

- Box office terminals connect to the same Inventory Service via internal API.
  - Same Redis-based seat holds apply for box office selections.
  - Box office has a reserved quota (e.g., 10% of seats) that can only be sold offline. This prevents a scenario where online sales consume 100% of seats, leaving nothing for walk-ins.
  - Reserved quota is released to online pool 2 hours before showtime if unsold.
  - Both channels write to the same MySQL booking table – single source of truth.
- 

#### 12.16.15 Q15: How would you add a waitlist feature for sold-out events?

##### Answer:

1. **Join waitlist:** `POST /v1/events/{id}/waitlist` adds user to Redis sorted set (`ZADD waitlist:{show_id} <timestamp> <user_id>`).
  2. **Notification on availability:** When a booked seat is cancelled or a held seat expires for a sold-out show:
    - Pop the first N users from the waitlist (`ZPOPMIN`).
    - Send push notification: "A seat is now available! You have 3 minutes to claim it."
    - Generate a pre-authorized booking token with 3-minute TTL.
  3. **Fair ordering:** Waitlist is FIFO (sorted by join time). First on the list gets notified first.
  4. **Expiry:** If the notified user doesn't book within 3 minutes, the next user in the waitlist is notified. This cascades until someone books or the waitlist is exhausted.
  5. **Limits:** Max 5,000 users per waitlist. Beyond that: "Waitlist is full. Check back later."
-

## 12.17 17. Summary

### SYSTEM DESIGN SUMMARY

**Core challenge:** Sell limited inventory (seats) to massive concurrent demand with zero overselling and sub-second latency.

**Key decisions:**

1. Redis SET NX for seat locking (sub-ms, atomic, auto-expiry via TTL)
2. Virtual waiting room for flash sales (CDN-served, controlled admission)
3. Two-phase booking (hold → pay → confirm) with saga-based rollback
4. Idempotent payments with per-request idempotency keys
5. Microservices with event-driven async processing (Kafka)
6. Elasticsearch for search with CDC-based near-real-time indexing

**Scaling numbers:**

- 10M daily bookings, 50M seat availability checks
- Flash sale: 500K users → 5K/sec admission → 50K seats in ~20 seconds
- Redis: <1ms seat hold, 100K ops/sec per node
- MySQL: sharded by show\_id (seats), user\_id (bookings)
- 99.99% availability with multi-layer failover

**What makes this design "hard":**

- Strong consistency for inventory + eventual consistency for rest
- Flash sale thundering herd mitigation
- Payment failure handling with distributed saga
- Real-time seat map updates at scale
- Bot prevention without degrading legitimate user experience

## Chapter 13

# HLD 13: Design a Typeahead / Autocomplete System

**Cross-reference:** For the low-level class design, see [LLD: Search Autocomplete](#)

---

### 13.1 Table of Contents

1. Problem Statement
  2. Functional Requirements
  3. Non-Functional Requirements
  4. Back-of-Envelope Estimation
  5. API Design
  6. Data Model
  7. High-Level Architecture
  8. Deep Dive
  9. Partitioning
  10. Caching
  11. Replication
  12. Fault Tolerance
  13. Scalability
  14. Monitoring and Observability
  15. Trade-offs and Alternatives
  16. Interview Questions
- 

### 13.2 1. Problem Statement

Design a typeahead (autocomplete) system that provides real-time search suggestions as a user types into a search box. The system must return top relevant suggestions with **sub-100ms end-to-end latency** from keystroke to rendered dropdown.

#### Why is this hard?

- **Latency:** Users expect instant results; even 200ms feels sluggish.
- **Volume:** Every keystroke generates a request. 10B daily searches x 4 chars = 40B requests/day.

- **Freshness:** Trending topics must surface within minutes, not hours.
- **Personalization at scale:** Tailored suggestions for billions of users.
- **Multi-language:** CJK, RTL, and Indic scripts need special tokenization and segmentation.

**Real-World Examples:** Google Search autocomplete, YouTube suggestions, Amazon product search, LinkedIn search, IDE code completion.

---

## 13.3 2. Functional Requirements

- **FR-1 - Prefix-based suggestions:** Return top 5-10 suggestions matching the typed prefix
- **FR-2 - Real-time response:** Update with every keystroke (debounced 50-100ms)
- **FR-3 - Ranked results:** Ordered by frequency, recency, popularity
- **FR-4 - Typo tolerance:** Handle misspellings (“pythn” -> “python”)
- **FR-5 - Personalized suggestions:** Blend user search history into rankings
- **FR-6 - Trending queries:** Surface trending queries within minutes
- **FR-7 - Multi-language support:** Latin, CJK, Cyrillic, Arabic, Indic scripts
- **FR-8 - Offensive content filtering:** Filter profanity, hate speech, NSFW
- **FR-9 - Multi-word prefix matching:** “how to le” -> “how to learn python”
- **FR-10 - Category-aware suggestions:** Scope by category (images, news, shopping)

**Out of Scope:** Full search results rendering, spell correction for submitted queries, voice-based autocomplete, image/visual search.

---

## 13.4 3. Non-Functional Requirements

- **Latency:** p50 < 50ms, p99 < 100ms; Rationale: Each keystroke triggers a call
  - **Availability:** 99.99% (52 min downtime/year); Rationale: Critical path of every search
  - **Throughput:** 500K+ QPS sustained; Rationale: 40B requests/day
  - **Consistency:** Eventual (seconds to minutes); Rationale: Stale suggestions acceptable
  - **Scalability:** Horizontal to 1M+ QPS; Rationale: Handle spikes (Super Bowl, elections)
  - **Data freshness:** Trending < 5 min, Batch < 1 hour; Rationale: Trending topics must surface quickly
-

## 13.5 4. Back-of-Envelope Estimation

### Query Volume:

Daily searches: 10 billion  
 Avg chars typed/search: 4 (users select early)  
 Typeahead requests/day: 40 billion  
 QPS (sustained): ~460,000  
 Peak QPS (3x): ~1.4 million

### Storage:

Unique queries (freq > 1): ~5 billion  
 Avg query length: 15 chars (~20 bytes UTF-8)  
 Raw query storage: 100 GB  
 Compressed trie + metadata: 300 - 700 GB  
 Per shard (100 shards): 3 - 7 GB

### Bandwidth:

Avg response: ~500 bytes (10 suggestions x 50 chars)  
 Outbound: 460K QPS x 500B = ~1.8 Gbps  
 After CDN: ~370 Mbps from origin

### Infrastructure:

Trie servers: 100 shards x 3 replicas = 300 servers (16 GB RAM)  
 Spark cluster: 50 nodes (offline aggregation)  
 Kafka: 20 brokers (query log ingestion)  
 Redis: 50 nodes (hot prefix cache)

---

## 13.6 5. API Design

### 13.6.1 5.1 Get Suggestions

GET /api/v1/suggestions?prefix={prefix}&user\_id={user\_id}&limit={limit}&lang={lang}&category={category}

- **prefix:** Type: string; Required: Yes; Default: -; Description: Typed prefix (URL-encoded, UTF-8)
- **user\_id:** Type: string; Required: No; Default: -; Description: For personalized suggestions
- **limit:** Type: int; Required: No; Default: 10; Description: Number of suggestions (max 20)
- **lang:** Type: string; Required: No; Default: "en"; Description: Language code (ISO 639-1)
- **category:** Type: string; Required: No; Default: "all"; Description: Scope: all, news, images, shopping
- **lat, lon:** Type: float; Required: No; Default: -; Description: For geo-aware suggestions

### Response (200 OK):



```

{
 "prefix": "how to le",
 "suggestions": [
 {
 "text": "how to learn python",
 "score": 0.95,
 "highlight": { "prefix": "how to le", "completion": "arn python" },
 "metadata": { "trending": false, "personalized": false }
 }
],
 "debug": { "latency_ms": 12, "shard": "prefix-h", "cache_hit": true }
}

```

**Headers:** Cache-Control: public, max-age=300 (5 min for common prefixes).

### 13.6.2 5.2 Report Selection (Analytics)

POST /api/v1/suggestions/select

```
{ "prefix": "how to le", "selected": "how to learn python", "position": 0, "user_id": "u_12345" }
```

### 13.6.3 5.3 Client-Side Optimizations

- **Debounce:** Wait 50-100ms after last keystroke before sending
- **Cancel in-flight:** Cancel previous request on new keystroke
- **Local cache:** Cache recent prefix responses in localStorage
- **Min prefix length:** Use CDN-cached responses for 1-char prefixes

## 13.7 6. Data Model

### 13.7.1 6.1 Trie Node (In-Memory)

| TrieNode          |                                  |
|-------------------|----------------------------------|
| char              | : character                      |
| children          | : map<char, TrieNode*>           |
| is_end_of_word    | : bool                           |
| frequency         | : uint64                         |
| top_k_suggestions | : list<(query, score)> [size=10] |
| last_updated      | : timestamp                      |

13.7.2 6.2 Query Log (Kafka Topic: query-logs)

| QueryLogEvent |                 |            |          |
|---------------|-----------------|------------|----------|
| query_id      | : UUID          | language   | : string |
| query_text    | : string        | country    | : string |
| user_id       | : string (null) | device     | : enum   |
| timestamp     | : int64 (ms)    | session_id | : string |

13.7.3 6.3 Aggregated Query Frequency (HDFS / S3)

| AggregatedQueryFrequency |                               |               |           |
|--------------------------|-------------------------------|---------------|-----------|
| query_text               | : string (partition key)      |               |           |
| time_window              | : string (e.g., "2026-04-09") |               |           |
| raw_count                | : uint64                      | decayed_score | : float64 |
| language                 | : string                      | is_filtered   | : bool    |

13.7.4 6.4 User Search History (Cassandra)

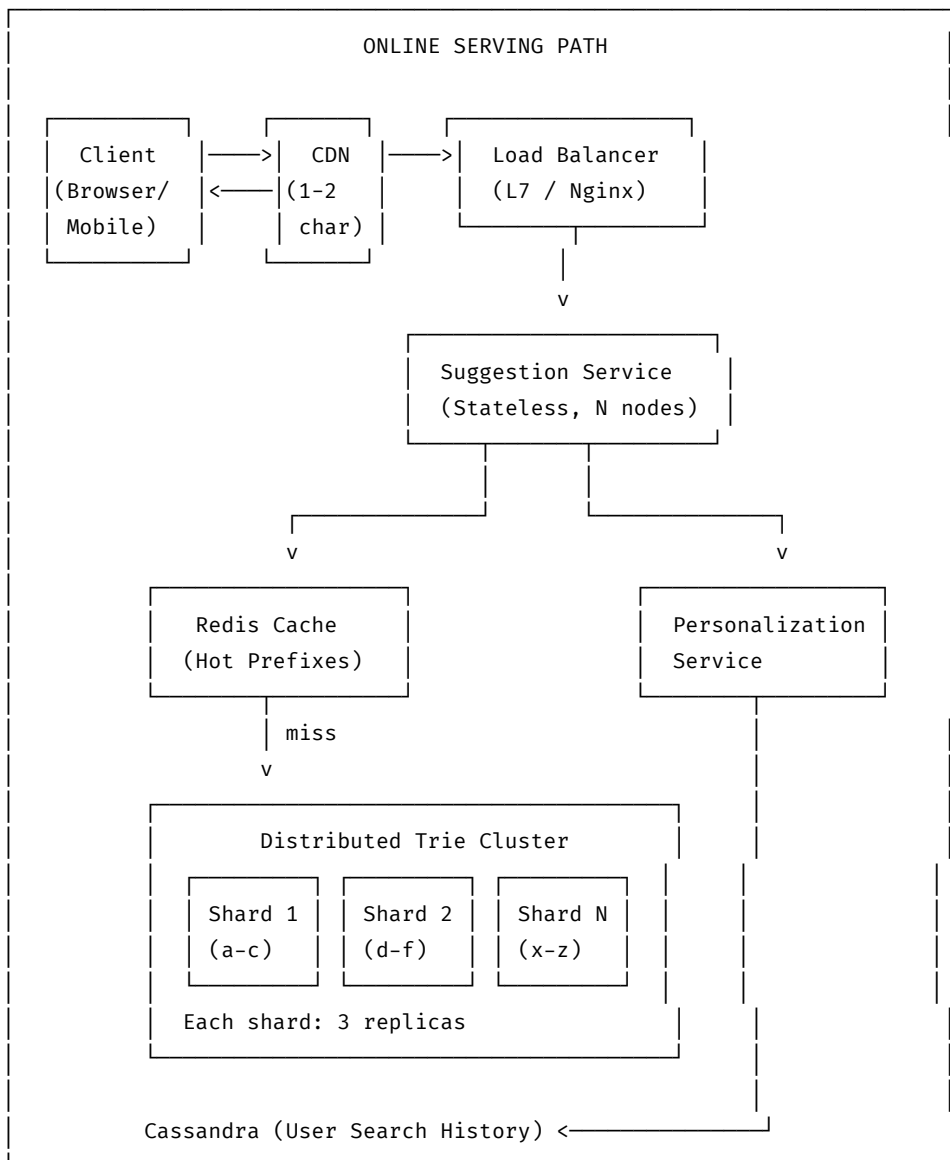
Table: user\_search\_history  
Partition key: user\_id (text)  
Clustering key: timestamp DESC  
Columns: query\_text, category, clicked  
TTL: 90 days

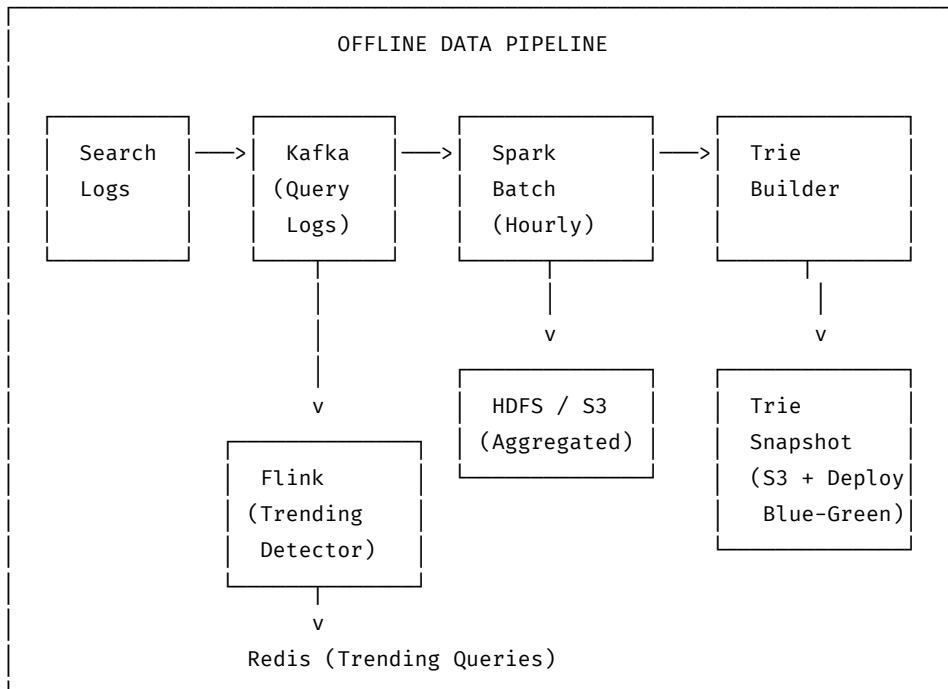
13.7.5 6.5 Trending Queries (Redis Sorted Set)

Key: trending:{lang}:{country}      Type: Sorted Set  
Member: query\_text      Score: trending velocity  
TTL: 1 hour (refreshed by streaming pipeline)

13.8 7. High-Level Architecture

### 13.8.1 7.1 System Overview





### 13.8.2 7.2 Request Flow

User types "pyth" ->

1. Client debounces (50ms), cancels previous in-flight request
2. Check browser local cache -> miss
3. GET /suggestions?prefix=pyth -> CDN miss -> LB -> Suggestion Service
4. Check Redis cache for "pyth" -> miss
5. Route to Trie Shard for prefix "p", lookup node p->y->t->h
6. Return pre-computed top-10 from that node
7. Fetch user history from Cassandra (parallel)
8. Blend global + personal + trending scores
9. Filter offensive content, apply diversity rules
10. Cache in Redis (TTL 5 min), return to client

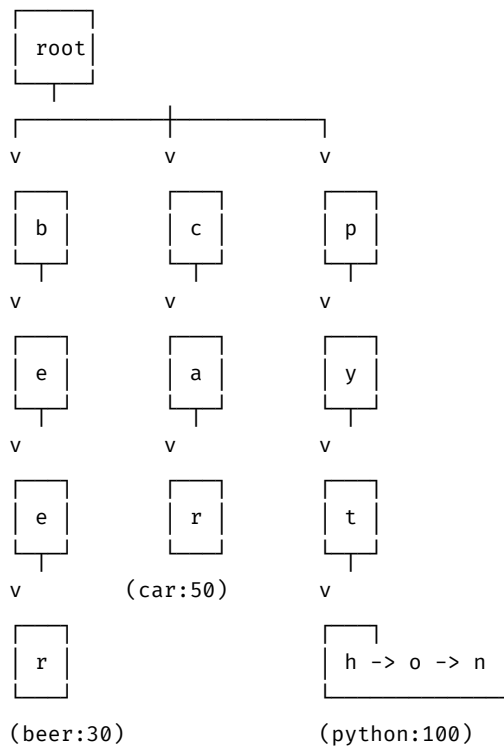
Latency budget:

|                        |                                   |
|------------------------|-----------------------------------|
| Network RTT:           | 20-30ms                           |
| Service + Trie lookup: | 5-15ms                            |
| Personalization:       | 5-10ms                            |
| Total:                 | 30-55ms (well under 100ms target) |

## 13.9 8. Deep Dive

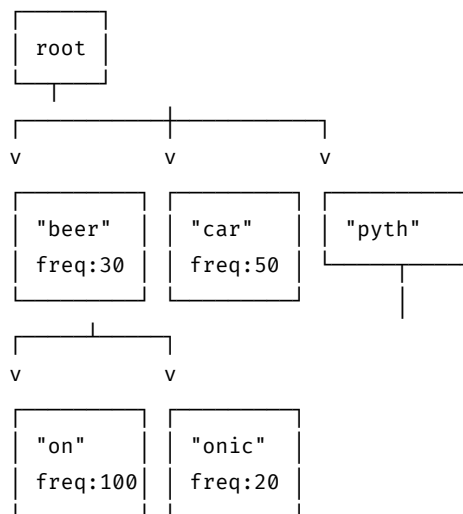
### 13.9.1 8.1 Trie Data Structure

**Standard Trie** – one character per node:



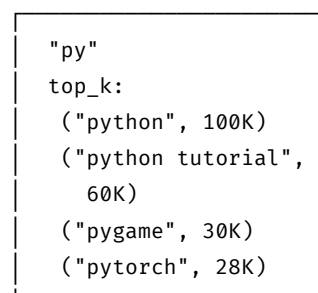
**Problem:** Finding top-K for prefix “py” requires traversing the entire subtree – too slow.

**Compressed Trie (Radix Tree)** – merges single-child chains:



Memory: Standard trie ~3 TB vs Compressed ~900 GB for 5B queries.

**Key optimization: Pre-computed Top-K at each node.** Store top-10 suggestions at every node, converting subtree traversal into a single  $O(L)$  node lookup.



Store query IDs (4 bytes) instead of strings at each node: 10 IDs x 4 bytes x 15B nodes = 600 GB + 100 GB string table = ~700 GB total. With 100 shards: ~7 GB per shard.

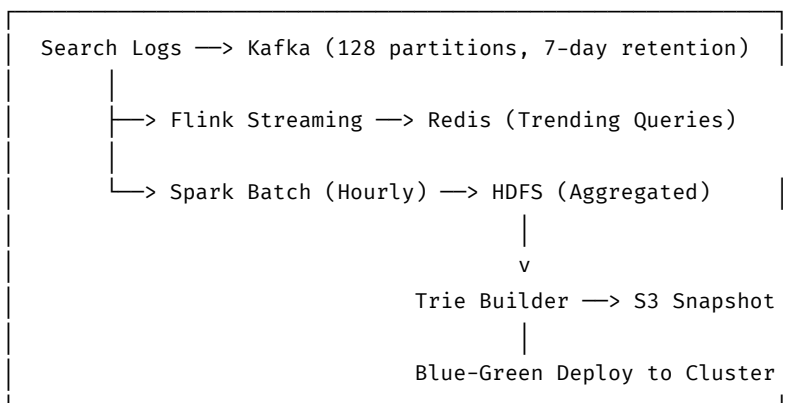
**Build-time:** Top-K is computed bottom-up. At each node, merge children's top-K lists and keep only top K using a min-heap:  $O(C \cdot K \cdot \log K)$  per node.

**Trie Serialization Format (binary, for fast deployment):**

| Trie Binary Format                                                                                                                                                                          |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Header:<br>magic ("TRIE"), version, node_count,<br>query_count, build_timestamp, checksum (MD5)                                                                                             |
| Query String Table:<br>[query_id -> (offset, length)] index<br>[concatenated query strings]                                                                                                 |
| Node Array (BFS order, cache-friendly):<br>label_offset(4B), label_len(1B),<br>child_offset(4B), child_count(1B),<br>is_terminal(1bit), frequency(4B),<br>top_k_offset(4B), top_k_count(1B) |
| Top-K Lists:<br>[query_id(4B), score(4B)] x K per node                                                                                                                                      |

BFS-ordered node layout ensures sequential memory access patterns for better CPU cache performance compared to pointer-chasing in a heap-allocated trie.

### 13.9.2 8.2 Data Collection Pipeline



**Sampling:** At 460K QPS, we sample: 1/1000 for 1-char prefixes, 1/100 for 2-3 char, 1/10 for 4+, and 100% for completed queries and click-throughs. Effective: ~50K events/sec.

**Aggregation (Spark):**

1. Map: (query, lang, country) → 1
2. Reduce: Sum counts per key
3. Decay:  $\text{new\_score} = \text{old\_score} * \exp(-\lambda * dt) + \text{new\_count}$  (half-life = 1 week)
4. Filter: Remove score < threshold, offensive content, garbage queries

5. Output: Parquet files to S3, partitioned by date/lang/country

#### Update modes:

- Full rebuild: 30-60 min, daily or on schema changes
- Incremental: 1-5 min, hourly frequency updates
- Real-time patch: Seconds, trending queries only (via Redis, not trie mutation)

### 13.9.3 8.3 Ranking and Scoring

```
final_score = w1 * global_frequency (0.30)
+ w2 * recency (0.25)
+ w3 * trending (0.15)
+ w4 * personalization (0.15)
+ w5 * contextual (0.10)
+ w6 * quality (0.05)
```

**Global frequency:**  $\log_{10}(1 + \text{decayed\_count}) / \log_{10}(1 + \text{max\_count})$  normalized to [0,1].

#### Personalization blend example:

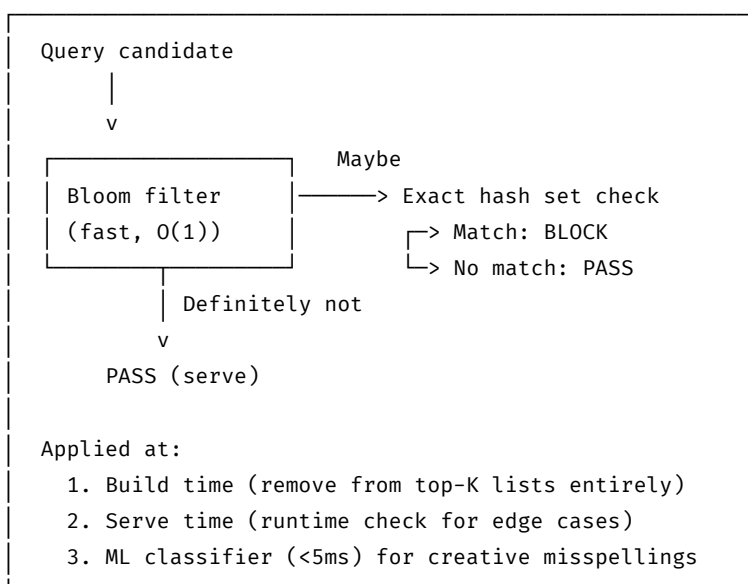
User types "ja", has history: [javascript tutorial x5, java spring boot x3]

| Global top-5:         | Blended result:                    |
|-----------------------|------------------------------------|
| 1. java (100K)        | 1. javascript (boosted by history) |
| 2. japan travel (80K) | 2. java (global + history)         |
| 3. javascript (75K)   | 3. java spring boot (personal)     |
| 4. jake paul (60K)    | 4. japan travel (global)           |
| 5. james bond (55K)   | 5. jake paul (global)              |

**Trending detection:**  $\text{velocity} = (\text{count\_last\_hour} - \text{avg\_7d\_hourly}) / (\text{avg\_7d\_hourly} + \text{epsilon})$ . If  $\text{velocity} > 3x$ , mark as trending and boost.

**Diversity:** Use Maximal Marginal Relevance (MMR) – max 3 suggestions with same first 2 words, at least 1 trending if available, at least 1 personalized if logged in.

#### Offensive filtering pipeline:



### 13.9.4 8.4 Trie Sharding

**Prefix range partitioning** with load-balanced boundaries:

```
Shard 0: "a" (~10% traffic)
Shard 1: "b" - "c" (~10%)
Shard 2: "d" - "f" (~10%)
...
Shard 7: "t" (~10%) -- "the", "to" are very common
Shard 9: "x"-"z", 0-9 (~10%)
Shard 10: CJK characters (separate scheme)
```

Boundaries are NOT uniform – weighted by query volume so each shard handles roughly equal QPS. Re-balanced weekly. Hot shards (e.g., “s”) can be further split by 2nd character.

**Replication:** 3 replicas per shard, spread across availability zones. All replicas serve reads (load balanced). Updates pushed during trie deployment.

**Routing:**

```
Suggestion Service receives prefix "python"

1. Normalize: lowercase, trim whitespace
2. Extract routing key: first_char = 'p'
3. Lookup shard map (cached from ZooKeeper): 'p' -> S5
4. Select replica: round-robin among healthy Shard 5
 replicas, prefer same-region
5. Send gRPC request to selected replica
6. On failure: retry on different replica
 (max 1 retry, timeout = 50ms)
```

## 13.10 9. Partitioning

- **Trie:** Prefix range (first 1-2 chars), 50-100 shards, load-balanced
- **Kafka logs:** 128 partitions, keyed by `hash(query_text)`, 7-day retention
- **HDFS/S3:** Date + language + country partitions, Parquet format, 90-day retain
- **Cassandra:** `user_id` (Murmur3 hash), clustering by `timestamp DESC`, RF=3

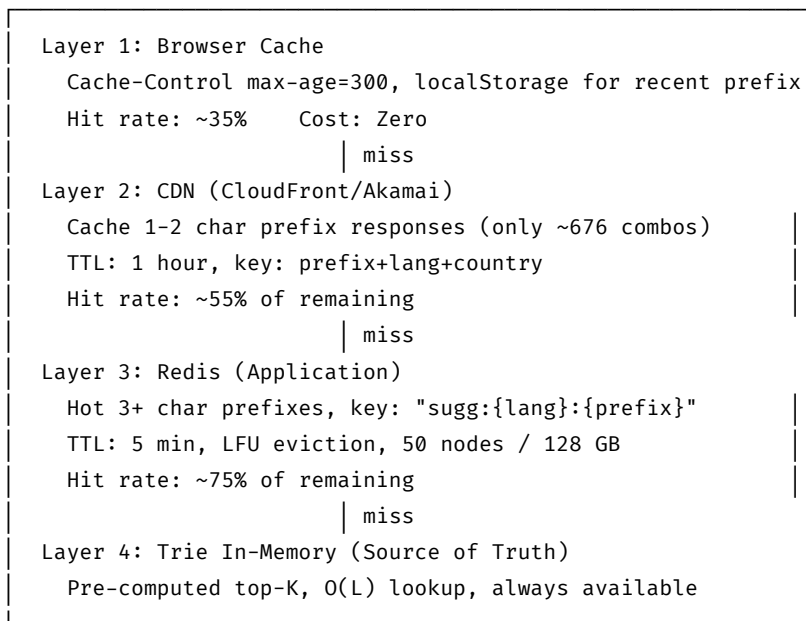
For HDFS/S3: `s3://typeahead/aggregated/dt=2026-04-09/lang=en/country=US/part-*.parquet`

Cassandra: Write CL=ONE, Read CL=ONE (low latency). Max 1000 queries/user. TTL 90 days. TimeWindowCompactionStrategy.

## 13.11 10. Caching



### 13.11.1 Multi-Layer Architecture



Net effect: 460K QPS total -> only ~34K QPS reach the trie (93% reduction)

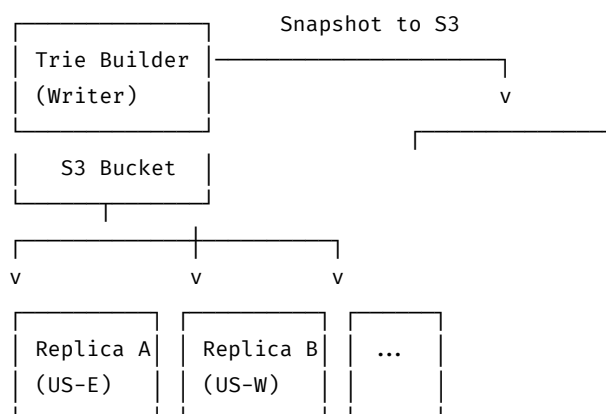
**Personalized caching:** Cache global top-20 at CDN/Redis. Personalization computed as a server-side re-ranking overlay – keeps CDN effective while still personalizing.

**Invalidation:** TTL-based natural expiry. Trending detection triggers surgical Redis purge for affected prefixes. Offensive content addition triggers immediate purge. Top-1000 prefixes proactively re-freshed before TTL expires (background refresh at 80% TTL).

**Thundering herd mitigation:** Staggered TTLs (5 min +/- 30s jitter), lock-based deduplication (Redis SETNX), stale-while-revalidate for async refresh.

## 13.12 11. Replication

### 13.12.1 Trie Replication Model



Each replica polls S3 (or receives SNS push) for new snapshots, loads in background alongside current trie, performs atomic pointer swap, frees old memory.

**Consistency:** Eventually consistent, minutes lag acceptable. All replicas converge within 1 rebuild cycle (~1 hour). Trending queries served from Redis (not trie) for near-real-time freshness. Version IDs tracked; alert if skew > 2.

**Blue-Green Deployment:** Canary replica gets new version first, serves 5% traffic for 10 min. If healthy: rolling update 10% -> 25% -> 50% -> 100% over 30 min. Previous snapshot retained in S3 for instant rollback (pointer swap, < 1 min).

## 13.13 12. Fault Tolerance

- **Single trie replica crash:** Route to other replicas (3x replication), <5s detection
- **Entire shard offline:** Serve stale cache; fallback to popular queries list
- **Redis cache down:** Bypass cache, serve from trie (auto-scale for load)
- **CDN outage:** Clients fall through to origin; auto-scaling kicks in
- **Trie build failure:** Continue serving previous version; alert and retry
- **Kafka broker failure:** RF=3 handles broker loss; some log loss acceptable
- **Cassandra node down:** RF=3, CL=ONE transparent; degrade to global-only
- **Network partition:** Each region operates independently with local tries

### Graceful Degradation Hierarchy:

Level 0: Full service (global + personalized + trending + filtered)

Level 1: No personalization (Cassandra down)

Level 2: No trending (Redis down)

Level 3: Cached only (trie cluster unreachable)

Level 4: Static fallback (top-100 popular queries per language, pre-computed)

Level 5: Empty suggestions (catastrophic failure, client hides dropdown)

**Circuit breakers** on all backends: open at >50% failure rate in 10s window, half-open after 30s trial at 10% traffic, close when success rate > 90%.

## 13.14 13. Scalability

1. **Trie Sharding:** Add shards as data grows  
100 -> 1000+ shards, split hot shards by 2nd char
2. **Read Replicas:** Add replicas per shard  
3 -> 10+ for hot shards
3. **CDN Offloading:** Absorbs 80%+ traffic  
Add PoPs in new regions as user base grows
4. **Suggestion Service:** Stateless, auto-scale  
K8s HPA: 60% CPU target, 50 -> 500 pods in minutes
5. **Redis Cache:** Redis Cluster with hash slots  
Add nodes for more memory/throughput

**Traffic spikes (e.g., Super Bowl at 10x normal):** CDN absorbs most; Redis handles warm prefixes; auto-scaling for compute; pre-warm caches for predicted events; rate limit at 20 req/sec per user; degrade gracefully (disable personalization, increase TTLs, reduce suggestions from 10 to 5).

**Multi-region:** Full trie copy in each region (US-East, US-West, EU, APAC). Central pipeline in US-East builds snapshots replicated to all regions via S3. DNS routes users to nearest region. Each region serves independently.

## 13.15 14. Monitoring and Observability

### 13.15.1 Key Metrics

|                 |                                                                              |
|-----------------|------------------------------------------------------------------------------|
| LATENCY         | p50: 12ms p95: 38ms p99: 72ms                                                |
| THROUGHPUT      | 462K QPS total, 21K to trie (5%)                                             |
| CACHE HIT RATES | CDN: 82% Redis: 76% Client: 35%                                              |
| TRIE HEALTH     | Version: v2847, 14.2B nodes, 6.2 GB<br>Last rebuild: 32 min ago (18 min dur) |
| QUALITY         | CTR: 42% MRR: 0.68 Diversity: 0.72                                           |
| PIPELINE        | Kafka lag: 12s Spark: SUCCESS                                                |

### 13.15.2 Alerting Rules

- **High latency:** Condition: p99 > 200ms for 2 min; Severity: P1
- **Low CDN hit rate:** Condition: < 60% for 10 min; Severity: P2
- **Trie version skew:** Condition: > 2 versions for 30 min; Severity: P2
- **Trie rebuild failure:** Condition: 2 consecutive failures; Severity: P1
- **Shard replicas < 2:** Condition: Any shard; Severity: P1
- **Error rate:** Condition: > 0.1% 5xx for 2 min; Severity: P1
- **CTR drop:** Condition: > 20% vs 7-day avg; Severity: P2
- **Offensive content spike:** Condition: > 5x normal filtered; Severity: P1

Distributed tracing (Jaeger) across LB -> Service -> Cache -> Trie -> Personalization. Sample rate 0.1%. Structured JSON logs for every request with request\_id, shard, cache\_layer, latency\_ms, trie\_version.

## 13.16 15. Trade-offs and Alternatives

### 13.16.1 15.1 Trie vs Inverted Index

- **Prefix lookup:** Trie:  $O(L)$  – excellent; Inverted Index:  $O(\log N)$  with sorted lists
- **Pre-computed top-K:** Trie: Natural fit at each node; Inverted Index: Requires separate index
- **Fuzzy matching:** Trie: Needs edit-distance walk; Inverted Index: Better (n-gram based)
- **Multi-word:** Trie: Full query as key; Inverted Index: Per-word + intersection
- **Best for:** Trie: Exact prefix, real-time; Inverted Index: Fuzzy, multi-field search

**Verdict:** Trie is standard for typeahead. Use inverted index as complement for fuzzy matching.

### 13.16.2 15.2 Prefix Match vs Fuzzy Match

Prefix match:  $O(L)$ , high precision, simple. Fuzzy match:  $O(L * \text{edit\_dist})$ , handles typos, more false positives.

**Hybrid:** Always do prefix first. If  $< 5$  results, trigger fuzzy match (n-gram index, edit distance 1-2, BK-tree or SymSpell). Merge results, prefix preferred. Fuzzy only for prefixes  $> 3$  chars.

### 13.16.3 15.3 Real-Time vs Batch Updates

- **Freshness:** Real-Time: Seconds; Batch: Minutes to hours
- **Complexity:** Real-Time: High (concurrent R/W); Batch: Low (rebuild and swap)
- **Risk:** Real-Time: Memory fragmentation; Batch: Stale data between builds

**Hybrid:** Batch rebuild hourly (99% of data) + streaming for trending only (1%, via Redis). Avoids real-time trie mutation complexity while keeping trending fresh.

### 13.16.4 15.4 Caching Strategy

- **Aggressive (long TTL):** Pro: 95%+ hit rate, low load; Con: Stale suggestions
- **Conservative (short):** Pro: Fresh results; Con: Higher trie load
- **Per-user caching:** Pro: Personalized + cached; Con: Huge cache, low per-user hits
- **Two-tier (recommended):** Pro: CDN for global, compute personal overlay; Con: Slightly more logic

## 13.17 16. Interview Questions

**Q1: How do you handle multi-word prefix matching?** The trie stores full query strings including spaces. “how to le” is a path h-o-w-’ -t-o-’ -l-e. The top-K at node “e” contains “how to learn python”. Multi-word and single-word queries are handled identically from the trie’s perspective. Compressed tries mitigate the cost of common chains like “how to”.

**Q2: How do you handle real-time trending updates without full trie rebuild?** Two-tier approach: main trie rebuilt hourly (Spark batch). Flink streaming detects queries with velocity  $> 3x$  their 7-day average and writes them to Redis sorted sets. At serving time, the Suggestion Service merges trie results with Redis trending results. This avoids the complexity of mutating a distributed in-memory trie.

**Q3: How do you support CJK languages?** (1) Use word segmenters (Jieba for Chinese, MeCab for Japanese) before trie lookup. (2) Maintain character-level CJK trie for character-by-character input. (3) Support pinyin/romaji input (“beijing”  $\rightarrow$  “□□”). (4) Dedicate separate shards for CJK (Unicode U+4E00-U+9FFF, ~20K chars). (5) More aggressive debounce (200ms) due to IME buffering.

**Q4: How do you filter offensive suggestions?** Multi-layer: (1) Build-time blocklist + regex removal from trie. (2) Serving-time Bloom filter pre-check ( $O(1)$ , 0.1% FPR). (3) Exact hash set verification on Bloom “maybe” results. (4) Lightweight ML classifier ( $< 5ms$ ) for creative misspellings. (5) Human review queue for borderline cases. (6) Monitor for spikes indicating coordinated abuse.

**Q5: How do you handle cold start for new users?** Default to global popularity. Use demographic signals (language, country, device, time-of-day). Track session context – if user searched “python tutorial”, bias toward programming. Boost trending weight ( $w_3$ : 0.15  $\rightarrow$  0.25) for new users. After 5-10 searches, begin basic personalization with short half-life.

**Q6: How do you handle thundering herd on cache expiry?** (1) Staggered TTLs (5 min +/- 30s jitter). (2) Background refresh for top-1000 prefixes at 80% TTL. (3) Lock-based dedup via Redis SETNX (only 1 request refills, others wait). (4) Stale-while-revalidate – serve expired value while async refresh.

**Q7: What if a trie shard exceeds single-machine memory?** Split by next character: shard “s” becomes sub-shards “sa-sf”, “sg-sm”, “sn-ss”, “st-sz”. Update shard map in ZooKeeper. Routing uses first 2 characters instead of 1. Transparent to clients. Can split on 3+ characters in extreme cases.

**Q8: How do you measure suggestion quality?** CTR (target >35%), MRR (target >0.6, higher = clicked suggestion ranks higher), Keystroke Savings Rate (target >50%), abandonment rate, time to selection. Every ranking change A/B tested before full rollout.

**Q9: How do you handle prefix “a” matching billions of queries?** CDN-cached with 1-hour TTL (only 36 possible 1-char prefixes). Pre-computed static top-10 lists baked at build time. O(1) lookup from pre-computed top-K at the “a” node. For logged-in users, personalization overlay computed server-side.

**Q10: How do you handle multiple languages?** Separate tries per language (recommended at scale). Routing key: `lang + prefix[0]` determines shard. Each language has different character distributions and query patterns, making separate tries more efficient. Multilingual users get suggestions from their configured language’s trie.

**Q11: How do you prevent the trie from growing indefinitely?** (1) Frequency threshold: only include score > 5. (2) Recency cutoff: drop queries unsearched for 90 days. (3) Hard cap at 5B unique queries; evict lowest-scoring. (4) Length limit: 100 chars. (5) Normalize (lowercase, trim, collapse spaces). (6) Weekly full rebuild prunes and compacts.

**Q12: How do you bootstrap with no query history?** Seed with Wikipedia titles, Common Crawl page titles, product catalogs, place names. Integrate Twitter/Google Trends. Manual curation for first weeks. Short half-life (1 day vs 1 week) for aggressive early learning. Gradually increase half-life as corpus grows.

**Q13: How do you implement “did you mean” alongside typeahead?** Separate systems: typeahead (before submission) vs “did you mean” (after submission). For typeahead fuzzy matching: maintain bigram/trigram index alongside trie. When prefix yields < 5 exact results, query n-gram index for edit distance 1-2 matches using SymSpell. Mark fuzzy results distinctly in UI. Only for prefixes > 3 chars.

**Q14: How do you address privacy concerns?** 90-day TTL on user history. Opt-out deletes all data. On-device personalization option (client-side re-ranking). Anonymize query logs after session. Differential privacy for aggregate frequencies. GDPR/CCPA deletion within 72 hours. Never surface one user’s private searches to another.

**Q15: How would e-commerce typeahead differ?**

#### E-Commerce Typeahead Differences

1. Catalog-aware: Only suggest existing products
2. Category suggest: "sho" -> "shoes" (category)
3. Attribute-based: "iphone 15 pro max 256gb" (SKU)
4. Inventory-aware: Deprioritize out-of-stock items
5. Revenue-weighted: Score by conversion rate x AOV
6. Rich results: Thumbnails, prices, ratings
7. Seasonal boost: "halloween costume" in October
8. Aggressive fuzzy: "adiddas" -> "adidas" (brand misspelling directly impacts revenue)

## 13.18 Summary

#### KEY DESIGN DECISIONS

1. Compressed trie + pre-computed top-K ->  $O(L)$  lookup
2. Multi-layer caching (client+CDN+Redis) -> 93% offload
3. Batch rebuild (hourly) + streaming trending (seconds)
4. Prefix-range sharding, load-balanced boundaries
5. Blue-green deployment with canary rollout
6. 5-level graceful degradation hierarchy
7. Two-tier personalization (cache global, blend at serve)
8. Offensive filtering at build + serve time

## Chapter 14

# Design a Distributed Key-Value Store (Dynamo-Style)

---

### 14.1 Table of Contents

1. Problem Statement
  2. Functional Requirements
  3. Non-Functional Requirements
  4. Back-of-Envelope Estimation
  5. API Design
  6. Data Model
  7. High-Level Architecture
  8. Deep Dive
  9. Data Partitioning
  10. Caching
  11. Replication and Consistency
  12. Fault Tolerance
  13. Scalability
  14. Monitoring
  15. Trade-offs
  16. Interview Questions
- 

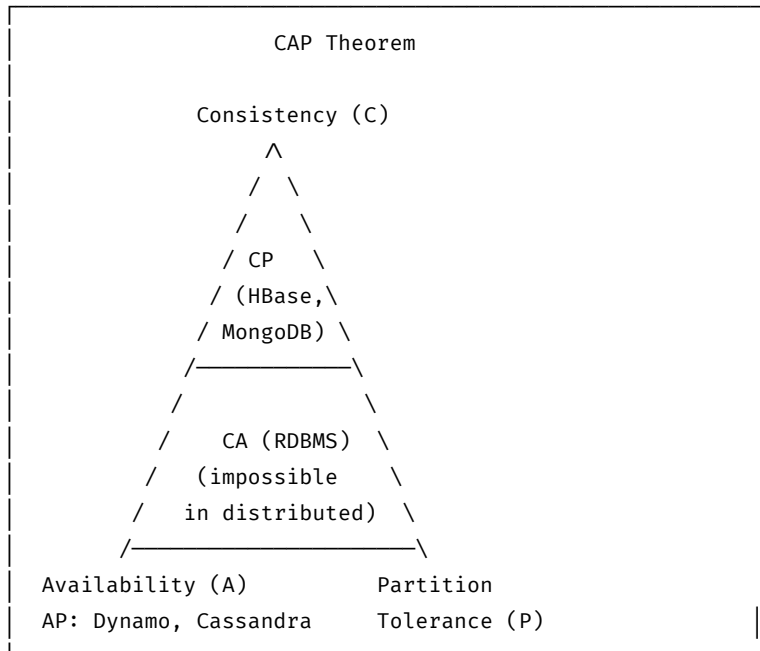
### 14.2 1. Problem Statement

Design a highly available, partition-tolerant distributed key-value store inspired by Amazon's Dynamo. The system must:

- Provide **always-on write availability** – writes must never be rejected, even during partial failures or network partitions.
- Offer **tunable consistency** so applications can choose their own trade-off between consistency and latency on a per-request basis.
- Scale **linearly** by adding commodity hardware – no special nodes, no single points of failure.
- Support **multi-datacenter replication** with eventual convergence.

Real-world systems that follow this architecture: Amazon DynamoDB, Apache Cassandra, Riak, Voldemort, ScyllaDB.

**Why Dynamo-style?** Traditional RDBMS systems sacrifice availability during partitions (CP). Dynamo chooses AP: it remains available and writable even when nodes are unreachable, resolving conflicts after the partition heals.



## 14.3 2. Functional Requirements

### 14.3.1 Core Operations

- **put:** Store a key-value pair with optional metadata (TTL, context)
- **get:** Retrieve value(s) for a key; may return multiple versions
- **delete:** Tombstone a key (soft delete with propagation)

### 14.3.2 Consistency Levels (per-request tunable)

- **ONE:** Behavior: Respond after 1 replica acknowledges. Lowest latency.
- **QUORUM:** Behavior: Respond after majority (W or R) replicas acknowledge.
- **ALL:** Behavior: Respond after all N replicas acknowledge. Strongest guarantee.

### 14.3.3 Additional Features

- **TTL (Time-To-Live):** Keys can have an expiration time. Expired keys are purged during compaction and filtered on reads.
- **Versioning:** Every write is tagged with a vector clock (or timestamp). Multiple concurrent versions may coexist until resolved.
- **Range Queries:** Not a primary feature (this is a hash-based store), but secondary indexes can be layered on top.
- **Batch Operations:** Multi-key get/put for throughput optimization.
- **Conditional Writes:** Compare-and-swap using the context (vector clock) returned from a prior get.



---

## 14.4 3. Non-Functional Requirements

- **Availability:** 99.99% (< 52.6 minutes downtime / year)
- **Read Latency:** p50 < 1ms, p99 < 10ms
- **Write Latency:** p50 < 2ms, p99 < 10ms
- **Scalability:** Linear throughput scaling with added nodes
- **Durability:** No data loss once acknowledged (WAL + replication)
- **Partition Tol.:** System remains read/write available during partitions
- **Data Size:** Support petabytes of data across the cluster
- **Key Size:** Up to 256 bytes
- **Value Size:** Up to 1 MB (typical: 1-10 KB)

### 14.4.1 SLA Targets

- 99.9th percentile latency (not averages) – tail latency matters.
  - Zero single points of failure.
  - Automated failure detection and recovery within seconds.
- 

## 14.5 4. Back-of-Envelope Estimation

### 14.5.1 Traffic

Read throughput: 100,000 requests/sec  
Write throughput: 50,000 requests/sec  
Total: 150,000 requests/sec

### 14.5.2 Storage

Average value size: 10 KB  
Writes per day:  $50,000 * 86,400 = 4.32$  billion  
Raw data per day:  $4.32B * 10\ KB = 43.2\ TB / day$   
With 3x replication:  $43.2 * 3 = 129.6\ TB / day$   
With 2x compaction overhead:  $129.6 * 2 = 259.2\ TB / day (peak)$

### 14.5.3 Bandwidth

Read bandwidth:  $100,000 * 10\ KB = 1\ GB/s$  inbound  
Write bandwidth:  $50,000 * 10\ KB = 500\ MB/s$  inbound  
Replication:  $500\ MB/s * 2$  (for 2 additional replicas) = 1 GB/s inter-node  
Total network:  $\sim 2.5\ GB/s = \sim 20\ Gbps$

### 14.5.4 Node Count

Per node storage capacity: 4 TB usable (8 TB disks, 50% utilization)  
 Data per day (replicated): 129.6 TB  
 30-day retention:  $129.6 * 30 = 3,888$  TB  
 Nodes needed (storage):  $3,888 / 4 = 972$  nodes  
 Round up for headroom: ~1,000 nodes

Per node throughput: ~500 ops/sec (conservative for 10KB values)  
 Nodes needed (throughput):  $150,000 / 500 = 300$  nodes

### 14.5.5 Memory

Per node RAM: 64 GB  
 MemTable size: 256 MB (flushed at threshold)  
 Key cache: ~10 GB (hot keys in LRU cache)  
 Bloom filters: ~2 GB (10 bits/key, millions of keys)  
 OS page cache: ~40 GB (SSTable reads)  
 Overhead: ~12 GB

---

## 14.6 5. API Design

### 14.6.1 Put (Write)

PUT /v1/kv/{key}

Headers:

Content-Type: application/octet-stream  
 X-Consistency: QUORUM # ONE | QUORUM | ALL  
 X-TTL: 86400 # optional, seconds  
 X-Context: <base64-vector-clock> # from prior GET, enables conflict detection

Body: <raw value bytes>

Response 200 OK:

```
{
 "key": "user:12345",
 "version": "vclock:a1b2c3...",
 "timestamp": 1712678400000,
 "replicas_acked": 2
}
```

### 14.6.2 Get (Read)

GET /v1/kv/{key}

Headers:

X-Consistency: QUORUM # ONE | QUORUM | ALL

Response 200 OK:

```
{
 "key": "user:12345",
 "values": [
 {
 "value": "<base64-encoded>",
 "context": "vclock:a1b2c3 ... ",
 "timestamp": 1712678400000
 }
],
 "replicas_read": 2
}
```

When there are conflicting versions (sibling values), the `values` array contains multiple entries. The client must resolve the conflict and write back with the merged context.

### 14.6.3 Delete

DELETE /v1/kv/{key}

Headers:

X-Consistency: QUORUM  
X-Context: <base64-vector-clock>

Response 200 OK:

```
{
 "key": "user:12345",
 "tombstone": true,
 "gc_grace_seconds": 864000
}
```

Deletes are **tombstones** – a special marker that propagates to all replicas. The tombstone is retained for `gc_grace_seconds` (default 10 days) to ensure all replicas learn about the deletion, then purged during compaction.

### 14.6.4 Internal RPC API (Node-to-Node)

```
// Coordinator -> Replica
rpc ReplicaWrite(key, value, vector_clock, ttl) -> (ack, updated_clock)
rpc ReplicaRead(key) -> (value, vector_clock, timestamp)
rpc ReplicaDelete(key, vector_clock) -> (ack)

// Anti-entropy
rpc MerkleTreeExchange(range, tree_root) -> (diff_ranges)
rpc StreamRepair(range, missing_keys) -> (stream of key-value pairs)

// Gossip
rpc GossipDigest(node_states[]) -> (node_states[])

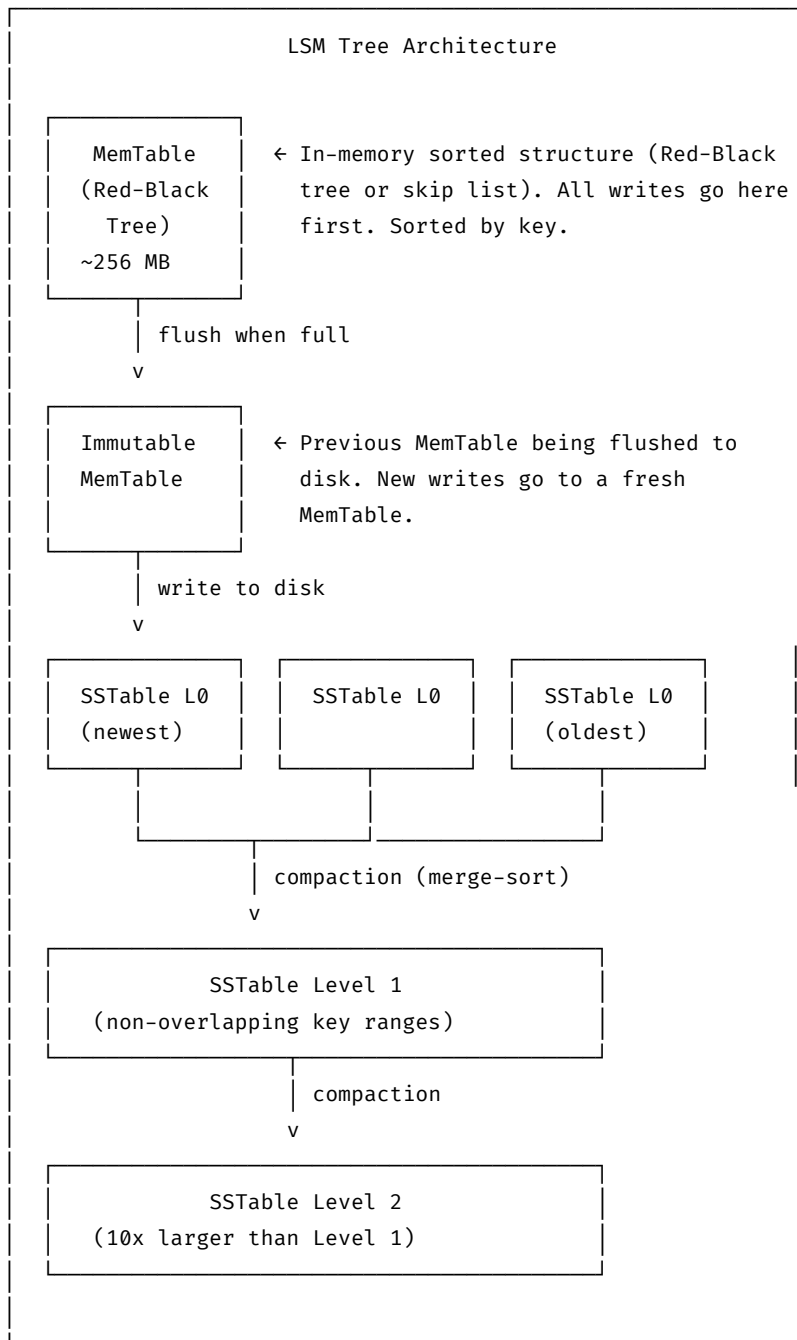
// Hinted Handoff
rpc DeliverHint(target_node, key, value, clock) -> (ack)
```

---

## 14.7 6. Data Model

### 14.7.1 6.1 LSM Tree (Log-Structured Merge Tree)

The LSM tree is the core on-disk data structure for Dynamo-style stores. It optimizes for **write throughput** at the cost of slightly more expensive reads.

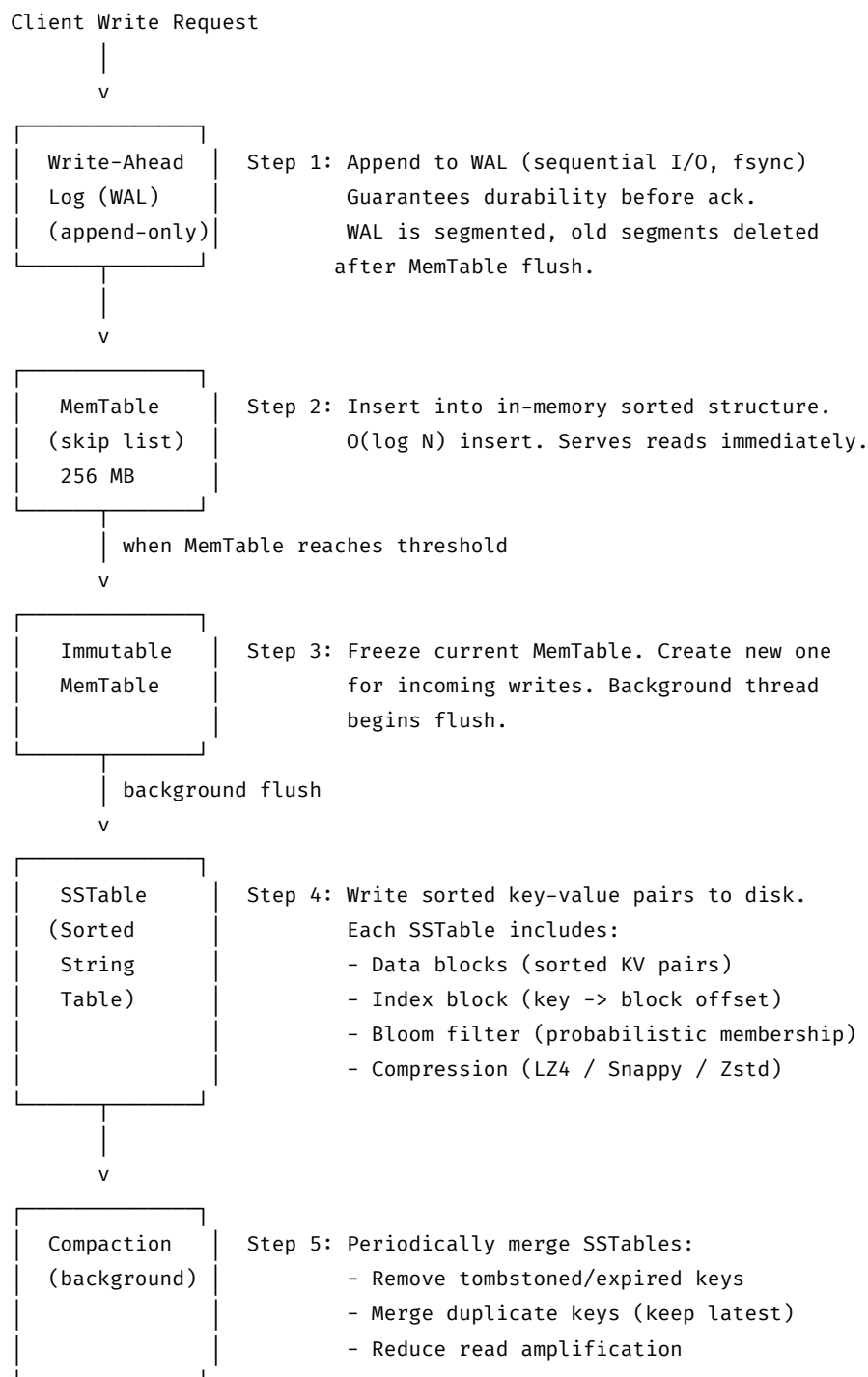


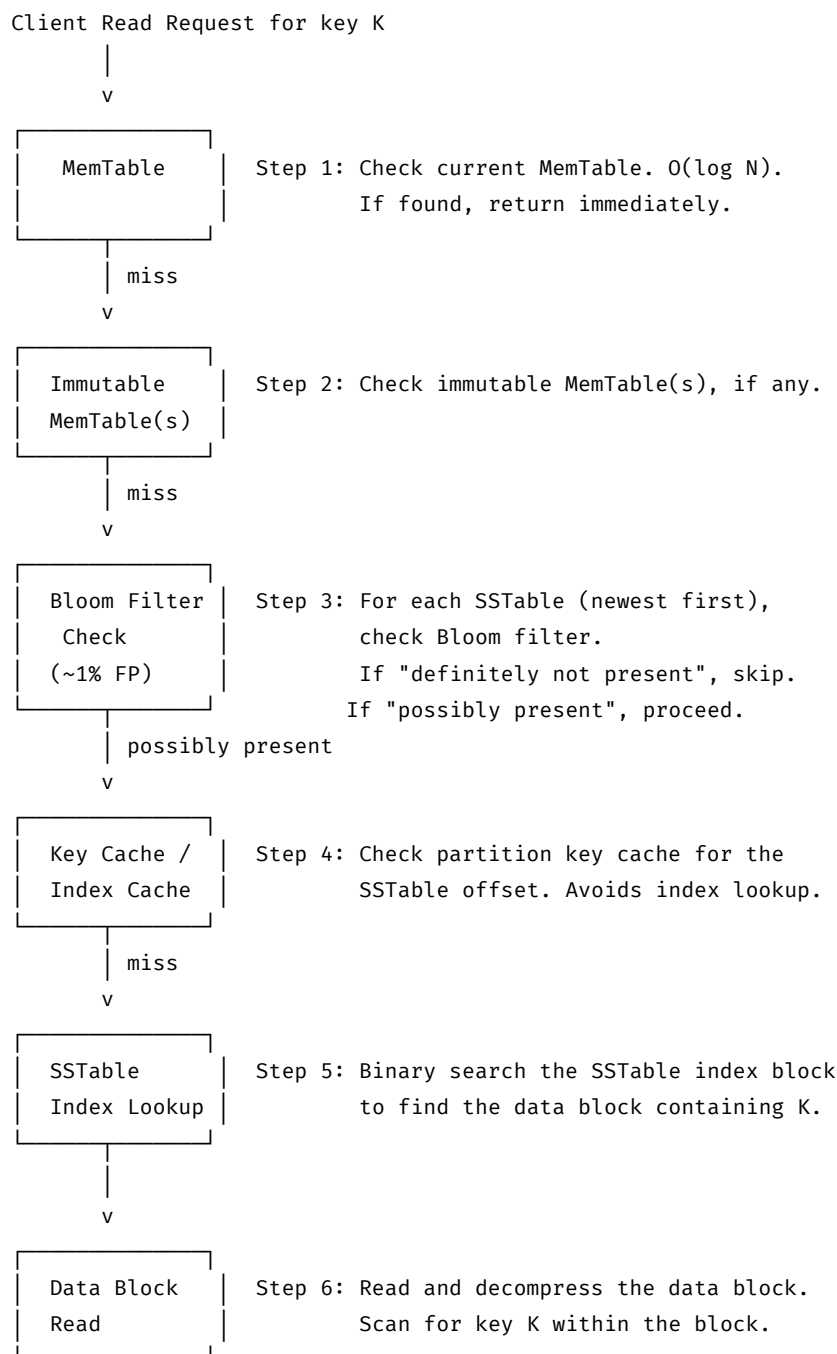
### Why LSM?

- **Write speed:** LSM Tree:  $O(1)$  amortized; B-Tree:  $O(\log N)$  random I/O
- **Read speed:** LSM Tree:  $O(L)$  levels checked; B-Tree:  $O(\log N)$
- **Write amp.:** LSM Tree: 10-30x; B-Tree: 2-3x
- **Space amp.:** LSM Tree: Low (compaction); B-Tree: ~50% page fill
- **Concurrency:** LSM Tree: Lock-free writes; B-Tree: Page-level locks

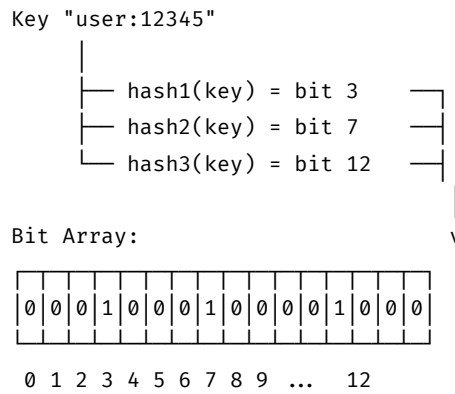
### 14.7.2 6.2 Storage Engine

The complete write and read paths through the storage engine:

**14.7.2.1 Write Path**

**14.7.2.2 Read Path****14.7.2.3 Bloom Filters**

A Bloom filter is a space-efficient probabilistic data structure that tests whether an element is a member of a set. It can have false positives but never false negatives.



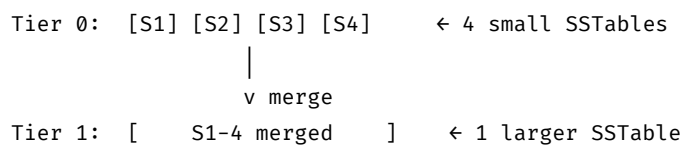
Lookup: All 3 bits are set → "possibly present"

If ANY bit is 0 → "definitely not present"

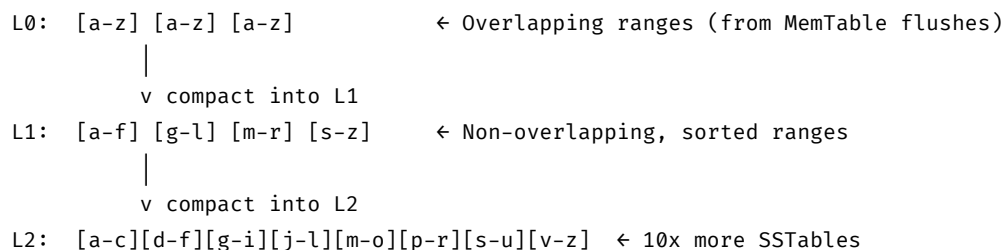
**Configuration:** - 10 bits per key, 3 hash functions: ~1% false positive rate - 15 bits per key, 4 hash functions: ~0.1% false positive rate - Memory: ~1.2 GB for 1 billion keys at 10 bits/key

#### 14.7.2.4 Compaction Strategies

**Size-Tiered Compaction (STCS):** - Group SSTables of similar size into tiers. - When a tier has enough SSTables (e.g., 4), merge them into one larger SSTable. - Pros: Good write throughput, simple. - Cons: High space amplification (up to 2x), wide key-range overlap.



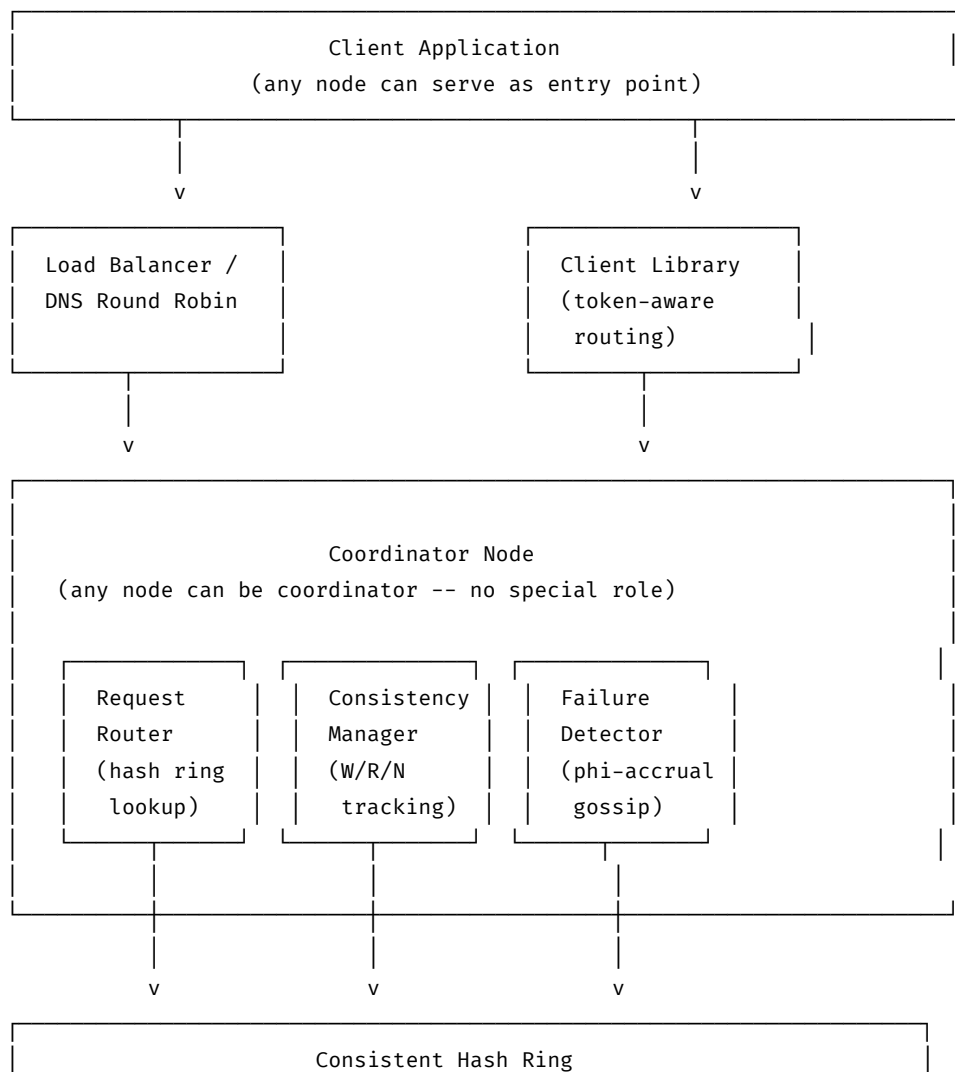
**Leveled Compaction (LCS):** - SSTables organized into levels (L0, L1, L2, ...). - Each level is 10x the size of the previous. - L1+ has non-overlapping key ranges. - When L(n) is full, pick an SSTable from L(n) and merge it with overlapping SSTables in L(n+1). - Pros: Low space amplification, bounded read amplification. - Cons: Higher write amplification (10-30x).

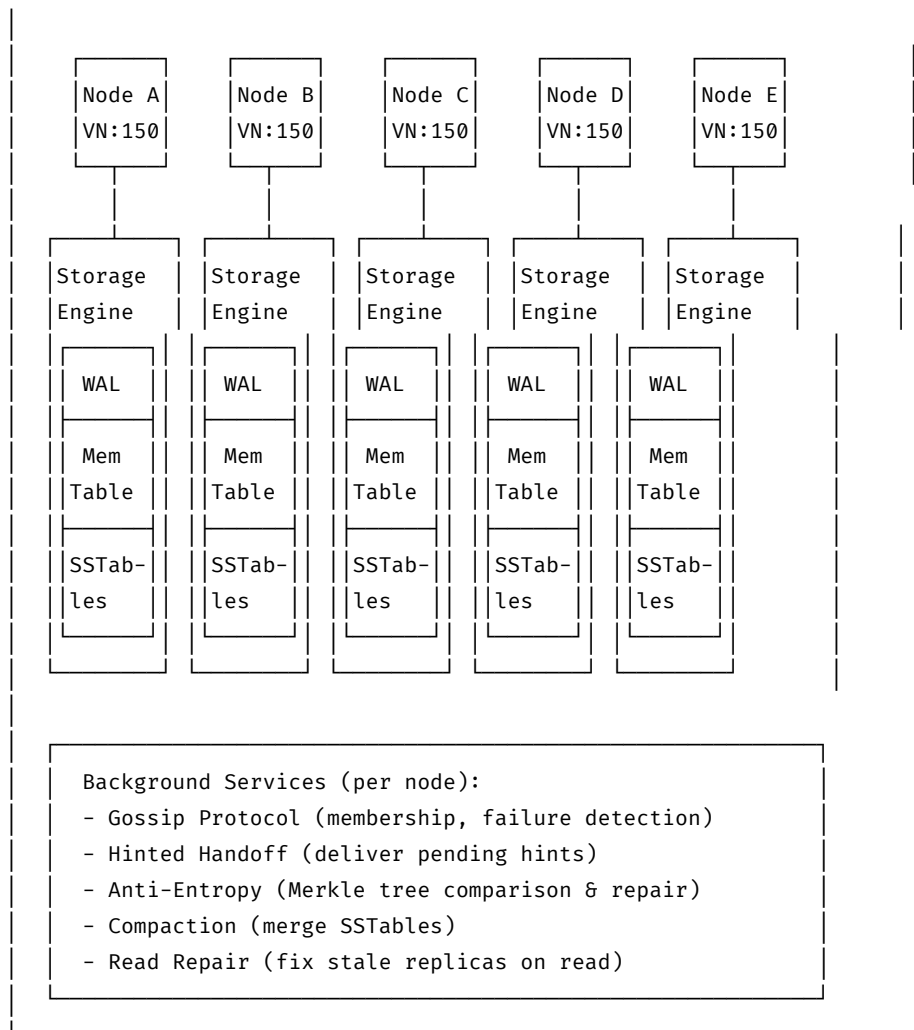


**Time-Window Compaction (TWCS):** - Groups SSTables by time window (e.g., 1 hour). - Only compacts SSTables within the same window. - Ideal for time-series data with TTL – entire windows can be dropped.

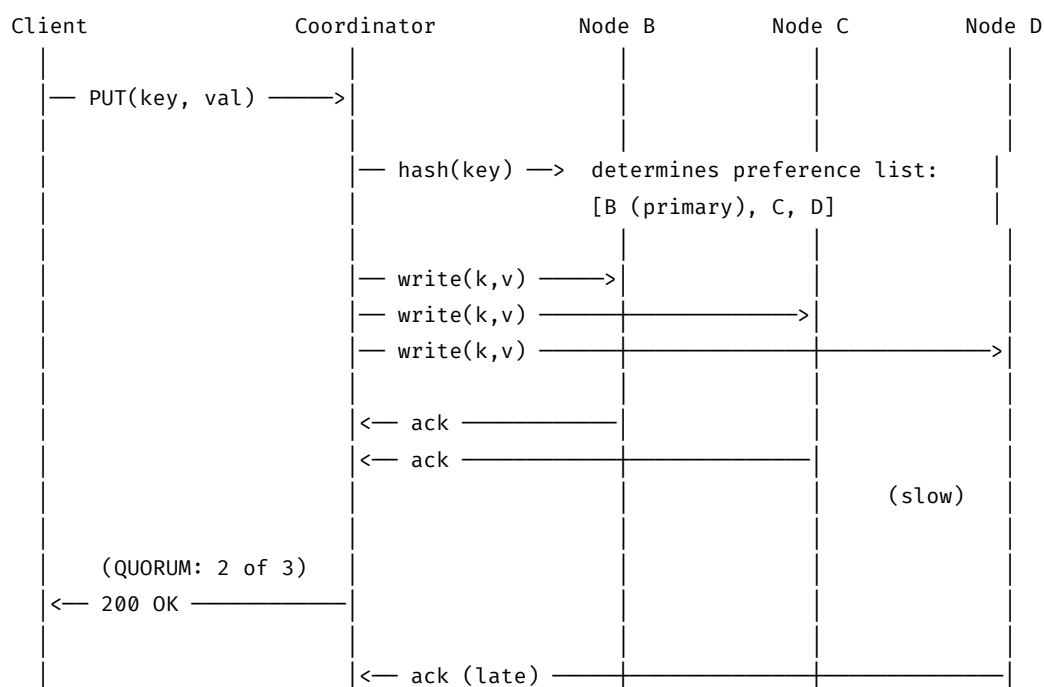


## 14.8 7. High-Level Architecture





### 14.8.1 Request Flow (Write)



## 14.9 8. Deep Dive

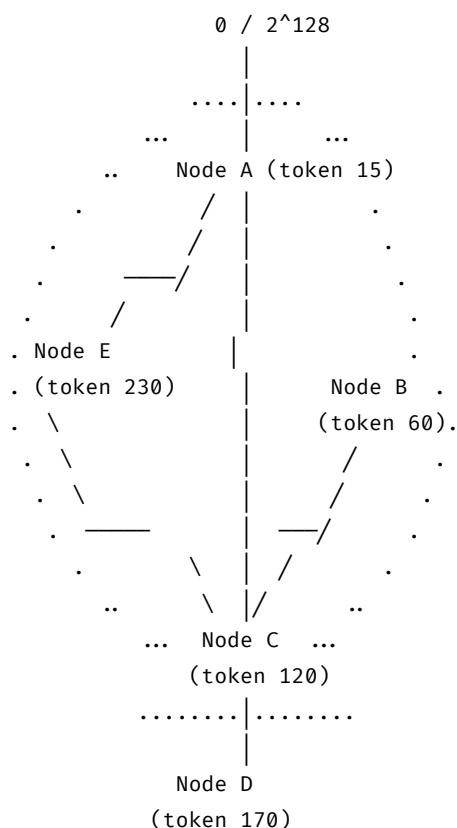
### 14.9.1 8.1 Consistent Hashing with Virtual Nodes

#### 14.9.1.1 The Problem with Naive Hashing

With  $\text{node} = \text{hash}(\text{key}) \% N$ , adding or removing a node changes the modulo, causing almost all keys to be remapped. This is catastrophic for a distributed store.

#### 14.9.1.2 Consistent Hash Ring

Map both keys and nodes onto the same hash ring (0 to  $2^{128} - 1$ ). A key is assigned to the first node encountered clockwise on the ring.



Key hash = 45 → walks clockwise → lands on Node B (token 60)  
 Key hash = 130 → walks clockwise → lands on Node D (token 170)  
 Key hash = 200 → walks clockwise → lands on Node E (token 230)

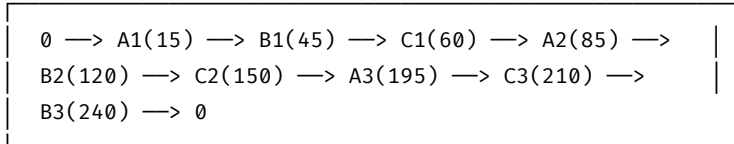
#### 14.9.1.3 Virtual Nodes (VNodes)

A single physical node owns **multiple tokens** (virtual nodes) on the ring. This provides:

1. **Even load distribution** – prevents hotspots from uneven token spacing.
2. **Smooth rebalancing** – adding a node steals small ranges from many nodes instead of one large range from one node.
3. **Heterogeneous hardware** – powerful nodes get more vnodes.

Physical Node A has vnodes: A1(15), A2(85), A3(195)  
 Physical Node B has vnodes: B1(45), B2(120), B3(240)  
 Physical Node C has vnodes: C1(60), C2(150), C3(210)

Ring with vnodes:

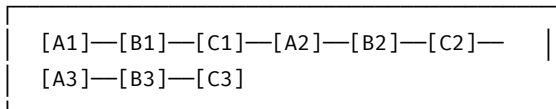


Recommended: 150-200 vnodes per physical node.  
 More vnodes = smoother distribution but more memory for ring metadata.

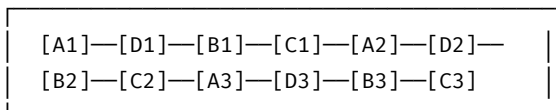
#### 14.9.1.4 Token Assignment and Rebalancing

When a new node joins:

Before (3 nodes, 3 vnodes each = 9 tokens):



Node D joins with 3 vnodes:



D1 steals a portion of B1's range  
 D2 steals a portion of B2's range  
 D3 steals a portion of B3's range

Only  $\sim 1/N$  of the data is moved (where  $N$  is the number of nodes).  
 Data streams from existing owners to D in the background.

### 14.9.2 8.2 Quorum Consensus

#### 14.9.2.1 N, W, R Parameters

$N$  = Number of replicas (typically 3)  
 $W$  = Write quorum (replicas that must ack a write)  
 $R$  = Read quorum (replicas that must respond to a read)

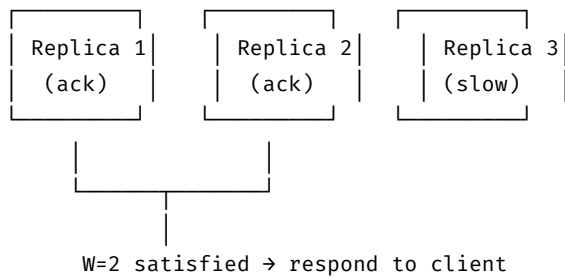
Strong consistency guarantee:  $R + W > N$

**Common configurations:**

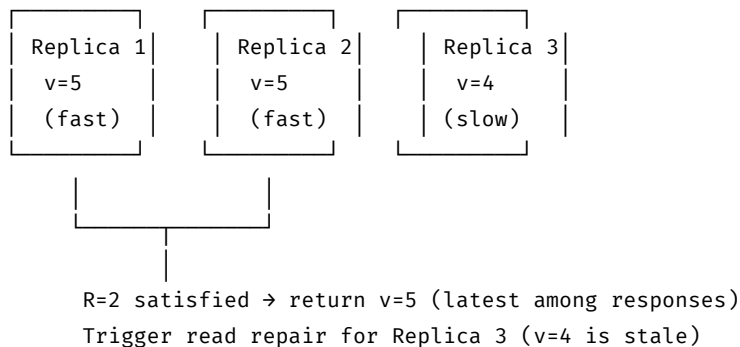
- **Strong read:**  $W: 1; R: 3$ ; Guarantees: Latest value guaranteed on read; Use Case: Read-heavy, stale OK on write
- **Strong write:**  $W: 3; R: 1$ ; Guarantees: All replicas have latest; Use Case: Write-heavy
- **Balanced:**  $W: 2; R: 2$ ; Guarantees: Overlap guarantees freshness; Use Case: General purpose
- **Weak (fast):**  $W: 1; R: 1$ ; Guarantees: No consistency guarantee; Use Case: Metrics, logs

Example:  $N=3$ ,  $W=2$ ,  $R=2$

Write to key K:



Read of key K:



### 14.9.2.2 Sloppy Quorum and Hinted Handoff

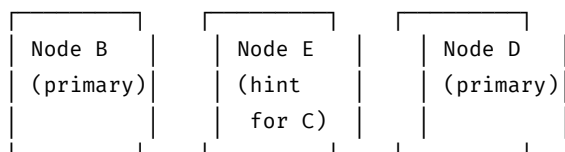
When a node is down, strict quorum would reduce availability. **Sloppy quorum** allows the coordinator to use the next healthy node on the ring as a temporary stand-in.

Preference list for key K: [B, C, D]

Node C is DOWN.

Strict quorum: Only B and D can participate → must wait for both.

Sloppy quorum: Use E as a stand-in for C.



Node E stores the value with a "hint" metadata:

```
{
 "intended_recipient": "C",
 "key": "K",
 "value": "...",
 "vector_clock": "... "
}
```

When C comes back online:

1. E detects C is up (via gossip)
2. E ships all hinted data to C
3. E deletes the hints

### 14.9.3 8.3 Conflict Resolution

#### 14.9.3.1 Vector Clocks

A vector clock is a list of (node, counter) pairs. Every time a node updates a key, it increments its own counter. Vector clocks establish a **partial ordering** of events.

Initial state: key K does not exist.

Step 1: Client writes to Node A

```
K = "value1"
VC = { A: 1 }
```

Step 2: Client reads from A, gets VC={A:1}, then writes to Node A

```
K = "value2"
VC = { A: 2 }
```

← A:1 happened-before A:2  
(no conflict, A:2 supersedes)

Step 3: Network partition! Two clients write concurrently:

Client X writes to Node A:  
(based on VC={A:2})

```
K = "value3a"
VC = { A: 3 }
```

Client Y writes to Node B:  
(based on VC={A:2})

```
K = "value3b"
VC = { A: 2, B: 1 }
```

Neither version dominates the other:

- {A:3} vs {A:2, B:1}: A is higher in first, B is higher in second.
- These are CONCURRENT → CONFLICT!

Step 4: Client reads key K, receives BOTH versions (siblings):

```
values: ["value3a", "value3b"]
contexts: [{A:3}, {A:2, B:1}]
```

Step 5: Client resolves conflict (application-specific merge):

Writes merged value with combined context:

```
K = "merged_value"
VC = { A: 3, B: 1 } ← merge of both
(written via Node A → { A: 4, B: 1 })
```

#### Comparing Vector Clocks:

```
VC1 = {A:2, B:3, C:1}
VC2 = {A:2, B:2, C:1}
```

For each node:

```
A: 2 ≥ 2 ✓
B: 3 ≥ 2 ✓
C: 1 ≥ 1 ✓
```

VC1 dominates VC2 → VC2 is stale, discard it.

```
VC1 = {A:3, B:1}
VC2 = {A:2, B:2}
```

A: 3 > 2 (VC1 wins)

B: 1 < 2 (VC2 wins)

Neither dominates → CONCURRENT → conflict!

### 14.9.3.2 Last-Write-Wins (LWW)

A simpler alternative: attach a physical timestamp to each write. The write with the highest timestamp wins. Used by Cassandra by default.

Write 1: K = "foo", timestamp = 1712678400000

Write 2: K = "bar", timestamp = 1712678400001 ← wins (higher timestamp)

Pros: Simple, no sibling explosion, no client-side resolution needed.

Cons: Clock skew can cause data loss. Write 1 may have been the "correct" value but loses due to clock difference.

### 14.9.3.3 Application-Level Resolution

For use cases like shopping carts, the application performs a **semantic merge**:

Cart version A: {item1, item2, item3} (user added item3)

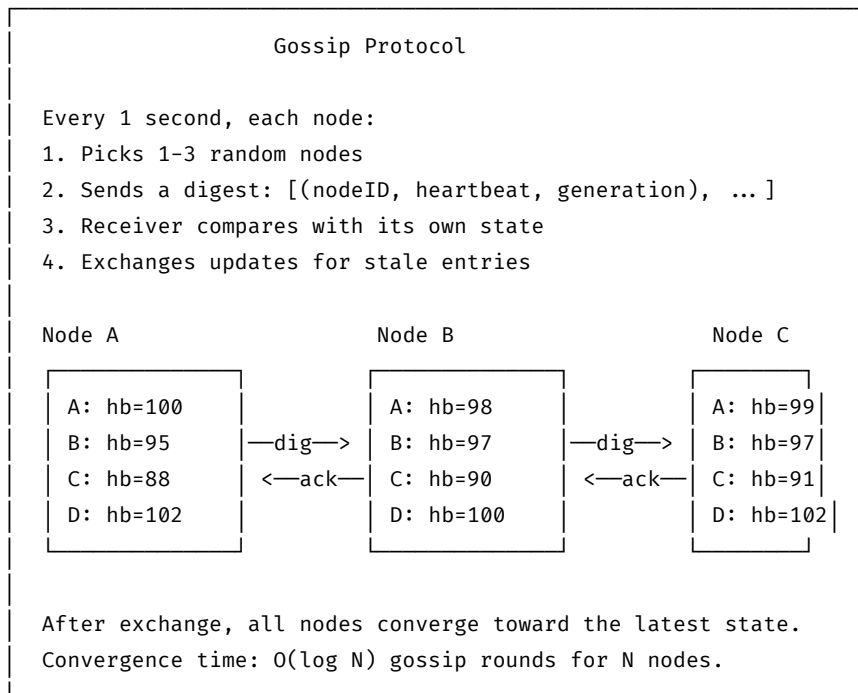
Cart version B: {item1, item2, item4} (user added item4)

Application merge: {item1, item2, item3, item4} (union)

## 14.9.4 8.4 Failure Detection

### 14.9.4.1 Gossip Protocol

Every node periodically exchanges state information with random peers. This decentralized approach has no single point of failure.



#### 14.9.4.2 Phi-Accrual Failure Detector

Instead of a binary “up/down” decision, the phi-accrual detector outputs a **suspicion level** ( $\phi$ ) based on the statistical distribution of heartbeat inter-arrival times.

$$\phi(t) = -\log_{10}(1 - F(t_{\text{now}} - t_{\text{last}}))$$

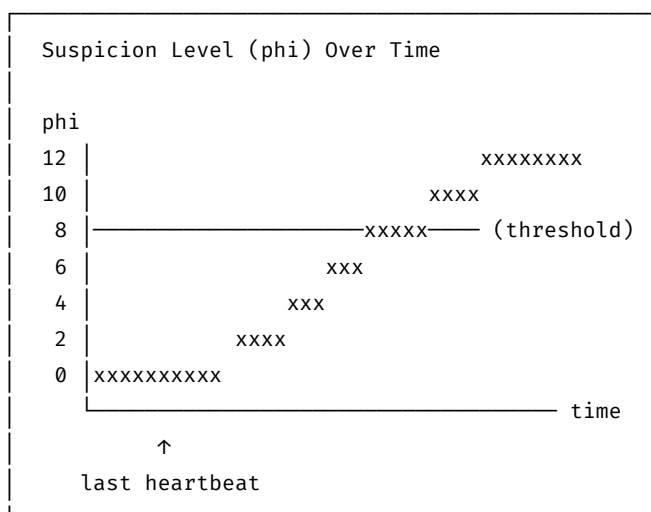
Where  $F$  is the CDF of the normal distribution fitted to historical inter-arrival times.

$\phi < 1$ : Normal heartbeat pattern. Node is definitely alive.

$\phi = 1-5$ : Getting suspicious. Heartbeat is late.

$\phi > 8$ : Very likely down (convict threshold, configurable).

$\phi > 12$ : Almost certainly down.

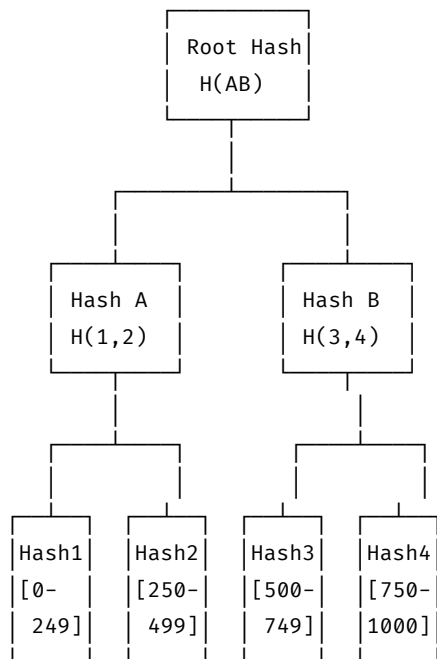




### 14.9.4.3 Merkle Trees for Anti-Entropy

Each node maintains a **Merkle tree** (hash tree) per key range. By comparing root hashes, two nodes can quickly identify which key ranges have diverged, then synchronize only the differing ranges.

Merkle Tree for key range  $[0, 1000]$ :



Node X's tree:

Root: abc123

```

├── A: def456
│ ├── 1: aaa
│ └── 2: bbb
└── B: ghi012
 ├── 3: ccc
 └── 4: ddd

```

Node Y's tree:

Root: abc789 ← DIFFERENT!

```

├── A: def456 ← same
│ ├── 1: aaa
│ └── 2: bbb
└── B: ghi345 ← DIFFERENT!
 ├── 3: ccc ← same
 └── 4: eee ← DIFFERENT!

```

Only range  $[750-1000]$  needs synchronization!

Compared  $O(\log N)$  hashes instead of scanning all keys.

#### 14.9.4.4 Hinted Handoff (Detailed)

Timeline of hinted handoff:

```
t=0: Node C goes down.
t=0+: Coordinator detects C is unresponsive (via gossip / phi detector).
t=0+: Writes destined for C are sent to Node E instead (sloppy quorum).
t=0+: E stores hints in a local hints directory:
 /hints/node_c/key1.hint
 /hints/node_c/key2.hint

t=30m: Node C comes back online.
t=30m: Gossip propagates C's liveness to all nodes.
t=31m: E detects C is alive, begins shipping hints:
 - Reads each hint file
 - Sends (key, value, vector_clock) to C
 - Deletes hint file after C acknowledges
```

Hint structure:

```
target_node: C
key: "user:12345"
value: <bytes>
vector_clock: {A:3, B:1}
timestamp: 1712678400000
ttl_remaining: 85400
```

Hints are expired after a configurable window (e.g., 3 hours).

If C is down longer, anti-entropy (Merkle trees) handles repair.

## 14.10 9. Data Partitioning

### 14.10.1 Consistent Hash Ring with Token Ranges

The entire key space is divided into **token ranges** assigned to virtual nodes.

Hash ring with tokens 0 to 360 (simplified):

|        |    |    |     |     |     |     |       |
|--------|----|----|-----|-----|-----|-----|-------|
| Token: | 0  | 60 | 120 | 180 | 240 | 300 | 360/0 |
| Range: |    |    |     |     |     |     |       |
|        | A1 | B1 | C1  | A2  | B2  | C2  |       |
| Owner: | A  | B  | C   | A   | B   | C   |       |

Key "user:12345" → hash = 145 → falls in range [120, 180) → owned by A2 (Node A)

Replicated to next 2 nodes on ring: B2 (Node B), C2 (Node C)

Preference list: [A, B, C]

### 14.10.2 Virtual Node Configuration

Recommended: 150–200 vnodes per physical node.

| Node Capacity  | VNodes | Rationale                |
|----------------|--------|--------------------------|
| Small (4 CPU)  | 100    | Proportional to capacity |
| Medium (8 CPU) | 150    | Standard configuration   |
| Large (16 CPU) | 200    | More tokens = more data  |

Total tokens in cluster:  $N_{\text{nodes}} * \text{vnodes\_per\_node}$

Example: 100 nodes \* 150 vnodes = 15,000 tokens

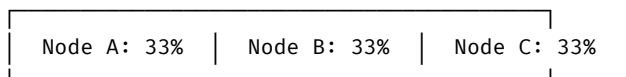
Memory per node for ring metadata:

15,000 tokens \* 32 bytes (token + node ID) = ~480 KB (negligible)

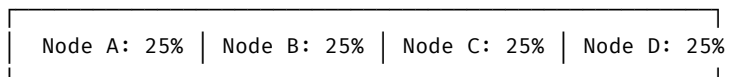
### 14.10.3 Range Ownership and Rebalancing

When a node is added or removed:

Before: 3 nodes, each owns ~33% of the ring.



After adding Node D: each node owns ~25%.



Node D's vnodes are interleaved, so it takes ~8% from each of A, B, C.

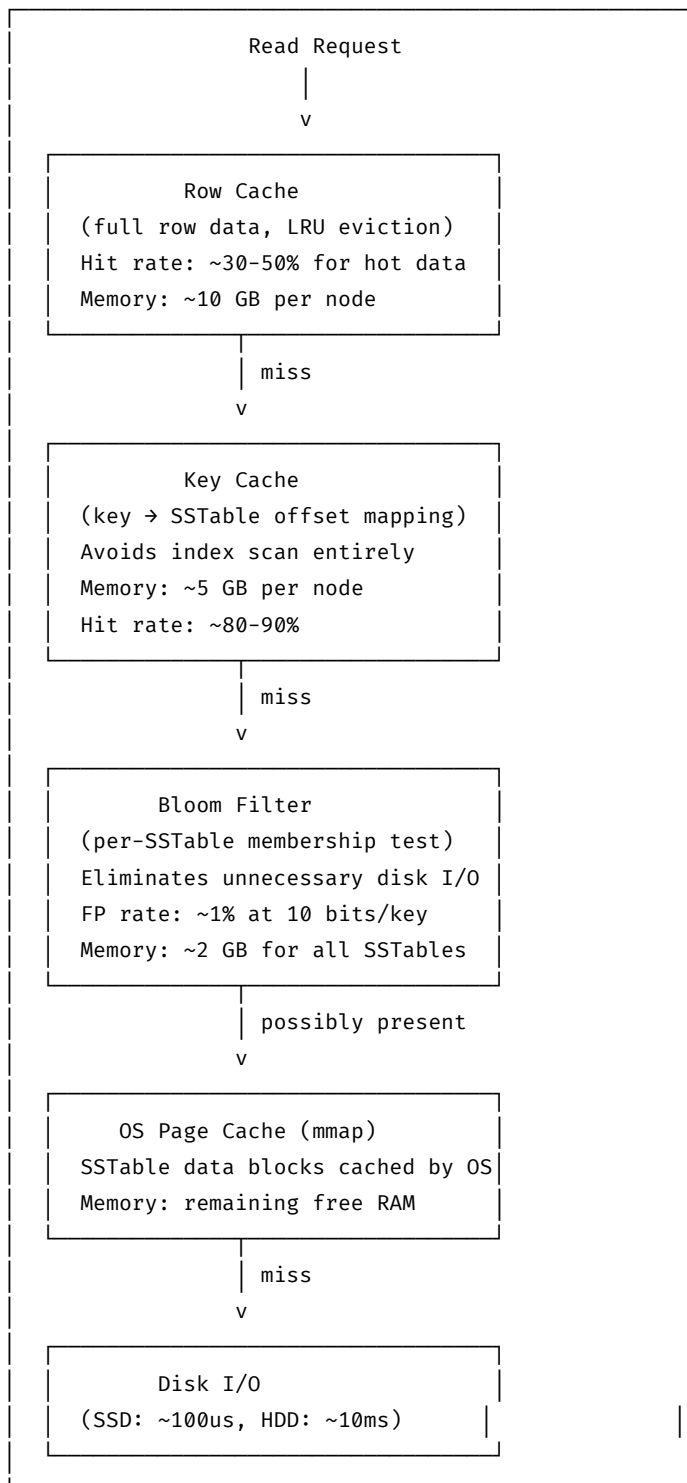
Data streams in the background. Reads/writes continue during rebalancing.

Rebalancing strategy:

1. New node announces itself via gossip.
2. Cluster assigns tokens to the new node.
3. Existing nodes stream data for the new node's token ranges.
4. Once streaming completes, the new node starts serving reads.
5. Old replicas for transferred ranges are eventually garbage collected.

## 14.11 10. Caching

### 14.11.1 Multi-Layer Caching Architecture



### 14.11.2 Row Cache

- Stores deserialized row data in memory.
- Best for **read-heavy, small, hot** datasets (e.g., user sessions).
- Invalidated on writes to the same key.
- NOT recommended for write-heavy workloads (frequent invalidation).

14.11.3 Key Cache

- Maps partition key to SSTable file + byte offset.
- Allows direct seeks, bypassing the SSTable index block entirely.
- Very high hit rate because key access patterns are often skewed (Zipfian).
- Populated on reads, saved to disk on graceful shutdown for warm restarts.

14.11.4 Bloom Filter Tuning

| Bits per key | Hash functions | FP Rate | Memory / 1M keys |
|--------------|----------------|---------|------------------|
| 5            | 3              | ~10%    | 625 KB           |
| 10           | 7              | ~1%     | 1.25 MB          |
| 15           | 10             | ~0.1%   | 1.87 MB          |
| 20           | 14             | ~0.01%  | 2.50 MB          |

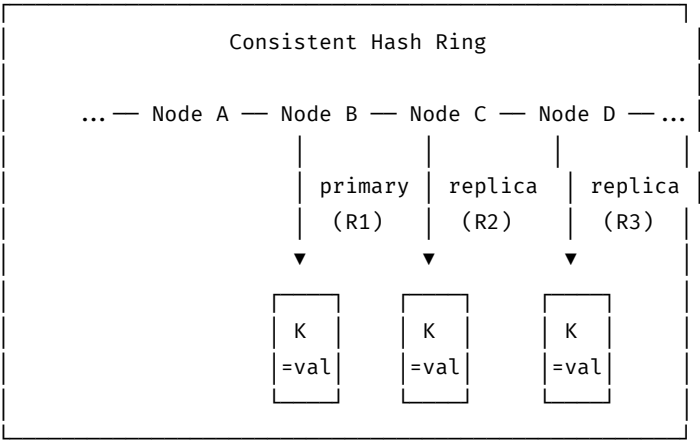
Recommendation: 10 bits/key (1% FP) is the sweet spot.  
At 1% FP rate with 100 SSTables, expected false disk reads per query: 1.

14.12 11. Replication and Consistency

14.12.1 Replication Strategy

Replication Factor (N) = 3

For key K with primary token on Node B:

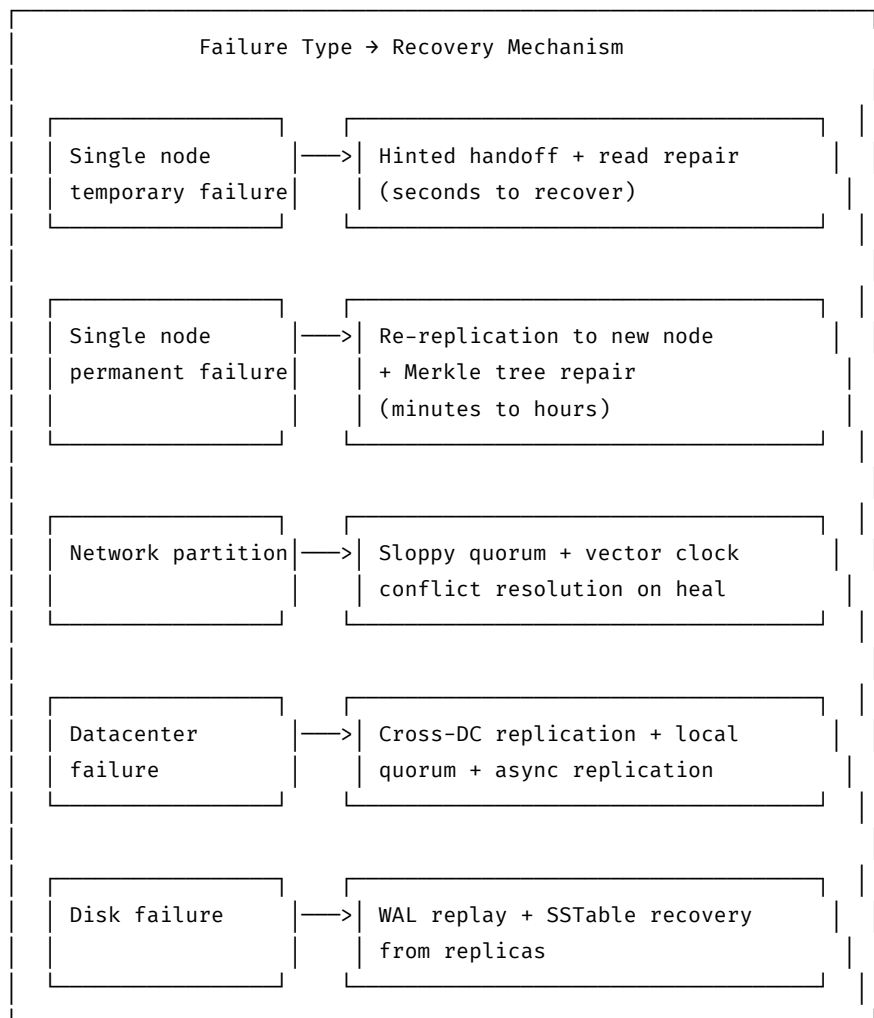


The "preference list" for K is [B, C, D].  
Skips duplicate physical nodes (if B and C are vnodes on the same machine, skip to the next distinct physical node).



## 14.13 12. Fault Tolerance

### 14.13.1 Failure Scenarios and Recovery Mechanisms



### 14.13.2 Hinted Handoff (Recovery Flow)

Normal state: Write for key  $K \rightarrow [B, C, D]$

Node C fails: Write for key  $K \rightarrow [B, E(\text{hint for } C), D]$

C recovers: E detects C alive  $\rightarrow$  ships hints  $\rightarrow$  C catches up  
Hint TTL = 3 hours (configurable)

If C is down  $> 3h$ : Hints expire. Anti-entropy (Merkle trees) handles the remaining drift when C comes back.

### 14.13.3 Read Repair (Consistency Recovery)

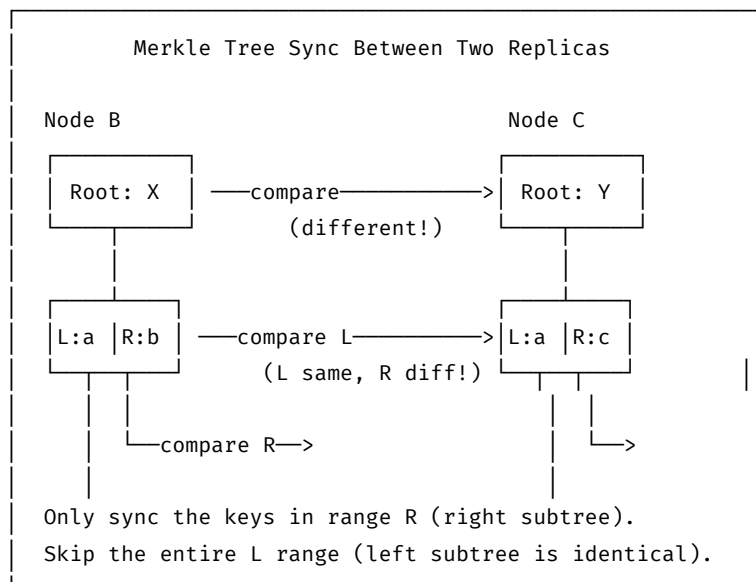
Read repair runs during normal read operations:

- Step 1: Coordinator reads from R replicas.
- Step 2: Compare versions from all R responses.
- Step 3: If any replica is behind:
  - Return the latest version to the client.
  - Asynchronously send the latest version to stale replicas.
- Step 4: Stale replicas update their local storage.

Read repair probability: 10% (configurable).

This means 1 in 10 reads will trigger a version comparison across ALL N replicas (not just R). This is called "full read repair."

### 14.13.4 Merkle Tree Synchronization (Anti-Entropy)



### 14.13.5 Re-Replication on Permanent Failure

Node C is declared permanently dead (after N hours of being down).

Before:

Key K replicas: [B, C, D]

Action:

1. Gossip marks C as permanently removed.
2. Token ranges owned by C are redistributed to other nodes.
3. For each range formerly on C:
  - The remaining replicas (B, D) stream data to the new owner (e.g., E).
4. Once streaming completes, E assumes responsibility for C's ranges.

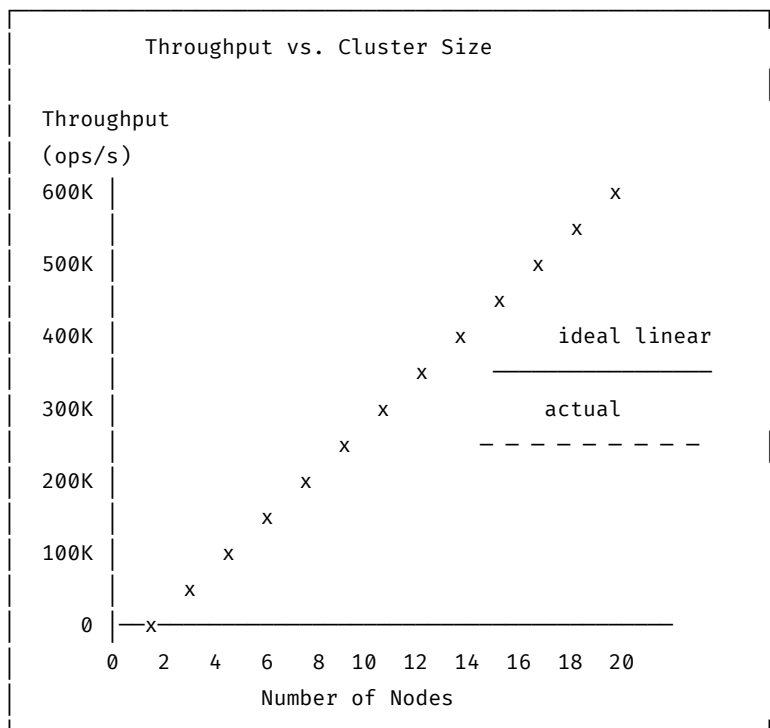
After:

Key K replicas: [B, E, D]      (E replaces C)



## 14.14 13. Scalability

### 14.14.1 Linear Scaling



Near-linear scaling because:

- No central coordinator or master node.
- Each node handles its own token ranges independently.
- Cross-node communication is limited to:
  - \* Coordinator  $\rightarrow$  replica (per-request,  $O(N_{\text{replicas}}) = O(3)$ )
  - \* Gossip ( $O(\log N)$  convergence)
  - \* Anti-entropy (background, per-range)

### 14.14.2 Auto Rebalancing

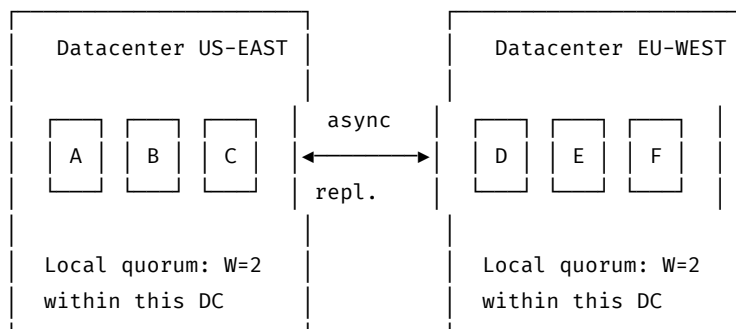
Adding a node:

1. Bootstrap: New node contacts a seed node.
  2. Token Assignment: New node claims tokens.
  3. Data Streaming: Existing nodes stream data for the new node's ranges.
  4. Join: New node starts serving requests.
  5. Cleanup: Source nodes remove migrated data.
- During streaming:
- Reads: Served by old owners until streaming done.
  - Writes: Dual-written to old + new owner.
  - Zero downtime. No manual intervention.

Removing a node:

1. Decommission: Node announces departure via gossip.
2. Streaming: Departing node streams all its data to the new owners for each range.
3. Token Release: Tokens are released and redistributed.
4. Departure: Node leaves the cluster cleanly.

### 14.14.3 Cross-Datcenter Replication

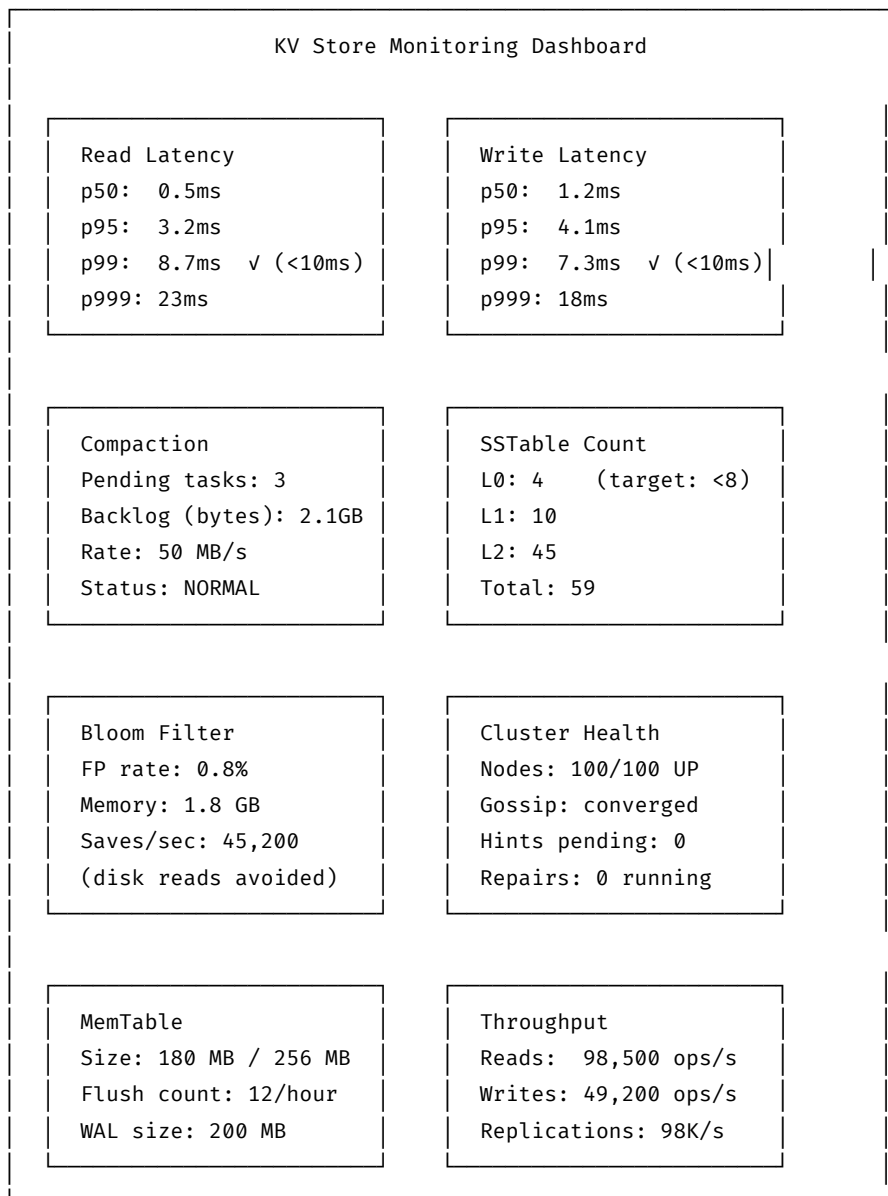


Strategy: LOCAL\_QUORUM + async cross-DC replication

- Writes ack after local quorum (fast, <10ms).
- Async replication to remote DC (50-200ms).
- Conflicts resolved via vector clocks or LWW.
- Each DC can serve reads independently (low latency).
- DC failure: other DC takes over with slight staleness.

## 14.15 14. Monitoring

### 14.15.1 Key Metrics Dashboard



### 14.15.2 Critical Alerts

- **Read p99 latency:** Warning Threshold: > 10ms; Critical Threshold: > 50ms
- **Write p99 latency:** Warning Threshold: > 10ms; Critical Threshold: > 50ms
- **Compaction backlog:** Warning Threshold: > 10 GB; Critical Threshold: > 50 GB
- **L0 SSTable count:** Warning Threshold: > 8; Critical Threshold: > 32 (stall writes)
- **Bloom filter FP rate:** Warning Threshold: > 5%; Critical Threshold: > 10%
- **Hints pending:** Warning Threshold: > 10,000; Critical Threshold: > 100,000
- **Node down duration:** Warning Threshold: > 5 minutes; Critical Threshold: > 30 minutes
- **Disk utilization:** Warning Threshold: > 70%; Critical Threshold: > 85%
- **MemTable flush latency:** Warning Threshold: > 5 seconds; Critical Threshold: > 30 seconds
- **Tombstone ratio:** Warning Threshold: > 20% of reads; Critical Threshold: > 50% of reads

### 14.15.3 Operational Runbooks

**High L0 SSTable Count:** 1. Check compaction throughput – is it keeping up? 2. Increase compaction threads or I/O priority. 3. If sustained, consider throttling writes temporarily. 4. Check for wide partitions causing slow compaction.

**High Bloom Filter FP Rate:** 1. Check bits-per-key setting (should be  $\geq 10$ ). 2. Rebuild Bloom filters during next compaction. 3. Consider if dataset characteristics have changed.

**Growing Hints Queue:** 1. Identify the target node – is it truly down or just slow? 2. If down, monitor for auto-recovery. 3. If hint TTL is approaching, prepare for anti-entropy repair.

## 14.16 15. Trade-offs

### 14.16.1 AP vs CP

AP (Dynamo-style):

- + Always writable, even during partitions
  - + Lower latency (no distributed consensus)
  - + Better availability
  - May return stale data
  - Conflict resolution complexity
  - Harder to reason about correctness
- Use: Shopping carts, session stores, user preferences, metrics

CP (Raft/Paxos-based):

- + Strong consistency (linearizable)
  - + No conflicts to resolve
  - + Easier to program against
  - Unavailable during partitions
  - Higher latency (consensus rounds)
  - Leader bottleneck
- Use: Financial transactions, inventory, coordination, distributed locks

### 14.16.2 Vector Clocks vs Last-Write-Wins (LWW)

- **Data loss:** Vector Clocks: None (preserves all versions); LWW: Possible (clock skew)
- **Client complexity:** Vector Clocks: High (must resolve siblings); LWW: None
- **Storage overhead:** Vector Clocks: VC metadata per key (grows); LWW: Single timestamp
- **Correctness:** Vector Clocks: Causally correct; LWW: Depends on clock accuracy
- **Sibling explosion:** Vector Clocks: Possible (mitigated by pruning); LWW: N/A
- **Clock dependency:** Vector Clocks: Logical (no physical clocks); LWW: Physical clocks (NTP)

- **Used by:** Vector Clocks: Riak, original Dynamo; LWW: Cassandra, ScyllaDB

**Vector Clock Pruning:** To prevent unbounded growth, prune entries older than a threshold (e.g., remove entries for nodes not seen in 7 days). Trades correctness for bounded metadata size.

### 14.16.3 LSM Tree vs B-Tree

- **Write throughput:** LSM Tree: Very high (sequential I/O); B-Tree: Lower (random I/O)
- **Read throughput:** LSM Tree: Lower (check multiple levels); B-Tree: Higher (single lookup path)
- **Write amplification:** LSM Tree: Higher (10-30x from compaction); B-Tree: Lower (2-3x)
- **Space amplification:** LSM Tree: Lower (after compaction); B-Tree: Higher (~50% page utilization)
- **Predictable latency:** LSM Tree: No (compaction spikes); B-Tree: Yes (consistent performance)
- **Range scans:** LSM Tree: Good (sorted SSTables); B-Tree: Excellent (sorted leaves)
- **Concurrency:** LSM Tree: Good (lock-free memtable); B-Tree: Complex (page-level locks)
- **Recovery:** LSM Tree: WAL replay; B-Tree: WAL + partial page repair
- **Used by:** LSM Tree: Cassandra, RocksDB, LevelDB; B-Tree: PostgreSQL, MySQL InnoDB

**When to choose LSM:** Write-heavy workloads (>50% writes), append-heavy patterns, time-series data. The Dynamo-style KV store favors LSM because it prioritizes write availability and throughput.

**When to choose B-Tree:** Read-heavy workloads, range scans, need for predictable latency, transactional workloads with MVCC.

### 14.16.4 Compaction Strategies Comparison

- **Size-Tiered:** Write Amp.: Low; Space Amp.: High (2x); Read Amp.: High; Best For: Write-heavy workloads
- **Leveled:** Write Amp.: High; Space Amp.: Low (1.1x); Read Amp.: Low; Best For: Read-heavy workloads
- **Time-Window:** Write Amp.: Low; Space Amp.: Low; Read Amp.: Low; Best For: Time-series with TTL
- **FIFO:** Write Amp.: None; Space Amp.: None; Read Amp.: Low; Best For: Cache-like workloads

## 14.17 16. Interview Questions

### 14.17.1 Q1: How do you handle hotspot keys?

**A:** Hotspot keys (e.g., a viral post's like counter) overwhelm a single partition. Solutions: 1. **Key salting / bucketing:** Append a random suffix (e.g., `hot_key_0` through `hot_key_99`) to distribute across 100 partitions. Reads must fan out to all 100 and aggregate. 2. **Local aggregation:** Buffer writes in the coordinator and flush periodically. 3. **Dedicated cache tier:** Put hot keys in a Redis/Memcached layer in front of the KV store. 4. **Read replicas:** For read-hot keys, add more read-only replicas for that partition.

### 14.17.2 Q2: What happens during a network partition?

**A:** With sloppy quorum, both sides of the partition continue serving reads and writes. Writes on each side use hinted handoff or local replicas. When the partition heals: 1. Gossip reconnects the two

halves. 2. Anti-entropy (Merkle trees) identifies divergent keys. 3. Vector clocks determine causal ordering. 4. Concurrent writes (neither dominates) create siblings that the application must resolve, or LWW picks the highest timestamp.

### 14.17.3 Q3: Explain vector clocks and when they produce conflicts.

**A:** A vector clock is a list of (node, counter) pairs. On every write, the writing node increments its counter. Two vector clocks are compared element-wise: - If VC1 dominates VC2 (every component  $\geq$  and at least one  $>$ ), VC2 is an ancestor – discard VC2. - If neither dominates, the writes are concurrent – they represent a conflict (sibling values).

Conflicts happen when two clients write to different replicas without seeing each other's writes (e.g., during a partition or race condition).

### 14.17.4 Q4: How does read repair work? When is it insufficient?

**A:** On a read, the coordinator compares versions from R replicas. If a replica has a stale version, the coordinator asynchronously sends the latest version. Read repair is insufficient for: - Keys that are **never read** (cold data) – use anti-entropy for these. - **High-write, low-read** workloads – divergence accumulates faster than reads repair it. - **Tombstone propagation** – a delete may not reach all replicas via read repair alone.

### 14.17.5 Q5: How does LSM differ from B-Tree for this use case?

**A:** LSM converts random writes to sequential I/O (write to MemTable, flush to sorted SSTables). This is ideal for a Dynamo-style store because: - Writes are the critical path (always-on availability). - Sequential I/O is 100x faster than random I/O on HDDs, 5x on SSDs. - Trade-off: reads may need to check multiple SSTables, mitigated by Bloom filters and caching.

B-Trees update in-place (random I/O), which is better for read-heavy workloads with predictable latency requirements (e.g., OLTP databases).

### 14.17.6 Q6: How do you handle deletes in a distributed KV store?

**A:** Deletes are tombstones (a marker with a timestamp/vector clock). The tombstone must propagate to all replicas before being garbage collected. `gc_grace_seconds` (default: 10 days) defines how long tombstones are kept. If a node is down longer than `gc_grace_seconds`, it may resurrect deleted data when it comes back – this is the **zombie data** problem. Mitigation: run anti-entropy repair before rejoining a long-absent node.

### 14.17.7 Q7: What is the impact of compaction on latency?

**A:** Compaction is I/O intensive and can cause latency spikes: - **Read latency:** During compaction, disk bandwidth is shared, increasing read times. - **Write stalls:** If L0 SSTable count exceeds a threshold (e.g., 32), writes are throttled or stalled until compaction catches up.

Mitigations: - Rate-limit compaction I/O. - Use separate disks for compaction output. - Prioritize L0 compaction (most impact on read performance). - Use leveled compaction for read-heavy workloads (fewer SSTables to check).

### 14.17.8 Q8: How do you tune N, W, R for different use cases?

**A:** - **Session store (availability > consistency):** N=3, W=1, R=1. Fastest, but may read stale sessions. - **User profiles (balanced):** N=3, W=2, R=2. Strong consistency ( $R+W=4 > N=3$ ), tolerates

1 failure. - **Financial ledger (strong consistency):**  $N=3$ ,  $W=3$ ,  $R=1$  or  $W=2$ ,  $R=2$ . No stale reads, but any node failure blocks writes ( $W=3$ ) or reduces availability. - **Analytics/metrics (high write throughput):**  $N=3$ ,  $W=1$ ,  $R=1$ . Eventual consistency is fine for aggregated metrics.

#### 14.17.9 Q9: How does consistent hashing handle heterogeneous hardware?

**A:** Assign more virtual nodes to more powerful machines. A node with 2x the CPU and storage gets 2x the vnodes, and thus owns 2x the token ranges. This is more flexible than fixed partitioning because:

- Adding a large node automatically takes proportionally more load.
- Rebalancing is granular (moves vnodes, not entire ranges).

#### 14.17.10 Q10: What is a sloppy quorum and why is it important?

**A:** A strict quorum requires exactly the nodes in the preference list to respond. A sloppy quorum allows the coordinator to use any  $N$  healthy nodes (not just the designated replicas) to meet the  $W$  or  $R$  requirement. This is critical for availability: without sloppy quorum, a single node failure could make a partition unavailable even though other healthy nodes exist. The trade-off is temporary inconsistency (the hint-holding node is not the designated replica).

#### 14.17.11 Q11: How do Merkle trees help with anti-entropy?

**A:** Merkle trees are hash trees where leaf nodes hash individual keys or key ranges and internal nodes hash their children. Two replicas compare root hashes:

- Same root = entire range is in sync ( $O(1)$  check).
- Different root = descend into children to find divergent subtrees.
- Only the differing leaf ranges need key-by-key comparison and repair.

This reduces the amount of data transferred from  $O(N)$  to  $O(D * \log N)$ , where  $D$  is the number of differences and  $N$  is the total number of keys.

#### 14.17.12 Q12: How do you handle cross-datacenter replication?

**A:** Use `LOCAL_QUORUM` for writes (ack after quorum within the local DC) plus asynchronous replication to remote DCs. This ensures:

- Low write latency (no cross-DC round trip in the critical path).
- Each DC can serve reads independently.
- Conflict resolution via vector clocks or LWW when DCs reconnect.
- DC failure: the other DC has a slightly stale but complete copy.

For stronger consistency across DCs, use `EACH_QUORUM` (quorum in every DC), but latency increases to cross-DC RTT (~50-200ms).

#### 14.17.13 Q13: What is the difference between hinted handoff and anti-entropy?

**A:**

- **Trigger:** Hinted Handoff: Node failure detected; Anti-Entropy: Periodic (scheduled)
- **Scope:** Hinted Handoff: Recent writes only; Anti-Entropy: Entire key ranges
- **Granularity:** Hinted Handoff: Individual keys; Anti-Entropy: Range-level (Merkle tree)
- **Time window:** Hinted Handoff: Short (hours); Anti-Entropy: Unlimited
- **Overhead:** Hinted Handoff: Low (just pending writes); Anti-Entropy: Higher (tree computation)
- **Speed:** Hinted Handoff: Fast (immediate on recovery); Anti-Entropy: Slow (background process)
- **Coverage:** Hinted Handoff: Incomplete (misses reads); Anti-Entropy: Complete

They are complementary: hinted handoff handles short outages quickly, anti-entropy handles long outages and data drift comprehensively.

**14.17.14 Q14: How do you prevent tombstone accumulation?**

**A:** Tombstone accumulation degrades read performance (reads must scan past tombstones). Prevention strategies: 1. **gc\_grace\_seconds:** Tombstones are garbage collected after this period during compaction. Set to slightly longer than your longest acceptable node downtime. 2. **Compaction:** Ensure compaction runs regularly and is not backlogged. 3. **TTL instead of delete:** Use TTL for data that naturally expires (e.g., sessions). TTL expiration produces fewer tombstones because the data and tombstone are in the same SSTable (time-window compaction). 4. **Avoid range deletes:** Deleting 1 million keys in a range creates 1 million tombstones. Instead, use TTL or partition-level drops.

**14.17.15 Q15: How would you migrate from a single-node KV store to this distributed design?**

**A:** Migration strategy: 1. **Dual-write phase:** Write to both old and new systems. Read from old. 2. **Backfill:** Scan the old store and write all existing data to the new cluster. Use background streaming to avoid overloading the new cluster. 3. **Validation:** Compare random samples between old and new stores. Use checksums for full validation. 4. **Shadow read phase:** Read from both systems, compare results, log discrepancies. Serve from old system. 5. **Cutover:** Switch reads to the new system. Keep the old system as fallback. 6. **Decommission:** After a confidence period (1-2 weeks), decommission the old store.

Key considerations: - Handle schema differences (add vector clocks, partition keys). - Set initial replication factor and consistency levels. - Pre-split tokens based on existing key distribution to avoid hotspots. - Monitor latency and error rates during each phase.

---



## 14.18 Summary

### Dynamo-Style KV Store Summary

#### Core Principles:

- AP over CP (always writable)
- Consistent hashing + virtual nodes (data distribution)
- Tunable consistency (ONE / QUORUM / ALL)
- LSM tree storage (write-optimized)
- Decentralized (no master, gossip protocol)

#### Key Mechanisms:

- Quorum (N/W/R) for consistency control
- Vector clocks / LWW for conflict resolution
- Gossip + phi-accrual for failure detection
- Hinted handoff for short-term failure recovery
- Merkle trees for anti-entropy repair
- Read repair for passive consistency convergence
- Bloom filters for read optimization
- Compaction for space reclamation

#### Scale:

- 150K ops/sec (100K reads + 50K writes)
- ~1000 nodes
- Petabytes of storage
- < 10ms p99 latency
- 99.99% availability

*References: Amazon Dynamo (DeCandia et al., 2007), Apache Cassandra Architecture, Google Bigtable, Facebook's RocksDB, LevelDB documentation.*

## Chapter 15

# Design a Metrics and Monitoring System (Datadog / Prometheus)

### 15.1 Table of Contents

1. Problem Statement
  2. Functional Requirements
  3. Non-Functional Requirements
  4. Back-of-Envelope Estimation
  5. API Design
  6. Data Model
  7. High-Level Architecture
  8. Deep Dive
    - 8.1 TSDB Storage Engine
    - 8.2 Ingestion Pipeline
    - 8.3 Query Engine
    - 8.4 Alerting Engine
  9. Partitioning Strategy
  10. Caching Strategy
  11. Replication
  12. Fault Tolerance
  13. Scalability
  14. Monitoring the Monitoring System (Meta)
  15. Trade-offs and Design Decisions
  16. Interview Questions and Answers
- 

### 15.2 1. Problem Statement

Modern distributed systems consist of thousands of microservices running across tens of thousands of hosts, containers, and serverless functions. Each component emits hundreds of metrics every few seconds – CPU utilization, request latency percentiles, error rates, queue depths, garbage collection pauses, disk IOPS, and custom business metrics. Operators need a unified system to:

- **Collect** billions of time-series data points per day from heterogeneous sources
- **Store** this data efficiently for both real-time queries and historical analysis
- **Query** across dimensions (service, host, region, version) with sub-second latency

- **Alert** on anomalies, threshold breaches, and SLO violations with minimal delay
- **Visualize** system health through dashboards that update in real time

The challenge lies in the extreme write volume (potentially millions of data points per second), the need for flexible multi-dimensional querying, long-term retention spanning years, and the requirement that the monitoring system itself must be more reliable than the systems it monitors.

### 15.2.1 Real-World Context

- **Prometheus:** Scale: Single-node TSDB, federation for scale; Approach: Pull-based, local storage
  - **Datadog:** Scale: Billions of pts/sec across all tenants; Approach: Push-based, cloud SaaS
  - **Thanos:** Scale: Long-term Prometheus with object store; Approach: Sidecar + compactor
  - **Cortex/Mimir:** Scale: Multi-tenant horizontally-scaled Prom; Approach: Shared-nothing ingesters
  - **InfluxDB:** Scale: Write-optimized columnar TSDB; Approach: Push, TSI index, TSM engine
  - **VictoriaMetrics:** Scale: Optimized single/cluster TSDB; Approach: Pull/push, vmselect/vmstorage
- 

## 15.3 2. Functional Requirements

### 15.3.1 Core Features

- **1 - Push ingestion:** Accept metrics via StatsD, OpenTelemetry, custom HTTP endpoints
- **2 - Pull ingestion:** Scrape Prometheus-format /metrics endpoints at configurable intervals
- **3 - Time-series storage:** Store (metric\_name, tags, timestamp, value) tuples efficiently
- **4 - PromQL-like queries:** rate(), avg\_over\_time(), histogram\_quantile(), aggregation by labels
- **5 - Dashboard creation:** Define panels with queries, layout, refresh intervals
- **6 - Alerting rules:** Define threshold/anomaly conditions, notification channels
- **7 - Tagging / labeling:** Arbitrary key-value labels on every metric for dimensional slicing
- **8 - Aggregation:** Pre-aggregate across hosts, regions, clusters at ingestion time
- **9 - Downsampling:** Reduce resolution for older data (10s -> 1m -> 5m -> 1h)
- **10 - Multi-tenancy:** Isolate metrics between teams/organizations

### 15.3.2 Out of Scope (for initial design)

- Log aggregation and full-text search (separate system like ELK)
  - Distributed tracing (separate system like Jaeger/Tempo)
  - Synthetic monitoring / uptime checks
  - AIOps / ML-based anomaly detection (can be layered on later)
- 

## 15.4 3. Non-Functional Requirements

- **Ingestion throughput:** 10M+ data points per second (cluster-wide)
- **Query latency:** < 1 second for dashboards over recent data (last 1-6 hours)
- **Query latency (long):** < 10 seconds for queries spanning days/weeks
- **Retention:** Raw: 15 days, 1m avg: 6 months, 1h avg: 3+ years
- **Availability:** 99.99% (52.6 min downtime/year) – must exceed monitored systems

- **Durability:** No data loss for committed writes (WAL + replication)
  - **Alert latency:** < 30 seconds from metric arrival to alert firing
  - **Cardinality:** Support 50M+ active time series per tenant
  - **Consistency:** Eventual consistency acceptable; prefer availability
- 

## 15.5 4. Back-of-Envelope Estimation

### 15.5.1 Ingestion Volume

Single cluster:

$1,000 \text{ hosts} \times 500 \text{ metrics/host} \times 1 \text{ sample/10s} = 50,000 \text{ data points/sec}$

Medium org (100 clusters):

$100 \times 50,000 = 5,000,000 \text{ data points/sec} = 5\text{M pts/sec}$

Large org / SaaS platform (10,000 clusters):

$10,000 \times 50,000 = 500,000,000 \text{ data points/sec} = 500\text{M pts/sec}$

### 15.5.2 Storage

Per data point (uncompressed):

|                   |                                         |
|-------------------|-----------------------------------------|
| metric_name hash: | 8 bytes                                 |
| tag set hash:     | 8 bytes                                 |
| timestamp:        | 8 bytes                                 |
| value (float64):  | 8 bytes                                 |
| overhead:         | ~88 bytes (labels, index entries, etc.) |
| Total:            | ~120 bytes/data point                   |

Daily raw volume (500M pts/sec):

$500\text{M} \times 86,400 \text{ sec} \times 120 \text{ bytes} = 5.18 \text{ PB/day (raw, uncompressed)}$

With Gorilla compression (~12x for timestamps + values):

~432 TB/day for values + timestamps

With delta encoding + dedup on tag indices:

~500 TB/day total (compressed)

15-day raw retention:

$500 \text{ TB} \times 15 = 7.5 \text{ PB}$

Downsampled (1m avg over 6 months):

$500\text{M pts/sec} / 6 (10\text{s} \rightarrow 1\text{m}) = 83\text{M pts/sec}$

$83\text{M} \times 86,400 \times 180 \text{ days} \times 20 \text{ bytes (compressed)} = \sim 26 \text{ PB}$

(But much smaller per-point after compression: ~2-3 PB)

### 15.5.3 Network Bandwidth

Ingestion at 500M pts/sec x 120 bytes:

60 GB/sec inbound = 480 Gbps

After batching and compression (10x reduction):

~48 Gbps inbound across all ingestion nodes

Typical query: scan 1M data points x 16 bytes = 16 MB

At 10,000 concurrent queries: 160 GB/sec read throughput

### 15.5.4 Node Count Estimates

TSDB storage nodes:

7.5 PB / 20 TB per node = 375 storage nodes (with replication: ~750-1125)

Ingestion nodes:

500M pts/sec / 2M pts/sec per node = 250 ingestion nodes

Query nodes:

10,000 QPS / 200 QPS per node = 50 query nodes

Kafka brokers:

48 Gbps / 2 Gbps per broker = 24+ brokers

---

## 15.6 5. API Design

### 15.6.1 5.1 Metrics Ingestion API

POST /api/v1/metrics

Content-Type: application/json

Authorization: Bearer <api\_key>

Request Body:

```

{
 "series": [
 {
 "metric": "system.cpu.utilization",
 "type": "gauge", // gauge | counter | histogram | summary
 "points": [
 [17000000000, 72.5], // [timestamp_epoch_sec, value]
 [17000000010, 74.1]
],
 "tags": {
 "host": "web-prod-001",
 "region": "us-east-1",
 "service": "api-gateway",
 "env": "production"
 },
 "interval": 10, // collection interval in seconds
 "unit": "percent"
 },
 {
 "metric": "http.request.duration",
 "type": "histogram",
 "points": [
 [17000000000, {"sum": 4520.5, "count": 1200, "buckets": {"0.1": 500, "0.5": 900, "1.0": 1100, "5.0": 1195, "+Inf": 1200}}],
 "tags": {
 "host": "web-prod-001",
 "method": "GET",
 "endpoint": "/api/users",
 "status_code": "200"
 }
 }
]
}

```

Response: 202 Accepted

```

{
 "status": "ok",
 "accepted": 2,
 "errors": []
}

```

## 15.6.2 5.2 Prometheus Remote Write (Pull-to-Push Bridge)

POST /api/v1/prom/write

Content-Type: application/x-protobuf

Content-Encoding: snappy

X-Prometheus-Remote-Write-Version: 0.1.0

Body: Snappy-compressed Protocol Buffer (prometheus.WriteRequest)

- Each TimeSeries has []Label and []Sample
- Label: name (string), value (string)
- Sample: value (float64), timestamp (int64 ms)

Response: 204 No Content

## 15.6.3 5.3 Query API

GET /api/v1/query

Parameters:

query: rate(http\_requests\_total{service="api",status=~"5.."}[5m])

time: 1700000000 // evaluation timestamp (optional, default=now)

GET /api/v1/query\_range

Parameters:

query: avg\_over\_time(system\_cpu\_utilization{region="us-east-1"}[5m])

start: 1700000000

end: 1700003600

step: 60 // resolution in seconds

Response:

```
{
 "status": "success",
 "data": {
 "resultType": "matrix",
 "result": [
 {
 "metric": {
 "__name__": "system_cpu_utilization",
 "host": "web-prod-001",
 "region": "us-east-1"
 },
 "values": [
 [1700000000, "72.5"],
 [1700000060, "73.2"],
 [1700000120, "71.8"]
]
 }
]
 }
}
```

### 15.6.4 5.4 Alerting API

POST /api/v1/alerts/rules

```
{
 "name": "HighErrorRate",
 "expr": "rate(http_requests_total{status=~'5..'}[5m]) / rate(http_requests_total[5m]) > 0.05",
 "for": "5m",
 "severity": "critical",
 "labels": {
 "team": "platform",
 "service": "api-gateway"
 },
 "annotations": {
 "summary": "Error rate above 5% for {{ $labels.service }}",
 "runbook": "https://wiki.internal/runbooks/high-error-rate"
 },
 "notifications": [
 {"channel": "pagerduty", "service_key": "abc123"},
 {"channel": "slack", "webhook": "https://hooks.slack.com/ ... "}
]
}
```

Response: 201 Created

```
{
 "id": "alert_rule_12345",
 "status": "active",
 "created_at": "2024-11-15T10:00:00Z"
}
```



### 15.6.5 5.5 Dashboard API

GET /api/v1/dashboards/{dashboard\_id}

Response:

```
{
 "id": "dash_001",
 "title": "API Gateway Overview",
 "refresh_interval": 30,
 "time_range": {"from": "now-1h", "to": "now"},
 "panels": [
 {
 "id": "panel_01",
 "title": "Request Rate",
 "type": "timeseries",
 "gridPos": {"x": 0, "y": 0, "w": 12, "h": 8},
 "queries": [
 {
 "expr": "sum(rate(http_requests_total{service='api'}[5m])) by (status_code)",
 "legend": "{{status_code}}"
 }
],
 "thresholds": [
 {"value": 1000, "color": "yellow"},
 {"value": 5000, "color": "red"}
]
 }
],
 "variables": [
 {"name": "region", "query": "label_values(http_requests_total, region)", "type": "query"}
]
}
```

---

## 15.7 6. Data Model

### 15.7.1 6.1 Time-Series Data Model

A time series is uniquely identified by its metric name plus a set of key-value label pairs.

Identity: metric\_name + sorted(labels) --> unique time series

Example:

```
http_requests_total{service="api", method="GET", status="200", host="web-001"}
http_requests_total{service="api", method="GET", status="200", host="web-002"}
```

These are TWO distinct time series.

Data Points (samples):

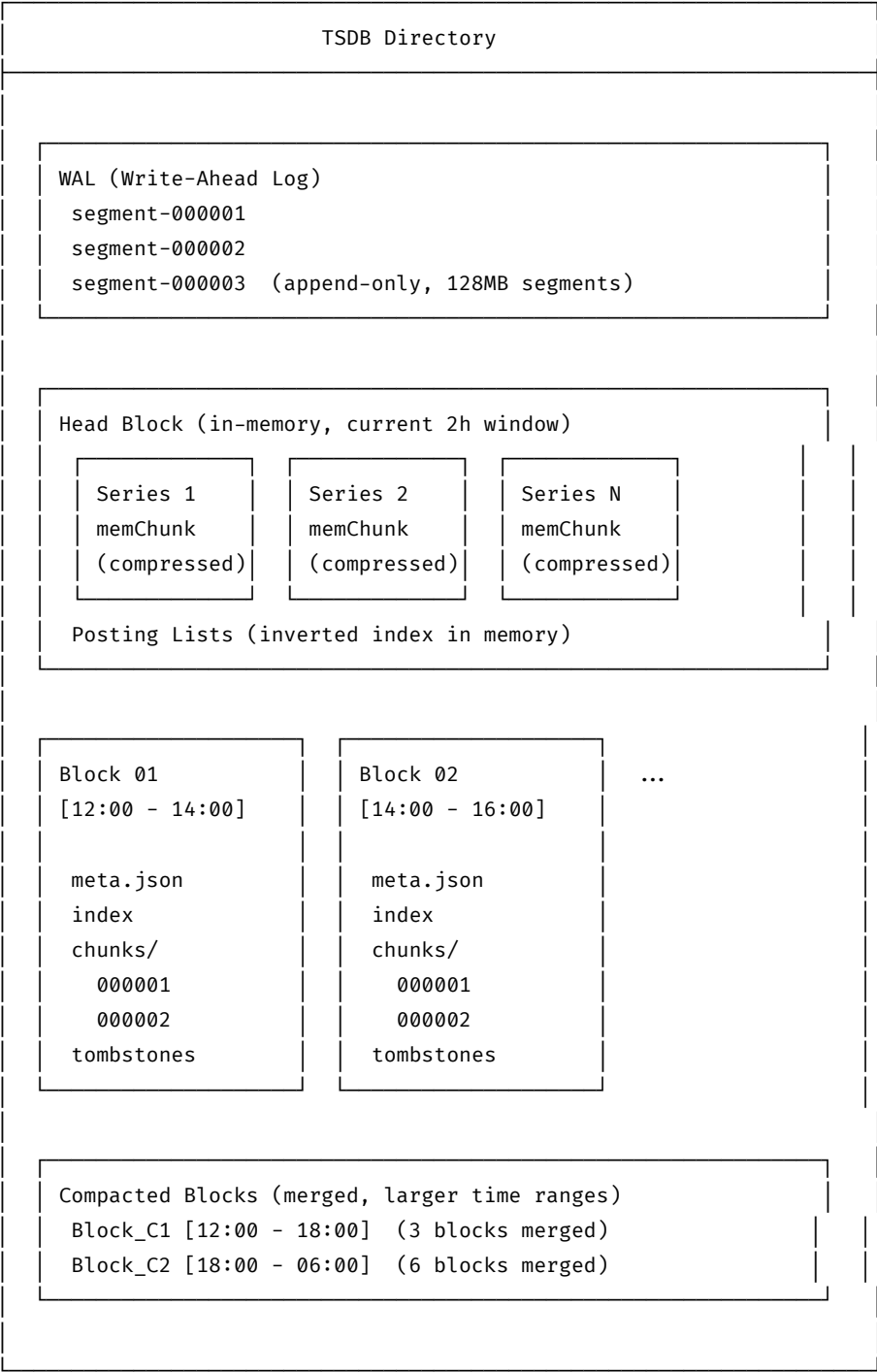
```
(timestamp_ms: int64, value: float64)
```

Cardinality = unique combinations of (metric\_name, label\_set)

```
500 metrics x 1000 hosts x 5 methods x 10 endpoints x 5 status = 125M series
```

(This is "cardinality explosion" -- a critical concern)

15.7.2 6.2 TSDB On-Disk Layout



**15.7.3 6.3 Inverted Index for Labels**

| Label Pair | --> Posting List (sorted series IDs) |
|------------|--------------------------------------|
|------------|--------------------------------------|

|               |                                   |
|---------------|-----------------------------------|
| service="api" | --> [1, 2, 3, 7, 15, 22, 45, ...] |
|---------------|-----------------------------------|

|               |                               |
|---------------|-------------------------------|
| service="web" | --> [4, 5, 8, 9, 16, 23, ...] |
|---------------|-------------------------------|

|                |                         |
|----------------|-------------------------|
| host="web-001" | --> [1, 4, 10, 33, ...] |
|----------------|-------------------------|

|                |                         |
|----------------|-------------------------|
| host="web-002" | --> [2, 5, 11, 34, ...] |
|----------------|-------------------------|

|              |                                 |
|--------------|---------------------------------|
| status="200" | --> [1, 2, 4, 5, 7, 8, 10, ...] |
|--------------|---------------------------------|

|              |                        |
|--------------|------------------------|
| status="500" | --> [3, 6, 9, 12, ...] |
|--------------|------------------------|

|                     |                                   |
|---------------------|-----------------------------------|
| __name__="http_req" | --> [1, 2, 3, 4, 5, 6, 7, 8, ...] |
|---------------------|-----------------------------------|

Query: http\_requests\_total{service="api", status="500"}

```
= Intersect(posting["__name__=http_requests_total"],
 posting["service=api"],
 posting["status=500"])
```

```
= Intersect([1,2,3,4,5,6,7,8], [1,2,3,7,15,22,45], [3,6,9,12])
```

```
= [3]
```

### 15.7.4 6.4 Metadata Store (MySQL / PostgreSQL)

-- Alert rules, dashboards, and tenant config stored in relational DB

```

CREATE TABLE tenants (
 tenant_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 name VARCHAR(255) NOT NULL,
 api_key_hash VARCHAR(64) NOT NULL,
 max_series BIGINT DEFAULT 10000000,
 retention_days INT DEFAULT 15,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE alert_rules (
 rule_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 tenant_id BIGINT NOT NULL REFERENCES tenants(tenant_id),
 name VARCHAR(255) NOT NULL,
 expression TEXT NOT NULL,
 for_duration INT DEFAULT 0, -- seconds
 severity ENUM('info','warning','critical') DEFAULT 'warning',
 labels JSON,
 annotations JSON,
 notification_channels JSON,
 enabled BOOLEAN DEFAULT TRUE,
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE alert_states (
 rule_id BIGINT NOT NULL REFERENCES alert_rules(rule_id),
 series_hash BIGINT NOT NULL,
 state ENUM('inactive','pending','firing','resolved') DEFAULT 'inactive',
 active_since TIMESTAMP NULL,
 fired_at TIMESTAMP NULL,
 resolved_at TIMESTAMP NULL,
 last_value DOUBLE,
 PRIMARY KEY (rule_id, series_hash)
);

```

```
CREATE TABLE dashboards (
 dashboard_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 tenant_id BIGINT NOT NULL REFERENCES tenants(tenant_id),
 title VARCHAR(255) NOT NULL,
 panels JSON NOT NULL,
 variables JSON,
 refresh_interval INT DEFAULT 30,
 created_by VARCHAR(255),
 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

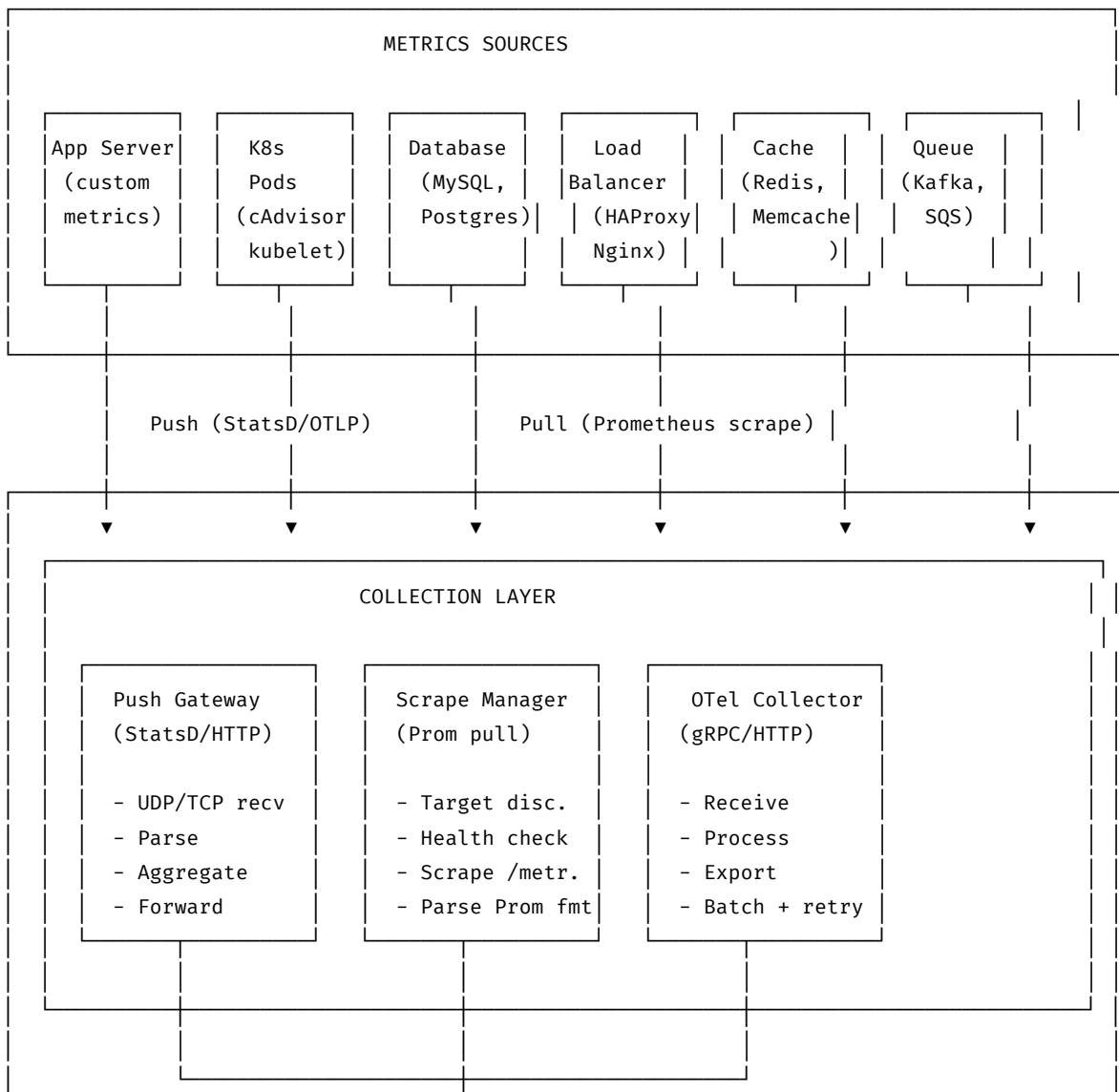
CREATE TABLE recording_rules (
 rule_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 tenant_id BIGINT NOT NULL REFERENCES tenants(tenant_id),
 name VARCHAR(255) NOT NULL, -- new metric name
 expression TEXT NOT NULL, -- PromQL expression
 interval INT DEFAULT 60, -- evaluation interval in seconds
 labels JSON,
 enabled BOOLEAN DEFAULT TRUE
);

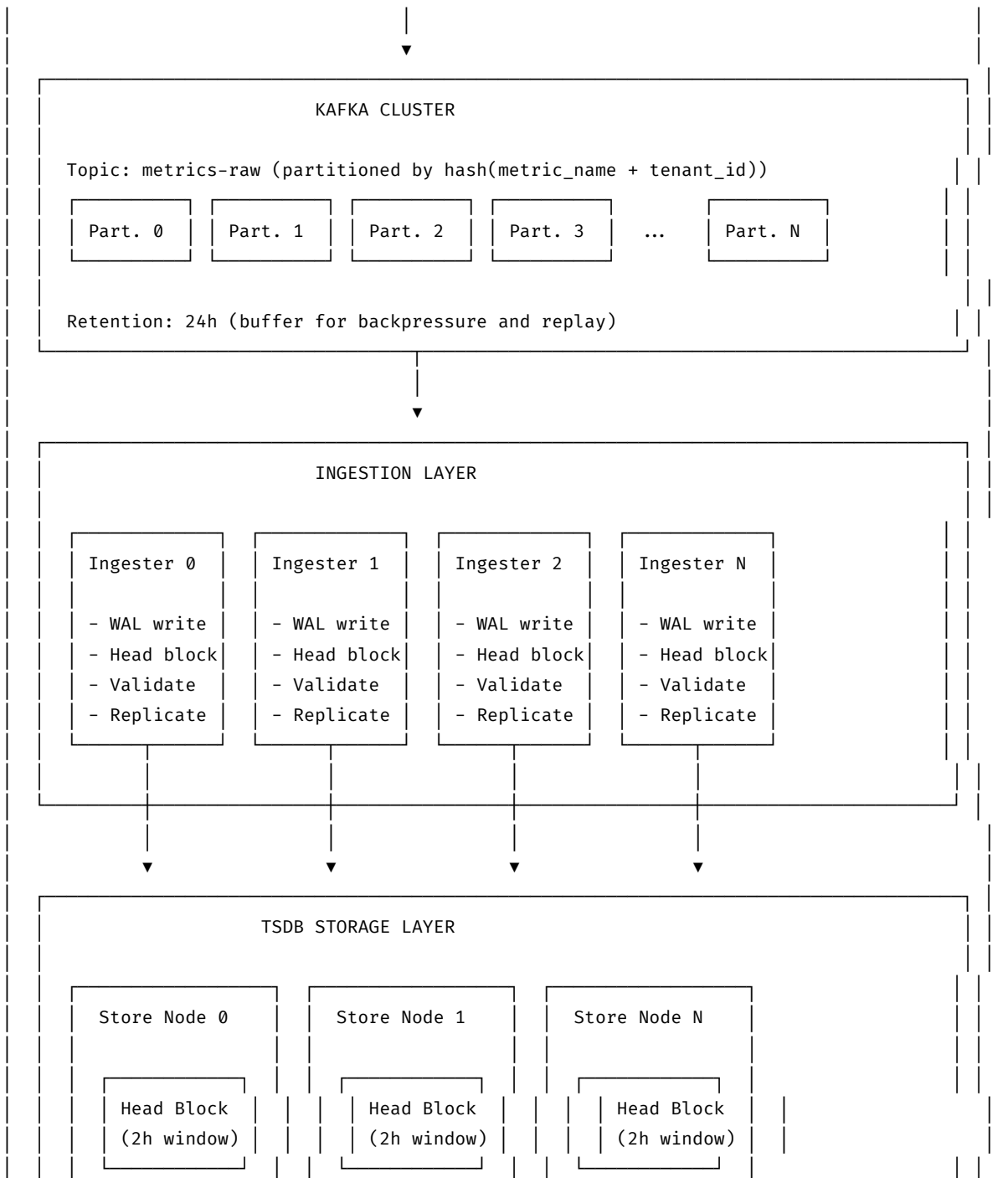
CREATE TABLE downsampling_policies (
 policy_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 tenant_id BIGINT NOT NULL REFERENCES tenants(tenant_id),
 source_resolution INT NOT NULL, -- e.g., 10 seconds
 target_resolution INT NOT NULL, -- e.g., 60 seconds
 aggregations JSON NOT NULL, -- ["avg", "min", "max", "count"]
 after_age INT NOT NULL, -- seconds after which to downsample
 enabled BOOLEAN DEFAULT TRUE
);
```

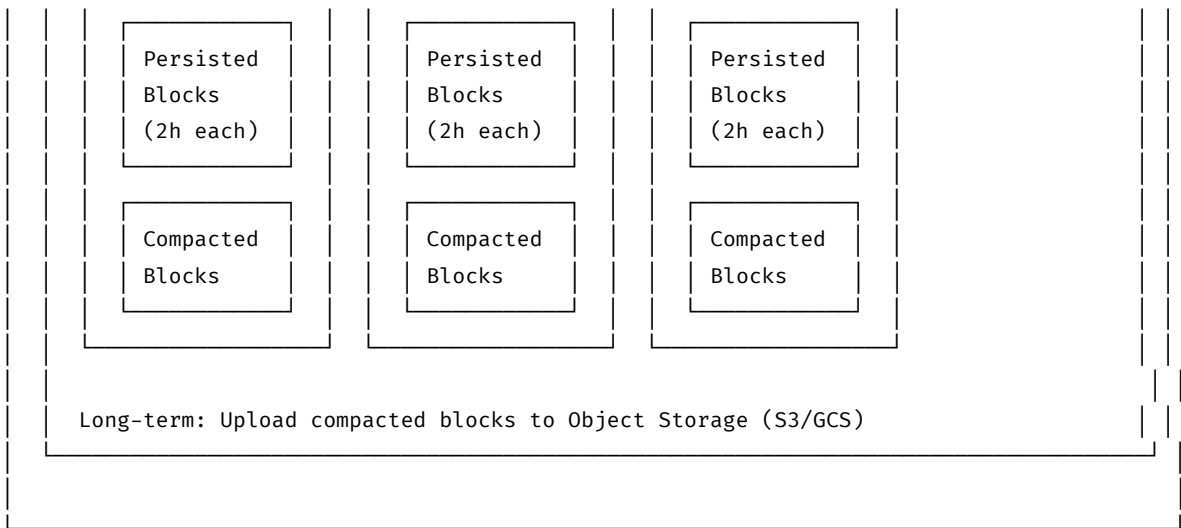
---

## 15.8 7. High-Level Architecture

### 15.8.1 7.1 System Overview

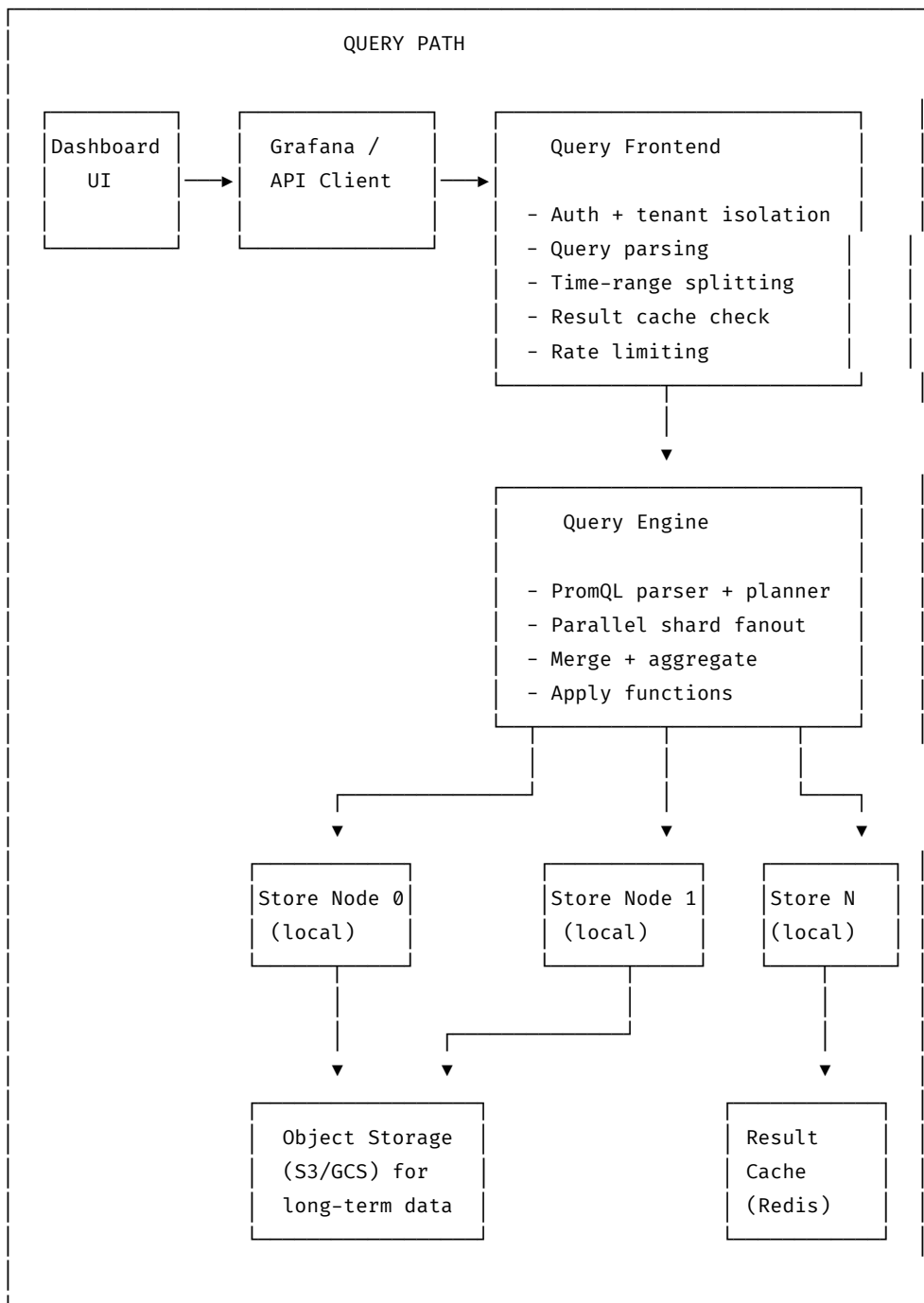




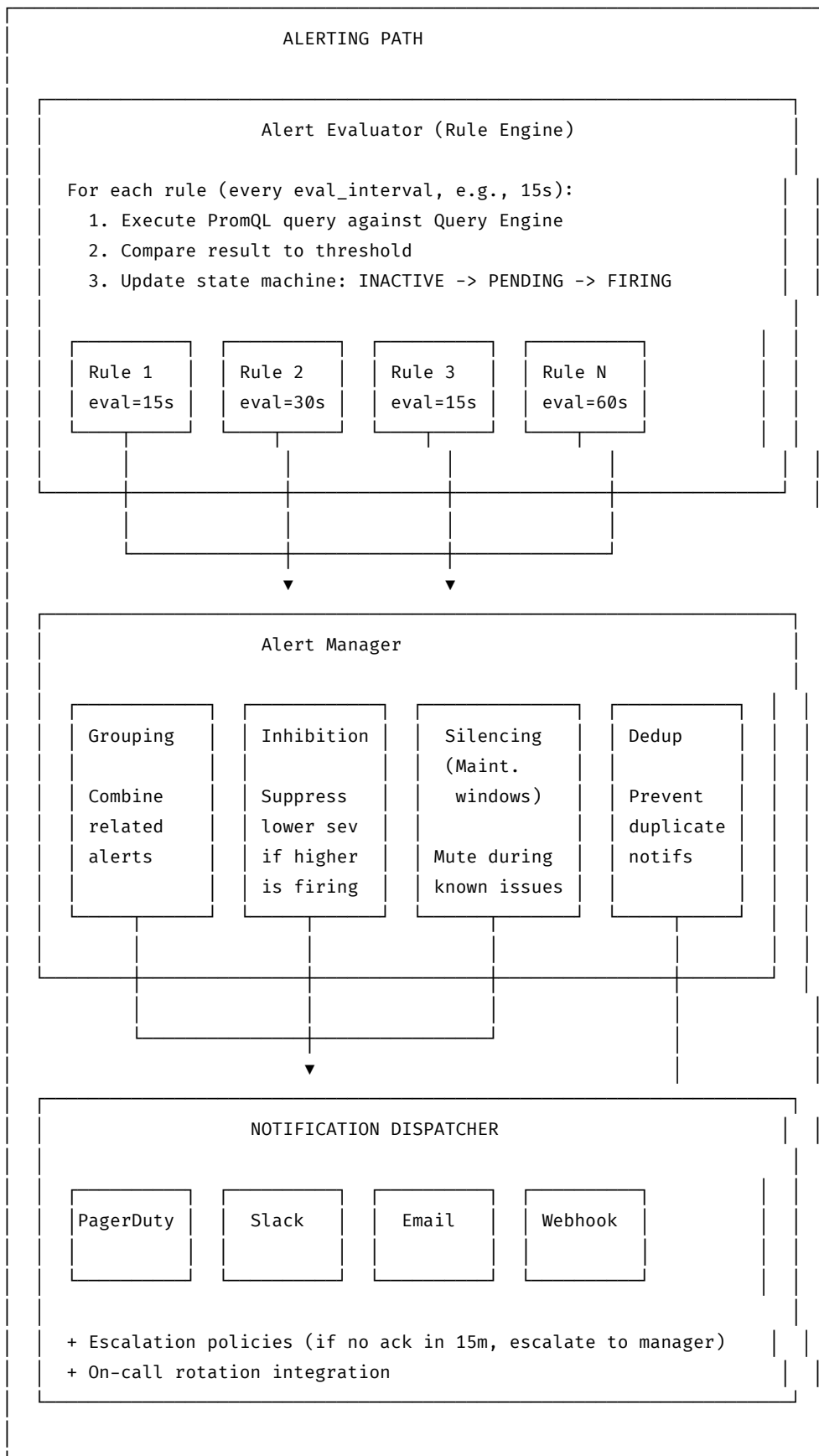




### 15.8.2 7.2 Query Path



### 15.8.3 7.3 Alerting Path



## 15.9 8. Deep Dive

### 15.9.1 8.1 TSDB Storage Engine

#### 15.9.1.1 Gorilla Compression (Facebook, 2015)

The key insight: consecutive samples from the same time series have highly predictable patterns. Timestamps increase by a near-constant interval, and values change slowly.

##### Timestamp Compression: Delta-of-Delta Encoding

```
Raw timestamps: 17000000000, 17000000010, 17000000020, 17000000030, 17000000040
Deltas: 10, 10, 10, 10
Delta-of-deltas: 0, 0, 0, 0
```

Encoding scheme:

```
delta-of-delta = 0: Store '0' (1 bit)
delta-of-delta in [-63,64]: Store '10' + value (7 bits) (9 bits)
delta-of-delta in [-255,256]: Store '110' + value (9 bits) (12 bits)
delta-of-delta in [-2047,2048]: Store '1110' + value (16 bits)
Otherwise: Store '1111' + full 32 bits (36 bits)
```

For regular 10s intervals: 1 bit per timestamp (vs 64 bits raw)

Compression ratio: 64:1 for timestamps!

##### Value Compression: XOR Encoding

For float64 values that change slowly:

```
v1 = 72.5 (IEEE 754: 0x4052200000000000)
v2 = 72.8 (IEEE 754: 0x4052333333333333)
```

```
XOR = v1 XOR v2 = 0x0000133333333333
```

Encoding:

```
XOR = 0: Store '0' (1 bit)
XOR ≠ 0, same position: Store '10' + meaningful bits (2 + N bits)
XOR ≠ 0, new position: Store '11' + leading zeros (5b)
 + length of meaningful (6b) + bits (variable)
```

For slowly changing values: ~15-25 bits per value (vs 64 bits raw)

Compression ratio: ~3-4x for values

##### Combined Compression

Overall per sample:

Timestamp: ~1-2 bits (best case) to ~36 bits (worst case)

Value: ~1 bit (best case) to ~77 bits (worst case)

Average: ~16 bits = 2 bytes per sample

Raw: 16 bytes per sample (8 timestamp + 8 value)

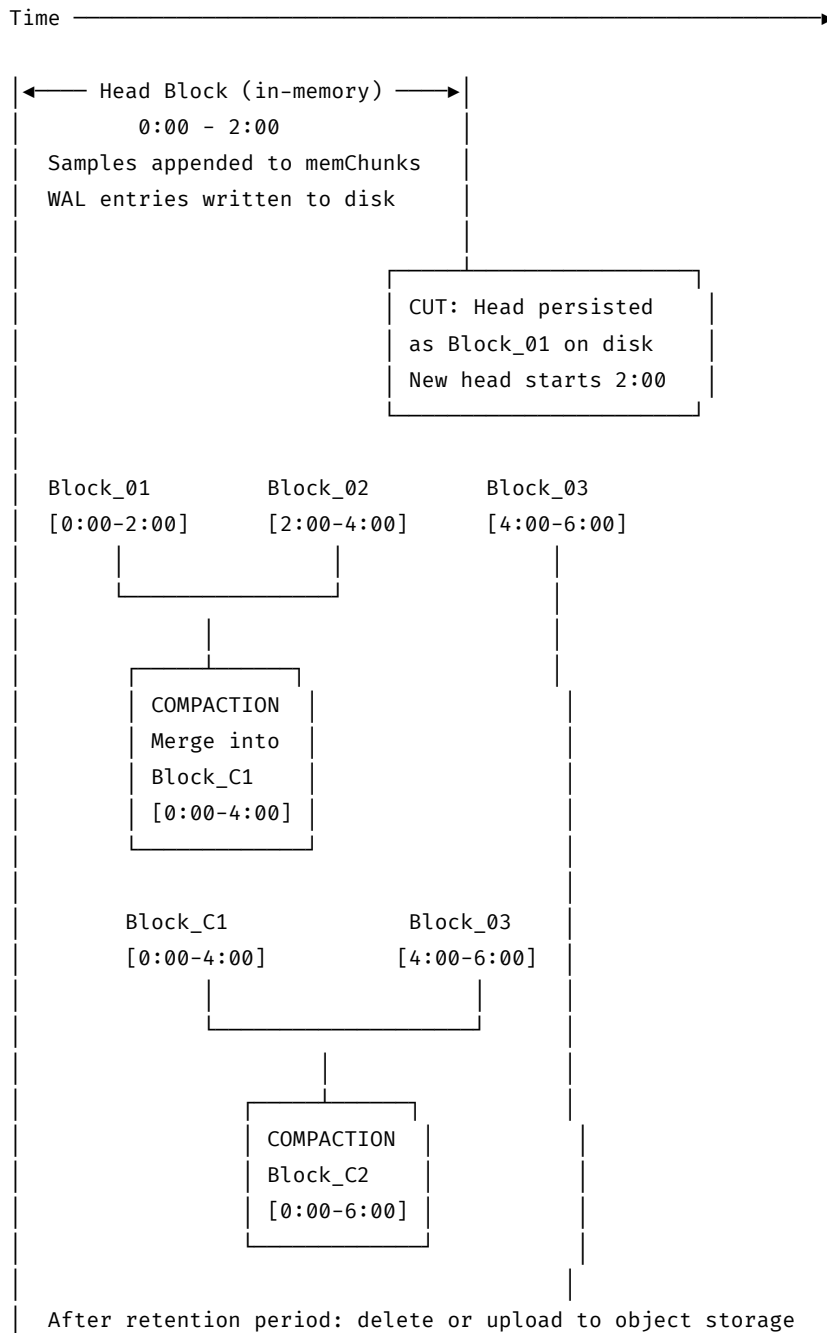
Compressed: ~2 bytes per sample

Ratio: ~8x compression on data alone

With index overhead factored in: ~12x overall compression

### 15.9.1.2 Head Block and Block Lifecycle

Timeline of block lifecycle:



### 15.9.1.3 Compaction Process

#### COMPACTION ENGINE

Input: Block\_A [0:00-2:00] + Block\_B [2:00-4:00]

##### Steps:

1. Merge-sort all series across blocks
2. Re-encode chunks for optimal compression
3. Rebuild inverted index
4. Apply tombstones (deleted series)
5. Drop out-of-retention samples
6. Write new block atomically
7. Swap references, delete old blocks

##### Levels:

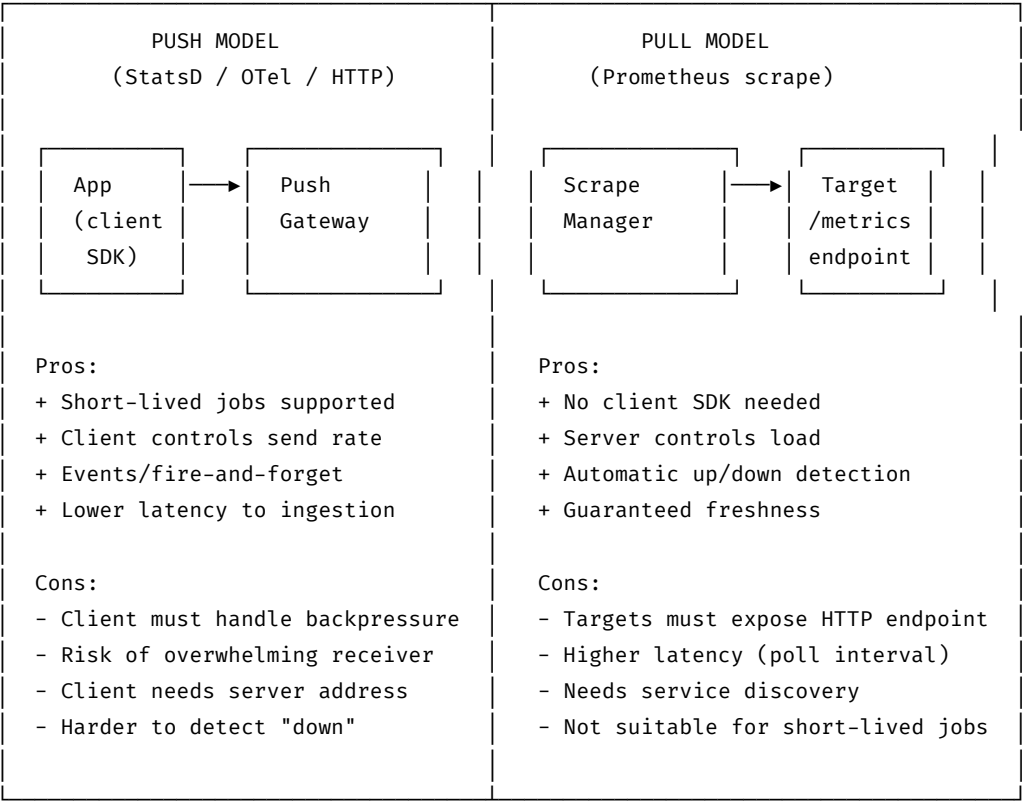
- L0: 2-hour blocks (raw from head)
- L1: 6-hour blocks (3 x L0 merged)
- L2: 24-hour blocks (4 x L1 merged)
- L3: 7-day blocks (7 x L2 merged) -- for long-term store

##### Benefits:

- Fewer blocks to scan for queries
- Better compression (more data to find patterns)
- Cleanup of deleted series
- Reduced file handle count

15.9.2 8.2 Ingestion Pipeline

15.9.2.1 Push vs Pull Model



### 15.9.2.2 Write-Ahead Log (WAL)

#### WAL WRITE PATH

Incoming sample: (metric="cpu", host="web-1", ts=1700000010, 72.5)

##### Step 1: Validate

- Check label cardinality limits
- Verify timestamp within acceptable range (+/- 1 hour)
- Check tenant quotas

##### Step 2: WAL Append (fsync to disk)

WAL Segment File (128 MB each)

| Record Type<br>(1 byte) | Series ID<br>(8 bytes) | Timestamp<br>(8 bytes) | Value<br>(8 bytes) |
|-------------------------|------------------------|------------------------|--------------------|
|-------------------------|------------------------|------------------------|--------------------|

Types: SERIES (new series + labels)  
SAMPLES (batch of data points)  
TOMBSTONE (deletions)

##### Step 3: Append to Head Block (in-memory)

- Lookup or create series by label hash
- Append to memChunk (Gorilla-compressed in memory)
- Update inverted index posting lists

##### Step 4: Acknowledge to client (after WAL + memory)

##### Recovery after crash:

- Replay WAL segments from last checkpoint
- Rebuild head block in memory
- Truncate WAL after successful head block persistence

### 15.9.2.3 Batching and Write Optimization

#### WRITE BATCHING PIPELINE

Source → Agent Batch → Network → Ingester Batch → WAL

##### Client-Side Batching:

Buffer: 1000 samples OR 10s flush interval  
Compress: Snappy/zstd before sending  
Retry: exponential backoff on failure  
Queue: 100MB disk queue for offline buffer

##### Server-Side Batching:

Receive batch from network  
Sort samples by series (locality)  
WAL: write single record for entire batch  
Head: batch append per series  
Fsync WAL every 100ms (tunable)

##### Throughput target per ingester node:

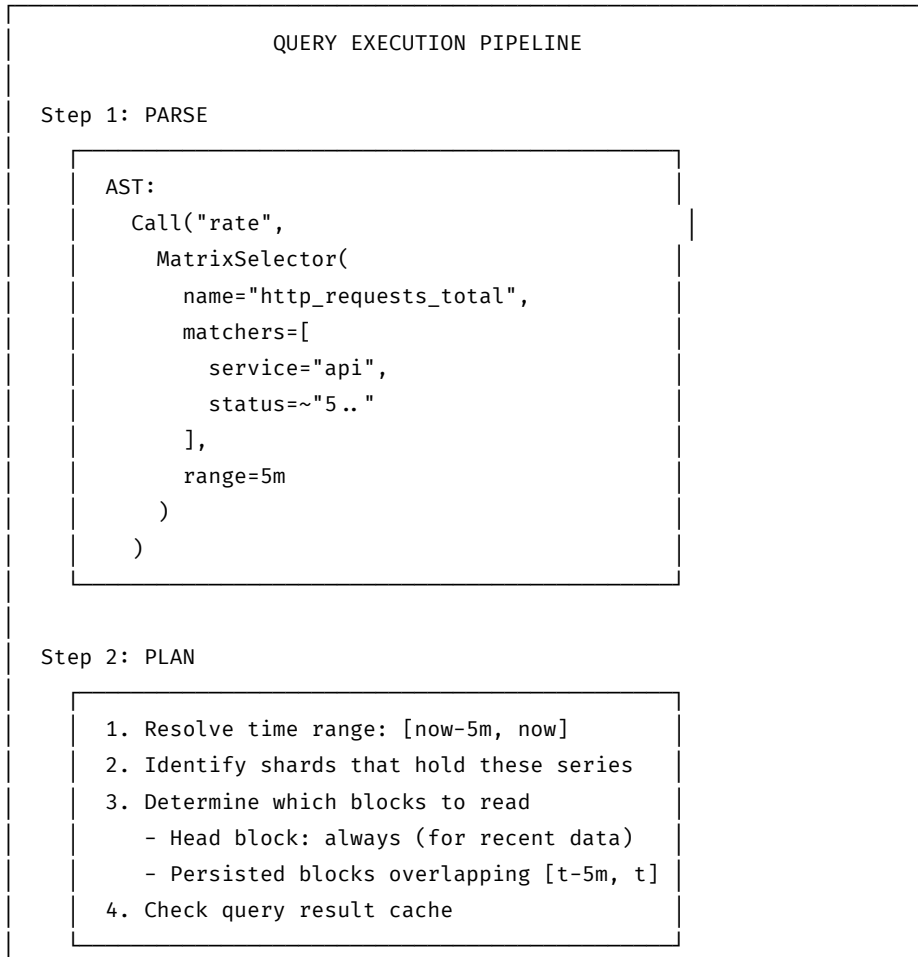
- 2M samples/sec sustained
- WAL write: ~32 MB/sec (2M x 16 bytes)
- Memory: ~4-8 GB for head block (2h window, compressed)



### 15.9.3 8.3 Query Engine

#### 15.9.3.1 PromQL Execution Model

Query: `rate(http_requests_total{service="api", status=~"5.."}[5m])`



## Step 3: FETCH (parallel across shards)

```
For each shard:
 a. Inverted index lookup:
 Intersect(posting["__name__=http_req"],
 posting["service=api"],
 posting_regex["status=5.."])
 = matching series IDs

 b. For each series ID:
 Read chunk data for [now-5m, now]
 Decompress Gorilla-encoded samples
 Return iterator of (timestamp, value)
```

## Step 4: EVALUATE

```
For each matched series:
 rate() = (last_value - first_value)
 / (last_ts - first_ts)
 (with counter reset handling)

Return vector of (labels, value) pairs
```

## Step 5: MERGE (for distributed queries)

```
Collect partial results from all shards
Merge series with same labels
Apply outer aggregations (sum, avg)
Cache result with TTL = step interval
```

**15.9.3.2 Common PromQL Functions and Execution**

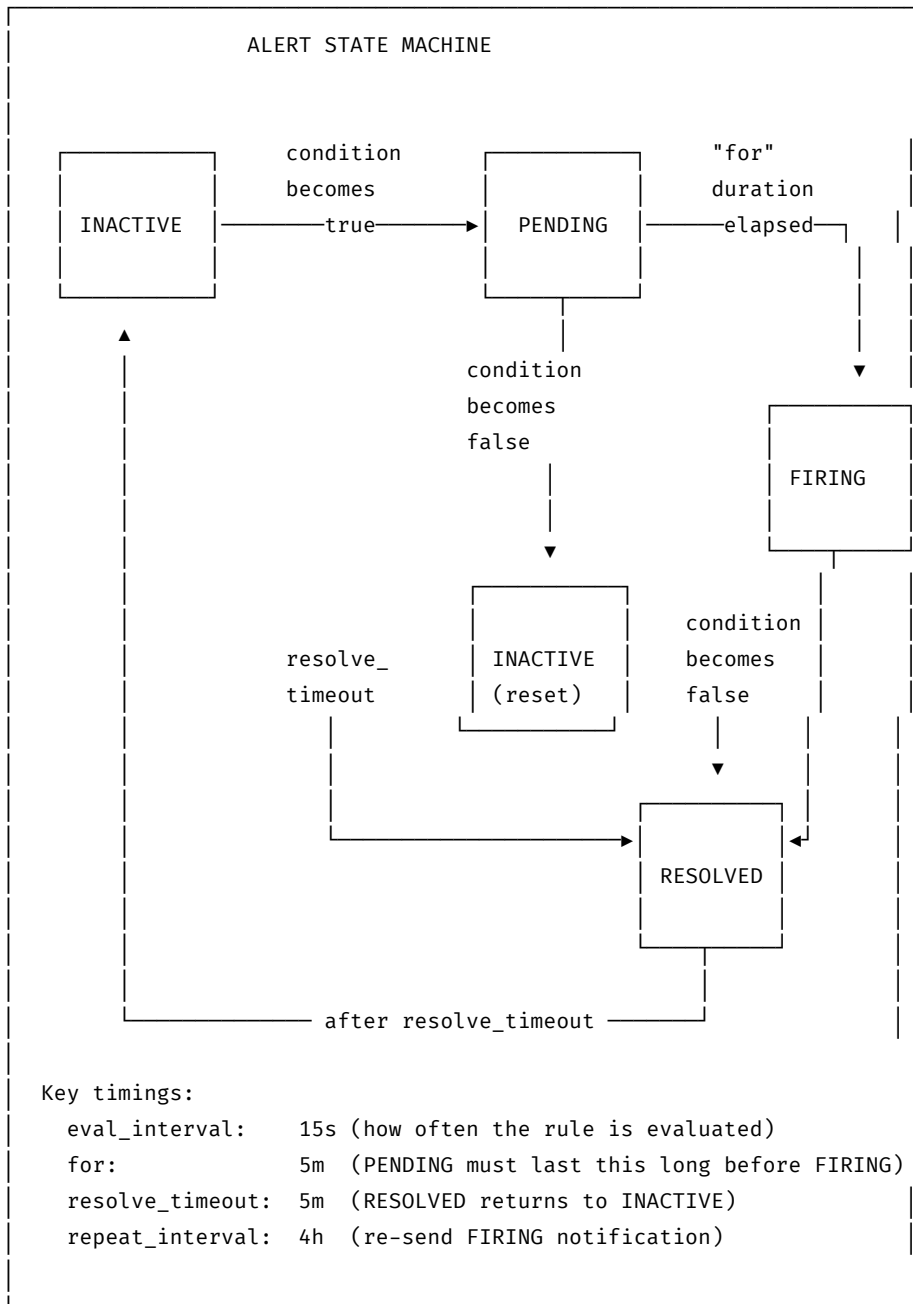
| Function                                   | Execution                                                                                           |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>rate(v[5m])</code>                   | Per-second rate of counter increase over 5m. Handles counter resets. Extrapolates edges.            |
| <code>avg_over_time(v[5m])</code>          | Average of all samples in the range window. Simple mean: <code>sum(values) / count(values)</code> . |
| <code>histogram_quantile(0.99, v)</code>   | Calculates the q-th quantile from histogram buckets using linear interpolation.                     |
| <code>sum by (svc)(rate(...))</code>       | Aggregation: sum across all series, group by the "svc" label. Executed after inner function.        |
| <code>topk(10, ...)</code>                 | Returns top-10 series by value. Requires full evaluation then sort. NOT distributable easily.       |
| <code>predict_linear(v[1h], 4*3600)</code> | Simple linear regression over range. Useful for "disk full in 4 hours" alerts.                      |

**15.9.3.3 Query Result Caching**

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>QUERY RESULT CACHE</p> <p>Cache Key: <code>hash(tenant_id, query_string, aligned_time_range, step)</code></p> <p>Strategy: Split-and-Cache</p> <p>Query: <code>avg(cpu) from 10:00 to 12:00 step=60s</code></p> <div> <p>Cache has: [10:00 - 11:00] HIT</p> <p>Missing: [11:00 - 12:00] MISS -&gt; query TSDB</p> <p>Result: Merge cached + fresh, cache [11:00-12:00]</p> </div> <p>Cache eviction:</p> <ul style="list-style-type: none"> <li>- TTL = step interval (e.g., 60s for step=60s queries)</li> <li>- For "now" queries: short TTL (15s)</li> <li>- For historical queries: long TTL (1h)</li> <li>- LRU eviction when memory pressure</li> </ul> <p>Implementation: Redis cluster with ~100 GB capacity</p> <p>Hit rate target: 60-80% for dashboard queries</p> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 15.9.4 8.4 Alerting Engine

### 15.9.4.1 Alert State Machine





### 15.9.4.2 Grouping, Inhibition, and Silencing

#### ALERT ROUTING PIPELINE

##### GROUPING:

```
group_by: [service, region]
```

##### Firing alerts:

```
HighErrorRate{service=api, host=web-001, region=us-east}
HighErrorRate{service=api, host=web-002, region=us-east}
HighErrorRate{service=api, host=web-003, region=us-east}
```

##### Grouped into ONE notification:

```
"3 instances of HighErrorRate for service=api,
region=us-east"
```

Purpose: Reduce notification noise (3 alerts -> 1 notif)

##### INHIBITION:

Rule: If "ClusterDown" is firing for service=X,  
suppress all "HighLatency" alerts for service=X

##### inhibit\_rules:

```
- source_match: {alertname: ClusterDown}
 target_match: {alertname: HighLatency}
 equal: [service]
```

Purpose: Don't alert on symptoms when root cause is known

##### SILENCING:

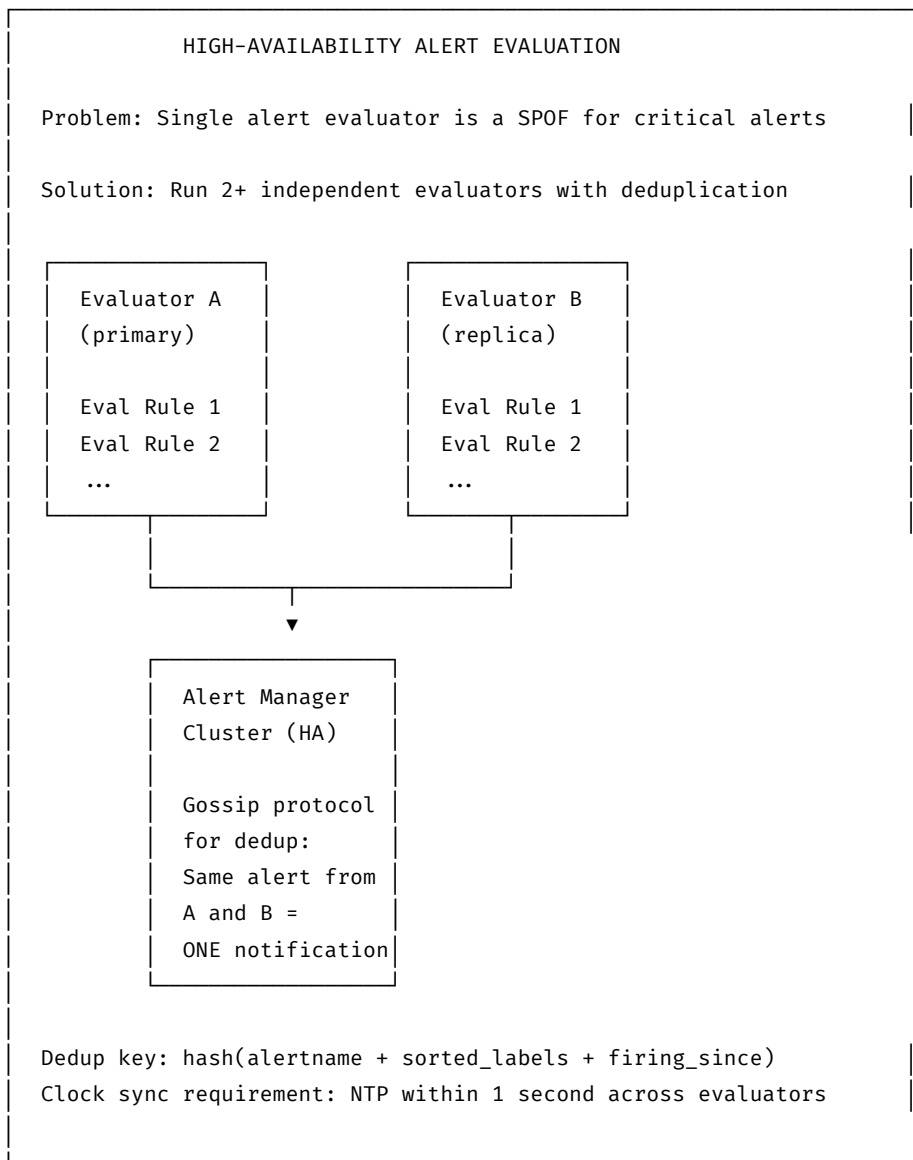
```
Silence: {matchers: [service=api, region=us-east],
 starts_at: 2024-01-15T02:00:00Z,
 ends_at: 2024-01-15T04:00:00Z,
 created_by: "oncall-engineer",
 comment: "Planned maintenance window"}
```

Purpose: Mute alerts during known maintenance windows

##### ESCALATION:

```
Level 1: Slack notification (immediate)
Level 2: PagerDuty to on-call (after 5 min no ack)
Level 3: PagerDuty to manager (after 15 min no ack)
Level 4: Phone call to VP Eng (after 30 min no ack)
```

### 15.9.4.3 Dual Alert Evaluation for HA



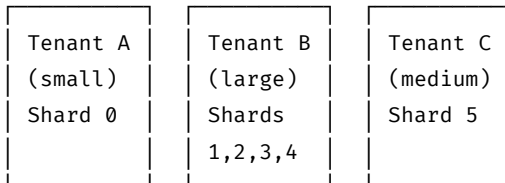
## 15.10 9. Partitioning Strategy

### 15.10.1 9.1 Partitioning Dimensions

#### PARTITIONING STRATEGY

Three dimensions of partitioning:

##### 1. BY TENANT (for multi-tenant SaaS)



Isolation: tenant B cannot impact tenant A's queries

##### 2. BY METRIC NAME HASH (within a tenant)

$\text{shard\_id} = \text{hash}(\text{tenant\_id} + \text{metric\_name}) \% \text{num\_shards}$

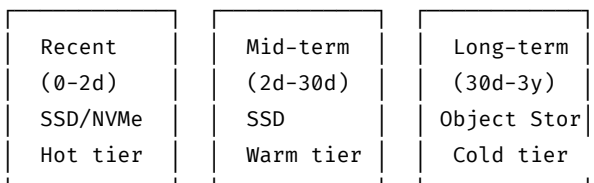
Benefits:

- + All samples for same metric on same shard
- + Efficient range queries for single metric
- + Even distribution across shards

Drawback:

- Cross-metric queries (joins) require scatter-gather

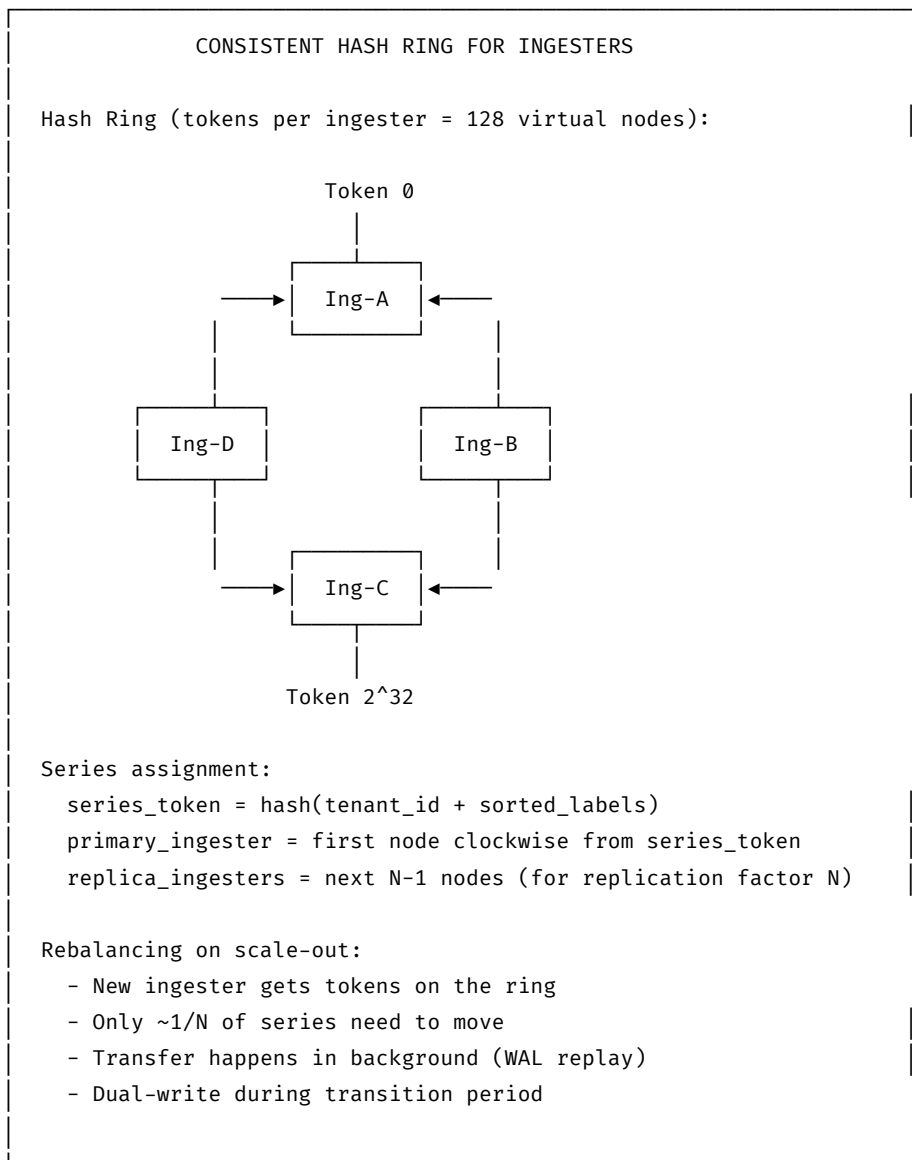
##### 3. BY TIME RANGE



Queries automatically fan out to appropriate tiers



### 15.10.2 9.2 Consistent Hashing for Shard Assignment



## 15.11 10. Caching Strategy

### CACHING LAYERS

#### LAYER 1: Head Block (In-Memory) -- Always Active

What: Latest 2 hours of all active series  
 Size: 4-16 GB per ingester (depends on cardinality)  
 Hit rate: 100% for "last N minutes" queries  
 Implementation: Custom memChunks with Gorilla compression

#### LAYER 2: Chunk Cache (LRU)

What: Recently accessed compressed chunks from disk blocks  
 Size: 16-64 GB per store node  
 Key: (block\_id, series\_id, chunk\_index)  
 Hit rate: 70-85% (dashboards hit same chunks repeatedly)  
 Implementation: In-process LRU or external Memcached

#### LAYER 3: Index / Posting List Cache

What: Inverted index posting lists for label lookups  
 Size: 2-8 GB per store node  
 Key: (block\_id, label\_name, label\_value)  
 Hit rate: 90%+ (label cardinality is bounded)  
 Implementation: In-process concurrent hashmap

#### LAYER 4: Query Result Cache (Shared)

What: Full query results for dashboard panels  
 Size: 50-200 GB (Redis cluster)  
 Key: hash(tenant, query, time\_range, step)  
 TTL: min(step, 60s) for recent; 1h for historical  
 Hit rate: 60-80% for dashboard traffic  
 Split-cache: partial hits for overlapping time ranges

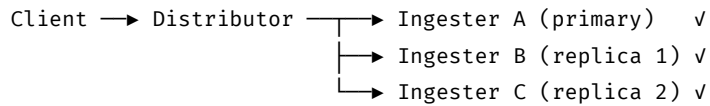
#### LAYER 5: Metadata Cache

What: Tenant configs, alert rules, dashboard definitions  
 Size: 1-2 GB  
 TTL: 30-60 seconds (near real-time updates)  
 Implementation: Local in-process + Redis

## 15.12 11. Replication

### REPLICATION STRATEGY

#### WRITE PATH (Quorum Writes, RF=3)



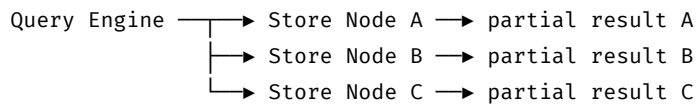
Write acknowledged when quorum (2 of 3) succeed

Third write completes asynchronously (hinted handoff)

Each ingester independently:

- Writes to its own WAL
- Appends to its own head block
- Persists its own blocks to disk
- Uploads compacted blocks to object storage

#### READ PATH (Merge Query Results)



Merge strategy:

- Deduplicate samples with same (series, timestamp)
- Prefer sample from primary replica
- For range queries: union of samples, sort by time
- If one replica is slow/down: return partial results with warning header

#### LONG-TERM STORAGE (Eventual Consistency)

##### Object Storage (S3/GCS):

- Each ingester uploads its compacted blocks
- Blocks from replicas may overlap in time
- Compactor deduplicates overlapping blocks
- Eventually consistent: dedup happens asynchronously

##### Downsampled data:

- Computed from compacted blocks (not from WAL)
- Eventual consistency is acceptable for historical data
- Downsampler runs as background job on object store data

---

## 15.13 12. Fault Tolerance

15.13.1 12.1 Failure Scenarios and Mitigations

| FAULT TOLERANCE MATRIX                            |                                                                                                               |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| FAILURE SCENARIO                                  | MITIGATION                                                                                                    |
| Single ingester crash                             | WAL recovery on restart.<br>Replicas serve queries.<br>Hash ring re-routes new writes.                        |
| Ingester disk full                                | WAL truncation after checkpoint.<br>Head block flush to free memory.<br>Alert on disk usage > 80%.            |
| Kafka broker failure                              | Kafka RF=3, auto leader election.<br>Producers retry with backoff.<br>Consumer lag monitored.                 |
| Store node failure                                | Query engine excludes failed node.<br>Returns partial results +warning.<br>Data still available on replicas.  |
| Object storage unavailable                        | Local blocks serve recent queries.<br>Upload retried with exp backoff.<br>Blocks cached on store nodes.       |
| Query engine OOM                                  | Query limits: max samples, series.<br>Timeout enforcement (30s default).<br>Query splitting for large ranges. |
| Network partition<br>(split-brain)                | Each side continues independently.<br>Duplicate data reconciled later.<br>Alert evaluators on both sides.     |
| Clock skew across hosts                           | Accept samples within +/- 1 hour.<br>Reject far-future timestamps.<br>NTP monitoring as meta-metric.          |
| Cardinality explosion<br>(label values unbounded) | Per-tenant series limit.<br>Auto-detection + alert.<br>Drop new series beyond limit.                          |

### 15.13.2 12.2 WAL Recovery Process

#### WAL RECOVERY ON RESTART

1. Find last checkpoint file  
checkpoint.000042  
└ Contains: snapshot of all active series + their labels
2. Find WAL segments after checkpoint  
segment-000043, segment-000044, segment-000045
3. Replay segments sequentially

For each record in segment:

SERIES record: register series + labels  
SAMPLES record: append to head block  
TOMBSTONE record: mark series deleted

Skip samples older than head block window  
(already persisted to disk blocks)

4. Resume normal operation
  - New head block with recovered data
  - WAL truncation of replayed segments
  - Create new checkpoint

Recovery time: ~30-60 seconds for 8 GB head block

Data loss: zero (WAL is fsync'd)

Data gap: samples missed while node was down  
(covered by replicas for queries)

## 15.14 13. Scalability

### 15.14.1 13.1 Horizontal Scaling Strategy

#### SCALABILITY APPROACH

##### INGESTION SCALING:

Kafka partitions: scale from 64 to 1024 as load grows  
Ingesters: stateless consumers, add more for throughput  
Hash ring: auto-rebalance with new ingesters

Scaling trigger: ingestion lag > 30s OR CPU > 70%  
Scale step: add 10% more ingesters  
Cooldown: 10 minutes between scale events

##### STORAGE SCALING:

TSDB sharding:  
Phase 1: 10 shards (50K series each)  
Phase 2: 100 shards (500K series each)  
Phase 3: 1000 shards (5M series each)

Each shard is an independent TSDB instance  
Shard splitting: when a shard exceeds size threshold

- Create two new shards with split hash range
- Copy relevant blocks to new shards
- Redirect new writes via updated hash ring
- Drain old shard after verification

Object storage: virtually unlimited (S3/GCS)  
Tier migration: hot (NVMe) -> warm (SSD) -> cold (S3)

## QUERY SCALING:

## Read replicas:

- Query frontends are stateless, scale horizontally
- Store gateways: read-only access to object storage
- Add store gateways for long-range query throughput

## Query sharding:

- Time-range queries split into sub-ranges
- Each sub-range fetched from different store node
- Results merged at query frontend

## Query limits (protect system):

- max\_samples\_per\_query: 50M
- max\_series\_per\_query: 100K
- query\_timeout: 120s
- max\_concurrent\_queries: 20 per tenant

## KAFKA BUFFERING:

Purpose: Decouple ingestion speed from processing speed

Normal: producer rate = consumer rate, lag ~0

Spike: producer rate > consumer rate temporarily  
Kafka buffers excess (24h retention)  
Consumers catch up after spike subsides

## Benefits:

- Absorbs burst traffic without data loss
- Allows ingester maintenance (drain + restart)
- Replay from offset for recovery / backfill



**DOWNSAMPLING:**

```
Raw (10s) —[after 15d]→ 1-minute avg/min/max/count
1-min —[after 90d]→ 5-minute avg/min/max/count
5-min —[after 1y] → 1-hour avg/min/max/count
```

**Storage savings:**

```
10s -> 1m: 6x reduction in samples
1m -> 5m: 5x reduction
5m -> 1h: 12x reduction
Total for 3-year data: ~360x fewer samples than raw
```

**Query engine auto-selects resolution based on time range:**

```
Last 6h: raw (10s)
Last 7d: 1-minute
Last 90d: 5-minute
Last 3y: 1-hour
```

## 15.15 14. Monitoring the Monitoring System (Meta)

The monitoring system must monitor itself. This is a bootstrapping problem – if the monitoring system is down, who alerts you?

## META-MONITORING METRICS

## INGESTION HEALTH:

|                                   |                     |
|-----------------------------------|---------------------|
| monitoring_ingestion_rate_total   | (target: baseline)  |
| monitoring_ingestion_errors_total | (target: 0)         |
| monitoring_kafka_consumer_lag     | (target: < 1000)    |
| monitoring_wal_size_bytes         | (target: < 4 GB)    |
| monitoring_head_series_active     | (track cardinality) |
| monitoring_samples_rejected_total | (cardinality limit) |

## QUERY HEALTH:

|                                    |                    |
|------------------------------------|--------------------|
| monitoring_query_duration_seconds  | (p99 < 2s)         |
| monitoring_query_errors_total      | (target: 0)        |
| monitoring_query_samples_scanned   | (detect expensive) |
| monitoring_query_cache_hit_ratio   | (target: > 0.6)    |
| monitoring_query_concurrent_active | (limit: 200)       |

## STORAGE HEALTH:

|                                         |                    |
|-----------------------------------------|--------------------|
| monitoring_tsdb_blocks_total            | (count per node)   |
| monitoring_tsdb_compactions_total       | (should be steady) |
| monitoring_tsdb_compaction_duration_sec | (track slowdowns)  |
| monitoring_storage_disk_usage_bytes     | (target: < 80%)    |
| monitoring_object_store_upload_failures | (target: 0)        |

## ALERTING HEALTH:

|                                        |                 |
|----------------------------------------|-----------------|
| monitoring_alert_eval_duration_sec     | (p99 < 5s)      |
| monitoring_alert_notification_latency  | (target: < 30s) |
| monitoring_alert_notification_failures | (target: 0)     |
| monitoring_alert_rules_active          | (count)         |
| monitoring_alert_firing_total          | (track trends)  |

## CARDINALITY EXPLOSION DETECTION:

|                                           |                 |
|-------------------------------------------|-----------------|
| monitoring_tsdb_head_series_created_total | (rate of new)   |
| monitoring_label_cardinality{label="X"}   | (unique values) |

Alert: rate(head\_series\_created\_total[10m]) > 10000  
 "More than 10K new series/min -- potential cardinality explosion. Check for unbounded labels like request\_id, user\_id, or trace\_id being used as metric labels."

## BOOTSTRAPPING SOLUTION:

|                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Option A: Separate small Prometheus instance monitors the main monitoring cluster (independent failure domain, alerts via separate PagerDuty) |
| Option B: Canary probes from external service (e.g., synthetic monitoring from cloud provider)                                                |
| Option C: Health check endpoints polled by load balancer with automatic page if all nodes fail                                                |

## 15.16 15. Trade-offs and Design Decisions

### 15.16.1 15.1 Push vs Pull Ingestion

DECISION: Support BOTH push and pull with push as primary

Rationale:

- Pull (Prometheus-style) is ideal for long-running services with HTTP endpoints and Kubernetes service discovery
- Push is necessary for short-lived batch jobs, serverless functions, and environments behind NAT/firewalls
- OTel Collector bridges both: scrapes targets AND receives push
- At scale, push through Kafka provides better buffering and decouples sources from ingesters

Implementation:

- Pull targets: Scrape Manager converts to push (remote\_write)
- Push sources: Direct to Kafka or Push Gateway
- Both converge at Kafka topic -> Ingesters

### 15.16.2 15.2 Custom TSDB vs Existing Solutions

DECISION: Build custom TSDB layer on top of proven primitives

Option A: Use Prometheus TSDB directly

- + Battle-tested, excellent single-node performance
- No native horizontal scaling
- Limited multi-tenancy

Option B: Use InfluxDB / TimescaleDB

- + Feature-rich, SQL-like queries
- Different query language (Flux / SQL vs PromQL)
- Licensing concerns at scale

Option C: Custom TSDB engine (our choice)

- + Optimized for our exact access patterns
- + Full control over sharding, replication, compaction
- + PromQL compatibility by design
- Higher engineering investment
- Must implement Gorilla compression, index, compaction ourselves

Hybrid approach: Use Prometheus TSDB library as embedded engine within custom distributed wrapper (like Thanos/Mimir approach)

### 15.16.3 15.3 PromQL vs SQL for Query Language

DECISION: PromQL as primary, with SQL adapter for analytics

PromQL advantages:

- + Purpose-built for time-series (rate, irate, histogram\_quantile)
- + Concise: `rate(http_req{svc="api"}[5m])` vs complex SQL
- + Industry standard, huge ecosystem (Grafana, exporters)
- + Implicit time alignment and step evaluation

SQL advantages:

- + More familiar to data analysts
- + Better for ad-hoc exploration and JOINS
- + Can express complex business logic

Approach: PromQL for ops dashboards and alerts (95% of queries),  
SQL adapter (translate SQL to PromQL) for analytics use cases

### 15.16.4 15.4 Memory vs Disk Trade-off

DECISION: Large memory head blocks with aggressive disk tiering

More memory (larger head block):

- + Faster queries for recent data (no disk I/O)
- + Better write throughput (batch WAL, no random I/O)
- + Fewer blocks to compact
- Higher cost per node
- Longer WAL recovery on restart
- Risk of data loss if WAL is corrupted

Less memory (smaller head block, more disk):

- + Cheaper per node
- + Faster recovery (less WAL to replay)
- + More data on disk = better durability
- Slower queries for recent data
- More frequent block flushes = more compaction work

Our choice: 2-hour head block (Prometheus default) with:

- 16-32 GB memory per ingester for head block + index
- NVMe SSD for persisted blocks (fast random read for queries)
- Object storage for blocks older than 24h (cheap, durable)
- Chunk cache to avoid disk I/O for repeated queries

### 15.16.5 15.5 Summary of Key Trade-offs

| Decision          | Option Chosen                  | Key Reason                                   |
|-------------------|--------------------------------|----------------------------------------------|
| Ingestion model   | Both push + pull               | Flexibility for all deployment models        |
| Storage engine    | Custom on Prom TSDB library    | Control + proven engine                      |
| Query language    | PromQL primary                 | Industry standard, concise for time-series   |
| Consistency       | Eventual                       | AP over CP; monitoring favors availability   |
| Replication       | RF=3, quorum write             | Balance durability + write latency           |
| Head block size   | 2 hours in-memory              | Sweet spot for query speed vs recovery time  |
| Long-term storage | Object store (S3)              | Cost-effective, durable, virtually unlimited |
| Alert dedup       | Gossip-based AM cluster        | No SPOF, distributed deduplication           |
| Compression       | Gorilla (delta-of-delta + XOR) | 8-12x compression, proven at Facebook scale  |

## 15.17 16. Interview Questions and Answers

### 15.17.1 Q1: How do you handle high cardinality labels that cause a cardinality explosion?

**Answer:** High cardinality occurs when label values are unbounded (e.g., user\_id, request\_id, IP address). A metric with 500K unique user IDs creates 500K separate time series.

Mitigations: 1. **Per-tenant series limits:** Enforce a hard cap (e.g., 10M active series). Reject new series beyond the limit with a clear error. 2. **Label validation at ingestion:** Block known high-cardinality labels (user\_id, trace\_id, email) from being used as metric labels. These belong in logs/traces. 3. **Cardinality monitoring:** Track `rate(head_series_created[10m])` and alert when it spikes (> 10K new series/min). 4. **Relabeling rules:** Drop or aggregate high-cardinality labels before storage (e.g., replace IP with subnet, replace user\_id with user\_tier). 5. **Adaptive limiting:** When a specific metric exceeds cardinality threshold, auto- drop the highest-cardinality label and alert the team to fix their instrumentation.

The key insight is that cardinality explosion is an **operational** problem, not just a technical one. Education and guardrails prevent it better than after-the-fact cleanup.

### 15.17.2 Q2: Explain Gorilla compression and why it works so well for time-series data.

**Answer:** Gorilla compression (Facebook, 2015 paper “Gorilla: A Fast, Scalable, In-Memory Time Series Database”) exploits two properties of monitoring data:

1. **Timestamps are regular:** Metrics arrive at fixed intervals (10s, 30s, 60s), so the delta between consecutive timestamps is nearly constant. Delta-of-delta encoding captures this: if delta is always 10s, the delta-of-delta is 0, stored in 1 bit. For perfectly regular data, timestamps compress from 64 bits to 1 bit (~64x).
2. **Values change slowly:** CPU at 72.5% then 72.8% – the IEEE 754 representations share most bits. XOR of consecutive values has many leading and trailing zeros. We encode: (a) identical value = 1 bit, (b) similar value = 2 bits + meaningful bits (~15-25 bits), (c) very different = 77 bits worst case. Average ~15 bits (~4x).

Combined: ~2 bytes per sample vs 16 bytes raw = ~8x compression. This means a server with 64 GB RAM can hold ~26 billion compressed samples in memory for sub-millisecond queries. For on-disk blocks, additional compaction across chunks achieves 12-15x overall.

---

### 15.17.3 Q3: How do you prevent alert fatigue in a large-scale monitoring system?

**Answer:** Alert fatigue occurs when operators receive so many alerts that they ignore them all, including critical ones. Prevention strategies:

1. **Grouping:** Combine related alerts into single notifications. If 50 hosts have high CPU, send one alert “50 hosts with HighCPU in us-east” not 50 separate pages.
  2. **Inhibition:** Suppress symptom alerts when the root cause is known. If “ClusterDown” is firing, suppress all “HighLatency” and “HighErrorRate” alerts for that cluster.
  3. **Severity tiers:** Only page for critical alerts. Warning = Slack. Info = dashboard. Audit alert severity quarterly – if a critical alert hasn’t led to action in 90 days, downgrade or delete it.
  4. **“For” duration:** Require the condition to persist for N minutes before firing. This eliminates transient spikes that self-resolve. `for: 5m` prevents 99% of flaps.
  5. **Dead-man’s switch:** Instead of alerting on every metric individually, use absence- of-heartbeat alerts. If the system stops reporting, that single alert covers everything.
  6. **Alert ownership:** Every alert must have a team owner. Unowned alerts are deleted. Track alert-to-action ratio: if an alert fires 100 times and is actioned 2 times, it needs to be tuned or removed.
- 

### 15.17.4 Q4: How do you handle clock skew when collecting metrics from thousands of hosts?

**Answer:** Clock skew is inevitable in distributed systems. A host with a drifted clock will send timestamps that are in the past or future relative to the monitoring system’s clock.

Strategies: 1. **Acceptance window:** Accept samples with timestamps within +/- 1 hour of server time. Reject anything outside this window with a clear error to the sender.

---

2. **Server-side timestamping:** For push-based ingestion, the server can optionally ignore the client timestamp and use its own receipt time. This avoids skew entirely but loses precision about when the metric was actually measured.
  3. **Out-of-order sample handling:** The TSDB must handle out-of-order samples within the head block window. Prometheus added OOO sample support (since 2.39). Samples arriving “late” (within the OOO window) are inserted into the correct position.
  4. **NTP monitoring:** Monitor NTP sync status as a meta-metric on every host. Alert when clock drift exceeds 500ms. Most environments use chrony/ntpd for < 10ms drift.
  5. **Query-time tolerance:** When aligning samples to step boundaries, use a tolerance window (e.g., +/- 5 seconds from the expected step timestamp) to accommodate minor skew without creating gaps in graphs.
- 

### 15.17.5 Q5: How would you design multi-tenant isolation in a shared monitoring platform?

**Answer:** Multi-tenancy requires isolation across several dimensions:

1. **Data isolation:** Every sample is tagged with `tenant_id`. The inverted index includes `tenant_id` in all posting lists. Queries are automatically scoped: the query engine prepends `{tenant_id="X"}` to every query, even if the user doesn't specify it.
  2. **Ingestion isolation:** Per-tenant rate limits (samples/sec), series limits (max active series), and label limits (max labels per series, max label length). Enforced at the distributor/ingester layer. One tenant's burst cannot starve another.
  3. **Query isolation:** Per-tenant query concurrency limits, max samples per query, and query timeout. Implemented via per-tenant queues in the query frontend with fair scheduling (weighted round-robin based on tenant tier).
  4. **Storage isolation:** Logical separation (all tenants in same TSDB but prefixed) or physical separation (dedicated shards for large tenants). Object storage uses tenant-prefixed paths: `s3://metrics/{tenant_id}/blocks/ ...`
  5. **Cost attribution:** Track ingestion volume, storage bytes, and query cost per tenant for charge-back. This also incentivizes tenants to reduce cardinality.
- 

### 15.17.6 Q6: How does the system handle a sudden 10x spike in ingestion rate?

**Answer:** Spikes occur during incidents (many error metrics), deployments (new metrics), or cardinality explosions. The system absorbs spikes through multiple layers:

1. **Kafka buffering:** The primary defense. Kafka absorbs the spike with its 24-hour retention. Consumer lag increases but no data is lost. The system processes the backlog after the spike subsides.
2. **Client-side buffering:** Well-behaved agents (OTel Collector, Datadog Agent) have local disk queues (100 MB - 1 GB). If the server is slow, the agent buffers locally and retries with exponential backoff.
3. **Backpressure signaling:** When ingesters are overloaded, they return HTTP 429 (Too Many Requests) with Retry-After header. Clients back off gracefully.



4. **Auto-scaling:** Ingestion nodes auto-scale based on Kafka consumer lag or CPU utilization. New ingesters join the hash ring and take ownership of partitions. Scale-up takes 2-5 minutes (container start + WAL catch-up).
  5. **Shedding:** As a last resort, drop samples for non-critical metrics. Priority classes: P0 (SLO metrics, never drop), P1 (infrastructure metrics, drop under extreme load), P2 (debug metrics, drop first). Shedding is logged and alerted.
- 

#### 15.17.7 Q7: How do you implement efficient range queries over months of data?

**Answer:** Querying months of data efficiently requires a combination of strategies:

1. **Automatic downsampling:** Instead of scanning 10-second raw data for a 90-day query (777M samples per series), use 5-minute downsampled data (26K samples per series). The query engine selects resolution based on the query time range and step size.
  2. **Block-level pruning:** The TSDB index records min/max time for each block. For a query [Jan 1 - Mar 31], skip blocks outside that range entirely. With compacted 7-day blocks, only 13 blocks need to be opened (not hundreds of 2-hour blocks).
  3. **Parallel block scanning:** Each relevant block is scanned in parallel. For a distributed setup, different blocks live on different store nodes, enabling natural parallelism across the cluster.
  4. **Object storage gateway caching:** Blocks from object storage (S3) are cached locally on store gateway nodes. Frequently queried historical blocks stay warm in the local NVMe cache, avoiding S3 latency.
  5. **Query result caching:** Historical queries are highly cacheable because the data doesn't change. Cache with long TTL (1 hour+). Split-cache enables partial hits for overlapping time ranges.
- 

#### 15.17.8 Q8: What happens when a TSDB storage node runs out of disk space?

**Answer:** This is a critical failure mode. Prevention and graceful handling:

1. **Prevention:** Monitor disk usage as a P0 meta-metric. Alert at 70% usage with automated remediation: trigger compaction (merges blocks, reclaims tombstoned space), upload old blocks to object storage and delete local copies, extend volume (if cloud).
  2. **Graceful degradation:** At 85% disk, the ingester stops accepting new series (existing series still append). At 90%, the ingester marks itself unhealthy in the hash ring, and the distributor routes writes to replicas.
  3. **Emergency actions:** At 95%, force-delete oldest blocks (data still available in object storage or on replicas). WAL truncation to the latest checkpoint.
  4. **Recovery:** Add a new ingester with fresh disk, or expand the existing volume. The hash ring rebalances, and the recovered node receives its share of new writes. Historical data is served from object storage or replicas.
-

**15.17.9 Q9: How would you implement recording rules for pre-computed aggregations?**

**Answer:** Recording rules pre-compute expensive queries and store the results as new time series. This is essential for dashboard performance.

Example: `recording_rule: job:http_requests:rate5m = sum(rate(http_requests_total[5m])) by (job)`

1. **Evaluation:** A dedicated rule evaluator runs each recording rule at its configured interval (e.g., every 60s). It executes the PromQL expression against the query engine and gets the result vector.
2. **Storage:** The result is written back into the TSDB as a new metric (`job:http_requests:rate5m`) with the original labels plus any additional labels from the rule definition.
3. **Query optimization:** Dashboards query the pre-computed metric instead of the raw expression. A query that would scan 10M samples now reads 1K pre-aggregated samples. This reduces query latency from seconds to milliseconds.
4. **Consistency:** Recording rules use “staleness” markers. If the source data has gaps, the recording rule produces NaN, which propagates correctly through PromQL functions.
5. **Naming convention:** Follow `level:metric:operations` (e.g., `job:http_requests:rate5m`). This makes it clear the metric is derived, not raw.

**15.17.10 Q10: How do you handle the “thundering herd” problem when many dashboards refresh simultaneously?**

**Answer:** When 1000 dashboards all refresh every 30 seconds, the query layer receives burst traffic at the refresh boundaries (especially if they’re all aligned to :00/:30).

1. **Jittered refresh:** The dashboard frontend adds random jitter (0-5 seconds) to the refresh interval. Instead of all dashboards hitting at  $T=30.000s$ , they spread across  $T=30.0s$  to  $T=35.0s$ . This smooths the query load.
2. **Query deduplication:** The query frontend detects identical queries arriving within a short window (e.g., 100ms). Only one query is executed; all requesters receive the same result. This is extremely effective when multiple users view the same dashboard.
3. **Result caching:** The query result cache serves repeat queries instantly. With a cache TTL of 15 seconds for “now” queries, at most 2 actual TSDB queries are made per 30-second refresh cycle per unique query.
4. **Request queuing:** Per-tenant query queues with fair scheduling. If a tenant sends 100 queries simultaneously, they are queued and processed at a controlled rate (e.g., 20 concurrent). This prevents one tenant’s dashboard storm from impacting others.
5. **Query coalescing at the store level:** Multiple queries reading the same time-series chunks can share the I/O. The chunk cache ensures that once a chunk is read from disk, subsequent queries for the same chunk get a cache hit.

**15.17.11 Q11: How would you implement anomaly detection without ML?**

**Answer:** Simple statistical methods are surprisingly effective for monitoring:

1. **Z-score alerting:** Calculate the rolling mean and standard deviation over a baseline window (e.g., same hour last 7 days). Alert when the current value exceeds mean +/- 3 standard deviations. PromQL: `(metric - avg_over_time(metric[7d])) / stddev_over_time(metric[7d]) > 3`
2. **Seasonal decomposition:** For metrics with daily/weekly patterns (traffic peaks at noon, drops at night), compare to the same period last week rather than a rolling average. PromQL: `metric / (metric offset 7d) > 2` (2x last week's value).
3. **Rate-of-change detection:** Alert on sudden changes rather than absolute values. `deriv(metric[5m]) > threshold` catches rapid spikes before they hit absolute limits.
4. **predict\_linear():** PromQL's built-in linear regression. Predict future values: `predict_linear(disk_usage[6h], 4*3600) > 0.9` means "disk will be 90% full in 4 hours at current rate."
5. **Histogram percentile shifts:** Monitor p50/p99 ratio. If p99 grows while p50 is stable, tail latency is increasing, indicating emerging issues.

### 15.17.12 Q12: How do you ensure alert notifications are delivered even when the monitoring system is partially down?

**Answer:** Alert delivery is the most critical path in the monitoring system. Failures here mean outages go unnoticed.

1. **HA Alert Manager cluster:** Run 3+ AlertManager instances using gossip protocol (Hashicorp memberlist). They share alert state and deduplicate notifications. If one fails, others continue sending.
2. **Dual evaluators:** Two independent alert evaluators run the same rules against (potentially different) data replicas. Both send to the AlertManager cluster, which deduplicates. If one evaluator is down, the other still fires alerts.
3. **Notification retry with persistence:** Failed notification attempts are persisted to a durable queue (not just in-memory retry). Retry with exponential backoff for up to 24 hours. After 24 hours, escalate through a different channel.
4. **Multi-channel redundancy:** Critical alerts go to both PagerDuty AND SMS AND Slack. If one channel's API is down, others still deliver. The AlertManager tracks delivery status per channel.
5. **Dead-man's switch:** An external watchdog (separate infrastructure) expects a heartbeat from the monitoring system every 60 seconds. If the heartbeat stops, the watchdog pages the on-call engineer through a completely independent channel (e.g., a simple cron job that calls the PagerDuty API directly).

### 15.17.13 Q13: How would you migrate from Prometheus to this distributed system with zero downtime?

**Answer:** Migration must be gradual and reversible:

1. **Phase 1 - Dual Write:** Configure Prometheus to `remote_write` to the new system while continuing to serve queries locally. Both systems have the same data. Duration: 2-4 weeks to build confidence.
2. **Phase 2 - Shadow Read:** Dashboard queries go to both systems. Compare results automatically (< 1% divergence acceptable for floating-point). Flag discrepancies for investigation. Duration: 1-2 weeks.

3. **Phase 3 - Read Cutover:** Point Grafana to the new system's query API. Keep Prometheus running as fallback. If issues arise, switch back instantly by changing the Grafana datasource.
4. **Phase 4 - Alert Cutover:** Migrate alert rules to the new system. Run dual evaluation for 1 week (both systems fire alerts, dedup at AlertManager level). Then disable Prometheus alerting.
5. **Phase 5 - Decommission:** Stop Prometheus scraping (new system handles pull now). Keep Prometheus data accessible for historical queries during transition. Archive and decommission after 30 days of successful operation.

Key: At every phase, rollback is possible within minutes. Never burn bridges.

---

#### 15.17.14 Q14: How do you handle metrics from ephemeral containers and serverless functions?

**Answer:** Ephemeral workloads (containers that live < 60 seconds, Lambda functions) break the pull model because the target disappears before it can be scraped.

1. **Push-based ingestion:** Use StatsD or OTLP push. The function/container sends metrics before it terminates. The OTel Collector sidecar or DaemonSet receives and forwards. No scrape endpoint needed.
  2. **Aggregation at the edge:** Instead of sending every invocation as a separate time series, aggregate at the collector level. Sum request counts, compute latency histograms across invocations, emit one aggregated sample per interval.
  3. **Staleness handling:** When a series stops receiving samples, mark it "stale" after 5 minutes (configurable). Stale series are excluded from queries but the data remains. This prevents graphs from showing flat lines at the last known value.
  4. **Instance labeling:** Use a stable label like `function_name` or `deployment_id` rather than `container_id` or `pod_name` for aggregation. The container ID changes on every restart, creating cardinality explosion. The function name is stable.
  5. **Pushgateway pattern:** For batch jobs, push final metrics to a Pushgateway that holds them until scraped. The Pushgateway persists the last-pushed value for each metric family, surviving the job's termination.
- 

#### 15.17.15 Q15: What are the most critical SLOs for a monitoring system itself, and how do you measure them?

**Answer:** A monitoring system must be more reliable than what it monitors. Key SLOs:

1. **Ingestion availability:** 99.99% of valid samples must be accepted within 5 seconds of receipt. Measured by: synthetic canary that sends known samples every 10s and verifies they appear in queries within 30s. Missing canary = ingestion failure.
2. **Query availability:** 99.9% of dashboard queries must return within 2 seconds. Measured by: synthetic queries against known data every 30s. Track p50, p95, p99 query latency. Error budget: 43 minutes of query downtime per month.
3. **Alert latency:** 99% of threshold breaches must trigger a notification within 60 seconds (eval\_interval + notification pipeline). Measured by: inject a known metric value that triggers a test alert, measure time to notification receipt. Run every 5 minutes.

4. **Data completeness:** < 0.1% sample loss end-to-end. Measured by: canary metrics with known values. Query count of canary samples per hour and compare to expected count (360 for 10s interval). Missing samples = data loss.
5. **Data correctness:** 100% of query results must be numerically accurate (within floating-point tolerance). Measured by: inject known values, query with known functions, verify results against precomputed expected values.

SLO violation response: - Burn rate alerts: If 1-hour burn rate exceeds 14.4x (exhausts monthly budget in 5 days), page immediately. If 6-hour burn rate exceeds 6x, create a P1 ticket.

---

## 15.18 Summary

The metrics and monitoring system design balances several competing concerns:

- **Write throughput:** Kafka buffering + horizontally-scaled ingesters
- **Storage efficiency:** Gorilla compression + tiered storage + downsampling
- **Query performance:** In-memory head block + chunk cache + result cache
- **Multi-dimensional query:** Inverted index on labels + PromQL
- **Alert reliability:** Dual evaluators + HA AlertManager + multi-channel
- **Multi-tenancy:** Per-tenant limits + data isolation + fair scheduling
- **Long-term retention:** Object storage + automatic downsampling
- **Operational simplicity:** Self-monitoring + auto-scaling + graceful degradation

The architecture follows the pattern established by systems like Cortex/Mimir (Grafana), Thanos, and VictoriaMetrics: use Prometheus TSDB as the embedded storage engine, add a distributed layer for horizontal scaling, Kafka for ingestion buffering, and object storage for durable long-term retention. This design handles 10M+ data points per second with sub-second query latency and 99.99% availability.